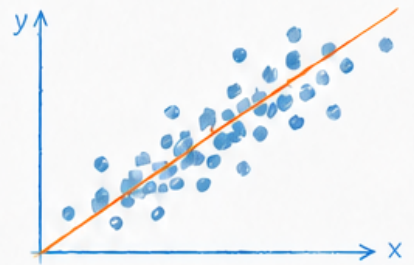
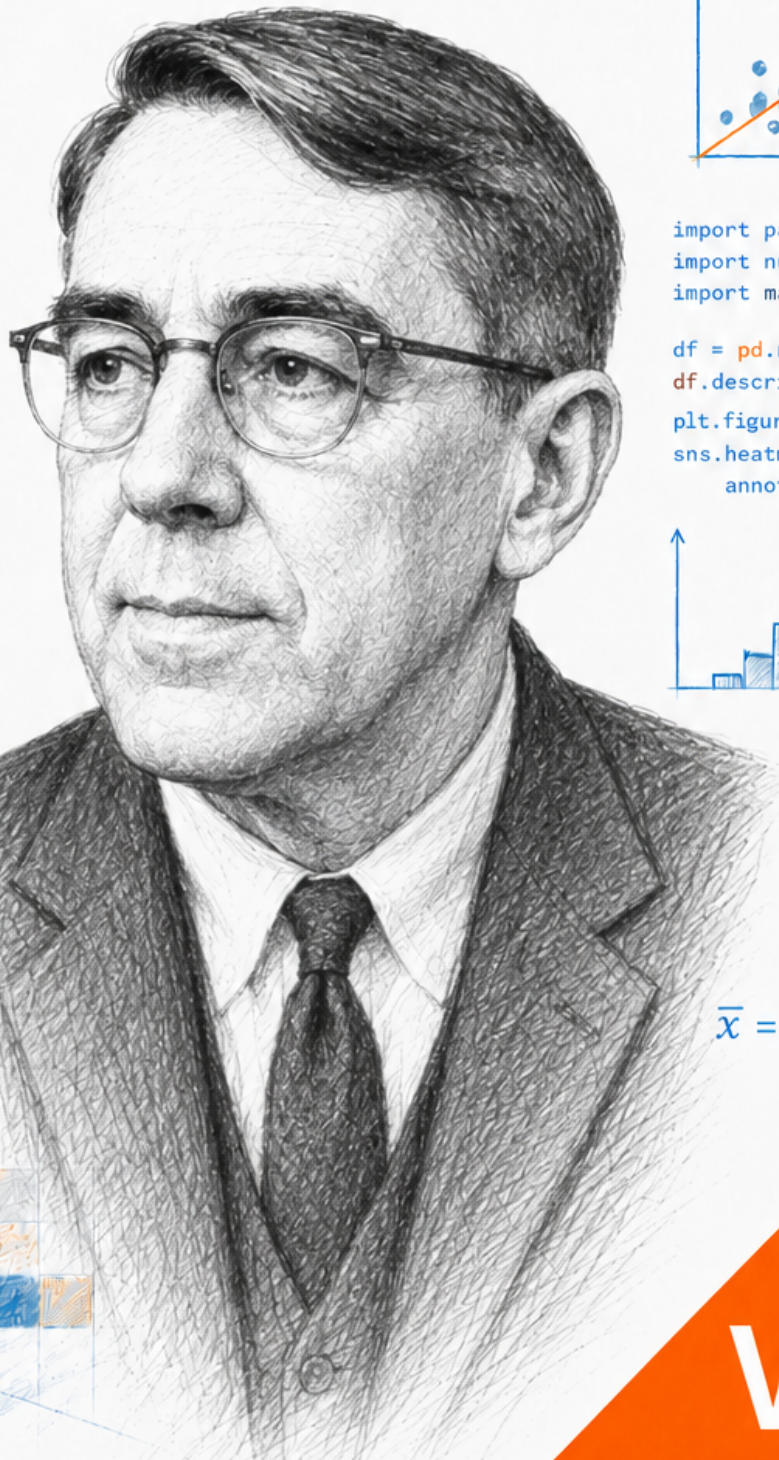


CIENCIA DE DATOS CON PYTHON

— DE LOS DATOS BRUTOS AL CONOCIMIENTO —



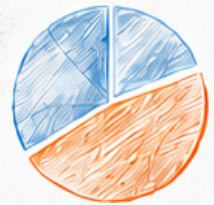
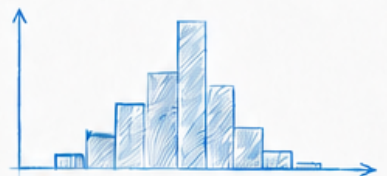
Arthur Samuel
(1901–1990)



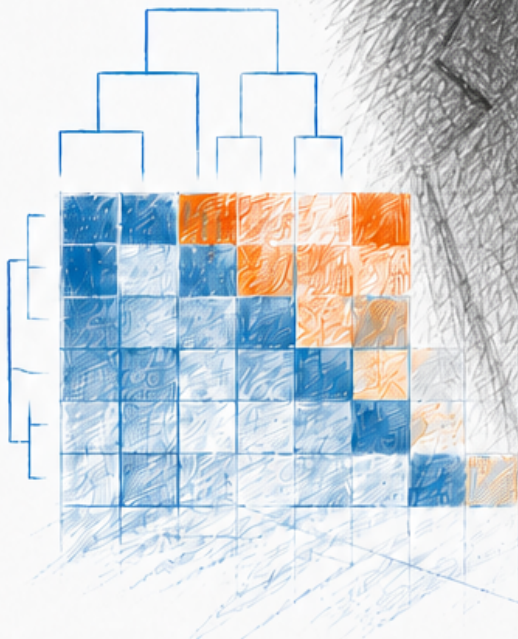
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

df = pd.read_csv('datos.csv')
df.describe()

plt.figure(figsize=(8,5))
sns.heatmap(df.corr(),
            annot=True, cmap='coolwarm')
```



$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$$



Vol. IV

MATERIAL GRATUITO

Última versión y apoyo al proyecto

Este documento forma parte de VintaBytes y se comparte de manera gratuita.

Escaneá el código QR para acceder a la última versión disponible, encontrar otros recursos del proyecto o colaborar de forma voluntaria.

Si este material te resultó útil, cualquier aporte ayuda a seguir preparando libros, cuadernos y ejemplos en español. También ayuda compartirlo, recomendarlo o avisarme si encontrás algo para mejorar.

Y si querés colaborar con el café que acompaña muchas de estas páginas, prometo usarlo con responsabilidad pedagógica.

Muchas gracias por formar parte de VintaBytes.



Escaneá para ir a la página oficial

También podés entrar desde:

<https://vintabytes.github.io/>

Arthur Samuel

En la portada de este volumen aparece la imagen de Arthur Samuel. Arthur fue un investigador estadounidense pionero en inteligencia artificial y aprendizaje automático. Su nombre está especialmente asociado con IBM y con uno de los primeros programas capaces de mejorar su desempeño a partir de la experiencia: un sistema que aprendía a jugar a las damas. En una época en la que las computadoras todavía eran muy limitadas, Samuel exploró una idea fundamental para la ciencia de datos moderna: que una máquina podía usar información previa para ajustar su comportamiento y tomar mejores decisiones.

Su trabajo más conocido fue publicado en 1959 bajo el título *Some Studies in Machine Learning Using the Game of Checkers*. Ese artículo suele citarse como uno de los hitos tempranos del campo y como el origen del uso moderno de la expresión “machine learning”. Allí Samuel planteó una meta que hoy resulta central: programar una computadora para que aprenda a realizar una tarea mejor que quien escribió el programa.

La importancia de Samuel para la ciencia de datos está en haber anticipado una forma de trabajo que hoy es habitual: usar datos, experiencia acumulada y evaluación de resultados para mejorar un modelo. Aunque su ejemplo era un juego de damas, la idea de fondo se extendió luego a problemas de clasificación, predicción, recomendación, detección de patrones y toma de decisiones basada en datos.

Por eso, Arthur Samuel puede considerarse una figura clave en la historia del aprendizaje automático. Su aporte ayudó a desplazar la idea de que las computadoras solo ejecutan reglas fijas, y abrió el camino hacia sistemas capaces de aprender a partir de ejemplos, un principio central en la inteligencia artificial y en la ciencia de datos contemporánea.

Créditos y uso del material

Ciencia de Datos con Python - Tomo 4

Machine Learning

Autor: **Ariel Palazzesi**

Año: **2026**

Versión actualizada de este documento:

<https://vintabytes.github.io/>

Este libro forma parte de una serie de materiales educativos sobre ciencia de datos con Python. La presente versión está aún en desarrollo, y además de este documento PDF incluye, en cada capítulo, un cuadernos de trabajo para Google Colab.

Los cuadernos de trabajo están disponibles de forma gratuita en el repositorio del proyecto:

Repositorio:

<https://github.com/VintaBytes/Ciencia-de-datos>

Desde allí, el lector puede acceder a la última versión de cada notebooks, abrirlos en Google Colab, ejecutar el código, modificar ejemplos y experimentar con los datos como parte del proceso de aprendizaje.

Derechos de autor

© 2026 Ariel Palazzesi. Todos los derechos reservados.

Este material se publica con fines educativos. Se permite su lectura, descarga y uso personal o educativo, así como la ejecución y modificación de los cuadernos en Google Colab con fines de aprendizaje.

No se permite la venta, redistribución, publicación de versiones modificadas, incorporación en cursos comerciales, reproducción total o parcial con fines comerciales, ni la creación de obras derivadas para su distribución pública sin autorización previa y por escrito del autor.

La disponibilidad pública de este libro o de sus cuadernos complementarios no implica cesión de derechos ni autorización para usos distintos de los indicados en esta página.

Para solicitar permisos de uso, adaptación o distribución, contactar al autor.

Uso educativo

Este material puede utilizarse como apoyo para el estudio personal, la consulta docente o el acompañamiento de cursos introductorios de programación, análisis de datos o ciencia de datos con Python, siempre respetando las condiciones de uso indicadas anteriormente.

Se recomienda acompañar la lectura del PDF con la ejecución de los cuadernos online, ya que buena parte del aprendizaje ocurre al probar instrucciones, observar resultados, modificar el código y revisar cómo cambian los datos después de cada operación.

Herramientas utilizadas

Los ejemplos y actividades de este tomo utilizan principalmente:

- Python;
- Pandas;
- Google Colab;
- Jupyter Notebooks.

Las menciones a tecnologías, bibliotecas, plataformas o servicios de terceros se realizan con fines educativos y descriptivos. Cada herramienta conserva sus propias marcas, licencias y condiciones de uso.

Nota sobre el alcance del material

Este tomo está pensado como una introducción progresiva al trabajo con datos tabulares en Python. No pretende cubrir todos los temas de ciencia de datos, sino construir una base sólida para realizar análisis exploratorio, visualización, comunicación de resultados y machine learning.

Introducción

Después de construir los primeros modelos de clasificación y regresión, aparece una nueva etapa en el aprendizaje de Machine Learning: comprender si esos modelos realmente generalizan, cómo pueden mejorarse, qué límites tienen y cómo deben interpretarse sus resultados. Ya no se trata solamente de entrenar un modelo y obtener una métrica inicial. Ahora comenzamos a preguntarnos si ese resultado es confiable, si se mantiene frente a nuevos datos, si el modelo está aprendiendo patrones útiles o si está ajustándose demasiado a los ejemplos que vio durante el entrenamiento.

Este cuarto tomo de **Ciencia de Datos con Python** continúa el recorrido iniciado en los volúmenes anteriores. El tercer tomo introdujo los fundamentos del Machine Learning: qué significa aprender a partir de datos, cómo separar variables de entrada y variable objetivo, cómo preparar datos para entrenar modelos, cómo abordar problemas de clasificación y regresión, y cómo evaluar los primeros resultados. Este volumen retoma ese punto y avanza hacia una mirada más profunda, crítica y aplicada.

El objetivo de este tomo es que el lector pueda desarrollar mayor criterio al construir, evaluar, comparar, interpretar y comunicar modelos de Machine Learning. Para eso, trabajaremos temas que aparecen cuando los primeros modelos ya funcionan, pero todavía necesitamos responder preguntas más exigentes: ¿el modelo generaliza bien?, ¿está sobreajustando?, ¿la métrica es estable?, ¿cómo se comparan modelos de manera justa?, ¿qué tan confiables son sus predicciones?, ¿qué sesgos pueden arrastrar los datos?, ¿cómo comunicar resultados sin exagerar?

Machine Learning suele asociarse con la búsqueda del modelo más potente o de la métrica más alta. Sin embargo, a medida que los problemas se vuelven más reales, esa mirada resulta insuficiente. Un modelo puede obtener un buen resultado en una partición particular de los datos y, aun así, fallar en producción. Puede parecer preciso, pero estar usando información que no existiría al momento de predecir. Puede funcionar bien en promedio, pero equivocarse en los casos más importantes. Puede ser técnicamente correcto y, aun así, conducir a conclusiones mal comunicadas.

Por eso, este volumen insiste en una idea central: entrenar modelos es solo una parte del trabajo. También necesitamos evaluar con cuidado, validar con mayor robustez, revisar errores, comparar alternativas, interpretar resultados, detectar riesgos y comunicar conclusiones responsables.

El contenido de este libro incluye una serie de cuadernos de trabajo preparados para Google Colab. En el libro en formato PDF se presentan los temas como texto organizado para facilitar la lectura continua, la consulta y el estudio. Sin embargo, la experiencia más completa se obtiene al trabajar también con los cuadernos disponibles en línea: allí es posible ejecutar el código, modificarlo, observar los resultados, cambiar parámetros, comparar modelos, revisar errores y experimentar con distintos datasets.

Siempre que sea posible, los ejemplos se apoyan en situaciones reales o en datasets disponibles públicamente, incluyendo conjuntos de datos clásicos y datasets provenientes de plataformas como Kaggle. La intención no es trabajar con ejemplos artificiales demasiado simples, sino acercar al lector a problemas parecidos a los que puede encontrar en contextos de estudio, investigación, análisis profesional o proyectos personales.

Qué vas a encontrar en este tomo

Este tomo se concentra en la profundización del trabajo con modelos de Machine Learning aplicado con Python. A lo largo de los capítulos se presentan herramientas y criterios para evaluar mejor, mejorar modelos, controlar su complejidad, trabajar con algoritmos más potentes, explorar datos no etiquetados, interpretar resultados y comunicar conclusiones con responsabilidad.

En los primeros capítulos se trabaja sobre la generalización. Esta idea es central en Machine Learning: no buscamos que un modelo funcione bien solo con los datos que ya vio, sino que pueda responder de manera razonable frente a casos nuevos. Para eso se estudian el sobreajuste y el subajuste, dos problemas fundamentales que ayudan a comprender cuándo un modelo es demasiado simple, cuándo es demasiado complejo y qué señales pueden alertarnos sobre un aprendizaje poco confiable.

Luego se introduce la validación cruzada. Una única separación entre entrenamiento y prueba puede ser útil, pero también puede depender demasiado del azar de esa partición. La validación cruzada permite evaluar modelos con más estabilidad y comparar alternativas con mayor criterio. En este tramo también se trabaja la búsqueda de hiperparámetros, incluyendo estrategias como Grid Search y Randomized Search, siempre vinculadas con la necesidad de evitar evaluaciones engañosas.

Más adelante se estudia la regularización y el control de la complejidad. Estos conceptos ayudan a entender cómo algunos modelos pueden limitar su flexibilidad para generalizar mejor. La idea no es hacer modelos cada vez más complejos, sino encontrar un equilibrio entre aprender patrones útiles y no memorizar detalles particulares del conjunto de entrenamiento.

El libro también presenta modelos más potentes que los abordados en el volumen anterior. Se estudian Random Forest, Gradient Boosting, Support Vector Machines y Naive Bayes. Cada modelo se introduce desde su idea principal, sus condiciones de uso, sus ventajas y sus limitaciones. El propósito no es agotar todos los detalles matemáticos o técnicos, sino construir una comprensión suficiente para usarlos con criterio y saber cuándo conviene profundizar.

Otro bloque importante está dedicado al aprendizaje no supervisado. Allí cambia el tipo de pregunta: ya no contamos con una variable objetivo que indique la respuesta correcta. En su lugar, buscamos explorar estructura en los datos. Se trabajan conceptos como datos no etiquetados, clustering con K-Means y reducción de dimensionalidad con PCA. Estos temas permiten ampliar la mirada más allá de la predicción supervisada y comprender que Machine Learning también puede utilizarse para agrupar, simplificar, visualizar y detectar patrones.

Hacia el final, el libro se concentra en la interpretación, los sesgos, los errores comunes y la comunicación de resultados. Un modelo no debería evaluarse únicamente por sus métricas. También debemos preguntarnos qué variables utiliza, cómo se comporta, en qué casos falla, qué sesgos pueden estar presentes en los datos, qué errores metodológicos debemos evitar y cómo presentar conclusiones sin exagerar.

En particular, se trabajan ideas como interpretación de modelos, importancia de variables, permutation importance, sesgo en los datos, variables sensibles, correlación y causalidad, decisiones automatizadas y responsabilidad humana. Estos temas son fundamentales para comprender que Machine Learning no es solo una técnica predictiva, sino también una práctica que puede influir en decisiones reales.

El cierre del tomo está dedicado a los errores frecuentes al construir modelos y a la comunicación responsable de resultados. Se revisan problemas como entrenar y evaluar con los mismos datos, fuga de datos, preprocesar antes de separar, usar variables que no existirían en producción, elegir métricas inadecuadas, comparar modelos de forma injusta, confiar en un único resultado y sobreinterpretar conclusiones. También se propone una forma básica de comunicar informes de Machine Learning, incluyendo problema, datos, enfoque, métricas, errores, limitaciones y conclusiones.

Finalmente, el último capítulo abre caminos para seguir aprendiendo. Se mencionan temas que quedan fuera de este recorrido, como Deep Learning, procesamiento de lenguaje natural, series temporales, sistemas de recomendación y MLOps. La intención no es cerrar el aprendizaje, sino mostrar que, a partir de los fundamentos construidos en estos tomos, el lector ya cuenta con una base para avanzar hacia áreas más especializadas.

Requisitos previos

Para aprovechar este libro conviene haber trabajado previamente con los contenidos de los tomos anteriores de Ciencia de Datos con Python o contar con conocimientos equivalentes. En particular, se recomienda conocer:

- nociones básicas de programación en Python;
- uso general de Google Colab o notebooks;
- estructura de un DataFrame de Pandas;
- selección de filas y columnas;
- filtros con condiciones;
- ordenamiento de datos;
- creación y modificación de columnas;
- detección básica de valores faltantes, duplicados y tipos de datos;
- cálculo de resúmenes estadísticos simples;
- uso básico de gráficos con Matplotlib;
- lectura e interpretación inicial de tablas, distribuciones y comparaciones;
- conceptos iniciales de Machine Learning;
- separación entre variables de entrada y variable objetivo;
- división entre datos de entrenamiento y prueba;
- preparación básica de datos para modelos;
- primeros modelos de clasificación y regresión;
- métricas iniciales para evaluar modelos.

Este tomo está pensado como continuación directa del volumen anterior. No se requiere experiencia avanzada en Machine Learning, pero sí conviene comprender las ideas básicas de entrenamiento, predicción, clasificación, regresión, evaluación y fuga de datos. Muchos de esos conceptos reaparecen aquí, pero se trabajan con mayor profundidad y en contextos más exigentes.

Tampoco se requiere un dominio avanzado de matemática, aunque sí se presentarán fundamentos matemáticos cuando sean necesarios para comprender una métrica, una penalización, una distancia, un margen, una probabilidad, una reducción de dimensionalidad o una medida de evaluación.

El objetivo no es evitar la matemática, sino incorporarla de forma gradual y conectada con problemas concretos. Cuando una fórmula aparezca, lo hará para explicar una idea útil: cómo se mide un error, cómo se controla la complejidad, cómo se evalúa un modelo, cómo se calcula una distancia o cómo se interpreta una transformación.

A quién está dirigido

Este material está pensado para personas que ya conocen Python, han trabajado con datos tabulares y tienen una base inicial de Machine Learning. Puede ser útil para estudiantes, docentes, programadores, analistas, profesionales que trabajan con datos, personas que preparan proyectos académicos y cualquier lector que quiera avanzar desde los primeros modelos hacia una comprensión más sólida del proceso completo.

El enfoque es introductorio en relación con los temas avanzados, pero no superficial. Se busca que el lector pueda evaluar modelos con mayor cuidado, comparar alternativas, ajustar hiperparámetros, reconocer sobreajuste, interpretar resultados, detectar errores frecuentes y comunicar conclusiones con prudencia.

Este libro puede acompañar a quienes estén continuando un curso de Machine Learning, preparando un proyecto académico, explorando datasets de Kaggle o intentando mejorar sus primeros modelos predictivos. No está pensado como un manual exhaustivo de todos los algoritmos existentes, sino como una base sólida para desarrollar criterio en el uso de modelos más complejos y en la interpretación de resultados.

Cómo trabajar con este libro y los cuadernos

Cada capítulo puede leerse directamente en el PDF, pero fue pensado también como una experiencia práctica apoyada en cuadernos ejecutables. La recomendación es acompañar la lectura con la versión online del cuaderno correspondiente siempre que sea posible.

En los cuadernos de Google Colab conviene ejecutar cada celda, observar la salida y preguntarse qué aporta ese resultado al problema que se está resolviendo. En esta etapa, muchas veces el valor no está solo en entrenar un modelo, sino en interpretar qué ocurrió: si el resultado es estable, si el modelo sobreajusta, qué hiperparámetros influyen, qué errores comete, qué variables parecen importantes y si la conclusión es razonable.

También es recomendable modificar los ejemplos. Cambiar un modelo, ajustar un hiperparámetro, alterar una métrica, comparar validación cruzada con una separación simple o revisar errores individuales ayuda a comprender que los resultados no son propiedades absolutas del algoritmo. Cada decisión del proceso puede influir en la evaluación final.

Los capítulos fueron diseñados como una secuencia progresiva. Algunos conceptos reaparecen en distintos momentos porque forman parte del trabajo cotidiano con

modelos. La separación entre entrenamiento y prueba, la evaluación sobre datos no vistos, el riesgo de sobreajuste, la importancia de elegir métricas adecuadas, el cuidado frente a la fuga de datos y la necesidad de comunicar límites no se aprenden en una única aparición: se consolidan al verlos funcionar en distintos contextos.

Enfoque del libro

El objetivo principal de este tomo es profundizar el trabajo con Machine Learning como una extensión natural del análisis de datos. Una vez que sabemos construir modelos iniciales, necesitamos aprender a evaluarlos mejor, compararlos con más criterio, detectar sus límites e interpretar sus resultados.

A lo largo del recorrido se priorizan cinco ideas:

- un modelo debe generalizar a datos nuevos, no solo funcionar sobre los datos que ya vio;
- evaluar correctamente exige más que mirar una única métrica;
- modelos más potentes requieren más cuidado, no menos;
- interpretar un modelo implica comprender su comportamiento sin exagerar sus conclusiones;
- usar Machine Learning de manera responsable exige revisar datos, errores, sesgos, decisiones y comunicación.

Con esa base, el lector queda preparado para seguir avanzando hacia temas más especializados, como Deep Learning, procesamiento de lenguaje natural, series temporales, sistemas de recomendación, modelos generativos o despliegue de modelos. Pero, sobre todo, queda mejor preparado para formular buenas preguntas, elegir herramientas con criterio, evitar errores comunes y comunicar resultados de Machine Learning de manera prudente y responsable.

Índice general

Parte VI · Generalización, validación y mejora

Capítulo 17 · Sobreajuste y subajuste

- Qué significa generalizar.
- Overfitting.
- Underfitting.
- Modelos demasiado simples.
- Modelos demasiado complejos.
- Señales de alerta.
- Resultados demasiado buenos para ser ciertos.
- Relación con fuga de datos.

Capítulo 18 · Validación cruzada

- Limitaciones de una única separación train/test.
- K-Fold Cross Validation.
- Stratified K-Fold.
- Promedio y variabilidad de métricas.
- Cómo comparar modelos con más criterio.
- Validación cruzada en `scikit-learn`.

Capítulo 19 · Hiperparámetros y búsqueda de mejores modelos

- Parámetros vs hiperparámetros.
- Ajuste manual.
- Grid Search.
- Randomized Search.
- Validación cruzada durante la búsqueda.
- Riesgos de probar combinaciones sin criterio.
- Selección del mejor modelo.

Capítulo 20 · Regularización y control de complejidad

- Por qué controlar la complejidad del modelo.
- Regularización L1 y L2.

- Ridge y Lasso.
- Regularización en regresión logística.
- Cuándo ayuda y cuándo puede perjudicar.

Parte VII · Modelos más potentes

Capítulo 21 · Random Forest

- Ensamblados de modelos.
- La idea de muchos árboles trabajando juntos.
- Bagging.
- Reducción del sobreajuste.
- Importancia de variables.
- Ventajas.
- Limitaciones.

Capítulo 22 · Gradient Boosting

- La idea del boosting.
- Modelos que corrigen errores anteriores.
- Diferencias con Random Forest.
- Potencia y riesgo de sobreajuste.
- Hiperparámetros importantes.
- Panorama de implementaciones populares.

Capítulo 23 · Support Vector Machines

- La idea del margen.
- Separación entre clases.
- Kernels.
- Importancia del escalado.
- Ventajas.
- Limitaciones.
- Cuándo puede ser una buena opción.

Capítulo 24 · Naive Bayes

- Un enfoque probabilístico.
- El supuesto de independencia.
- Clasificación rápida y simple.

- Clasificación de texto como ejemplo típico.
- Ventajas.
- Limitaciones.

Parte VIII · Aprendizaje no supervisado

Capítulo 25 · Aprender sin etiquetas

- Qué cambia cuando no tenemos variable objetivo.
- Datos no etiquetados.
- Buscar estructura en los datos.
- Agrupamiento.
- Reducción de dimensionalidad.
- Detección de anomalías.
- Cuidado al interpretar resultados no supervisados.

Capítulo 26 · Clustering con K-Means

- Qué significa agrupar datos.
- Centroides.
- Distancia a los centros.
- Número de clusters.
- Método del codo.
- Escalado de variables.
- Limitaciones de K-Means.

Capítulo 27 · Reducción de dimensionalidad con PCA

- Por qué reducir dimensiones.
- Variables correlacionadas.
- Componentes principales.
- Varianza explicada.
- Usos frecuentes.
- Visualización de datos.
- Limitaciones interpretativas.

Parte IX · Interpretación, sesgos y comunicación

Capítulo 28 · Interpretar modelos

- Qué significa interpretar un modelo.
- Modelos transparentes y modelos opacos.
- Coeficientes.
- Importancia de variables.
- Permutation importance.
- Interpretación local y global.
- Riesgos de interpretar de más.

Capítulo 29 · Sesgos, datos y decisiones

- Sesgo en los datos.
- Sesgo en el modelo.
- Datos faltantes y representación desigual.
- Variables sensibles.
- Correlación y causalidad.
- Impacto de las decisiones automatizadas.
- Responsabilidad humana.

Capítulo 30 · Errores comunes al construir modelos

- Entrenar y evaluar con los mismos datos.
- Fuga de datos.
- Preprocesar antes de separar.
- Usar variables que no existirían en producción.
- Elegir métricas inadecuadas.
- Comparar modelos de forma injusta.
- Confiar en un único resultado.
- No revisar errores individuales.
- Sobreinterpretar conclusiones.

Capítulo 31 · Comunicar resultados de Machine Learning

- Explicar el problema.
- Describir los datos.
- Justificar el enfoque.
- Presentar métricas relevantes.

- Mostrar errores y limitaciones.
- Comparar modelos sin exagerar.
- Escribir conclusiones responsables.
- Qué debería incluir un informe básico.

Parte X · Cierre y próximos caminos

Capítulo 32 · Cómo seguir aprendiendo Machine Learning

- Qué temas quedaron fuera del libro.
- Deep Learning.
- Procesamiento de lenguaje natural.
- Series temporales.
- Sistemas de recomendación.
- MLOps.
- Ética y regulación.
- Cómo practicar con datasets reales.
- Cómo leer documentación.
- Cómo construir criterio propio.

Capítulo 17 · Sobreajuste y subajuste

Qué significa generalizar

En Machine Learning, entrenamos modelos usando datos disponibles. Esos datos contienen ejemplos que ya conocemos: clientes históricos, viviendas ya vendidas, transacciones pasadas, registros de ventas, observaciones anteriores. Durante el entrenamiento, el modelo busca patrones en esos ejemplos para poder realizar predicciones.

Pero el objetivo final no es que el modelo funcione bien únicamente sobre los datos que ya vio. El objetivo es que pueda comportarse razonablemente bien con **datos nuevos**. A esa capacidad la llamamos **generalización**.

Generalizar significa aprender patrones útiles en los datos de entrenamiento y aplicarlos correctamente a casos nuevos.

Esta idea está en el centro del Machine Learning. Un modelo no se entrena para memorizar una tabla. Se entrena para encontrar regularidades que puedan servir más allá de los ejemplos conocidos.

Por ejemplo, si entrenamos un modelo para predecir el precio de viviendas, no nos interesa solamente que estime bien los precios de las viviendas que ya estaban en el dataset. Queremos que pueda estimar el precio de una vivienda nueva, que todavía no fue usada durante el entrenamiento.

Datos de entrenamiento: viviendas ya conocidas.

Caso nuevo: vivienda que el modelo no vio antes.

Objetivo: producir una estimación razonable para el caso nuevo.

Lo mismo ocurre en clasificación. Si entrenamos un modelo para detectar fraude, no alcanza con que clasifique bien transacciones históricas. Queremos que ayude a detectar fraudes futuros. Si entrenamos un modelo para predecir abandono de clientes, queremos que identifique clientes actuales en riesgo, no solo que explique correctamente abandonos pasados.

Por eso separamos datos de entrenamiento y prueba. El conjunto de entrenamiento sirve para ajustar el modelo. El conjunto de prueba sirve para simular, de manera controlada, qué ocurre cuando el modelo se enfrenta a casos que no vio durante el entrenamiento. Durante el entrenamiento el modelo aprende. Durante la prueba evaluamos si ese aprendizaje funciona en datos no vistos.

Si un modelo obtiene buen desempeño en entrenamiento y también buen desempeño en prueba, tenemos una señal positiva. No garantiza que funcionará perfectamente en el futuro, pero indica que los patrones aprendidos no dependen solo de memorizar los datos de entrenamiento.

En cambio, si un modelo funciona muy bien en entrenamiento pero mal en prueba, hay un problema. El modelo aprendió demasiado los detalles específicos de los datos conocidos, pero no logró capturar patrones que se mantengan en casos nuevos. Este fenómeno se llama **sobreajuste**, y lo estudiaremos en la próxima sección.

También puede ocurrir lo contrario: que el modelo funcione mal tanto en entrenamiento como en prueba. En ese caso, probablemente no está aprendiendo lo suficiente. Tal vez el modelo es demasiado simple, las variables no contienen señal útil, el problema está mal formulado o los datos no alcanzan. Este fenómeno se relaciona con el **subajuste**.

La generalización se ubica entre esos dos extremos. Queremos un modelo que aprenda lo suficiente como para capturar patrones reales, pero no tanto como para memorizar detalles accidentales.

Aprender demasiado poco: **subajuste**. Aprender detalles específicos del entrenamiento: **sobreajuste**. Aprender patrones que funcionan en datos nuevos: **generalización**.

Una forma intuitiva de pensar esto es comparar un modelo con un estudiante. Si un estudiante memoriza las respuestas exactas de una guía, puede obtener buen resultado si el examen repite las mismas preguntas. Pero si el examen cambia los datos o plantea situaciones nuevas, probablemente tendrá dificultades. No aprendió el concepto; memorizó ejemplos.

Un modelo sobreajustado se parece a ese estudiante. Puede funcionar muy bien con los datos conocidos, pero falla cuando aparecen casos nuevos.

En cambio, un estudiante que entiende el procedimiento puede resolver ejercicios diferentes, aunque no sean idénticos a los practicados. No necesita que el problema sea exactamente igual. Reconoce la estructura general y la aplica a una situación nueva. Eso se parece más a generalizar.

Memorizar ejemplos puede funcionar en entrenamiento, pero comprender patrones permite responder mejor ante casos nuevos.

En Machine Learning, esta diferencia es fundamental porque los datos de entrenamiento siempre son una muestra limitada de la realidad. No contienen todos los casos posibles. Pueden tener ruido, errores, valores extremos, ausencias, sesgos o situaciones particulares del período en que fueron recolectados. Si el modelo se ajusta demasiado a esos detalles, puede perder capacidad para funcionar fuera de esa muestra.

Generalizar no significa acertar siempre. Ningún modelo real predice perfectamente todos los casos nuevos. Generalizar significa que el modelo mantiene un desempeño razonable cuando se enfrenta a datos que no fueron usados para entrenarlo.

Por ejemplo, un modelo de regresión puede tener cierto error promedio en entrenamiento y un error parecido en prueba. Eso sugiere que el modelo está aprendiendo una relación que se mantiene en datos no vistos. Si el error de prueba es mucho mayor, la señal es distinta: el modelo aprendió algo que no se sostiene fuera del entrenamiento.

Buen indicio: error de entrenamiento y error de prueba razonablemente cercanos. Señal de alerta: error de entrenamiento muy bajo y error de prueba mucho más alto.

En clasificación ocurre algo similar. Si la accuracy, el F1-score o las métricas relevantes son muy altas en entrenamiento pero caen mucho en prueba, debemos sospechar. El modelo puede estar adaptándose demasiado a los casos conocidos.

Por eso, la evaluación sobre datos no vistos no es un detalle técnico. Es la forma básica de comprobar si el modelo generaliza.

También debemos entender que generalizar depende del parecido entre los datos de entrenamiento y los datos futuros. Si entrenamos un modelo con datos de una ciudad y luego lo usamos en otra con características muy distintas, el desempeño puede cambiar. Si entrenamos con datos de un período económico estable y luego lo usamos durante una crisis, las relaciones aprendidas pueden dejar de funcionar. Si entrenamos con un tipo de usuario y luego aplicamos el modelo a otro grupo, puede fallar.

Un modelo generaliza mejor cuando los datos nuevos se parecen razonablemente a los datos usados para entrenar. Esto no significa que los casos nuevos deban ser idénticos. Si fueran idénticos, no necesitaríamos generalizar. Pero sí debe existir cierta continuidad entre el pasado que usamos para entrenar y el futuro o contexto donde aplicaremos el modelo.

Esta idea ya había aparecido cuando hablamos de variables disponibles antes de la predicción y de fuga de datos. Un modelo puede parecer excelente si se entrena con información que no estará disponible en el momento real de uso. Pero ese desempeño no representa verdadera generalización. Representa una evaluación engañosa.

Generalizar también implica evitar aprender ruido. En los datos puede haber patrones accidentales que no se repetirán. Por ejemplo, una promoción puntual puede haber aumentado ventas en una semana específica. Un error de carga puede hacer que ciertos valores parezcan relacionados. Un grupo pequeño de casos puede tener una coincidencia casual. Si el modelo toma esas señales como si fueran reglas generales, puede equivocarse en el futuro.

No todo patrón en los datos de entrenamiento es un patrón útil para datos nuevos. Por eso, un modelo más complejo no siempre generaliza mejor. Un modelo muy flexible puede adaptarse a muchos detalles del entrenamiento, incluso a los que no representan relaciones reales. En cambio, un modelo más simple puede ignorar parte del ruido y capturar una tendencia más estable. La clave está en encontrar un equilibrio.

Este equilibrio será el tema central del capítulo. El sobreajuste aparece cuando el modelo se adapta demasiado a los datos de entrenamiento. El subajuste aparece cuando el modelo es tan simple o insuficiente que no captura ni siquiera los patrones principales. La buena generalización está entre ambos. Queremos un modelo que aprenda patrones, no un modelo que memorice ruido ni uno que ignore la señal disponible.

En la práctica, para evaluar generalización comparamos el desempeño en entrenamiento y en prueba. Más adelante, usaremos validación cruzada para tener una evaluación más estable. Pero la intuición ya está presente: un modelo útil debe funcionar en datos que no vio.

Generalizar significa que un modelo aprende relaciones a partir de los datos de entrenamiento y logra aplicarlas razonablemente bien a casos nuevos. Es la diferencia entre memorizar ejemplos y capturar patrones útiles. La separación entre entrenamiento y prueba existe justamente para medir esa capacidad. Cuando un modelo no generaliza bien, puede estar sobreajustado, subajustado o afectado por problemas en los datos, las variables o la forma de evaluación.

Overfitting

El **overfitting**, o **sobreajuste**, ocurre cuando un modelo aprende demasiado bien los datos de entrenamiento, al punto de capturar no solo patrones útiles, sino también detalles accidentales, ruido, valores particulares o irregularidades que no se repetirán en datos nuevos.

Un modelo sobreajustado puede parecer excelente durante el entrenamiento. Sus métricas sobre los datos conocidos pueden ser muy buenas: pocos errores, alta accuracy, bajo MAE, alto R^2 . Pero cuando lo evaluamos con datos de prueba, su rendimiento cae. Esa diferencia revela que el modelo no aprendió una regla general, sino una versión demasiado ajustada a los ejemplos que vio.

La idea puede parecer contradictoria al principio. ¿Cómo puede ser malo que un modelo aprenda mucho? El problema es que no todo lo que aparece en los datos de entrenamiento representa una regularidad útil. Los datos reales contienen ruido. Pueden incluir errores de carga, casos atípicos, coincidencias casuales, pequeñas variaciones sin importancia o patrones que solo existen en esa muestra.

Si el modelo tiene suficiente flexibilidad, puede adaptarse a esos detalles. En lugar de aprender la tendencia general, empieza a aprender particularidades del conjunto de entrenamiento.

Por ejemplo, imaginemos un modelo de regresión que predice precios de viviendas. Una relación general útil podría ser que, en promedio, viviendas con más superficie tienden a tener precios más altos. Pero el dataset puede contener casos particulares: una vivienda pequeña muy cara por su ubicación, una vivienda grande barata por su estado, una operación registrada con error o una propiedad excepcional que no representa al mercado general.

Un modelo demasiado flexible podría intentar ajustar todos esos casos particulares. En entrenamiento, sus errores serían muy bajos. Pero al recibir nuevas viviendas, esas reglas tan específicas probablemente no se mantendrán. Por ejemplo:

- **Patrón útil:** a mayor superficie, mayor precio estimado, en promedio.
- **Detalle accidental:** esta vivienda particular se comporta distinto por una razón que el modelo no puede generalizar.

En clasificación ocurre algo similar. Supongamos un modelo para detectar abandono de clientes. Puede aprender patrones razonables, como baja frecuencia de uso, muchos reclamos recientes o caída en la interacción. Pero también podría aprender coincidencias demasiado específicas: combinaciones raras de variables que aparecen en pocos clientes del entrenamiento y que no representan un patrón estable.

El modelo puede clasificar perfectamente a esos clientes conocidos, pero fallar con clientes nuevos.

Una forma intuitiva de entender el sobreajuste es volver a la analogía del estudiante. Un estudiante que memoriza las respuestas exactas de una guía puede obtener un resultado perfecto si el examen repite los mismos ejercicios. Pero si el examen cambia los valores o plantea una situación parecida con otra redacción, se equivoca. No aprendió el concepto; memorizó los ejemplos.

Un modelo sobreajustado se comporta de manera parecida. Aprende demasiado los ejemplos de entrenamiento y pierde capacidad para resolver casos nuevos.

El sobreajuste suele aparecer cuando el modelo es demasiado complejo para la cantidad o calidad de datos disponibles. Un árbol de decisión muy profundo, por ejemplo, puede crear muchas divisiones hasta separar casi perfectamente los casos de entrenamiento. Puede terminar con hojas que contienen muy pocos ejemplos, incluso uno solo. Eso le permite memorizar el dataset, pero no necesariamente generalizar.

En regresión, un modelo demasiado flexible puede seguir cada ondulación de los datos, incluso las que corresponden a ruido. En lugar de capturar una tendencia general, se ajusta a cada punto conocido. Visualmente, sería como dibujar una línea que pasa por casi todos los puntos, pero con una forma tan irregular que no representa bien el fenómeno.

Un modelo demasiado flexible puede adaptarse al ruido. Un modelo que generaliza captura la tendencia estable.

El sobreajuste no depende solo del algoritmo. También depende de la cantidad de datos, la cantidad de variables, la presencia de ruido, la calidad de las etiquetas, la forma de preprocesar y la dificultad del problema. Un modelo puede sobreajustar porque es muy complejo, pero también porque tiene demasiadas variables irrelevantes o porque el conjunto de entrenamiento es pequeño.

Si tenemos pocas filas y muchas columnas, el modelo puede encontrar relaciones accidentales. Algunas variables parecerán útiles simplemente por coincidencia. En entrenamiento, esas coincidencias pueden mejorar las métricas. En datos nuevos, desaparecen.

También puede haber sobreajuste cuando probamos muchas variantes del modelo y elegimos la que mejor funciona en el conjunto de prueba. Aunque parezca que estamos evaluando correctamente, si usamos el test una y otra vez para tomar

decisiones, podemos terminar ajustándonos indirectamente a ese conjunto. Este punto se entenderá mejor cuando estudiemos validación cruzada y búsqueda de hiperparámetros.

Por ahora, la señal más importante es comparar entrenamiento y prueba. Si el desempeño en entrenamiento es mucho mejor que en prueba, debemos sospechar sobreajuste.

En clasificación, podríamos ver algo como:

```
Accuracy en entrenamiento: 0.99
Accuracy en prueba:        0.72
```

O en regresión:

```
MAE en entrenamiento: 2.1
MAE en prueba:        18.7
```

En ambos casos, el modelo parece haber aprendido muy bien los datos de entrenamiento, pero no logra trasladar ese desempeño a datos no vistos.

Esto no significa que las métricas de entrenamiento deban ser idénticas a las de prueba. Es normal que el modelo funcione algo mejor en entrenamiento, porque esos datos fueron usados para ajustarlo. Lo preocupante es que la diferencia sea grande.

El sobreajuste también puede producir una falsa sensación de éxito. Si solo miramos las métricas de entrenamiento, podríamos creer que el modelo es excelente. Por eso insistimos tanto en separar datos de prueba y evaluar sobre casos no vistos. La evaluación en entrenamiento no alcanza para saber si un modelo aprendió algo útil.

Un caso especialmente engañoso aparece cuando el modelo obtiene resultados casi perfectos. En problemas reales, una accuracy de 1.00, un F1-score perfecto o un error prácticamente cero pueden ser señales de que algo anda mal. No siempre, pero conviene revisar. Puede haber sobreajuste, fuga de datos, evaluación sobre entrenamiento o una variable que revela directamente la respuesta.

Para reducir el sobreajuste existen varias estrategias. Algunas dependen del modelo. En árboles de decisión, por ejemplo, podemos limitar la profundidad, exigir más casos por hoja o evitar divisiones demasiado específicas. En otros modelos, podemos usar regularización, ajustar hiperparámetros o elegir una estructura más simple.

También podemos mejorar los datos: conseguir más ejemplos, eliminar variables irrelevantes, corregir errores, revisar outliers, evitar fuga de datos o construir

variables más estables. Muchas veces, mejorar la calidad de los datos ayuda más que cambiar de algoritmo. Para reducir sobreajuste podemos:

- simplificar el modelo,
- limitar su flexibilidad,
- mejorar los datos,
- revisar variables,
- evaluar con más cuidado.

Sin embargo, no debemos interpretar el sobreajuste como algo que se elimina por completo. Siempre existe cierta diferencia entre aprender de una muestra y aplicar el modelo a nuevos casos. El objetivo es controlar ese riesgo y elegir modelos que mantengan un desempeño razonable fuera del entrenamiento.

También debemos evitar la reacción opuesta: simplificar tanto el modelo que ya no aprenda lo suficiente. Si reducimos demasiado la complejidad, podemos caer en subajuste. Por eso, el problema no es que un modelo aprenda mucho, sino que aprenda detalles que no generalizan.

El equilibrio consiste en permitir que el modelo capture patrones reales sin perseguir cada ruido del entrenamiento. Con demasiada flexibilidad existe el riesgo de sobreajuste. Con demasiada simplicidad aparece el riesgo de subajuste. Un buen modelo aprende patrones útiles y generalizables.

El overfitting o sobreajuste ocurre cuando un modelo se adapta demasiado a los datos de entrenamiento y pierde capacidad para funcionar bien con datos nuevos. Suele verse como una gran diferencia entre el desempeño en entrenamiento y en prueba. Puede deberse a modelos demasiado complejos, pocos datos, ruido, variables irrelevantes o evaluación inadecuada. Detectarlo y controlarlo es fundamental para construir modelos que realmente generalicen.

Underfitting

El **underfitting**, o **subajuste**, ocurre cuando un modelo es demasiado simple o insuficiente para capturar los patrones importantes de los datos. A diferencia del sobreajuste, donde el modelo aprende demasiado los detalles del entrenamiento, en el subajuste el modelo aprende demasiado poco.

Un modelo subajustado suele funcionar mal tanto en entrenamiento como en prueba. No logra buenos resultados ni siquiera sobre los datos que vio durante el ajuste. Esto

indica que no está capturando adecuadamente la relación entre las variables de entrada y la variable objetivo.

Esta es una diferencia importante con el sobreajuste. En el sobreajuste, el modelo parece excelente en entrenamiento, pero falla en prueba. En el subajuste, el problema aparece desde el principio: el modelo no logra explicar bien los datos conocidos.

Por ejemplo, imaginemos que queremos predecir precios de viviendas usando solo la superficie. Si el precio depende mucho de la ubicación, el estado, la antigüedad, la cantidad de ambientes y otros factores, una regresión lineal simple con una sola variable puede quedarse corta. Tal vez capture una tendencia general, pero sus errores serán grandes porque le falta información relevante.

En ese caso, el modelo no está sobreajustando. No está memorizando detalles. Al contrario: no tiene suficiente capacidad o información para representar el fenómeno. Está subajustando.

El subajuste también puede aparecer cuando elegimos un modelo cuya forma no se adapta al problema. Supongamos que la relación entre una variable y el objetivo es claramente curva, pero usamos una recta. La regresión lineal intentará aproximar esa relación con una línea recta, pero tal vez falle de manera sistemática.

Por ejemplo, el consumo eléctrico puede ser bajo en temperaturas templadas y aumentar tanto en días muy fríos como en días muy calurosos. Esa relación tiene forma curva. Si usamos una regresión lineal simple con temperatura como única variable, el modelo puede no capturar bien ese comportamiento. La relación real es curva, pero el modelo elegido intenta representarla mediante una recta. El resultado posible es el subajuste.

Esto no significa que la regresión lineal sea mala. Significa que, para algunos problemas, una relación lineal básica puede ser demasiado limitada. Un buen modelo debe tener una complejidad adecuada para el fenómeno que intenta representar.

También puede haber subajuste cuando las variables de entrada no contienen suficiente señal. Si queremos predecir ventas usando columnas poco relacionadas con las ventas, el modelo tendrá dificultades aunque el algoritmo sea correcto. No puede aprender una relación que no está presente en los datos.

Por ejemplo, intentar predecir el precio de una vivienda usando solo un identificador interno del registro no tendría sentido. El modelo podría encontrar coincidencias accidentales, pero no una relación útil. Lo mismo ocurriría si queremos predecir el abandono de clientes sin variables que describan su comportamiento, historial, uso del servicio o interacciones relevantes.

El subajuste puede deberse, entonces, a varias causas: un modelo demasiado simple, pocas variables relevantes, variables mal preparadas, una relación no lineal que el modelo no logra capturar, datos con poca señal o un problema mal formulado.

Una señal típica de subajuste es que las métricas sean malas tanto en entrenamiento como en prueba. En regresión, podríamos observar errores altos en ambos conjuntos:

```
MAE en entrenamiento: 28.4
MAE en prueba:        30.1
```

La diferencia entre entrenamiento y prueba no es muy grande, pero ambos errores son altos. Eso sugiere que el modelo no está aprendiendo lo suficiente.

En clasificación podría ocurrir algo similar:

```
Accuracy en entrenamiento: 0.62
Accuracy en prueba:        0.60
```

Si el problema no es extremadamente difícil y una referencia simple obtiene valores parecidos, el modelo probablemente esté subajustando o no esté aprovechando información suficiente.

El subajuste también puede aparecer cuando aplicamos restricciones demasiado fuertes al modelo. Por ejemplo, un árbol de decisión con una profundidad máxima muy baja puede ser incapaz de separar adecuadamente los casos. Si permitimos solo una o dos preguntas, quizá el árbol resulte demasiado simple para representar patrones importantes.

En modelos con regularización, una regularización excesiva también puede producir subajuste. La regularización busca limitar la complejidad del modelo para evitar sobreajuste. Pero si la restricción es demasiado fuerte, el modelo puede quedar tan limitado que no aprenda relaciones útiles.

Esta idea muestra que no existe una complejidad ideal universal. Un modelo demasiado simple puede subajustar y uno demasiado complejo puede sobreajustar. La dificultad está en encontrar un punto intermedio que permita una mejor generalización.

Una forma intuitiva de entender el subajuste es imaginar un estudiante que estudió demasiado poco o que aprendió una regla demasiado básica para resolver problemas variados. Si el examen contiene ejercicios apenas distintos, no puede resolverlos. No porque haya memorizado demasiado, sino porque no incorporó suficiente estructura conceptual.

Un modelo subajustado se parece a ese estudiante: no aprendió los patrones principales del problema.

También puede ocurrir que el modelo tenga suficiente capacidad, pero los datos estén mal preparados. Si una variable categórica importante no fue codificada correctamente, si los valores faltantes fueron tratados de forma inadecuada o si una variable numérica quedó en un formato incorrecto, el modelo puede no aprovechar la información disponible.

Por ejemplo, si una columna de fechas se usa como texto sin extraer información útil, quizá el modelo no capture patrones relacionados con el mes, el día de la semana o la temporada. Si una columna de barrio se elimina por completo, tal vez se pierda una señal importante para predecir precios. A veces, por lo tanto, el problema no está en el algoritmo, sino en la representación de los datos.

Para reducir el subajuste, podemos probar distintas estrategias. Una es agregar variables relevantes. Si el modelo no tiene suficiente información, mejorar las variables de entrada puede ayudar. Otra es usar un modelo más flexible, capaz de capturar relaciones no lineales o interacciones. También podemos revisar el preprocesamiento, corregir errores, construir nuevas características o reformular el problema.

Pero debemos hacerlo con cuidado. Si aumentamos demasiado la complejidad, podemos pasar del subajuste al sobreajuste. Mejorar un modelo no significa hacerlo cada vez más complejo sin límite. Significa darle la capacidad suficiente para aprender patrones reales, pero seguir evaluando si generaliza.

Por ejemplo, si una regresión lineal simple funciona mal, podemos probar una regresión lineal múltiple con más variables. Si todavía falla porque la relación es no lineal, más adelante podríamos considerar árboles, Random Forest, Gradient Boosting u otros modelos más flexibles. Pero cada paso debe evaluarse con datos de prueba o validación.

El subajuste también nos recuerda que no todos los problemas pueden resolverse solo cambiando el modelo. Si los datos no contienen información suficiente, ningún algoritmo podrá producir buenas predicciones. Si la variable objetivo está mal definida, si los registros son inconsistentes o si el fenómeno es muy aleatorio, el margen de mejora puede ser limitado. Un modelo no puede aprender una señal que no está en los datos.

Por eso, ante un posible subajuste, no conviene culpar automáticamente al algoritmo. Debemos revisar el problema completo: la pregunta, la variable objetivo, las variables

predictoras, la calidad de los datos, el preprocesamiento y las métricas usadas para evaluar.

También es importante comparar con una referencia simple. Si nuestro modelo funciona apenas igual que predecir siempre la clase mayoritaria o el promedio de la variable objetivo, probablemente no esté aprendiendo mucho. Una mejora clara sobre esa referencia es una señal mínima de que el modelo está capturando alguna relación útil. Si no logra mejorarla, debemos revisar si realmente está aprendiendo.

El underfitting o subajuste ocurre cuando un modelo no logra aprender suficientemente los patrones del problema. Suele manifestarse como mal desempeño tanto en entrenamiento como en prueba. Puede deberse a modelos demasiado simples, variables insuficientes, mala preparación de datos, relaciones no lineales no capturadas o problemas en la formulación. Reducir el subajuste implica mejorar la información disponible o aumentar la capacidad del modelo, siempre cuidando no caer en sobreajuste.

Modelos demasiado simples

Un modelo demasiado simple es aquel que no tiene suficiente capacidad para representar los patrones importantes del problema. Puede aprender una tendencia muy general, pero no capturar detalles relevantes de la relación entre las variables de entrada y la variable objetivo.

Este tipo de modelo suele estar asociado al subajuste. No falla porque haya aprendido demasiado los datos de entrenamiento, sino porque no aprende lo suficiente. Sus predicciones son pobres incluso sobre los datos que usó para entrenar.

Un ejemplo claro aparece en regresión. Supongamos que queremos predecir el consumo eléctrico de un edificio usando solo la temperatura exterior. Además, imaginemos que el consumo aumenta en días muy fríos por la calefacción y también en días muy calurosos por la refrigeración. En temperaturas templadas, en cambio, el consumo es menor.

La relación real podría tener forma de curva: alta en los extremos y baja en el centro. Si intentamos representarla con una regresión lineal simple, el modelo usará una recta. Esa recta difícilmente capture bien el patrón.

En este caso, el modelo puede equivocarse de manera sistemática. Tal vez subestime el consumo en los días extremos y lo sobreestime en los días templados. No importa

cuánto lo entrenemos: si la forma del modelo no permite representar esa curva, el problema persistirá.

Algo similar puede ocurrir en clasificación. Imaginemos que queremos separar dos clases que se distribuyen de manera no lineal. Si usamos un modelo que solo puede trazar una frontera de decisión muy simple, quizá no logre separar adecuadamente los grupos.

Por ejemplo, una regresión logística básica puede funcionar muy bien cuando las clases se separan aproximadamente mediante una frontera lineal. Pero si la separación real tiene una forma más compleja, puede ser necesario utilizar un modelo más flexible.

Esto no significa que los modelos simples sean malos. Al contrario, muchas veces son una excelente primera opción. Son más fáciles de entrenar, interpretar, comunicar y controlar. Una regresión lineal o una regresión logística pueden ser muy útiles si la relación principal es razonablemente simple.

El problema aparece cuando esa simplicidad impide aprender lo necesario. Un modelo simple puede ser útil e incluso preferible; un modelo demasiado simple, en cambio, no alcanza a representar el problema.

La diferencia no se decide por el nombre del algoritmo, sino por su desempeño y por el análisis de sus errores. Un modelo simple puede ser suficiente en un problema y totalmente insuficiente en otro. La pregunta no es si el modelo es simple, sino si su simplicidad alcanza para capturar los patrones relevantes.

Una señal de que un modelo es demasiado simple es el mal desempeño tanto en entrenamiento como en prueba. Si no logra buenos resultados ni siquiera con los datos que vio, probablemente no tenga suficiente capacidad, no cuente con variables adecuadas o la relación entre las variables y el objetivo no esté bien representada.

Por ejemplo, en regresión podríamos obtener:

MAE en entrenamiento:	35
MAE en prueba:	37

Si esos errores son altos para el problema, el modelo no está aprendiendo lo suficiente. La diferencia entre entrenamiento y prueba no es el problema principal. El problema es que ambos resultados son malos.

En clasificación podríamos observar algo similar:

F1-score en entrenamiento: 0.55
F1-score en prueba: 0.53

Si una referencia simple obtiene resultados parecidos, el modelo no está aportando demasiado.

Un modelo puede ser demasiado simple por varias razones. Una de ellas es que utilice pocas variables. Si queremos predecir precios de viviendas usando solo la superficie, podemos capturar una parte del problema, pero no toda. Si la ubicación es fundamental y no está incluida, el modelo no podrá usar esa información.

Otra razón es que las variables estén representadas de manera pobre. Tal vez tenemos una fecha, pero no extraemos el mes, el día de la semana o si se trata de un feriado. Tal vez tenemos una dirección, pero no transformamos esa información en una zona, una distancia o características del barrio. Tal vez disponemos de texto, pero lo ignoramos por completo.

En esos casos, el modelo parece simple no solo por el algoritmo elegido, sino también por la forma en que le presentamos los datos. Un modelo aprende de las variables que recibe. Si su representación es pobre, el aprendizaje también puede serlo.

El modelo también puede ser demasiado simple porque sus restricciones son muy fuertes. Un árbol de decisión con una profundidad máxima de 1 solo puede hacer una pregunta antes de decidir. A este tipo de árbol se lo suele llamar *stump* o tocón de decisión. Puede ser útil como componente de algunos métodos, pero por sí solo puede resultar insuficiente para problemas complejos. Si el problema requiere encadenar varias decisiones, un árbol de profundidad 1 puede subajustar.

En los modelos lineales, la simplicidad aparece en la forma de la relación que pueden representar. Una regresión lineal básica suma aportes proporcionales de las variables. Si el problema necesita curvas, interacciones o cambios de comportamiento según determinados rangos, esa forma puede quedarse corta.

Por ejemplo, el efecto de una promoción sobre las ventas puede depender del tipo de producto. La influencia de la temperatura sobre el consumo puede depender del horario. El impacto de la superficie sobre el precio puede depender del barrio. Una suma lineal básica no representa automáticamente esas interacciones. Si el efecto de una variable depende de otra, un modelo demasiado simple puede no capturarlo.

Para mejorar un modelo demasiado simple, una posibilidad es agregar variables relevantes. Si el modelo no tiene suficiente información, proporcionarle mejores datos puede ayudar más que cambiar de algoritmo.

Otra posibilidad es construir mejores características. Por ejemplo, podemos transformar fechas en variables útiles, agrupar categorías, crear indicadores, calcular promedios históricos o representar relaciones que el modelo no capturaría por sí solo.

También podemos usar un modelo más flexible. Si una regresión lineal no alcanza, más adelante podríamos probar árboles, Random Forest, Gradient Boosting u otros métodos. Si una frontera lineal no separa bien las clases, podríamos utilizar modelos capaces de representar fronteras más complejas. Además, siempre conviene revisar la formulación del problema para comprobar que la variable objetivo y las variables predictoras representen adecuadamente aquello que queremos estudiar.

Pero este aumento de complejidad debe hacerse con cuidado. Un modelo más flexible puede reducir el subajuste, pero también aumentar el riesgo de sobreajuste. La mejora debe medirse sobre datos no vistos y no únicamente sobre los datos de entrenamiento.

Por ejemplo, si un árbol de decisión poco profundo funciona mal, podemos permitirle una mayor profundidad. Al principio, eso puede mejorar el desempeño tanto en entrenamiento como en prueba. Sin embargo, si seguimos aumentando la profundidad sin control, el árbol puede comenzar a memorizar los datos de entrenamiento y perder capacidad de generalización. Aumentar la complejidad puede ayudar, pero solo hasta cierto punto; después puede aparecer el sobreajuste.

Por eso, no buscamos necesariamente el modelo más simple ni el más complejo. Buscamos el nivel de complejidad adecuado para el problema y los datos disponibles.

Un modelo simple tiene ventajas claras: es más fácil de interpretar, más rápido, más estable y más sencillo de comunicar. Si su desempeño es suficiente, puede ser preferible a un modelo complejo. Pero si sus errores son altos y sistemáticos, necesitamos darle más capacidad o mejor información.

La decisión debe basarse en evidencia. Primero evaluamos el modelo y luego analizamos sus errores. Si vemos que falla tanto en entrenamiento como en prueba y que no mejora una referencia simple, podemos sospechar que existe subajuste. A partir de ahí, debemos revisar si el modelo es demasiado simple, si faltan variables relevantes, si la representación de los datos es deficiente o si el problema está mal planteado.

Un modelo demasiado simple es aquel que no alcanza a capturar los patrones relevantes de los datos. Suele producir un mal desempeño tanto en entrenamiento como en prueba y se relaciona con el subajuste. La solución puede implicar agregar variables, mejorar la representación de los datos o usar modelos más flexibles. Sin embargo, aumentar la complejidad siempre debe evaluarse con cuidado para no pasar al extremo opuesto: el sobreajuste.

Modelos demasiado complejos

Un modelo demasiado complejo es aquel que tiene tanta flexibilidad que puede adaptarse en exceso a los datos de entrenamiento. En lugar de aprender solo los patrones generales, también puede capturar ruido, irregularidades, valores extremos o coincidencias accidentales. Cuando eso ocurre, el modelo puede funcionar muy bien con los datos conocidos, pero peor con datos nuevos.

Este tipo de modelo se relaciona directamente con el sobreajuste. Aprende muy bien el entrenamiento, pero puede perder capacidad de generalización.

La complejidad de un modelo no siempre es fácil de ver a simple vista. No depende únicamente de cuántas líneas de código usamos o de si el algoritmo parece avanzado. Depende de cuánta libertad tiene el modelo para ajustarse a los datos.

Un árbol de decisión muy profundo, por ejemplo, puede ser muy complejo. Si permitimos que siga dividiendo los datos una y otra vez, puede terminar creando reglas muy específicas para pequeños grupos de casos. Incluso puede llegar a hojas con uno o muy pocos ejemplos. En entrenamiento, eso puede producir resultados excelentes. Pero esas reglas pueden no servir para casos nuevos.

Supongamos un modelo para predecir abandono de clientes. Un árbol demasiado profundo podría aprender reglas como:

```
si el cliente tiene 37 años,  
vive en cierta zona,  
hizo 2 reclamos,  
compró cierto producto,  
y se conectó exactamente 4 veces,  
entonces abandona.
```

Tal vez esa regla funcione para algunos clientes del entrenamiento, pero no necesariamente representa un patrón estable. Puede ser una coincidencia. Si el modelo construye muchas reglas de ese tipo, corre riesgo de sobreajustar.

En regresión ocurre algo parecido. Un modelo muy flexible puede seguir demasiado de cerca los puntos del entrenamiento. Si los datos tienen ruido, la curva aprendida puede ondular para acercarse a cada observación, pero perder la tendencia general. Visualmente, sería una línea que parece encajar muy bien con los puntos conocidos, pero que se comporta de manera extraña entre ellos o frente a nuevos casos. Ajustarse demasiado a los puntos observados puede producir malas predicciones para puntos nuevos.

Esto no significa que la complejidad sea mala en sí misma. Muchos problemas reales necesitan modelos flexibles. Si la relación entre variables es no lineal, si hay interacciones importantes o si la frontera de decisión es compleja, un modelo simple puede quedarse corto. En esos casos, aumentar la complejidad puede mejorar la generalización.

El problema aparece cuando la complejidad supera lo que los datos pueden sostener. Una complejidad útil permite capturar patrones reales. Una complejidad excesiva, en cambio, puede capturar ruido y detalles accidentales.

La cantidad de datos disponibles influye mucho. Con pocos ejemplos, un modelo complejo tiene más riesgo de encontrar patrones casuales. Si hay muchas variables y pocas filas, el riesgo aumenta aún más. Algunas combinaciones de variables pueden parecer predictivas por azar, aunque no funcionen en datos nuevos.

También influye la calidad de los datos. Si hay errores de carga, etiquetas incorrectas, valores extremos no revisados o ruido fuerte, un modelo complejo puede intentar aprender esas irregularidades. Un modelo más simple quizá ignore parte de ese ruido y capture una tendencia más estable.

Por eso, ante un modelo demasiado complejo, no siempre la solución es cambiar inmediatamente de algoritmo. A veces conviene revisar primero los datos, las variables y el proceso de evaluación. Tal vez el problema no sea solo la complejidad del modelo, sino la información con la que se lo alimenta.

Una señal típica de exceso de complejidad es una gran diferencia entre el desempeño en entrenamiento y el desempeño en prueba. Por ejemplo:

Entrenamiento:
MAE = 1.5

Prueba:
MAE = 22.0

O en clasificación:

Entrenamiento:
F1-score = 0.98

Prueba:
F1-score = 0.61

En estos casos, el modelo parece muy bueno cuando evalúa los datos que ya vio, pero no conserva ese desempeño en datos no vistos. Esa brecha es una señal fuerte de sobreajuste.

También puede aparecer una mejora engañosa cuando miramos solo entrenamiento. Si aumentamos la complejidad de un modelo, muchas veces el desempeño en entrenamiento mejora. Un árbol más profundo, más variables o más flexibilidad pueden reducir errores sobre los datos conocidos. Pero esa mejora no siempre se traslada al conjunto de prueba. Más complejidad casi siempre puede mejorar el entrenamiento; lo importante es si mejora el desempeño sobre datos no vistos.

Esta es una idea central. No elegimos el modelo que mejor ajusta el entrenamiento. Elegimos el modelo que mejor generaliza.

Un ejemplo claro aparece con árboles de decisión. Si aumentamos `max_depth`, el árbol puede hacer más preguntas y construir reglas más específicas. Al principio, eso puede ayudar: el modelo deja de ser demasiado simple y captura patrones importantes. Pero, pasado cierto punto, más profundidad puede empezar a memorizar detalles del entrenamiento.

Una profundidad baja puede producir subajuste. Una profundidad intermedia puede ofrecer un buen equilibrio. Una profundidad muy alta puede aumentar el riesgo de sobreajuste.

Este tipo de comportamiento se observa también con otros hiperparámetros. Muchos modelos tienen controles que regulan su flexibilidad. En KNN, un valor muy bajo de `k` puede producir decisiones muy sensibles a casos individuales. En regresión lineal con muchas variables irrelevantes, el modelo puede ajustarse a ruido. En modelos más avanzados, también habrá parámetros que controlen la complejidad.

No necesitamos estudiar todos esos casos ahora. Lo importante es entender la idea general: la complejidad permite aprender patrones más ricos, pero también aumenta el riesgo de aprender detalles que no generalizan.

Para reducir el riesgo de modelos demasiado complejos, podemos usar distintas estrategias. Una es limitar la flexibilidad del modelo. En árboles, podemos limitar la profundidad, exigir más muestras por hoja o impedir divisiones con poca información. En otros modelos, podemos usar regularización o ajustar hiperparámetros que restrinjan el ajuste.

Controlar la complejidad significa darle al modelo suficiente libertad para aprender, pero no tanta como para memorizar.

Otra estrategia es mejorar la evaluación. Si usamos solo una separación train/test, podemos tener una idea inicial, pero tal vez dependa demasiado de esa partición. Más adelante estudiaremos validación cruzada, que permite evaluar de manera más estable usando varias divisiones de los datos.

También podemos reducir variables irrelevantes, revisar outliers, mejorar el preprocesamiento o conseguir más datos. Más datos de buena calidad suelen ayudar a los modelos complejos, porque les dan más ejemplos para distinguir patrones reales de coincidencias accidentales.

Para controlar modelos demasiado complejos, entonces, podemos limitar hiperparámetros, usar regularización, mejorar los datos, eliminar ruido, evaluar con validación y comparar siempre el desempeño en entrenamiento y prueba.

De todos modos, simplificar no significa elegir siempre el modelo más básico. Si simplificamos demasiado, podemos volver al subajuste. El objetivo es encontrar un equilibrio entre capacidad y generalización.

Un modelo demasiado simple no aprende suficiente. Un modelo demasiado complejo aprende demasiado de los detalles del entrenamiento. Un buen modelo aprende lo necesario para resolver el problema en datos nuevos.

En términos generales, un modelo demasiado simple suele mostrar alto error en entrenamiento y prueba. Un modelo demasiado complejo puede mostrar bajo error en entrenamiento, pero alto error en prueba. Un modelo adecuado, en cambio, mantiene un error razonable en entrenamiento y un buen desempeño en prueba.

También conviene considerar la interpretabilidad. Los modelos complejos pueden ser más difíciles de explicar. Si la mejora en métricas es pequeña, tal vez no justifique perder claridad. En cambio, si la mejora es grande y el problema lo requiere, puede tener sentido aceptar mayor complejidad, siempre que la evaluación sea sólida.

Por ejemplo, una regresión lineal puede ser preferible a un modelo muy complejo si sus errores son apenas mayores, pero sus resultados son más fáciles de comunicar. En cambio, si un modelo más flexible reduce mucho los errores en datos no vistos, puede ser la mejor opción para una tarea predictiva.

La decisión no es “simple contra complejo”. La decisión es qué nivel de complejidad necesita el problema y qué costo tiene esa complejidad.

La complejidad debe justificarse por una mejora real en datos no vistos y por el uso que tendrá el modelo.

Un modelo demasiado complejo tiene tanta flexibilidad que puede adaptarse a detalles específicos del conjunto de entrenamiento y perder capacidad de generalización. Suele mostrar muy buen desempeño en entrenamiento, pero peor desempeño en prueba. La complejidad puede ser necesaria para capturar patrones reales, pero debe controlarse mediante hiperparámetros, regularización, buena evaluación y análisis de errores. El objetivo no es usar el modelo más complejo, sino el que logre el mejor equilibrio entre aprendizaje, generalización e interpretabilidad.

Señales de alerta

El sobreajuste y el subajuste no siempre se ven de manera inmediata. Un modelo puede producir métricas, gráficos y predicciones que parecen razonables a primera vista, pero aun así tener problemas importantes. Por eso, al evaluar modelos, conviene prestar atención a ciertas señales de alerta.

La primera señal aparece al comparar el desempeño en entrenamiento y en prueba. Si el modelo funciona muy bien en entrenamiento, pero mucho peor en prueba, debemos sospechar sobreajuste.

Por ejemplo, en clasificación:

```
Accuracy en entrenamiento: 0.98
Accuracy en prueba:        0.71
```

O en regresión:

```
MAE en entrenamiento: 3.2
MAE en prueba:        28.5
```

En ambos casos, el modelo parece haber aprendido muy bien los datos de entrenamiento, pero no logra mantener ese rendimiento en datos nuevos. La diferencia entre ambos resultados es demasiado grande como para ignorarla.

No existe un número universal que indique cuándo la diferencia es aceptable y cuándo no. Depende del problema, del tamaño del dataset, de la métrica y de la variabilidad natural de los datos. Pero la idea general es clara: una brecha grande entre entrenamiento y prueba debe investigarse.

La segunda señal aparece cuando el modelo funciona mal tanto en entrenamiento como en prueba. En ese caso, el problema probablemente no sea sobreajuste, sino subajuste o falta de señal en los datos.

Por ejemplo:

```
MAE en entrenamiento: 42.0
MAE en prueba:        44.3
```

Si ese nivel de error es alto para el problema, el modelo no está aprendiendo lo necesario. La cercanía entre entrenamiento y prueba no significa que el modelo sea bueno. Solo indica que falla de manera parecida en ambos conjuntos.

En clasificación, una señal parecida podría ser:

```
F1-score en entrenamiento: 0.48
F1-score en prueba:        0.46
```

Si una referencia simple obtiene resultados similares, el modelo no está aportando mucho.

La tercera señal de alerta aparece cuando el modelo no mejora una estrategia básica. En regresión, una referencia simple puede ser predecir siempre el promedio o la mediana de la variable objetivo. En clasificación, puede ser predecir siempre la clase mayoritaria. Si el modelo entrenado apenas mejora esa referencia, debemos revisar si realmente está aprendiendo patrones útiles.

```
Referencia simple:
MAE = 20.0
```

```
Modelo entrenado:
MAE = 19.4
```

La mejora existe, pero es pequeña. Tal vez no justifique la complejidad del modelo. También puede indicar que las variables predictoras tienen poca señal, que el problema está mal planteado o que el preprocesamiento no está aprovechando la información disponible.

La cuarta señal aparece cuando los resultados son demasiado buenos para ser ciertos. En problemas reales, un desempeño casi perfecto merece una revisión cuidadosa. Puede ser real en problemas muy simples o datasets muy limpios, pero muchas veces indica una evaluación incorrecta.

```
Accuracy = 1.00
F1-score = 1.00
```

$$\begin{aligned} \text{MAE} &\approx 0 \\ R^2 &\approx 1.00 \end{aligned}$$

Un resultado perfecto puede deberse a fuga de datos, evaluación sobre el conjunto de entrenamiento, duplicados entre entrenamiento y prueba, variables que revelan la respuesta o errores en la separación de datos. No debemos celebrar automáticamente un resultado perfecto sin revisar cómo se obtuvo.

La quinta señal son métricas muy diferentes entre clases o grupos. Un modelo puede tener una métrica global aceptable y, sin embargo, fallar en una clase minoritaria o en un segmento importante.

Por ejemplo, en clasificación:

```
Clase mayoritaria:  
recall = 0.95
```

```
Clase minoritaria:  
recall = 0.30
```

La accuracy global podría parecer buena, pero el modelo está fallando en la clase que quizá más importa. Esto es frecuente en problemas con desbalance de clases.

En regresión también puede ocurrir algo parecido. El modelo puede tener buen error promedio, pero fallar mucho en valores altos, en ciertas zonas, en determinados productos o en períodos específicos.

```
MAE general: 8 unidades
```

```
MAE en días normales: 5 unidades  
MAE en días de promoción: 35 unidades
```

El promedio general oculta que el modelo falla justamente en un segmento relevante. Por eso, las métricas globales deben complementarse con análisis por grupos, rangos o condiciones importantes.

La sexta señal aparece en los residuos o errores. Si los errores muestran un patrón claro, el modelo puede estar dejando información sin capturar. En regresión, por ejemplo, si el modelo subestima sistemáticamente los valores altos y sobreestima los valores bajos, puede estar usando una forma demasiado simple.

Un buen análisis de errores no busca solo calcular una métrica. Busca entender dónde y cómo se equivoca el modelo. Si los errores se concentran en una zona, un grupo o un tipo de caso, hay una pista para mejorar.

La séptima señal de alerta es la inestabilidad. Si pequeños cambios en los datos producen resultados muy diferentes, el modelo puede ser demasiado sensible. Esto puede ocurrir con datasets pequeños, modelos muy flexibles o datos con mucho ruido.

Por ejemplo, si al cambiar la separación train/test el modelo pasa de un R^2 de 0.82 a uno de 0.35, conviene investigar. Tal vez el conjunto de datos sea pequeño, la partición sea poco representativa o el modelo dependa demasiado de algunos casos particulares.

Más adelante, la validación cruzada nos ayudará a evaluar esta variabilidad con más criterio. Por ahora, alcanza con reconocer que una única métrica obtenida en una única separación no siempre cuenta toda la historia.

La octava señal aparece cuando el modelo depende demasiado de variables sospechosas. Si una variable tiene una importancia enorme o un coeficiente inesperadamente dominante, debemos revisar si esa variable es legítima. Tal vez contiene información futura, resume indirectamente la respuesta o no estaría disponible al momento de predicción.

Por ejemplo, en un modelo de abandono de clientes, una variable como `fecha_baja` no debería usarse para predecir si el cliente abandonará. En un modelo de aprobación de créditos, una variable calculada después de la decisión tampoco sería válida. Si el modelo se apoya en ese tipo de columnas, puede parecer excelente en evaluación, pero fallar en uso real.

Esta señal conecta directamente con la fuga de datos, que retomaremos al final del capítulo.

La novena señal aparece cuando el desempeño en entrenamiento mejora al aumentar complejidad, pero el desempeño en prueba empeora. Este comportamiento es típico cuando cruzamos el punto de equilibrio y empezamos a sobreajustar.

Por ejemplo, en un árbol de decisión:

Profundidad 3:
prueba razonable.

Profundidad 8:
mejor entrenamiento, prueba similar.

Profundidad 20:
entrenamiento casi perfecto, prueba peor.

El modelo se vuelve cada vez más capaz de memorizar entrenamiento, pero eso no mejora la generalización. En ese caso, conviene limitar la complejidad.

La décima señal es la falta de coherencia con el conocimiento del problema. Si el modelo produce predicciones absurdas, coeficientes difíciles de justificar o reglas que contradicen de manera extraña lo esperable, no necesariamente está mal, pero merece revisión.

Por ejemplo, un modelo de consumo que predice valores negativos, un modelo de precios que asigna enorme importancia a un identificador interno o un modelo de ventas que ignora por completo una variable claramente relevante pueden indicar problemas de datos, preprocesamiento o formulación.

La evaluación técnica debe dialogar con el sentido del problema. Las métricas son necesarias, pero no sustituyen la revisión conceptual.

Estas señales de alerta no deben interpretarse como diagnósticos automáticos. Una brecha entre entrenamiento y prueba puede indicar sobreajuste, pero debemos confirmar. Un mal desempeño general puede indicar subajuste, pero también datos insuficientes o un problema muy difícil. Un resultado perfecto puede tener explicación legítima, aunque siempre conviene revisar.

Lo importante es usar estas señales como disparadores de análisis. Una señal de alerta no cierra el diagnóstico: abre una pregunta.

Al evaluar modelos debemos mirar más que una métrica final. Señales como gran diferencia entre entrenamiento y prueba, mal desempeño en ambos conjuntos, resultados demasiado perfectos, errores concentrados en ciertos grupos, patrones en residuos, inestabilidad o dependencia de variables sospechosas pueden indicar sobreajuste, subajuste, fuga de datos o problemas de formulación. Detectar estas señales a tiempo permite corregir el modelo antes de confiar en predicciones que no generalizan.

Relación con fuga de datos

La fuga de datos es una de las causas más peligrosas de evaluaciones engañosas. Puede hacer que un modelo parezca excelente durante la evaluación, pero que falle cuando se lo intenta usar en una situación real. Por eso, al estudiar sobreajuste, subajuste y generalización, debemos volver a esta idea.

La fuga de datos ocurre cuando el modelo recibe información que no debería estar disponible en el momento de hacer la predicción. Esa información puede revelar directa o indirectamente la respuesta. Como resultado, el modelo obtiene métricas muy altas, pero no porque haya aprendido patrones útiles, sino porque tuvo acceso a pistas indebidas.

Esto se relaciona con la generalización porque una evaluación con fuga no mide realmente la capacidad del modelo para funcionar con datos nuevos. Mide su capacidad para aprovechar información que no debería estar ahí.

Supongamos que queremos predecir si un cliente abandonará un servicio. La variable objetivo es:

`abandona = sí / no`

Si entre las variables de entrada incluimos una columna como `fecha_baja`, el modelo tendrá una pista directa. Los clientes que tienen una fecha de baja probablemente abandonaron. El modelo podrá lograr un desempeño altísimo, pero esa predicción no será válida, porque en el momento en que queremos anticipar el abandono esa fecha todavía no existiría.

En este caso, el modelo no está prediciendo el futuro. Está leyendo una consecuencia del evento que supuestamente debía anticipar.

Algo similar puede ocurrir en modelos de crédito. Si queremos predecir el riesgo de una solicitud antes de aprobarla, no podemos usar variables que solo se conocen después de la aprobación, como comportamiento de pago posterior, mora futura o decisiones administrativas tomadas después del análisis.

En diagnóstico asistido, si un modelo se supone que debe ayudar con una decisión inicial, no debería usar resultados de estudios que solo se realizan después de esa decisión. En ventas, si queremos predecir demanda futura, no podemos usar columnas calculadas con ventas reales futuras. En mantenimiento predictivo, si queremos anticipar una falla, no debemos incluir variables registradas después de que la falla ocurrió.

La pregunta clave siempre es temporal: ¿esta información estaría disponible en el momento real de hacer la predicción?

Si la respuesta es no, la variable no debería estar en `X`.

La fuga de datos puede ser muy evidente, como una columna que contiene directamente la respuesta. Pero muchas veces es más sutil. Puede aparecer en variables derivadas, acumulados, promedios, indicadores, fechas, estados administrativos o transformaciones realizadas antes de separar entrenamiento y prueba.

Por ejemplo, una columna llamada `cantidad_reclamos_ultimos_30_dias` podría ser válida o inválida según cómo se haya calculado. Si se calcula usando solo información anterior al momento de predicción, puede ser una variable legítima. Pero si se calcula

mirando un período que incluye días posteriores al evento que queremos predecir, puede introducir fuga.

Una variable puede parecer válida por su nombre, pero ser inválida por la forma en que fue calculada.

Este punto es importante porque no alcanza con revisar la tabla final. Debemos entender el proceso que generó cada columna. En proyectos reales, muchas variables ya vienen construidas por sistemas anteriores, reportes o consultas. Si no sabemos qué período usan o qué información resumen, podemos incluir fuga sin darnos cuenta.

La fuga también puede aparecer durante el preprocesamiento. Por ejemplo, si imputamos valores faltantes usando todo el dataset antes de separar entrenamiento y prueba, el conjunto de entrenamiento recibe información estadística del conjunto de prueba. Lo mismo ocurre si escalamos variables, seleccionamos características o calculamos transformaciones usando todos los datos antes del split.

Por eso insistimos en que las transformaciones que aprenden algo de los datos deben ajustarse solo con el conjunto de entrenamiento. Luego se aplican al conjunto de prueba. Esta es una de las razones para usar `Pipeline`: ayuda a ordenar el flujo y reducir errores metodológicos.

La fuga de datos puede confundirse con sobreajuste porque ambos producen problemas de generalización. Pero no son exactamente lo mismo.

En el sobreajuste, el modelo aprende demasiado los detalles del conjunto de entrenamiento. El problema está en que se adapta a ruido o particularidades que no se repiten.

En la fuga de datos, el problema es que el modelo recibe información que no debería tener. Puede funcionar bien incluso en el conjunto de prueba, si esa información filtrada también está presente allí. Por eso la fuga puede ser más peligrosa: la evaluación puede parecer excelente y no mostrar la caída hasta el uso real.

Un modelo con fuga puede obtener buen desempeño tanto en entrenamiento como en prueba. Eso lo hace difícil de detectar si solo miramos métricas. Por eso, ante resultados muy buenos, debemos revisar las variables y el flujo de datos, no solo las métricas.

Por ejemplo, imaginemos un modelo de abandono con estos resultados:

```
Accuracy en entrenamiento: 0.99
Accuracy en prueba:        0.98
```

A simple vista parece excelente. La diferencia entre entrenamiento y prueba es pequeña, así que no parece un caso clásico de sobreajuste. Pero si el modelo usa una columna que revela el abandono, la evaluación está contaminada. El problema no se ve comparando solo métricas.

Esto muestra que la fuga de datos puede producir una falsa generalización. El modelo parece generalizar porque funciona bien en prueba, pero en realidad el conjunto de prueba no representa correctamente el escenario futuro. Contiene información que tampoco estará disponible cuando se use el modelo.

Para prevenir fuga de datos, debemos definir claramente el momento de predicción. Antes de construir variables, tenemos que saber en qué instante se supone que el modelo hará su predicción y qué información existe en ese momento.

Por ejemplo, una predicción de abandono podría hacerse al cierre de cada mes, con información disponible hasta ese día. Una predicción de demanda podría hacerse antes de iniciar la semana, usando solo datos históricos anteriores. Una predicción de riesgo crediticio debería hacerse antes de aprobar o rechazar la solicitud.

Una vez definido ese momento, podemos revisar cada variable con algunas preguntas simples: ¿existe antes de la predicción?, ¿fue calculada solo con datos anteriores?, ¿depende directa o indirectamente de la respuesta?, ¿estaría disponible para casos nuevos?

Si una variable no pasa estas preguntas, debemos excluirla o reconstruirla correctamente.

Otra forma de prevenir fuga es separar los datos antes de realizar transformaciones que aprendan información estadística. Imputadores, escaladores, codificadores y selectores de variables deben ajustarse con entrenamiento. El conjunto de prueba debe permanecer como datos no vistos hasta el momento de evaluación.

También conviene revisar duplicados y relaciones entre registros. Si el mismo caso, cliente, paciente, producto o evento aparece en entrenamiento y prueba de formas muy parecidas, la evaluación puede estar inflada. En algunos problemas, no basta con separar filas al azar. Puede ser necesario separar por grupo o por tiempo.

Por ejemplo, si queremos evaluar desempeño sobre clientes nuevos, todas las filas de un mismo cliente deberían quedar en el mismo conjunto. Si algunas están en entrenamiento y otras en prueba, el modelo puede aprovechar información del cliente que no tendría para un cliente completamente nuevo.

La fuga de datos también se relaciona con resultados demasiado buenos para ser ciertos. Un modelo con métricas perfectas o casi perfectas debe revisarse

cuidadosamente. No porque sea imposible obtener buenos resultados, sino porque una de las explicaciones más frecuentes de resultados sospechosamente altos es la presencia de fuga.

En esos casos, conviene inspeccionar variables demasiado predictivas, columnas calculadas después del evento, duplicados entre entrenamiento y prueba, preprocesamiento antes del split, uso repetido del conjunto de prueba y errores en el cálculo de métricas.

Detectar fuga puede ser incómodo porque suele reducir las métricas. Al quitar una variable filtrada o corregir el flujo, el desempeño puede bajar mucho. Pero esa caída no significa que empeoramos el modelo. Significa que dejamos de medir una ventaja artificial y empezamos a evaluar de manera más honesta.

Una métrica más baja pero honesta es mejor que una métrica alta producida por fuga.

Esta idea es fundamental para construir modelos confiables. En Machine Learning, el objetivo no es ganar una métrica en un entorno artificial, sino producir predicciones útiles en el contexto real de uso. Si la evaluación no respeta ese contexto, las conclusiones serán engañosas.

La fuga de datos ocurre cuando el modelo usa información que no estaría disponible al momento real de predicción. Puede generar resultados extremadamente altos y una falsa sensación de generalización. Se relaciona con el sobreajuste porque ambos afectan la capacidad de funcionar en datos nuevos, pero la fuga puede ser más difícil de detectar solo con métricas. Para prevenirla, debemos definir el momento de predicción, revisar la disponibilidad de variables, separar correctamente los datos y aplicar transformaciones sin aprender del conjunto de prueba.

Aprender lo suficiente, pero no demasiado

En este capítulo estudiamos una de las ideas centrales para construir modelos útiles: la generalización. Un modelo no se entrena para repetir los datos que ya conoce, sino para aplicar lo aprendido a casos nuevos. Por eso, evaluar solo el desempeño en entrenamiento no alcanza. Necesitamos saber si el modelo mantiene un comportamiento razonable cuando se enfrenta a datos que no vio.

El sobreajuste aparece cuando el modelo aprende demasiado los detalles del conjunto de entrenamiento. Puede capturar ruido, valores extremos, coincidencias

accidentales o reglas muy específicas que no se repiten en datos nuevos. La señal típica es un desempeño muy bueno en entrenamiento y mucho peor en prueba.

Sobreajuste: muy buen desempeño en entrenamiento, peor desempeño en prueba.

El subajuste aparece cuando el modelo no aprende lo suficiente. Puede ser demasiado simple, recibir pocas variables relevantes, estar mal preparado o no tener capacidad para representar la relación entre las variables y la variable objetivo. La señal típica es un mal desempeño tanto en entrenamiento como en prueba.

Subajuste: mal desempeño en entrenamiento, mal desempeño en prueba.

Estos dos extremos ayudan a entender el equilibrio que buscamos. Un modelo demasiado simple puede ignorar patrones importantes. Un modelo demasiado complejo puede memorizar detalles irrelevantes. Un buen modelo debe tener la capacidad suficiente para aprender relaciones reales, pero no tanta flexibilidad como para adaptarse al ruido.

Modelo demasiado simple: no aprende lo necesario.

Modelo demasiado complejo: aprende detalles que no generalizan.

Modelo adecuado: aprende patrones útiles para datos nuevos.

También vimos que las señales de alerta no deben interpretarse de manera automática, pero sí deben motivar revisión. Una gran diferencia entre entrenamiento y prueba puede indicar sobreajuste. Malos resultados en ambos conjuntos pueden indicar subajuste. Métricas casi perfectas pueden revelar fuga de datos, duplicados, evaluación incorrecta o variables que contienen la respuesta.

Los resultados demasiado buenos para ser ciertos merecen especial atención. En problemas reales, un modelo perfecto no debe celebrarse sin revisar primero el flujo completo: separación de datos, variables usadas, preprocesamiento, duplicados, momento de predicción y cálculo de métricas.

La fuga de datos es una causa especialmente peligrosa de evaluaciones engañosas. Ocurre cuando el modelo recibe información que no estaría disponible en el momento real de hacer la predicción. Esto puede producir métricas muy altas, incluso en prueba, pero no representa verdadera capacidad de generalización. Una métrica alta no garantiza que la evaluación sea válida. Primero hay que verificar que el modelo no haya recibido información indebida.

Para prevenir estos problemas, es fundamental definir con claridad el momento de predicción, separar correctamente los datos, ajustar transformaciones solo con entrenamiento, cuidar el conjunto de prueba y revisar si las variables estarían realmente disponibles en el uso futuro del modelo.

Construir modelos útiles implica aprender lo suficiente, pero no demasiado. El objetivo no es obtener el mejor resultado posible sobre los datos conocidos, sino lograr un desempeño confiable en datos nuevos. Sobreajuste, subajuste y fuga de datos son riesgos distintos, pero todos afectan la confianza que podemos tener en un modelo. Reconocerlos nos prepara para el próximo paso: evaluar de manera más estable usando validación cruzada.

Capítulo 18 · Validación cruzada

Limitaciones de una única separación train/test

Hasta ahora evaluamos los modelos separando los datos en dos partes: un conjunto de entrenamiento y un conjunto de prueba. Esta estrategia es fundamental porque nos permite entrenar el modelo con algunos datos y evaluar su desempeño sobre casos que no vio durante el entrenamiento.

La lógica general era sencilla: partimos de los datos disponibles, los dividimos en entrenamiento y prueba, el modelo aprende con el conjunto de entrenamiento y luego evaluamos su desempeño con el conjunto de prueba.

Esta separación es mucho mejor que entrenar y evaluar con los mismos datos. Nos da una primera aproximación a la capacidad de generalización del modelo. Sin embargo, también tiene una limitación importante: el resultado puede depender demasiado de **cómo se realizó esa única separación**.

Cuando usamos una única división train/test, algunos casos quedan en entrenamiento y otros quedan en prueba. Si cambiamos la semilla aleatoria, o si hacemos otra división, los casos pueden distribuirse de otra manera. Como consecuencia, las métricas pueden cambiar. Una única separación train/test puede dar una primera mirada del desempeño, pero no siempre ofrece una estimación estable.

Supongamos que tenemos un dataset de viviendas y queremos predecir precios. Si en el conjunto de prueba caen muchas viviendas difíciles de predecir, el modelo puede parecer peor de lo que realmente es. Si, en cambio, caen casos muy parecidos a los del entrenamiento, el modelo puede parecer mejor.

El modelo es el mismo, pero la evaluación cambia porque cambió el conjunto de prueba. En un primer split, la prueba podría contener casos difíciles y producir métricas peores. En otro split, podría contener casos más simples y producir métricas mejores.

Esto no significa que la separación train/test esté mal. Significa que una sola separación puede ser insuficiente para tener una idea robusta del desempeño, especialmente cuando el dataset no es muy grande.

El problema se vuelve más evidente con pocos datos. Si tenemos miles o millones de ejemplos, una separación aleatoria razonable puede representar bastante bien al conjunto completo. Pero si tenemos pocas filas, la selección de casos para entrenamiento y prueba puede influir mucho en el resultado.

Por ejemplo, si tenemos solo 100 registros y usamos 20 para prueba, esos 20 casos tienen un peso enorme en la métrica final. Si por azar contienen muchos casos atípicos, el modelo parecerá malo. Si contienen casos fáciles, parecerá bueno. Con pocos datos, cada caso del conjunto de prueba influye mucho en la evaluación.

También puede ocurrir que algunas clases queden mal representadas. En clasificación, si una clase es minoritaria, una única separación puede dejar pocos ejemplos de esa clase en prueba o en entrenamiento. Esto puede afectar tanto el aprendizaje como la evaluación.

Ya vimos que `stratify=y` ayuda a conservar proporciones de clases en una separación train/test. Pero incluso con estratificación, seguimos teniendo una sola división. La métrica obtenida sigue dependiendo de ese reparto particular.

En regresión también puede haber problemas de representación. Tal vez el conjunto de prueba contiene pocos valores altos de la variable objetivo, o pocos casos de cierta zona, producto, período o segmento. Entonces, la métrica puede no reflejar bien cómo funcionaría el modelo en toda la variedad de casos. El conjunto de prueba debería representar la diversidad del problema, pero con una sola separación eso no siempre ocurre suficientemente bien.

Otra limitación es que una única separación usa solo una parte de los datos para entrenar. Si reservamos 20% para prueba, el modelo se entrena con el 80%. Eso es razonable, pero cuando el dataset es pequeño puede sentirse costoso. Nos gustaría evaluar con datos no vistos, pero también nos gustaría que el modelo pudiera aprender de la mayor cantidad posible de ejemplos.

Hay una tensión natural: más datos para entrenamiento permiten que el modelo aprenda con más ejemplos, mientras que más datos para prueba pueden hacer que la evaluación sea más confiable. Con una sola separación, tenemos que elegir un reparto fijo.

La validación cruzada aparece como una forma de aprovechar mejor los datos disponibles para evaluar modelos. En lugar de hacer una única separación, realizamos varias divisiones. Cada parte del dataset tiene oportunidad de funcionar como prueba en algún momento, mientras el resto se usa para entrenar.

Todavía no necesitamos entrar en el detalle del procedimiento. La idea general es que, en lugar de obtener una única métrica, obtenemos varias métricas, una por cada división. Luego podemos calcular un promedio y observar cuánta variabilidad hay entre resultados. Con una única separación obtenemos una sola métrica. Con validación cruzada obtenemos varias separaciones, varias métricas y una estimación más estable.

Esto permite responder preguntas más ricas. No solo preguntamos “¿cuánto dio la métrica en este split?”, sino también “¿el modelo funciona de manera parecida en distintas divisiones?” y “¿su desempeño es estable o depende mucho de qué casos caen en prueba?”.

Por ejemplo, imaginemos que evaluamos un modelo de regresión con una sola separación y obtenemos un `MAE = 12.4`. Ese número puede parecer claro, pero no sabemos si es estable. Tal vez con otra separación habría dado 9.8 o 18.5.

Con validación cruzada, podríamos obtener varios valores: `MAE = 11.9` en una división, `MAE = 12.7` en otra, luego `13.1`, `12.2` y `12.5`. En este caso, los valores son bastante parecidos. Eso da más confianza en que el desempeño del modelo es estable.

Pero también podríamos obtener valores mucho más dispersos: por ejemplo, `MAE = 7.5` en una división, `22.8` en otra, luego `10.1`, `19.6` y `8.3`. Aquí el promedio puede ser útil, pero la variabilidad también es una señal importante. El modelo parece depender mucho de qué datos se usan para entrenar y evaluar. Tal vez el dataset es pequeño, hay casos atípicos, faltan variables o el modelo es inestable.

Esta información se pierde si miramos una sola separación.

En clasificación sucede lo mismo. Una única separación puede dar un F1-score determinado, pero otra puede dar un valor distinto. Esto es especialmente importante si hay desbalance de clases o si algunos casos son difíciles de clasificar. Una sola métrica puede ocultar si el modelo es estable o si tuvo suerte con la división de datos.

También debemos considerar la comparación entre modelos. Supongamos que entrenamos dos modelos y los evaluamos con una única separación. El Modelo A obtiene `accuracy = 0.84` y el Modelo B obtiene `accuracy = 0.82`.

A primera vista, el Modelo A parece mejor. Pero la diferencia es pequeña. ¿Qué pasaría con otra separación? Tal vez el Modelo B ganaría. Tal vez ambos serían prácticamente iguales. Tal vez uno sería mucho más estable que el otro.

Con una única separación, es fácil sacar conclusiones demasiado fuertes a partir de diferencias pequeñas. Por eso, las diferencias pequeñas en un único split deben interpretarse con prudencia.

La validación cruzada ayuda a comparar con más criterio porque evalúa cada modelo en varias divisiones. Así podemos observar no solo el promedio de desempeño, sino también la variabilidad. Un modelo con promedio apenas mayor pero resultados muy inestables puede no ser preferible a otro un poco más simple y estable.

Esto no significa que la validación cruzada reemplace completamente al conjunto de prueba final. En muchos flujos de trabajo, se usa validación cruzada para comparar modelos y elegir configuraciones, y se reserva un conjunto de prueba final para una última evaluación honesta. Esa distinción se volverá más importante cuando trabajemos con búsqueda de hiperparámetros.

Por ahora, lo central es entender la limitación de una única separación: puede dar una estimación útil, simple y necesaria, pero dependiente de una partición particular de los datos. El train/test split sigue siendo una herramienta fundamental, aunque puede depender demasiado de una única división.

También debemos tener cuidado con datos temporales o agrupados. La validación cruzada tradicional no siempre es adecuada para todos los casos. Si los datos tienen orden temporal, no podemos mezclar libremente pasado y futuro. Si hay múltiples registros por cliente, paciente o producto, debemos evitar que información de una misma entidad quede repartida de manera engañosa entre entrenamiento y prueba.

Esto no invalida la validación cruzada, pero nos recuerda que las divisiones deben respetar el problema. Evaluar bien no es solo aplicar una función; es dividir los datos de una manera que represente el uso real del modelo.

Una única separación train/test es una herramienta fundamental para evaluar generalización, pero tiene limitaciones. El resultado puede depender del reparto particular de los datos, especialmente en datasets pequeños, desbalanceados o con casos muy variados. Además, una única métrica no muestra la variabilidad del desempeño ni permite comparar modelos con suficiente estabilidad. La validación cruzada surge justamente como una forma de evaluar modelos usando varias divisiones, obteniendo una estimación más robusta y menos dependiente de un único split.

K-Fold Cross Validation

La validación cruzada permite evaluar un modelo usando varias divisiones de los datos, en lugar de depender de una única separación entre entrenamiento y prueba. La forma más conocida es **K-Fold Cross Validation**, o validación cruzada con K particiones.

La idea es dividir el dataset en **K** partes aproximadamente iguales. A cada una de esas partes se la llama **fold**. Luego, el modelo se entrena varias veces. En cada repetición, una de las partes se usa como conjunto de validación y las demás se usan para entrenar.

Por ejemplo, si usamos 5 folds, dividimos los datos en cinco partes: fold 1, fold 2, fold 3, fold 4 y fold 5.

En la primera repetición, el modelo se entrena con los folds 2, 3, 4 y 5, y se evalúa con el fold 1. En la segunda repetición, se entrena con los folds 1, 3, 4 y 5, y se evalúa con el fold 2. En la tercera repetición, se entrena con los folds 1, 2, 4 y 5, y se evalúa con el fold 3. Luego se repite el mismo criterio hasta que cada fold haya sido usado una vez como validación.

Al final, obtenemos cinco resultados de evaluación. Si la métrica es accuracy, tendremos cinco valores de accuracy. Si usamos MAE, tendremos cinco valores de MAE. Si usamos F1-score, tendremos cinco valores de F1-score.

Luego podemos calcular el promedio de esos resultados. Ese promedio ofrece una estimación más estable que una única separación train/test, porque cada caso del dataset tuvo oportunidad de formar parte de la validación una vez. Dicho de otra manera, en K-Fold cada parte del dataset se usa una vez para validar y varias veces para entrenar.

Esto permite aprovechar mejor los datos disponibles. En una única separación train/test, algunos casos quedan reservados para prueba y no participan del entrenamiento. En K-Fold, en cambio, el modelo se entrena varias veces y cada caso se usa para evaluación en alguna repetición.

Es importante entender que no se obtiene un único modelo durante la validación cruzada, sino varios entrenamientos temporales. Si usamos 5 folds, el algoritmo entrena 5 modelos, uno por cada división. Estos modelos sirven para evaluar el desempeño esperado. Después, cuando ya elegimos el enfoque, podemos entrenar un modelo final con todos los datos de entrenamiento disponibles.

Este punto suele generar confusión. La validación cruzada no es un modelo distinto. Es una estrategia de evaluación. Nos ayuda a medir qué tan bien funciona un modelo o una configuración cuando cambia la partición de los datos. Durante la validación cruzada se entrenan varios modelos temporales para estimar el desempeño; el modelo final se entrena después, con la configuración elegida.

El valor de K puede variar. Uno de los valores más frecuentes es $K = 5$. También es común usar $K = 10$. Si usamos $K = 5$, tendremos cinco entrenamientos y cinco evaluaciones. Si usamos $K = 10$, tendremos diez entrenamientos y diez evaluaciones. En general, si usamos más folds, cada entrenamiento se realiza con una proporción mayor de los datos, pero también se hacen más entrenamientos, por lo que el costo computacional aumenta.

No existe un valor de K perfecto para todos los casos. En muchos problemas tabulares, 5 o 10 folds son opciones razonables. La elección depende del tamaño del dataset, del costo de entrenar el modelo y de la estabilidad que necesitamos en la evaluación.

Si el dataset es pequeño, la validación cruzada puede ser especialmente útil porque permite que todos los casos participen en la evaluación sin reservar permanentemente una parte fija. Si el dataset es muy grande, una separación train/test puede ser suficiente para una primera evaluación, y la validación cruzada puede volverse costosa, según el modelo, el objetivo y el tiempo de entrenamiento disponible.

La validación cruzada también permite observar la variabilidad del desempeño. No solo nos interesa el promedio. También queremos saber si el modelo funciona de manera parecida en todos los folds o si depende mucho de qué casos caen en validación.

Por ejemplo, si obtenemos valores de accuracy como 0.81, 0.83, 0.82, 0.84 y 0.82, podemos decir que los resultados son bastante estables. El promedio será cercano a 0.82 y la variabilidad será baja.

En cambio, si obtenemos valores como 0.92, 0.61, 0.88, 0.57 y 0.90, el promedio puede no contar toda la historia. El modelo funciona muy bien en algunos folds y mal en otros. Eso puede indicar que el dataset es pequeño, que hay casos muy diferentes entre sí, que hay clases mal distribuidas, que el modelo es inestable o que la división no está respetando alguna estructura importante de los datos.

Por eso, K-Fold no solo mejora la estimación promedio. También nos permite detectar inestabilidad. El promedio resume el desempeño general, pero la variabilidad muestra qué tan estable es ese desempeño.

K-Fold se usa tanto en clasificación como en regresión. Lo que cambia es la métrica. En clasificación podemos evaluar accuracy, F1-score, recall, precision u otras métricas. En regresión podemos evaluar MAE, RMSE, R^2 u otras. La estructura de la validación es la misma.

Sin embargo, en clasificación hay un detalle importante: si las clases están desbalanceadas, una división K-Fold común puede dejar algunos folds con proporciones de clases poco representativas. Para resolver esto, suele usarse una variante llamada **Stratified K-Fold**, que veremos en la próxima sección.

En regresión, no hay clases para estratificar de la misma manera, aunque igualmente debemos revisar que los folds representen bien la diversidad de valores. Si todos los valores extremos quedan en un fold, la evaluación puede volverse inestable.

También debemos recordar que K-Fold tradicional supone que podemos mezclar los datos sin romper la lógica del problema. Esto no siempre es cierto. Si los datos tienen un orden temporal, como ventas por día, consumo por hora o precios a lo largo del tiempo, no deberíamos mezclar pasado y futuro libremente. En esos casos se necesitan estrategias de validación que respeten el tiempo.

Lo mismo ocurre si hay grupos. Si varias filas pertenecen al mismo cliente, paciente, producto, curso o dispositivo, puede ser necesario evitar que datos de la misma entidad aparezcan en entrenamiento y validación al mismo tiempo. De lo contrario, la evaluación puede ser demasiado optimista.

Estas excepciones no invalidan K-Fold. Simplemente muestran que la validación debe adaptarse al problema. Dividir datos no es un paso mecánico: debe respetar cómo se usará el modelo.

En `scikit-learn`, la validación cruzada puede realizarse con `cross_val_score`. Por ejemplo, para evaluar un modelo con 5 folds:

```
from sklearn.model_selection import cross_val_score

scores = cross_val_score(modelo, X, y, cv=5)
scores
```

Este código entrena y evalúa el modelo cinco veces, usando distintas particiones. El resultado es una lista de cinco puntajes, uno por cada fold.

Luego podemos calcular el promedio:

```
scores.mean()
```

Y también la variabilidad:

```
scores.std()
```

La interpretación dependerá de la métrica usada. Por defecto, `cross_val_score` utiliza la métrica asociada al método `score` del modelo. En clasificadores, suele ser accuracy. En regresores, suele ser R^2 . Si queremos otra métrica, debemos indicarla con el parámetro `scoring`.

Por ejemplo, para usar MAE en regresión, `scikit-learn` emplea una convención particular: muchas métricas de error se expresan como valores negativos dentro de `cross_val_score`, porque la función está pensada para que valores mayores sean mejores. Este detalle puede resultar confuso al principio y conviene tratarlo con cuidado en los cuadernos prácticos.

Por ahora, lo importante no es memorizar todos los parámetros, sino entender el procedimiento: K-Fold Cross Validation divide los datos en K partes, entrena K veces, evalúa K veces y obtiene una estimación más estable del desempeño.

K-Fold Cross Validation es una estrategia de evaluación que divide el dataset en K partes. En cada repetición, una parte se usa para validar y las restantes para entrenar. Al final, obtenemos varias métricas que permiten calcular un promedio y observar la variabilidad del desempeño. Esta técnica reduce la dependencia de una única separación train/test y ayuda a comparar modelos con más criterio, siempre que la forma de dividir los datos respete la estructura del problema.

Stratified K-Fold

K-Fold Cross Validation divide el dataset en varias partes y usa cada una de ellas como validación una vez. Esta estrategia mejora la evaluación porque reduce la dependencia de una única separación train/test. Sin embargo, en problemas de clasificación puede aparecer una dificultad importante: que las clases no queden representadas de manera equilibrada en cada fold.

Este problema es especialmente relevante cuando las clases están desbalanceadas.

Supongamos un dataset con 1000 casos, donde la clase positiva aparece pocas veces. Podríamos tener, por ejemplo, 950 casos de la clase negativa y solo 50 casos de la clase positiva. En ese caso, la clase positiva representa apenas el 5% del total.

Si dividimos los datos en 5 folds sin cuidar la proporción de clases, podría ocurrir que algunos folds tengan muy pocos positivos, o incluso ninguno, especialmente si el dataset es pequeño. Un fold podría quedar con muchos negativos y pocos positivos; otro, con muchos negativos y casi ningún positivo; otro, con algunos positivos más. Esa distribución irregular puede afectar la evaluación.

Si un fold de validación tiene muy pocos casos positivos, métricas como recall, precision o F1-score pueden volverse inestables. El modelo podría parecer mejor o peor según cómo quedaron repartidos esos pocos casos.

Para evitar este problema, usamos **Stratified K-Fold**.

La idea de Stratified K-Fold es dividir los datos en folds manteniendo, en lo posible, la proporción de clases en cada uno. Es decir, si en el dataset completo la clase positiva representa el 5%, cada fold intentará conservar una proporción parecida.

Esto es similar a lo que ocurría con `stratify=y` en `train_test_split`. En una única separación train/test, `stratify=y` ayuda a que entrenamiento y prueba tengan proporciones de clases parecidas al dataset original. En validación cruzada, Stratified K-Fold aplica la misma idea, pero sobre varias divisiones. Dicho de otra manera: `train_test_split` con `stratify` mantiene proporciones en una separación; Stratified K-Fold intenta mantener proporciones en varias divisiones.

Veamos un ejemplo sencillo. Supongamos un problema binario con 100 casos: 80 pertenecen a la clase 0 y 20 pertenecen a la clase 1. Si usamos 5 folds estratificados, cada fold tendrá aproximadamente 20 casos. Como la proporción general es 80% clase 0 y 20% clase 1, cada fold intentará contener algo parecido a 16 casos de clase 0 y 4 casos de clase 1.

De esta forma, cada evaluación se realiza sobre una muestra que conserva mejor la estructura del problema. No garantiza que todos los folds sean idénticos, pero reduce el riesgo de particiones poco representativas.

Esto es muy importante en problemas con clases minoritarias. Si estamos detectando fraude, abandono, fallas, enfermedades, intrusiones o cualquier evento poco frecuente, no queremos que un fold de validación quede casi sin ejemplos positivos. Necesitamos evaluar el modelo en condiciones que reflejen razonablemente la distribución real de clases. En clasificación desbalanceada, la estratificación ayuda a que cada fold incluya ejemplos de todas las clases.

También es útil en clasificación multiclase. Supongamos un modelo que clasifica reclamos en cuatro categorías. Podríamos tener 600 casos de la categoría A, 250 de la categoría B, 120 de la categoría C y solo 30 de la categoría D.

La categoría D es minoritaria. Si no estratificamos, algunos folds podrían contener muy pocos casos D. Entonces la evaluación del desempeño sobre esa clase sería poco confiable. Con estratificación, cada fold intentará incluir una proporción similar de casos D.

Esto no resuelve todos los problemas del desbalance, pero mejora la calidad de la evaluación. Un modelo puede seguir funcionando mal sobre la clase minoritaria, pero al menos tendremos una medición más representativa de ese comportamiento.

Esta distinción es importante. Stratified K-Fold no balancea las clases para entrenar, no inventa nuevos datos y no mejora automáticamente las predicciones. Su objetivo es construir folds más representativos para que la evaluación sea más estable y justa. La estratificación no corrige el modelo: corrige la forma de dividir los datos para evaluar con más cuidado.

Si el modelo ignora la clase minoritaria, las métricas seguirán mostrando ese problema. Pero será menos probable que el resultado dependa de una mala distribución accidental de los casos entre folds.

En `scikit-learn`, muchas funciones de validación cruzada usan estratificación automáticamente en ciertos casos de clasificación cuando se pasa un número entero a `cv`, dependiendo del estimador y de la función. Sin embargo, también podemos crear explícitamente un objeto `StratifiedKFold` para controlar el procedimiento.

```
from sklearn.model_selection import StratifiedKFold

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
```

Aquí `n_splits=5` indica que queremos cinco folds. `shuffle=True` mezcla los datos antes de dividirlos, y `random_state=42` permite reproducir la misma división.

Luego podemos usar ese objeto en una función como `cross_val_score`:

```
from sklearn.model_selection import cross_val_score

scores = cross_val_score(modelo, X, y, cv=cv, scoring="f1")
scores
```

En este ejemplo, el modelo se evalúa con F1-score usando folds estratificados.

La elección de la métrica sigue siendo importante. En problemas desbalanceados, accuracy puede ser poco informativa. Si la clase positiva es rara, un modelo que predice siempre la clase mayoritaria puede obtener accuracy alta y aun así ser inútil para detectar positivos. Por eso, al usar Stratified K-Fold en clasificación desbalanceada, muchas veces conviene evaluar también precision, recall o F1-score. Estratificar ayuda a dividir mejor; elegir una métrica adecuada ayuda a interpretar mejor.

También debemos tener cuidado cuando la clase minoritaria tiene muy pocos casos. Si tenemos 5 folds, necesitamos que cada clase tenga suficientes ejemplos para poder distribuirse entre ellos. Si una clase aparece solo 3 veces, no puede estar presente en los 5 folds de validación.

En esa situación, puede ser necesario reducir la cantidad de folds, conseguir más datos, agrupar clases o revisar si el problema está bien formulado. No siempre se puede resolver con un parámetro.

Por ejemplo, si una clase tiene solo 4 casos y usamos `n_splits=10`, la división no podrá representar esa clase en cada fold. La evaluación por clase será muy inestable.

En datasets pequeños o extremadamente desbalanceados, la validación cruzada debe diseñarse con cuidado.

Stratified K-Fold se usa principalmente para clasificación. En regresión, la variable objetivo es continua, por lo que no hay clases que conservar de la misma manera. Existen estrategias para estratificar regresión mediante rangos de la variable objetivo, pero no forman parte de la validación cruzada básica que necesitamos ahora. En regresión, lo importante es revisar que los folds representen razonablemente la diversidad de valores.

También debemos recordar las limitaciones generales de cualquier división. Si los datos son temporales, no deberíamos usar Stratified K-Fold común sin pensar, porque mezclar pasado y futuro puede producir una evaluación engañosa. Si hay grupos, como múltiples registros por cliente o paciente, quizá necesitemos una estrategia que mantenga los grupos separados. La estratificación por clases no corrige esos problemas por sí sola.

Por eso, antes de elegir la estrategia de validación, debemos entender la estructura del dataset. Stratified K-Fold cuida las proporciones de clases, pero no reemplaza el cuidado temporal ni el cuidado por grupos. Si las filas son independientes y el problema es de clasificación, Stratified K-Fold suele ser una buena opción. Si hay tiempo, grupos o dependencias fuertes, necesitamos adaptar la evaluación.

Stratified K-Fold es una variante de K-Fold Cross Validation que mantiene aproximadamente la proporción de clases en cada fold. Es especialmente útil en problemas de clasificación con clases desbalanceadas, porque evita que algunas divisiones queden poco representativas. No mejora el modelo por sí mismo ni reemplaza la elección de métricas adecuadas, pero ayuda a obtener una evaluación más estable y justa del desempeño.

Promedio y variabilidad de métricas

Una de las principales ventajas de la validación cruzada es que no nos da una sola métrica, sino varias. Si usamos 5 folds, obtenemos 5 resultados. Si usamos 10 folds, obtenemos 10 resultados. Cada resultado corresponde a una división diferente de los datos.

Esto cambia la forma de interpretar la evaluación. Ya no miramos solamente un número final, sino una serie de valores que muestran cómo se comportó el modelo en distintas particiones.

Por ejemplo, si evaluamos un modelo de clasificación con 5 folds, podríamos obtener valores de accuracy como 0.82, 0.84, 0.81, 0.83 y 0.82. En este caso, las métricas son bastante parecidas entre sí. El modelo parece tener un desempeño estable. Si calculamos el promedio, obtenemos una estimación general cercana a 0.824, que podríamos comunicar como una accuracy promedio de aproximadamente 0.82.

Pero también es importante observar que los valores individuales no varían demasiado. Esa estabilidad nos da más confianza. Un buen promedio acompañado por poca variabilidad sugiere un desempeño más estable.

Ahora imaginemos otro caso, con valores de accuracy como 0.95, 0.61, 0.88, 0.57 y 0.91. El promedio puede parecer aceptable, pero la variabilidad es muy grande. El modelo funciona muy bien en algunos folds y muy mal en otros.

Esta situación es una señal de alerta. Puede indicar que el dataset es pequeño, que los folds quedaron muy distintos, que hay casos atípicos, que el modelo es inestable o que la forma de dividir los datos no está respetando la estructura del problema.

Por eso, en validación cruzada no debemos mirar solo el promedio. También debemos mirar la dispersión de los resultados. El promedio responde cómo funciona el modelo en términos generales. La variabilidad responde qué tan estable es ese desempeño.

Una forma común de resumir la variabilidad es calcular la desviación estándar de las métricas obtenidas en los folds. No necesitamos profundizar ahora en la estadística detrás de la desviación estándar. Para nuestro uso, alcanza con interpretarla como una medida de cuánto cambian los resultados entre una división y otra.

Si los valores de los folds son muy parecidos, la desviación estándar será baja. Si son muy diferentes, será alta. Una desviación estándar baja indica que los resultados entre folds son parecidos. Una desviación estándar alta indica que los resultados cambian mucho entre folds.

Por ejemplo, imaginemos un Modelo A con accuracy promedio de 0.83 y desviación estándar de 0.01. Este modelo parece estable: en las distintas divisiones, su desempeño se mantiene alrededor de 0.83.

En cambio, imaginemos un Modelo B con accuracy promedio de 0.83 y desviación estándar de 0.12. Aunque el promedio sea el mismo, el Modelo B es mucho más inestable. En algunos folds puede funcionar muy bien y en otros bastante mal. Dependiendo del problema, esa inestabilidad puede ser preocupante.

Este punto es fundamental al comparar modelos. Dos modelos pueden tener el mismo promedio, pero distinta variabilidad. El modelo más estable puede ser preferible,

especialmente si buscamos un comportamiento confiable. Mismo promedio no significa mismo comportamiento.

Supongamos dos modelos de regresión evaluados con MAE. El Modelo A obtiene valores por fold como 10.1, 10.4, 9.8, 10.2 y 10.0. El Modelo B obtiene valores como 5.0, 18.0, 7.5, 16.2 y 3.8.

Ambos podrían tener promedios no tan lejanos, pero el comportamiento es muy distinto. El Modelo A se equivoca de manera bastante consistente. El Modelo B a veces funciona muy bien y a veces muy mal. Si necesitamos estabilidad, el Modelo A puede ser más confiable, aunque el promedio no sea siempre el más bajo.

En regresión, debemos recordar que métricas como MAE y RMSE son errores. En esas métricas, valores más bajos son mejores. En clasificación, en cambio, métricas como accuracy o F1-score suelen ser mejores cuando son más altas.

En general, accuracy, precision, recall, F1-score y R^2 se interpretan mejor cuando son más altos. MAE, MSE y RMSE, en cambio, se interpretan mejor cuando son más bajos. Esta diferencia debe tenerse presente al interpretar promedios.

Cuando usamos validación cruzada, el promedio de las métricas nos ayuda a estimar el desempeño esperado. Pero no debemos interpretarlo como una garantía exacta. Sigue siendo una estimación basada en los datos disponibles. Si los datos futuros cambian, el desempeño también puede cambiar.

Por ejemplo, un modelo puede tener un MAE promedio de 12 unidades en validación cruzada. Eso no significa que siempre se equivocará exactamente por 12 unidades. Significa que, en las particiones evaluadas, su error promedio estuvo alrededor de ese valor. La validación cruzada estima el desempeño, pero no promete un resultado exacto para todos los datos futuros.

También es útil observar si algún fold tiene un resultado especialmente malo. Un valor aislado muy distinto puede indicar que ese fold contiene casos difíciles, valores extremos o una distribución particular. En lugar de ignorarlo, conviene preguntarse qué tiene de especial esa división.

Por ejemplo, si los valores de MAE por fold son 8.5, 9.1, 8.8, 27.4 y 9.0, el cuarto fold muestra un error mucho mayor. Esto puede ser ruido, pero también puede revelar un segmento del problema que el modelo no maneja bien. Tal vez allí quedaron muchos casos de alto valor, una zona específica, un período particular o registros con características diferentes.

La variabilidad entre folds puede transformarse en una herramienta de diagnóstico. Un fold con mal desempeño puede señalar casos difíciles que merecen revisión.

En `scikit-learn`, después de usar `cross_val_score`, podemos calcular promedio y desviación estándar de los resultados:

```
from sklearn.model_selection import cross_val_score

scores = cross_val_score(modelo, X, y, cv=5)

scores.mean()
scores.std()
```

Si estamos usando una métrica como accuracy, una salida posible podría ser una lista de valores como `0.82`, `0.84`, `0.81`, `0.83` y `0.82`. En ese caso, el promedio sería `0.824` y la desviación estándar sería aproximadamente `0.010`. Esto indica un desempeño promedio cercano a `0.82`, con baja variabilidad entre folds.

Para regresión, debemos tener cuidado con algunas métricas en `scikit-learn`. Cuando usamos `cross_val_score` con errores como MAE o RMSE, la biblioteca suele usar versiones negativas de esas métricas, porque internamente espera que “más alto sea mejor”. Por ejemplo, para MAE puede usarse `neg_mean_absolute_error`.

```
scores = cross_val_score(
    modelo,
    X,
    y,
    cv=5,
    scoring="neg_mean_absolute_error"
)
```

El resultado serán valores negativos. Para interpretarlos como MAE, se cambia el signo:

```
mae_scores = -scores
mae_scores.mean()
mae_scores.std()
```

Este detalle es técnico, pero importante para evitar confusiones. Un valor de `-12` en `neg_mean_absolute_error` representa un MAE de `12`, no un error negativo en sentido real.

En los cuadernos prácticos, esta convención aparecerá con ejemplos concretos. En el texto teórico, lo importante es comprender por qué tenemos varios resultados y cómo leerlos.

Al comunicar validación cruzada, conviene presentar el promedio junto con la variabilidad. Por ejemplo, podemos informar una accuracy promedio de `0.82` con una desviación estándar de `0.01`. O, en un problema de regresión, un MAE promedio de `12.4` unidades con una desviación estándar de `1.8` unidades.

Esto es más informativo que dar solo el promedio. La desviación estándar ayuda a entender si el desempeño es consistente o si depende mucho de la partición de los datos.

También podemos usar una formulación más narrativa. Por ejemplo: “El modelo obtuvo un MAE promedio de 12.4 unidades en validación cruzada de 5 folds. La variabilidad entre folds fue de 1.8 unidades, lo que sugiere un desempeño relativamente estable”.

Si la variabilidad es alta, podríamos decir: “El modelo obtuvo un MAE promedio de 12.4 unidades, pero con alta variabilidad entre folds. Esto indica que su desempeño depende bastante de qué casos quedan en entrenamiento y validación”.

Esta forma de comunicar evita conclusiones exageradas. No basta con decir que un modelo “dio bien”. Debemos indicar cuán estable fue ese resultado.

La variabilidad también ayuda a interpretar diferencias entre modelos. Si un modelo tiene un MAE promedio de 10.2 y otro de 10.0, pero la desviación estándar es de 2.5, la diferencia promedio es muy pequeña comparada con la variabilidad. En ese caso, no conviene afirmar con fuerza que el segundo modelo es mejor. Podrían estar prácticamente empatados.

En cambio, si un modelo tiene MAE promedio de 10 y otro de 16, con baja variabilidad, la diferencia parece más clara.

Por eso, la validación cruzada permite comparar con más criterio. No solo miramos quién tiene el mejor promedio, sino si esa mejora parece consistente.

La validación cruzada produce varias métricas, una por cada fold. El promedio resume el desempeño general del modelo, mientras que la variabilidad muestra qué tan estable es ese desempeño entre distintas divisiones de los datos. Un buen promedio con baja variabilidad da más confianza que un buen promedio con resultados muy inestables. Al comparar modelos, debemos mirar ambas cosas para evitar conclusiones apresuradas basadas en una única métrica o en diferencias demasiado pequeñas.

Cómo comparar modelos con más criterio

Comparar modelos no consiste simplemente en elegir el que obtiene el mejor número en una única métrica. Esa puede ser una primera señal, pero no alcanza para tomar una decisión confiable. Un modelo puede ganar por muy poca diferencia, ser más

inestable, ser más difícil de interpretar o funcionar peor justo en los casos que más importan.

La validación cruzada nos ayuda a comparar modelos con más criterio porque nos da más información que una única separación train/test. En lugar de mirar un solo resultado, podemos observar el desempeño promedio y la variabilidad entre folds.

Comparar con más criterio significa mirar varias cosas al mismo tiempo: el promedio de las métricas, la variabilidad entre folds, la importancia práctica de las diferencias, la estabilidad del modelo, su simplicidad y el tipo de errores que resultan relevantes para el problema.

Supongamos que evaluamos dos modelos de clasificación usando F1-score en validación cruzada de 5 folds:

Modelo	F1-score promedio	Desviación estándar
Modelo A	0.81	0.02
Modelo B	0.83	0.09

Si solo miramos el promedio, el Modelo B parece mejor. Pero su variabilidad es mucho mayor. Esto significa que en algunos folds puede funcionar muy bien y en otros bastante peor. El Modelo A, aunque tiene un promedio un poco menor, parece más estable.

La elección no es automática. Si la diferencia de promedio fuera grande, quizá aceptaríamos mayor variabilidad. Pero si la diferencia es pequeña, la estabilidad del Modelo A puede hacerlo preferible. El modelo con mejor promedio no siempre es el modelo más conveniente; un modelo más estable puede ser preferible cuando la diferencia promedio es pequeña.

Lo mismo ocurre en regresión. Imaginemos dos modelos evaluados con MAE:

Modelo	MAE promedio	Desviación estándar
Modelo A	10.2	0.8
Modelo B	9.9	3.5

El Modelo B tiene un MAE promedio apenas menor. Pero su variabilidad es bastante mayor. Si el problema requiere resultados consistentes, la diferencia de 0.3 unidades en el promedio tal vez no justifique elegir un modelo más inestable.

Este tipo de lectura es mucho más prudente que quedarse con el menor MAE o el mayor F1-score sin mirar nada más.

También debemos pensar si la diferencia entre modelos es importante en términos prácticos. Una mejora muy pequeña puede no justificar mayor complejidad. Si un modelo reduce el MAE de 10.1 a 10.0, quizás la mejora sea irrelevante para la decisión real. En cambio, reducirlo de 10.1 a 6.5 probablemente sea más significativo.

La magnitud de la mejora debe interpretarse en el contexto del problema. En predicción de ventas, mejorar 0.5 unidades puede ser irrelevante si las ventas típicas son miles de unidades. Pero puede ser importante si se predicen cantidades muy pequeñas. En predicción de temperatura, reducir el error promedio de 2.5 °C a 1.0 °C puede ser una mejora clara. En precios de viviendas, bajar el error promedio de 10000 a 9500 dólares puede o no justificar un modelo más complejo, según el uso.

Por eso, comparar modelos exige traducir las métricas al problema real. Una diferencia entre métricas debe ayudarnos a responder preguntas concretas: si mejora una decisión, si reduce un costo relevante, si evita errores importantes o si justifica una mayor complejidad.

También es importante comparar todos los modelos bajo las mismas condiciones. Deben usar las mismas particiones, la misma métrica principal y un flujo de preprocesamiento equivalente. Si un modelo se evalúa con una estrategia y otro con otra, la comparación puede ser injusta.

Por ejemplo, no sería correcto comparar un modelo evaluado con una única separación train/test contra otro evaluado con validación cruzada y concluir directamente que uno es mejor. Tampoco sería justo comparar un modelo entrenado con variables cuidadosamente preparadas contra otro al que no se le aplicó el mismo preprocesamiento, salvo que estemos comparando pipelines completos y lo declaremos así.

Una comparación justa necesita mismas divisiones, mismas métricas, mismo criterio de preprocesamiento y misma variable objetivo.

En la práctica, conviene comparar pipelines completos. Un pipeline incluye las transformaciones necesarias y el modelo final. Esto evita aplicar un preprocesamiento a un modelo y otro distinto a otro modelo sin control. También ayuda a evitar fuga de datos durante la validación cruzada.

Por ejemplo, si usamos imputación, escalado y un modelo, todo eso debería estar dentro del pipeline evaluado. Así, en cada fold, las transformaciones se ajustan solo con los datos de entrenamiento de ese fold y luego se aplican a la validación correspondiente. Validar pipelines completos ayuda a comparar modelos sin mezclar información indebida entre folds.

Otro criterio importante es comparar contra una referencia simple. Antes de decidir entre modelos sofisticados, deberíamos saber cuánto mejora cada uno respecto de una estrategia básica. En clasificación, la referencia puede ser predecir siempre la clase mayoritaria. En regresión, puede ser predecir siempre el promedio o la mediana.

Si todos los modelos apenas mejoran la referencia, tal vez el problema no esté en la elección del algoritmo, sino en los datos, las variables o la formulación.

Supongamos un problema de regresión:

Modelo	MAE promedio
Modelo base	18.0
Regresión lineal	17.2
Árbol de decisión	16.9

Aunque el árbol sea el mejor de los tres, la mejora respecto del modelo base es pequeña. Quizás las variables disponibles no contienen suficiente señal. Tal vez falta información importante. Tal vez la variable objetivo es muy ruidosa. La comparación entre modelos no debería ocultar esa conclusión.

En cambio:

Modelo	MAE promedio
Modelo base	18.0
Regresión lineal	11.0
Árbol de decisión	9.5

Aquí ambos modelos entrenados mejoran claramente la referencia. Luego podemos discutir si la mejora adicional del árbol justifica su complejidad frente a la regresión lineal.

La simplicidad del modelo también debe formar parte de la comparación. Si dos modelos obtienen desempeños parecidos, suele ser razonable preferir el más simple, más interpretable o más fácil de mantener. Esto no es una regla absoluta, pero sí un criterio práctico importante.

Un modelo más simple puede ser más fácil de explicar, depurar, implementar y monitorear. También puede ser menos sensible a cambios pequeños en los datos. En contextos educativos, institucionales o de toma de decisiones sensibles, la interpretabilidad puede ser tan importante como una mejora pequeña en la métrica.

Sin embargo, si un modelo más complejo mejora claramente el desempeño en datos no vistos, puede valer la pena usarlo. La clave es que esa complejidad esté justificada por una mejora real y relevante.

También debemos revisar el tipo de error que comete cada modelo. Dos modelos pueden tener métricas similares y fallar de maneras distintas. Uno puede tener buen desempeño promedio pero cometer errores grandes en pocos casos. Otro puede tener errores más moderados y distribuidos. Uno puede funcionar bien para la clase mayoritaria y mal para la minoritaria. Otro puede equilibrar mejor ambas.

Las métricas promedio nos dicen cuánto rinde el modelo. El análisis de errores nos ayuda a entender cómo falla.

Por ejemplo, en un problema de fraude, un modelo con F1-score levemente menor pero mayor recall sobre la clase positiva puede ser preferible si el objetivo principal es detectar fraudes. En un problema de spam, un modelo con mayor precisión puede ser preferible si queremos evitar que correos legítimos sean marcados como spam. En regresión, un modelo con MAE similar pero menor RMSE puede ser preferible si queremos evitar errores extremos.

Esto muestra que la métrica principal debe elegirse según el problema. La validación cruzada mejora la forma de medir, pero no decide por sí sola qué métrica importa más. La validación cruzada mejora la estimación del desempeño; el criterio del problema define qué métrica y qué errores importan.

También conviene observar si los modelos son estables frente a distintas divisiones. Un modelo puede tener el mejor promedio, pero con alta variabilidad. Otro puede tener un promedio apenas menor, pero comportarse de forma más consistente. En problemas donde la confiabilidad importa, esa estabilidad puede pesar mucho.

La variabilidad también puede indicar que necesitamos más datos, mejores variables o una estrategia de validación diferente. Si todos los modelos muestran alta variabilidad, tal vez el dataset es pequeño o heterogéneo. Si solo un modelo es inestable, tal vez ese modelo es demasiado sensible o complejo para los datos disponibles.

Al comparar modelos, también debemos evitar usar el conjunto de prueba final como espacio de experimentación permanente. Si probamos muchas variantes y elegimos la que mejor funciona en el test, el test deja de ser una evaluación honesta. Para comparar y seleccionar modelos, conviene usar validación cruzada dentro del conjunto de entrenamiento. Luego, una vez elegida la opción, se evalúa una última vez en el conjunto de prueba final.

Esta separación de roles ayuda a evitar evaluaciones demasiado optimistas. La validación cruzada sirve para comparar y seleccionar. El conjunto de prueba final sirve para una evaluación honesta al final del proceso y debe permanecer lo más independiente posible hasta ese momento.

En `scikit-learn`, comparar modelos con validación cruzada puede hacerse calculando los scores de cada uno con la misma estrategia de `cv` y la misma métrica. Conceptualmente, el flujo sería:

```
scores_modelo_a = cross_val_score(modelo_a, X, y, cv=cv, scoring=metrica)
scores_modelo_b = cross_val_score(modelo_b, X, y, cv=cv, scoring=metrica)
```

Luego comparamos promedio y desviación estándar:

```
scores_modelo_a.mean(), scores_modelo_a.std()
scores_modelo_b.mean(), scores_modelo_b.std()
```

En un cuaderno práctico, estos resultados pueden organizarse en una tabla para comparar varios modelos:

Modelo	Promedio	Desviación estándar
Regresión lineal	0.78	0.03
Árbol de decisión	0.75	0.09
KNN	0.80	0.04

La tabla ayuda a ordenar la información, pero la conclusión debe escribirse con criterio. No basta con señalar el número más alto. Hay que interpretar diferencias, estabilidad y adecuación al problema.

Una conclusión responsable podría decir que KNN obtuvo el mejor promedio, pero que la regresión lineal fue más simple y tuvo un desempeño cercano. También podría señalar que el árbol de decisión mostró mayor variabilidad, por lo que conviene revisar si está sobreajustando o si necesita restricciones de complejidad.

Este tipo de conclusión es más útil que afirmar simplemente que “KNN ganó”.

Comparar modelos con más criterio implica evaluar alternativas bajo condiciones justas, usando validación cruzada, observando promedio y variabilidad, comparando contra una referencia simple y considerando la importancia práctica de las diferencias. La mejor elección no siempre es el modelo con el número más alto o más bajo en una tabla, sino aquel que ofrece un buen equilibrio entre desempeño, estabilidad, simplicidad, interpretabilidad y adecuación al problema real.

Validación cruzada en `scikit-learn`

Después de comprender la idea de validación cruzada, podemos ver cómo se aplica en `scikit-learn`. La biblioteca ofrece herramientas para dividir los datos en folds, entrenar el modelo varias veces, evaluar cada repetición y devolver los resultados obtenidos.

La función más directa para comenzar es `cross_val_score`, ubicada en `sklearn.model_selection`.

```
from sklearn.model_selection import cross_val_score
```

Su uso básico consiste en pasarle un modelo, las variables de entrada `X`, la variable objetivo `y` y la cantidad de folds que queremos utilizar.

```
scores = cross_val_score(modelo, X, y, cv=5)
```

En este ejemplo, `cv=5` indica una validación cruzada de 5 folds. El modelo se entrenará cinco veces. En cada repetición, una parte distinta de los datos se usará para validar y las restantes para entrenar.

El resultado, `scores`, contiene una métrica por cada fold. Por ejemplo, en un problema de clasificación, podríamos obtener:

```
[0.82, 0.84, 0.81, 0.83, 0.82]
```

Estos valores podrían representar accuracy en cada fold, si esa es la métrica usada. Luego podemos calcular el promedio:

```
scores.mean()
```

Y también la variabilidad:

```
scores.std()
```

El promedio resume el desempeño general estimado del modelo, mientras que la desviación estándar muestra cuánto variaron los resultados entre folds.

Es importante recordar que, por defecto, `cross_val_score` usa la métrica asociada al método `score` del modelo. En muchos clasificadores, `score` devuelve accuracy. En muchos regresores, `score` devuelve R^2 . Esto puede ser cómodo, pero también puede generar confusiones si no recordamos qué métrica estamos usando.

Por eso, muchas veces conviene indicar explícitamente la métrica con el parámetro `scoring`.

Por ejemplo, en clasificación podríamos evaluar con F1-score:

```
scores = cross_val_score(
    modelo,
    X,
    y,
    cv=5,
    scoring="f1"
)
```

O con accuracy:

```
scores = cross_val_score(
    modelo,
    X,
    y,
    cv=5,
    scoring="accuracy"
)
```

En problemas multiclase o cuando necesitamos una forma particular de promediar las clases, el valor de `scoring` puede cambiar. Por ejemplo, existen variantes como `f1_macro` o `f1_weighted`. No necesitamos profundizar ahora en todas ellas, pero sí conviene saber que la métrica debe elegirse según el problema.

En regresión, podemos evaluar con R^2 :

```
scores = cross_val_score(
    modelo,
    X,
    y,
    cv=5,
    scoring="r2"
)
```


Para métricas de error, como MAE, aparece una convención particular de `scikit-learn`. Como `cross_val_score` está pensada para que valores más altos sean mejores, las métricas de error se devuelven con signo negativo. Por eso, para MAE se usa `"neg_mean_absolute_error"`.

```
scores = cross_val_score(
    modelo,
    X,
    y,
    cv=5,
    scoring="neg_mean_absolute_error"
)
```

El resultado serán valores negativos. Si obtenemos:

```
[-10.2, -11.4, -9.8, -12.1, -10.7]
```

eso significa que los MAE reales son:

```
[10.2, 11.4, 9.8, 12.1, 10.7]
```

Para interpretarlos como errores positivos, cambiamos el signo:

```
mae_scores = -scores
```

Luego calculamos promedio y desviación estándar:

```
mae_scores.mean()
mae_scores.std()
```

Este detalle técnico es importante. Un error negativo no significa que el modelo se haya equivocado “menos que cero”. Es solo una convención de la biblioteca para mantener la idea de que, internamente, valores mayores son mejores.

Algo parecido ocurre si usamos MSE:

```
scores = cross_val_score(
    modelo,
    X,
    y,
    cv=5,
    scoring="neg_mean_squared_error"
)
```

Para obtener el MSE positivo:

```
mse_scores = -scores
```

Y para obtener RMSE por fold:

```
import numpy as np

rmse_scores = np.sqrt(mse_scores)
```

En los cuadernos prácticos convendrá mostrar estos pasos con calma, porque la convención de métricas negativas suele confundir al principio. En el texto teórico, lo esencial es recordar que `scoring` define qué métrica se calcula y que, para errores de regresión, `scikit-learn` suele devolver valores negativos por convención.

También podemos controlar explícitamente la estrategia de partición. Si pasamos `cv=5`, `scikit-learn` usa una división de 5 folds apropiada para el caso. Pero si queremos más control, podemos crear un objeto de validación cruzada.

Para clasificación con clases desbalanceadas, podemos usar `StratifiedKFold`:

```
from sklearn.model_selection import StratifiedKFold

cv = StratifiedKFold(
    n_splits=5,
    shuffle=True,
    random_state=42
)
```

Luego usamos ese objeto en `cross_val_score`:

```
scores = cross_val_score(
    modelo,
    X,
    y,
    cv=cv,
    scoring="f1"
)
```

Esta forma permite mantener proporciones de clases en cada fold, mezclar los datos antes de dividirlos y reproducir la misma partición con `random_state`.

Para regresión, podemos usar `KFold`:

```
from sklearn.model_selection import KFold

cv = KFold(
    n_splits=5,
    shuffle=True,
    random_state=42
)
```

Y luego:

```
scores = cross_val_score(
    modelo,
    X,
    y,
    cv=cv,
    scoring="r2"
)
```

Usar `shuffle=True` puede ser conveniente cuando los datos no tienen un orden que deba respetarse. Si las filas están ordenadas por fecha, por grupo o por alguna estructura importante, mezclar sin pensar puede producir una evaluación engañosa. La validación debe respetar el problema: mezclar puede ser razonable en datos independientes, pero puede ser incorrecto cuando hay tiempo, grupos o dependencias fuertes.

Otro punto clave es usar validación cruzada junto con `Pipeline`. Si el modelo necesita preprocesamiento, como imputación, escalado u One-Hot Encoding, esas transformaciones deben formar parte del pipeline evaluado.

Por ejemplo:

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression

pipeline = Pipeline(steps=[
    ("scaler", StandardScaler()),
    ("modelo", LinearRegression())
])
```

Luego evaluamos el pipeline completo:

```
scores = cross_val_score(
    pipeline,
    X,
    y,
    cv=5,
    scoring="r2"
)
```

Esto es importante porque, en cada fold, el escalador se ajustará solo con los datos de entrenamiento de ese fold y luego se aplicará a la parte de validación. Así evitamos que la transformación aprenda información del fold de validación.

Si escaláramos todo `X` antes de aplicar validación cruzada, estaríamos usando información estadística de todos los datos, incluidos los que luego deberían funcionar como validación. Esto puede producir una evaluación demasiado optimista. Por eso, el pipeline no es solo una comodidad: también ayuda a ordenar correctamente el flujo metodológico y a evitar fuga de datos durante el preprocesamiento.

Lo mismo vale para imputadores, codificadores y transformaciones por tipo de columna con `ColumnTransformer`. Si esas transformaciones aprenden algo de los datos, deben ajustarse dentro de cada fold, no antes de la validación.

Una salida típica de validación cruzada debería comunicarse con promedio y variabilidad. Por ejemplo:

```
scores = cross_val_score(
    pipeline,
    X,
    y,
    cv=5,
    scoring="r2"
)

print(scores)
print(scores.mean())
print(scores.std())
```

La interpretación podría ser: el modelo obtuvo un R^2 promedio de `0.74`, con una desviación estándar de `0.04`, en validación cruzada de 5 folds.

Para MAE:

```
scores = cross_val_score(
    pipeline,
    X,
    y,
    cv=5,
    scoring="neg_mean_absolute_error"
)

mae_scores = -scores

print(mae_scores.mean())
print(mae_scores.std())
```

La interpretación podría ser: el modelo obtuvo un MAE promedio de `12.6` unidades, con una desviación estándar de `1.9` unidades entre folds.

Esta forma de comunicar es más completa que dar un solo valor. Permite saber no solo cuánto rindió el modelo en promedio, sino también qué tan estable fue la evaluación.

También podemos usar validación cruzada para comparar varios modelos. La idea es evaluar cada uno con la misma estrategia de `cv` y la misma métrica.

```
scores_a = cross_val_score(modelo_a, X, y, cv=cv, scoring=metrica)
scores_b = cross_val_score(modelo_b, X, y, cv=cv, scoring=metrica)
```

Luego comparamos promedios y desviaciones estándar. Si estamos usando pipelines, cada modelo puede tener su propio pipeline, pero la comparación debe ser justa: misma variable objetivo, mismos datos, misma métrica, misma estrategia de validación y mismo criterio de preprocesamiento.

No debemos olvidar que la validación cruzada aumenta el costo computacional. Si usamos 5 folds, el modelo se entrena 5 veces. Si usamos 10 folds, se entrena 10 veces. Para modelos simples esto no suele ser un problema, pero para modelos pesados o datasets grandes puede ser importante.

Por eso, en la práctica, `cv=5` suele ser un buen punto de partida. No es una regla obligatoria, pero ofrece un equilibrio razonable entre estabilidad y costo para muchos problemas tabulares.

Finalmente, conviene recordar el rol del conjunto de prueba final. La validación cruzada se usa para estimar desempeño y comparar modelos durante el desarrollo. Si tenemos un conjunto de prueba separado, ese conjunto debería reservarse para una evaluación final, después de elegir el modelo o la configuración. No conviene usarlo repetidamente para tomar decisiones, porque dejaría de ser una prueba independiente.

La validación cruzada sirve para comparar y seleccionar modelos durante el desarrollo. El conjunto de prueba final sirve para evaluar una vez al final, con la menor intervención posible.

`scikit-learn` permite aplicar validación cruzada con herramientas como `cross_val_score`, `KFold` y `StratifiedKFold`. El parámetro `cv` define cómo se dividen los datos, y `scoring` define qué métrica se calcula. Para problemas con preprocesamiento, conviene evaluar pipelines completos para evitar fuga de datos. Los resultados deben interpretarse mediante promedio y variabilidad entre folds, recordando que la validación cruzada es una estrategia de evaluación, no un modelo en sí misma.

Evaluar con más estabilidad

En este capítulo estudiamos la validación cruzada como una forma de evaluar modelos con más estabilidad que una única separación entre entrenamiento y prueba. La idea central es simple: en lugar de depender de un solo reparto de los datos, usamos varias divisiones y observamos cómo se comporta el modelo en cada una.

Una única separación train/test es útil y necesaria para introducir la evaluación con datos no vistos. Pero también tiene limitaciones. El resultado puede depender de qué

casos quedaron en entrenamiento y cuáles quedaron en prueba. Si el dataset es pequeño, desbalanceado o tiene casos muy variados, esa dependencia puede ser importante.

Con una única separación obtenemos una sola estimación del desempeño. Con validación cruzada, en cambio, obtenemos varias estimaciones y, por lo tanto, una evaluación más estable.

K-Fold Cross Validation divide los datos en K partes o folds. En cada repetición, una parte se usa para validar y las restantes para entrenar. Al final, cada fold fue usado una vez como validación. Esto permite obtener varias métricas y calcular un promedio.

También vimos que, en clasificación, puede ser importante usar Stratified K-Fold. Esta variante conserva aproximadamente la proporción de clases en cada fold, lo que resulta especialmente útil cuando las clases están desbalanceadas. La estratificación no mejora el modelo por sí misma, pero ayuda a que la evaluación sea más representativa y estable.

Una ventaja clave de la validación cruzada es que no solo produce un promedio, sino también información sobre la variabilidad. El promedio resume el desempeño general estimado, mientras que la variabilidad nos ayuda a observar la estabilidad del desempeño entre distintas divisiones.

Dos modelos pueden tener el mismo desempeño promedio, pero uno ser mucho más estable que el otro. Por eso, conviene mirar tanto la media como la desviación estándar de las métricas obtenidas en los folds.

Esta forma de evaluar permite comparar modelos con más criterio. No alcanza con elegir el número más alto o más bajo en una tabla. Debemos observar si la diferencia es práctica, si el modelo es estable, si mejora una referencia simple, si sus errores son aceptables y si su complejidad está justificada.

En `scikit-learn`, la validación cruzada puede implementarse con herramientas como `cross_val_score`, `KFold` y `StratifiedKFold`. El parámetro `cv` define la estrategia de división y `scoring` define la métrica que se calculará. En regresión, debemos recordar la convención de métricas negativas para errores como MAE o MSE, por ejemplo `neg_mean_absolute_error`.

También es importante evaluar pipelines completos. Si el modelo necesita imputación, escalado, codificación de categorías u otras transformaciones, esas transformaciones deben ajustarse dentro de cada fold, usando solo los datos de entrenamiento correspondientes. Esto ayuda a evitar fuga de datos durante la validación.

Validar un pipeline completo significa evaluar el preprocesamiento y el modelo como una sola unidad. De esa manera, cada fold respeta mejor la lógica de datos no vistos: las transformaciones se aprenden con los datos de entrenamiento de ese fold y luego se aplican a los datos de validación.

La validación cruzada no reemplaza necesariamente al conjunto de prueba final. En un flujo de trabajo cuidadoso, puede usarse para comparar modelos y elegir configuraciones durante el desarrollo. Luego, si tenemos un conjunto de prueba separado, lo reservamos para una evaluación final más honesta.

La validación cruzada permite evaluar modelos de manera menos dependiente de una única partición de los datos. Nos da una estimación promedio del desempeño y una medida de su estabilidad. Usarla correctamente ayuda a comparar modelos con más prudencia, detectar variabilidad, evitar conclusiones apresuradas y preparar el terreno para el ajuste de hiperparámetros, que será el tema del próximo capítulo.

Capítulo 19 · Ajuste de hiperparámetros

¿Qué son los hiperparámetros?

Cuando entrenamos un modelo de Machine Learning, no todo lo que determina su comportamiento se aprende automáticamente a partir de los datos. Algunos valores deben definirse antes del entrenamiento. A esos valores los llamamos **hiperparámetros**.

Un hiperparámetro es una configuración del modelo o del proceso de entrenamiento que influye en cómo el modelo aprende, pero que no se obtiene directamente como resultado del ajuste con los datos.

Concepto	Qué significa
Parámetros	Valores que el modelo aprende durante el entrenamiento.
Hiperparámetros	Valores que definimos antes de entrenar.

Esta diferencia es importante. En una regresión lineal, por ejemplo, los coeficientes y el intercepto son parámetros. El modelo los aprende a partir de los datos. Nosotros no elegimos manualmente cuánto debe valer cada coeficiente; el algoritmo los calcula durante el entrenamiento.

En cambio, en un modelo KNN, el valor de `k` no se aprende automáticamente de los datos. Debemos elegirlo antes de entrenar o evaluar el modelo. Un KNN con `k = 3` clasifica mirando los 3 vecinos más cercanos. Un KNN con `k = 15` clasifica mirando los 15 vecinos más cercanos.

Ambos modelos usan el mismo algoritmo general, pero no se comportan igual. El valor de `k` modifica la forma en que el modelo toma decisiones. Por eso `k` es un hiperparámetro.

En un árbol de decisión ocurre algo parecido con la profundidad máxima. Si permitimos que el árbol crezca sin límite, puede volverse muy complejo y adaptarse demasiado a los datos de entrenamiento. Si limitamos su profundidad, el árbol será más simple.

Un árbol con poca profundidad tiende a producir un modelo más simple. Un árbol con mucha profundidad es más flexible, pero también tiene mayor riesgo de sobreajuste. El valor de `max_depth`, entonces, es un hiperparámetro. No es una regla aprendida por el árbol, sino una restricción que nosotros fijamos antes del entrenamiento.

Los hiperparámetros existen en muchos modelos. Algunos controlan la complejidad, otros la forma de medir distancias, la cantidad de vecinos, la cantidad de iteraciones, la intensidad de una penalización, la profundidad de un árbol o la forma en que se construyen las divisiones.

No necesitamos memorizar todos los hiperparámetros de todos los modelos. Lo importante es entender que cada modelo tiene ciertas configuraciones que pueden afectar su desempeño y su capacidad de generalizar.

Ajustar hiperparámetros significa probar distintas configuraciones para encontrar una opción adecuada. La palabra “adecuada” es importante. No siempre buscamos la configuración que obtiene el mejor resultado en una sola prueba. Queremos una configuración que funcione bien en datos no vistos, que sea estable y que no se limite a aprovechar una división particular de los datos.

Por eso, el ajuste de hiperparámetros se relaciona directamente con la validación cruzada. En el capítulo anterior vimos que una única separación train/test puede ser insuficiente para comparar modelos. Ahora usaremos esa misma idea para comparar configuraciones de un mismo modelo.

Por ejemplo, en KNN podríamos probar valores como `k = 3`, `k = 5`, `k = 7`, `k = 11` y `k = 15`. Cada valor de `k` produce una versión distinta del modelo. Todas siguen siendo KNN, pero con comportamientos diferentes.

Un valor pequeño de `k` puede adaptarse demasiado a detalles locales. Un valor grande puede suavizar demasiado las decisiones y perder patrones importantes. No hay una respuesta universal. El mejor valor depende del dataset, de las variables, del preprocesamiento, de la métrica usada y del objetivo del problema.

Por eso no existe un hiperparámetro óptimo para todos los problemas. Cada configuración debe evaluarse en el contexto de los datos y de la tarea.

En árboles de decisión podríamos probar distintas profundidades, por ejemplo `max_depth = 2`, `max_depth = 4`, `max_depth = 6`, `max_depth = 8` o `max_depth = None`. Un árbol con profundidad 2 puede ser demasiado simple y subajustar. Un árbol sin límite puede ser demasiado complejo y sobreajustar. Entre esos extremos puede haber configuraciones que logren un equilibrio más razonable.

Este equilibrio es una de las ideas centrales del ajuste de hiperparámetros. Muchos hiperparámetros controlan, directa o indirectamente, la relación entre simplicidad y complejidad. Si son demasiado restrictivos, aumenta el riesgo de subajuste. Si son demasiado flexibles, aumenta el riesgo de sobreajuste. Una configuración adecuada busca un punto intermedio que ayude al modelo a generalizar mejor.

También hay hiperparámetros asociados al preprocesamiento. Por ejemplo, en una imputación de valores faltantes podemos elegir si reemplazamos con la media, la mediana o el valor más frecuente. En One-Hot Encoding podemos decidir cómo manejar categorías desconocidas. En algunos modelos, podemos decidir si escalamos o no las variables.

Cuando trabajamos con pipelines, el ajuste de hiperparámetros puede incluir tanto configuraciones del modelo como configuraciones del preprocesamiento. Esto permite evaluar el flujo completo de manera ordenada: preprocesamiento y modelo se prueban como una sola unidad.

Este punto es importante porque no queremos ajustar el preprocesamiento por fuera de la validación. Si probamos configuraciones usando información de todos los datos antes de validar, podemos introducir fuga de datos. Por eso, más adelante veremos que herramientas como `GridSearchCV` y `RandomizedSearchCV` pueden trabajar directamente con pipelines.

Un error frecuente al comenzar es ajustar hiperparámetros mirando el conjunto de prueba final. Por ejemplo, entrenar un modelo con `k = 3`, medir en test, luego probar `k = 5`, volver a medir en test, después `k = 7`, y quedarse con el mejor resultado.

A primera vista parece razonable, pero tiene un problema: el conjunto de prueba empieza a participar en la elección del modelo. Deja de ser una evaluación final independiente. El test final no debería usarse para elegir hiperparámetros. Debe reservarse para evaluar al final.

La forma más cuidadosa de trabajar es usar validación cruzada sobre los datos de entrenamiento para elegir hiperparámetros. Luego, una vez elegida la configuración, evaluamos el modelo final en el conjunto de prueba.

El flujo general sería: primero se separan los datos disponibles en entrenamiento y prueba. Luego, dentro del conjunto de entrenamiento, se usa validación cruzada para comparar configuraciones. Después se entrena el modelo final con la configuración elegida. Por último, se evalúa ese modelo en el conjunto de prueba final.

Este flujo protege mejor la evaluación. El conjunto de prueba queda reservado para una última medición, después de haber tomado las decisiones principales.

Ajustar hiperparámetros tampoco significa probar infinitas combinaciones hasta encontrar un resultado perfecto. Esa búsqueda puede llevar a sobreajustar la validación, especialmente si probamos demasiadas opciones sin criterio. Por eso, conviene definir rangos razonables, usar métricas adecuadas y mantener una mirada crítica sobre los resultados.

El ajuste de hiperparámetros no reemplaza el criterio. Ayuda a comparar configuraciones, pero las decisiones siguen dependiendo del problema.

También debemos considerar el costo computacional. Probar muchas configuraciones con validación cruzada puede requerir muchos entrenamientos. Si evaluamos 10 combinaciones con 5 folds, el modelo se entrena 50 veces. Si evaluamos 100 combinaciones con 10 folds, se entrena 1000 veces.

En general, la cantidad de entrenamientos es igual a la cantidad de configuraciones multiplicada por la cantidad de folds. Esto no siempre es un problema con modelos simples y datasets pequeños, pero puede volverse costoso con modelos pesados o grandes volúmenes de datos.

Por eso, ajustar hiperparámetros implica un equilibrio entre búsqueda, tiempo y utilidad práctica. A veces una búsqueda pequeña y bien pensada es suficiente. Otras veces necesitaremos explorar más opciones. Lo importante es que la búsqueda tenga sentido para el problema.

Los hiperparámetros son configuraciones que definimos antes de entrenar un modelo y que influyen en su comportamiento. A diferencia de los parámetros, no se aprenden automáticamente durante el entrenamiento. Ajustarlos significa comparar distintas configuraciones para buscar un mejor equilibrio entre desempeño, estabilidad y generalización. Para hacerlo correctamente, debemos usar validación cruzada, evitar usar el test final como herramienta de selección y evaluar siempre las configuraciones dentro del contexto del problema.

Búsqueda manual de hiperparámetros

Una forma sencilla de empezar a ajustar hiperparámetros es probar algunas configuraciones manualmente. Esto significa elegir ciertos valores posibles, entrenar el modelo con cada uno, evaluar los resultados y comparar.

Por ejemplo, si usamos KNN, podemos probar distintos valores de k , como 3, 5, 7, 11 y 15. Cada valor produce un modelo distinto. Todos son modelos KNN, pero cada uno decide usando una cantidad diferente de vecinos. Un k pequeño puede adaptarse mucho a los casos cercanos. Un k grande puede suavizar la decisión y hacer que el modelo sea menos sensible a detalles locales.

La búsqueda manual consiste en probar varias configuraciones, comparar sus métricas y elegir una opción razonable. La palabra “razonable” vuelve a ser importante. No buscamos solamente el valor que gana por una mínima diferencia en

una única evaluación. Buscamos una configuración que tenga buen desempeño, sea estable y tenga sentido para el problema.

Supongamos que evaluamos KNN con validación cruzada y obtenemos estos resultados:

Valor de k	F1-score promedio
3	0.78
5	0.81
7	0.82
11	0.82
15	0.80

En este caso, `k = 7` y `k = 11` parecen funcionar mejor. Si la diferencia entre ambos es muy pequeña, no conviene afirmar con demasiada seguridad que uno es superior al otro. Podríamos mirar también la variabilidad entre folds, el tipo de errores que cometen o elegir la opción más simple de justificar.

La búsqueda manual es útil porque nos obliga a pensar. No delegamos todo en una función automática. Elegimos valores posibles, observamos qué ocurre y relacionamos los resultados con lo que sabemos del modelo. De este modo, también podemos comprender cómo cambia su comportamiento cuando modificamos un hiperparámetro.

En un árbol de decisión, por ejemplo, podríamos probar valores de `max_depth` iguales a `2`, `3`, `4`, `5` y `8`. Luego evaluamos cada opción con la misma métrica y la misma estrategia de validación.

Un resultado posible sería:

max_depth	Accuracy promedio
2	0.74
3	0.79
4	0.82
5	0.81
8	0.76

Aquí puede verse una idea conocida: un árbol demasiado poco profundo puede subajustar, porque no tiene suficiente capacidad para representar patrones importantes. Pero un árbol demasiado profundo puede sobreajustar, porque empieza

a capturar detalles específicos del entrenamiento. Una profundidad intermedia puede ofrecer un mejor equilibrio entre ambos extremos.

La búsqueda manual permite visualizar este equilibrio. No se trata solo de obtener una tabla de resultados, sino de comprender cómo los hiperparámetros afectan el comportamiento del modelo.

En `scikit-learn`, una búsqueda manual puede hacerse con un ciclo. Por ejemplo, para probar distintos valores de `k` en KNN, podríamos escribir una estructura como esta:

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import cross_val_score

valores_k = [3, 5, 7, 11, 15]

for k in valores_k:
    modelo = KNeighborsClassifier(n_neighbors=k)
    scores = cross_val_score(modelo, X, y, cv=5, scoring="f1")
    print(k, scores.mean(), scores.std())
```

Este código entrena y evalúa un modelo por cada valor de `k`. Para cada configuración, calcula los resultados de validación cruzada y muestra el promedio y la desviación estándar.

La misma idea puede aplicarse a un árbol de decisión:

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import cross_val_score

profundidades = [2, 3, 4, 5, 8]

for profundidad in profundidades:
    modelo = DecisionTreeClassifier(
        max_depth=profundidad,
        random_state=42
    )

    scores = cross_val_score(modelo, X, y, cv=5, scoring="accuracy")
    print(profundidad, scores.mean(), scores.std())
```

Este tipo de código es simple y transparente. Nos muestra explícitamente qué valores probamos y qué resultado obtuvo cada uno.

Sin embargo, debemos tener cuidado: si el modelo necesita preprocesamiento, no deberíamos evaluar el modelo aislado, sino un pipeline completo. Esto es especialmente importante si usamos imputación, escalado u One-Hot Encoding.

Por ejemplo, KNN suele requerir escalado de variables numéricas. Entonces, en lugar de evaluar solo el modelo KNN, conviene evaluar un pipeline que incluya el escalado y el modelo.

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import cross_val_score

valores_k = [3, 5, 7, 11, 15]

for k in valores_k:
    pipeline = Pipeline(steps=[
        ("scaler", StandardScaler()),
        ("modelo", KNeighborsClassifier(n_neighbors=k))
    ])

    scores = cross_val_score(pipeline, X, y, cv=5, scoring="f1")
    print(k, scores.mean(), scores.std())
```

De esta manera, en cada fold, el escalador se ajusta solo con la parte de entrenamiento correspondiente. Luego se aplica a la parte de validación. Esto evita que la validación reciba información que no debería tener. Al ajustar hiperparámetros, también debemos cuidar el preprocesamiento, porque la validación debe evaluar el flujo completo y no solamente el modelo final.

La búsqueda manual tiene varias ventajas. Es fácil de entender, permite controlar cada paso y resulta muy útil cuando queremos explorar pocos valores. También es una buena herramienta didáctica, porque muestra la relación entre configuración y desempeño.

Pero también tiene limitaciones. Si queremos probar muchos hiperparámetros al mismo tiempo, la cantidad de combinaciones puede crecer rápidamente.

Supongamos que en un árbol de decisión queremos probar las siguientes opciones:

Hiperparámetro	Valores posibles
max_depth	2, 3, 4, 5, 8
min_samples_leaf	1, 5, 10
criterion	gini, entropy

En total tendríamos $5 \times 3 \times 2 = 30$ combinaciones. Si además usamos validación cruzada de 5 folds, el modelo se entrenará $30 \times 5 = 150$ veces.

Hacer esto manualmente empieza a ser incómodo. No es imposible, pero puede volverse repetitivo y propenso a errores.

Además, cuando probamos muchas combinaciones, debemos ser más cuidadosos con la interpretación. Siempre puede aparecer una configuración que gana por casualidad en la validación. Por eso, conviene definir rangos razonables y no probar valores sin criterio solo para perseguir una mejora mínima. Más combinaciones no siempre significan mejores decisiones: también aumentan el costo y el riesgo de sobreajustar la validación.

La búsqueda manual también puede quedar limitada por nuestra intuición. Si solo probamos valores muy cercanos entre sí, tal vez no exploremos configuraciones importantes. Si probamos valores demasiado amplios sin criterio, podemos perder tiempo. Por eso, el conocimiento del modelo y del problema sigue siendo necesario.

Una estrategia práctica es comenzar con una búsqueda manual pequeña. Esto permite entender cómo responde el modelo. Luego, si hace falta explorar más combinaciones, podemos usar herramientas automáticas como `GridSearchCV` o `RandomizedSearchCV`, que veremos en las próximas secciones. En otras palabras, la búsqueda manual es una buena opción para empezar, comprender el comportamiento del modelo y explorar pocos valores, mientras que la búsqueda automática resulta más útil cuando el número de combinaciones crece.

También debemos recordar que el resultado de la búsqueda depende de la métrica elegida. Si ajustamos KNN usando accuracy, podríamos elegir un valor de `k`. Si lo ajustamos usando recall, podríamos elegir otro. Esto no es un error: significa que distintos objetivos llevan a distintas configuraciones.

Por ejemplo, en un problema de detección de fraude, tal vez prioricemos recall para detectar la mayor cantidad posible de casos sospechosos. En un problema donde las falsas alarmas son muy costosas, tal vez prioricemos precision. El hiperparámetro elegido debe responder a esa decisión, porque el mejor valor depende de la métrica y la métrica, a su vez, depende del problema.

En regresión ocurre algo similar. Si ajustamos un modelo usando MAE, podríamos favorecer configuraciones con errores promedio moderados. Si usamos RMSE, penalizaremos más los errores grandes. La elección de la métrica afecta la elección de hiperparámetros.

Por eso, antes de ajustar, debemos tener claro qué significa “mejor” en el contexto del problema. Ajustar hiperparámetros sin una métrica adecuada puede llevar a optimizar algo que no representa la necesidad real.

La búsqueda manual no debe hacerse sobre el conjunto de prueba final. Este punto es fundamental. Si usamos el test para probar valores y elegir el mejor, el test deja de ser una evaluación independiente. Los hiperparámetros deben elegirse mediante

validación cruzada sobre los datos de entrenamiento. El test debe reservarse para el final y utilizarse únicamente para evaluar la configuración elegida.

La búsqueda manual de hiperparámetros consiste, entonces, en probar algunas configuraciones elegidas por nosotros, evaluarlas con una estrategia consistente y comparar sus resultados. Es una forma simple, transparente y muy útil para comprender cómo los hiperparámetros afectan el desempeño del modelo. Sin embargo, cuando hay muchas combinaciones, puede volverse costosa y repetitiva. Por eso, después de entender esta idea, podemos pasar a herramientas que automatizan la búsqueda manteniendo la lógica de validación cruzada.

GridSearchCV

La búsqueda manual de hiperparámetros es útil para entender cómo cambia un modelo cuando modificamos algunas configuraciones. Pero cuando queremos probar muchas combinaciones, escribir ciclos manuales puede volverse repetitivo. Para esos casos, `scikit-learn` ofrece una herramienta muy utilizada: `GridSearchCV`.

`GridSearchCV` permite definir una grilla de hiperparámetros, probar todas las combinaciones posibles incluidas en esa grilla y evaluarlas mediante validación cruzada. En lugar de probar manualmente un valor por vez, definimos un conjunto de valores posibles para cada hiperparámetro. Luego, la herramienta combina esos valores y evalúa cada configuración.

Por ejemplo, para un árbol de decisión podríamos querer probar los siguientes valores:

Hiperparámetro	Valores que se probarán
<code>max_depth</code>	2, 3, 4, 5
<code>min_samples_leaf</code>	1, 5, 10
<code>criterion</code>	"gini", "entropy"

A partir de esta grilla se generan combinaciones como las siguientes:

<code>max_depth</code>	<code>min_samples_leaf</code>	<code>criterion</code>
2	1	"gini"
2	5	"gini"
2	10	"gini"
2	1	"entropy"
...	...	...

Si tenemos cuatro valores para `max_depth`, tres valores para `min_samples_leaf` y dos valores para `criterion`, la cantidad total de combinaciones es:

$4 \times 3 \times 2 = 24$ combinaciones

Si además usamos validación cruzada de cinco folds, cada combinación se evalúa cinco veces. En total, se realizan:

$24 \text{ combinaciones} \times 5 \text{ folds} = 120 \text{ entrenamientos}$

Esto muestra por qué `GridSearchCV` puede ser muy práctico, pero también costoso. Automatiza la búsqueda, pero no elimina el trabajo computacional: cada combinación debe entrenarse y evaluarse.

La estructura básica en `scikit-learn` comienza con la importación de la clase:

```
from sklearn.model_selection import GridSearchCV
```

Primero definimos el modelo:

```
from sklearn.tree import DecisionTreeClassifier

modelo = DecisionTreeClassifier(random_state=42)
```

Luego definimos la grilla de hiperparámetros. En Python, esta grilla se representa mediante un diccionario:

```
param_grid = {
    "max_depth": [2, 3, 4, 5],
    "min_samples_leaf": [1, 5, 10],
    "criterion": ["gini", "entropy"]
}
```

Cada clave del diccionario es el nombre de un hiperparámetro, y cada valor es una lista con las opciones que queremos probar.

Después creamos el objeto `GridSearchCV`:

```
busqueda = GridSearchCV(
    estimator=modelo,
    param_grid=param_grid,
    cv=5,
    scoring="accuracy"
)
```

Aquí indicamos el modelo base mediante `estimator`, la grilla de hiperparámetros mediante `param_grid`, la cantidad de folds mediante `cv` y la métrica que queremos usar mediante `scoring`.

Luego ajustamos la búsqueda con los datos:

```
busqueda.fit(X_train, y_train)
```

Este paso ejecuta la búsqueda. `GridSearchCV` prueba todas las combinaciones de la grilla usando validación cruzada sobre los datos de entrenamiento.

Es importante notar que usamos `X_train` e `y_train`, y no el conjunto de prueba final. La búsqueda de hiperparámetros debe ocurrir dentro del proceso de entrenamiento. El conjunto de prueba debe permanecer reservado y no debe utilizarse para elegir hiperparámetros, ya que su función es proporcionar una evaluación final independiente.

Una vez finalizada la búsqueda, podemos consultar cuál fue la mejor combinación encontrada:

```
busqueda.best_params_
```

También podemos ver el mejor puntaje promedio obtenido durante la validación cruzada:

```
busqueda.best_score_
```

Por ejemplo, podríamos obtener un resultado como este:

```
best_params_:
{
  "criterion": "gini",
  "max_depth": 4,
  "min_samples_leaf": 5
}

best_score_:
0.82
```

Esto significa que, dentro de la grilla que definimos, esa combinación obtuvo el mejor desempeño promedio según la métrica elegida.

La expresión “dentro de la grilla” es importante. `GridSearchCV` no busca entre todos los valores posibles ni inventa configuraciones nuevas. Solo evalúa las combinaciones que nosotros incluimos en `param_grid`.

Si nuestra grilla es muy limitada, la búsqueda también lo será. Si nunca incluimos una profundidad adecuada, `GridSearchCV` no podrá encontrarla. Si incluimos valores demasiado extremos, puede perder tiempo probando configuraciones poco razonables. Por eso, definir la grilla también requiere conocimiento del modelo y criterio para elegir valores relevantes.

Una grilla pequeña puede ser suficiente para una primera exploración:

```
param_grid = {
    "max_depth": [3, 5, 7],
    "min_samples_leaf": [1, 5]
}
```

Una grilla más grande explora más opciones, pero también aumenta el costo:

```
param_grid = {
    "max_depth": [2, 3, 4, 5, 6, 8, 10],
    "min_samples_leaf": [1, 2, 5, 10, 20],
    "criterion": ["gini", "entropy"]
}
```

En este segundo caso hay siete valores para `max_depth`, cinco para `min_samples_leaf` y dos para `criterion`. Esto produce:

$7 \times 5 \times 2 = 70$ combinaciones

Con una validación cruzada de cinco folds, se realizan:

$70 \times 5 = 350$ entrenamientos

Por eso no conviene construir grillas enormes sin necesidad. Ajustar hiperparámetros no significa probar todo indiscriminadamente, sino explorar configuraciones razonables de manera ordenada.

GridSearchCV y pipelines

`GridSearchCV` también permite trabajar con pipelines. Esto es especialmente importante cuando el modelo requiere preprocesamiento. Si tenemos imputación, escalado o codificación de variables categóricas, esas transformaciones deben formar parte del pipeline evaluado.

Supongamos que construimos un pipeline con escalado y un modelo KNN:

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier

pipeline = Pipeline(steps=[
```

```

        ("scaler", StandardScaler()),
        ("modelo", KNeighborsClassifier())
    ])

```

Si queremos ajustar el hiperparámetro `n_neighbors` de KNN, debemos indicar el nombre del paso del pipeline y luego el nombre del hiperparámetro, separados por un doble guion bajo:

```

param_grid = {
    "modelo__n_neighbors": [3, 5, 7, 11, 15]
}

```

En la expresión `modelo__n_neighbors`, `modelo` es el nombre que le dimos al paso del pipeline y `n_neighbors` es el hiperparámetro de `KNeighborsClassifier` que queremos modificar.

Luego usamos `GridSearchCV` sobre el pipeline completo:

```

busqueda = GridSearchCV(
    estimator=pipeline,
    param_grid=param_grid,
    cv=5,
    scoring="f1"
)

busqueda.fit(X_train, y_train)

```

Esto evalúa cada valor de `n_neighbors` usando validación cruzada. En cada fold, el escalador se ajusta solamente con los datos de entrenamiento correspondientes y luego se aplica a los datos de validación. De esta forma, `GridSearchCV` prueba los hiperparámetros sin romper la lógica de validación y sin introducir fuga de datos durante el preprocesamiento.

También podemos ajustar hiperparámetros de distintos pasos del pipeline. Por ejemplo, si un pipeline incluye un modelo y alguna transformación configurable, la grilla puede contener opciones para ambos pasos. Lo importante es indicar correctamente el nombre de cada paso seguido de un doble guion bajo y del hiperparámetro correspondiente.

El mejor estimador

Después de ajustar `GridSearchCV`, podemos utilizar el mejor modelo encontrado. Por defecto, la herramienta vuelve a entrenar el mejor estimador usando todos los datos que recibió en el método `fit`. Ese modelo queda disponible en el atributo:

```

busqueda.best_estimator_

```

Esto resulta práctico porque podemos usarlo directamente para predecir sobre el conjunto de prueba final:

```
mejor_modelo = busqueda.best_estimator_  
y_pred = mejor_modelo.predict(X_test)
```

Luego evaluamos esas predicciones con la métrica que corresponda:

```
from sklearn.metrics import accuracy_score  
  
accuracy_score(y_test, y_pred)
```

Este resultado sobre `X_test` es diferente de `best_score_`. El atributo `best_score_` contiene el promedio obtenido mediante validación cruzada dentro del conjunto de entrenamiento. La evaluación sobre `X_test`, en cambio, es la medición final sobre datos que no participaron en la búsqueda.

Resultado	Qué representa
<code>best_score_</code>	Resultado promedio de validación cruzada obtenido durante la búsqueda
Evaluación sobre <code>X_test</code>	Resultado final sobre los datos reservados para prueba

Esta distinción es fundamental. Si elegimos los hiperparámetros observando la validación cruzada y luego evaluamos una sola vez sobre el conjunto de prueba, mantenemos una separación razonable entre la selección del modelo y su evaluación final.

Análisis de todos los resultados

También podemos revisar los resultados completos de la búsqueda. `GridSearchCV` guarda información sobre todas las combinaciones evaluadas en el atributo `cv_results_`:

```
busqueda.cv_results_
```

Este objeto contiene numerosos datos, entre ellos los parámetros probados, los puntajes medios, las posiciones en el ranking y los tiempos de entrenamiento. En los cuadernos prácticos puede convertirse en un `DataFrame` para analizar mejor qué configuraciones funcionaron bien y cuáles no:

```
import pandas as pd

resultados = pd.DataFrame(busqueda.cv_results_)
```

Luego podemos ordenar las combinaciones según su desempeño:

```
resultados.sort_values("rank_test_score").head()
```

Esto permite observar no solamente la configuración ganadora, sino también otras configuraciones con resultados cercanos. A veces varias opciones obtienen desempeños muy parecidos. En esos casos puede ser preferible elegir una configuración más simple, más estable o más fácil de interpretar, aunque no sea la que gana por una diferencia mínima.

Por ejemplo, si un árbol con `max_depth=8` obtiene un puntaje de 0.823 y otro con `max_depth=4` obtiene 0.821, la diferencia es muy pequeña. Podría resultar conveniente elegir el árbol menos profundo porque es más simple e interpretable.

Por lo tanto, la mejor configuración numérica no siempre es la única opción razonable. `GridSearchCV` automatiza la comparación, pero la decisión final sigue requiriendo interpretación. Debemos considerar la magnitud de las diferencias, la variabilidad de los resultados, el costo computacional, la complejidad del modelo y el sentido que tiene la configuración elegida.

Elección de la métrica

También es importante elegir correctamente la métrica indicada en `scoring`. Si usamos accuracy en un problema desbalanceado, la búsqueda puede favorecer configuraciones que funcionan bien para la clase mayoritaria, pero mal para la minoritaria. Si el objetivo es detectar la clase positiva, quizás convenga utilizar F1-score, recall u otra métrica más adecuada.

En regresión ocurre algo similar. Ajustar con R^2 , MAE o RMSE puede llevar a configuraciones diferentes. `GridSearchCV` busca la mejor configuración según la métrica que le indicamos. Si esa métrica no representa correctamente el problema, la búsqueda optimizará un objetivo equivocado.

Por eso, antes de iniciar la búsqueda, debemos tener claro qué significa que un modelo sea “mejor” dentro del contexto del problema que estamos resolviendo.

En regresión debemos recordar, además, la convención de las métricas negativas. Para ajustar un modelo usando MAE, podemos escribir:

```
busqueda = GridSearchCV(
    estimator=modelo,
```

```
param_grid=param_grid,  
cv=5,  
scoring="neg_mean_absolute_error"  
)
```

El mejor resultado será el menos negativo, porque internamente `scikit-learn` utiliza la convención de que los valores más altos son mejores. Para interpretarlo como un MAE positivo, habrá que cambiar el signo cuando corresponda.

Costo de una búsqueda exhaustiva

Otra limitación de `GridSearchCV` es que puede volverse muy costoso cuando la grilla es grande. Como prueba todas las combinaciones, el número de entrenamientos crece rápidamente. Una grilla más grande permite una exploración más amplia, pero también exige mucho más tiempo y recursos computacionales.

Si queremos explorar muchos hiperparámetros con una gran cantidad de valores posibles, una búsqueda exhaustiva puede no ser eficiente. En esos casos puede resultar conveniente utilizar una estrategia diferente: en lugar de probar todas las combinaciones, probar solamente algunas combinaciones seleccionadas al azar. Esa es la idea de `RandomizedSearchCV`, que veremos en la próxima sección.

`GridSearchCV` automatiza la búsqueda de hiperparámetros probando todas las combinaciones de una grilla definida por nosotros y evaluándolas mediante validación cruzada. Puede trabajar tanto con modelos individuales como con pipelines completos, permite elegir una métrica de evaluación y devuelve la mejor configuración encontrada dentro de la grilla. Es una herramienta poderosa, pero debe utilizarse con criterio. La grilla debe contener valores razonables, la métrica debe representar las necesidades reales del problema, el conjunto de prueba debe reservarse para la evaluación final y los resultados deben interpretarse más allá del número ganador.

RandomizedSearchCV

`GridSearchCV` prueba todas las combinaciones posibles dentro de una grilla. Esa estrategia es clara y ordenada, pero puede volverse costosa cuando tenemos muchos hiperparámetros o muchos valores posibles para cada uno.

En esos casos, una alternativa útil es `RandomizedSearchCV`.

La idea de `RandomizedSearchCV` es probar una cantidad limitada de combinaciones elegidas al azar dentro de un espacio de búsqueda definido. En lugar de evaluar

todas las combinaciones, selecciona algunas y las evalúa mediante validación cruzada.

La diferencia principal entre ambas herramientas puede resumirse de esta manera:

Herramienta	Estrategia
GridSearchCV	Prueba todas las combinaciones de la grilla
RandomizedSearchCV	Prueba algunas combinaciones seleccionadas al azar

Esto puede parecer menos completo, pero tiene una ventaja importante: permite explorar espacios de búsqueda más grandes sin entrenar una cantidad enorme de modelos.

Supongamos que queremos ajustar tres hiperparámetros de un modelo y que cada uno tiene diez valores posibles. Si usamos una grilla completa, la cantidad de combinaciones puede crecer muy rápido:

$10 \times 10 \times 10 = 1000$ combinaciones

Si además usamos validación cruzada de cinco folds, eso implicaría:

1000 combinaciones $\times$ 5 folds = 5000 entrenamientos

En muchos proyectos, ese costo puede ser demasiado alto. `RandomizedSearchCV` permite indicar que, de todas esas posibilidades, queremos probar solamente 30, 50 o 100 combinaciones, según el tiempo y los recursos disponibles.

Ese control se realiza mediante el parámetro `n_iter`, que indica la cantidad de combinaciones que se evaluarán.

Por ejemplo:

```
from sklearn.model_selection import RandomizedSearchCV

busqueda = RandomizedSearchCV(
    estimator=modelo,
    param_distributions=param_distributions,
    n_iter=30,
    cv=5,
    scoring="accuracy",
    random_state=42
)
```

En este caso se probarán 30 combinaciones de hiperparámetros. Cada una se evaluará con cinco folds, por lo que se realizarán:

30 combinaciones $\times$ 5 folds = 150 entrenamientos

Esto puede ser mucho más razonable que probar miles de combinaciones.

La estructura es parecida a la de `GridSearchCV`, pero en lugar de usar `param_grid`, usamos `param_distributions`.

```
param_distributions = {
    "max_depth": [2, 3, 4, 5, 6, 8, 10, None],
    "min_samples_leaf": [1, 2, 5, 10, 20],
    "criterion": ["gini", "entropy"]
}
```

Aunque el nombre diga “distributions”, también podemos usar listas de valores. En ese caso, `RandomizedSearchCV` toma combinaciones al azar a partir de esas listas.

Para un árbol de decisión, el ejemplo completo podría ser:

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import RandomizedSearchCV

modelo = DecisionTreeClassifier(random_state=42)

param_distributions = {
    "max_depth": [2, 3, 4, 5, 6, 8, 10, None],
    "min_samples_leaf": [1, 2, 5, 10, 20],
    "criterion": ["gini", "entropy"]
}

busqueda = RandomizedSearchCV(
    estimator=modelo,
    param_distributions=param_distributions,
    n_iter=20,
    cv=5,
    scoring="accuracy",
    random_state=42
)

busqueda.fit(X_train, y_train)
```

Después de ejecutar la búsqueda, podemos consultar la mejor combinación encontrada:

```
busqueda.best_params_
```

También podemos consultar el mejor puntaje promedio de validación cruzada:

```
busqueda.best_score_
```

Al igual que ocurre con `GridSearchCV`, este resultado corresponde a la validación cruzada realizada sobre el conjunto de entrenamiento. No es todavía la evaluación final sobre el conjunto de prueba.

El atributo `best_score_` muestra el mejor promedio obtenido durante la búsqueda. La evaluación final debe hacerse después, utilizando el conjunto de prueba que se mantuvo reservado.

Una diferencia importante es que `RandomizedSearchCV` no garantiza probar todas las combinaciones. Por eso, la mejor configuración encontrada es la mejor entre las combinaciones que fueron seleccionadas al azar, y no necesariamente la mejor dentro de todo el espacio posible.

Esto no es un defecto, sino parte de su diseño. La herramienta busca un equilibrio entre exploración y costo computacional. Muchas veces, explorar una muestra amplia de combinaciones puede ser más útil que recorrer exhaustivamente una grilla pequeña.

Podemos imaginar, por ejemplo, dos estrategias:

Estrategia	Alcance
<code>GridSearchCV</code>	Prueba 20 combinaciones muy específicas
<code>RandomizedSearchCV</code>	Prueba 50 combinaciones variadas dentro de un espacio más amplio

La segunda estrategia puede descubrir regiones interesantes que una grilla demasiado rígida no habría considerado. Esto es especialmente útil cuando no sabemos bien qué valores pueden funcionar mejor.

Uso de distribuciones

`RandomizedSearchCV` también permite usar distribuciones estadísticas para algunos hiperparámetros. Esto resulta útil cuando un hiperparámetro puede tomar muchos valores posibles, como sucede con números continuos o rangos amplios.

Por ejemplo, algunos modelos tienen un hiperparámetro de regularización, como `C` en regresión logística. En lugar de definir una lista pequeña de valores, podríamos querer explorar valores dentro de un rango. En la práctica, esto puede hacerse usando distribuciones de `scipy.stats`.

No necesitamos profundizar ahora en ese uso avanzado. Para este capítulo, alcanza con comprender que `RandomizedSearchCV` puede trabajar tanto con listas de valores como con distribuciones, y que su objetivo es explorar un espacio de búsqueda sin probarlo todo.

Por este motivo, `RandomizedSearchCV` resulta especialmente útil cuando el espacio de búsqueda es grande y queremos limitar la cantidad de pruebas.

RandomizedSearchCV y pipelines

`RandomizedSearchCV` también puede trabajar con pipelines, del mismo modo que `GridSearchCV`. Si tenemos un pipeline con preprocesamiento y modelo, podemos ajustar los hiperparámetros del modelo usando el nombre del paso y el doble guion bajo.

Por ejemplo, con KNN:

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import RandomizedSearchCV

pipeline = Pipeline(steps=[
    ("scaler", StandardScaler()),
    ("modelo", KNeighborsClassifier())
])

param_distributions = {
    "modelo__n_neighbors": [3, 5, 7, 9, 11, 15, 21],
    "modelo__weights": ["uniform", "distance"]
}

busqueda = RandomizedSearchCV(
    estimator=pipeline,
    param_distributions=param_distributions,
    n_iter=8,
    cv=5,
    scoring="f1",
    random_state=42
)

busqueda.fit(X_train, y_train)
```

En este caso se prueban ocho combinaciones al azar entre los valores de `n_neighbors` y `weights`. Como el modelo está dentro de un pipeline, el escalado se ajusta correctamente dentro de cada fold.

De esta manera, `RandomizedSearchCV` puede explorar distintas configuraciones sin separar el preprocesamiento de la validación. Esto evita que las transformaciones aprendan información a partir de los datos de validación y ayuda a prevenir la fuga de datos.

Reproducibilidad y `random_state`

El parámetro `random_state` es importante porque permite reproducir la selección aleatoria de combinaciones. Si usamos el mismo valor de `random_state`, la búsqueda elegirá las mismas combinaciones.

Esto facilita comparar resultados y documentar el proceso. Sin `random_state`, dos ejecuciones pueden probar combinaciones distintas y obtener resultados diferentes.

Esto no siempre es un problema, pero para aprender, comparar y documentar suele ser conveniente fijarlo.

Elección de `n_iter`

También debemos elegir cuidadosamente el valor de `n_iter`. Si es muy bajo, la búsqueda puede ser demasiado limitada. Si es muy alto, el costo computacional puede acercarse al de una grilla exhaustiva.

Valor de <code>n_iter</code>	Consecuencia
Pequeño	Búsqueda rápida, pero menor exploración
Grande	Mayor exploración, pero también mayor costo

La elección depende del tamaño del espacio de búsqueda, del costo de entrenar el modelo y de cuánto queremos explorar. Para una primera aproximación puede bastar con un número pequeño. Si los resultados parecen prometedores o necesitamos una búsqueda más completa, podemos aumentar `n_iter`.

Elección de la métrica

Al igual que en `GridSearchCV`, la métrica indicada mediante `scoring` define qué significa que una configuración sea “mejor”.

Si usamos `accuracy`, la búsqueda seleccionará la configuración con mejor accuracy promedio. Si usamos `f1`, seleccionará la de mejor F1-score promedio. Si usamos `neg_mean_absolute_error`, seleccionará la que minimice el MAE, siguiendo la convención de valores negativos de `scikit-learn`.

Por lo tanto, `RandomizedSearchCV` optimiza la métrica que le indicamos. Si elegimos mal esa métrica, buscará una configuración inadecuada para el problema.

Antes de iniciar la búsqueda, debemos definir qué tipo de error importa más. En clasificación desbalanceada, `accuracy` puede no ser una buena elección. En regresión, MAE y RMSE pueden llevar a decisiones diferentes. La herramienta automatiza la exploración, pero no decide por nosotros qué objetivo tiene sentido.

Evaluación del mejor modelo

Después de la búsqueda, podemos acceder al mejor estimador encontrado:

```
mejor_modelo = busqueda.best_estimator_
```

Luego podemos usarlo para predecir sobre el conjunto de prueba final:

```
y_pred = mejor_modelo.predict(X_test)
```

Después evaluamos las predicciones con la métrica correspondiente. Esta evaluación final es importante porque el conjunto de prueba no participó en la elección de los hiperparámetros.

La búsqueda se realiza utilizando los datos de entrenamiento y validación cruzada. El conjunto de prueba se utiliza después, para medir el desempeño del modelo elegido.

Análisis de los resultados

También podemos revisar todas las combinaciones evaluadas mediante `cv_results_`, igual que en `GridSearchCV`.

```
import pandas as pd

resultados = pd.DataFrame(busqueda.cv_results_)
resultados.sort_values("rank_test_score").head()
```

Esto permite ver no solo la mejor combinación, sino también otras configuraciones cercanas. A veces varias combinaciones tienen resultados parecidos, y puede ser razonable elegir una alternativa más simple, más estable o menos costosa.

No conviene interpretar el resultado de `RandomizedSearchCV` como una verdad absoluta. Al haber una selección aleatoria de combinaciones, otra ejecución con otro `random_state` o con un valor mayor de `n_iter` podría encontrar una configuración diferente.

Por eso, cuando los resultados son muy cercanos, debemos evitar conclusiones exageradas. La búsqueda aleatoria ayuda a explorar, pero las diferencias pequeñas deben interpretarse con prudencia.

Combinar `RandomizedSearchCV` y `GridSearchCV`

En algunos flujos de trabajo, una estrategia práctica consiste en usar `RandomizedSearchCV` para explorar un espacio amplio y luego, si hace falta, usar `GridSearchCV` en una zona más reducida.

Primero buscamos regiones prometedoras mediante una búsqueda aleatoria. Después podemos afinar la búsqueda con una grilla pequeña y más específica.

No siempre es necesario utilizar ambas herramientas. En muchos proyectos educativos o iniciales, una búsqueda manual o una grilla pequeña resulta suficiente. Sin embargo, es útil conocer las diferencias entre las herramientas para elegir la más conveniente.

`RandomizedSearchCV` es especialmente útil cuando hay muchos hiperparámetros, muchos valores posibles, el modelo es costoso de entrenar, queremos limitar la cantidad de pruebas o no sabemos bien qué regiones del espacio pueden funcionar mejor.

En cambio, `GridSearchCV` puede ser preferible cuando la grilla es pequeña, los valores son pocos y queremos una búsqueda exhaustiva dentro de esas opciones.

Herramienta	Cuándo puede resultar más conveniente
<code>GridSearchCV</code>	Cuando la grilla es pequeña y queremos probar todas las combinaciones
<code>RandomizedSearchCV</code>	Cuando el espacio es grande y queremos controlar el costo computacional

Ambas herramientas comparten una idea fundamental: utilizan validación cruzada para comparar configuraciones sin tocar el conjunto de prueba final. Este principio es más importante que la diferencia entre una búsqueda exhaustiva y una búsqueda aleatoria.

`RandomizedSearchCV`, en conclusión, es una herramienta de `scikit-learn` que permite ajustar hiperparámetros probando una cantidad limitada de combinaciones seleccionadas al azar dentro de un espacio de búsqueda. Es útil cuando una búsqueda exhaustiva sería demasiado costosa.

Permite controlar el número de combinaciones mediante `n_iter`, trabajar con pipelines, elegir una métrica mediante `scoring` y conservar el conjunto de prueba para la evaluación definitiva. Al igual que cualquier búsqueda de hiperparámetros, sus resultados deben interpretarse con criterio, considerando la métrica, la estabilidad, la complejidad y el contexto del problema.

RandomizedSearchCV

`GridSearchCV` prueba todas las combinaciones posibles dentro de una grilla. Esa estrategia es clara y ordenada, pero puede volverse costosa cuando tenemos muchos hiperparámetros o muchos valores posibles para cada uno.

En esos casos, una alternativa útil es `RandomizedSearchCV`.

La idea de `RandomizedSearchCV` es probar una cantidad limitada de combinaciones elegidas al azar dentro de un espacio de búsqueda definido. En lugar de evaluar todas las combinaciones, selecciona algunas y las evalúa mediante validación cruzada.

La diferencia principal entre ambas herramientas puede resumirse de esta manera:

Herramienta	Estrategia
GridSearchCV	Prueba todas las combinaciones de la grilla
RandomizedSearchCV	Prueba algunas combinaciones seleccionadas al azar

Esto puede parecer menos completo, pero tiene una ventaja importante: permite explorar espacios de búsqueda más grandes sin entrenar una cantidad enorme de modelos.

Supongamos que queremos ajustar tres hiperparámetros de un modelo y que cada uno tiene diez valores posibles. Si usamos una grilla completa, la cantidad de combinaciones puede crecer muy rápido:

$10 \times 10 \times 10 = 1000$ combinaciones

Si además usamos validación cruzada de cinco folds, eso implicaría:

1000 combinaciones $\times$ 5 folds = 5000 entrenamientos

En muchos proyectos, ese costo puede ser demasiado alto. `RandomizedSearchCV` permite indicar que, de todas esas posibilidades, queremos probar solamente 30, 50 o 100 combinaciones, según el tiempo y los recursos disponibles.

Ese control se realiza mediante el parámetro `n_iter`, que indica la cantidad de combinaciones que se evaluarán.

Por ejemplo:

```
from sklearn.model_selection import RandomizedSearchCV

busqueda = RandomizedSearchCV(
    estimator=modelo,
    param_distributions=param_distributions,
    n_iter=30,
    cv=5,
    scoring="accuracy",
    random_state=42
)
```

En este caso se probarán 30 combinaciones de hiperparámetros. Cada una se evaluará con cinco folds, por lo que se realizarán:

30 combinaciones $\times$ 5 folds = 150 entrenamientos

Esto puede ser mucho más razonable que probar miles de combinaciones.

La estructura es parecida a la de `GridSearchCV`, pero en lugar de usar `param_grid`, usamos `param_distributions`.

```
param_distributions = {
    "max_depth": [2, 3, 4, 5, 6, 8, 10, None],
    "min_samples_leaf": [1, 2, 5, 10, 20],
    "criterion": ["gini", "entropy"]
}
```

Aunque el nombre diga “distributions”, también podemos usar listas de valores. En ese caso, `RandomizedSearchCV` toma combinaciones al azar a partir de esas listas.

Para un árbol de decisión, el ejemplo completo podría ser:

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import RandomizedSearchCV

modelo = DecisionTreeClassifier(random_state=42)

param_distributions = {
    "max_depth": [2, 3, 4, 5, 6, 8, 10, None],
    "min_samples_leaf": [1, 2, 5, 10, 20],
    "criterion": ["gini", "entropy"]
}

busqueda = RandomizedSearchCV(
    estimator=modelo,
    param_distributions=param_distributions,
    n_iter=20,
    cv=5,
    scoring="accuracy",
    random_state=42
)

busqueda.fit(X_train, y_train)
```

Después de ejecutar la búsqueda, podemos consultar la mejor combinación encontrada:

```
busqueda.best_params_
```

También podemos consultar el mejor puntaje promedio de validación cruzada:

```
busqueda.best_score_
```

Al igual que ocurre con `GridSearchCV`, este resultado corresponde a la validación cruzada realizada sobre el conjunto de entrenamiento. No es todavía la evaluación final sobre el conjunto de prueba.

El atributo `best_score_` muestra el mejor promedio obtenido durante la búsqueda. La evaluación final debe hacerse después, utilizando el conjunto de prueba que se mantuvo reservado.

Una diferencia importante es que `RandomizedSearchCV` no garantiza probar todas las combinaciones. Por eso, la mejor configuración encontrada es la mejor entre las combinaciones que fueron seleccionadas al azar, y no necesariamente la mejor dentro de todo el espacio posible.

Esto no es un defecto, sino parte de su diseño. La herramienta busca un equilibrio entre exploración y costo computacional. Muchas veces, explorar una muestra amplia de combinaciones puede ser más útil que recorrer exhaustivamente una grilla pequeña.

Podemos imaginar, por ejemplo, dos estrategias:

Estrategia	Alcance
<code>GridSearchCV</code>	Prueba 20 combinaciones muy específicas
<code>RandomizedSearchCV</code>	Prueba 50 combinaciones variadas dentro de un espacio más amplio

La segunda estrategia puede descubrir regiones interesantes que una grilla demasiado rígida no habría considerado. Esto es especialmente útil cuando no sabemos bien qué valores pueden funcionar mejor.

Uso de distribuciones

`RandomizedSearchCV` también permite usar distribuciones estadísticas para algunos hiperparámetros. Esto resulta útil cuando un hiperparámetro puede tomar muchos valores posibles, como sucede con números continuos o rangos amplios.

Por ejemplo, algunos modelos tienen un hiperparámetro de regularización, como `C` en regresión logística. En lugar de definir una lista pequeña de valores, podríamos querer explorar valores dentro de un rango. En la práctica, esto puede hacerse usando distribuciones de `scipy.stats`.

No necesitamos profundizar ahora en ese uso avanzado. Para este capítulo, alcanza con comprender que `RandomizedSearchCV` puede trabajar tanto con listas de valores como con distribuciones, y que su objetivo es explorar un espacio de búsqueda sin probarlo todo.

Por este motivo, `RandomizedSearchCV` resulta especialmente útil cuando el espacio de búsqueda es grande y queremos limitar la cantidad de pruebas.

RandomizedSearchCV y pipelines

`RandomizedSearchCV` también puede trabajar con pipelines, del mismo modo que `GridSearchCV`. Si tenemos un pipeline con preprocesamiento y modelo, podemos

ajustar los hiperparámetros del modelo usando el nombre del paso y el doble guion bajo.

Por ejemplo, con KNN:

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import RandomizedSearchCV

pipeline = Pipeline(steps=[
    ("scaler", StandardScaler()),
    ("modelo", KNeighborsClassifier())
])

param_distributions = {
    "modelo__n_neighbors": [3, 5, 7, 9, 11, 15, 21],
    "modelo__weights": ["uniform", "distance"]
}

busqueda = RandomizedSearchCV(
    estimator=pipeline,
    param_distributions=param_distributions,
    n_iter=8,
    cv=5,
    scoring="f1",
    random_state=42
)

busqueda.fit(X_train, y_train)
```

En este caso se prueban ocho combinaciones al azar entre los valores de `n_neighbors` y `weights`. Como el modelo está dentro de un pipeline, el escalado se ajusta correctamente dentro de cada fold.

De esta manera, `RandomizedSearchCV` puede explorar distintas configuraciones sin separar el preprocesamiento de la validación. Esto evita que las transformaciones aprendan información a partir de los datos de validación y ayuda a prevenir la fuga de datos.

Reproducibilidad y `random_state`

El parámetro `random_state` es importante porque permite reproducir la selección aleatoria de combinaciones. Si usamos el mismo valor de `random_state`, la búsqueda elegirá las mismas combinaciones.

Esto facilita comparar resultados y documentar el proceso. Sin `random_state`, dos ejecuciones pueden probar combinaciones distintas y obtener resultados diferentes. Esto no siempre es un problema, pero para aprender, comparar y documentar suele ser conveniente fijarlo.

Elección de `n_iter`

También debemos elegir cuidadosamente el valor de `n_iter`. Si es muy bajo, la búsqueda puede ser demasiado limitada. Si es muy alto, el costo computacional puede acercarse al de una grilla exhaustiva.

Valor de <code>n_iter</code>	Consecuencia
Pequeño	Búsqueda rápida, pero menor exploración
Grande	Mayor exploración, pero también mayor costo

La elección depende del tamaño del espacio de búsqueda, del costo de entrenar el modelo y de cuánto queremos explorar. Para una primera aproximación puede bastar con un número pequeño. Si los resultados parecen prometedores o necesitamos una búsqueda más completa, podemos aumentar `n_iter`.

Elección de la métrica

Al igual que en `GridSearchCV`, la métrica indicada mediante `scoring` define qué significa que una configuración sea “mejor”.

Si usamos `accuracy`, la búsqueda seleccionará la configuración con mejor `accuracy` promedio. Si usamos `f1`, seleccionará la de mejor F1-score promedio. Si usamos `neg_mean_absolute_error`, seleccionará la que minimice el MAE, siguiendo la convención de valores negativos de `scikit-learn`.

Por lo tanto, `RandomizedSearchCV` optimiza la métrica que le indicamos. Si elegimos mal esa métrica, buscará una configuración inadecuada para el problema.

Antes de iniciar la búsqueda, debemos definir qué tipo de error importa más. En clasificación desbalanceada, `accuracy` puede no ser una buena elección. En regresión, MAE y RMSE pueden llevar a decisiones diferentes. La herramienta automatiza la exploración, pero no decide por nosotros qué objetivo tiene sentido.

Evaluación del mejor modelo

Después de la búsqueda, podemos acceder al mejor estimador encontrado:

```
mejor_modelo = busqueda.best_estimator_
```

Luego podemos usarlo para predecir sobre el conjunto de prueba final:

```
y_pred = mejor_modelo.predict(X_test)
```

Después evaluamos las predicciones con la métrica correspondiente. Esta evaluación final es importante porque el conjunto de prueba no participó en la elección de los hiperparámetros.

La búsqueda se realiza utilizando los datos de entrenamiento y validación cruzada. El conjunto de prueba se utiliza después, para medir el desempeño del modelo elegido.

Análisis de los resultados

También podemos revisar todas las combinaciones evaluadas mediante `cv_results_`, igual que en `GridSearchCV`.

```
import pandas as pd

resultados = pd.DataFrame(busqueda.cv_results_)
resultados.sort_values("rank_test_score").head()
```

Esto permite ver no solo la mejor combinación, sino también otras configuraciones cercanas. A veces varias combinaciones tienen resultados parecidos, y puede ser razonable elegir una alternativa más simple, más estable o menos costosa.

No conviene interpretar el resultado de `RandomizedSearchCV` como una verdad absoluta. Al haber una selección aleatoria de combinaciones, otra ejecución con otro `random_state` o con un valor mayor de `n_iter` podría encontrar una configuración diferente.

Por eso, cuando los resultados son muy cercanos, debemos evitar conclusiones exageradas. La búsqueda aleatoria ayuda a explorar, pero las diferencias pequeñas deben interpretarse con prudencia.

Combinar `RandomizedSearchCV` y `GridSearchCV`

En algunos flujos de trabajo, una estrategia práctica consiste en usar `RandomizedSearchCV` para explorar un espacio amplio y luego, si hace falta, usar `GridSearchCV` en una zona más reducida.

Primero buscamos regiones prometedoras mediante una búsqueda aleatoria. Después podemos afinar la búsqueda con una grilla pequeña y más específica.

No siempre es necesario utilizar ambas herramientas. En muchos proyectos educativos o iniciales, una búsqueda manual o una grilla pequeña resulta suficiente. Sin embargo, es útil conocer las diferencias entre las herramientas para elegir la más conveniente.

`RandomizedSearchCV` es especialmente útil cuando hay muchos hiperparámetros, muchos valores posibles, el modelo es costoso de entrenar, queremos limitar la cantidad de pruebas o no sabemos bien qué regiones del espacio pueden funcionar mejor.

En cambio, `GridSearchCV` puede ser preferible cuando la grilla es pequeña, los valores son pocos y queremos una búsqueda exhaustiva dentro de esas opciones.

Herramienta	Cuándo puede resultar más conveniente
<code>GridSearchCV</code>	Cuando la grilla es pequeña y queremos probar todas las combinaciones
<code>RandomizedSearchCV</code>	Cuando el espacio es grande y queremos controlar el costo computacional

Ambas herramientas comparten una idea fundamental: utilizan validación cruzada para comparar configuraciones sin tocar el conjunto de prueba final. Este principio es más importante que la diferencia entre una búsqueda exhaustiva y una búsqueda aleatoria.

`RandomizedSearchCV`, en conclusión, es una herramienta de `scikit-learn` que permite ajustar hiperparámetros probando una cantidad limitada de combinaciones seleccionadas al azar dentro de un espacio de búsqueda. Es útil cuando una búsqueda exhaustiva sería demasiado costosa. Permite controlar el número de combinaciones mediante `n_iter`, trabajar con pipelines, elegir una métrica mediante `scoring` y conservar el conjunto de prueba para la evaluación definitiva. Al igual que cualquier búsqueda de hiperparámetros, sus resultados deben interpretarse con criterio, considerando la métrica, la estabilidad, la complejidad y el contexto del problema.

Buscar mejores configuraciones sin engañar la evaluación

En este capítulo estudiamos el ajuste de hiperparámetros, una etapa importante para mejorar modelos sin perder de vista la generalización. Vimos que un modelo no solo depende de los datos y del algoritmo elegido, sino también de ciertas configuraciones que se definen antes del entrenamiento.

Los hiperparámetros no son valores que el modelo aprende directamente de los datos. Son decisiones previas que influyen en cómo aprende. En KNN, por ejemplo, el valor de `k` define cuántos vecinos se consideran. En un árbol de decisión, la profundidad máxima controla cuánto puede crecer el árbol. En otros modelos, los hiperparámetros pueden regular la complejidad, la penalización, la cantidad de iteraciones o la forma de construir las decisiones.

A diferencia de los parámetros, que se aprenden durante el entrenamiento, los hiperparámetros se definen antes de entrenar y modifican el comportamiento del modelo.

Ajustar hiperparámetros significa probar distintas configuraciones y elegir una opción adecuada según una métrica y una estrategia de evaluación. Esa elección no debe basarse en una sola intuición ni en un resultado aislado, sino en una comparación ordenada.

Primero vimos la búsqueda manual. Esta estrategia consiste en elegir algunos valores, entrenar y evaluar cada configuración. Es simple, transparente y muy útil para comenzar, explorar pocos valores y comprender cómo cambia el desempeño del modelo cuando cambia un hiperparámetro.

Luego vimos `GridSearchCV`, que automatiza la búsqueda probando todas las combinaciones de una grilla definida por nosotros y evaluándolas mediante validación cruzada. Esta herramienta es muy útil cuando la cantidad de combinaciones es razonable y queremos una búsqueda exhaustiva dentro de ese conjunto de opciones.

También vimos `RandomizedSearchCV`, que prueba solamente una cantidad limitada de combinaciones seleccionadas al azar. Esta estrategia resulta especialmente útil cuando el espacio de búsqueda es grande y probar todas las combinaciones sería demasiado costoso. Además, permite controlar la cantidad de pruebas mediante el parámetro `n_iter`.

Ambas herramientas se apoyan en la misma idea central: usar validación cruzada para comparar configuraciones sin tocar el conjunto de prueba final. La diferencia está en cómo exploran las combinaciones.

Herramienta	Forma de explorar
<code>GridSearchCV</code>	Prueba todas las combinaciones incluidas en la grilla
<code>RandomizedSearchCV</code>	Prueba una muestra de combinaciones dentro de un espacio definido

Ninguna de estas herramientas reemplaza el criterio. Si la grilla está mal elegida, si la métrica no representa el objetivo real o si el flujo de evaluación está contaminado, la búsqueda puede llevarnos a una configuración aparentemente buena, pero poco útil.

La herramienta automatiza la búsqueda. El criterio sigue siendo necesario para decidir qué buscar, cómo evaluar y cómo interpretar el resultado.

Un punto central del capítulo fue evitar ajustar sobre el conjunto de prueba. El test final no debe usarse para elegir hiperparámetros, comparar muchas configuraciones

o tomar decisiones durante el desarrollo. Si lo hacemos, deja de ser una evaluación independiente.

El flujo más cuidadoso consiste en separar primero los datos en entrenamiento y prueba. Luego, dentro del conjunto de entrenamiento, usamos validación cruzada para comparar configuraciones. Después elegimos el modelo y recién al final evaluamos una vez sobre el conjunto de prueba.

Etapas	Función
Entrenamiento	Servir como base para aprender y validar configuraciones
Validación cruzada	Comparar hiperparámetros
Prueba final	Evaluar el modelo elegido

Esta separación de roles ayuda a proteger la evaluación. Si usamos el test durante la búsqueda, podemos terminar eligiendo una configuración que tuvo suerte con ese conjunto particular. La métrica final sería demasiado optimista y no representaría tan bien el desempeño ante datos realmente nuevos.

También vimos que, cuando hay preprocesamiento, conviene ajustar hiperparámetros sobre pipelines completos. Si el modelo necesita imputación, escalado, codificación de categorías u otras transformaciones, esas etapas deben estar dentro del pipeline. Así, durante cada fold, las transformaciones se ajustan solamente con los datos de entrenamiento correspondientes.

En otras palabras, el ajuste debe incluir el flujo completo de preprocesamiento, modelo y validación cruzada. Esto ayuda a evitar fuga de datos.

Este punto conecta el ajuste de hiperparámetros con varios temas anteriores: separación entre entrenamiento y prueba, validación cruzada, fuga de datos, sobreajuste y evaluación honesta. Ajustar hiperparámetros puede mejorar el desempeño, pero también puede generar falsas mejoras si se hace sin cuidado.

También debemos recordar que la mejor configuración depende de la métrica. En clasificación desbalanceada, elegir hiperparámetros usando accuracy puede no ser adecuado si lo importante es detectar la clase minoritaria. En regresión, optimizar MAE o RMSE puede llevar a decisiones distintas. La búsqueda siempre optimiza lo que le indicamos.

Por eso, la mejor configuración depende de la métrica, y la métrica depende del problema.

Antes de buscar hiperparámetros, debemos decidir qué error importa más. No alcanza con encontrar el mejor número en una tabla. Hay que interpretar si la mejora

es relevante, si el modelo es estable, si la configuración es razonable y si la complejidad adicional se justifica.

Un modelo con una mejora mínima pero mucha más complejidad puede no ser la mejor elección. Un modelo apenas menos preciso, pero más simple, estable e interpretable, puede ser preferible en muchos contextos.

Ajustar hiperparámetros no significa perseguir la métrica más alta a cualquier costo. Significa buscar una configuración útil, estable y coherente con el problema.

El ajuste de hiperparámetros permite mejorar modelos comparando distintas configuraciones de manera ordenada. La búsqueda puede ser manual, exhaustiva con `GridSearchCV` o aleatoria con `RandomizedSearchCV`. Pero en todos los casos debemos usar validación cruzada, elegir métricas adecuadas, trabajar con pipelines cuando haya preprocesamiento y reservar el conjunto de prueba para la evaluación final. Ajustar bien no es solo encontrar una mejor configuración: es mejorar el modelo sin engañar la evaluación.

Capítulo 20 · Regularización y control de complejidad

¿Por qué controlar la complejidad del modelo?

En los capítulos anteriores vimos que un modelo debe aprender patrones útiles, pero no debería memorizar detalles accidentales de los datos de entrenamiento. Ese equilibrio es una de las ideas centrales de Machine Learning. Si el modelo es demasiado simple, puede subajustar. Si es demasiado flexible, puede sobreajustar.

La regularización aparece como una forma de controlar esa flexibilidad.

Regularizar un modelo significa limitar o penalizar su complejidad para favorecer una mejor generalización.

La idea no es impedir que el modelo aprenda. La idea es evitar que aprenda de una manera excesivamente ajustada a los datos de entrenamiento. Un modelo regularizado acepta, en cierto sentido, no perseguir cada detalle de los datos conocidos si eso permite funcionar mejor con datos nuevos.

Esto puede parecer extraño al principio. Uno podría pensar que el mejor modelo es el que logra el menor error posible durante el entrenamiento. Pero ya sabemos que eso no siempre es cierto. Un error de entrenamiento muy bajo puede ser una señal de sobreajuste.

No buscamos solamente minimizar el error en entrenamiento.

Buscamos un modelo que generalice bien.

Supongamos que queremos predecir precios de viviendas. Un modelo muy flexible podría ajustarse demasiado a cada caso del conjunto de entrenamiento: viviendas particulares, valores extremos, errores de carga, barrios poco representados o combinaciones accidentales de variables. Tal vez consiga un error muy bajo sobre esos datos, pero cuando aparezcan nuevas viviendas, su desempeño puede caer.

Un modelo más controlado quizá no ajuste perfectamente todos los casos de entrenamiento, pero puede capturar una relación más general y estable entre las variables y el precio.

Concepto	Descripción
Modelo sin control	puede seguir detalles accidentales.
Modelo regularizado	tiende a preferir soluciones más simples.

Esta preferencia por soluciones más simples no significa que siempre debamos elegir el modelo más básico. Significa que, entre varias soluciones con desempeño parecido, suele ser mejor una que dependa menos de detalles frágiles de los datos.

En regresión lineal, por ejemplo, el modelo aprende coeficientes para las variables predictoras. Si algunos coeficientes toman valores muy grandes, el modelo puede volverse muy sensible a pequeñas variaciones en los datos. Un pequeño cambio en una variable puede producir un gran cambio en la predicción.

La regularización puede penalizar coeficientes demasiado grandes. De esta forma, el modelo no solo intenta reducir el error, sino también mantener sus coeficientes bajo cierto control.

Concepto	Descripción
Sin regularización	el modelo busca reducir el error.
Con regularización	el modelo busca reducir el error sin volverse innecesariamente complejo.

Esta idea conecta directamente con el sobreajuste. Cuando un modelo se adapta demasiado al entrenamiento, suele estar usando demasiada flexibilidad. La regularización reduce esa libertad. No garantiza por sí sola un buen modelo, pero puede ayudar a que el aprendizaje sea más estable.

En modelos lineales, controlar la complejidad puede significar limitar el tamaño de los coeficientes. En árboles de decisión, puede significar limitar la profundidad, exigir una cantidad mínima de muestras por hoja o impedir divisiones demasiado específicas. En otros modelos, la regularización puede tomar formas distintas, pero la intención general es parecida: evitar que el modelo se vuelva excesivamente ajustado a los datos conocidos.

Distintos modelos, distintas formas de controlar complejidad.

Misma idea general: reducir el riesgo de sobreajuste.

No todos los controles de complejidad se llaman formalmente “regularización” en todos los modelos, pero comparten una lógica similar. Por ejemplo, limitar la profundidad de un árbol no es exactamente lo mismo que agregar una penalización a los coeficientes de una regresión, pero en ambos casos estamos restringiendo la capacidad del modelo para adaptarse demasiado.

La regularización también puede ser útil cuando hay muchas variables predictoras. Si un dataset tiene muchas columnas, algunas pueden aportar información real y otras pueden ser débiles, redundantes o ruidosas. Un modelo sin suficiente control puede

usar esas variables para encontrar patrones aparentes que no se sostienen en datos nuevos.

Tener muchas variables ofrece más posibilidades de encontrar patrones reales, pero también aumenta el riesgo de capturar ruido.

Esto se vuelve especialmente importante cuando la cantidad de variables es grande en relación con la cantidad de filas. Si tenemos pocos datos y muchas columnas, el modelo puede encontrar combinaciones que parecen explicar muy bien el entrenamiento, aunque en realidad sean coincidencias.

Regularizar ayuda a reducir ese riesgo porque desalienta soluciones demasiado complicadas. En lugar de permitir que el modelo use coeficientes enormes o reglas muy específicas, introduce una presión hacia soluciones más moderadas.

Podemos pensarlo como una negociación entre dos objetivos:

Concepto	Descripción
Objetivo 1	ajustar bien los datos de entrenamiento.
Objetivo 2	mantener el modelo bajo control para generalizar mejor.

Sin regularización, el primer objetivo domina. El modelo intenta reducir el error tanto como pueda. Con regularización, agregamos el segundo objetivo. El modelo sigue intentando predecir bien, pero también recibe una penalización si la solución se vuelve demasiado compleja.

Esta tensión es fundamental. Si regularizamos demasiado poco, el modelo puede seguir sobreajustando. Si regularizamos demasiado, el modelo puede volverse demasiado simple y subajustar.

Concepto	Descripción
Poca regularización	riesgo de sobreajuste.
Mucha regularización	riesgo de subajuste.
Regularización adecuada	mejor equilibrio.

Por eso, la intensidad de la regularización suele ser un hiperparámetro. No se elige automáticamente de manera universal. Debe ajustarse según los datos, el modelo y la métrica del problema. En la práctica, se puede evaluar mediante validación cruzada, como vimos en los capítulos anteriores.

Esta conexión es importante: primero aprendimos a evaluar con más estabilidad; luego aprendimos a ajustar hiperparámetros; ahora vemos que muchos hiperparámetros sirven justamente para controlar la complejidad del modelo.

Concepto	Descripción
Evaluación	nos dice cómo generaliza el modelo.
Ajuste de hiperparámetros	nos permite comparar configuraciones.
Regularización	usa esas configuraciones para controlar la complejidad.

También es importante evitar una interpretación equivocada: regularizar no es una solución mágica. Si los datos son malos, si hay fuga de datos, si la variable objetivo está mal definida o si las variables predictoras no contienen señal, la regularización no resolverá el problema de fondo. Puede mejorar la estabilidad, pero no inventa información útil.

La regularización ayuda a controlar el modelo. No corrige por sí sola datos mal definidos, fuga de datos o falta de señal.

Tampoco debemos pensar que un modelo regularizado siempre tendrá mejores métricas en entrenamiento. De hecho, muchas veces ocurre lo contrario: el desempeño en entrenamiento puede bajar un poco. Pero si el desempeño en prueba o validación mejora, esa pérdida es aceptable.

Esto es coherente con la idea de generalización. No buscamos que el modelo sea perfecto con lo conocido; buscamos que sea confiable con lo nuevo.

Por ejemplo:

Concepto	Descripción
Modelo sin regularización	muy buen resultado en entrenamiento, peor resultado en validación.
Modelo regularizado	resultado algo menor en entrenamiento, mejor resultado en validación.

En ese caso, la regularización está cumpliendo su función. Está sacrificando parte del ajuste a los datos conocidos para mejorar el comportamiento ante datos no vistos.

La regularización también puede favorecer modelos más interpretables. En algunos casos, especialmente con ciertos tipos de regularización, los coeficientes pueden volverse más pequeños o incluso quedar en cero. Esto puede ayudar a identificar variables menos relevantes, aunque no debe confundirse automáticamente con una explicación causal.

Más adelante veremos dos formas muy conocidas de regularización en modelos lineales: Ridge y Lasso. Ambas penalizan los coeficientes, pero lo hacen de maneras distintas. Ridge tiende a reducir el tamaño de los coeficientes. Lasso puede llevar algunos coeficientes exactamente a cero, lo que puede funcionar como una forma de selección de variables.

Por ahora, lo importante es comprender la motivación general.

Por lo tanto, la regularización no es un modelo específico, sino una estrategia para controlar la complejidad.

Controlar la complejidad del modelo es necesario porque un modelo demasiado flexible puede aprender detalles accidentales del entrenamiento y fallar en datos nuevos. La regularización introduce restricciones o penalizaciones que desalientan soluciones excesivamente complejas. Su objetivo es mejorar la generalización, no obtener el menor error posible en entrenamiento. Como toda decisión de modelado, debe ajustarse y evaluarse con cuidado, usando validación cruzada y métricas adecuadas para el problema.

Regularización L1 y L2

En los modelos lineales, la regularización suele aplicarse penalizando el tamaño de los coeficientes. La idea es que el modelo no solo busque reducir el error de predicción, sino que también evite construir una solución con coeficientes demasiado grandes o innecesariamente complejos.

Recordemos que, en una regresión lineal, el modelo combina las variables predictoras usando coeficientes:

```
predicción = coeficiente_1 × variable_1 + coeficiente_2 × variable_2 +  
coeficiente_3 × variable_3 + intercepto
```

Los coeficientes indican cuánto peso tiene cada variable dentro de la predicción. Si algunos coeficientes son muy grandes, el modelo puede volverse muy sensible a pequeñas variaciones en los datos. Eso puede aumentar el riesgo de sobreajuste, especialmente cuando hay muchas variables, variables correlacionadas o ruido.

La regularización introduce una penalización asociada a esos coeficientes. En lugar de elegir solamente la solución que minimiza el error, el modelo busca una solución que equilibre dos cosas:

La regularización busca combinar un buen ajuste a los datos con coeficientes bajo control.

Las dos formas más conocidas de regularización en modelos lineales son **L1** y **L2**.

La regularización **L2** penaliza el tamaño de los coeficientes de una manera que tiende a reducirlos, pero normalmente no los lleva exactamente a cero. En `scikit-learn`, una regresión lineal con regularización L2 se conoce como **Ridge Regression**.

La regularización L2 reduce el tamaño de los coeficientes, pero suele mantenerlos distintos de cero.

Esto significa que Ridge puede hacer que el modelo sea más moderado. Las variables siguen participando, pero con pesos más controlados. Si una variable tenía un coeficiente muy grande, la regularización puede empujarlo hacia un valor menor.

Por ejemplo, imaginemos un modelo de precio de viviendas con muchas variables: superficie, cantidad de ambientes, antigüedad, distancia al centro, barrio codificado, servicios cercanos, tipo de propiedad y otras características. Sin regularización, algunos coeficientes podrían crecer mucho para ajustarse mejor a los datos de entrenamiento. Con Ridge, el modelo tiende a distribuir los pesos de manera más controlada.

Ridge resulta útil cuando queremos reducir coeficientes grandes sin eliminar variables por completo.

La regularización **L1**, en cambio, tiene un comportamiento particular: puede llevar algunos coeficientes exactamente a cero. En `scikit-learn`, una regresión lineal con regularización L1 se conoce como **Lasso Regression**.

La regularización L1 puede reducir algunos coeficientes hasta dejarlos exactamente en cero.

Cuando un coeficiente queda en cero, esa variable deja de influir en la predicción. Por eso, Lasso puede funcionar como una forma de selección de variables. No solo controla la complejidad del modelo, sino que también puede descartar variables que no aportan suficiente información según la regularización aplicada.

Lasso puede simplificar el modelo dejando algunas variables sin efecto.

Esto puede ser útil cuando tenemos muchas variables y sospechamos que solo algunas son realmente importantes. Si varias columnas aportan poca señal o son redundantes, Lasso puede reducir su influencia hasta eliminarlas del modelo.

Sin embargo, debemos interpretar esto con cuidado. Que Lasso deje una variable en cero no significa necesariamente que esa variable “no tenga importancia” en sentido

absoluto. Puede ocurrir que esté correlacionada con otras variables, que su aporte dependa de transformaciones no incluidas o que el resultado cambie con otra muestra de datos.

Coefficiente cero: la variable no fue usada por ese modelo con esa configuración y esos datos.

No es una conclusión definitiva sobre el mundo real.

La diferencia entre L1 y L2 puede resumirse así:

Concepto	Descripción
L2 / Ridge	reduce coeficientes, mantiene muchas variables activas.
L1 / Lasso	reduce coeficientes, y puede dejar algunos en cero.

Ambas técnicas buscan controlar la complejidad, pero lo hacen con efectos distintos. Ridge suele ser una buena opción cuando creemos que muchas variables pueden aportar algo de información y queremos evitar coeficientes exagerados. Lasso puede ser útil cuando queremos un modelo más compacto o cuando hay muchas variables y esperamos que solo algunas sean relevantes.

También existe una combinación de ambas, llamada **Elastic Net**, que mezcla regularización L1 y L2. No necesitamos profundizar en ella ahora, pero conviene saber que existe porque, en algunos problemas, combinar ambos efectos puede ser útil.

La intensidad de la regularización se controla mediante un hiperparámetro. En modelos como Ridge y Lasso, ese hiperparámetro suele llamarse **alpha**.

Concepto	Descripción
alpha pequeño	regularización débil.
alpha grande	regularización fuerte.

Si **alpha** es muy pequeño, la regularización tiene poco efecto. El modelo se parece más a una regresión lineal común. Esto puede ser adecuado si el modelo no está sobreajustando, pero puede ser insuficiente si hay demasiada flexibilidad.

Si **alpha** es muy grande, la regularización se vuelve muy fuerte. Los coeficientes se reducen mucho y el modelo puede perder capacidad para representar relaciones importantes. En ese caso, puede aparecer subajuste.

Concepto	Descripción
Regularización demasiado débil	puede no controlar el sobreajuste.
Regularización demasiado fuerte	puede producir subajuste.

Por eso, `alpha` debe elegirse con cuidado. No hay un valor universal que sirva para todos los datasets. Como vimos en el capítulo anterior, este tipo de decisión puede tratarse como un ajuste de hiperparámetros mediante validación cruzada.

Por ejemplo, podríamos probar varios valores de `alpha`:

Por ejemplo, podemos probar los valores `alpha = 0.001`, `alpha = 0.01`, `alpha = 0.1`, `alpha = 1`, `alpha = 10`.

Cada valor produce una versión distinta del modelo. Luego evaluamos cuál ofrece mejor desempeño en validación cruzada, sin usar el conjunto de prueba final para elegir.

En `scikit-learn`, Ridge puede usarse así:

```
from sklearn.linear_model import Ridge

modelo = Ridge(alpha=1.0)
```

Y Lasso puede usarse así:

```
from sklearn.linear_model import Lasso

modelo = Lasso(alpha=1.0)
```

Estos ejemplos no buscan reemplazar el trabajo práctico del cuaderno, sino mostrar que la regularización aparece como una configuración concreta del modelo. El valor de `alpha` no se aprende solo: lo definimos nosotros o lo buscamos mediante validación cruzada.

También debemos recordar que la interpretación de los coeficientes depende de la escala de las variables. Si una variable está medida en metros, otra en años y otra en pesos, sus coeficientes no son directamente comparables. Además, la regularización se ve afectada por la escala. Por eso, en muchos modelos lineales regularizados es importante escalar las variables numéricas.

En modelos lineales regularizados, el escalado suele ser importante para que la penalización sea comparable entre variables.

Si una variable tiene valores muy grandes y otra valores pequeños, la penalización puede afectar sus coeficientes de manera desigual. Usar un `StandardScaler` dentro de un pipeline ayuda a ordenar este flujo correctamente y evita ajustar el escalado con datos que no corresponden.

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import Ridge

pipeline = Pipeline(steps=[
    ("scaler", StandardScaler()),
    ("modelo", Ridge(alpha=1.0))
])
```

En una validación cruzada, este pipeline asegura que el escalador se ajuste dentro de cada fold, usando solo la parte de entrenamiento correspondiente. Así evitamos fuga de datos durante el preprocesamiento.

En problemas con variables categóricas, también puede haber muchas columnas generadas por One-Hot Encoding. En esos casos, la regularización puede ser especialmente útil porque el modelo puede terminar con una gran cantidad de coeficientes. L1 puede dejar algunos en cero, mientras que L2 puede reducirlos de manera más gradual.

Sin embargo, regularizar no significa que ya no debemos pensar en las variables. Si incluimos columnas inválidas, variables con fuga de datos o información posterior al momento de predicción, la regularización no corrige el problema. Puede reducir algunos pesos, pero el diseño del dataset sigue siendo responsabilidad nuestra.

Concepto	Descripción
Regularización	controla coeficientes.
No reemplaza	buena selección de variables, prevención de fuga de datos ni comprensión del problema.

También puede ocurrir que una regularización fuerte mejore una métrica, pero vuelva el modelo demasiado simple para algunos segmentos. Por eso, además del promedio de validación, conviene revisar errores, estabilidad y sentido de las predicciones.

La regularización debe evaluarse como parte de un flujo completo. No es una receta automática. Sirve para controlar la complejidad, pero su efecto depende del modelo, de las variables, de la escala, del ruido, de la cantidad de datos y de la métrica que estamos optimizando.

L1 y L2 son dos formas comunes de regularización en modelos lineales. L2, usada en Ridge, reduce el tamaño de los coeficientes y suele mantener todas las variables activas. L1, usada en Lasso, también reduce coeficientes, pero puede llevar algunos exactamente a cero, funcionando como una forma de selección de variables. La intensidad de la regularización se controla mediante hiperparámetros como `alpha`, que deben elegirse con validación cruzada y no usando el conjunto de prueba final. Ambas técnicas buscan el mismo objetivo general: construir modelos menos propensos a sobreajustar y más capaces de generalizar.

Ridge y Lasso

Ridge y Lasso son dos modelos lineales regularizados. Ambos parten de la idea de la regresión lineal, pero agregan una penalización para controlar el tamaño de los coeficientes. Esa penalización modifica la forma en que el modelo aprende y puede ayudar a reducir el sobreajuste.

En una regresión lineal común, el modelo busca los coeficientes que reducen el error de predicción sobre los datos de entrenamiento. Si los datos tienen muchas variables, ruido o columnas relacionadas entre sí, esos coeficientes pueden volverse demasiado grandes o inestables.

Ridge y Lasso agregan una restricción adicional:

No alcanza con ajustar bien los datos.

También hay que mantener los coeficientes bajo control.

La diferencia principal está en el tipo de penalización que usan.

Ridge aplica regularización L2. Esto tiende a reducir el tamaño de los coeficientes, pero generalmente no los lleva exactamente a cero. Las variables siguen participando del modelo, aunque con pesos más moderados.

Ridge reduce los coeficientes grandes, pero suele conservar todas las variables.

Lasso aplica regularización L1. Esta penalización también reduce coeficientes, pero tiene una característica especial: puede hacer que algunos queden exactamente en cero. Cuando eso ocurre, esas variables dejan de influir en la predicción.

Lasso puede reducir algunos coeficientes a cero y dejar variables fuera del modelo.

Por eso, Ridge y Lasso suelen usarse con intenciones algo distintas.

Ridge es útil cuando creemos que muchas variables pueden aportar algo de información, pero queremos evitar que algunas tengan pesos excesivos. En lugar de eliminar variables, las mantiene activas y reduce su influencia cuando corresponde.

Por ejemplo, en un modelo para predecir precios de viviendas, podríamos tener muchas variables relacionadas: superficie, cantidad de ambientes, cantidad de dormitorios, metros cubiertos, metros totales, antigüedad, ubicación, cercanía a servicios y otras características. Algunas de estas variables pueden estar correlacionadas entre sí. Ridge suele manejar razonablemente bien esa situación porque distribuye los pesos de manera más moderada.

Ridge suele ser útil cuando hay muchas variables relacionadas y no queremos eliminar columnas por completo.

Lasso, en cambio, puede ser útil cuando tenemos muchas variables y sospechamos que solo algunas son realmente relevantes para la predicción. Al llevar ciertos coeficientes a cero, puede producir un modelo más compacto.

Por ejemplo, si luego de una codificación One-Hot tenemos muchas columnas categóricas, Lasso puede dejar algunas sin efecto. Esto puede facilitar la interpretación, aunque siempre con cuidado: una variable con coeficiente cero no queda descartada para siempre ni se demuestra irrelevante en sentido absoluto. Solo significa que ese modelo, con esos datos y ese valor de regularización, no la usó.

Lasso puede simplificar el modelo, pero sus coeficientes deben interpretarse dentro del contexto del entrenamiento.

Ambos modelos tienen un hiperparámetro clave: `alpha`. Este valor controla la intensidad de la regularización.

Concepto	Descripción
alpha pequeño	regularización débil.
alpha grande	regularización fuerte.

Si `alpha` es muy bajo, Ridge o Lasso se parecen más a una regresión lineal común. Los coeficientes tienen más libertad para ajustarse a los datos. Esto puede funcionar bien si no hay sobreajuste, pero puede ser insuficiente si el modelo es demasiado flexible para el dataset.

Si `alpha` es muy alto, la regularización se vuelve fuerte. Los coeficientes se reducen mucho. En Ridge, tenderán a hacerse pequeños. En Lasso, varios pueden quedar en cero. Si la regularización es excesiva, el modelo puede perder capacidad predictiva y subajustar.

Concepto	Descripción
alpha demasiado bajo	puede no controlar el sobreajuste.
alpha demasiado alto	puede simplificar demasiado el modelo.

Por eso, `alpha` debe elegirse mediante evaluación. No hay un valor universal. Lo correcto es probar distintos valores con validación cruzada y elegir una configuración adecuada según la métrica del problema.

Por ejemplo:

Por ejemplo, podemos probar los valores `alpha = 0.001`, `alpha = 0.01`, `alpha = 0.1`, `alpha = 1`, `alpha = 10`.

Cada valor produce un modelo distinto. En una búsqueda de hiperparámetros, podríamos comparar esos valores usando validación cruzada y observar cuál ofrece mejor equilibrio entre error, estabilidad y simplicidad.

En `scikit-learn`, Ridge se importa desde `sklearn.linear_model`:

```
from sklearn.linear_model import Ridge  
  
modelo = Ridge(alpha=1.0)
```

Lasso se usa de forma similar:

```
from sklearn.linear_model import Lasso  
  
modelo = Lasso(alpha=1.0)
```

Estos modelos pueden evaluarse como cualquier otro estimador de `scikit-learn`, usando `fit`, `predict`, validación cruzada o búsqueda de hiperparámetros. Su diferencia está en la forma en que aprenden los coeficientes.

Una cuestión importante es el escalado. En modelos lineales regularizados, la penalización se aplica sobre los coeficientes. Si las variables están en escalas muy distintas, la regularización puede afectar de manera desigual el aprendizaje.

Por ejemplo, una variable medida en pesos puede tener valores muy grandes, mientras que una variable como cantidad de ambientes puede estar entre 1 y 6. Si no escalamos, los coeficientes no son comparables de forma directa y la penalización puede comportarse de manera poco conveniente.

En Ridge y Lasso, el escalado de variables numéricas suele ser importante.

Por eso, es común usar `StandardScaler` dentro de un pipeline:

```

from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import Ridge

pipeline = Pipeline(steps=[
    ("scaler", StandardScaler()),
    ("modelo", Ridge(alpha=1.0))
])

```

Si usamos validación cruzada, este pipeline asegura que el escalado se ajuste correctamente dentro de cada fold, usando solo la parte de entrenamiento correspondiente. Esto evita fuga de datos.

También podemos ajustar `alpha` con `GridSearchCV`:

```

from sklearn.model_selection import GridSearchCV

param_grid = {
    "modelo__alpha": [0.001, 0.01, 0.1, 1, 10]
}

busqueda = GridSearchCV(
    estimator=pipeline,
    param_grid=param_grid,
    cv=5,
    scoring="neg_mean_absolute_error"
)

```

En este ejemplo, `modelo__alpha` indica que queremos ajustar el hiperparámetro `alpha` del paso llamado `"modelo"` dentro del pipeline.

Luego la búsqueda se ajusta con los datos de entrenamiento:

```

busqueda.fit(X_train, y_train)

```

Como siempre, el conjunto de prueba final no debe usarse para elegir `alpha`. Se reserva para evaluar el modelo ya elegido.

Ridge y Lasso pueden aplicarse en regresión, pero también existen modelos de clasificación que incorporan regularización. Por ejemplo, la regresión logística puede usar penalizaciones L1 o L2. En ese caso, el objetivo no es predecir un valor numérico continuo, sino clasificar, pero la idea de controlar coeficientes sigue siendo similar.

No necesitamos profundizar ahora en todas las variantes. Lo importante es reconocer que la regularización no pertenece a un único modelo. Es una idea general que aparece en distintas formas dentro de varios algoritmos.

Ridge y Lasso son ejemplos claros de regularización en modelos lineales.

También es importante no exagerar lo que estos modelos pueden hacer. Ridge y Lasso siguen siendo modelos lineales. La regularización controla coeficientes, pero no transforma automáticamente una relación no lineal en una relación lineal adecuada. Si el problema requiere representar interacciones complejas o patrones no lineales, la regularización por sí sola puede no alcanzar.

Regularizar un modelo lineal no lo convierte en un modelo capaz de aprender cualquier tipo de relación.

Por eso, si Ridge o Lasso tienen mal desempeño tanto en entrenamiento como en validación, tal vez el problema no sea la falta de regularización, sino que el modelo lineal es demasiado simple para la relación que intentamos representar. En ese caso, podríamos estar frente a un problema de subajuste.

En cambio, si una regresión lineal común funciona muy bien en entrenamiento pero peor en validación, Ridge o Lasso pueden ayudar a controlar el sobreajuste.

La comparación debe hacerse con métricas y validación cruzada. No deberíamos elegir Ridge, Lasso o regresión lineal común solo por preferencia. Debemos evaluar cuál funciona mejor para el problema y también considerar interpretabilidad, estabilidad y sentido de los coeficientes.

La elección entre regresión lineal, Ridge y Lasso debe basarse en evaluación, no en una regla fija.

En algunos problemas, Ridge puede ofrecer mejor desempeño porque conserva información distribuida entre muchas variables. En otros, Lasso puede ser atractivo porque produce modelos más simples al dejar algunos coeficientes en cero. En otros casos, una regresión lineal sin regularización puede ser suficiente.

Ridge y Lasso son dos formas de regresión lineal regularizada. Ridge usa regularización L2 y tiende a reducir coeficientes sin eliminar variables. Lasso usa regularización L1 y puede llevar algunos coeficientes exactamente a cero, funcionando como una forma de selección de variables. Ambos modelos dependen de la intensidad de regularización, controlada por `alpha`, y deben evaluarse mediante validación cruzada dentro de un pipeline cuando haya preprocesamiento. Su objetivo no es obtener el menor error posible en entrenamiento, sino mejorar la generalización controlando la complejidad del modelo.

Regularización en regresión logística

Aunque venimos presentando Ridge y Lasso desde la regresión lineal, la regularización también aparece en modelos de clasificación. Un caso muy importante es la regresión logística.

Recordemos que la regresión logística se usa para clasificar. En problemas binarios, el modelo estima la probabilidad de pertenecer a la clase positiva y luego aplica un umbral para convertir esa probabilidad en una clase.

La regresión logística estima probabilidades y luego clasifica según un umbral.

Aunque el objetivo sea clasificar, la regresión logística también aprende coeficientes. Cada variable predictora tiene un peso dentro del modelo. Esos coeficientes influyen en la probabilidad estimada para la clase positiva.

Si una variable tiene un coeficiente positivo, tiende a aumentar la probabilidad de la clase positiva. Si tiene un coeficiente negativo, tiende a disminuirla. La magnitud del coeficiente indica cuánta influencia tiene esa variable dentro del modelo, siempre con las precauciones que ya mencionamos: escala, correlación entre variables, codificación y contexto.

Los coeficientes de la regresión logística controlan cómo las variables modifican la probabilidad estimada.

Como ocurre en la regresión lineal, esos coeficientes pueden volverse demasiado grandes o inestables si el modelo intenta ajustarse demasiado a los datos de entrenamiento. Esto puede aumentar el riesgo de sobreajuste, especialmente cuando hay muchas variables, variables redundantes, columnas generadas por One-Hot Encoding o pocos datos en relación con la cantidad de características.

La regularización ayuda a controlar ese comportamiento. En lugar de permitir que los coeficientes crezcan sin restricción, la regresión logística regularizada penaliza soluciones con coeficientes demasiado grandes.

La regresión logística regularizada clasifica estimando probabilidades, pero controla el tamaño de sus coeficientes.

En `scikit-learn`, la regresión logística incluye regularización por defecto. Esto es importante: cuando usamos `LogisticRegression`, no estamos usando una versión completamente “libre” del modelo, sino una versión que ya incorpora una penalización, salvo que indiquemos lo contrario.

La penalización más habitual es L2. Como vimos antes, L2 tiende a reducir el tamaño de los coeficientes sin llevarlos normalmente a cero. En regresión logística, esto ayuda a que el modelo sea más estable y menos sensible a detalles particulares del entrenamiento.

En la regresión logística, la regularización L2 reduce los coeficientes grandes y favorece modelos más estables.

También puede usarse regularización L1 en regresión logística. En ese caso, algunos coeficientes pueden quedar exactamente en cero. Esto significa que ciertas variables dejan de influir en la predicción del modelo.

En la regresión logística, la regularización L1 puede dejar algunos coeficientes en cero y simplificar el modelo.

Esto puede ser útil cuando tenemos muchas variables y sospechamos que solo algunas aportan señal relevante. Por ejemplo, después de aplicar One-Hot Encoding a variables categóricas con muchas categorías, podemos terminar con una gran cantidad de columnas. La regularización L1 puede ayudar a que el modelo use solo una parte de ellas.

Pero, como antes, hay que interpretar esto con prudencia. Que un coeficiente quede en cero no significa que esa variable sea inútil en cualquier contexto. Solo significa que ese modelo, con esos datos, esa preparación y esa intensidad de regularización, no la necesitó para mejorar la métrica elegida.

En `scikit-learn`, la intensidad de la regularización en `LogisticRegression` suele controlarse con el hiperparámetro `C`.

Aquí aparece una diferencia importante respecto de Ridge y Lasso. En esos modelos hablábamos de `alpha`: valores más grandes de `alpha` significan más regularización. En regresión logística de `scikit-learn`, `C` funciona al revés.

En `LogisticRegression`, un valor pequeño de `C` implica una regularización más fuerte, mientras que un valor grande de `C` implica una regularización más débil.

Esto puede resultar poco intuitivo al principio. Para recordarlo, podemos pensar que `C` controla cuánta libertad le damos al modelo. Si `C` es grande, el modelo tiene más libertad para ajustar los coeficientes. Si `C` es pequeño, la penalización pesa más y los coeficientes quedan más controlados.

Concepto	Descripción
C grande	más libertad, más riesgo de sobreajuste.

Concepto	Descripción
C pequeño	más control, más riesgo de subajuste si es excesivo.

Por ejemplo, podríamos comparar valores como:

Por ejemplo, podemos probar los valores `C = 0.01`, `C = 0.1`, `C = 1`, `C = 10`, `C = 100`.

Cada valor produce una versión distinta de la regresión logística. No hay un valor universalmente mejor. Debe evaluarse mediante validación cruzada, usando una métrica adecuada para el problema.

Si trabajamos con clasificación desbalanceada, por ejemplo, no conviene ajustar `C` mirando solo accuracy sin revisar si el modelo está detectando correctamente la clase minoritaria. En esos casos, puede ser más útil evaluar F1-score, recall, precision u otra métrica alineada con el objetivo.

La regularización se ajusta con una métrica.

Si la métrica no representa el problema, la configuración elegida puede no ser adecuada.

Un ejemplo básico de regresión logística con regularización L2 sería:

```
from sklearn.linear_model import LogisticRegression

modelo = LogisticRegression(
    penalty="l2",
    C=1.0,
    max_iter=1000
)
```

Aquí `penalty="l2"` indica el tipo de regularización. `C=1.0` define su intensidad. `max_iter=1000` aumenta la cantidad máxima de iteraciones, algo que puede ser necesario para que el algoritmo termine correctamente en algunos datasets.

Para usar L1, podemos escribir:

```
from sklearn.linear_model import LogisticRegression

modelo = LogisticRegression(
    penalty="l1",
    C=1.0,
    solver="liblinear",
    max_iter=1000
)
```

El parámetro `solver` indica el método de optimización usado por el modelo. No todos los solvers aceptan todas las penalizaciones. Por eso, cuando usamos L1, en ejemplos iniciales puede aparecer `solver="liblinear"` u otro solver compatible.

No necesitamos profundizar ahora en los detalles internos de los solvers. Lo importante es saber que la regularización forma parte de la configuración del modelo y que algunas combinaciones de parámetros deben ser compatibles.

Concepto	Descripción
penalty	tipo de regularización.
C	intensidad de la regularización.
solver	método que usa el modelo para ajustar los coeficientes.

Como ocurre con Ridge y Lasso, el escalado de variables suele ser importante. La regularización penaliza coeficientes, y los coeficientes dependen de la escala de las variables. Si una variable está medida en unidades muy grandes y otra en unidades pequeñas, la penalización puede afectar el modelo de manera poco equilibrada.

Por eso, es común usar `StandardScaler` dentro de un pipeline:

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

pipeline = Pipeline(steps=[
    ("scaler", StandardScaler()),
    ("modelo", LogisticRegression(
        penalty="l2",
        C=1.0,
        max_iter=1000
    ))
])
```

Si además queremos ajustar `C` con validación cruzada, podemos usar `GridSearchCV`:

```
from sklearn.model_selection import GridSearchCV

param_grid = {
    "modelo__C": [0.01, 0.1, 1, 10, 100]
}

busqueda = GridSearchCV(
    estimator=pipeline,
    param_grid=param_grid,
    cv=5,
    scoring="f1"
)
```

Luego se ajusta la búsqueda con los datos de entrenamiento:

```
busqueda.fit(X_train, y_train)
```

Este flujo evalúa distintos valores de `C` mediante validación cruzada. En cada fold, el escalado se ajusta solo con la parte de entrenamiento correspondiente, y la regresión logística se entrena con ese mismo fold de entrenamiento. Esto evita que la validación reciba información indebida del fold que debería permanecer no visto.

La combinación de un pipeline con `GridSearchCV` permite ajustar la regularización sin romper la lógica de validación.

Una vez elegida la mejor configuración, podemos evaluar el modelo final en el conjunto de prueba reservado. Como ya vimos, el conjunto de prueba no debe usarse para elegir `C`, el tipo de penalización o cualquier otra configuración.

La regularización también afecta la interpretación de los coeficientes. Si usamos una regularización fuerte, los coeficientes pueden quedar más pequeños. Esto puede hacer que el modelo sea más estable, pero también puede reducir la influencia de variables que sí tenían señal. Si usamos L1, algunas variables pueden quedar en cero. Esto puede simplificar el modelo, pero no debe interpretarse como una verdad definitiva sobre la importancia de esas variables.

La regularización cambia los coeficientes. Por eso, también cambia su interpretación.

En problemas donde necesitamos explicar el modelo, conviene tener presente qué tipo de regularización se usó y con qué intensidad. No es lo mismo interpretar coeficientes de una regresión logística con regularización débil que con regularización fuerte.

También es importante recordar que la regularización no resuelve todos los problemas. Si hay fuga de datos, variables posteriores al momento de predicción, etiquetas mal definidas o una métrica inadecuada, regularizar no convierte automáticamente al modelo en confiable. Puede controlar los coeficientes, pero no corrige un mal diseño del dataset.

Regularizar ayuda a controlar el modelo, pero no reemplaza una evaluación bien diseñada.

La regresión logística también puede regularizarse. Aunque se usa para clasificación, aprende coeficientes igual que otros modelos lineales. La regularización L2 reduce coeficientes grandes y favorece estabilidad; la regularización L1 puede llevar algunos coeficientes a cero y simplificar el modelo. En `scikit-learn`, la intensidad se controla con `C`, donde valores pequeños significan regularización más fuerte y valores grandes significan regularización más débil. Como siempre, estas decisiones

deben tomarse con validación cruzada, pipelines completos y una métrica coherente con el problema.

¿Cuándo ayuda y cuándo puede perjudicar?

La regularización es una herramienta muy útil para controlar la complejidad de un modelo, pero no siempre produce una mejora automática. Como ocurre con muchas decisiones en Machine Learning, su efecto depende del problema, de los datos, del modelo y de la intensidad con que se aplique.

Regularizar ayuda especialmente cuando el modelo tiene riesgo de sobreajuste. Esto puede ocurrir cuando hay muchas variables, cuando el dataset no es muy grande, cuando existen variables correlacionadas, cuando hay ruido en los datos o cuando el modelo tiene demasiada libertad para adaptarse a los detalles del entrenamiento.

La regularización suele ayudar cuando el modelo es demasiado flexible, hay muchas variables, hay ruido o el desempeño en entrenamiento es mucho mejor que en validación.

Supongamos que entrenamos un modelo y observamos este comportamiento:

Concepto	Descripción
Entrenamiento	muy buen desempeño.
Validación	desempeño bastante peor.

Esta diferencia puede ser una señal de sobreajuste. El modelo aprendió demasiado bien los datos de entrenamiento, pero no logra mantener ese rendimiento en datos nuevos. En un caso así, aumentar la regularización puede ayudar porque limita la complejidad y desalienta soluciones demasiado ajustadas al entrenamiento.

En modelos lineales, esto puede significar reducir coeficientes excesivamente grandes. En árboles, puede significar limitar la profundidad o exigir más muestras por hoja. En otros modelos, puede tomar formas diferentes, pero la intención general es la misma: evitar que el modelo dependa demasiado de detalles frágiles.

Cuando hay sobreajuste, controlar la complejidad puede mejorar la generalización.

La regularización también puede ayudar cuando tenemos muchas variables predictoras. Con muchas columnas, el modelo tiene más oportunidades de encontrar patrones reales, pero también más oportunidades de encontrar coincidencias accidentales. Si algunas variables aportan poca señal o son redundantes, una regularización adecuada puede reducir su influencia.

Por ejemplo, después de aplicar One-Hot Encoding a varias variables categóricas, un dataset puede pasar de tener pocas columnas a tener muchas. En ese contexto, un modelo lineal sin regularización puede volverse más sensible a detalles específicos del entrenamiento. Ridge puede moderar los coeficientes. Lasso puede incluso dejar algunos en cero.

Concepto	Descripción
Muchas columnas	más información posible, pero también más riesgo de ruido.
Regularización	ayuda a controlar ese riesgo.

También puede ser útil cuando las variables están correlacionadas. Si dos o más variables contienen información parecida, un modelo lineal puede distribuir los coeficientes de manera inestable. Pequeños cambios en los datos pueden modificar bastante qué variable recibe más peso. Ridge suele ser útil en estos casos porque reduce coeficientes grandes y tiende a producir soluciones más estables.

Por ejemplo, en un modelo de viviendas, `superficie_total`, `superficie_cubierta` y `cantidad_ambientes` pueden estar relacionadas. No significan exactamente lo mismo, pero contienen información parcialmente solapada. La regularización puede ayudar a evitar que el modelo exagere el peso de alguna de ellas por una particularidad del entrenamiento.

Concepto	Descripción
Variables correlacionadas	pueden volver inestables los coeficientes.
Regularización	puede ayudar a moderar esa inestabilidad.

Sin embargo, la regularización también puede perjudicar. Si se aplica con demasiada intensidad, el modelo puede volverse demasiado simple. En ese caso, deja de capturar patrones reales y aparece subajuste.

Una regularización demasiado fuerte puede producir coeficientes demasiado pequeños, limitar en exceso el modelo y aumentar el riesgo de subajuste.

Supongamos que un modelo lineal regularizado reduce demasiado sus coeficientes. Las predicciones pueden acercarse excesivamente a un valor promedio y perder sensibilidad frente a las variables de entrada. En regresión, esto puede producir predicciones demasiado conservadoras. En clasificación, puede generar probabilidades menos diferenciadas entre casos.

Por ejemplo, si queremos predecir precios de viviendas y la regularización es excesiva, el modelo puede dejar de reflejar suficientemente diferencias importantes

entre viviendas grandes y pequeñas, entre ubicaciones distintas o entre propiedades nuevas y antiguas. El resultado será un modelo más simple, pero también menos útil.

Regularizar no significa aplanar el modelo hasta que deje de aprender.

La regularización también puede perjudicar si el problema ya era de subajuste. Si un modelo es demasiado simple y además lo regularizamos fuertemente, lo estamos limitando todavía más. En ese caso, no resolvemos el problema: lo empeoramos.

Supongamos que entrenamos un modelo y obtenemos:

Concepto	Descripción
Entrenamiento	mal desempeño.
Validación	mal desempeño.

Esto sugiere que el modelo no está aprendiendo lo suficiente. Tal vez faltan variables importantes, la relación no es lineal, el modelo es demasiado simple o el problema está mal formulado. Agregar más regularización probablemente no ayude, porque el problema no es exceso de complejidad, sino falta de capacidad o falta de señal.

Si hay subajuste, más regularización suele ser mala idea.

En estos casos, podría ser mejor agregar variables relevantes, transformar algunas variables, usar un modelo más flexible o revisar la formulación del problema. La regularización no reemplaza esas decisiones.

También debemos tener cuidado con la interpretación de modelos regularizados. Si usamos Lasso y algunas variables quedan con coeficiente cero, puede ser tentador concluir que esas variables “no importan”. Pero esa conclusión puede ser demasiado fuerte. El resultado depende del valor de regularización, de las otras variables presentes, de la escala, de la muestra de datos y de la métrica optimizada.

Una variable con coeficiente cero no queda descartada para siempre.

Solo quedó fuera en ese modelo específico.

Esto es especialmente importante cuando hay variables correlacionadas. Lasso puede elegir una variable y dejar otra en cero, aunque ambas contengan información parecida. Con otra muestra o con otro valor de regularización, la selección podría cambiar.

La regularización también puede afectar de manera distinta a distintos grupos de datos. Un modelo más simple y regularizado puede mejorar el promedio general, pero empeorar en un segmento particular. Por eso, no conviene mirar solo la métrica

global. Si el problema lo requiere, también hay que revisar errores por grupos, rangos de valores o clases.

Una mejora promedio puede ocultar empeoramientos en algunos segmentos.

Por ejemplo, en regresión de precios, un modelo regularizado puede mejorar el error promedio, pero predecir peor viviendas de alto valor. En clasificación, puede mejorar F1-score global, pero reducir recall en una clase minoritaria. La regularización debe evaluarse no solo por la métrica principal, sino también por el tipo de errores que produce.

Otra situación en la que la regularización no alcanza es cuando hay fuga de datos. Si el modelo recibe una variable que revela la respuesta, regularizar puede reducir parcialmente su coeficiente, pero el problema de fondo sigue existiendo. La evaluación seguirá contaminada si esa información indebida permanece en los datos.

Regularización no corrige fuga de datos. La variable inválida debe eliminarse o reconstruirse correctamente.

Lo mismo ocurre con etiquetas mal definidas, errores sistemáticos de carga, datos no representativos o cambios fuertes entre entrenamiento y uso futuro. La regularización puede hacer que el modelo sea más moderado, pero no convierte datos problemáticos en datos confiables.

Por eso, antes de atribuir todo a “falta de regularización”, conviene revisar el flujo completo: definición del problema, calidad de variables, separación de datos, preprocesamiento, métrica y análisis de errores.

La regularización es una parte del flujo. No reemplaza una buena preparación ni una evaluación honesta.

La intensidad adecuada de regularización debe elegirse con validación cruzada. Si probamos varios valores de `alpha` en Ridge o Lasso, o varios valores de `C` en regresión logística, debemos compararlos dentro del conjunto de entrenamiento, sin usar el test final para elegir.

Un patrón posible sería:

Concepto	Descripción
Regularización débil	buen entrenamiento, peor validación.
Regularización intermedia	mejor validación.
Regularización fuerte	bajan entrenamiento y validación.

Ese comportamiento muestra el equilibrio que buscamos. Al principio, regularizar ayuda porque reduce sobreajuste. Pero si seguimos aumentando la penalización, el modelo pierde capacidad y empieza a subajustar.

No siempre los resultados serán tan claros. A veces varias intensidades tienen desempeños muy parecidos. En esos casos, podemos preferir una configuración más simple, más estable o más fácil de interpretar, siempre que la diferencia de métrica sea pequeña.

Si varias configuraciones rinden parecido, conviene considerar estabilidad, simplicidad e interpretabilidad.

También debemos considerar el costo de la regularización en términos de explicación. Un modelo muy regularizado puede ser más estable, pero sus coeficientes reflejan una solución penalizada. Si vamos a interpretar esos coeficientes, debemos aclarar que no son los mismos que obtendríamos en una regresión sin regularización.

Esto no es un problema, pero sí una condición de lectura. La regularización modifica el modelo para favorecer la generalización. Por lo tanto, también modifica la forma en que los pesos quedan distribuidos.

La regularización ayuda cuando el modelo tiende a sobreajustar, cuando hay muchas variables, ruido, coeficientes inestables o alta diferencia entre desempeño de entrenamiento y validación. Puede perjudicar si se aplica con demasiada fuerza o si el problema principal es subajuste. Tampoco corrige fuga de datos, mala calidad de variables ni errores de formulación. Como siempre, debe evaluarse con validación cruzada, métricas adecuadas y análisis de errores. Regularizar bien significa controlar la complejidad sin impedir que el modelo aprenda los patrones que realmente importan.

Regularizar para generalizar mejor

En este capítulo estudiamos la regularización como una estrategia para controlar la complejidad de los modelos. La idea central fue que un modelo no debe limitarse a ajustar lo mejor posible los datos de entrenamiento, sino que debe aprender de una manera que le permita funcionar bien con datos nuevos.

La regularización introduce una restricción o penalización que desalienta soluciones demasiado complejas. En los modelos lineales, esto suele hacerse controlando el tamaño de los coeficientes. En otros modelos, puede aparecer como una limitación de profundidad, una cantidad mínima de muestras o algún mecanismo equivalente para evitar que el modelo se adapte demasiado a detalles accidentales.

En este contexto, regularizar significa controlar la complejidad para reducir el riesgo de sobreajuste.

Vimos que la regularización se conecta directamente con el equilibrio entre sobreajuste y subajuste. Si el modelo tiene demasiada libertad, puede aprender ruido o particularidades del entrenamiento. Si lo restringimos demasiado, puede dejar de aprender patrones importantes.

Concepto	Descripción
Poca regularización	más riesgo de sobreajuste.
Demasiada regularización	más riesgo de subajuste.
Regularización adecuada	mejor equilibrio para generalizar.

En modelos lineales, estudiamos dos formas frecuentes de regularización: L1 y L2. La regularización L2, asociada a Ridge, reduce el tamaño de los coeficientes y suele mantener todas las variables activas. La regularización L1, asociada a Lasso, también reduce coeficientes, pero puede llevar algunos exactamente a cero.

Concepto	Descripción
Ridge / L2	coeficientes más pequeños, variables generalmente activas.
Lasso / L1	coeficientes más pequeños, algunas variables pueden quedar en cero.

Esta diferencia hace que Ridge y Lasso puedan ser útiles en situaciones distintas. Ridge suele ser conveniente cuando varias variables pueden aportar algo de información y queremos evitar pesos exagerados. Lasso puede ser útil cuando hay muchas variables y buscamos un modelo más compacto, aunque sus coeficientes deben interpretarse con cuidado.

También vimos que la intensidad de la regularización se controla mediante hiperparámetros. En Ridge y Lasso usamos `alpha`: valores más grandes implican regularización más fuerte. En la regresión logística de `scikit-learn`, en cambio, aparece el hiperparámetro `C`, que funciona al revés: valores pequeños implican mayor regularización y valores grandes implican menor regularización.

Concepto	Descripción
Ridge y Lasso	<code>alpha</code> grande → más regularización.
LogisticRegression	<code>C</code> pequeño → más regularización.

Esta diferencia es importante porque muestra que no basta con saber que un modelo está regularizado. También debemos entender qué hiperparámetro controla esa regularización y cómo se interpreta.

En regresión logística, la regularización cumple una función similar a la que cumple en los modelos lineales de regresión: controla el tamaño de los coeficientes. Aunque el modelo se use para clasificación, sigue aprendiendo pesos para las variables predictoras. Por eso, la regularización puede ayudar a evitar coeficientes demasiado grandes, mejorar la estabilidad y reducir el riesgo de sobreajuste.

La regresión logística regularizada clasifica estimando probabilidades, pero controla sus coeficientes.

Otro punto importante fue el escalado. Como la regularización penaliza coeficientes, la escala de las variables puede afectar el resultado. Por eso, en muchos modelos lineales regularizados conviene usar `StandardScaler` dentro de un `Pipeline`. De esa manera, el escalado se ajusta correctamente dentro de cada fold durante la validación cruzada.

Concepto	Descripción
Pipeline	preprocesamiento + modelo
Validación cruzada	evalúa el flujo completo sin filtrar información indebida.

La regularización debe evaluarse con el mismo cuidado que cualquier otra decisión de modelado. No se debe elegir la intensidad de regularización mirando el conjunto de prueba final. Lo correcto es probar configuraciones mediante validación cruzada dentro del conjunto de entrenamiento, elegir una opción adecuada y reservar el test para una evaluación final.

También vimos que la regularización no es una solución mágica. Puede ayudar cuando hay sobreajuste, muchas variables, ruido o coeficientes inestables. Pero puede perjudicar si se aplica con demasiada fuerza o si el problema principal es subajuste. Tampoco corrige fuga de datos, mala calidad de variables, etiquetas mal definidas ni una métrica mal elegida.

Regularizar ayuda a controlar el modelo.

No reemplaza: buenos datos, variables válidas, evaluación honesta y una métrica adecuada.

La regularización es una herramienta para mejorar la generalización controlando la complejidad. Ridge, Lasso y la regresión logística regularizada muestran distintas formas de aplicar esta idea en modelos lineales. La intensidad debe elegirse mediante validación cruzada y siempre dentro de un flujo que respete la separación entre entrenamiento, validación y prueba. Regularizar bien no significa hacer el modelo lo más simple posible, sino encontrar un equilibrio: permitir que aprenda patrones útiles sin dejar que memorice detalles que no se sostendrán en datos nuevos.

Capítulo 21 · Random Forest

Ensamblados de modelos

Hasta ahora estudiamos modelos individuales. Un KNN clasifica observando vecinos cercanos. Una regresión logística estima probabilidades a partir de una combinación de variables. Un árbol de decisión toma decisiones mediante una secuencia de preguntas. Cada uno de estos modelos tiene su propia forma de aprender y de producir predicciones.

Un **ensamble** parte de una idea distinta: en lugar de confiar en un único modelo, combina varios modelos para producir una predicción final.

Concepto	Descripción
Modelo individual	una sola predicción.
Ensamble	varios modelos colaboran para producir una predicción final.

La intuición es sencilla. Muchas veces, un conjunto de modelos puede funcionar mejor que un modelo aislado. Si cada modelo comete errores diferentes, la combinación puede reducir algunos de esos errores. En lugar de depender de una única mirada sobre los datos, el ensamble reúne varias miradas.

Podemos pensarlo con una analogía. Si una persona debe tomar una decisión difícil, puede equivocarse por su experiencia limitada, por una interpretación parcial o por un sesgo. Si varias personas analizan el mismo problema de manera independiente y luego se combina su opinión, la decisión final puede ser más robusta. No siempre será perfecta, pero puede ser menos dependiente de un único punto de vista.

En Machine Learning ocurre algo parecido. Un ensamble intenta aprovechar la diversidad entre modelos para mejorar la generalización.

La fuerza de un ensamble no está en que cada modelo sea perfecto, sino en que sus errores no sean todos iguales.

Esta idea es especialmente útil con modelos que pueden variar bastante según los datos con los que se entrenan. Los árboles de decisión son un buen ejemplo. Un árbol individual puede ser muy sensible a pequeñas diferencias en el conjunto de entrenamiento. Si cambiamos algunas filas, si aparece un valor atípico o si una división inicial cambia, el árbol resultante puede ser bastante distinto.

Esa inestabilidad puede ser una limitación cuando usamos un solo árbol. Pero también puede convertirse en una oportunidad: si construimos muchos árboles diferentes y combinamos sus predicciones, podemos obtener un modelo más estable.

Concepto	Descripción
Un árbol individual	puede ser inestable.
Muchos árboles combinados	pueden producir una predicción más robusta.

Random Forest se basa precisamente en esta idea. Su nombre puede traducirse como “bosque aleatorio”. En lugar de construir un único árbol de decisión, construye muchos árboles. Cada árbol aporta una predicción, y el bosque combina esas predicciones para tomar una decisión final.

En clasificación, cada árbol puede votar por una clase. La clase que recibe más votos se convierte en la predicción final del bosque.

Elemento	Resultado
Árbol 1	clase A
Árbol 2	clase B
Árbol 3	clase A
Árbol 4	clase A
Árbol 5	clase B
Predicción final	clase A

En regresión, cada árbol produce un valor numérico. El bosque combina esos valores, generalmente mediante un promedio.

Elemento	Resultado
Árbol 1	120.000
Árbol 2	125.000
Árbol 3	118.000
Árbol 4	123.000
Predicción final	promedio aproximado

La lógica general es la misma: muchos modelos producen predicciones y el ensamble las combina.

Los ensambles pueden construirse de distintas maneras. Algunos entrenan varios modelos de forma independiente y luego combinan sus resultados. Otros entrenan modelos de manera secuencial, donde cada nuevo modelo intenta corregir errores de los anteriores. Random Forest pertenece a una familia de métodos basada en combinar muchos árboles entrenados con cierta diversidad entre ellos.

Por ahora, lo importante es entender la idea general antes de entrar en los detalles.

Random Forest: un ensamble formado por muchos árboles de decisión.

Un punto clave es que no alcanza con repetir el mismo árbol muchas veces. Si todos los árboles fueran idénticos, todos cometerían los mismos errores y el ensamble no aportaría demasiado. Para que el bosque sea útil, los árboles deben ser diferentes entre sí.

Random Forest logra esa diversidad introduciendo aleatoriedad durante la construcción de los árboles. Esa aleatoriedad aparece principalmente de dos formas: cada árbol se entrena con una muestra distinta de los datos, y en cada división del árbol se considera solo una parte de las variables disponibles. Veremos estos mecanismos con más detalle en las próximas secciones.

Un buen ensamble necesita diversidad.

Si todos los modelos son iguales, la combinación aporta poco.

Esta diversidad hace que cada árbol observe una versión algo distinta del problema. Algunos árboles pueden captar ciertos patrones; otros pueden captar patrones diferentes. Algunos pueden equivocarse en ciertos casos; otros pueden corregirlos. Al combinar muchos árboles, el bosque tiende a reducir la dependencia de un único árbol particular.

Esto no significa que Random Forest elimine todos los errores. Tampoco significa que siempre será el mejor modelo. Pero sí suele ser una opción muy potente para datos tabulares, tanto en clasificación como en regresión.

Una de sus ventajas es que puede capturar relaciones no lineales e interacciones entre variables sin exigir que las especifiquemos manualmente. Un árbol ya tenía esa capacidad; Random Forest la conserva, pero agrega mayor estabilidad al combinar muchos árboles.

Concepto	Descripción
Árbol de decisión	flexible, interpretable, pero puede sobreajustar.
Random Forest	combina muchos árboles para mejorar estabilidad y generalización.

También suele requerir menos preparación que otros modelos en algunos aspectos. Por ejemplo, no necesita escalado de variables numéricas del mismo modo que KNN o los modelos lineales regularizados. Una variable medida en miles y otra medida en unidades pequeñas no afectan a los árboles de la misma manera que afectarían a un modelo basado en distancias o coeficientes.

Sin embargo, esto no significa que podamos descuidar el preprocesamiento. Los valores faltantes, las variables categóricas, la fuga de datos, las columnas inválidas y la definición del problema siguen siendo importantes. Random Forest no corrige automáticamente un dataset mal construido.

Un modelo más potente no reemplaza una buena preparación de datos.

Otra ventaja de Random Forest es que puede ofrecer una estimación de la importancia de las variables. Esto permite analizar qué columnas parecen haber sido más útiles para construir las predicciones. Esa información puede ser muy valiosa, aunque debe interpretarse con cuidado. La importancia de variables no equivale automáticamente a causalidad ni garantiza que una variable sea válida desde el punto de vista del problema.

Por ejemplo, si una variable aparece como muy importante, debemos preguntarnos si realmente estaría disponible al momento de predicción. Si esa variable contiene fuga de datos, el bosque también puede aprovecharla y producir una evaluación engañosa.

Importancia de variables: ayuda a interpretar el modelo, pero no reemplaza la revisión del contexto.

Los ensambles también tienen un costo. Un Random Forest suele ser menos interpretable que un único árbol. Un árbol individual puede dibujarse y recorrerse como una secuencia de preguntas. En cambio, un bosque con cientos de árboles es mucho más difícil de explicar paso a paso.

Podemos entender su lógica general, revisar importancias de variables y analizar errores, pero no resulta tan simple como mostrar una única ruta de decisión.

Concepto	Descripción
Un árbol	más fácil de visualizar.
Un bosque	más potente y estable, pero menos transparente.

También puede ser más costoso computacionalmente. Entrenar un árbol es relativamente simple. Entrenar muchos árboles requiere más tiempo y más memoria. En datasets pequeños esto puede no ser un problema, pero en datasets grandes o con búsquedas de hiperparámetros amplias, el costo puede crecer.

Aun así, Random Forest suele ser una herramienta muy útil porque ofrece un buen equilibrio entre potencia, estabilidad y facilidad de uso. Por eso aparece con frecuencia como un modelo de referencia fuerte en problemas tabulares.

Random Forest suele ser un buen candidato cuando queremos un modelo más potente que un árbol individual.

En el contexto de este libro, Random Forest representa un paso importante. Ya no estamos observando un único modelo simple, sino una estrategia que combina muchos modelos para mejorar el desempeño. Esto abre la puerta a una familia más amplia de métodos de ensamble, donde la predicción final surge de la colaboración entre varios modelos.

Un ensamble combina varios modelos para producir una predicción final. La idea es que varios modelos, si son suficientemente diversos, pueden compensar errores individuales y ofrecer un desempeño más estable que un único modelo. Random Forest es un ensamble formado por muchos árboles de decisión. Cada árbol aporta una predicción, y el bosque combina esas predicciones mediante votación en clasificación o promedio en regresión. Esta estrategia suele mejorar la estabilidad y reducir el riesgo de depender de un único árbol, aunque también aumenta la complejidad y reduce la interpretabilidad directa.

Muchos árboles trabajando juntos

Random Forest se construye a partir de muchos árboles de decisión. Cada árbol aprende una serie de preguntas sobre los datos y produce una predicción. La diferencia es que, en lugar de quedarnos con la decisión de un solo árbol, combinamos las decisiones de todos.

La idea puede resumirse así:

Concepto	Descripción
Un árbol	una secuencia de preguntas, una predicción.
Random Forest	muchos árboles, muchas predicciones, una decisión combinada.

En clasificación, cada árbol vota por una clase. Si el problema consiste en predecir si un cliente abandonará o no un servicio, cada árbol del bosque produce su propia clasificación:

Elemento	Resultado
Árbol 1	abandona
Árbol 2	no abandona
Árbol 3	abandona

Elemento	Resultado
Árbol 4	abandona
Árbol 5	no abandona

La predicción final se obtiene por votación. Si la mayoría de los árboles vota por “abandona”, el bosque predice “abandona”. Si la mayoría vota por “no abandona”, predice “no abandona”.

Predicción final en clasificación: gana la clase con más votos.

En regresión, la lógica es parecida, pero en lugar de votar clases, cada árbol produce un valor numérico. Por ejemplo, si queremos estimar el precio de una vivienda, cada árbol puede proponer un precio:

Elemento	Resultado
Árbol 1	118.000
Árbol 3	121.000
Árbol 5	120.000
Árbol 2	130.000
Árbol 4	125.000

Un árbol aislado podría darnos una predicción muy alta o muy baja. El promedio del bosque puede ser más razonable que depender de cualquiera de esas predicciones individuales.

Por supuesto, el promedio no garantiza la verdad. Si todos los árboles están sesgados por datos mal preparados, variables inválidas o fuga de datos, el bosque también

producirá resultados engañosos. La combinación reduce ciertos tipos de inestabilidad, pero no corrige problemas de fondo en los datos.

Muchos árboles pueden mejorar estabilidad. No corrigen automáticamente datos mal contruidos.

La cantidad de árboles del bosque es un hiperparámetro. En `scikit-learn`, se controla con `n_estimators`.

```
from sklearn.ensemble import RandomForestClassifier

modelo = RandomForestClassifier(
    n_estimators=100,
    random_state=42
)
```

En este ejemplo, `n_estimators=100` indica que el bosque tendrá 100 árboles. Cada uno será entrenado como parte del ensamble y participará en la predicción final.

En general, aumentar la cantidad de árboles puede mejorar la estabilidad del modelo, porque la predicción final se apoya en más votos o más valores promediados. Sin embargo, llega un punto en que agregar más árboles produce mejoras pequeñas y aumenta el costo computacional.

Más árboles: más estabilidad potencial, pero también más tiempo de entrenamiento y más memoria.

No existe una cantidad perfecta de árboles para todos los problemas. Valores como 100, 200 o 500 pueden aparecer con frecuencia en ejemplos prácticos, pero la elección depende del tamaño del dataset, del tiempo disponible y del desempeño observado.

A diferencia de un árbol individual, Random Forest no suele interpretarse dibujando todos sus árboles. Técnicamente podríamos inspeccionarlos, pero un bosque con muchos árboles no es fácil de recorrer manualmente. Por eso, su interpretación suele hacerse de otras formas: importancia de variables, análisis de errores, comparación de métricas y revisión de predicciones.

Un árbol se puede recorrer. Un bosque se analiza de manera agregada.

Esta es una de las compensaciones del modelo. Ganamos estabilidad y potencia, pero perdemos parte de la transparencia directa. Ya no tenemos una única secuencia de preguntas fácil de explicar. Tenemos muchas secuencias de preguntas combinadas.

Aun así, Random Forest conserva una intuición clara: varios árboles entrenados con diversidad pueden producir una predicción más robusta que un árbol individual. Esa

intuición es una de las razones por las que este modelo resulta tan útil en problemas tabulares.

También debemos recordar que Random Forest puede usarse tanto en clasificación como en regresión. En `scikit-learn`, existen dos clases principales:

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.ensemble import RandomForestRegressor
```

`RandomForestClassifier` se usa cuando la variable objetivo es categórica. `RandomForestRegressor` se usa cuando la variable objetivo es numérica continua.

La diferencia está en cómo se combinan las predicciones:

Concepto	Descripción
Clasificación	los árboles votan clases.
Regresión	los árboles producen valores y se calcula un promedio.

El resto de la intuición es común: muchos árboles, diversidad entre ellos y combinación de sus resultados.

Random Forest funciona haciendo que muchos árboles trabajen juntos. Cada árbol produce una predicción y el bosque combina esas predicciones mediante votación en clasificación o promedio en regresión. Para que el ensamble sea útil, los árboles deben ser diversos; por eso el modelo introduce aleatoriedad durante su construcción. Esta combinación suele producir predicciones más estables que las de un único árbol, aunque a costa de menor interpretabilidad directa y mayor costo computacional.

Bagging

Para entender mejor cómo funciona Random Forest, necesitamos presentar una idea importante: **bagging**. El término viene de *bootstrap aggregating*, que puede traducirse como “agregación mediante muestras bootstrap”. Aunque el nombre parezca técnico, la intuición es bastante accesible.

Bagging consiste en entrenar varios modelos usando distintas muestras del conjunto de entrenamiento y luego combinar sus predicciones.

Bagging: entrenar varios modelos con muestras diferentes de los datos y combinar sus resultados.

Esta estrategia busca reducir la inestabilidad de modelos que pueden cambiar mucho según los datos con los que se entrenan. Los árboles de decisión son un ejemplo típico. Un árbol individual puede variar bastante si modificamos algunas filas del dataset. Bagging aprovecha esa variabilidad construyendo muchos árboles diferentes y combinándolos.

La primera parte de la idea es generar varias muestras a partir del conjunto de entrenamiento. Pero no se generan de cualquier manera. En bagging, cada muestra se construye mediante una técnica llamada **bootstrap**.

Una muestra bootstrap se obtiene tomando filas del conjunto de entrenamiento **con reemplazo**. Esto significa que, después de elegir una fila, esa misma fila puede volver a ser elegida.

Muestreo con reemplazo: una misma fila puede aparecer más de una vez en la muestra.

Supongamos un conjunto de entrenamiento muy pequeño, con cinco filas:

Posición 1	Posición 2	Posición 3	Posición 4	Posición 5
Fila A	Fila B	Fila C	Fila D	Fila E

Una muestra bootstrap podría ser:

Posición 1	Posición 2	Posición 3	Posición 4	Posición 5
Fila B	Fila E	Fila B	Fila A	Fila D

En esta muestra, la fila B aparece dos veces y la fila C no aparece. Esto es normal en el muestreo con reemplazo. Otra muestra bootstrap podría ser distinta:

Posición 1	Posición 2	Posición 3	Posición 4	Posición 5
Fila C	Fila C	Fila A	Fila E	Fila B

Cada árbol del Random Forest se entrena con una muestra bootstrap diferente. Como las muestras no son idénticas, los árboles tampoco tienden a ser idénticos. Cada uno aprende a partir de una versión ligeramente distinta del conjunto de entrenamiento.

Muestra bootstrap 1 → árbol 1 Muestra bootstrap 2 → árbol 2 Muestra bootstrap 3 → árbol 3 ...

Luego, las predicciones de esos árboles se agregan. En clasificación, se combinan mediante votación. En regresión, mediante promedio. Esa agregación es la segunda parte de la palabra *bootstrap aggregating*.

Concepto	Descripción
Bootstrap	crear muestras con reemplazo.
Aggregating	combinar las predicciones.

La utilidad de bagging está en que reduce la varianza del modelo. Dicho de manera sencilla, ayuda a que el resultado final dependa menos de una muestra particular de datos. Si un árbol individual cambia mucho al cambiar el conjunto de entrenamiento, el promedio o la votación de muchos árboles puede producir un comportamiento más estable.

Bagging reduce la dependencia de un único árbol entrenado con una única muestra de datos.

Este punto se conecta con lo que vimos sobre sobreajuste. Un árbol de decisión profundo puede ajustarse mucho a los datos de entrenamiento. Si usamos un solo árbol, sus decisiones pueden estar muy influenciadas por detalles particulares del dataset. Si usamos muchos árboles entrenados sobre muestras diferentes, esos detalles individuales tienden a compensarse en la combinación final.

Por ejemplo, un árbol puede construir una regla muy específica porque en su muestra aparecieron ciertos casos repetidos. Otro árbol, entrenado con otra muestra, puede construir una regla distinta. Al combinar muchos árboles, el bosque no depende completamente de ninguna de esas reglas individuales.

Un árbol puede aprender una regla frágil. El bosque reduce el peso de esa regla al combinar muchos árboles.

Sin embargo, bagging por sí solo no siempre alcanza. Si todos los árboles consideran siempre todas las variables y hay una variable muy dominante, muchos árboles pueden terminar pareciéndose bastante. Random Forest agrega un segundo componente de aleatoriedad: además de usar muestras bootstrap de filas, también selecciona subconjuntos de variables al construir las divisiones de cada árbol.

Esta diferencia es importante. Bagging con árboles ya es una técnica útil. Random Forest puede verse como una extensión de esa idea, porque introduce más diversidad entre árboles.

Concepto	Descripción
Bagging	diversidad por muestras de filas.
Random Forest	diversidad por muestras de filas y por selección de variables.

La selección aleatoria de variables ayuda a que los árboles no dependan todos de las mismas columnas. Si una variable es muy fuerte, un árbol individual podría usarla una y otra vez para dividir los datos. Random Forest evita que todos los árboles sigan exactamente el mismo camino, porque en cada división solo evalúan una parte de las variables disponibles.

Esto hace que los árboles sean más diversos. Y, como vimos, la diversidad es clave para que un ensamble aporte valor.

Más diversidad entre árboles: más posibilidad de que sus errores no sean todos iguales.

En `scikit-learn`, Random Forest usa bootstrap por defecto. En un clasificador, podríamos escribir:

```
from sklearn.ensemble import RandomForestClassifier

modelo = RandomForestClassifier(
    n_estimators=100,
    bootstrap=True,
    random_state=42
)
```

El parámetro `bootstrap=True` indica que cada árbol se entrenará con una muestra bootstrap del conjunto de entrenamiento. Como es el valor habitual por defecto, muchas veces no necesitamos escribirlo explícitamente, pero es útil saber que está allí.

En regresión, la lógica es similar:

```
from sklearn.ensemble import RandomForestRegressor

modelo = RandomForestRegressor(
    n_estimators=100,
    bootstrap=True,
    random_state=42
)
```

El parámetro `n_estimators` indica cuántos árboles se construirán. Cada uno usará una muestra bootstrap distinta y luego sus predicciones se combinarán.

Una consecuencia interesante del muestreo bootstrap es que, para cada árbol, algunas filas del conjunto de entrenamiento no aparecen en su muestra. Esas filas quedan fuera de esa muestra específica. Se las suele llamar casos **out-of-bag**, o fuera de bolsa.

Out-of-bag: filas que no fueron usadas para entrenar un árbol particular.

En Random Forest, estos casos pueden usarse para estimar el desempeño del modelo sin una validación separada, mediante lo que se conoce como evaluación *out-of-bag*. No necesitamos profundizar ahora en este mecanismo, pero conviene saber que existe porque surge naturalmente del bagging.

```
modelo = RandomForestClassifier(  
    n_estimators=100,  
    bootstrap=True,  
    oob_score=True,  
    random_state=42  
)
```

Con `oob_score=True`, `scikit-learn` puede calcular una estimación de desempeño usando esas filas que quedaron fuera de las muestras bootstrap de ciertos árboles. Aun así, en un flujo general de aprendizaje, seguimos usando validación cruzada o un conjunto de prueba final cuando corresponde. La evaluación out-of-bag es una herramienta adicional, no una excusa para descuidar la evaluación.

Out-of-bag: puede aportar una estimación adicional.

No reemplaza automáticamente un diseño cuidadoso de evaluación.

Bagging suele funcionar especialmente bien con modelos de alta varianza, como árboles de decisión. Un modelo de alta varianza es aquel cuyo resultado puede cambiar mucho si cambia el conjunto de entrenamiento. Al promediar o votar muchos modelos de ese tipo, se logra una predicción más estable.

No todos los modelos se benefician de la misma manera. Si un modelo ya es muy estable y todos los entrenamientos producen resultados parecidos, hacer bagging puede aportar menos. Por eso, los árboles son una base natural para esta técnica: son flexibles, potentes, pero también inestables cuando se usan individualmente.

Bagging es especialmente útil cuando los modelos individuales son potentes pero inestables.

También debemos recordar sus límites. Bagging no transforma datos malos en datos buenos. Si el dataset contiene fuga de datos, errores sistemáticos, variables inválidas o una variable objetivo mal definida, el ensamble puede aprender esos problemas con mucha eficacia. Incluso puede parecer muy bueno en evaluación si el flujo está contaminado.

Bagging reduce inestabilidad. No corrige fuga de datos ni errores de formulación.

Tampoco elimina completamente el sobreajuste. Random Forest suele sobreajustar menos que un árbol individual, pero aún puede hacerlo si las condiciones son

desfavorables, si el modelo es demasiado flexible, si hay pocas observaciones o si se ajustan hiperparámetros sin cuidado. Por eso seguimos necesitando evaluación con datos no vistos.

El costo computacional también aumenta. Bagging implica entrenar muchos modelos. Si usamos 100 árboles, estamos entrenando 100 modelos base. Si usamos 500 árboles, entrenamos 500. Aunque cada árbol pueda entrenarse de manera relativamente eficiente, el costo total debe considerarse.

Bagging mejora estabilidad, pero aumenta la cantidad de modelos entrenados.

Bagging es una estrategia de ensamble que entrena varios modelos sobre muestras bootstrap del conjunto de entrenamiento y luego combina sus predicciones. En Random Forest, cada árbol se entrena con una muestra diferente, lo que ayuda a generar diversidad. Esa diversidad permite reducir la inestabilidad de los árboles individuales y mejorar la generalización. Random Forest agrega además aleatoriedad en la selección de variables, aumentando aún más la diversidad entre árboles. Bagging no reemplaza la evaluación cuidadosa ni la buena preparación de datos, pero es una de las ideas centrales que explican por qué Random Forest suele funcionar mejor que un árbol de decisión aislado.

Reducción del sobreajuste

Una de las razones principales para usar Random Forest es reducir el sobreajuste que puede aparecer en un árbol de decisión individual. Para entender esto, conviene recordar cómo se comporta un árbol cuando se le permite crecer demasiado.

Un árbol profundo puede crear muchas divisiones. Cada división separa los datos en grupos más pequeños. Si el árbol sigue creciendo, puede llegar a construir hojas con muy pocos casos, incluso con un solo ejemplo. En ese punto, el árbol puede estar aprendiendo detalles muy específicos del conjunto de entrenamiento.

Árbol muy profundo: puede adaptarse demasiado a los datos de entrenamiento.

Ese comportamiento puede producir un resultado excelente sobre los datos conocidos, pero más débil sobre datos nuevos. El árbol no aprendió solamente patrones generales; también aprendió ruido, excepciones y particularidades de la muestra de entrenamiento.

Random Forest reduce este problema combinando muchos árboles. Cada árbol puede ser flexible, pero el bosque no depende de uno solo. La predicción final surge de la

votación o del promedio de todos los árboles. Esto tiende a suavizar decisiones demasiado específicas.

Un árbol puede sobreajustar. Un bosque puede reducir ese efecto al combinar muchos árboles diferentes.

La clave está en la diversidad. Si cada árbol se entrena con una muestra bootstrap distinta y considera diferentes subconjuntos de variables, no todos aprenderán las mismas reglas. Algunos árboles pueden capturar patrones útiles; otros pueden equivocarse en ciertos casos; otros pueden construir divisiones menos convenientes. Al combinar sus predicciones, los errores particulares de algunos árboles pierden peso.

En clasificación, si un árbol vota por una clase incorrecta, puede ser compensado por la mayoría de los demás árboles. En regresión, si un árbol produce una predicción demasiado alta o demasiado baja, el promedio con otros árboles puede moderar ese error.

La combinación reduce el peso de decisiones individuales extremas.

Esto no significa que Random Forest no pueda sobreajustar nunca. Puede hacerlo, especialmente si los datos son pocos, ruidosos, tienen fuga de datos o si los hiperparámetros permiten una flexibilidad excesiva. Pero, en general, suele ser menos propenso al sobreajuste que un único árbol de decisión profundo.

Un árbol individual puede tener alta varianza. Esto significa que su estructura puede cambiar bastante si cambia el conjunto de entrenamiento. Random Forest reduce esa varianza al promediar o votar muchos árboles contruidos a partir de muestras diferentes.

Concepto	Descripción
Árbol individual	alta varianza.
Random Forest	menor varianza gracias a la combinación.

La reducción de varianza es una de las ideas centrales del bagging. Si muchos modelos flexibles cometen errores distintos, la combinación puede producir un resultado más estable. Random Forest aprovecha esta idea con árboles de decisión.

Podemos imaginarlo de esta manera: un árbol aislado puede tomar una decisión muy influida por algunos casos particulares. En cambio, en un bosque, esa decisión individual queda mezclada con muchas otras. El resultado final depende menos de una regla aislada y más de un patrón compartido por muchos árboles.

Random Forest busca que la predicción final dependa menos de una única estructura de árbol.

Este mecanismo ayuda especialmente cuando los árboles son profundos. Un árbol profundo tiene mucha capacidad para ajustar el entrenamiento. Si usamos uno solo, esa capacidad puede volverse un problema. Pero si usamos muchos árboles profundos y diversos, la combinación puede conservar parte de la potencia del árbol y reducir parte de su inestabilidad.

Por eso, Random Forest suele funcionar bien con árboles relativamente flexibles. No se basa en construir árboles muy simples, sino en combinar muchos árboles que no sean idénticos entre sí.

Random Forest no evita el sobreajuste haciendo un árbol muy simple.

Lo reduce combinando muchos árboles diversos.

Esto lo diferencia de la regularización o de limitar fuertemente la profundidad de un árbol individual. En un árbol único, controlar la profundidad puede ser necesario para evitar que memorice. En Random Forest, cada árbol puede crecer más, porque el ensamble completo estabiliza la predicción.

De todos modos, los hiperparámetros siguen importando. Podemos controlar, por ejemplo, la cantidad de árboles, la profundidad máxima, la cantidad mínima de muestras por hoja y la cantidad de variables consideradas en cada división. Todos estos elementos influyen en el equilibrio entre flexibilidad y generalización.

Random Forest reduce sobreajuste, pero sus hiperparámetros siguen afectando el comportamiento del modelo.

Uno de los hiperparámetros más conocidos es `n_estimators`, que indica la cantidad de árboles del bosque. Aumentar la cantidad de árboles suele mejorar la estabilidad, aunque después de cierto punto las mejoras pueden volverse pequeñas.

```
from sklearn.ensemble import RandomForestClassifier

modelo = RandomForestClassifier(
    n_estimators=200,
    random_state=42
)
```

Más árboles significan más predicciones combinadas. Esto puede reducir la variabilidad del resultado final, pero también aumenta el tiempo de entrenamiento y el uso de memoria.

Otro hiperparámetro importante es `max_depth`, que limita la profundidad de cada árbol. Aunque Random Forest suele tolerar árboles más profundos que un árbol individual, en algunos problemas puede convenir limitar la profundidad para evitar modelos demasiado complejos.

```
modelo = RandomForestClassifier(  
    n_estimators=200,  
    max_depth=8,  
    random_state=42  
)
```

También podemos usar `min_samples_leaf`, que exige una cantidad mínima de muestras en cada hoja. Si aumentamos este valor, evitamos hojas demasiado pequeñas y reglas excesivamente específicas.

```
modelo = RandomForestClassifier(  
    n_estimators=200,  
    min_samples_leaf=5,  
    random_state=42  
)
```

Este hiperparámetro puede ser útil cuando sospechamos que el bosque sigue aprendiendo detalles demasiado finos del entrenamiento. Hojas con más casos suelen producir decisiones más estables, aunque también pueden hacer que el modelo pierda capacidad para capturar patrones locales.

Concepto	Descripción
Hojas muy pequeñas	más flexibilidad, más riesgo de capturar ruido.
Hojas más grandes	más estabilidad, pero posible pérdida de detalle.

Otro elemento clave es cuántas variables se consideran en cada división. En `scikit-learn`, esto se controla con `max_features`. Al limitar las variables disponibles para cada split, se obliga a los árboles a explorar caminos diferentes. Esto aumenta la diversidad del bosque.

`max_features`: controla cuántas variables puede considerar cada división de cada árbol.

Si todos los árboles consideran siempre todas las variables, pueden parecerse demasiado, sobre todo si hay variables muy predictivas. Al considerar solo un subconjunto, Random Forest fomenta que distintos árboles usen distintas señales.

Este mecanismo también ayuda a reducir el sobreajuste, porque el bosque no se apoya de manera excesiva en una única variable o en una única estructura de

decisión. Sin embargo, si limitamos demasiado las variables disponibles, cada árbol puede volverse demasiado débil. Como siempre, buscamos equilibrio.

Concepto	Descripción
Poca aleatoriedad	árboles más parecidos.
Demasiada aleatoriedad	árboles demasiado débiles.
Equilibrio	diversidad útil entre árboles.

Para saber si Random Forest está reduciendo el sobreajuste, conviene comparar el desempeño en entrenamiento y validación. Si el modelo obtiene un resultado perfecto en entrenamiento pero claramente peor en validación, puede seguir habiendo sobreajuste. Si el desempeño de validación mejora respecto de un árbol individual y la diferencia entre entrenamiento y validación se reduce, el ensamble está ayudando.

Por ejemplo:

Modelo	Accuracy en entrenamiento	Accuracy en validación
Árbol individual	1.00	0.76
Random Forest	0.98	0.84

En este caso, Random Forest no elimina totalmente la diferencia, pero mejora la validación y reduce el problema. La evaluación con validación cruzada o con un conjunto de prueba reservado permite comprobarlo con más cuidado.

También puede ocurrir que Random Forest tenga un resultado muy alto en entrenamiento y también alto en validación. Eso puede ser una buena señal, pero igualmente debemos revisar que no haya fuga de datos, duplicados o variables que revelen la respuesta. Un modelo potente puede aprovechar esos problemas con mucha eficacia.

Un modelo potente con datos contaminados puede producir métricas muy engañosas.

Random Forest reduce el sobreajuste asociado a la inestabilidad de los árboles, pero no resuelve errores metodológicos. Si la separación de datos está mal hecha, si se usa el test para ajustar hiperparámetros o si hay variables posteriores al momento de predicción, el bosque no nos protegerá.

Esta aclaración es importante porque los modelos más potentes pueden dar una falsa sensación de seguridad. Si el resultado mejora, podemos pensar que el problema está resuelto. Pero una buena métrica solo es confiable si el flujo completo de evaluación también lo es.

Mejor modelo no significa automáticamente mejor evaluación.

Random Forest reduce el sobreajuste de los árboles individuales combinando muchos árboles diferentes. Cada árbol puede aprender reglas específicas, pero la votación o el promedio del bosque reduce el peso de errores particulares. Esta estrategia disminuye la varianza y suele mejorar la generalización frente a un árbol aislado. Sin embargo, Random Forest todavía debe evaluarse con datos no vistos, métricas adecuadas y separación correcta de entrenamiento, validación y prueba. Sus hiperparámetros, como `n_estimators`, `max_depth`, `min_samples_leaf` y `max_features`, permiten controlar el equilibrio entre flexibilidad, diversidad y estabilidad.

Importancia de variables

Una ventaja práctica de Random Forest es que permite estimar qué variables fueron más relevantes para construir las predicciones. A esta información se la suele llamar **importancia de variables** o *feature importance*.

La idea general es identificar qué columnas ayudaron más al bosque a separar los datos, reducir errores o construir mejores decisiones dentro de los árboles.

Importancia de variables: estimación de cuánto aportó cada variable al funcionamiento del modelo.

Esto puede ser muy útil cuando trabajamos con muchas columnas. Un dataset puede incluir variables numéricas, categóricas codificadas, indicadores, fechas transformadas y otras características. Después de entrenar un modelo, es natural preguntarse cuáles de esas variables parecieron influir más en las predicciones.

Por ejemplo, si usamos Random Forest para estimar precios de viviendas, podríamos encontrar que algunas variables tienen mucha importancia:

Las variables mencionadas son `superficie_m2`, `ubicacion`, `antiguedad`, `cantidad_ambientes`.

Y otras tienen menor importancia:

Las variables mencionadas son `tipo_de_publicacion`, `codigo_interno`, `cantidad_de_fotos`.

Este análisis puede ayudarnos a entender mejor el comportamiento del modelo. También puede orientar preguntas sobre los datos: qué variables parecen aportar

señal, cuáles parecen redundantes, cuáles podrían estar mal construidas o cuáles merecen una revisión más cuidadosa.

La importancia de variables ayuda a interpretar el modelo, pero también ayuda a revisar el dataset.

En Random Forest, cada árbol toma decisiones dividiendo los datos en función de ciertas variables. Si una variable aparece muchas veces en divisiones relevantes y ayuda a mejorar la separación de los datos, puede recibir una importancia alta. Si una variable se usa poco o no mejora demasiado las divisiones, su importancia será menor.

En `scikit-learn`, después de entrenar un Random Forest, podemos acceder a esta información con `feature_importances_`.

```
modelo.feature_importances_
```

Este atributo devuelve un valor de importancia para cada variable de entrada. Si `X` es un DataFrame de pandas, podemos asociar esos valores con los nombres de las columnas.

```
import pandas as pd

importancias = pd.Series(
    modelo.feature_importances_,
    index=X.columns
).sort_values(ascending=False)

importancias
```

El resultado podría verse así:

Variable	Importancia
superficie_m2	0.38
ubicacion	0.27
antiguedad	0.14
cantidad_ambientes	0.11
tipo_de_propiedad	0.06
codigo_interno	0.04

Estos valores no deben interpretarse como porcentajes exactos de causalidad. Indican cómo el modelo usó las variables durante el entrenamiento, no cuánto “causan” realmente el resultado.

Concepto	Descripción
Importancia alta	la variable fue útil para el modelo.
No significa automáticamente	la variable causa el resultado.

Esta distinción es fundamental. Un modelo puede encontrar que una variable es muy importante porque está asociada con la variable objetivo, pero esa asociación puede no ser causal. También puede ocurrir que una variable sea importante porque contiene fuga de datos o porque codifica de manera indirecta la respuesta.

Por ejemplo, si estamos prediciendo abandono de clientes y una variable como `fecha_baja` aparece como muy importante, eso no significa que descubrimos una causa del abandono. Significa que probablemente incluimos una variable inválida, porque esa fecha se conoce después del evento que queríamos predecir.

Variable muy importante + disponible después del evento = posible fuga de datos.

Por eso, la importancia de variables debe interpretarse junto con el contexto del problema. Una variable importante no queda automáticamente aceptada. Debemos preguntarnos si es válida, si estaría disponible al momento de predicción, si fue calculada correctamente y si no revela la respuesta de manera directa o indirecta.

Antes de confiar en una variable importante, conviene preguntar:

- ¿estaría disponible al momento de predicción?
- ¿fue calculada solo con información válida?
- ¿podría estar revelando la respuesta?

La importancia de variables también puede verse afectada por la forma en que preparamos los datos. Si aplicamos One-Hot Encoding a una variable categórica, una sola variable original puede transformarse en muchas columnas binarias. En ese caso, la importancia puede quedar repartida entre varias columnas.

Por ejemplo, una variable como `barrio` puede convertirse en:

Las variables mencionadas son `barrio_Centro`, `barrio_Norte`, `barrio_Sur`, `barrio_Oeste`.

Cada columna tendrá su propia importancia. Si miramos solo columnas individuales, podríamos subestimar la importancia total de la variable original `barrio`.

Después de One-Hot Encoding, la importancia de una variable categórica puede quedar distribuida entre varias columnas.

Esto no invalida el análisis, pero exige cuidado. A veces necesitamos agrupar las importancias de las columnas generadas para entender el aporte de la variable original.

También puede haber problemas cuando las variables están correlacionadas. Si dos variables contienen información parecida, el modelo puede distribuir la importancia entre ellas. En otros casos, puede usar más una y menos otra, aunque ambas estén relacionadas con la variable objetivo.

Por ejemplo, en viviendas, `superficie_m2` y `cantidad_ambientes` pueden estar asociadas. Si ambas aportan información parecida, la importancia puede dividirse o desplazarse entre ellas.

Variables correlacionadas: pueden compartir o competir por la importancia dentro del modelo.

Por eso, una variable con baja importancia no siempre es inútil. Puede estar siendo reemplazada por otra variable similar. También puede aportar en segmentos específicos, aunque su importancia global sea baja.

Del mismo modo, una variable con alta importancia no siempre debe conservarse sin discusión. Puede ser válida y útil, pero también puede ser sospechosa. La importancia de variables es una pista, no una sentencia definitiva.

Importancia de variables: sirve para formular preguntas, no para cerrar la interpretación.

Otra precaución importante es que la importancia calculada por Random Forest puede favorecer variables con muchas posibilidades de división. Algunas variables numéricas continuas o categóricas con muchas categorías pueden tener más oportunidades de generar divisiones útiles dentro de los árboles. Esto puede influir en el valor de importancia.

Por eso, cuando el análisis de variables es muy importante, conviene complementar esta información con otras técnicas. Una alternativa frecuente es la **permutation importance**, o importancia por permutación. La idea es medir cuánto empeora el modelo cuando desordenamos los valores de una variable. Si al desordenar una columna el desempeño cae mucho, esa variable parece ser importante para las predicciones.

No necesitamos desarrollar esta técnica en profundidad ahora, pero conviene saber que existe porque ofrece otra forma de analizar el aporte de las variables.

Permutation importance: evalúa cuánto cae el desempeño cuando se rompe la información de una variable.

En términos prácticos, la importancia de variables puede usarse para varias tareas. Puede ayudar a explicar un modelo a otras personas, detectar variables sospechosas, comparar con el conocimiento del dominio, identificar columnas poco útiles o priorizar revisiones del dataset.

Por ejemplo, si un modelo de riesgo crediticio indica que una variable administrativa interna es la más importante, eso merece revisión. Tal vez esa variable esté filtrando información de la decisión final. Si un modelo de demanda muestra que la fecha o la temporada son importantes, eso puede tener sentido. Si un modelo de precios muestra importancia alta para superficie y ubicación, también parece coherente.

Una importancia coherente con el dominio aumenta la confianza.

Una importancia extraña invita a revisar los datos.

Esto no significa que el modelo deba coincidir siempre con nuestras expectativas. A veces los datos revelan patrones que no habíamos considerado. Pero cuando una variable importante contradice fuertemente el contexto, debemos investigar antes de aceptar el resultado.

También podemos usar la importancia de variables para simplificar modelos. Si varias columnas tienen importancia muy baja, podríamos probar un modelo sin ellas y evaluar si el desempeño se mantiene. Pero esta decisión debe validarse. No conviene eliminar variables solo porque aparecen abajo en un ranking, sin medir el efecto.

Eliminar variables: debe evaluarse con validación, no decidirse solo por un ranking.

En un flujo cuidadoso, la importancia de variables puede ser parte del análisis posterior al entrenamiento. Primero entrenamos y evaluamos el modelo. Luego revisamos qué variables usó más. Si detectamos algo sospechoso, podemos volver al dataset, corregir variables, repetir la evaluación y comparar.

Este proceso ayuda a mejorar tanto el modelo como la comprensión del problema.

El proceso puede resumirse en esta secuencia: Entrenar. Evaluar. Revisar importancias. Investigar variables. Corregir si hace falta. Volver a evaluar.

La importancia de variables también debe comunicarse con prudencia. No conviene decir “esta variable es la causa principal” si solo tenemos una importancia de modelo. Es mejor decir que “el modelo utilizó fuertemente esta variable” o que “esta variable aparece como relevante para las predicciones del modelo”.

Concepto	Descripción
Formulación prudente	la variable fue importante para el modelo.
Formulación riesgosa	la variable causa el resultado.

Esta diferencia es especialmente importante en contextos sensibles, como crédito, salud, educación, empleo o seguridad. Una mala interpretación puede llevar a conclusiones injustas o decisiones mal fundamentadas.

Random Forest permite estimar la importancia de las variables usadas por el modelo. Esta información ayuda a interpretar el comportamiento del bosque, revisar el dataset y detectar variables útiles o sospechosas. Sin embargo, debe interpretarse con cuidado: no implica causalidad, puede verse afectada por correlaciones, codificación de variables, fuga de datos y características propias del algoritmo. La importancia de variables es una herramienta de análisis, no una explicación definitiva. Su valor está en orientar preguntas y mejorar la comprensión del modelo y de los datos.

Ventajas y limitaciones

Random Forest es uno de los modelos más utilizados en problemas tabulares porque combina potencia, estabilidad y una implementación relativamente sencilla. No exige comprender una estructura matemática demasiado compleja para usarlo con criterio, pero al mismo tiempo puede capturar relaciones que modelos más simples no representan bien.

Una de sus principales ventajas es que suele generalizar mejor que un árbol de decisión individual. Un árbol puede sobreajustar con facilidad si crece demasiado. Random Forest reduce ese riesgo combinando muchos árboles entrenados con diversidad. La predicción final no depende de una única estructura, sino de la votación o el promedio de todo el bosque.

Ventaja central: más estabilidad que un árbol individual.

Esta estabilidad lo convierte en un buen candidato cuando necesitamos un modelo más potente que los primeros modelos simples, pero sin entrar todavía en técnicas más complejas. En muchos problemas reales, Random Forest puede ofrecer un desempeño sólido como punto de comparación.

Otra ventaja importante es que puede capturar relaciones no lineales. La regresión lineal y la regresión logística construyen decisiones a partir de combinaciones lineales de variables. Eso puede ser suficiente en muchos casos, pero no en todos.

Los árboles, y por lo tanto Random Forest, pueden representar divisiones más flexibles.

Por ejemplo, el efecto de una variable puede depender del valor de otra. Tal vez la antigüedad de una vivienda afecte el precio de manera distinta según la ubicación. Tal vez el consumo de un cliente tenga un significado diferente según su antigüedad en el servicio. Random Forest puede capturar este tipo de interacciones sin que tengamos que escribirlas manualmente.

Random Forest puede aprender: relaciones no lineales e interacciones entre variables.

También tiene la ventaja de no requerir escalado de variables numéricas como sí ocurre con KNN o con muchos modelos lineales regularizados. Los árboles dividen los datos mediante condiciones sobre variables, no mediante distancias ni coeficientes comparables. Por eso, una variable medida en miles y otra medida en unidades pequeñas no generan el mismo problema de escala que en otros modelos.

Random Forest: no suele necesitar escalado para variables numéricas.

Esto simplifica parte del preprocesamiento. Sin embargo, no significa que podamos descuidar la preparación de datos. Los valores faltantes, las variables categóricas, las columnas inválidas, los duplicados, la fuga de datos y la definición de la variable objetivo siguen siendo cuestiones fundamentales.

En particular, en `scikit-learn`, las variables categóricas suelen necesitar alguna forma de codificación antes de entrenar el modelo. Random Forest no entiende directamente textos como "norte", "sur" o "tarjeta" si están como cadenas en una columna. Debemos transformarlas en una representación numérica adecuada, por ejemplo mediante One-Hot Encoding u otras estrategias.

No necesitar escalado no significa no necesitar preprocesamiento.

Otra ventaja de Random Forest es que puede usarse tanto para clasificación como para regresión. Si la variable objetivo es categórica, usamos `RandomForestClassifier`. Si la variable objetivo es numérica continua, usamos `RandomForestRegressor`. La lógica de muchos árboles trabajando juntos se mantiene, aunque cambie la forma de combinar las predicciones.

Concepto	Descripción
Clasificación	votación entre árboles.
Regresión	promedio de predicciones.

También ofrece información sobre importancia de variables. Esto puede ayudar a comprender qué columnas fueron más utilizadas por el modelo y a detectar variables sospechosas o poco útiles. No es una explicación definitiva, pero es una herramienta práctica para analizar el comportamiento del bosque.

Sin embargo, Random Forest también tiene limitaciones.

La primera es la interpretabilidad. Un árbol de decisión individual puede visualizarse y explicarse como una secuencia de preguntas. Random Forest, en cambio, combina muchos árboles. Si el bosque tiene 100, 200 o 500 árboles, ya no es práctico explicar la predicción recorriendo todas las reglas.

Concepto	Descripción
Un árbol	más fácil de explicar.
Random Forest	más potente, pero menos transparente.

Podemos analizar importancias de variables, revisar errores y estudiar predicciones, pero no tendremos una explicación tan directa como con un árbol pequeño o una regresión lineal. En contextos donde la explicación paso a paso es muy importante, esta menor transparencia puede ser un problema.

Otra limitación es el costo computacional. Entrenar un árbol puede ser relativamente rápido. Entrenar muchos árboles requiere más tiempo y más memoria. Además, si usamos validación cruzada o búsqueda de hiperparámetros, el costo se multiplica.

Por ejemplo, si entrenamos un Random Forest con 200 árboles y lo evaluamos con validación cruzada de 5 folds, se entrenan muchos árboles en total. Si además probamos varias configuraciones con `GridSearchCV`, el número de entrenamientos puede crecer rápidamente.

Más árboles: más estabilidad potencial, pero también más costo computacional.

En datasets pequeños o medianos esto suele ser manejable. En datasets grandes, o en entornos con recursos limitados, debemos prestar atención al tiempo de entrenamiento, la memoria y la cantidad de combinaciones que probamos.

Otra limitación es que Random Forest no extrapola de la misma manera que un modelo lineal. En regresión, sus predicciones suelen estar basadas en promedios de valores observados en las hojas de los árboles. Esto puede ser problemático si necesitamos predecir valores más allá del rango de entrenamiento.

Por ejemplo, si entrenamos un modelo con precios de viviendas entre 50.000y 200.000, Random Forest puede tener dificultades para producir predicciones

razonables para casos que realmente deberían estar muy por encima de ese rango si no vio situaciones similares durante el entrenamiento.

Random Forest funciona muy bien dentro de patrones observados.

Puede tener dificultades cuando debe extrapolar más allá de ellos.

Esto no significa que un modelo lineal siempre extrapole bien. También puede equivocarse mucho fuera del rango conocido. Pero Random Forest, por su forma de construir predicciones, no está pensado para proyectar tendencias lineales indefinidamente.

También puede verse afectado por datos ruidosos o variables inválidas. Aunque sea más robusto que un árbol individual, no es inmune a problemas de calidad. Si una columna contiene fuga de datos, Random Forest puede aprovecharla muy bien y producir métricas engañosamente altas. Si hay duplicados mal separados entre entrenamiento y prueba, también puede beneficiarse de esa contaminación.

Un modelo potente puede aprender muy bien información que no debería usar.

Por eso, cuanto más potente es el modelo, más importante es revisar el flujo de evaluación. Una métrica alta no alcanza si los datos no están bien definidos, si la separación temporal es incorrecta o si las variables no estarían disponibles al momento real de predicción.

Otra limitación es que Random Forest puede no ser la mejor opción cuando el problema requiere una explicación simple o una relación clara entre variables. Si un modelo lineal obtiene un desempeño parecido y es más interpretable, quizá sea preferible. No siempre conviene elegir el modelo más complejo.

Si dos modelos rinden parecido, el más simple puede ser más conveniente.

La elección depende del objetivo. En un contexto exploratorio, Random Forest puede ser excelente para obtener buen desempeño y detectar variables importantes. En un contexto donde se necesita justificar cada decisión, puede requerir herramientas adicionales de interpretación o quizá convenga un modelo más simple.

También debemos recordar que Random Forest tiene hiperparámetros que afectan su comportamiento. `n_estimators`, `max_depth`, `min_samples_leaf`, `max_features` y otros valores pueden modificar el equilibrio entre flexibilidad, estabilidad y costo. Aunque suele funcionar razonablemente bien con configuraciones por defecto, ajustar estos hiperparámetros puede mejorar el resultado o hacerlo más adecuado al problema.

Random Forest suele funcionar bien como punto de partida, pero sus hiperparámetros siguen importando.

Aun así, no conviene ajustar hiperparámetros mirando el conjunto de prueba final. Como vimos en capítulos anteriores, la búsqueda debe realizarse con validación cruzada sobre entrenamiento, y el test debe reservarse para la evaluación final.

Random Forest ofrece varias ventajas: suele reducir el sobreajuste de un árbol individual, captura relaciones no lineales, maneja interacciones entre variables, no requiere escalado numérico en la misma medida que otros modelos y puede usarse en clasificación y regresión. También permite analizar importancia de variables. Sus limitaciones principales son menor interpretabilidad directa, mayor costo computacional, dificultades para extrapolar, sensibilidad a datos mal definidos y necesidad de una evaluación cuidadosa. Es un modelo potente y muy útil, pero no reemplaza el criterio, la preparación de datos ni la elección de métricas adecuadas.

Combinar árboles para ganar estabilidad

En este capítulo estudiamos Random Forest como un modelo de ensamble construido a partir de muchos árboles de decisión. La idea central fue que, en lugar de confiar en un único árbol, el modelo combina las predicciones de varios árboles para obtener una decisión final más estable.

Un árbol individual puede ser útil, flexible e interpretable, pero también puede sobreajustar con facilidad si crece demasiado. Random Forest reduce ese riesgo construyendo muchos árboles diferentes y combinando sus respuestas.

Concepto	Descripción
Árbol individual	una estructura de decisión.
Random Forest	muchos árboles, muchas predicciones, una decisión combinada.

En clasificación, los árboles votan por una clase y gana la clase más votada. En regresión, cada árbol produce un valor numérico y el bosque devuelve un promedio. En ambos casos, la predicción final depende del conjunto de árboles y no de una única estructura.

El concepto de ensamble fue clave para entender este modelo. Un ensamble busca combinar varios modelos para mejorar el desempeño o la estabilidad. No necesita que cada modelo individual sea perfecto. Lo importante es que los modelos sean suficientemente diversos para que sus errores no sean todos iguales.

La diversidad entre modelos permite que la combinación compense errores individuales.

Random Forest logra esa diversidad mediante bagging y selección aleatoria de variables. Con bagging, cada árbol se entrena con una muestra bootstrap del conjunto de entrenamiento. Esto significa que cada árbol ve una versión distinta de los datos. Además, en cada división, el árbol considera solo una parte de las variables disponibles. Esto evita que todos los árboles sigan exactamente los mismos caminos.

Random Forest combina: muestras distintas de filas + subconjuntos de variables + muchos árboles de decisión.

Esa combinación ayuda a reducir la varianza de los árboles individuales. Un solo árbol puede cambiar bastante si cambian algunos datos de entrenamiento. Un bosque, en cambio, tiende a ser más estable porque la predicción final surge de la votación o del promedio de muchos árboles.

También vimos que Random Forest puede reducir el sobreajuste de un árbol individual, pero no lo elimina por completo. Sigue siendo necesario evaluar con datos no vistos, usar validación cruzada cuando corresponda y cuidar la separación entre entrenamiento, validación y prueba.

Random Forest suele sobreajustar menos que un árbol individual, pero no reemplaza una evaluación honesta.

Sus hiperparámetros permiten controlar el equilibrio entre flexibilidad, diversidad y costo computacional. `n_estimators` define la cantidad de árboles. `max_depth` limita la profundidad. `min_samples_leaf` controla el tamaño mínimo de las hojas. `max_features` regula cuántas variables se consideran en cada división.

Estos hiperparámetros no deben elegirse mirando el conjunto de prueba final. Como vimos en capítulos anteriores, deben ajustarse mediante validación cruzada dentro del conjunto de entrenamiento.

Una ventaja práctica de Random Forest es que permite estimar importancia de variables. Esta información ayuda a analizar qué columnas fueron más utilizadas por el modelo para construir sus predicciones. Puede servir para interpretar, revisar el dataset y detectar variables sospechosas.

Pero esa importancia debe leerse con cuidado. No implica causalidad, puede verse afectada por variables correlacionadas, por One-Hot Encoding, por columnas con muchas posibilidades de división y, especialmente, por fuga de datos.

Una variable importante para el modelo no necesariamente es una causa real.

Puede ser una señal útil, pero también una pista sospechosa.

Random Forest tiene varias ventajas: suele ser potente en datos tabulares, captura relaciones no lineales, maneja interacciones entre variables y no requiere escalado numérico como KNN o algunos modelos lineales. Además, puede usarse tanto en clasificación como en regresión.

También tiene limitaciones. Es menos interpretable que un árbol individual, puede ser más costoso computacionalmente, no extrapola bien fuera de los patrones observados y puede aprender con mucha eficacia variables inválidas si el dataset está mal construido.

Modelo más potente no significa automáticamente modelo más confiable.

Random Forest es un ensamble de muchos árboles de decisión que trabajan juntos. Su fortaleza proviene de combinar modelos diversos para producir predicciones más estables que las de un único árbol. Es una herramienta muy útil para problemas tabulares y suele ofrecer buen desempeño con relativamente poco ajuste inicial. Sin embargo, debe usarse con el mismo cuidado que cualquier modelo: datos válidos, preprocesamiento correcto, evaluación honesta, métricas adecuadas e interpretación prudente de sus resultados.

Capítulo 22 · Gradient Boosting

La idea del boosting

Random Forest nos mostró una forma de construir modelos más potentes combinando muchos árboles de decisión. En ese modelo, los árboles se entrenan de manera relativamente independiente y luego sus predicciones se combinan mediante votación o promedio. La fuerza del bosque aparece al reunir muchos árboles diversos.

Gradient Boosting también combina varios modelos, pero lo hace con una lógica diferente. En lugar de entrenar muchos modelos en paralelo, los construye de manera secuencial. Cada nuevo modelo intenta mejorar lo que los anteriores todavía no resolvieron bien.

Random Forest: muchos árboles trabajan en paralelo. Gradient Boosting: los modelos se construyen en secuencia, corrigiendo errores anteriores.

La palabra **boosting** se refiere a una familia de técnicas que buscan transformar modelos relativamente simples en un modelo final potente. La idea es entrenar muchos modelos débiles, donde cada uno aporta una pequeña mejora. Al combinarlos, el resultado puede ser mucho más fuerte que cualquiera de ellos por separado.

Un modelo débil no significa un modelo inútil. Significa un modelo que, por sí solo, tiene capacidad limitada. En Gradient Boosting, muchas veces se usan árboles pequeños, de poca profundidad. Cada árbol no intenta resolver todo el problema. Su tarea es aportar una corrección parcial.

Modelo débil: aprende una parte del problema. Muchos modelos débiles combinados: pueden formar un modelo muy potente.

Podemos pensarlo como un proceso de mejora progresiva. Primero se construye un modelo inicial. Ese modelo comete errores. Luego se entrena otro modelo para enfocarse en esos errores. Después se agrega otro, y otro, y otro. Cada paso intenta reducir lo que todavía falta corregir.

La lógica no es “construyamos muchos modelos y votemos”, como en Random Forest. La lógica es más parecida a:

primera aproximación ↓ revisar errores ↓ agregar una corrección ↓ revisar nuevos errores ↓ agregar otra corrección ↓ modelo final.

Esta idea puede verse con un ejemplo de regresión. Supongamos que queremos predecir el consumo eléctrico de una vivienda. El primer modelo produce una predicción inicial. Para algunos casos se equivoca por poco; para otros se equivoca bastante.

El segundo modelo no empieza desde cero. Intenta aprender algo sobre los errores del primer modelo. Si el primer modelo subestimó mucho el consumo en viviendas grandes con aire acondicionado, el siguiente modelo puede aprender una corrección para ese tipo de casos. Después, un tercer modelo puede corregir errores que todavía quedaron pendientes.

Modelo 1: produce una predicción inicial. Modelo 2: corrige parte de los errores del modelo 1. Modelo 3: corrige parte de los errores restantes.

En clasificación, la intuición es parecida, aunque los detalles matemáticos son distintos. El modelo final se construye agregando modelos que ayudan a mejorar las decisiones donde las versiones anteriores todavía fallaban o estaban poco seguras.

Lo importante, por ahora, es entender el enfoque: Gradient Boosting aprende de manera aditiva y secuencial. El modelo final no aparece de una sola vez. Se forma acumulando pequeñas contribuciones.

Gradient Boosting construye el modelo final sumando muchas correcciones pequeñas.

Esta forma de aprender puede ser muy poderosa. Al enfocarse gradualmente en los errores, el modelo puede capturar patrones complejos que un único árbol pequeño no podría representar. También puede modelar relaciones no lineales e interacciones entre variables, como Random Forest, pero con una estrategia diferente.

Sin embargo, esa potencia tiene un costo: si no se controla bien, Gradient Boosting puede sobreajustar. Como el modelo va corrigiendo errores paso a paso, existe el riesgo de que empiece a corregir no solo patrones reales, sino también ruido o particularidades del conjunto de entrenamiento.

Gradient Boosting puede aprender patrones complejos, pero también puede adaptarse demasiado si se lo deja avanzar sin control.

Por eso, en este modelo son muy importantes los hiperparámetros. La cantidad de árboles, la profundidad de cada árbol, la velocidad con la que se agregan correcciones y otros parámetros influyen mucho en el equilibrio entre aprendizaje y sobreajuste.

Uno de los conceptos centrales es la **tasa de aprendizaje**, conocida en inglés como `learning_rate`. Esta tasa controla cuánto aporta cada nuevo árbol al modelo final. Si cada árbol corrige demasiado, el modelo puede avanzar rápido pero sobreajustar. Si cada árbol aporta poco, el aprendizaje es más lento, pero puede ser más estable.

`learning_rate` alto: correcciones más fuertes, más riesgo de sobreajuste.
`learning_rate` bajo: correcciones más pequeñas, aprendizaje más gradual.

En general, Gradient Boosting suele combinar una tasa de aprendizaje pequeña con una cantidad mayor de árboles. Esto permite construir el modelo de manera más progresiva. No es una regla absoluta, pero es una intuición importante.

Correcciones pequeñas + muchos pasos = aprendizaje gradual.

Esta forma de trabajar se parece a mejorar un texto por revisiones sucesivas. En una primera lectura corregimos los errores más evidentes. En una segunda, ajustamos frases. En una tercera, mejoramos detalles. Pero si seguimos corrigiendo indefinidamente, podemos terminar sobreeditando: cambiando cosas que no hacía falta cambiar o adaptando el texto a una situación demasiado particular.

Con Gradient Boosting ocurre algo similar. Cada nuevo modelo mejora lo anterior, pero demasiadas correcciones pueden hacer que el modelo se adapte excesivamente al conjunto de entrenamiento.

También conviene aclarar que Gradient Boosting no es un único algoritmo cerrado. Es una idea general que tiene distintas implementaciones. En `scikit-learn` existen modelos como `GradientBoostingClassifier` y `GradientBoostingRegressor`. También existen implementaciones muy populares y optimizadas, como XGBoost, LightGBM y CatBoost, que se usan mucho en problemas reales y competencias de datos.

No necesitamos estudiar todas esas implementaciones en detalle en este momento. Lo importante es comprender la idea común: construir un modelo fuerte agregando modelos más simples que corrigen errores de manera secuencial.

Boosting: combinar modelos simples de forma secuencial para mejorar progresivamente.

Gradient Boosting puede usarse tanto en clasificación como en regresión. En regresión, suele pensarse de manera intuitiva como la corrección progresiva de errores numéricos. En clasificación, la construcción también es progresiva, pero orientada a mejorar la capacidad del modelo para asignar clases o probabilidades.

En `scikit-learn`, por ejemplo, podríamos encontrar:

```
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.ensemble import GradientBoostingRegressor
```

El primero se usa para problemas de clasificación. El segundo, para problemas de regresión.

Como ocurre con Random Forest, Gradient Boosting puede trabajar bien con relaciones no lineales y combinaciones entre variables. También suele funcionar muy bien con datos tabulares. Pero requiere más cuidado en el ajuste. Sus resultados pueden ser excelentes, pero una mala configuración puede llevar a sobreajuste, tiempos de entrenamiento mayores o conclusiones demasiado optimistas.

Gradient Boosting: modelo muy potente, pero sensible a la configuración.

Esta sensibilidad es una diferencia importante respecto de Random Forest. Random Forest suele funcionar razonablemente bien con configuraciones iniciales. Gradient Boosting puede necesitar más atención a hiperparámetros como `n_estimators`, `learning_rate`, `max_depth` y otros controles de complejidad.

Por eso, los capítulos anteriores son necesarios para entenderlo con criterio. Para usar Gradient Boosting no alcanza con entrenarlo y mirar una métrica. Necesitamos validación cruzada, cuidado con el conjunto de prueba, revisión del sobreajuste, elección adecuada de métricas y búsqueda de hiperparámetros.

Modelos más potentes exigen evaluaciones más cuidadosas.

Boosting es una estrategia de ensamble que construye un modelo fuerte combinando muchos modelos más simples de manera secuencial. Cada nuevo modelo intenta corregir errores o debilidades de los modelos anteriores. Gradient Boosting aplica esta idea usando correcciones progresivas guiadas por el error. Puede lograr muy buen desempeño en problemas de clasificación y regresión, especialmente con datos tabulares, pero debe usarse con cuidado porque su potencia también aumenta el riesgo de sobreajuste si no se controlan bien sus hiperparámetros.

Modelos que corrigen errores anteriores

La idea central de Gradient Boosting es construir un modelo fuerte a partir de una secuencia de modelos más simples. Cada nuevo modelo se agrega para mejorar lo que el conjunto anterior todavía no resolvió bien. Por eso decimos que son modelos que corrigen errores anteriores.

En Random Forest, los árboles se entrenan de manera relativamente independiente. En Gradient Boosting, en cambio, hay un orden. El segundo modelo depende de lo que hizo el primero. El tercero depende de lo que ya hicieron los anteriores. Y así sucesivamente.

Gradient Boosting: cada nuevo modelo se entrena teniendo en cuenta los errores de los modelos anteriores.

Esta lógica secuencial es una de las diferencias más importantes del boosting. No se trata de entrenar muchos modelos y combinarlos todos al final sin relación entre ellos. Se trata de construir una solución paso a paso.

Podemos imaginar un primer modelo que intenta resolver el problema de manera aproximada. Ese modelo produce predicciones. Algunas serán buenas y otras no tanto. Luego observamos dónde se equivocó. El siguiente modelo se enfoca en reducir esos errores. Después se vuelven a revisar los errores restantes y se agrega otra corrección.

Modelo inicial ↓ errores del modelo inicial ↓ nuevo modelo para corregir parte de esos errores ↓ errores restantes ↓ nueva corrección.

En regresión, esta idea puede verse de manera bastante intuitiva. Supongamos que queremos predecir el precio de viviendas. El primer modelo produce una estimación. Para una vivienda cuyo precio real es 120.000, tal vez predice 100.000. El error es de 20.000: el modelo se quedó corto.

Un modelo posterior puede aprender una corrección para casos parecidos. No necesita volver a predecir el precio real. Podemos resumir esto en una tabla:

Concepto	Descripción
120.000	Predicción inicial 100.000
	Error restante 20.000

Si el siguiente modelo aprende una corrección de 15.000 para este tipo de viviendas, la predicción combinada se acerca más al valor real:

Concepto	Descripción
	Predicción inicial 100.000
	Corrección 15.000
	Predicción nueva 115.000

Todavía puede quedar error, pero ya es menor. Luego otro modelo podría corregir una parte adicional. Así, el modelo final se forma sumando muchas correcciones.

Predicción final = predicción inicial + corrección 1 + corrección 2 + corrección 3 + ...

Esta forma de aprender es aditiva. Cada nuevo modelo no reemplaza completamente a los anteriores; se suma a ellos. Por eso, Gradient Boosting puede capturar patrones complejos a partir de muchas contribuciones pequeñas.

En clasificación, la intuición es similar, aunque el cálculo interno no se expresa tan directamente como “sumar pesos” o “sumar pesos en pesos”. El modelo también se construye progresivamente, ajustando nuevas partes para mejorar la clasificación o las probabilidades estimadas. Lo importante es que cada paso intenta reducir los errores que quedaron de los pasos anteriores.

En clasificación: cada nuevo modelo ayuda a mejorar las decisiones que el conjunto anterior todavía resolvía mal.

Una forma de entenderlo es pensar en casos difíciles. Un primer modelo puede clasificar correctamente los ejemplos más evidentes, pero confundirse con casos cercanos a la frontera de decisión. Los modelos posteriores pueden enfocarse más en esas zonas problemáticas, agregando ajustes que mejoran la separación.

Esto hace que Gradient Boosting sea muy potente. No se limita a una sola estructura de árbol ni a una votación independiente. Construye una secuencia donde cada parte aporta algo específico al conjunto.

Pero esa misma potencia exige cuidado. Si el modelo sigue corrigiendo errores durante demasiado tiempo, puede empezar a corregir ruido. Es decir, puede intentar ajustar diferencias que no representan patrones reales, sino accidentes del conjunto de entrenamiento.

Corregir errores reales: mejora la generalización. Corregir ruido: aumenta el riesgo de sobreajuste.

Esta es una idea fundamental. No todo error del entrenamiento merece ser corregido. Algunos errores existen porque los datos contienen ruido, casos atípicos, mediciones imprecisas o situaciones que no se repetirán. Si el modelo intenta ajustar todos esos detalles, puede perder capacidad de generalizar.

Por eso, Gradient Boosting debe avanzar con moderación. Cada nuevo modelo suele aportar una corrección pequeña, controlada por la tasa de aprendizaje. En lugar de permitir que cada árbol modifique mucho la predicción final, se reduce su influencia para que el aprendizaje sea gradual.

Correcciones pequeñas: aprendizaje más lento, pero generalmente más controlado.

La tasa de aprendizaje, o `learning_rate`, regula precisamente cuánto pesa cada nuevo árbol en el modelo final. Si es alta, cada árbol corrige con fuerza. Si es baja, cada árbol corrige poco y se necesitan más árboles para construir el modelo.

Esta relación es importante:

`learning_rate` alto: menos pasos, correcciones más fuertes, más riesgo de sobreajuste. `learning_rate` bajo: más pasos, correcciones más suaves, aprendizaje más gradual.

La cantidad de árboles también importa. Si usamos muy pocos, el modelo puede quedarse corto y no aprender lo suficiente. Si usamos demasiados, puede terminar ajustándose demasiado al entrenamiento, especialmente si la tasa de aprendizaje no es suficientemente baja o si los árboles son demasiado complejos.

Pocos árboles: riesgo de subajuste. Demasiados árboles: riesgo de sobreajuste. Cantidad adecuada: depende de los datos, la tasa de aprendizaje y la complejidad de cada árbol.

Otro aspecto importante es que los modelos base suelen ser árboles pequeños. A diferencia de un árbol de decisión profundo que intenta resolver todo el problema, en Gradient Boosting cada árbol cumple un papel más limitado. Es una pieza dentro de una secuencia.

Esto se puede pensar como una suma de pequeñas reglas. Un árbol pequeño puede capturar un ajuste simple. Muchos árboles pequeños, agregados ordenadamente, pueden representar un patrón complejo.

Un árbol pequeño: corrección simple. Muchos árboles pequeños: modelo final complejo.

Esta combinación es una de las razones por las que Gradient Boosting puede lograr resultados muy buenos en datos tabulares. Puede aprender relaciones no lineales, interacciones entre variables y patrones difíciles de capturar con modelos lineales simples.

Pero, nuevamente, el control es esencial. Si los árboles base son demasiado profundos, cada corrección puede ser demasiado específica. Si además usamos muchos árboles, el modelo puede volverse excesivamente flexible.

Gradient Boosting no solo depende de cuántos árboles usamos, sino también de qué tan complejos son esos árboles.

Por eso, hiperparámetros como `n_estimators`, `learning_rate`, `max_depth` y otros controles de tamaño de los árboles trabajan juntos. No conviene interpretarlos de manera aislada.

Por ejemplo, un `learning_rate` bajo puede permitir usar más árboles. Un `learning_rate` alto puede requerir menos árboles para evitar sobreajuste. Árboles más profundos pueden necesitar más regularización o una búsqueda más cuidadosa.

Los hiperparámetros de Gradient Boosting interactúan entre sí.

En la práctica, esto significa que Gradient Boosting suele requerir más ajuste que Random Forest. No porque sea necesariamente más difícil de usar, sino porque su desempeño depende mucho de cómo se controle la secuencia de correcciones.

Un buen modelo de boosting no es simplemente el que corrige más. Es el que corrige lo suficiente, en el ritmo adecuado, sin empezar a perseguir ruido.

El objetivo no es corregir todo. El objetivo es corregir lo que ayuda a generalizar mejor.

Esta idea conecta con el capítulo sobre regularización y control de complejidad. Gradient Boosting necesita mecanismos que eviten que la secuencia crezca sin control. La tasa de aprendizaje, la cantidad de árboles y las restricciones sobre cada árbol actúan como formas de controlar la complejidad del modelo final.

También conecta con validación cruzada y conjunto de prueba. Como Gradient Boosting puede ajustarse mucho a los datos, no debemos elegir configuraciones mirando el test final. Debemos usar validación cruzada o una estrategia de validación adecuada dentro del entrenamiento, y reservar el test para una evaluación final.

Modelo potente + búsqueda descuidada = riesgo de evaluación optimista.

Gradient Boosting construye modelos de manera secuencial. Cada nuevo modelo intenta corregir errores o debilidades del conjunto anterior. En regresión, esto puede entenderse como una suma progresiva de correcciones sobre la predicción. En clasificación, la lógica también es progresiva, aunque el cálculo interno sea distinto. Esta estrategia permite formar modelos muy potentes a partir de muchos modelos simples, pero requiere controlar cuidadosamente la cantidad de pasos, el tamaño de las correcciones y la complejidad de los modelos base para evitar sobreajuste.

Diferencias con Random Forest

Random Forest y Gradient Boosting pertenecen a la familia de los métodos de ensamble. Ambos combinan muchos árboles de decisión para producir un modelo final más potente que un árbol individual. Sin embargo, la forma en que construyen esos árboles y la manera en que los combinan son muy diferentes.

Random Forest construye muchos árboles de manera relativamente independiente. Cada árbol se entrena con una muestra distinta de los datos y considera subconjuntos de variables para generar diversidad. Luego, las predicciones se combinan por votación en clasificación o por promedio en regresión.

Random Forest: árboles independientes, entrenados con diversidad, combinados al final.

Gradient Boosting, en cambio, construye los árboles de manera secuencial. Cada nuevo árbol intenta corregir errores o debilidades del conjunto de árboles anteriores. El modelo final se forma como una suma progresiva de contribuciones.

Gradient Boosting: árboles secuenciales, cada uno corrige errores anteriores, el modelo mejora paso a paso.

Esta diferencia cambia la lógica del aprendizaje. En Random Forest, cada árbol aporta una opinión independiente. La fuerza del modelo está en combinar muchas opiniones diversas. En Gradient Boosting, cada árbol aporta una corrección. La fuerza del modelo está en mejorar progresivamente una predicción acumulada.

Podemos pensarlo así:

Random Forest: muchos árboles opinan al mismo tiempo. Gradient Boosting: cada árbol aprende después del anterior.

En Random Forest, si un árbol se equivoca, otros árboles pueden compensarlo mediante la votación o el promedio. No hay una secuencia de correcciones. El bosque no analiza explícitamente los errores de un árbol para construir el siguiente.

En Gradient Boosting, los errores anteriores son parte central del proceso. El siguiente árbol se construye mirando qué falta mejorar. Por eso, el orden de construcción importa.

En Random Forest: los árboles no dependen fuertemente entre sí. En Gradient Boosting: cada árbol depende del modelo acumulado anterior.

Otra diferencia está en el tipo de árboles que suelen usarse. Random Forest suele construir árboles relativamente profundos y variados. Cada árbol puede capturar

patrones complejos de su muestra bootstrap. Luego, el ensamble reduce la inestabilidad al combinar muchos árboles.

Gradient Boosting, en cambio, suele usar árboles más pequeños. Cada árbol no intenta resolver todo el problema. Su función es hacer una corrección parcial. Muchos árboles simples, sumados de manera ordenada, pueden formar un modelo final muy potente.

Random Forest: árboles más fuertes e independientes. Gradient Boosting: árboles más simples, agregados como correcciones sucesivas.

Esto no significa que siempre sea así en todos los casos, porque los hiperparámetros pueden cambiar la profundidad y el comportamiento de los árboles. Pero como intuición general, Random Forest se apoya en combinar árboles con bastante capacidad individual, mientras que Gradient Boosting se apoya en acumular muchas pequeñas mejoras.

También cambia la forma en que ambos modelos reducen errores. Random Forest reduce principalmente la varianza de los árboles individuales. Un árbol puede ser inestable, pero el promedio o la votación de muchos árboles diversos produce una predicción más estable.

Gradient Boosting busca reducir progresivamente el error del modelo acumulado. No promedia opiniones independientes, sino que va agregando correcciones para mejorar donde todavía hay errores.

Random Forest: reduce inestabilidad combinando árboles. Gradient Boosting: reduce errores agregando correcciones sucesivas.

Esta diferencia hace que Gradient Boosting pueda lograr desempeños muy altos, especialmente cuando se ajusta bien. Al concentrarse en los errores restantes, puede capturar patrones sutiles que otros modelos no detectan. Pero también puede sobreajustar si sigue corrigiendo demasiado, si los árboles son muy complejos o si se usan demasiadas iteraciones sin control.

Random Forest también puede sobreajustar, pero suele ser más robusto con configuraciones iniciales. En muchos problemas tabulares, puede funcionar razonablemente bien sin una búsqueda extensa de hiperparámetros. Gradient Boosting, en cambio, suele ser más sensible a la configuración.

Random Forest: suele ser más robusto como punto de partida. Gradient Boosting: puede ser más potente, pero suele requerir más ajuste.

Esto no significa que Random Forest no necesite evaluación ni que Gradient Boosting sea siempre difícil. Significa que Gradient Boosting tiene más riesgo de mejorar mucho o empeorar según la configuración elegida. Hiperparámetros como `learning_rate`, `n_estimators` y la profundidad de los árboles tienen un papel muy importante.

Otra diferencia práctica es el costo de entrenamiento. Random Forest puede entrenar muchos árboles de manera más independiente. Esto facilita paralelizar el trabajo: distintos árboles pueden construirse al mismo tiempo, dependiendo de la implementación y los recursos disponibles.

Gradient Boosting es más secuencial. Como cada árbol depende del modelo anterior, no se puede construir toda la secuencia de árboles de manera completamente independiente. Esto puede hacer que el entrenamiento sea más delicado o más costoso, aunque las implementaciones modernas han optimizado mucho este proceso.

Random Forest: más naturalmente paralelizable. Gradient Boosting: más secuencial, porque cada paso depende del anterior.

También cambia la interpretación de la predicción final. En Random Forest, podemos decir que la predicción surge de muchos votos o muchos valores promediados. En Gradient Boosting, la predicción surge de una predicción inicial más una serie de ajustes acumulados.

Random Forest: predicción final por votación o promedio. Gradient Boosting: predicción final por suma de correcciones.

Ambos modelos pueden ofrecer importancia de variables, pero esa importancia debe interpretarse con las mismas precauciones: no implica causalidad, puede verse afectada por variables correlacionadas, por la codificación de categorías, por fuga de datos o por características propias del algoritmo.

Otra diferencia es la relación con los errores difíciles. Random Forest no se enfoca explícitamente en los casos donde el modelo anterior falló, porque no hay un modelo anterior en sentido secuencial. Gradient Boosting sí tiende a concentrarse en lo que todavía no está bien resuelto. Esa es parte de su potencia, pero también parte de su riesgo.

Gradient Boosting presta atención a los errores que quedan pendientes. Eso puede mejorar el desempeño, pero también puede llevarlo a perseguir ruido.

Por ejemplo, si algunos casos del conjunto de entrenamiento tienen etiquetas incorrectas o valores atípicos, Gradient Boosting puede intentar corregirlos de manera insistente. Si esos casos no representan patrones reales, el modelo puede

sobreajustar. Random Forest también puede verse afectado por ruido, pero su mecanismo de promedio puede suavizar parte de ese impacto.

En términos de uso práctico, Random Forest suele ser una excelente opción cuando queremos un modelo fuerte, relativamente estable y con menor necesidad de ajuste fino inicial. Gradient Boosting suele ser una opción muy poderosa cuando buscamos alto desempeño y estamos dispuestos a ajustar hiperparámetros con más cuidado.

Random Forest: buen desempeño, estabilidad, uso relativamente directo. Gradient Boosting: muy alto desempeño posible, más sensibilidad, mayor necesidad de control.

La elección entre ambos no debe hacerse por nombre ni por popularidad. Debe hacerse evaluando el problema, los datos, las métricas, el costo computacional, la interpretabilidad necesaria y la estabilidad de los resultados.

En algunos problemas, Random Forest puede ser suficiente y preferible por su robustez. En otros, Gradient Boosting puede superar claramente su desempeño. En otros, un modelo más simple puede ser mejor si obtiene resultados parecidos y es más fácil de explicar.

No existe un modelo mejor para todo. La comparación debe hacerse con evaluación justa y métricas adecuadas.

Random Forest y Gradient Boosting son ensambles basados en árboles, pero construyen sus modelos de manera distinta. Random Forest entrena muchos árboles diversos e independientes y combina sus predicciones por votación o promedio. Gradient Boosting construye árboles en secuencia, donde cada nuevo árbol corrige errores del modelo acumulado. Random Forest suele ser más robusto como punto de partida; Gradient Boosting puede alcanzar desempeños muy altos, pero requiere más cuidado para evitar sobreajuste y ajustar correctamente sus hiperparámetros.

Potencia y riesgo de sobreajuste

Gradient Boosting es un modelo muy potente porque construye una solución de manera progresiva. En lugar de depender de un único árbol o de una votación entre árboles independientes, agrega modelos que intentan corregir errores anteriores. Esa estrategia le permite capturar relaciones complejas, interacciones entre variables y patrones difíciles de representar con modelos más simples.

Esta potencia es una de sus grandes ventajas. Pero también es una de sus principales fuentes de riesgo.

Gradient Boosting: puede lograr muy buen desempeño, pero necesita control cuidadoso.

El riesgo principal es el sobreajuste. Como el modelo va agregando correcciones una tras otra, puede terminar ajustándose demasiado al conjunto de entrenamiento. Al principio, cada nuevo árbol puede ayudar a corregir errores reales. Pero si el proceso continúa demasiado, puede empezar a corregir ruido, valores atípicos o particularidades que no se repetirán en datos nuevos.

Esto se conecta directamente con lo que vimos en capítulos anteriores: no todo error del entrenamiento debe ser perseguido. Algunos errores representan límites naturales de los datos, mediciones imperfectas, etiquetas discutibles o casos excepcionales. Si el modelo intenta ajustar todo eso, puede perder capacidad de generalizar.

Corregir patrones reales: mejora el modelo. Corregir ruido: puede producir sobreajuste.

Supongamos un problema de regresión donde queremos predecir ventas semanales. Hay patrones reales: temporada, promociones, precios, feriados, ubicación y tipo de producto. Gradient Boosting puede aprender combinaciones complejas entre esas variables. Por ejemplo, una promoción puede tener efectos distintos según la categoría del producto o la época del año.

Pero también puede encontrar detalles accidentales. Quizá una semana tuvo ventas inusuales por un evento aislado que no está registrado en las variables. Quizá hubo un error de carga. Quizá una sucursal tuvo un comportamiento excepcional por una causa externa. Si el modelo intenta corregir con demasiada precisión esos casos, puede mejorar en entrenamiento, pero empeorar en validación o prueba.

Buen aprendizaje: captura relaciones que se repiten. Sobreajuste: captura detalles particulares del conjunto de entrenamiento.

En clasificación ocurre algo parecido. Imaginemos un modelo para detectar abandono de clientes. Gradient Boosting puede aprender patrones útiles: menor uso del servicio, reclamos recientes, caída de actividad, cambios en el comportamiento de pago. Pero también puede adaptarse a casos muy específicos si se lo deja crecer demasiado o si se usan muchas iteraciones sin control.

La potencia del modelo no debe confundirse con confiabilidad automática. Un modelo más potente puede encontrar mejores patrones, pero también puede aprovechar mejor problemas del dataset: fuga de datos, duplicados, variables posteriores al momento de predicción o etiquetas mal definidas.

Un modelo potente aprende señales útiles con facilidad. Pero también puede aprender señales inválidas con facilidad.

Por eso, con Gradient Boosting es especialmente importante revisar la diferencia entre desempeño en entrenamiento y desempeño en validación. Si el resultado en entrenamiento mejora continuamente, pero el resultado en validación deja de mejorar o empieza a empeorar, estamos ante una señal de sobreajuste.

Entrenamiento mejora. Validación mejora al principio. Luego validación se estanca o empeora. Señal posible: el modelo empezó a sobreajustar.

Este comportamiento es frecuente en modelos que aprenden de manera progresiva. Al agregar más árboles, el modelo tiene más capacidad para ajustar los datos. Al comienzo, eso suele ayudar. Pero después de cierto punto, agregar más árboles puede sumar complejidad innecesaria.

Más árboles: más capacidad de corrección. Demasiados árboles: más riesgo de ajustar ruido.

La tasa de aprendizaje también influye mucho. Si `learning_rate` es alto, cada nuevo árbol tiene mucho peso en el modelo final. Esto puede hacer que el modelo aprenda rápido, pero también que avance de manera brusca y se adapte demasiado a errores particulares.

Si `learning_rate` es bajo, cada árbol aporta menos. El aprendizaje es más gradual. Esto suele requerir más árboles, pero puede mejorar la estabilidad.

`learning_rate` alto: aprendizaje rápido, correcciones fuertes, mayor riesgo.
`learning_rate` bajo: aprendizaje más lento, correcciones suaves, mayor control.

No hay una combinación universalmente correcta entre cantidad de árboles y tasa de aprendizaje. Ambos hiperparámetros interactúan. Un `learning_rate` bajo suele necesitar más árboles. Un `learning_rate` alto puede requerir menos árboles para no sobreajustar. La elección debe evaluarse con validación cruzada o una estrategia de validación adecuada.

También influye la complejidad de cada árbol. Si los árboles son muy profundos, cada uno puede aprender correcciones muy específicas. Si además usamos muchos árboles, el modelo final puede volverse excesivamente flexible.

Árboles pequeños: correcciones simples. Árboles profundos: correcciones más complejas, más riesgo de sobreajuste.

Por eso, Gradient Boosting suele usar árboles relativamente pequeños. Cada árbol no necesita resolver todo el problema. Solo debe aportar una mejora parcial. La potencia aparece por la acumulación de muchas correcciones, no por la complejidad extrema de cada árbol individual.

Este punto es importante porque marca una diferencia de enfoque. En un árbol de decisión individual, podríamos intentar aumentar la profundidad para capturar más patrones. En Gradient Boosting, muchas veces es preferible usar árboles pequeños y permitir que la secuencia de modelos construya la complejidad de manera gradual.

Gradient Boosting suele preferir: muchas correcciones pequeñas antes que pocas correcciones enormes.

Otra herramienta para controlar el sobreajuste es limitar la cantidad mínima de muestras en las hojas. Si permitimos hojas con muy pocos casos, cada árbol puede capturar detalles demasiado específicos. Al exigir más muestras por hoja, hacemos que las correcciones sean más generales.

Hojas con pocos casos: más detalle, más riesgo de ruido. Hojas con más casos: correcciones más estables.

Además, algunas implementaciones permiten usar una fracción de los datos o de las variables en cada paso, incorporando aleatoriedad. Esto puede ayudar a reducir el sobreajuste y mejorar la generalización, aunque los detalles dependen de la implementación utilizada.

La validación es clave. Con Gradient Boosting no conviene confiar solo en el resultado de entrenamiento. Tampoco conviene ajustar hiperparámetros mirando el conjunto de prueba final. Lo adecuado es usar validación cruzada o una separación de validación dentro del conjunto de entrenamiento para comparar configuraciones.

Gradient Boosting debe evaluarse con datos que no participaron en el ajuste de cada configuración.

Esto incluye el preprocesamiento. Si usamos imputación, codificación de variables categóricas u otras transformaciones, conviene trabajar con pipelines para evitar fuga de datos. Un modelo potente puede beneficiarse mucho de una filtración indebida y producir métricas artificialmente altas.

También debemos revisar errores individuales y segmentos. Un Gradient Boosting puede mejorar la métrica global, pero empeorar en ciertos grupos o rangos de valores. Por ejemplo, en regresión puede mejorar el error promedio, pero seguir fallando en valores extremos. En clasificación, puede mejorar accuracy, pero no necesariamente mejorar recall de la clase minoritaria.

Una mejora global no garantiza mejoras en todos los casos importantes.

Por eso, al usar modelos potentes, el análisis de errores se vuelve más importante, no menos. Una métrica alta puede ser buena noticia, pero debe acompañarse con revisión de estabilidad, clases minoritarias, valores extremos, casos difíciles y coherencia con el dominio.

La potencia de Gradient Boosting también puede hacerlo atractivo para competencias o problemas donde se busca máximo desempeño predictivo. Muchas implementaciones modernas de boosting han logrado resultados excelentes en datos tabulares. Pero en un contexto educativo o profesional, no alcanza con obtener el mejor número. Hay que poder justificar el flujo, documentar decisiones, evaluar correctamente y comunicar limitaciones.

Al usar modelos potentes, la responsabilidad de evaluar bien también aumenta.

Un modelo simple puede fallar por falta de capacidad. Un modelo potente puede fallar de una manera más sutil: dando resultados excelentes en una evaluación mal diseñada, pero funcionando peor en el uso real. Por eso, la potencia debe ir acompañada de criterio.

Gradient Boosting es potente porque construye un modelo final mediante muchas correcciones sucesivas. Esto le permite capturar patrones complejos y lograr muy buen desempeño en muchos problemas tabulares. Pero esa misma capacidad aumenta el riesgo de sobreajuste si se agregan demasiados árboles, si las correcciones son demasiado fuertes, si los árboles son demasiado complejos o si la evaluación está mal diseñada. Para usarlo correctamente, necesitamos controlar hiperparámetros, evaluar con validación adecuada, reservar el conjunto de prueba final y revisar no solo la métrica global, sino también los errores y la coherencia de las predicciones.

Hiperparámetros importantes

Gradient Boosting puede lograr muy buen desempeño, pero su comportamiento depende mucho de sus hiperparámetros. A diferencia de modelos más simples, donde unas pocas configuraciones suelen alcanzar para obtener una primera versión razonable, en Gradient Boosting la combinación de hiperparámetros puede cambiar de manera importante el equilibrio entre aprendizaje, estabilidad y sobreajuste.

Esto no significa que debamos ajustar todo desde el comienzo. Pero sí conviene conocer cuáles son las configuraciones principales y qué efecto general tienen sobre el modelo.

En Gradient Boosting, los hiperparámetros controlan: cuántas correcciones se agregan, qué tan fuerte corrige cada árbol, qué tan complejo es cada árbol, y cuánto riesgo de sobreajuste aceptamos.

Uno de los hiperparámetros más importantes es `n_estimators`. Este valor indica cuántos árboles se agregarán al modelo. Cada árbol representa una nueva corrección dentro de la secuencia.

```
from sklearn.ensemble import GradientBoostingClassifier

modelo = GradientBoostingClassifier(
    n_estimators=100,
    random_state=42
)
```

Si usamos pocos árboles, el modelo puede quedarse corto. No tendrá suficientes pasos para corregir los errores iniciales. En ese caso, puede aparecer subajuste.

Si usamos demasiados árboles, el modelo puede seguir corrigiendo detalles del entrenamiento incluso después de haber capturado los patrones principales. En ese caso, aumenta el riesgo de sobreajuste.

Pocos árboles: riesgo de subajuste. Muchos árboles: más capacidad, pero también más riesgo de sobreajuste.

La cantidad adecuada de árboles depende de otros hiperparámetros, especialmente de la tasa de aprendizaje. No conviene analizar `n_estimators` de manera aislada.

La tasa de aprendizaje, o `learning_rate`, controla cuánto aporta cada árbol nuevo al modelo final. Si el valor es alto, cada árbol tiene mucho peso. Si el valor es bajo, cada árbol aporta una corrección más pequeña.

```
modelo = GradientBoostingClassifier(
    n_estimators=200,
    learning_rate=0.05,
    random_state=42
)
```

Un `learning_rate` alto puede hacer que el modelo aprenda rápido, pero también que se adapte demasiado a errores particulares. Un `learning_rate` bajo hace que el aprendizaje sea más gradual, aunque suele requerir más árboles.

`learning_rate` alto: correcciones fuertes, aprendizaje rápido, más riesgo de sobreajuste. `learning_rate` bajo: correcciones suaves, aprendizaje gradual, más necesidad de árboles.

Por eso, `learning_rate` y `n_estimators` están muy relacionados. Una tasa baja con pocos árboles puede subajustar, porque el modelo no alcanza a corregir lo suficiente. Una tasa alta con muchos árboles puede sobreajustar, porque cada paso corrige fuerte y hay demasiados pasos.

Una combinación frecuente es usar un `learning_rate` moderado o bajo y aumentar la cantidad de árboles. Esto permite que el modelo aprenda en pasos pequeños.

Correcciones pequeñas + cantidad suficiente de árboles = aprendizaje más controlado.

Otro hiperparámetro importante es la profundidad de los árboles. En `scikit-learn`, puede controlarse con `max_depth`, dentro del árbol base que usa el modelo.

```
modelo = GradientBoostingClassifier(  
    n_estimators=200,  
    learning_rate=0.05,  
    max_depth=3,  
    random_state=42  
)
```

La profundidad define qué tan compleja puede ser cada corrección. Un árbol muy pequeño aprende reglas simples. Un árbol más profundo puede aprender interacciones más complejas entre variables.

En Gradient Boosting, muchas veces no necesitamos árboles muy profundos. Como el modelo final se construye sumando muchas correcciones, cada árbol puede ser relativamente simple.

Árboles poco profundos: correcciones simples, menor riesgo de sobreajuste. Árboles profundos: correcciones complejas, mayor riesgo de capturar ruido.

Esto diferencia a Gradient Boosting de un árbol de decisión individual. Un único árbol necesita suficiente profundidad para representar el problema. En boosting, la complejidad surge de la suma de muchos árboles. Por eso, usar árboles pequeños puede ser una buena estrategia.

También podemos controlar el tamaño mínimo de las hojas con `min_samples_leaf`. Este hiperparámetro indica cuántos casos, como mínimo, debe tener una hoja del árbol.

```
modelo = GradientBoostingClassifier(  
    n_estimators=200,  
    learning_rate=0.05,  
    max_depth=3,  
    min_samples_leaf=10,
```

```
        random_state=42
    )
```

Si permitimos hojas con muy pocos casos, los árboles pueden aprender correcciones muy específicas. Esto puede mejorar el ajuste en entrenamiento, pero también aumentar el riesgo de sobreajuste.

Si exigimos más casos por hoja, las correcciones serán más generales. Esto puede mejorar la estabilidad, aunque si el valor es demasiado alto el modelo puede perder capacidad para capturar patrones útiles.

`min_samples_leaf` bajo: más detalle, más flexibilidad. `min_samples_leaf` alto: más estabilidad, pero posible pérdida de patrones locales.

Otro hiperparámetro relacionado es `min_samples_split`, que indica cuántos casos debe tener un nodo para poder dividirse. Si aumentamos este valor, evitamos divisiones basadas en muy pocos ejemplos. Esto también puede funcionar como una forma de controlar la complejidad de los árboles.

```
modelo = GradientBoostingClassifier(
    n_estimators=200,
    learning_rate=0.05,
    max_depth=3,
    min_samples_split=20,
    random_state=42
)
```

Estos hiperparámetros no deben elegirse de manera aislada. Una profundidad mayor puede necesitar más control en las hojas. Una tasa de aprendizaje baja puede combinarse con más árboles. Una tasa alta puede necesitar menos árboles o árboles más simples.

En Gradient Boosting, los hiperparámetros interactúan. Cambiar uno puede modificar el efecto de los demás.

También aparece el hiperparámetro `subsample`. Este valor indica qué proporción de los datos se usa para entrenar cada árbol. Si `subsample=1.0`, cada árbol usa todos los datos disponibles en esa etapa. Si usamos un valor menor, como `0.8`, cada árbol se entrena con una fracción de los datos.

```
modelo = GradientBoostingClassifier(
    n_estimators=200,
    learning_rate=0.05,
    max_depth=3,
    subsample=0.8,
    random_state=42
)
```

Usar `subsample` menor que 1.0 introduce aleatoriedad. Esto puede ayudar a reducir el sobreajuste, porque los árboles no se entrenan siempre con exactamente los mismos datos. Esta variante suele llamarse **Stochastic Gradient Boosting**.

`subsample` menor que 1: agrega aleatoriedad, puede mejorar la generalización, pero también vuelve el entrenamiento más dependiente de la muestra elegida.

No siempre es necesario modificar `subsample`, pero es útil conocerlo como una herramienta adicional de control.

En problemas de clasificación, también importa la métrica que usamos para evaluar las configuraciones. Si ajustamos hiperparámetros usando accuracy en un dataset desbalanceado, la búsqueda puede favorecer modelos que funcionan bien para la clase mayoritaria y mal para la minoritaria. Si el objetivo es detectar la clase positiva, puede convenir usar F1-score, recall o precision, según el costo de los errores.

El mejor conjunto de hiperparámetros depende de la métrica elegida. La métrica debe depender del problema.

En regresión ocurre algo similar. Ajustar con MAE puede favorecer errores promedio más moderados. Ajustar con RMSE penaliza más los errores grandes. Ajustar con R^2 responde a otra forma de medir desempeño. La configuración elegida puede cambiar según la métrica.

Por eso, antes de buscar hiperparámetros, necesitamos definir qué significa “mejor” para el problema. Gradient Boosting puede optimizar muy bien una métrica, pero si esa métrica no representa la necesidad real, el modelo elegido no será necesariamente adecuado.

Una búsqueda de hiperparámetros podría incluir valores como:

Concepto	Descripción
n_estimators	100, 200, 300
learning_rate	0.01, 0.05, 0.1
max_depth	2, 3, 4
min_samples_leaf	1, 5, 10

La cantidad de combinaciones crece rápido. En este ejemplo habría:

$$3 \times 3 \times 3 \times 3 = 81 \text{ combinaciones}$$

Si usamos validación cruzada de 5 folds, eso implica:

$81 \times 5 = 405$ entrenamientos

Esto muestra por qué ajustar Gradient Boosting puede ser costoso. No conviene probar combinaciones sin criterio. Es preferible comenzar con rangos razonables, observar resultados y ajustar de manera progresiva.

Búsquedas enormes sin criterio pueden consumir mucho tiempo y aumentar el riesgo de elegir una configuración poco robusta.

Una estrategia práctica es empezar con una búsqueda pequeña. Por ejemplo, probar algunas combinaciones de `learning_rate` y `n_estimators`, manteniendo árboles pequeños. Luego, si el modelo muestra potencial, explorar con más detalle profundidad, hojas mínimas o `subsample`.

También puede ser útil comparar contra modelos ya estudiados. Si Gradient Boosting mejora apenas a Random Forest, pero requiere mucho más ajuste y tiempo, quizá no justifique su complejidad. Si mejora claramente y la evaluación es estable, puede ser una opción muy fuerte.

La mejora de un modelo más complejo debe ser suficiente para justificar su costo y su menor simplicidad.

En `scikit-learn`, podemos ajustar hiperparámetros de Gradient Boosting con `GridSearchCV` o `RandomizedSearchCV`, igual que con otros modelos.

```
from sklearn.model_selection import GridSearchCV
from sklearn.ensemble import GradientBoostingClassifier

modelo = GradientBoostingClassifier(random_state=42)

param_grid = {
    "n_estimators": [100, 200],
    "learning_rate": [0.05, 0.1],
    "max_depth": [2, 3],
    "min_samples_leaf": [1, 5]
}

busqueda = GridSearchCV(
    estimator=modelo,
    param_grid=param_grid,
    cv=5,
    scoring="f1"
)

busqueda.fit(X_train, y_train)
```

Este flujo busca la mejor combinación dentro de la grilla definida, usando validación cruzada sobre el conjunto de entrenamiento. El conjunto de prueba final debe reservarse para evaluar después el modelo elegido.

Si hay preprocesamiento, conviene usar un pipeline. Esto permite aplicar transformaciones dentro de cada fold y evitar fuga de datos. Gradient Boosting no necesita escalado numérico de la misma manera que KNN o modelos lineales regularizados, pero sí puede necesitar imputación, codificación de categorías u otras transformaciones según el dataset.

Aunque Gradient Boosting sea potente, el preprocesamiento y la evaluación siguen siendo parte del modelo real.

Los hiperparámetros más importantes de Gradient Boosting controlan la cantidad de árboles, el peso de cada corrección, la complejidad de cada árbol y el grado de aleatoriedad del entrenamiento. `n_estimators`, `learning_rate`, `max_depth`, `min_samples_leaf`, `min_samples_split` y `subsample` influyen en el equilibrio entre aprendizaje y sobreajuste. Estos valores deben elegirse con validación adecuada, métricas coherentes con el problema y cuidado para no usar el conjunto de prueba final durante la búsqueda.

Panorama de implementaciones populares

Gradient Boosting no es una única implementación. Es una familia de métodos que comparten una idea general: construir un modelo fuerte agregando modelos más simples de manera secuencial, donde cada nuevo modelo intenta corregir errores o debilidades de los anteriores.

Esa idea puede implementarse de distintas formas. Algunas implementaciones son más simples y didácticas. Otras están muy optimizadas para trabajar con grandes volúmenes de datos, muchas variables, valores faltantes, variables categóricas o entrenamiento eficiente.

Gradient Boosting: idea general. Implementaciones: formas concretas de aplicar esa idea en distintas bibliotecas.

En `scikit-learn`, una implementación clásica es `GradientBoostingClassifier` para clasificación y `GradientBoostingRegressor` para regresión.

```
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.ensemble import GradientBoostingRegressor
```

Estas clases son útiles para aprender la lógica del modelo y trabajar con problemas de tamaño moderado. Permiten controlar hiperparámetros como `n_estimators`, `learning_rate`, `max_depth`, `min_samples_leaf` y `subsample`, entre otros.

Por ejemplo:

```
from sklearn.ensemble import GradientBoostingClassifier

modelo = GradientBoostingClassifier(
    n_estimators=200,
    learning_rate=0.05,
    max_depth=3,
    random_state=42
)
```

Este modelo representa muy bien la idea general de boosting: muchos árboles pequeños, agregados de manera secuencial, con una tasa de aprendizaje que controla cuánto aporta cada uno.

Sin embargo, `scikit-learn` también incluye una familia más moderna y eficiente basada en histogramas: `HistGradientBoostingClassifier` y `HistGradientBoostingRegressor`.

```
from sklearn.ensemble import HistGradientBoostingClassifier
from sklearn.ensemble import HistGradientBoostingRegressor
```

Estas implementaciones suelen ser más rápidas en datasets medianos o grandes. Internamente utilizan una estrategia basada en agrupar valores continuos en intervalos, lo que permite acelerar el entrenamiento. Además, incorporan algunas capacidades prácticas adicionales, como manejo más eficiente de ciertos tipos de datos y valores faltantes, según el caso.

GradientBoosting: implementación clásica. HistGradientBoosting: implementación más eficiente para muchos problemas tabulares.

Para un curso introductorio, conviene empezar por la idea conceptual y por ejemplos simples. No necesitamos dominar todas las diferencias internas. Lo importante es reconocer que, incluso dentro de `scikit-learn`, hay más de una forma de trabajar con Gradient Boosting.

Fuera de `scikit-learn`, existen implementaciones muy populares en el mundo profesional y en competencias de ciencia de datos. Las más conocidas son **XGBoost**, **LightGBM** y **CatBoost**.

Implementaciones populares: XGBoost LightGBM CatBoost.

XGBoost significa *Extreme Gradient Boosting*. Es una de las implementaciones más difundidas de boosting con árboles. Se volvió muy popular por su rendimiento, su flexibilidad y su buen desempeño en problemas tabulares. Durante años fue una

herramienta muy frecuente en competencias de Machine Learning y también en proyectos aplicados.

XGBoost incorpora varias optimizaciones y controles de regularización. Permite ajustar muchos hiperparámetros relacionados con la cantidad de árboles, la profundidad, la tasa de aprendizaje, la regularización, el muestreo de filas, el muestreo de columnas y otros aspectos del entrenamiento.

XGBoost: implementación potente y flexible de Gradient Boosting con árboles.

Esa flexibilidad es una ventaja, pero también exige criterio. Cuantos más hiperparámetros tenemos disponibles, más importante se vuelve evaluar bien, evitar búsquedas sin sentido y no usar el conjunto de prueba final para elegir configuraciones.

LightGBM es otra implementación muy usada. Fue diseñada para ser eficiente, especialmente en datasets grandes. Su nombre viene de *Light Gradient Boosting Machine*. Suele destacarse por su velocidad de entrenamiento y su capacidad para trabajar con grandes volúmenes de datos.

LightGBM: implementación orientada a eficiencia y buen rendimiento en datasets grandes.

LightGBM tiene algunas decisiones internas diferentes respecto de otras implementaciones, por ejemplo en la forma de construir árboles y organizar los datos durante el entrenamiento. No necesitamos entrar en esos detalles ahora. Para este capítulo, alcanza con saber que es una herramienta muy usada cuando se busca buen desempeño con entrenamiento eficiente.

CatBoost es otra implementación popular, especialmente conocida por su tratamiento de variables categóricas. Su nombre viene de *Categorical Boosting*. En muchos problemas tabulares, las variables categóricas son muy importantes: ciudad, producto, sector, tipo de cliente, medio de pago, categoría de reclamo, entre muchas otras.

CatBoost: implementación de boosting con especial atención a variables categóricas.

En flujos tradicionales con `scikit-learn`, muchas veces transformamos variables categóricas mediante One-Hot Encoding. CatBoost ofrece estrategias propias para trabajar con este tipo de variables de manera eficiente. Esto puede ser una ventaja en datasets con muchas categorías o con variables categóricas de alta cardinalidad.

Aunque XGBoost, LightGBM y CatBoost son herramientas muy potentes, no conviene presentarlas como “mejores” en todos los casos. Cada una tiene fortalezas,

requisitos, hiperparámetros y formas de uso. Además, un modelo más avanzado no resuelve automáticamente problemas de datos, fuga, evaluación o mala definición del objetivo.

Una implementación más potente no reemplaza: datos bien preparados, métricas adecuadas, validación correcta, interpretación prudente.

Para un estudiante que recién se acerca a Machine Learning, puede ser más conveniente comenzar con las implementaciones de `scikit-learn`, porque mantienen una interfaz familiar: `fit`, `predict`, `predict_proba`, pipelines, validación cruzada y búsqueda de hiperparámetros. Luego, cuando la idea está clara, se puede explorar XGBoost, LightGBM o CatBoost.

Esta progresión evita confundir dos aprendizajes distintos. Primero debemos entender qué hace el modelo. Después podemos estudiar qué ventajas ofrece cada biblioteca.

Primero: entender la idea de boosting. Después: comparar implementaciones y optimizaciones.

También es importante considerar el contexto de uso. En un proyecto educativo, `GradientBoostingClassifier` o `HistGradientBoostingClassifier` pueden ser suficientes para comprender el proceso. En un proyecto profesional con muchos datos, restricciones de tiempo o necesidad de máximo rendimiento, puede tener sentido explorar implementaciones especializadas.

Por ejemplo, si estamos trabajando con un dataset tabular mediano y queremos enseñar el funcionamiento del modelo, una implementación de `scikit-learn` puede ser adecuada. Si trabajamos con millones de filas, muchas variables y necesitamos optimizar desempeño, tal vez LightGBM o XGBoost resulten más convenientes. Si hay muchas variables categóricas con muchas categorías, CatBoost puede ser una opción interesante.

La elección de implementación depende de: tamaño del dataset, tipo de variables, tiempo disponible, necesidad de interpretación, experiencia del equipo, objetivo del proyecto.

En todos los casos, la comparación debe hacerse de manera justa. No alcanza con entrenar una implementación con hiperparámetros por defecto y otra con una búsqueda cuidadosa, y luego concluir que una es mejor. Tampoco conviene comparar modelos usando métricas distintas o divisiones de datos diferentes.

Si queremos comparar implementaciones, debemos mantener el mismo conjunto de datos, la misma variable objetivo, la misma estrategia de validación y una métrica coherente con el problema.

Comparación justa: mismos datos, misma métrica, misma validación, mismo rol del conjunto de prueba.

Otro punto a tener en cuenta es la interpretabilidad. Los modelos de boosting pueden ofrecer importancia de variables y otras herramientas de análisis, pero siguen siendo modelos de ensamble complejos. No son tan fáciles de explicar como una regresión lineal o un árbol pequeño. Cuanto más optimizada y compleja es la implementación, más importante es complementar la métrica con análisis de errores, revisión de variables e interpretación prudente.

También debemos cuidar el sobreajuste. Algunas implementaciones modernas ofrecen muchos mecanismos para controlarlo: regularización, muestreo de filas, muestreo de columnas, profundidad máxima, cantidad mínima de muestras, tasa de aprendizaje, detención temprana y otros. Pero esos mecanismos deben usarse con criterio. La herramienta ofrece posibilidades; el diseño correcto sigue siendo responsabilidad de quien modela.

Más opciones de ajuste significan más control, pero también más decisiones que cuidar.

Un concepto que suele aparecer en implementaciones de boosting es la **detención temprana**, o *early stopping*. La idea es detener el entrenamiento cuando el desempeño en validación deja de mejorar. Esto puede ayudar a evitar que el modelo siga agregando árboles cuando ya no aporta mejoras reales.

No necesitamos desarrollarlo en profundidad ahora, pero conviene mencionarlo porque se relaciona directamente con el riesgo de sobreajuste. Si el modelo sigue entrenando después de que la validación dejó de mejorar, puede empezar a ajustar ruido.

Early stopping: detener el entrenamiento cuando la validación deja de mejorar.

Este mecanismo es muy útil, pero también debe aplicarse correctamente. Necesita una separación de validación adecuada y no debe usar el conjunto de prueba final como guía para detener el entrenamiento.

Gradient Boosting es una idea general que puede implementarse de varias maneras. `scikit-learn` ofrece implementaciones clásicas y versiones basadas en histogramas. Fuera de `scikit-learn`, XGBoost, LightGBM y CatBoost son herramientas muy populares y potentes para datos tabulares. Cada una tiene ventajas y particularidades, pero todas deben usarse con el mismo cuidado metodológico: buena preparación de datos, validación adecuada, elección correcta de métricas, control del sobreajuste y evaluación final sobre datos reservados. La implementación puede mejorar el rendimiento, pero no reemplaza el criterio necesario para construir modelos confiables.

Corregir errores paso a paso

En este capítulo estudiamos Gradient Boosting, un modelo de ensamble que combina muchos modelos simples de manera secuencial. A diferencia de Random Forest, donde los árboles se entrenan de forma relativamente independiente, Gradient Boosting construye el modelo paso a paso: cada nuevo árbol intenta corregir errores o debilidades de los anteriores.

La idea central del boosting es que muchos modelos débiles, agregados de manera ordenada, pueden formar un modelo final muy potente.

Gradient Boosting: muchas correcciones pequeñas construyen un modelo fuerte.

En regresión, esta lógica puede imaginarse como una predicción inicial a la que se le van sumando correcciones. El primer modelo se equivoca en algunos casos; el siguiente intenta reducir parte de esos errores; los modelos posteriores siguen ajustando lo que todavía falta mejorar. En clasificación, la lógica también es progresiva, aunque el cálculo interno sea distinto.

Esta forma de aprendizaje hace que Gradient Boosting sea muy flexible. Puede capturar relaciones no lineales, interacciones entre variables y patrones complejos. Por eso suele funcionar muy bien en problemas tabulares y aparece con frecuencia en aplicaciones reales y competencias de ciencia de datos.

Pero esa potencia tiene un riesgo: el sobreajuste. Si el modelo agrega demasiados árboles, si cada corrección pesa demasiado o si los árboles son demasiado complejos, puede empezar a ajustar ruido y detalles específicos del conjunto de entrenamiento.

Corregir errores reales: mejora la generalización. Corregir ruido: aumenta el riesgo de sobreajuste.

Por eso, los hiperparámetros de Gradient Boosting son especialmente importantes. `n_estimators` controla cuántos árboles se agregan. `learning_rate` controla cuánto aporta cada árbol. `max_depth` regula la complejidad de cada árbol. `min_samples_leaf`, `min_samples_split` y `subsample` ayudan a controlar la estabilidad, la flexibilidad y el grado de aleatoriedad del entrenamiento.

Estos hiperparámetros interactúan entre sí. No conviene ajustar uno sin pensar en los demás. Una tasa de aprendizaje baja suele requerir más árboles. Una tasa alta puede necesitar menos árboles. Árboles más profundos pueden capturar relaciones más complejas, pero también aumentar el riesgo de sobreajuste.

Gradient Boosting requiere equilibrio entre: cantidad de árboles, tamaño de cada corrección, complejidad de los árboles, y evaluación honesta.

También comparamos Gradient Boosting con Random Forest. Ambos son ensambles basados en árboles, pero tienen lógicas distintas. Random Forest combina árboles diversos entrenados en paralelo y reduce principalmente la inestabilidad de un árbol individual. Gradient Boosting construye árboles en secuencia y busca mejorar progresivamente los errores del modelo acumulado.

Random Forest: muchos árboles votan o promedian. Gradient Boosting: muchos árboles corrigen paso a paso.

Random Forest suele ser más robusto como punto de partida y menos sensible a la configuración inicial. Gradient Boosting puede alcanzar desempeños muy altos, pero suele requerir más cuidado en el ajuste de hiperparámetros y en la validación.

También vimos que Gradient Boosting no es una única herramienta. Es una familia de métodos con distintas implementaciones. En `scikit-learn` existen `GradientBoostingClassifier`, `GradientBoostingRegressor`, `HistGradientBoostingClassifier` y `HistGradientBoostingRegressor`. Fuera de `scikit-learn`, herramientas como XGBoost, LightGBM y CatBoost son muy populares por su eficiencia y desempeño en datos tabulares.

Gradient Boosting: idea general. XGBoost, LightGBM, CatBoost: implementaciones populares de esa idea.

Sin embargo, una implementación más potente no reemplaza el criterio. Todas estas herramientas necesitan datos bien preparados, variables válidas, métricas adecuadas, validación correcta y una evaluación final sobre datos reservados. Un modelo potente puede aprender señales útiles, pero también puede aprovechar fuga de datos o errores metodológicos con mucha eficacia.

Gradient Boosting construye un modelo fuerte mediante una secuencia de correcciones. Su potencia lo convierte en una herramienta muy valiosa para problemas de clasificación y regresión, especialmente con datos tabulares. Pero esa misma potencia exige cuidado: controlar hiperparámetros, evitar sobreajuste, usar validación adecuada, no ajustar sobre el conjunto de prueba final y analizar los errores más allá de una única métrica. Bien utilizado, Gradient Boosting puede ser uno de los modelos más efectivos del flujo de Machine Learning clásico.

Capítulo 23 · Support Vector Machines

La idea del margen

Support Vector Machines, o SVM, es una familia de modelos que puede usarse para clasificación y también para regresión. En este capítulo nos concentraremos principalmente en su intuición para clasificación, porque allí aparece con más claridad su idea central: separar clases buscando un **margen** lo más amplio posible.

Para empezar, imaginemos un problema binario con dos clases. Por ejemplo, clientes que abandonan y clientes que no abandonan, correos spam y correos legítimos, operaciones fraudulentas y operaciones normales. Si representamos los datos usando dos variables, podemos imaginar cada caso como un punto en un plano.

Cada punto representa un caso del dataset, mientras que su color o clase indica la categoría real de ese caso.

Un clasificador intenta separar esos puntos para poder decidir a qué clase pertenece un caso nuevo. Si las clases pueden separarse con una línea, podríamos trazar distintas rectas entre ellas. Algunas rectas separan bien los puntos. Otras los separan mal. Pero incluso entre las rectas que separan correctamente, no todas son igual de convenientes.

SVM propone una pregunta muy interesante:

Si existen varias formas de separar las clases, aparece entonces una pregunta importante: ¿cuál de esas separaciones conviene elegir?

La respuesta de SVM es: elegir la separación que deje el mayor margen posible entre las clases.

El **margen** es la distancia entre la frontera de decisión y los puntos más cercanos de cada clase. En un problema de dos dimensiones, la frontera puede imaginarse como una línea. SVM busca una línea que no solo separe las clases, sino que quede lo más lejos posible de los puntos más cercanos.

SVM no busca solamente separar las clases, sino hacerlo con el mayor margen posible.

Esta idea es importante porque una separación con margen amplio suele ser más robusta. Si la frontera queda demasiado cerca de algunos puntos, pequeños cambios

en los datos podrían cambiar la clasificación. En cambio, si la frontera queda más alejada de los puntos de ambas clases, el modelo tiene más “espacio” para decidir.

Podemos pensarlo con una analogía. Si dos grupos de personas están ubicados en una sala y queremos marcar una línea divisoria en el piso, no conviene trazarla pegada a uno de los grupos. Conviene ubicarla de manera que deje espacio razonable hacia ambos lados. Esa separación sería menos frágil.

Tipo de margen	Consecuencia
Margen pequeño	La frontera queda cerca de los datos.
Margen amplio	La frontera queda más separada de los puntos cercanos.

En un plano, podríamos imaginar varias líneas posibles. Algunas separan las clases, pero pasan muy cerca de ciertos puntos. SVM prefiere la línea que maximiza la distancia a los puntos más cercanos. Esa línea se conoce como **hiperplano de separación**.

La palabra hiperplano puede sonar extraña, pero la idea es simple. En dos dimensiones, el hiperplano es una recta. En tres dimensiones, es un plano. En más dimensiones, no podemos visualizarlo fácilmente, pero matemáticamente cumple la misma función: separar regiones del espacio de variables.

Cantidad de variables	Forma de la frontera
2 variables	Puede verse como una recta.
3 variables	Puede verse como un plano.
Muchas variables	Es un hiperplano.

En Machine Learning, cada variable predictora agrega una dimensión al espacio donde ubicamos los datos. Si usamos dos variables, podemos dibujar puntos en un plano. Si usamos tres, podríamos imaginar puntos en un espacio tridimensional. Si usamos diez, veinte o cien variables, ya no podemos visualizarlo directamente, pero el modelo sigue trabajando con esa representación.

SVM busca, en ese espacio, una frontera que separe las clases con el mayor margen posible.

Los puntos más cercanos a la frontera son especialmente importantes. Se llaman **support vectors**, o vectores de soporte. Son los casos que sostienen la posición de la frontera. Si esos puntos cambiaran, la frontera podría cambiar. En cambio, puntos que están muy lejos de la frontera influyen menos en la decisión final.

Los vectores de soporte son los puntos cercanos a la frontera que ayudan a definir el margen.

Esta es una diferencia interesante respecto de otros modelos. SVM no necesita que todos los puntos tengan la misma influencia en la frontera. Los casos más críticos son aquellos que están cerca de la separación entre clases. Esos puntos son los que definen cuán amplio puede ser el margen.

Supongamos que tenemos dos grupos claramente separados. Los puntos que están en el centro de cada grupo no generan demasiada duda. Los casos difíciles son los que están cerca de la región donde ambas clases se aproximan. SVM presta especial atención a esos casos porque allí se define la frontera.

Tipo de punto	Ubicación e importancia
Puntos fáciles	Están lejos de la frontera.
Puntos críticos	Están cerca de la frontera y ayudan a definir la separación.

La idea del margen también ayuda a entender por qué SVM puede generalizar bien en ciertos problemas. En lugar de buscar una separación cualquiera que funcione en entrenamiento, busca una separación que deje espacio. Ese espacio funciona como una forma de protección frente a pequeñas variaciones.

Sin embargo, en datos reales no siempre es posible separar perfectamente las clases. Muchas veces hay puntos mezclados, errores de medición, ruido o casos ambiguos. Si exigimos una separación perfecta, el modelo podría terminar construyendo una frontera demasiado rígida o directamente no encontrar una separación razonable.

Por eso, en la práctica, SVM permite cierto grado de error. Esta idea se conoce como **margen suave**, o *soft margin*. En lugar de obligar a que todos los puntos queden perfectamente del lado correcto, el modelo permite algunas violaciones del margen o incluso algunas clasificaciones incorrectas, si eso ayuda a obtener una separación más general.

Tipo de margen	Comportamiento
Margen rígido	Intenta separar todos los casos perfectamente.
Margen suave	Permite ciertos errores para lograr una separación más robusta.

Esta flexibilidad es fundamental. En problemas reales, insistir en clasificar perfectamente todos los casos de entrenamiento puede llevar al sobreajuste. Un margen suave permite equilibrar dos objetivos: separar bien las clases y mantener una frontera razonable.

Ese equilibrio se controla mediante un hiperparámetro llamado C .

El hiperparámetro C controla cuánto penalizamos los errores o las violaciones del margen.

Si C es muy grande, el modelo penaliza mucho los errores. Entonces intentará clasificar correctamente la mayor cantidad posible de puntos de entrenamiento, incluso si eso reduce el margen o produce una frontera más ajustada.

Si C es más pequeño, el modelo tolera más errores. Esto puede permitir un margen más amplio y una frontera más simple, aunque algunos puntos de entrenamiento queden mal clasificados.

Valor de C	Comportamiento
C grande	Menor tolerancia a errores y mayor ajuste al entrenamiento.
C pequeño	Mayor tolerancia a errores y un margen más flexible.

Este hiperparámetro se relaciona con ideas que ya vimos: sobreajuste, subajuste, regularización e hiperparámetros. Un valor de C demasiado alto puede hacer que el modelo se ajuste demasiado al entrenamiento. Un valor demasiado bajo puede volverlo demasiado tolerante y producir subajuste.

No existe un valor universal de C . Debe evaluarse según el problema, los datos y la métrica elegida. Como siempre, la búsqueda de hiperparámetros debe hacerse con validación cruzada dentro del conjunto de entrenamiento, no mirando el conjunto de prueba final.

La idea del margen puede parecer geométrica, pero su importancia es práctica. Cuando SVM busca una frontera con margen amplio, está intentando construir una regla de decisión que no dependa de una separación frágil. Esto puede ser útil en datasets donde las clases tienen una estructura clara en el espacio de variables.

SVM busca una frontera que separe las clases manteniendo cierta distancia de seguridad.

Sin embargo, si las variables no están bien preparadas, esa geometría puede volverse engañosa. SVM depende fuertemente de las escalas de las variables, especialmente cuando usa distancias, productos internos o kernels. Una variable con valores grandes puede dominar el cálculo. Por eso, el escalado será un tema central más adelante en este capítulo.

Por ahora, lo importante es comprender la intuición inicial: SVM no solo pregunta de qué lado de la frontera cae un punto. También pregunta cuán buena es esa frontera en términos de margen.

La frontera importa, pero también importa qué tan lejos quedan de ella los puntos más cercanos.

Support Vector Machines se basa en la idea de separar clases buscando un margen amplio. En un problema de dos variables, podemos imaginar una recta que separa los puntos. En más dimensiones, esa frontera se generaliza como un hiperplano. Los puntos más cercanos a la frontera se llaman vectores de soporte y son los que ayudan a definir la separación. En datos reales, SVM suele trabajar con un margen suave, permitiendo ciertos errores para lograr una frontera más robusta. Esta combinación de separación y margen es la base conceptual del modelo.

Separación entre clases

La idea del margen nos permite entender qué intenta hacer SVM cuando las clases pueden separarse. Pero para completar la intuición, conviene detenernos en la separación entre clases. Después de todo, un modelo de clasificación necesita decidir de qué lado queda cada caso.

En un problema binario, tenemos dos clases. Si trabajamos con dos variables, podemos imaginar los datos como puntos en un plano. Algunos puntos pertenecen a una clase y otros a la otra. El objetivo del modelo es encontrar una frontera que divida el espacio en dos regiones.

Región	Clase asignada
Un lado de la frontera	Clase 0
Otro lado de la frontera	Clase 1

Cuando llega un nuevo caso, el modelo observa sus variables, lo ubica en ese espacio y decide según la región en la que cae. Si queda de un lado, predice una clase. Si queda del otro, predice la clase contraria.

Para clasificar un nuevo punto, primero se lo ubica en el espacio de variables. Según el lado de la frontera en el que quede, se le asigna una clase.

En los casos más simples, las clases pueden separarse con una línea recta. A esto se lo llama separación lineal. En dos dimensiones, la frontera es una recta. En tres dimensiones, sería un plano. En más dimensiones, hablamos de un hiperplano.

Separación lineal: las clases pueden dividirse con una frontera recta o plana.

Por ejemplo, imaginemos un problema donde queremos clasificar frutas según dos variables: peso y diámetro. Si las manzanas tienden a ser más grandes y pesadas, y las ciruelas más pequeñas y livianas, tal vez una línea pueda separar razonablemente ambos grupos. Esa línea no necesita pasar por los puntos; simplemente divide el espacio en dos zonas.

En ese caso, SVM buscaría no solo una línea que separe, sino una línea que deje el mayor margen posible.

No todas las separaciones lineales son equivalentes: SVM prefiere aquella que separa las clases con un margen mayor.

Pero en problemas reales, las clases rara vez se presentan como dos grupos perfectamente separados. Puede haber superposición. Algunos puntos de una clase pueden aparecer dentro de una zona donde predominan puntos de la otra. Puede haber ruido, errores en las etiquetas, variables insuficientes o simplemente casos ambiguos.

En los datos reales pueden aparecer clases parcialmente mezcladas, casos atípicos, ruido y zonas ambiguas.

Cuando las clases se mezclan, una frontera perfecta puede no existir. O puede existir solo si construimos una frontera demasiado compleja, ajustada a detalles particulares del entrenamiento. Eso podría mejorar el resultado sobre los datos conocidos, pero perjudicar la generalización.

SVM enfrenta este problema usando la idea de margen suave. En lugar de exigir que todos los casos queden perfectamente clasificados, permite algunos errores o violaciones del margen. Esto hace que la separación sea más flexible.

Separar bien no siempre significa clasificar perfectamente todos los casos de entrenamiento.

Esta idea es importante. A veces, para construir un buen modelo, debemos aceptar que algunos casos de entrenamiento quedarán mal clasificados. Si esos casos son atípicos, ruidosos o ambiguos, forzar al modelo a clasificarlos correctamente puede empeorar la frontera general.

Por ejemplo, supongamos que casi todos los puntos de la clase A están a la izquierda y casi todos los de la clase B están a la derecha, pero hay un punto de la clase A muy metido entre los de la clase B. Si intentamos clasificar perfectamente ese punto, la frontera puede torcerse demasiado. Tal vez sea mejor aceptar ese error y mantener una separación más estable.

Aceptar algunos errores puede ayudar a construir una frontera más razonable.

El hiperparámetro C controla este equilibrio. Si C es alto, el modelo penaliza mucho los errores y tratará de clasificar correctamente más puntos del entrenamiento. Si C es bajo, tolerará más errores para favorecer un margen más amplio o una frontera menos forzada.

Valor de C	Consecuencia
C alto	Los errores se penalizan mucho.
C bajo	Se toleran más errores para mantener una separación más flexible.

Esto no significa que un valor bajo de C sea siempre mejor. Si toleramos demasiados errores, el modelo puede volverse demasiado simple y subajustar. Tampoco significa que un valor alto sea siempre mejor. Si penalizamos demasiado los errores, el modelo puede sobreajustar.

Como en otros casos, el valor adecuado debe buscarse mediante validación.

La separación entre clases también depende de las variables disponibles. Si las variables contienen buena señal, puede existir una frontera razonable. Si las variables no permiten distinguir las clases, ningún modelo podrá separarlas bien de manera confiable.

Una buena frontera requiere variables que contengan información útil.

Por ejemplo, si queremos clasificar correos como spam o no spam, variables como presencia de ciertos términos, remitente, enlaces o patrones de escritura pueden aportar señal. Si solo usamos una variable irrelevante, como la longitud exacta del asunto sin más contexto, tal vez la separación sea pobre.

El modelo no inventa información. Solo puede construir fronteras a partir de las variables que recibe.

También puede ocurrir que las clases no sean separables linealmente en el espacio original de variables. Es decir, no hay una recta o un plano que las divida bien. Esto no significa necesariamente que SVM no sirva. Una de sus fortalezas es que puede

trabajar con transformaciones llamadas kernels, que permiten construir fronteras no lineales. Veremos esa idea en la próxima sección.

Por ahora, podemos quedarnos con esta distinción:

Tipo de separación	Frontera necesaria
Lineal	Una frontera recta o plana es suficiente.
No lineal	Se necesita una frontera más flexible.

La frontera de decisión es importante porque determina cómo se clasifican los nuevos casos. Pero también importa cómo se obtuvo esa frontera. Una separación que funciona muy bien en entrenamiento puede no generalizar si está demasiado ajustada. Una separación más simple, aunque cometa algunos errores, puede funcionar mejor con datos nuevos.

Esta idea conecta con todo lo que venimos estudiando: generalización, sobreajuste, regularización, validación cruzada e hiperparámetros. SVM no está aislado de esos conceptos. Su manera de separar clases también debe evaluarse con datos no vistos.

La pregunta no debe ser solamente si el modelo separó bien los datos de entrenamiento, sino también si podrá separar correctamente datos nuevos.

En `scikit-learn`, un clasificador SVM lineal puede representarse con `SVC` usando un kernel lineal:

```
from sklearn.svm import SVC

modelo = SVC(
    kernel="linear",
    C=1.0
)
```

También existe `LinearSVC`, una implementación pensada específicamente para SVM lineal:

```
from sklearn.svm import LinearSVC

modelo = LinearSVC(C=1.0)
```

No necesitamos comparar ahora todos los detalles entre ambas opciones. Lo importante es reconocer que SVM puede buscar fronteras lineales, y que el parámetro `C` controla el equilibrio entre margen amplio y penalización de errores.

Cuando usamos SVM, la evaluación debe realizarse con las mismas precauciones que en otros modelos. Debemos separar entrenamiento y prueba, ajustar hiperparámetros

con validación cruzada y cuidar el preprocesamiento. Además, el escalado de variables será especialmente importante, porque la separación depende de distancias y márgenes en el espacio de variables.

En SVM importa la geometría del espacio y, por esa razón, también importa la escala de las variables.

SVM clasifica construyendo una frontera que separa las clases. En problemas simples, esa frontera puede ser lineal: una recta, un plano o un hiperplano. En datos reales, las clases pueden mezclarse, por lo que SVM permite márgenes suaves y cierta tolerancia a errores. El hiperparámetro C controla el equilibrio entre penalizar errores y mantener una separación más flexible. La calidad de la separación depende de las variables disponibles, del preprocesamiento y de una evaluación cuidadosa con datos no vistos.

Kernels

Hasta ahora imaginamos a SVM como un modelo que busca una frontera lineal: una recta en dos dimensiones, un plano en tres dimensiones o un hiperplano cuando hay muchas variables. Esa idea es muy importante, pero no alcanza para todos los problemas.

En muchos datasets, las clases no pueden separarse bien con una frontera recta. Puede ocurrir que una clase quede rodeada por otra, que los puntos formen curvas, que haya regiones mezcladas o que la separación dependa de combinaciones no lineales entre variables.

Tipo de separación	Frontera necesaria
Lineal	Una frontera recta o plana es suficiente.
No lineal	Se necesita una frontera más flexible.

Supongamos un ejemplo sencillo. Imaginemos puntos de una clase ubicados cerca del centro de un plano y puntos de otra clase ubicados formando un anillo alrededor. No existe una línea recta que separe bien el centro del anillo. Cualquier recta dejaría puntos mezclados de ambos lados.

Sin embargo, visualmente sí vemos una estructura: una clase está adentro y la otra afuera. El problema no es que no haya patrón. El problema es que el patrón no se separa con una línea recta en el espacio original.

Que un problema no sea separable linealmente no significa que no exista un patrón, sino que una frontera recta no alcanza para representarlo.

Aquí aparece la idea de los **kernels**. Un kernel permite que SVM construya fronteras no lineales sin que tengamos que crear manualmente todas las transformaciones de variables.

La intuición es la siguiente: a veces, si miramos los datos en otro espacio, la separación se vuelve más simple. Un patrón que parece difícil en el espacio original puede volverse separable si transformamos las variables.

La idea general consiste en transformar la forma de mirar los datos para que la separación resulte más sencilla.

Volvamos al ejemplo del centro y el anillo. En el plano original, una recta no separa bien. Pero si agregáramos una nueva variable relacionada con la distancia al centro, la separación podría volverse más clara. Los puntos del centro tendrían distancias pequeñas; los del anillo, distancias mayores. En ese nuevo espacio, la diferencia entre clases sería más fácil de representar.

No necesitamos calcular ese ejemplo ahora. Lo importante es la intuición: una transformación adecuada puede convertir un problema no lineal en uno que SVM puede separar mejor.

Los kernels permiten trabajar con esa idea de manera eficiente. En lugar de transformar explícitamente los datos a un espacio enorme o difícil de construir, el kernel calcula relaciones entre puntos como si estuvieran en ese espacio transformado.

Un kernel es una forma de medir las relaciones entre los puntos como si los datos se encontraran en un espacio transformado.

Esta es una de las razones por las que SVM fue históricamente tan importante. Los kernels permiten construir fronteras complejas manteniendo una formulación matemática elegante. Para usar el modelo, no necesitamos entender todos los detalles internos del cálculo. Pero sí necesitamos comprender qué efecto conceptual tiene: permite separaciones no lineales.

En `scikit-learn`, el kernel se indica con el parámetro `kernel` de `SVC`.

```
from sklearn.svm import SVC

modelo = SVC(
    kernel="linear",
    C=1.0
)
```

Con `kernel="linear"`, SVM busca una frontera lineal. Es decir, trabaja con la idea que venimos usando hasta ahora: una separación mediante recta, plano o hiperplano.

Pero también podemos usar otros kernels. Uno de los más conocidos es el kernel radial, llamado RBF por *Radial Basis Function*.

```
modelo = SVC(  
    kernel="rbf",  
    C=1.0  
)
```

El kernel RBF permite construir fronteras no lineales. Puede adaptarse a formas curvas y regiones más complejas. Por eso suele ser una opción fuerte cuando sospechamos que las clases no se separan bien con una frontera lineal.

Kernel	Tipo de frontera
Lineal	Frontera lineal.
RBF	Frontera no lineal más flexible.

La flexibilidad del kernel RBF puede ser una ventaja, pero también un riesgo. Si el modelo se vuelve demasiado flexible, puede sobreajustar. Puede construir una frontera que rodee demasiado los puntos de entrenamiento y generalice mal a datos nuevos.

Por eso, al usar kernels no lineales, la validación es fundamental. No alcanza con ver que el modelo clasifica muy bien el entrenamiento. Debemos evaluar con datos no vistos y ajustar hiperparámetros con cuidado.

Un kernel más flexible ofrece mayor capacidad para aprender patrones complejos, pero también aumenta el riesgo de sobreajuste.

En SVM con kernel RBF aparece un hiperparámetro importante llamado `gamma`. Este valor controla, de manera intuitiva, qué tan local es la influencia de cada punto en la forma de la frontera.

Si `gamma` es alto, cada punto puede influir en una zona más pequeña. Esto permite fronteras muy ajustadas y complejas. Si `gamma` es bajo, la influencia de los puntos se extiende más y la frontera tiende a ser más suave.

Valor de <code>gamma</code>	Comportamiento
<code>gamma</code> alto	Frontera más ajustada y mayor riesgo de sobreajuste.
<code>gamma</code> bajo	Frontera más suave, con posible subajuste si es demasiado bajo.

`gamma` interactúa con `C`. Como vimos, `C` controla cuánto penalizamos errores o violaciones del margen. Un `C` alto presiona al modelo para clasificar bien los datos de entrenamiento. Un `gamma` alto permite fronteras muy locales. Combinados sin cuidado, pueden producir un modelo demasiado ajustado.

Configuración	Riesgo principal
<code>C</code> alto y <code>gamma</code> alto	Mayor riesgo de sobreajuste.
<code>C</code> o <code>gamma</code> excesivamente bajos	Mayor riesgo de subajuste.

En cambio, valores más moderados pueden producir fronteras más estables. No hay una combinación universal. Debe buscarse mediante validación cruzada, con una métrica adecuada para el problema.

Un ejemplo de búsqueda podría probar distintos valores de `C` y `gamma`:

```
param_grid = {
    "modelo__C": [0.1, 1, 10],
    "modelo__gamma": [0.01, 0.1, 1],
    "modelo__kernel": ["rbf"]
}
```

Si el SVM está dentro de un pipeline, usamos el nombre del paso seguido de doble guion bajo, como ya vimos en capítulos anteriores.

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.model_selection import GridSearchCV

pipeline = Pipeline(steps=[
    ("scaler", StandardScaler()),
    ("modelo", SVC())
])

param_grid = {
    "modelo__kernel": ["rbf"],
    "modelo__C": [0.1, 1, 10],
    "modelo__gamma": [0.01, 0.1, 1]
}

busqueda = GridSearchCV(
    estimator=pipeline,
    param_grid=param_grid,
    cv=5,
    scoring="f1"
)
```

Este ejemplo muestra dos ideas importantes. Primero, que los kernels y sus hiperparámetros pueden ajustarse con validación cruzada. Segundo, que SVM suele necesitar escalado, por lo que conviene incluir `StandardScaler` dentro del pipeline.

El escalado será tratado con más detalle en la próxima sección, pero vale anticipar el motivo: SVM trabaja con geometría, márgenes y relaciones entre puntos. Si una variable tiene una escala mucho más grande que otra, puede dominar la separación.

SVM con kernels es muy sensible a la escala de las variables.

Además del kernel lineal y el RBF, existen otros kernels. Por ejemplo, el kernel polinómico permite construir fronteras basadas en relaciones polinómicas entre variables.

```
modelo = SVC(  
    kernel="poly",  
    degree=3,  
    C=1.0  
)
```

En este caso, `degree` indica el grado del polinomio. Un grado mayor permite fronteras más complejas, pero también puede aumentar el riesgo de sobreajuste y el costo computacional.

También existe el kernel sigmoide, aunque en la práctica introductoria suele aparecer menos.

Entre los kernels más comunes se encuentran `linear`, `rbf`, `poly` y `sigmoid`.

No es necesario probar todos los kernels en todos los problemas. Muchas veces, una comparación razonable entre un SVM lineal y un SVM con RBF ya permite explorar dos situaciones importantes: separación lineal y separación no lineal flexible.

Si el dataset tiene muchas variables y las clases parecen separarse de manera relativamente simple, un kernel lineal puede funcionar bien y ser más eficiente. Si la frontera parece no lineal, RBF puede ser una opción interesante, siempre con buena validación.

Kernel	Características
Lineal	Más simple, eficiente y menos flexible.
RBF	Más flexible y potente, pero también más sensible a los hiperparámetros.

También debemos recordar que más flexibilidad no siempre es mejor. Un kernel complejo puede mejorar la métrica de validación, pero también aumentar el costo computacional y reducir la interpretabilidad. Si un modelo lineal funciona casi igual de bien, puede ser preferible por simplicidad.

La elección del kernel debe responder al problema, no a la idea de que “más complejo es mejor”.

Elegir un kernel equivale a decidir qué tipo de frontera le permitimos construir al modelo.

Los kernels permiten que SVM construya fronteras no lineales sin transformar manualmente todas las variables. El kernel lineal busca separaciones rectas o planas. El kernel RBF permite fronteras más flexibles y suele ser útil cuando las clases no son linealmente separables. Esa flexibilidad debe controlarse mediante hiperparámetros como C y γ , usando validación cruzada y pipelines que incluyan escalado. Los kernels hacen que SVM sea una herramienta poderosa, pero también exigen cuidado para no confundir potencia con generalización real.

Importancia del escalado

En SVM, el escalado de variables es especialmente importante. Esto se debe a que el modelo trabaja con una idea geométrica: puntos ubicados en un espacio de variables, fronteras de decisión, márgenes y distancias. Si las variables están en escalas muy diferentes, esa geometría puede quedar distorsionada.

Supongamos que queremos clasificar viviendas usando dos variables:

Por ejemplo, podríamos trabajar con las variables `superficie_m2` y `cantidad_ambientes`.

La superficie puede tomar valores como 40, 80, 120 o 200. La cantidad de ambientes, en cambio, puede tomar valores como 1, 2, 3, 4 o 5. Si usamos esas variables sin escalar, la superficie tendrá números mucho más grandes. Como consecuencia, puede dominar el cálculo geométrico del modelo.

Escala de la variable	Posible efecto
Valores grandes	Puede influir demasiado.
Valores pequeños	Puede quedar con menor peso relativo.

Esto no significa que la superficie sea necesariamente más importante que la cantidad de ambientes. Significa que, por su escala numérica, puede afectar más la forma en que el modelo mide separaciones, márgenes y relaciones entre puntos.

En modelos como SVM, esa diferencia de escala puede cambiar completamente la frontera de decisión. Dos variables pueden ser relevantes, pero si una está medida en miles y otra en unidades pequeñas, el modelo puede comportarse como si la variable grande fuera mucho más importante.

Sin escalado, la escala de medición puede confundirse con la importancia que una variable tiene para el modelo.

Esto también ocurre con otros modelos que dependen de distancias o geometría, como KNN. Ya vimos en KNN que una variable con valores grandes podía dominar la distancia entre puntos. En SVM pasa algo similar, especialmente cuando usamos kernels como RBF, donde las relaciones entre puntos son fundamentales.

Por eso, antes de entrenar un SVM, suele ser necesario escalar las variables numéricas.

Una transformación habitual es la estandarización, que lleva cada variable a una escala donde la media queda cerca de 0 y la desviación estándar cerca de 1. En `scikit-learn`, esto puede hacerse con `StandardScaler`.

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
```

Pero no conviene aplicar el escalador de cualquier manera. Como ya vimos en capítulos anteriores, las transformaciones que aprenden información de los datos deben ajustarse solo con el conjunto de entrenamiento. Si ajustamos el escalador usando todo el dataset antes de separar entrenamiento y prueba, estaríamos filtrando información del test hacia el entrenamiento.

El escalador aprende la media y la desviación estándar, por lo que debe ajustarse únicamente con los datos de entrenamiento.

La forma más segura de trabajar es usar un `Pipeline`. Así, el escalado y el modelo quedan unidos en un mismo flujo. Durante la validación cruzada, el escalador se ajusta dentro de cada fold usando solo los datos de entrenamiento correspondientes.

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC

pipeline = Pipeline(steps=[
    ("scaler", StandardScaler()),
    ("modelo", SVC(kernel="rbf", C=1.0, gamma="scale"))
])
```

Este pipeline primero escala las variables y luego entrena el SVM. Si lo usamos con `cross_val_score`, `GridSearchCV` o `RandomizedSearchCV`, el escalado se hará correctamente en cada división.

Un pipeline ajusta el escalado con los datos de entrenamiento, aplica la transformación, entrena el modelo y ayuda a evitar la fuga de datos.

El escalado es importante tanto para SVM lineal como para SVM con kernels no lineales. En un SVM lineal, la escala afecta la orientación y la posición de la frontera. En un SVM con RBF, la escala afecta cómo se mide la cercanía entre puntos y, por lo tanto, la forma de la frontera no lineal.

Variante	Efecto de la escala
SVM lineal	La escala afecta la frontera.
SVM con RBF	La escala afecta la relación entre puntos y la forma de la frontera.

Esto se vuelve todavía más importante cuando ajustamos hiperparámetros como `C` y `gamma`. Si las variables no están escaladas, un valor de `gamma` puede comportarse de manera muy distinta según las unidades de las columnas. Una búsqueda de hiperparámetros sobre datos sin escalar puede llevar a conclusiones poco confiables.

Por ejemplo, una variable medida en pesos puede tener valores como 10000, 50000 o 200000. Otra variable, como cantidad de compras, puede tener valores entre 0 y 20. Otra, como antigüedad en años, puede estar entre 1 y 10. Si esas escalas se mezclan sin preparación, el espacio geométrico del modelo queda dominado por algunas variables.

Las variables pueden estar expresadas en pesos, metros, años, cantidades o porcentajes. SVM necesita que puedan compararse dentro de una escala razonable.

Esto no significa que todas las variables deban tener el mismo significado, sino que su magnitud numérica no debería dominar por accidente. Escalar permite que el modelo evalúe las variables de manera más equilibrada.

También debemos considerar qué ocurre con variables categóricas. SVM no trabaja directamente con categorías de texto. Si tenemos una variable como `medio_pago`, `barrio` o `tipo_cliente`, debemos codificarla. Una estrategia común es One-Hot Encoding.

Cuando combinamos variables numéricas y categóricas, el preprocesamiento puede incluir escalado para las numéricas y codificación para las categóricas. En ese caso, conviene usar `ColumnTransformer` dentro de un pipeline.

```
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.pipeline import Pipeline
from sklearn.svm import SVC

preprocesamiento = ColumnTransformer(
```

```

transformers=[
    ("num", StandardScaler(), columnas_numericas),
    ("cat", OneHotEncoder(handle_unknown="ignore"), columnas_categoricas)
]

pipeline = Pipeline(steps=[
    ("preprocesamiento", preprocesamiento),
    ("modelo", SVC(kernel="rbf", C=1.0, gamma="scale"))
])

```

Este flujo mantiene el preprocesamiento dentro del pipeline. Es más ordenado y reduce el riesgo de fuga de datos. Además, permite evaluar el modelo completo mediante validación cruzada.

En SVM, el preprocesamiento no es un detalle menor, sino una parte del comportamiento real del modelo.

También es posible usar otros escaladores. `MinMaxScaler`, por ejemplo, lleva las variables a un rango determinado, como entre 0 y 1. `StandardScaler` suele ser una opción frecuente, pero no es la única. La elección puede depender del modelo, de la distribución de las variables y de la presencia de valores extremos.

No necesitamos estudiar todos los escaladores en detalle ahora. Lo importante es comprender que SVM no debería entrenarse sobre variables numéricas con escalas muy diferentes sin revisar ese punto.

Antes de entrenar una SVM debemos preguntarnos si las variables numéricas están en escalas comparables, si existen valores extremos, si el escalado se ajustó solo con los datos de entrenamiento y si el preprocesamiento está incluido dentro de un pipeline.

Si olvidamos escalar, el modelo puede rendir mal o elegir fronteras poco razonables. También puede parecer que una variable es muy influyente solo porque sus valores son más grandes. Esto afecta tanto el desempeño como la interpretación.

En algunos modelos, como árboles de decisión o Random Forest, el escalado numérico no suele ser tan importante. Los árboles dividen los datos usando condiciones del tipo “variable menor o mayor que cierto valor”, por lo que la escala no afecta de la misma manera. Pero en SVM, la geometría del espacio sí depende directamente de la escala.

Modelo	Sensibilidad al escalado
Árboles	Menor sensibilidad al escalado.
SVM	Alta sensibilidad al escalado.

Esta diferencia ayuda a entender por qué no existe un único preprocesamiento universal para todos los modelos. Las transformaciones dependen del tipo de algoritmo y de cómo ese algoritmo aprende.

El escalado es una etapa fundamental al trabajar con SVM. Como el modelo busca fronteras, márgenes y relaciones geométricas entre puntos, las variables numéricas deben estar en escalas razonablemente comparables. Si no escalamos, las variables con valores grandes pueden dominar el modelo. La forma recomendada de hacerlo es incluir `StandardScaler` u otra transformación adecuada dentro de un `Pipeline`, especialmente cuando usamos validación cruzada o búsqueda de hiperparámetros. En SVM, el preprocesamiento no es accesorio: forma parte esencial del modelo que realmente estamos evaluando.

Ventajas

Support Vector Machines es un modelo especialmente interesante porque combina una idea geométrica clara con una capacidad considerable para construir fronteras de decisión. Aunque puede resultar menos intuitivo al principio que KNN, regresión logística o árboles de decisión, ofrece ventajas importantes en ciertos tipos de problemas.

Una primera ventaja es su enfoque en el margen. SVM no busca simplemente una frontera que separe las clases, sino una frontera que lo haga dejando la mayor distancia posible respecto de los puntos más cercanos. Esta idea puede favorecer modelos con buena capacidad de generalización, especialmente cuando las clases tienen una separación razonablemente clara.

SVM busca una frontera con un margen amplio, no una separación cualquiera.

Esto lo diferencia de modelos que solo intentan reducir errores de entrenamiento. Al priorizar el margen, SVM introduce una forma de control sobre la frontera de decisión. No se limita a pasar entre los puntos de cualquier manera, sino que intenta encontrar una separación más robusta.

Esta característica puede ser útil cuando tenemos datasets donde la frontera entre clases está bien definida, aunque no necesariamente sea evidente a simple vista. En esos casos, SVM puede encontrar una separación eficaz y relativamente estable.

Otra ventaja es que SVM puede trabajar bien en espacios con muchas variables. En algunos problemas, especialmente aquellos donde la cantidad de columnas es grande, SVM puede ser competitivo. Un ejemplo clásico es la clasificación de textos, donde cada palabra o término puede convertirse en una variable. En esos contextos,

puede haber muchísimas variables, pero cada documento contiene solo algunas de ellas activas.

SVM puede funcionar bien en problemas con muchas variables, siempre que esas variables contengan una señal útil.

Esto no significa que siempre sea la mejor opción para datos de alta dimensionalidad, pero históricamente ha sido un modelo fuerte en ese tipo de escenarios. Su capacidad para construir fronteras en espacios de muchas dimensiones lo vuelve una herramienta interesante cuando otros modelos simples no alcanzan.

También es una ventaja la posibilidad de usar distintos kernels. Con un kernel lineal, SVM puede construir una frontera simple y eficiente. Con kernels como RBF o polinómico, puede construir fronteras no lineales más flexibles.

Kernel	Características
Lineal	Más simple, eficiente y menos flexible.
RBF	Más flexible y potente, pero también más sensible a los hiperparámetros.

Esto permite adaptar el modelo a distintos tipos de problemas. Si las clases se separan razonablemente con una frontera lineal, podemos usar un SVM lineal. Si la separación requiere formas más complejas, podemos explorar kernels no lineales.

La posibilidad de usar kernels hace que SVM sea más flexible que un modelo lineal puro. Pero, a diferencia de un árbol o un ensamble de árboles, mantiene una formulación basada en márgenes y distancias en el espacio de variables. Esto le da una identidad propia dentro de los modelos de clasificación.

Otra ventaja es que, en muchos casos, la frontera del modelo depende principalmente de los vectores de soporte, es decir, de los puntos cercanos al margen. Los puntos que están muy lejos de la frontera suelen tener menor influencia en la posición final de la separación.

Los vectores de soporte son los casos cercanos a la frontera que ayudan a definir la separación.

Esto puede ser útil porque el modelo se concentra en los casos críticos, aquellos que realmente están cerca de la zona de decisión. Los casos fáciles, ubicados claramente dentro de una clase, no necesitan determinar tanto la frontera.

SVM también puede comportarse bien en problemas donde la cantidad de filas no es enorme. En datasets pequeños o medianos, bien preparados y con variables informativas, puede ofrecer buenos resultados. Esto lo vuelve una opción interesante

para comparar contra modelos como regresión logística, KNN, árboles o Random Forest.

SVM puede ser una buena alternativa para datasets pequeños o medianos cuyas variables estén bien preparadas.

Otra ventaja es que el modelo permite controlar el equilibrio entre margen y errores mediante el hiperparámetro `C`. Esto ofrece una forma explícita de regular la tolerancia a errores del entrenamiento. Si queremos una frontera más estricta, podemos aumentar `C`. Si queremos permitir más errores para favorecer una separación más amplia, podemos disminuirlo.

El hiperparámetro `C` controla el equilibrio entre penalizar los errores y mantener un margen flexible.

Cuando usamos kernel RBF, también podemos controlar la flexibilidad de la frontera mediante `gamma`. Esto permite ajustar qué tan local o suave será la separación. Aunque esto aumenta la cantidad de decisiones a tomar, también da control sobre el comportamiento del modelo.

SVM puede ser especialmente útil cuando queremos comparar una frontera lineal con una no lineal sin cambiar completamente de familia de modelos. Podemos evaluar un SVM lineal y luego un SVM con RBF, usando el mismo esquema de validación. Esto permite observar si el problema parece necesitar una frontera más flexible.

SVM permite comparar una separación lineal con una no lineal dentro de una misma familia de modelos.

Otra ventaja práctica es que `scikit-learn` ofrece implementaciones accesibles y compatibles con el flujo que ya venimos usando. Podemos trabajar con `SVC`, `LinearSVC`, pipelines, validación cruzada, `GridSearchCV` y métricas de clasificación.

Por ejemplo:

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC

pipeline = Pipeline(steps=[
    ("scaler", StandardScaler()),
    ("modelo", SVC(kernel="rbf", C=1.0, gamma="scale"))
])
```

Este tipo de estructura permite integrar el escalado y el modelo en un solo flujo. Como vimos, esto es fundamental para evitar fuga de datos durante la validación.

También puede obtener probabilidades estimadas si se configura explícitamente con `probability=True`, aunque esto tiene costos y precauciones. En muchos casos, SVM produce una función de decisión que indica de qué lado de la frontera queda el punto y con qué distancia relativa. Si necesitamos probabilidades calibradas, debemos tratar ese punto con cuidado.

```
modelo = SVC(  
    kernel="rbf",  
    C=1.0,  
    gamma="scale",  
    probability=True  
)
```

No siempre necesitamos activar probabilidades. Si solo queremos clasificar, `predict` puede ser suficiente. Si necesitamos ajustar umbrales, comparar probabilidades o construir curvas, debemos entender cómo se obtienen esas probabilidades y evaluar si son confiables.

Entre sus ventajas, entonces, SVM ofrece una combinación atractiva: margen, flexibilidad mediante kernels, buen desempeño potencial en espacios de muchas variables y control mediante hiperparámetros.

Entre las ventajas principales de SVM se encuentran la búsqueda de un margen amplio, el uso de kernels, el buen desempeño en ciertos espacios con muchas variables, el control mediante `C` y `gamma`, y su compatibilidad con pipelines.

Sin embargo, estas ventajas dependen de una condición importante: los datos deben estar bien preparados. SVM no suele ser un modelo para aplicar de manera descuidada. Necesita escalado, selección razonable de variables, cuidado con valores extremos y una búsqueda de hiperparámetros bien diseñada.

Por eso, sus ventajas aparecen con más claridad cuando el flujo completo está bien construido. Un SVM sin escalado o con hiperparámetros mal elegidos puede rendir mucho peor de lo esperado.

SVM puede ser muy potente, pero exige un buen preprocesamiento y una evaluación cuidadosa.

SVM tiene varias ventajas importantes. Busca fronteras con margen amplio, puede trabajar con kernels para construir separaciones no lineales, puede funcionar bien en problemas con muchas variables y permite controlar su flexibilidad mediante hiperparámetros como `C` y `gamma`. También se integra bien con pipelines y herramientas de validación de `scikit-learn`. Su fortaleza aparece especialmente cuando las variables están bien preparadas, escaladas y evaluadas con una estrategia adecuada.

Limitaciones

Support Vector Machines tiene ventajas importantes, pero también presenta limitaciones que conviene conocer antes de elegirlo para un problema. Como ocurre con todos los modelos, no alcanza con saber qué puede hacer bien. También debemos entender en qué situaciones puede resultar incómodo, costoso o poco adecuado.

Una primera limitación es su sensibilidad al escalado. Ya vimos que SVM trabaja con márgenes, distancias y relaciones geométricas entre puntos. Si las variables numéricas están en escalas muy diferentes, el modelo puede construir una frontera dominada por las variables con valores más grandes.

SVM necesita variables bien escaladas para que la geometría del modelo tenga sentido.

Esto no es un detalle menor. En un árbol de decisión, la escala numérica de una variable suele ser menos problemática. En SVM, en cambio, olvidar el escalado puede cambiar mucho el resultado. Por eso, casi siempre conviene usar `StandardScaler` u otra transformación adecuada dentro de un `Pipeline`.

Una segunda limitación es que puede ser sensible a la elección de hiperparámetros. El valor de `C` controla el equilibrio entre margen y penalización de errores. Si usamos kernel RBF, `gamma` controla la flexibilidad de la frontera. Si elegimos mal estos valores, el modelo puede subajustar o sobreajustar.

Los hiperparámetros `C` y `gamma` pueden modificar considerablemente el comportamiento del modelo.

Por ejemplo, un `C` muy alto y un `gamma` muy alto pueden producir una frontera muy ajustada a los datos de entrenamiento. Esto puede mejorar el desempeño en entrenamiento, pero reducir la capacidad de generalizar. En cambio, valores demasiado bajos pueden hacer que la frontera sea demasiado simple y no capture patrones importantes.

Configuración	Riesgo principal
C alto y gamma alto	Mayor riesgo de sobreajuste.
C o gamma excesivamente bajos	Mayor riesgo de subajuste.

Esta sensibilidad obliga a evaluar con cuidado. SVM suele necesitar validación cruzada y búsqueda de hiperparámetros para encontrar una configuración adecuada. Aplicarlo con valores por defecto puede funcionar en algunos casos, pero no siempre es suficiente.

Otra limitación es el costo computacional. SVM puede volverse lento cuando el dataset tiene muchas filas, especialmente con kernels no lineales como RBF. Esto se debe a que el modelo necesita trabajar con relaciones entre puntos, y esa tarea puede crecer mucho cuando aumenta la cantidad de observaciones.

SVM con kernels no lineales puede resultar costoso en datasets grandes.

En datasets pequeños o medianos, este costo puede ser manejable. Pero en datasets con cientos de miles o millones de filas, SVM con kernels no lineales puede no ser la opción más práctica. En esos casos, modelos como Random Forest, Gradient Boosting o modelos lineales más eficientes pueden ser más convenientes, según el problema.

El SVM lineal suele ser más eficiente que el SVM con kernel RBF. Si el dataset tiene muchas filas y muchas variables, y sospechamos que una frontera lineal puede ser suficiente, `LinearSVC` puede ser una opción más práctica que `SVC(kernel="rbf")`.

Variante	Características
SVM lineal	Más simple y eficiente.
SVM con RBF	Más flexible, pero también más costoso.

Otra limitación es la interpretabilidad. Aunque la idea del margen es clara, un SVM entrenado, especialmente con kernel no lineal, no siempre es fácil de explicar. En una regresión logística podemos observar coeficientes. En un árbol pequeño podemos seguir una secuencia de preguntas. En SVM con RBF, la frontera puede ser compleja y difícil de traducir a reglas comprensibles.

Una SVM con kernel no lineal puede construir buenas fronteras, aunque estas no siempre sean fáciles de explicar.

Esto no significa que sea imposible analizarlo. Podemos revisar métricas, errores, vectores de soporte y comportamiento general. Pero si necesitamos una explicación sencilla para cada predicción, SVM puede ser menos conveniente que modelos más transparentes.

También puede ser incómodo cuando necesitamos probabilidades bien interpretables. Por defecto, `SVC` no entrega probabilidades directamente, sino clases y una función de decisión. Es posible activar probabilidades con `probability=True`, pero eso agrega costo computacional y las probabilidades resultantes deben interpretarse con cuidado.

SVM clasifica naturalmente a partir de una frontera, mientras que la obtención de probabilidades requiere un tratamiento adicional.

Esto puede ser una desventaja en problemas donde las probabilidades son centrales. Por ejemplo, si necesitamos ordenar clientes por riesgo, ajustar umbrales con precisión, construir curvas de decisión o comunicar probabilidades calibradas, quizá convenga comparar SVM con modelos que entregan probabilidades de manera más directa, como regresión logística o algunos ensambles.

Otra limitación es que SVM puede verse afectado por ruido y valores extremos. Aunque el margen suave permite tolerar ciertos errores, casos atípicos o etiquetas incorrectas pueden influir en la posición de la frontera, especialmente si están cerca del margen. Si el dataset tiene muchas etiquetas ruidosas, la búsqueda de una buena frontera puede complicarse.

Los puntos cercanos al margen pueden influir mucho en la frontera de decisión.

Esto se relaciona con los vectores de soporte. Como los puntos cercanos a la frontera son importantes para definir el modelo, errores en esa zona pueden tener más impacto que errores en puntos muy alejados. Por eso, la calidad de los datos y de las etiquetas sigue siendo fundamental.

SVM tampoco resuelve por sí solo problemas de variables irrelevantes, fuga de datos o mala formulación del objetivo. Si incluimos una variable que revela la respuesta, el modelo puede aprovecharla. Si las variables no contienen señal, no podrá inventarla. Si las clases están mal definidas, la frontera resultante también será problemática.

SVM puede separar patrones, pero no puede corregir por sí sola un problema mal definido.

Otra dificultad aparece cuando hay muchas variables categóricas. SVM no trabaja directamente con categorías de texto, por lo que necesitamos codificarlas. Si usamos One-Hot Encoding con variables de alta cardinalidad, el número de columnas puede crecer mucho. En algunos casos, esto puede funcionar bien; en otros, puede aumentar el costo o exigir más cuidado en la preparación.

Las variables categóricas deben codificarse y, cuando contienen muchas categorías, pueden generar una gran cantidad de columnas.

También debemos considerar que la elección del kernel no siempre es obvia. Un kernel lineal puede ser demasiado simple. Un kernel RBF puede ser demasiado flexible o costoso. Un kernel polinómico puede introducir complejidad adicional. La decisión debe basarse en evaluación, no en preferencias.

La elección del kernel es una decisión de modelado que debe validarse.

En algunos problemas, SVM puede rendir muy bien, pero no mejorar lo suficiente respecto de modelos más simples. Si una regresión logística obtiene una métrica parecida, entrega probabilidades más directas y es más fácil de interpretar, puede ser preferible. Si Random Forest o Gradient Boosting funcionan mejor con menor esfuerzo de preprocesamiento, también pueden ser alternativas más prácticas.

Esto no le quita valor a SVM. Simplemente recuerda que un modelo debe elegirse por su utilidad en el flujo completo, no por su elegancia matemática.

La mejor opción no siempre es el modelo más sofisticado, sino el más adecuado para el problema y sus restricciones.

SVM tiene varias limitaciones. Es sensible al escalado, requiere cuidado en la elección de hiperparámetros, puede ser costoso en datasets grandes con kernels no lineales, no siempre es fácil de interpretar y no entrega probabilidades de manera tan directa como otros modelos. También depende fuertemente de la calidad de los datos, de la codificación de variables y de una evaluación bien diseñada. Puede ser una herramienta muy poderosa, pero no es una solución universal.

¿Cuándo puede ser una buena opción?

Support Vector Machines puede ser una buena opción cuando el problema tiene una estructura de separación clara, las variables están bien preparadas y el tamaño del dataset permite entrenar el modelo sin un costo excesivo. No es un modelo que convenga usar de manera automática en cualquier situación, pero puede resultar muy competitivo en ciertos escenarios.

Una primera situación favorable es aquella en la que las clases pueden separarse razonablemente bien mediante una frontera. Esa frontera puede ser lineal o no lineal. Si una separación recta alcanza, un SVM lineal puede ser una alternativa sólida. Si la separación requiere curvas o regiones más flexibles, un SVM con kernel RBF puede ser una opción interesante.

SVM puede ser útil cuando existe una frontera razonable entre las clases.

Por ejemplo, si tenemos dos grupos de clientes con comportamientos bastante distintos, o documentos de texto que pertenecen a categorías diferenciables, SVM puede encontrar una separación eficaz. Su objetivo de maximizar el margen puede ayudar a construir una frontera más robusta que una separación cualquiera.

También puede ser una buena opción cuando el dataset no es demasiado grande. En conjuntos pequeños o medianos, especialmente con variables bien seleccionadas, SVM puede ofrecer muy buen desempeño. En cambio, si tenemos millones de filas, un SVM con kernel no lineal puede volverse costoso y quizá convenga usar otros modelos.

Tamaño del dataset	Consideración
Pequeño o mediano	SVM puede ser competitivo.
Muy grande	Conviene revisar el costo computacional.

Otra situación favorable aparece cuando tenemos muchas variables, pero una cantidad manejable de filas. SVM, especialmente en su versión lineal, puede funcionar bien en espacios de alta dimensionalidad. Esto lo hace interesante en problemas donde cada observación se representa con muchas características.

Un ejemplo típico es la clasificación de textos. Después de transformar documentos en variables numéricas, podemos terminar con miles de columnas, una por término, expresión o característica del texto. En ese contexto, un SVM lineal puede ser una alternativa fuerte.

La clasificación de texto, con muchas variables y datos dispersos, puede ser un buen escenario para una SVM lineal.

Esto no significa que SVM sea siempre el mejor modelo para texto. Existen muchas alternativas, incluidas técnicas modernas más avanzadas. Pero dentro del Machine Learning clásico, SVM ha sido históricamente una herramienta importante para clasificación textual.

SVM también puede ser una buena opción cuando queremos comparar una frontera lineal contra una frontera no lineal dentro de una misma familia de modelos. Podemos empezar con un kernel lineal y luego probar un kernel RBF. Si el RBF mejora mucho en validación, eso puede indicar que la separación lineal era insuficiente. Si el lineal funciona parecido, quizás convenga preferirlo por simplicidad y eficiencia.

Una comparación útil consiste en evaluar una SVM lineal frente a una SVM con kernel RBF.

Esta comparación debe hacerse con cuidado, usando el mismo conjunto de datos, la misma métrica, la misma estrategia de validación y el mismo preprocesamiento. Si no, podríamos atribuir al modelo una diferencia que en realidad viene de otro aspecto del flujo.

Otra situación donde SVM puede resultar conveniente es cuando queremos un modelo con control explícito sobre el margen y los errores. El hiperparámetro `C` permite ajustar cuánto penalizamos las clasificaciones incorrectas o las violaciones del margen. Esto da una forma clara de regular el equilibrio entre una frontera más estricta y una frontera más flexible.

El hiperparámetro `C` permite regular cuánto tolera el modelo ciertos errores de entrenamiento.

Si usamos un kernel RBF, `gamma` agrega otro control sobre la flexibilidad de la frontera. Esto permite ajustar el modelo a distintos grados de complejidad, siempre con el riesgo de sobreajuste si no se valida correctamente.

SVM puede ser especialmente útil cuando el desempeño de modelos simples no alcanza, pero todavía queremos probar una alternativa distinta a los ensambles de árboles. Por ejemplo, si una regresión logística resulta demasiado limitada y un Random Forest no mejora lo suficiente, un SVM con kernel puede ofrecer otra forma de separar las clases.

SVM puede ser una alternativa intermedia: es más flexible que un modelo lineal simple, pero sigue una lógica diferente de la de los ensambles de árboles.

También puede ser útil cuando las variables numéricas están cuidadosamente preparadas y escaladas. Si ya tenemos un buen pipeline de preprocesamiento, SVM se integra bien con validación cruzada y búsqueda de hiperparámetros.

Un flujo razonable podría ser:

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC

pipeline = Pipeline(steps=[
    ("scaler", StandardScaler()),
    ("modelo", SVC(kernel="rbf", C=1.0, gamma="scale"))
])
```

Luego podemos evaluar ese pipeline con validación cruzada y ajustar `C`, `gamma` o el tipo de kernel con `GridSearchCV` o `RandomizedSearchCV`.

SVM debería evaluarse como parte de un pipeline completo, y no como un modelo aislado y desconectado del preprocesamiento.

SVM puede ser una opción menos conveniente si no podemos escalar correctamente las variables, si hay muchas categorías mal codificadas, si el dataset es enorme o si necesitamos interpretaciones muy simples para cada predicción. También puede no ser ideal si necesitamos probabilidades bien calibradas como salida principal.

Por ejemplo, en un problema donde el objetivo es ordenar clientes por probabilidad de abandono y ajustar umbrales de negocio, quizá una regresión logística, un Gradient Boosting o un modelo calibrado resulten más cómodos. Si necesitamos explicar cada decisión con reglas simples, un árbol pequeño o un modelo lineal pueden ser preferibles.

SVM puede no ser la mejor opción cuando necesitamos máxima interpretabilidad, probabilidades directas o un entrenamiento rápido con una cantidad muy grande de filas.

La decisión depende del contexto. Como ya vimos en otros capítulos, no existe un modelo universalmente mejor. SVM debe compararse con alternativas usando una evaluación justa. Puede rendir muy bien en algunos problemas y ser poco práctico en otros.

Una estrategia razonable es incluir SVM como candidato cuando:

Puede ser una buena alternativa cuando el dataset es pequeño o mediano, las variables pueden escalarse correctamente, existen muchas variables informativas, la separación entre clases parece posible, queremos probar kernels y podemos ajustar los hiperparámetros mediante validación.

En cambio, si el problema tiene muchos datos, muchas variables categóricas de alta cardinalidad, necesidad de explicaciones simples o restricciones fuertes de tiempo, quizá convenga comenzar por otros modelos.

También debemos observar la métrica adecuada. En clasificación desbalanceada, no alcanza con mirar accuracy. Si usamos SVM para detectar fraude, abandono, fallas o eventos poco frecuentes, debemos revisar precision, recall, F1-score, matriz de confusión y el costo de cada tipo de error.

SVM puede separar bien, pero la métrica debe indicar si lo hace de la manera que el problema necesita.

Como ocurre con todos los modelos potentes, también debemos cuidar la fuga de datos. Si una variable revela la respuesta, SVM puede aprovecharla. Si el

preprocesamiento se ajusta antes de separar los datos, también puede contaminar la evaluación. Un buen resultado solo es confiable si el flujo completo está bien diseñado.

SVM puede ser una buena opción cuando las clases tienen una separación razonable, el dataset es pequeño o mediano, las variables están bien escaladas y buscamos una frontera lineal o no lineal con control sobre margen y errores. Puede funcionar especialmente bien en problemas con muchas variables, como ciertos casos de clasificación de texto. Sin embargo, debe compararse con otros modelos bajo una evaluación justa, usando pipelines, validación cruzada, métricas adecuadas y cuidado con hiperparámetros como C y γ . Su valor no está en ser siempre el mejor modelo, sino en ofrecer una forma potente y geoméricamente clara de separar clases cuando el problema se presta para ello.

Separar clases con márgenes

En este capítulo estudiamos Support Vector Machines, una familia de modelos cuya intuición principal se apoya en la geometría. Para clasificación, SVM busca construir una frontera que separe las clases, pero no cualquier frontera: intenta encontrar una separación con el mayor margen posible.

La idea del margen fue el punto central. En un problema con dos variables, podemos imaginar los casos como puntos en un plano y la frontera como una recta. SVM busca ubicar esa recta de manera que quede lo más alejada posible de los puntos más cercanos de cada clase.

SVM no busca solamente separar, sino hacerlo con un margen amplio.

Los puntos cercanos a la frontera se llaman vectores de soporte. Son importantes porque ayudan a definir la posición del margen y de la frontera de decisión. Los puntos que están muy lejos de la separación suelen influir menos en la forma final del modelo.

También vimos que, en datos reales, no siempre es posible separar perfectamente las clases. Puede haber ruido, casos ambiguos o puntos mezclados. Por eso SVM suele trabajar con un margen suave, permitiendo algunos errores o violaciones del margen para construir una separación más robusta.

Un margen suave permite ciertos errores para evitar una frontera demasiado rígida.

El hiperparámetro C controla este equilibrio. Un C alto penaliza más los errores y empuja al modelo a ajustarse mejor al entrenamiento. Un C bajo permite más

tolerancia, lo que puede favorecer un margen más amplio, aunque si es demasiado bajo puede llevar a subajuste.

Luego analizamos la separación entre clases. En los casos más simples, una frontera lineal puede alcanzar. En dos dimensiones, esa frontera es una recta; en tres dimensiones, un plano; en más dimensiones, un hiperplano. Pero no todos los problemas son linealmente separables.

Para esos casos aparecen los kernels. Un kernel permite que SVM construya fronteras no lineales, como ocurre con el kernel RBF. La intuición es que el modelo puede analizar relaciones entre puntos como si estuvieran en un espacio transformado, donde la separación puede ser más sencilla.

Kernel	Tipo de frontera
Lineal	Frontera más simple.
RBF	Frontera no lineal más flexible.

Esa flexibilidad es una ventaja, pero también aumenta el riesgo de sobreajuste. Por eso, al usar kernels no lineales, hiperparámetros como `C` y `gamma` deben ajustarse con validación cruzada y sin tocar el conjunto de prueba final.

Un tema fundamental del capítulo fue el escalado. SVM es muy sensible a la escala de las variables porque trabaja con márgenes, distancias y relaciones geométricas entre puntos. Si una variable tiene valores mucho más grandes que otra, puede dominar la frontera de decisión por una razón puramente numérica.

En SVM, la escala de las variables forma parte del problema geométrico.

Por eso, SVM suele entrenarse dentro de un pipeline que incluya `StandardScaler` u otra transformación adecuada. Si además hay variables categóricas, estas deben codificarse, por ejemplo con One-Hot Encoding, preferentemente dentro de un `ColumnTransformer`.

También revisamos sus ventajas. SVM puede funcionar bien cuando las clases tienen una separación razonable, cuando el dataset es pequeño o mediano, cuando hay muchas variables informativas y cuando queremos comparar una frontera lineal con una no lineal dentro de una misma familia de modelos. Su enfoque en el margen y su capacidad de usar kernels lo convierten en una herramienta potente.

Pero también tiene limitaciones. Puede ser costoso en datasets grandes, especialmente con kernels no lineales. Requiere escalado cuidadoso, puede ser sensible a hiperparámetros, no siempre es fácil de interpretar y no entrega probabilidades de manera tan directa como otros modelos.

SVM puede ser potente, pero exige buen preprocesamiento, una validación adecuada y un ajuste cuidadoso.

Support Vector Machines clasifica buscando una frontera que separe las clases con margen amplio. Con kernels, puede construir fronteras no lineales y adaptarse a problemas más complejos. Sin embargo, su desempeño depende fuertemente del escalado, de la elección de hiperparámetros y del tamaño del dataset. SVM no es una solución universal, pero puede ser una excelente opción cuando el problema tiene una estructura de separación clara, las variables están bien preparadas y la evaluación se realiza de manera cuidadosa.

Capítulo 24 · Naive Bayes

Un enfoque probabilístico

Naive Bayes es un modelo de clasificación basado en probabilidades. A diferencia de otros modelos que imaginamos mediante distancias, fronteras, árboles o márgenes, Naive Bayes intenta responder una pregunta probabilística:

Dadas estas variables observadas, ¿qué clase parece más probable?

Por ejemplo, si queremos clasificar un correo como spam o no spam, el modelo observa ciertas características del mensaje: palabras presentes, frecuencia de términos, aparición de enlaces, longitud del texto u otras señales. A partir de esa información, estima qué clase es más probable.

El proceso sigue esta secuencia: Correo observado, luego características del correo, luego probabilidad de spam, luego probabilidad de no spam y, finalmente, clase más probable.

El modelo no dice “este correo es spam porque cumple una regla exacta”. Tampoco busca vecinos cercanos como KNN ni construye una frontera geométrica como SVM. Su lógica es probabilística: compara qué tan probable sería observar esas características si el correo perteneciera a cada clase.

La idea general puede expresarse así:

Elegimos la clase que resulta más probable dadas las evidencias observadas.

Naive Bayes se apoya en el teorema de Bayes, una regla de la probabilidad que permite actualizar la probabilidad de una hipótesis cuando aparece nueva evidencia. No necesitamos desarrollar toda la matemática formal para entender su uso en clasificación, pero sí conviene captar la intuición.

Una hipótesis podría ser:

El correo es spam.

La evidencia serían las características observadas:

aparece la palabra "oferta" aparece un enlace aparece la palabra "gratis" el remitente no está en contactos.

Naive Bayes intenta estimar qué tan compatible es esa evidencia con cada clase posible. Si esas características son mucho más frecuentes en correos spam que en correos legítimos, el modelo aumentará la probabilidad de la clase spam.

Si ciertas características aparecen con frecuencia en una clase, el modelo las usa como evidencia a favor de esa clase.

Lo mismo puede pensarse en otros problemas. Si queremos clasificar reseñas como positivas o negativas, ciertas palabras pueden ser más frecuentes en reseñas positivas y otras en reseñas negativas. Si queremos clasificar noticias por tema, ciertos términos pueden aparecer más en notas deportivas, otros en notas económicas y otros en notas de tecnología.

Naive Bayes aprende esas asociaciones a partir de los datos de entrenamiento.

Durante el entrenamiento: el modelo estima qué tan frecuentes son ciertas características dentro de cada clase.

Luego, cuando llega un caso nuevo, usa esas frecuencias para calcular qué clase parece más probable.

Este enfoque tiene una ventaja importante: es simple y rápido. Naive Bayes no necesita construir árboles grandes, buscar vecinos entre muchos puntos ni ajustar fronteras complejas. Trabaja con conteos, frecuencias y probabilidades estimadas a partir del entrenamiento.

Naive Bayes: modelo simple, rápido, basado en probabilidades.

La palabra “Bayes” viene justamente de esa base probabilística. La palabra “naive”, que significa ingenuo o simple, se relaciona con un supuesto fuerte que el modelo hace sobre las variables. Ese supuesto lo estudiaremos en la próxima sección: asume que las características aportan información de manera independiente entre sí, dado el valor de la clase.

Por ahora, alcanza con saber que esa simplificación permite que el modelo sea muy eficiente, aunque también introduce limitaciones.

Naive Bayes funciona gracias a una simplificación fuerte. Esa simplificación lo hace rápido, pero también limita su realismo.

Para comprender mejor el enfoque probabilístico, imaginemos un pequeño ejemplo. Supongamos que tenemos mensajes clasificados como spam o no spam. En los mensajes spam, la palabra “gratis” aparece muchas veces. En los mensajes no spam,

aparece pocas veces. Si llega un mensaje nuevo con esa palabra, Naive Bayes la considera una evidencia a favor de spam.

Pero no mira una sola palabra. Puede combinar varias evidencias. Si el mensaje contiene “gratis”, “oferta”, “urgente” y “clic”, cada una de esas palabras puede aportar información. El modelo combina esas señales para estimar cuál clase resulta más probable.

Una señal aislada puede ser débil. Muchas señales combinadas pueden orientar la clasificación.

Esto es especialmente útil en texto, donde cada palabra puede funcionar como una característica. Un documento puede representarse mediante conteos de palabras o mediante indicadores de presencia o ausencia de términos. Naive Bayes puede usar esa representación para clasificar documentos con bastante eficiencia.

Por eso, Naive Bayes aparece con frecuencia como ejemplo clásico en clasificación de texto: spam, análisis de sentimiento, clasificación de noticias, temas de documentos o filtros simples de contenido.

Texto: muchas palabras, muchas señales pequeñas, clasificación probabilística.

Aunque hoy existen modelos mucho más avanzados para lenguaje, Naive Bayes sigue siendo valioso para aprender. Es claro, rápido, fácil de entrenar y ayuda a entender cómo un modelo puede usar probabilidades para clasificar.

También puede ser útil como modelo base. Si un modelo simple como Naive Bayes obtiene un desempeño razonable, puede servir como punto de comparación para modelos más complejos. Si un modelo mucho más sofisticado apenas lo mejora, quizá no justifique su complejidad en ciertos contextos.

Naive Bayes puede funcionar como referencia simple para comparar modelos más complejos.

En `scikit-learn`, existen distintas variantes de Naive Bayes. Una de las más usadas para texto con conteos de palabras es `MultinomialNB`.

```
from sklearn.naive_bayes import MultinomialNB

modelo = MultinomialNB()
```

El nombre “multinomial” se relaciona con datos de conteo, como la cantidad de veces que aparece cada palabra en un documento. Más adelante, cuando trabajemos ejemplos prácticos, esta variante será especialmente útil para clasificación de texto.

También existe `GaussianNB`, que se usa cuando las variables numéricas continuas se modelan bajo una distribución normal aproximada.

```
from sklearn.naive_bayes import GaussianNB

modelo = GaussianNB()
```

No necesitamos profundizar todavía en todas las variantes. Lo importante es reconocer que Naive Bayes no es una única implementación aislada, sino una familia de clasificadores basados en la misma idea general: estimar probabilidades por clase y elegir la clase más probable.

Naive Bayes: familia de clasificadores probabilísticos.

Como todos los modelos, Naive Bayes necesita datos bien definidos. La variable objetivo debe ser una clase. Las variables predictoras deben aportar señales relevantes. La evaluación debe hacerse con datos no vistos. Y, si hay preprocesamiento, debe integrarse correctamente para evitar fuga de datos.

Aunque sea un modelo simple, no está exento de errores metodológicos. Un Naive Bayes con fuga de datos también puede dar resultados engañosos. Un Naive Bayes evaluado sobre entrenamiento también puede producir una impresión demasiado optimista.

Modelo simple no significa evaluación simple. Las reglas metodológicas siguen siendo necesarias.

Naive Bayes es un enfoque probabilístico para clasificación. Observa las características de un caso y estima qué clase resulta más probable dadas esas evidencias. Aprende a partir de frecuencias y probabilidades en los datos de entrenamiento, y suele ser especialmente útil como modelo rápido, simple y claro. Su uso clásico aparece en clasificación de texto, donde muchas señales pequeñas, como palabras o términos, pueden combinarse para decidir una clase. Su simplicidad es una fortaleza, pero también depende de un supuesto fuerte que estudiaremos a continuación.

El supuesto de independencia

Naive Bayes es un modelo probabilístico, pero su simplicidad depende de una suposición muy fuerte: considera que las variables predictoras aportan información de manera independiente entre sí, una vez conocida la clase.

Esta idea es la razón por la que el modelo se llama “naive”, es decir, ingenuo. No porque el modelo sea inútil, sino porque hace una simplificación que muchas veces no se cumple exactamente en la realidad.

Supuesto de independencia: cada característica aporta información como si no dependiera de las demás, dada la clase.

Para entenderlo, volvamos al ejemplo de clasificación de correos. Supongamos que queremos decidir si un mensaje es spam o no spam. Algunas palabras pueden ser señales importantes: “oferta”, “gratis”, “urgente”, “clic”, “promoción”, “ganaste”.

Naive Bayes analiza esas señales y estima qué tan frecuentes son en cada clase. Pero, para simplificar el cálculo, trata a esas características como si cada una aportara evidencia por separado.

Palabra “gratis”: evidencia a favor o en contra de spam. Palabra “oferta”: otra evidencia. Palabra “urgente”: otra evidencia.

El problema es que, en el lenguaje real, las palabras no aparecen de forma completamente independiente. La palabra “oferta” puede aparecer junto con “promoción”. La palabra “clic” puede aparecer junto con “enlace”. La palabra “urgente” puede estar asociada con ciertos tipos de mensajes. Es decir, las características pueden estar relacionadas entre sí.

Naive Bayes simplifica esa relación. Actúa como si pudiera combinar esas evidencias sin modelar todas las dependencias entre ellas.

En la realidad: las variables pueden estar relacionadas. En Naive Bayes: se las trata como si aportaran evidencia de manera independiente.

Esta simplificación puede parecer demasiado fuerte. Y, en cierto sentido, lo es. En muchos problemas reales, las variables no son independientes. Las palabras de un texto se relacionan entre sí. Las características de una vivienda también: superficie, cantidad de ambientes y precio de publicación pueden estar asociadas. Las variables de un cliente pueden depender unas de otras.

Sin embargo, Naive Bayes puede funcionar sorprendentemente bien aun cuando el supuesto de independencia no se cumple perfectamente. Esto ocurre porque, para clasificar, muchas veces no necesitamos estimar probabilidades perfectas. Necesitamos que la clase correcta reciba una puntuación mayor que las demás.

Naive Bayes puede clasificar bien aunque sus probabilidades no sean perfectamente realistas.

Esta es una distinción importante. El modelo puede estar simplificando demasiado la realidad y aun así ordenar correctamente las clases en muchos casos. Por ejemplo, si un mensaje contiene muchas palabras típicas de spam, el modelo puede clasificarlo como spam aunque no haya modelado todas las relaciones entre esas palabras.

Dicho de otro modo, el supuesto puede ser falso en sentido estricto, pero útil en la práctica.

El supuesto de independencia no siempre describe bien el mundo, pero puede producir clasificaciones útiles.

La ventaja de esta simplificación es que el modelo se vuelve muy rápido y eficiente. Si intentáramos modelar todas las relaciones posibles entre todas las variables, el cálculo sería mucho más complejo. En texto, donde puede haber miles o decenas de miles de palabras, eso sería especialmente costoso.

Naive Bayes evita ese problema tratando cada característica de manera separada. Esto permite entrenar modelos con muchas variables de forma rápida.

Muchas variables + supuesto de independencia = entrenamiento simple y rápido.

Por ejemplo, en clasificación de texto, cada palabra puede ser una variable. Un documento puede tener miles de posibles términos. Modelar todas las combinaciones entre palabras sería poco práctico. Naive Bayes, en cambio, estima qué tan frecuente es cada palabra en cada clase y combina esas evidencias.

Esta eficiencia es una de las razones por las que Naive Bayes fue tan usado en tareas de texto. Permite construir un clasificador razonable con pocos recursos y con una implementación relativamente simple.

El supuesto de independencia también hace que el modelo sea más fácil de explicar. Podemos decir que ciertas palabras, valores o características aportan evidencia a favor de una clase. No necesitamos recorrer árboles, analizar márgenes ni combinar cientos de modelos. La lógica básica es más directa.

Naive Bayes: estima evidencias por clase y las combina para elegir la clase más probable.

Pero esta simplicidad tiene límites. Si la relación entre variables es esencial para distinguir las clases, Naive Bayes puede quedarse corto. Hay problemas donde una variable aislada no alcanza, pero la combinación entre variables sí es decisiva. Si el modelo no representa bien esas dependencias, puede perder información importante.

Supongamos un ejemplo simple. Una palabra por sí sola puede no indicar demasiado, pero la combinación de dos palabras sí puede cambiar el sentido. En lenguaje, expresiones como “no bueno” o “sin problemas” dependen de la relación entre términos. Analizar cada palabra por separado puede ser insuficiente.

Algunas señales no están en variables aisladas, sino en combinaciones entre variables.

Esto también puede ocurrir fuera del texto. En datos de clientes, una compra aislada puede no significar mucho, pero una caída de compras junto con reclamos recientes puede indicar riesgo de abandono. En salud, una medición aislada puede ser normal, pero su combinación con otra puede ser relevante. En esos casos, un modelo que represente interacciones puede superar a Naive Bayes.

El supuesto de independencia, entonces, debe entenderse como una simplificación útil, no como una descripción completa del problema.

Naive Bayes simplifica para poder aprender rápido. Esa simplificación puede perder relaciones importantes.

También afecta la interpretación de las probabilidades. Como el modelo combina evidencias bajo un supuesto fuerte, las probabilidades que produce pueden no estar bien calibradas. Es decir, una probabilidad estimada de 0.90 no necesariamente significa que, de cada 100 casos similares, 90 pertenecerán a esa clase. Puede ser una puntuación útil para comparar clases, pero no siempre una probabilidad confiable en sentido estricto.

Esto es importante si el problema requiere decisiones basadas en probabilidades precisas. Por ejemplo, si queremos ordenar riesgos, definir umbrales económicos o comunicar probabilidades a usuarios, debemos evaluar con cuidado la calibración del modelo.

Naive Bayes puede ordenar clases correctamente, pero sus probabilidades pueden ser demasiado confiadas o poco calibradas.

Aun así, para muchas tareas de clasificación, especialmente como modelo base, puede ser suficiente. Si queremos una primera solución rápida, clara y fácil de evaluar, Naive Bayes puede ser una excelente opción inicial.

En `scikit-learn`, las distintas variantes de Naive Bayes aplican esta idea general a distintos tipos de datos. `MultinomialNB` suele usarse con conteos, como conteos de palabras. `BernoulliNB` puede usarse con variables binarias, como presencia o ausencia de términos. `GaussianNB` se usa con variables numéricas continuas bajo ciertos supuestos de distribución.

```
from sklearn.naive_bayes import MultinomialNB
from sklearn.naive_bayes import BernoulliNB
from sklearn.naive_bayes import GaussianNB
```

No necesitamos profundizar ahora en las diferencias técnicas entre estas variantes. Lo importante es que todas comparten la misma intuición: estimar probabilidades por clase usando características tratadas de manera simplificada.

El supuesto de independencia es la simplificación central de Naive Bayes. El modelo trata las variables como si aportaran evidencia independiente una vez conocida la clase. En muchos problemas reales esto no se cumple exactamente, pero la simplificación permite entrenar modelos rápidos, simples y efectivos, especialmente en clasificación de texto. Su principal riesgo es perder relaciones importantes entre variables y producir probabilidades que no siempre deben interpretarse como estimaciones perfectamente calibradas.

Clasificación rápida y simple

Una de las grandes fortalezas de Naive Bayes es su simplicidad. El modelo no necesita construir una frontera compleja, recorrer vecinos, entrenar muchos árboles ni ajustar una larga secuencia de correcciones. Su lógica se basa en estimar probabilidades a partir de frecuencias observadas en los datos de entrenamiento.

Esa simplicidad hace que el entrenamiento sea rápido y que las predicciones también puedan calcularse con eficiencia.

Naive Bayes: entrenamiento rápido, predicción rápida, implementación simple.

Esto lo convierte en un modelo muy útil cuando necesitamos una primera solución de clasificación. En muchos proyectos, antes de probar modelos más complejos, conviene construir una referencia simple. Naive Bayes puede cumplir ese rol, especialmente cuando los datos tienen muchas variables simples y discretas, como ocurre con textos representados mediante palabras.

Por ejemplo, en un problema de spam, podemos entrenar un modelo que aprenda qué palabras son más frecuentes en correos spam y cuáles son más frecuentes en correos legítimos. Una vez entrenado, clasificar un nuevo correo consiste en observar sus palabras y calcular qué clase resulta más probable según lo aprendido.

El proceso sigue esta secuencia: Mensaje nuevo, luego palabras observadas, luego probabilidades por clase y, finalmente, clasificación final.

El modelo no necesita revisar todos los mensajes anteriores cada vez que clasifica uno nuevo, como sí ocurre con KNN. Tampoco necesita recorrer cientos de árboles, como en Random Forest, ni construir una secuencia de correcciones como Gradient Boosting. Una vez estimadas las probabilidades, la predicción es directa.

Esta eficiencia es especialmente importante cuando trabajamos con texto. Un conjunto de documentos puede transformarse en miles de variables, una por palabra o término. Aun así, Naive Bayes puede entrenarse rápidamente porque trata cada característica de manera simple.

Muchas palabras posibles no impiden necesariamente un entrenamiento rápido.

Por eso, Naive Bayes ha sido muy usado como clasificador base en tareas de texto. Puede servir para filtros de spam, clasificación de noticias, análisis de sentimiento básico o categorización de documentos. Aunque hoy existen modelos mucho más avanzados, sigue siendo una herramienta valiosa para aprender y para construir primeras soluciones.

Otra ventaja de su simplicidad es que resulta relativamente fácil de explicar. Podemos decir que el modelo aprendió qué características son más frecuentes en cada clase y luego usa esas evidencias para decidir. No necesitamos introducir estructuras demasiado complejas para comprender la intuición general.

Naive Bayes clasifica combinando evidencias a favor de cada clase.

Esta explicación no muestra todos los detalles matemáticos, pero sí transmite la lógica del modelo. Si ciertas palabras aparecen mucho más en una clase que en otra, funcionarán como señales. Si un documento contiene muchas señales asociadas a una clase, esa clase recibirá mayor probabilidad.

Por ejemplo, en una clasificación de noticias, palabras como “partido”, “gol”, “equipo” y “torneo” pueden aportar evidencia a favor de la categoría deportes. Palabras como “inflación”, “mercado”, “dólar” o “inversión” pueden aportar evidencia a favor de economía. El modelo aprende esas asociaciones desde los datos.

Las palabras no deciden solas. Aportan evidencia que el modelo combina.

También puede utilizarse con variables que no sean texto. Por ejemplo, podríamos tener variables categóricas o binarias que indiquen presencia o ausencia de ciertas características. En esos casos, Naive Bayes también puede combinar señales para estimar la clase más probable.

Sin embargo, su forma más típica y didáctica sigue siendo el texto, porque allí se ve con claridad la idea de muchas evidencias pequeñas.

La rapidez de Naive Bayes lo hace útil para establecer una línea de base. Una línea de base es un primer resultado razonable contra el cual comparar modelos más complejos. Si Naive Bayes obtiene una accuracy o un F1-score aceptable, los modelos posteriores deberían justificar su mayor complejidad con una mejora real.

Modelo base: primer punto de comparación para saber si un modelo más complejo realmente aporta valor.

Supongamos que Naive Bayes obtiene un F1-score de 0.82 en una tarea de clasificación de texto. Luego entrenamos un modelo mucho más costoso que obtiene 0.83. La mejora existe, pero quizá no justifique el aumento de complejidad. En cambio, si el modelo complejo obtiene 0.90, la diferencia puede ser más relevante.

Este tipo de comparación ayuda a construir criterio. Un modelo simple que funciona bien no debe descartarse solo por ser simple. A veces, la solución más sencilla es suficiente para el objetivo del proyecto.

Si un modelo simple funciona bien, la complejidad adicional debe estar justificada.

Otra ventaja práctica es que Naive Bayes puede funcionar con relativamente pocos datos en algunos contextos. Como estima frecuencias por clase y por característica, no siempre necesita grandes volúmenes para producir una primera aproximación. Esto no significa que los datos no importen. Si hay muy pocos ejemplos, las probabilidades pueden ser inestables. Pero, comparado con modelos más complejos, puede ser una opción inicial razonable cuando el dataset no es enorme.

También es relativamente robusto frente a muchas variables irrelevantes en contextos como texto, aunque no es inmune a ruido. Si una palabra no aporta demasiada información, su efecto puede ser limitado. Si muchas señales útiles aparecen combinadas, el modelo puede clasificar bien aunque cada señal individual sea simple.

Muchas señales pequeñas pueden producir una clasificación útil.

En `scikit-learn`, una forma típica de usar Naive Bayes para texto es combinar una transformación de texto con `MultinomialNB`. Por ejemplo, `CountVectorizer` transforma documentos en conteos de palabras y luego el modelo usa esos conteos para clasificar.

```
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.pipeline import Pipeline

pipeline = Pipeline(steps=[
    ("vectorizador", CountVectorizer()),
```

```
    ("modelo", MultinomialNB())  
  ])
```

Este pipeline representa un flujo completo: primero convierte el texto en variables numéricas y luego entrena el clasificador. Como ocurre con otros pipelines, esto ayuda a mantener el preprocesamiento dentro del proceso de evaluación.

El proceso sigue esta secuencia: Texto original, luego vectorización, luego modelo Naive Bayes y, finalmente, clase predicha.

Si usamos validación cruzada, el vectorizador debe ajustarse dentro de cada fold, no antes. Esto es importante porque el vocabulario aprendido a partir del texto también forma parte del preprocesamiento. Si lo aprendemos usando todos los datos antes de separar, podemos filtrar información del conjunto de validación o prueba.

En texto, el vocabulario aprendido también debe construirse solo con datos de entrenamiento.

Aunque Naive Bayes sea simple, seguimos necesitando las mismas precauciones metodológicas: separar datos, evitar fuga, usar métricas adecuadas y comparar de forma justa. Su simplicidad no elimina la necesidad de una evaluación correcta.

También debemos recordar que rapidez no significa perfección. Naive Bayes puede ser rápido y útil, pero puede quedarse corto si el problema depende mucho de relaciones entre variables. Como asume independencia, puede perder información cuando la combinación entre características es más importante que las características por separado.

Naive Bayes es rápido porque simplifica el problema. Esa simplificación puede limitar su desempeño.

Por eso, una forma responsable de usarlo es entrenarlo, evaluarlo y compararlo. Si funciona bien, puede ser una solución suficiente o una referencia sólida. Si funciona mal, puede indicar que necesitamos variables mejores, otra representación de los datos o modelos capaces de capturar interacciones más complejas.

Naive Bayes permite construir clasificadores rápidos y simples. Aprende probabilidades a partir de los datos de entrenamiento y usa esas probabilidades para elegir la clase más probable de un caso nuevo. Su eficiencia lo hace especialmente útil en clasificación de texto y como modelo base para comparar alternativas más complejas. Sin embargo, debe evaluarse con el mismo cuidado que cualquier otro modelo: usando pipelines, datos no vistos, métricas adecuadas y atención a sus supuestos.

Clasificación de texto como ejemplo típico

Naive Bayes suele aparecer asociado a la clasificación de texto porque en ese tipo de problemas su lógica se entiende con mucha claridad. Un texto puede convertirse en un conjunto de señales: palabras, términos, expresiones o conteos. Luego, el modelo estima qué clase es más probable a partir de esas señales.

Por ejemplo, si queremos clasificar correos electrónicos como spam o no spam, cada mensaje puede representarse mediante las palabras que contiene. Algunas palabras pueden ser más frecuentes en spam, otras en mensajes legítimos. Naive Bayes aprende esas frecuencias durante el entrenamiento y las usa para clasificar mensajes nuevos.

El proceso sigue esta secuencia: Texto, luego palabras o términos, luego frecuencias por clase, luego probabilidad de cada clase y, finalmente, clasificación final.

La idea no se limita al spam. También puede usarse para clasificar noticias por tema, reseñas como positivas o negativas, consultas de soporte por categoría, mensajes de estudiantes según tipo de pedido o documentos según área temática.

En todos esos casos, el texto original debe transformarse en variables numéricas. Los modelos de Machine Learning no trabajan directamente con párrafos como los leemos las personas. Necesitan una representación estructurada.

Una forma simple de representar textos es contar palabras. Si tenemos una colección de documentos, podemos construir un vocabulario con los términos que aparecen y luego representar cada documento según cuántas veces aparece cada palabra.

El proceso sigue esta secuencia: Documento:, luego "oferta exclusiva gratis", luego Variables posibles:, luego oferta $\rightarrow 1$, luego exclusiva $\rightarrow 1$, luego gratis $\rightarrow 1$, luego reunión $\rightarrow 0$ y, finalmente, factura $\rightarrow 0$.

Este enfoque se conoce como bolsa de palabras, o *bag of words*. El orden de las palabras se pierde, pero conservamos información sobre qué términos aparecen. Para Naive Bayes, esta representación suele ser suficiente para construir un clasificador rápido y razonable.

Bolsa de palabras: representa un texto por las palabras que contiene, sin considerar demasiado el orden.

La pérdida del orden es una simplificación importante. La frase "no es bueno" y la frase "es bueno" comparten palabras, pero significan cosas distintas. Un modelo basado solo en conteos puede tener dificultades para captar esas diferencias. Aun

así, en muchas tareas sencillas, la frecuencia de ciertos términos aporta suficiente señal para clasificar de manera útil.

En `scikit-learn`, una herramienta común para convertir textos en conteos de palabras es `CountVectorizer`.

```
from sklearn.feature_extraction.text import CountVectorizer

vectorizador = CountVectorizer()
```

Si combinamos `CountVectorizer` con `MultinomialNB`, obtenemos un flujo clásico para clasificación de texto:

```
from sklearn.pipeline import Pipeline
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB

pipeline = Pipeline(steps=[
    ("vectorizador", CountVectorizer()),
    ("modelo", MultinomialNB())
])
```

Este pipeline primero transforma los textos en una matriz de conteos y luego entrena el modelo Naive Bayes. La estructura es importante porque la creación del vocabulario también forma parte del aprendizaje. El vectorizador debe ajustarse solo con los textos de entrenamiento.

El vocabulario aprendido también debe considerarse parte del preprocesamiento.

Si construimos el vocabulario usando todos los textos antes de separar entrenamiento y prueba, podemos filtrar información del conjunto de prueba. Tal vez aparezcan palabras que solo estaban en el test y que no deberían haber sido conocidas durante el entrenamiento. Por eso, el vectorizador debe estar dentro del pipeline.

Durante la validación cruzada, el pipeline asegura que en cada fold el vocabulario se aprenda solo con la parte de entrenamiento. Luego se aplica a la parte de validación. Este cuidado evita una forma de fuga de datos.

Pipeline en texto: aprende vocabulario con entrenamiento, transforma los documentos, entrena el modelo y evalúa con datos no vistos.

Veamos una intuición más concreta. Supongamos que entrenamos un clasificador de noticias con tres categorías: deportes, economía y tecnología. Durante el entrenamiento, el modelo observa que ciertas palabras aparecen con más frecuencia en cada clase.

Deportes: partido, equipo, gol, torneo Economía: inflación, mercado, dólar, inversión
Tecnología: software, datos, inteligencia, dispositivo.

Si llega un texto nuevo con palabras como “equipo”, “torneo” y “gol”, el modelo encontrará evidencia a favor de deportes. Si aparecen “mercado”, “inversión” y “dólar”, encontrará evidencia a favor de economía. Si aparecen “software”, “modelo” y “datos”, puede favorecer tecnología.

Esto no significa que una sola palabra determine la clase. Cada término aporta una señal. Naive Bayes combina esas señales de manera probabilística.

Una palabra: puede aportar evidencia. Varias palabras: pueden reforzar una clasificación.

Una dificultad habitual en texto es que muchas palabras son muy frecuentes pero poco informativas. Palabras como “de”, “la”, “el”, “y”, “en” pueden aparecer en casi todos los documentos. En algunas tareas, estas palabras no ayudan demasiado a distinguir clases.

Por eso, los vectorizadores suelen permitir eliminar palabras muy comunes, conocidas como *stop words*. En español, esta tarea debe hacerse con cuidado, porque las listas de palabras vacías dependen del idioma y del contexto.

Palabras muy frecuentes: pueden aportar poca información para distinguir clases.

Otra posibilidad es limitar el vocabulario, ignorar términos demasiado raros o demasiado frecuentes, o usar representaciones alternativas como TF-IDF. TF-IDF no cuenta solamente cuántas veces aparece una palabra, sino que también considera qué tan común es en el conjunto de documentos. Una palabra muy frecuente en todos los textos suele recibir menos peso que una palabra más específica.

```
from sklearn.feature_extraction.text import TfidfVectorizer  
  
vectorizador = TfidfVectorizer()
```

Naive Bayes puede combinarse con distintas representaciones de texto, aunque **MultinomialNB** aparece muy asociado a conteos o frecuencias no negativas. En los cuadernos prácticos podremos comparar alguna de estas opciones de manera sencilla.

Una ventaja de Naive Bayes en texto es que puede funcionar incluso cuando tenemos muchas columnas. Si el vocabulario tiene miles de términos, cada documento se representa en un espacio con miles de variables. Muchos de esos valores serán cero,

porque cada documento contiene solo una parte del vocabulario. Esta estructura se llama dispersa.

Texto vectorizado: muchas columnas posibles, pero cada documento usa pocas. Eso genera matrices dispersas.

Naive Bayes maneja bien este tipo de representación. Por eso puede entrenarse rápido en tareas donde otros modelos podrían resultar más costosos.

Sin embargo, también tiene limitaciones. Al trabajar con bolsa de palabras, pierde parte del orden y del contexto. Puede confundir frases donde la relación entre palabras cambia el sentido. También puede fallar con ironía, ambigüedad, negaciones o significados que dependen del contexto.

El texto no es solo una lista de palabras. También tiene orden, contexto, tono e intención.

Por ejemplo, una reseña que dice “no fue una buena experiencia” contiene la palabra “buena”, pero el sentido general es negativo. Un modelo basado en conteos simples puede necesitar muchas evidencias adicionales para clasificar correctamente ese caso.

Esto no significa que Naive Bayes no sirva para texto. Significa que debemos entender qué tipo de representación estamos usando y qué información estamos perdiendo. Para problemas sencillos o como línea de base, puede ser excelente. Para análisis más finos del lenguaje, puede quedarse corto.

También debemos elegir métricas adecuadas. En clasificación de spam, por ejemplo, no alcanza con mirar accuracy si las clases están desbalanceadas. Tal vez haya muchos más correos legítimos que spam. En ese caso, debemos revisar precision, recall, F1-score y matriz de confusión.

En clasificación de texto, la métrica debe reflejar el costo real de los errores.

Marcar un correo legítimo como spam puede ser grave porque el usuario podría perder un mensaje importante. No detectar un spam también tiene costo, pero quizá sea distinto. La evaluación debe tener en cuenta ese equilibrio.

En análisis de sentimiento ocurre algo parecido. Si las clases positiva y negativa están desbalanceadas, accuracy puede ser insuficiente. Si agregamos una clase neutral, el problema se vuelve multiclase y debemos revisar el desempeño por clase.

Naive Bayes también puede ser útil para prototipos rápidos. Si tenemos un conjunto de textos etiquetados y queremos una primera solución, un pipeline con vectorizador

y `MultinomialNB` puede construirse con pocas líneas. Luego podemos comparar contra modelos más complejos.

Naive Bayes en texto: rápido para empezar, útil como referencia, fácil de comparar.

Si un modelo más complejo mejora poco, quizá Naive Bayes sea suficiente. Si mejora mucho, Naive Bayes habrá servido como referencia inicial para medir el valor de esa complejidad.

La clasificación de texto es un ejemplo típico para Naive Bayes porque los textos pueden representarse como muchas señales simples, especialmente palabras o conteos de términos. El modelo aprende qué señales son frecuentes en cada clase y las combina para elegir la clase más probable. Su rapidez y simplicidad lo hacen útil para spam, noticias, reseñas y categorización de documentos. Sin embargo, al usar representaciones simples como bolsa de palabras, puede perder orden, contexto, negaciones e ironía. Por eso debe evaluarse con métricas adecuadas y compararse con otros modelos cuando el problema lo requiera.

Ventajas

Naive Bayes tiene varias ventajas que explican por qué sigue siendo un modelo importante, especialmente como primera aproximación a problemas de clasificación. Aunque no suele ser el modelo más sofisticado, su simplicidad, rapidez y claridad lo convierten en una herramienta muy útil para aprender y para construir líneas de base.

Una primera ventaja es que se entrena muy rápido. El modelo no necesita recorrer vecinos, construir muchos árboles ni ajustar una frontera compleja. Aprende a partir de frecuencias y probabilidades estimadas en los datos de entrenamiento. Esto hace que el costo de entrenamiento sea bajo en comparación con otros modelos.

Naive Bayes: entrenamiento rápido a partir de conteos, frecuencias y probabilidades.

Esta rapidez es especialmente valiosa cuando tenemos muchas variables. En clasificación de texto, por ejemplo, un conjunto de documentos puede generar miles o decenas de miles de columnas, una por palabra o término. Aun así, Naive Bayes puede trabajar de manera eficiente con ese tipo de representación.

Esto lo convierte en una excelente opción para obtener un primer resultado. Cuando comenzamos un proyecto, no siempre conviene ir directamente al modelo más complejo. Muchas veces es mejor empezar con un modelo simple, evaluar su desempeño y usarlo como referencia.

Un modelo simple y rápido puede ser una excelente línea de base.

Si Naive Bayes obtiene un resultado razonable, nos da un punto de comparación. Cualquier modelo más complejo debería justificar su uso con una mejora suficiente. Si un Gradient Boosting o un SVM apenas mejora el resultado, tal vez la complejidad adicional no valga la pena. Si la mejora es clara, entonces el modelo base nos ayudó a medir cuánto aporta esa complejidad.

Otra ventaja es que sus predicciones también son rápidas. Una vez que el modelo estimó las probabilidades, clasificar un nuevo caso es relativamente directo. Esto puede ser importante en aplicaciones donde se deben clasificar muchos casos en poco tiempo.

Entrenar rápido y predecir rápido lo vuelve útil para prototipos y sistemas simples.

Naive Bayes también suele funcionar bien con datos de texto representados como conteos o frecuencias. Esta es quizá su aplicación clásica. En tareas como detección de spam, clasificación de noticias o análisis de sentimiento básico, muchas palabras funcionan como señales parciales. El modelo combina esas señales para estimar la clase más probable.

La representación de bolsa de palabras, aunque simple, encaja bien con la lógica del modelo. Cada término aporta evidencia a favor o en contra de una clase. Naive Bayes no necesita comprender el texto como una persona; solo necesita encontrar patrones estadísticos útiles en la presencia o frecuencia de términos.

En texto, muchas palabras pueden actuar como pequeñas evidencias a favor de distintas clases.

Otra ventaja es su relativa facilidad de explicación. Podemos describir su funcionamiento diciendo que aprende qué características son más frecuentes en cada clase y luego usa esas frecuencias para clasificar casos nuevos. Esta explicación es mucho más accesible que la de modelos con fronteras muy complejas o grandes ensambles.

Por ejemplo, en un clasificador de noticias, podemos explicar que ciertas palabras aparecen con más frecuencia en noticias deportivas, otras en noticias económicas y otras en noticias tecnológicas. El modelo usa esas asociaciones para decidir.

Naive Bayes permite explicar la clasificación como combinación de evidencias probabilísticas.

Esta claridad lo vuelve útil en contextos educativos. Ayuda a mostrar cómo un modelo puede aprender a partir de datos sin necesidad de introducir estructuras demasiado

complejas. También conecta de manera natural con ideas de probabilidad, frecuencias y evidencia.

Otra ventaja es que puede funcionar razonablemente bien incluso cuando el supuesto de independencia no se cumple perfectamente. Como vimos, ese supuesto es una simplificación fuerte. En muchos problemas reales, las variables están relacionadas. Sin embargo, para clasificar, el modelo puede seguir siendo útil si las evidencias apuntan con suficiente fuerza hacia una clase.

Aunque el supuesto sea simplificado, el modelo puede clasificar bien si las señales son informativas.

Esto es especialmente interesante porque muestra que un modelo no necesita representar toda la complejidad del mundo para ser útil. A veces, una aproximación simple alcanza para tomar decisiones razonables.

Naive Bayes también puede ser útil cuando tenemos pocos recursos computacionales. Como es liviano, puede entrenarse y evaluarse rápidamente en computadoras comunes. Esto lo hace atractivo para tareas iniciales, prototipos, actividades educativas o sistemas donde la eficiencia importa.

También puede funcionar con datasets no demasiado grandes. Si hay suficientes ejemplos por clase para estimar frecuencias útiles, el modelo puede producir una primera solución sin requerir grandes volúmenes de datos. Por supuesto, con muy pocos datos las estimaciones pueden ser inestables, pero esta limitación no le quita valor como punto de partida.

Naive Bayes puede ser útil cuando necesitamos una solución inicial, rápida y de bajo costo.

Otra ventaja práctica es que se integra fácilmente en pipelines. En texto, podemos combinar un vectorizador con el modelo en una estructura clara. Esto ayuda a evitar fuga de datos y permite usar validación cruzada de manera correcta.

```
from sklearn.pipeline import Pipeline
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB

pipeline = Pipeline(steps=[
    ("vectorizador", CountVectorizer()),
    ("modelo", MultinomialNB())
])
```

Este pipeline representa todo el flujo: convertir texto en variables numéricas y luego clasificar. Si lo evaluamos con validación cruzada, el vocabulario se aprende dentro de cada fold, usando solo los datos de entrenamiento correspondientes.

Pipeline: preprocesamiento + modelo en un flujo evaluable y ordenado.

También existen distintas variantes de Naive Bayes según el tipo de datos. `MultinomialNB` se usa mucho con conteos, `BernoulliNB` con variables binarias y `GaussianNB` con variables numéricas continuas bajo ciertos supuestos. Esto permite adaptar la idea general a diferentes representaciones.

Variantes comunes: `MultinomialNB` `BernoulliNB` `GaussianNB`.

No significa que cualquiera sirva para cualquier dato. Elegir la variante adecuada sigue siendo importante. Pero la existencia de estas opciones permite aplicar la intuición probabilística de Naive Bayes en distintos escenarios.

Otra ventaja es que, por su simplicidad, puede ayudar a detectar si el problema tiene señal. Si un modelo simple obtiene un desempeño claramente superior a una referencia trivial, eso sugiere que las variables contienen información útil. Si no logra mejorar casi nada, tal vez el problema necesite mejores datos, otra representación o una revisión de la variable objetivo.

Un modelo simple puede ayudar a preguntar: ¿hay señal suficiente en los datos?

Esto lo convierte en una herramienta de diagnóstico inicial. No solo sirve para clasificar, sino también para conocer mejor el problema.

Naive Bayes tiene ventajas importantes: es rápido de entrenar, rápido de predecir, simple de implementar, relativamente fácil de explicar y especialmente útil en clasificación de texto. Puede funcionar bien como línea de base y permite evaluar si los datos contienen señales útiles antes de avanzar hacia modelos más complejos. Aunque se apoya en un supuesto fuerte de independencia, muchas veces produce resultados razonables en la práctica. Su valor principal está en ofrecer una primera solución clara, eficiente y fácil de comparar.

Limitaciones

Naive Bayes es simple, rápido y útil, pero también tiene limitaciones importantes. Como todos los modelos, no debe elegirse solo por sus ventajas. Conviene entender qué simplifica, qué puede perder y en qué situaciones puede quedarse corto frente a otros enfoques.

La principal limitación está en su supuesto de independencia. Naive Bayes trata las variables como si aportaran evidencia de manera independiente una vez conocida la clase. En muchos problemas reales, esto no se cumple.

Supuesto fuerte: las variables aportan evidencia como si fueran independientes dada la clase.

En texto, por ejemplo, las palabras no aparecen aisladas. El significado de una palabra puede depender de las palabras que la rodean. La expresión “muy bueno” no transmite lo mismo que “no muy bueno”. La palabra “gratis” puede ser una señal de spam en algunos contextos, pero puede aparecer en un correo legítimo sobre una actividad gratuita.

Naive Bayes, especialmente cuando se usa con bolsa de palabras, no comprende esas relaciones como lo haría una persona. Trabaja con señales separadas y las combina de manera simplificada.

El texto tiene contexto, orden, negaciones e intención. Una bolsa de palabras pierde parte de esa información.

Esto puede provocar errores en casos donde el sentido depende mucho de combinaciones de términos. Frases con negación, ironía, doble sentido o expresiones ambiguas pueden ser difíciles para una representación basada solamente en conteos de palabras.

Por ejemplo, una reseña que dice “no fue una buena experiencia” contiene la palabra “buena”, pero su sentido general es negativo. Un modelo muy simple podría interpretar esa palabra como evidencia positiva si no cuenta con suficientes señales adicionales.

Algunas señales importantes no están en palabras aisladas, sino en cómo se combinan.

Esta limitación no aparece solo en texto. En datos tabulares, también puede ocurrir que la clase dependa de interacciones entre variables. Una variable aislada puede no ser demasiado informativa, pero su combinación con otra sí puede ser decisiva.

Por ejemplo, en un problema de abandono de clientes, una baja reciente en el uso del servicio puede ser importante. Un reclamo reciente también puede ser importante. Pero la combinación de baja de uso y reclamo reciente puede ser mucho más relevante que cada señal por separado.

Naive Bayes puede perder información cuando la clave está en la interacción entre variables.

Otro límite importante es que sus probabilidades no siempre deben interpretarse como probabilidades bien calibradas. El modelo puede asignar una clase

correctamente, pero dar valores de probabilidad demasiado extremos o poco realistas. Esto ocurre, en parte, por el supuesto de independencia.

Si varias variables relacionadas apuntan a la misma clase, Naive Bayes puede contar esas evidencias como si fueran más independientes de lo que realmente son. Como consecuencia, puede mostrarse demasiado seguro.

Naive Bayes puede clasificar bien y aun así producir probabilidades poco confiables.

Esto es relevante cuando las probabilidades tienen un rol central en la decisión. Si solo necesitamos asignar una clase, puede alcanzar con que el modelo ordene correctamente las alternativas. Pero si vamos a usar la probabilidad para definir umbrales, priorizar casos, estimar riesgos o comunicar certezas, debemos tener más cuidado.

Por ejemplo, si un sistema marca un mensaje como spam con probabilidad 0.98, no necesariamente significa que, de cada 100 mensajes similares, 98 sean spam. Puede ser una puntuación útil para decidir la clase, pero no una probabilidad perfectamente calibrada.

Probabilidad alta no siempre equivale a certeza bien calibrada.

Otra limitación aparece cuando hay características poco frecuentes. En clasificación de texto, muchas palabras pueden aparecer muy pocas veces. Si una palabra aparece solo en una clase durante el entrenamiento, el modelo puede asignar probabilidades problemáticas a documentos nuevos.

Para evitar algunos de estos problemas se usan técnicas de suavizado, como el parámetro `alpha` en `MultinomialNB`. El suavizado evita que ciertas probabilidades queden en cero por falta de observaciones.

```
from sklearn.naive_bayes import MultinomialNB

modelo = MultinomialNB(alpha=1.0)
```

No necesitamos profundizar ahora en la matemática del suavizado. Lo importante es entender que, cuando trabajamos con conteos y frecuencias, los términos raros o no observados pueden generar dificultades. El suavizado ayuda a reducir ese problema.

El suavizado ayuda cuando hay palabras o valores poco frecuentes en entrenamiento.

Naive Bayes también puede verse afectado por la calidad de la representación. En texto, el resultado depende mucho de cómo convertimos documentos en variables. No es lo mismo usar conteos simples, presencia o ausencia de palabras, TF-IDF, n-

gramas o limpieza previa del texto. El modelo puede ser simple, pero la representación sigue siendo fundamental.

En texto, la representación elegida puede ser tan importante como el modelo.

Si eliminamos palabras importantes, dejamos demasiados términos irrelevantes o no tratamos adecuadamente el idioma, el rendimiento puede verse afectado. En español, por ejemplo, debemos tener cuidado con acentos, plurales, conjugaciones verbales y palabras frecuentes que tal vez no ayuden a distinguir clases.

Otra limitación es que Naive Bayes puede quedarse corto cuando el problema necesita fronteras complejas. Modelos como SVM con kernel, Random Forest o Gradient Boosting pueden capturar relaciones no lineales e interacciones de manera más flexible. Si esas relaciones son importantes, Naive Bayes puede ofrecer un desempeño inferior.

Naive Bayes simplifica mucho. Esa simplicidad puede ser insuficiente cuando el problema es complejo.

Esto no significa que debamos descartarlo. Significa que debe evaluarse y compararse. Puede ser excelente como línea de base, pero no siempre será la solución final.

También puede ser menos adecuado cuando las variables numéricas continuas no encajan bien con la variante elegida. `GaussianNB`, por ejemplo, supone una forma de distribución para las variables numéricas dentro de cada clase. Si esa suposición es muy inadecuada, el modelo puede perder precisión.

Cada variante de Naive Bayes tiene supuestos propios sobre el tipo de datos.

Por eso, no alcanza con importar cualquier clase de `scikit-learn` y usarla sin pensar. `MultinomialNB`, `BernoulliNB` y `GaussianNB` responden a representaciones distintas. Elegir la variante incorrecta puede perjudicar el resultado.

Otra limitación es que, aunque el modelo sea rápido, puede no ser el mejor si necesitamos máxima precisión predictiva. En algunos problemas, un modelo más complejo puede capturar patrones que Naive Bayes ignora. Si el objetivo requiere el mejor desempeño posible, Naive Bayes suele ser un punto de partida, no necesariamente el destino final.

Naive Bayes: excelente punto de partida, pero no siempre el modelo más potente.

También debemos tener cuidado con el desbalance de clases. Como en cualquier clasificador, si una clase es mucho más frecuente que otra, la evaluación debe

hacerse con métricas adecuadas. Accuracy puede resultar engañosa si el modelo acierta muchos casos de la clase mayoritaria pero falla en la clase que más nos importa.

En spam, fraude, reclamos urgentes o detección de incidentes, no alcanza con mirar el porcentaje total de aciertos. Debemos revisar matriz de confusión, precision, recall y F1-score.

Modelo simple o modelo complejo: la métrica debe responder al objetivo del problema.

Por último, Naive Bayes no corrige problemas de datos. Si las etiquetas están mal, si hay fuga de datos, si las clases están mal definidas o si las variables no contienen señal, el modelo no puede resolverlo mágicamente. Puede entrenarse rápido, pero eso no garantiza que el problema esté bien planteado.

Naive Bayes aprende rápido, pero aprende desde los datos que le damos.

Naive Bayes tiene limitaciones claras. Su supuesto de independencia puede ser poco realista, puede perder interacciones importantes entre variables, sus probabilidades pueden no estar bien calibradas y su desempeño depende mucho de la representación elegida. En texto, puede fallar con contexto, negaciones, ironía o expresiones donde el orden importa. Además, cada variante del modelo tiene supuestos propios y debe elegirse según el tipo de datos. Aun así, estas limitaciones no eliminan su valor: Naive Bayes sigue siendo un modelo rápido, claro y útil como línea de base, siempre que se lo evalúe con criterio.

Clasificar con probabilidades simples

En este capítulo estudiamos Naive Bayes, una familia de modelos de clasificación basada en probabilidades. Su idea central es sencilla: observar las características de un caso y estimar qué clase resulta más probable dadas esas evidencias.

A diferencia de otros modelos, Naive Bayes no se apoya principalmente en distancias, árboles, márgenes o ensambles. Su lógica consiste en aprender, a partir de los datos de entrenamiento, qué características aparecen con mayor frecuencia en cada clase y usar esa información para clasificar nuevos casos.

El proceso sigue esta secuencia: Caso nuevo, luego características observadas, luego probabilidades por clase y, finalmente, clase más probable.

El modelo se apoya en una idea probabilística relacionada con el teorema de Bayes. No necesitamos dominar toda la matemática para comprender su intuición: si ciertas evidencias son más comunes en una clase que en otra, esas evidencias aumentan la probabilidad de esa clase.

También vimos que el nombre “naive” se debe a su supuesto principal: trata las variables como si aportaran información de manera independiente una vez conocida la clase. Este supuesto suele ser poco realista, porque en los problemas reales las variables muchas veces están relacionadas entre sí.

Supuesto de independencia: cada característica aporta evidencia como si no dependiera de las demás, dada la clase.

Sin embargo, esa simplificación es justamente lo que permite que el modelo sea rápido, simple y eficiente. En muchos problemas, especialmente en clasificación de texto, Naive Bayes puede funcionar razonablemente bien aunque el supuesto de independencia no se cumpla de manera perfecta.

La clasificación de texto es uno de sus ejemplos más típicos. Un correo, una reseña, una noticia o un mensaje pueden representarse mediante palabras o términos. Cada palabra puede funcionar como una pequeña señal a favor de una clase. El modelo combina esas señales para estimar la clase más probable.

El proceso sigue esta secuencia: Texto, luego palabras o conteos, luego evidencias por clase y, finalmente, clasificación.

En este contexto, aparecen herramientas como `CountVectorizer`, que transforma textos en conteos de palabras, y `MultinomialNB`, una variante de Naive Bayes adecuada para trabajar con conteos o frecuencias no negativas.

```
from sklearn.pipeline import Pipeline
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB

pipeline = Pipeline(steps=[
    ("vectorizador", CountVectorizer()),
    ("modelo", MultinomialNB())
])
```

Este ejemplo también nos permitió recordar una idea metodológica importante: el preprocesamiento debe formar parte del pipeline. En texto, el vocabulario aprendido por el vectorizador también se ajusta a partir de los datos, por lo que debe aprenderse solo con el conjunto de entrenamiento.

El vocabulario aprendido también forma parte del preprocesamiento.

Entre sus ventajas, Naive Bayes se destaca por ser rápido de entrenar, rápido de predecir, simple de implementar y relativamente fácil de explicar. Puede funcionar como una excelente línea de base para comparar modelos más complejos. Si un modelo más sofisticado apenas mejora su desempeño, tal vez no justifique la complejidad adicional.

Naive Bayes: simple, rápido, claro, útil como modelo base.

Pero también tiene limitaciones. Su supuesto de independencia puede hacer que pierda relaciones importantes entre variables. En texto, puede fallar cuando el sentido depende del orden, el contexto, la negación, la ironía o combinaciones específicas de palabras. Además, sus probabilidades no siempre deben interpretarse como probabilidades perfectamente calibradas.

Puede clasificar bien, pero sus probabilidades pueden no ser totalmente confiables como estimaciones exactas.

También depende mucho de la representación elegida. En texto, no es lo mismo usar conteos simples, presencia o ausencia de palabras, TF-IDF, n-gramas o distintas formas de limpieza. Aunque el modelo sea simple, la forma en que convertimos los datos en variables sigue siendo decisiva.

Naive Bayes clasifica usando probabilidades simples aprendidas desde los datos. Su fuerza está en la rapidez, la claridad y la capacidad de combinar muchas señales pequeñas, especialmente en problemas de texto. Su debilidad está en la simplificación que realiza: trata las variables como independientes y puede perder relaciones importantes. Por eso, es una herramienta muy valiosa como modelo inicial o línea de base, siempre que se la evalúe con métricas adecuadas y se la compare con otras alternativas cuando el problema lo requiera.

Capítulo 25 · Aprender sin etiquetas

¿Qué cambia cuando no tenemos variable objetivo?

Hasta ahora, la mayor parte del recorrido estuvo centrada en problemas supervisados. En esos casos, el modelo aprende a partir de ejemplos donde conocemos la respuesta correcta. Tenemos variables de entrada, que llamamos X , y una variable objetivo, que llamamos y .

El proceso parte de Aprendizaje supervisado: $X \rightarrow$ datos de entrada y conduce a $y \rightarrow$ respuesta conocida.

Si el problema es de clasificación, y contiene categorías: spam o no spam, abandona o no abandona, positivo o negativo, aprobado o desaprobado. Si el problema es de regresión, y contiene valores numéricos: precio, demanda, consumo, temperatura o ventas.

En ambos casos, el modelo aprende comparando sus predicciones con una respuesta conocida. Esa respuesta funciona como guía. Si el modelo se equivoca, podemos medir el error. Si acierta, podemos medir el desempeño. Por eso hablamos de aprendizaje supervisado: durante el entrenamiento, existe una señal que orienta el aprendizaje.

En el aprendizaje supervisado, el modelo dispone de ejemplos con una respuesta conocida.

Pero no siempre contamos con esa respuesta. Muchas veces tenemos datos, pero no etiquetas. Podemos tener información sobre clientes, productos, transacciones, documentos, imágenes, mediciones o comportamientos, sin que exista una columna que indique la clase correcta o el valor que queremos predecir.

En esos casos no podemos plantear el problema de la misma manera. Ya no tenemos una variable objetivo y que el modelo deba aprender a predecir. Solo tenemos X .

El proceso parte de Aprendizaje no supervisado: $X \rightarrow$ datos disponibles y conduce a $y \rightarrow$ no existe o no está disponible.

Este cambio es fundamental. Sin variable objetivo, el modelo no puede aprender a responder una pregunta del tipo “¿cuál es la clase correcta?” o “¿qué valor numérico debería predecir?”. Tampoco podemos evaluar el resultado comparando predicciones contra respuestas conocidas, porque esas respuestas no están.

Entonces, el objetivo cambia. En lugar de predecir una etiqueta o un valor, buscamos descubrir estructura en los datos.

Sin **y**, no buscamos predecir una respuesta conocida, sino encontrar estructuras, patrones o relaciones internas.

Por ejemplo, una empresa puede tener datos de sus clientes: edad, zona, frecuencia de compra, monto promedio, productos elegidos y antigüedad. Pero quizá no tenga una etiqueta que diga qué tipo de cliente es cada uno. En ese caso, un modelo no supervisado puede ayudar a buscar grupos de clientes con comportamientos parecidos.

El proceso parte de Clientes, continúa con variables de comportamiento y conduce finalmente a grupos sugeridos por similitud.

Esos grupos no vienen definidos de antemano. El modelo no sabe que existe una clase llamada “cliente frecuente”, “cliente ocasional” o “cliente sensible al precio”. Lo que hace es analizar las variables disponibles y proponer agrupamientos según la similitud entre casos.

Este es un cambio importante respecto de la clasificación. En clasificación, las clases ya existen en los datos de entrenamiento. En agrupamiento, los grupos son sugeridos por el modelo.

Concepto	Descripción
Clasificación	las clases ya están definidas.
Agrupamiento	los grupos se descubren a partir de los datos.

También podemos tener un conjunto de documentos sin categorías asignadas. Un modelo no supervisado podría ayudar a detectar temas, organizar documentos similares o encontrar conjuntos de textos que comparten vocabulario. No estaría prediciendo una categoría previamente conocida, sino proponiendo una organización posible.

Lo mismo puede ocurrir con mediciones de sensores. Podemos tener miles de registros de funcionamiento de una máquina sin saber cuáles corresponden a fallas. Un enfoque no supervisado puede buscar patrones normales y detectar casos que se apartan mucho de ellos. Esos casos podrían revisarse como posibles anomalías.

El proceso parte de Datos sin etiquetas, continúa con patrones habituales, luego casos que se alejan del patrón y conduce finalmente a posibles anomalías.

En este tipo de problemas, el aprendizaje no supervisado no reemplaza el juicio humano. Sus resultados suelen necesitar interpretación. Un grupo encontrado por el modelo no necesariamente tiene un significado claro por sí mismo. Una anomalía detectada no necesariamente es un error, un fraude o una falla. Puede ser simplemente un caso distinto.

El modelo puede sugerir una estructura, pero su interpretación sigue siendo responsabilidad humana.

Esta diferencia hace que la evaluación sea más difícil. En clasificación, podemos calcular accuracy, precision, recall o F1-score porque conocemos la clase verdadera. En regresión, podemos calcular MAE, RMSE o R^2 porque conocemos el valor real. En aprendizaje no supervisado, muchas veces no tenemos una respuesta externa contra la cual comparar.

Por eso, evaluar un resultado no supervisado requiere otros criterios. Podemos revisar si los grupos son coherentes, si son útiles para tomar decisiones, si se mantienen al cambiar ciertos parámetros, si tienen sentido para personas que conocen el dominio o si ayudan a explorar mejor los datos.

Sin etiquetas, la evaluación no depende solamente de una métrica automática.

Esto no significa que no existan métricas para aprendizaje no supervisado. Existen medidas para evaluar agrupamientos, reducción de dimensionalidad o detección de anomalías. Pero esas métricas no siempre tienen una interpretación tan directa como comparar contra una respuesta conocida. Deben usarse con cuidado.

El aprendizaje no supervisado suele estar más cerca de la exploración que de la predicción directa. Puede ayudar a formular hipótesis, descubrir patrones, ordenar datos, reducir complejidad o preparar análisis posteriores.

El aprendizaje no supervisado se utiliza para explorar, organizar, resumir y detectar estructuras.

Por ejemplo, antes de construir una estrategia comercial, una empresa puede usar agrupamiento para explorar perfiles de clientes. Antes de entrenar un modelo supervisado, un equipo puede usar reducción de dimensionalidad para visualizar datos complejos. Antes de investigar incidentes, puede usar detección de anomalías para encontrar casos que merecen revisión.

En todos estos casos, el modelo no entrega una respuesta definitiva. Entrega una forma de mirar los datos.

El resultado de un método no supervisado no suele ser una conclusión final, sino una herramienta de análisis.

Otra diferencia importante es que los nombres de los grupos o patrones no los da el modelo. Si un algoritmo encuentra tres grupos de clientes, esos grupos pueden aparecer como `0`, `1` y `2`. Esas etiquetas son internas y no significan nada por sí mismas. Para interpretarlas, debemos observar las características de cada grupo.

Los nombres `Cluster 0`, `Cluster 1` y `Cluster 2` son etiquetas técnicas; su significado debe analizarse después.

Quizá el grupo 0 tenga clientes con compras frecuentes y montos altos. El grupo 1, clientes nuevos con pocas compras. El grupo 2, clientes antiguos pero con baja actividad reciente. Recién después de analizar sus características podemos darles una interpretación útil.

Esto exige prudencia. No conviene presentar grupos descubiertos por un algoritmo como si fueran verdades naturales. Son resultados de una técnica aplicada sobre ciertos datos, con ciertas variables y ciertos parámetros. Si cambiamos las variables, el algoritmo o la escala de los datos, los grupos pueden cambiar.

Los grupos encontrados dependen de los datos, las variables, el preprocesamiento y el algoritmo elegido.

Por eso, el aprendizaje no supervisado requiere tanto cuidado metodológico como el aprendizaje supervisado. Aunque no haya `y`, seguimos necesitando datos limpios, variables relevantes, preprocesamiento adecuado, escalado cuando corresponda y una interpretación responsable.

En especial, cuando los algoritmos dependen de distancias, como ocurre con K-Means, la escala de las variables puede influir mucho. Una variable con valores grandes puede dominar el agrupamiento, igual que ocurría en modelos como KNN o SVM.

Trabajar sin etiquetas no significa trabajar sin preparación de datos.

También debemos recordar que “no supervisado” no significa “objetivo” o “neutral”. Los datos pueden contener sesgos. Las variables elegidas pueden representar solo una parte del problema. Las decisiones humanas siguen apareciendo al seleccionar datos, limpiar información, elegir algoritmos, configurar parámetros e interpretar resultados.

Cuando no tenemos variable objetivo, cambia la naturaleza del aprendizaje. El modelo ya no aprende a predecir una respuesta conocida, sino que busca estructura en los datos disponibles. Esto permite agrupar casos similares, reducir dimensiones, detectar anomalías o explorar patrones ocultos. Pero también hace que la evaluación y la interpretación sean más delicadas. En aprendizaje no supervisado, el modelo puede sugerir formas de organizar los datos, pero el significado de esas formas debe construirse con análisis, contexto y criterio.

Datos no etiquetados

En aprendizaje no supervisado trabajamos con datos no etiquetados. Esto significa que tenemos observaciones y variables descriptivas, pero no tenemos una columna que indique la respuesta correcta que el modelo debe aprender.

En un problema supervisado, cada fila suele venir acompañada por una etiqueta o valor objetivo. En cambio, en un problema no supervisado, las filas están disponibles, pero no sabemos de antemano a qué clase pertenecen, qué grupo forman o qué resultado deberían producir.

El proceso parte de Datos etiquetados: $X \rightarrow$ variables de entrada y conduce a $y \rightarrow$ respuesta conocida.

El proceso parte de Datos no etiquetados: $X \rightarrow$ variables disponibles y conduce a $y \rightarrow$ no está presente.

Por ejemplo, imaginemos una tabla de clientes con información sobre sus compras. Podemos conocer cuántas veces compraron, cuánto gastaron en promedio, qué categorías de productos eligieron, desde cuándo son clientes y en qué zona viven. Pero quizá no tengamos una columna que diga “tipo de cliente”.

cliente | frecuencia_compra | gasto_promedio | antigüedad | zona.

Esa tabla contiene información útil, pero no contiene etiquetas. No hay una respuesta previamente definida. El modelo no puede aprender a predecir “cliente frecuente”, “cliente ocasional” o “cliente en riesgo”, porque esas categorías no existen todavía en los datos.

En este contexto, un algoritmo no supervisado puede ayudarnos a buscar grupos de clientes parecidos. Pero esos grupos no son etiquetas verdaderas. Son agrupamientos sugeridos a partir de las variables disponibles.

El modelo no descubre una etiqueta correcta, sino que sugiere una posible organización de los datos.

Esta diferencia es importante. En clasificación, si el modelo predice mal, podemos comparar con la etiqueta verdadera. En agrupamiento, no tenemos esa etiqueta. Si el algoritmo propone tres grupos, no podemos decir automáticamente que esos tres grupos son “correctos”. Debemos analizarlos.

Los datos no etiquetados son muy frecuentes en la práctica. Muchas organizaciones acumulan registros de actividad, transacciones, consultas, productos, usuarios, sensores o documentos sin que cada caso esté clasificado manualmente.

Muchos datos reales existen antes de que alguien los etiquete o los interprete.

Etiquetar datos puede ser costoso. Requiere tiempo, criterio y, en algunos casos, conocimiento experto. Clasificar miles de correos, imágenes, noticias, reclamos o historias clínicas puede demandar mucho trabajo humano. Por eso, muchas veces se empieza explorando datos sin etiquetas.

Por ejemplo, una plataforma educativa puede tener registros de uso: cantidad de ingresos, actividades realizadas, tiempos de conexión, entregas completadas y consultas enviadas. Pero quizá no tenga una etiqueta confiable que indique qué estudiantes necesitan acompañamiento adicional.

Un enfoque no supervisado podría ayudar a identificar patrones de uso. Tal vez aparezcan grupos con comportamientos diferentes: estudiantes muy activos, estudiantes que ingresan poco, estudiantes que comienzan actividades pero no las completan, o estudiantes con actividad irregular.

El proceso parte de Datos de uso, continúa con patrones de comportamiento, luego grupos posibles y conduce finalmente a análisis posterior.

Pero sería incorrecto asumir automáticamente que esos grupos representan categorías definitivas. Un grupo con poca actividad no necesariamente significa “bajo compromiso”. Puede deberse a problemas de conectividad, horarios laborales, dificultades familiares, uso compartido de dispositivos o simplemente otra forma de organizar el estudio.

Un patrón observado no equivale automáticamente a una explicación.

Esta advertencia es central en aprendizaje no supervisado. Los datos no etiquetados permiten descubrir estructuras posibles, pero el significado de esas estructuras no está garantizado. Debe interpretarse con contexto.

Otro ejemplo puede aparecer en documentos. Supongamos que tenemos cientos de consultas recibidas por una mesa de ayuda. No están clasificadas por tema. Un algoritmo puede agrupar textos con vocabulario similar. Tal vez algunos grupos correspondan a problemas de contraseña, otros a errores de carga, otros a consultas administrativas.

El proceso parte de Consultas sin categoría, continúa con textos similares, luego grupos sugeridos y conduce finalmente a posibles temas.

El algoritmo no sabe qué es una contraseña, una inscripción o un error administrativo. Solo analiza similitudes en la representación de los textos. Luego, una persona debe revisar los grupos para decidir si tienen sentido y qué nombre podrían recibir.

Esto muestra otra característica de los datos no etiquetados: muchas veces el resultado es exploratorio. No buscamos una respuesta final, sino una forma de ordenar información que sería difícil revisar manualmente desde cero.

Los datos no etiquetados suelen utilizarse para explorar, organizar y descubrir patrones iniciales.

También puede ocurrir en detección de anomalías. Podemos tener registros de transacciones sin una etiqueta que diga cuáles son fraudulentas. En lugar de entrenar un clasificador supervisado, podemos buscar casos que se alejan mucho del comportamiento habitual.

El proceso parte de Transacciones sin etiqueta, continúa con comportamiento habitual, luego casos inusuales y conduce finalmente a posibles anomalías.

Pero una anomalía no es necesariamente un fraude. Puede ser una compra legítima poco común, un cliente nuevo, un evento estacional o un error de carga. El modelo señala casos que merecen revisión, no dicta una conclusión definitiva.

Trabajar con datos no etiquetados también exige cuidar la selección de variables. Como no hay una variable objetivo que guíe el aprendizaje, las variables que elegimos tienen mucho peso en la estructura que el algoritmo puede encontrar. Si incluimos variables poco relevantes, redundantes o mal escaladas, el resultado puede ser confuso.

Sin etiquetas, la elección de variables influye mucho en los patrones encontrados.

Por ejemplo, si agrupamos clientes usando solo su zona geográfica, los grupos reflejarán principalmente ubicación. Si usamos frecuencia de compra y gasto promedio, los grupos reflejarán comportamiento comercial. Si usamos edad, ingresos

y productos comprados, aparecerá otra estructura. El algoritmo no decide qué dimensión del problema es importante; trabaja con lo que le damos.

Esto significa que los datos no etiquetados no eliminan las decisiones humanas. Al contrario, hacen más importante definir qué queremos explorar y qué variables representan mejor esa pregunta.

El algoritmo organiza los casos según las variables disponibles, pero la elección de esas variables es una decisión humana.

También debemos considerar el preprocesamiento. Si las variables están en escalas muy diferentes, algunos algoritmos no supervisados pueden quedar dominados por las variables con valores más grandes. Esto es especialmente relevante en métodos basados en distancia, como K-Means, que estudiaremos en el próximo capítulo.

Supongamos que agrupamos personas usando dos variables: ingresos mensuales en pesos y cantidad de compras. Los ingresos pueden tener valores de cientos de miles, mientras que la cantidad de compras puede estar entre 0 y 20. Si no escalamos, la variable de ingresos puede dominar el agrupamiento.

Las escalas diferentes pueden producir agrupamientos dominados por una sola variable.

Por eso, aunque no tengamos `y`, seguimos necesitando un flujo de trabajo cuidadoso: revisar datos faltantes, codificar variables categóricas si corresponde, escalar variables numéricas cuando el algoritmo lo requiera y documentar las decisiones tomadas.

Los datos no etiquetados también pueden ser un punto de partida para construir etiquetas. A veces, los grupos encontrados por un algoritmo ayudan a que especialistas revisen casos y definan categorías más claras. En otros casos, permiten seleccionar ejemplos representativos para etiquetar manualmente.

El aprendizaje no supervisado puede ayudar a explorar los datos antes de crear etiquetas formales.

Esto no convierte automáticamente el problema en supervisado, pero puede ser parte de un proceso mayor. Primero exploramos estructuras. Luego interpretamos. Después, si tiene sentido, podemos definir etiquetas y construir modelos supervisados.

Los datos no etiquetados son datos que no incluyen una respuesta conocida para entrenar un modelo supervisado. En lugar de aprender a predecir y , los algoritmos no supervisados buscan estructuras internas en X : grupos, patrones, dimensiones principales o casos inusuales. Esta exploración puede ser muy útil, pero requiere prudencia. Los resultados no vienen con significado garantizado; deben interpretarse según las variables usadas, el preprocesamiento aplicado y el contexto del problema.

Buscar estructura en los datos

Cuando no tenemos etiquetas, el objetivo principal no es predecir una respuesta conocida, sino buscar estructura en los datos. Esa estructura puede tomar distintas formas: grupos de casos parecidos, patrones frecuentes, relaciones entre variables, dimensiones principales o casos que se comportan de manera inusual.

En aprendizaje supervisado, el modelo tiene una guía externa: la variable objetivo. En aprendizaje no supervisado, esa guía no está. Por eso, el modelo debe trabajar únicamente con las variables disponibles y tratar de encontrar regularidades internas.

Concepto	Descripción
Aprendizaje supervisado	aprender una relación entre X e y .
Aprendizaje no supervisado	buscar estructura dentro de X .

Esta diferencia cambia la forma de pensar el problema. Ya no preguntamos “¿qué clase corresponde a este caso?” ni “¿qué valor debería predecir?”. Preguntamos algo más exploratorio: “¿hay casos parecidos entre sí?”, “¿existen patrones que se repiten?”, “¿podemos resumir mejor estos datos?”, “¿hay observaciones que se alejan del comportamiento habitual?”.

Preguntas típicas:

¿Qué casos se parecen? ¿Qué grupos aparecen? ¿Qué variables se relacionan? ¿Qué casos son inusuales?

Buscar estructura significa encontrar algún tipo de organización que no estaba explícita en los datos. Esa organización puede ayudarnos a comprender mejor el conjunto, pero no debe interpretarse automáticamente como una verdad definitiva.

Por ejemplo, si tenemos datos de clientes, un algoritmo puede encontrar grupos según frecuencia de compra, monto promedio, antigüedad y categorías elegidas.

Quizá algunos clientes compren poco pero con montos altos. Otros compren frecuentemente, pero montos pequeños. Otros hayan comprado mucho en el pasado, pero estén reduciendo su actividad.

El proceso parte de Clientes, continúa con variables de comportamiento y conduce finalmente a estructura posible: grupos con hábitos parecidos.

El algoritmo no sabe qué significa cada grupo desde el punto de vista del negocio. Solo encuentra similitudes según las variables disponibles. Después, una persona debe analizar los grupos, describirlos y decidir si son útiles.

Esa interpretación puede transformar un resultado técnico en conocimiento práctico. Un grupo puede convertirse en “clientes frecuentes”, “clientes nuevos”, “clientes de alto gasto” o “clientes con baja actividad reciente”. Pero esos nombres no los descubre el algoritmo: los construimos nosotros al observar las características de cada grupo.

El modelo puede sugerir grupos, pero su significado se construye mediante el análisis humano.

También podemos buscar estructura en documentos. Si tenemos muchas consultas, reclamos o comentarios sin clasificar, podemos intentar agrupar textos similares. Tal vez aparezcan temas relacionados con pagos, acceso a la plataforma, problemas técnicos o solicitudes administrativas.

En ese caso, la estructura no está en una etiqueta previa, sino en las semejanzas entre textos. Los documentos que usan vocabulario parecido o tratan temas cercanos pueden quedar cerca en la representación numérica.

El proceso parte de Textos sin categoría, continúa con representación numérica, luego similitudes y conduce finalmente a posibles temas.

Pero nuevamente debemos ser cuidadosos. Dos textos pueden usar palabras similares y tratar temas distintos. También pueden tratar el mismo tema usando palabras diferentes. El modelo trabaja con la representación que le damos, no con una comprensión humana completa del contenido.

Buscar estructura también puede significar reducir la complejidad de los datos. A veces tenemos muchas variables y resulta difícil analizarlas directamente. Un método de reducción de dimensionalidad puede ayudarnos a construir una versión más resumida del dataset, conservando parte importante de la información.

El proceso parte de Muchas variables, continúa con representación más compacta y conduce finalmente a análisis o visualización más manejable.

Esto puede ser útil para visualizar datos, detectar patrones globales o preparar información para otros análisis. En capítulos posteriores estudiaremos PCA, una técnica clásica de reducción de dimensionalidad.

Otra forma de estructura aparece en la detección de anomalías. Si tenemos muchos casos que siguen un comportamiento habitual, podemos buscar observaciones que se apartan de ese patrón. Esas observaciones no necesariamente son errores, fraudes o fallas, pero pueden merecer atención.

El proceso parte de Comportamiento habitual, continúa con casos que se alejan mucho y conduce finalmente a posibles anomalías.

Por ejemplo, en datos de transacciones, una compra muy distinta al comportamiento usual de una persona podría ser una anomalía. En datos de sensores, una medición muy fuera de lo habitual podría indicar una falla o una condición especial. En registros de acceso, un patrón poco común podría requerir revisión.

En todos estos casos, el modelo no entrega una explicación final. Señala estructuras posibles. Luego debemos preguntar si esas estructuras tienen sentido, si son estables, si son útiles y si se relacionan con el problema real.

Encontrar una estructura no equivale automáticamente a comprender el fenómeno.

La estructura encontrada depende de muchas decisiones previas. Depende de qué variables elegimos, cómo tratamos los valores faltantes, cómo codificamos variables categóricas, si escalamos o no las variables numéricas, qué algoritmo usamos y con qué parámetros.

Por ejemplo, si agrupamos clientes usando edad e ingresos, aparecerá una estructura. Si usamos frecuencia de compra y gasto promedio, puede aparecer otra. Si agregamos zona geográfica, el resultado puede cambiar. El algoritmo no descubre una única verdad escondida; encuentra una organización según la información que le damos.

Si cambian las variables, también puede cambiar la estructura encontrada.

Esto no es un defecto del aprendizaje no supervisado, sino una característica que debemos entender. Los resultados son sensibles al punto de vista desde el cual analizamos los datos. Por eso, antes de aplicar un algoritmo, conviene preguntarnos qué tipo de estructura estamos buscando y qué variables representan mejor esa pregunta.

Si queremos segmentar clientes por comportamiento de compra, probablemente importen variables como frecuencia, monto promedio, categorías compradas y

recencia. Si queremos analizar distribución geográfica, importarán otras variables. Si queremos detectar uso inusual de una plataforma, necesitaremos variables relacionadas con actividad, horarios, accesos o secuencia de eventos.

La estructura que resulte útil dependerá de la pregunta que queremos explorar.

Otro aspecto importante es que algunas técnicas no supervisadas son muy sensibles a la escala. Si usamos un algoritmo basado en distancia, una variable con valores grandes puede dominar el resultado. Por eso, el escalado puede ser tan importante como en KNN o SVM.

Supongamos que agrupamos usuarios usando dos variables: ingresos mensuales en pesos y cantidad de compras. Si los ingresos varían entre 100.000 y 900.000, mientras que las compras varían entre 0 y 20, la distancia entre usuarios puede quedar dominada por los ingresos.

Variable en pesos: valores muy grandes.

Variable en cantidad: valores pequeños.

Sin escalado, una puede dominar a la otra.

En ese caso, los grupos encontrados podrían reflejar principalmente diferencias de ingresos, aunque nuestro interés fuera analizar hábitos de compra. El preprocesamiento, entonces, no es un detalle técnico secundario: define qué estructura podrá ver el algoritmo.

Buscar estructura también requiere revisar si los resultados son útiles. Un algoritmo puede producir grupos matemáticamente separados, pero poco relevantes para la decisión que queremos tomar. Puede encontrar patrones que existen en los datos, pero que no aportan comprensión o acción.

Una estructura puede existir y, aun así, no resultar útil para el objetivo del análisis.

Por eso, en aprendizaje no supervisado la evaluación debe combinar criterios técnicos y criterios de contexto. Podemos observar métricas, visualizaciones, estabilidad de resultados y coherencia interna. Pero también necesitamos interpretar si los grupos, dimensiones o anomalías ayudan a responder una pregunta real.

Un buen resultado no supervisado suele ser aquel que permite mirar los datos de una forma más clara. Puede ordenar casos, revelar diferencias, sugerir hipótesis o señalar situaciones que merecen revisión. No necesariamente produce una conclusión definitiva.

El aprendizaje no supervisado ayuda a explorar, pero no reemplaza la interpretación del problema.

Buscar estructura en los datos significa encontrar formas de organización que no estaban dadas explícitamente por una variable objetivo. Esa estructura puede aparecer como grupos, dimensiones principales, patrones habituales o casos inusuales. Pero los resultados dependen de las variables elegidas, del preprocesamiento, del algoritmo y de los parámetros. Por eso, el aprendizaje no supervisado debe entenderse como una herramienta para explorar y comprender mejor los datos, no como una fuente automática de verdades definitivas.

Agrupamiento

Una de las formas más conocidas de aprendizaje no supervisado es el agrupamiento, también llamado *clustering*. Su objetivo es formar grupos de casos parecidos entre sí, sin usar etiquetas previas.

En clasificación, las clases ya existen. El modelo aprende a reconocerlas a partir de ejemplos etiquetados. En agrupamiento, en cambio, no hay una columna que diga a qué grupo pertenece cada caso. El algoritmo analiza las variables disponibles y propone una organización posible.

Concepto	Descripción
Clasificación	las clases están dadas.
Agrupamiento	los grupos se construyen a partir de similitudes.

Supongamos que tenemos datos de clientes. Para cada cliente conocemos la frecuencia de compra, el gasto promedio, la antigüedad, la cantidad de categorías compradas y la fecha de la última compra. No tenemos una etiqueta que diga “cliente frecuente”, “cliente ocasional” o “cliente inactivo”. Sin embargo, es posible que existan patrones en esos datos.

Un algoritmo de agrupamiento podría encontrar casos parecidos y organizarlos en grupos. Tal vez un grupo reúna clientes que compren con frecuencia y gastan montos altos. Otro podría reunir clientes nuevos con pocas compras. Otro, clientes antiguos con baja actividad reciente.

El proceso parte de Clientes, continúa con variables de comportamiento, luego clientes parecidos entre sí y conduce finalmente a grupos sugeridos.

Estos grupos no son verdades previas. Son una propuesta del algoritmo según las variables que usamos y la forma en que medimos la similitud. Luego debemos analizarlos para decidir si tienen sentido.

El agrupamiento es útil cuando queremos explorar datos sin categorías conocidas. Puede ayudarnos a segmentar clientes, organizar documentos, encontrar perfiles de uso, resumir comportamientos o detectar conjuntos de casos con características similares.

Agrupar puede servir para segmentar, explorar, resumir y organizar los datos.

La palabra “similitud” es clave. Para agrupar, el algoritmo necesita alguna forma de decidir qué casos se parecen. En muchos métodos, esa similitud se relaciona con distancia. Dos casos están cerca si sus valores son parecidos en las variables consideradas. Dos casos están lejos si sus valores son muy diferentes.

Por ejemplo, si representamos clientes usando frecuencia de compra y gasto promedio, dos clientes con valores similares quedarán cerca. Si uno compra todos los meses y otro compró una sola vez, probablemente queden más lejos.

Concepto	Descripción
Casos cercanos	valores parecidos.
Casos lejanos	valores diferentes.

Esta idea parece simple, pero depende mucho de las variables elegidas. Si usamos variables de compra, los grupos reflejarán comportamiento comercial. Si usamos variables geográficas, los grupos reflejarán ubicación. Si usamos variables demográficas, reflejarán otro tipo de estructura.

Por eso, antes de agrupar, debemos preguntarnos qué tipo de parecido nos interesa. No existe una única forma correcta de parecerse. Dos clientes pueden parecerse por edad, por gasto, por productos elegidos, por zona, por frecuencia de uso o por muchas otras dimensiones.

Agrupar exige decidir qué significa que dos casos sean parecidos.

El algoritmo no responde esa pregunta por nosotros. Trabaja con las variables que le damos. Si esas variables no representan bien el objetivo exploratorio, los grupos encontrados pueden ser poco útiles.

Otro aspecto importante es la escala. En métodos basados en distancia, una variable con valores grandes puede dominar el cálculo de similitud. Si agrupamos usando ingresos mensuales en pesos y cantidad de compras, los ingresos pueden pesar

mucho más por su escala numérica, aunque no necesariamente sean más importantes.

En agrupamiento, la escala de las variables puede cambiar los grupos encontrados.

Por eso, en muchos casos es necesario escalar las variables numéricas antes de agrupar. Este cuidado será especialmente importante en K-Means, que estudiaremos en el próximo capítulo.

El resultado de un agrupamiento suele expresarse mediante números de grupo: cluster 0, cluster 1, cluster 2. Esos números no tienen significado propio. No indican jerarquía, calidad ni valor. Solo identifican grupos encontrados por el algoritmo.

Los nombres `Cluster 0`, `Cluster 1` y `Cluster 2` identifican grupos, pero no explican su significado.

Después de obtener los grupos, debemos analizarlos. Podemos calcular promedios por grupo, revisar distribuciones, observar ejemplos representativos y comparar características. Recién entonces podemos proponer nombres o interpretaciones.

Por ejemplo:

Concepto	Descripción
Cluster 0	clientes frecuentes y de alto gasto.
Cluster 1	clientes nuevos con pocas compras.
Cluster 2	clientes antiguos con baja actividad.

Estos nombres son interpretaciones humanas. El algoritmo no sabe qué significa “alto gasto” o “baja actividad” desde el punto de vista del negocio. Solo agrupó casos según sus valores.

También debemos tener cuidado con la cantidad de grupos. Algunos algoritmos requieren que indiquemos cuántos grupos queremos encontrar. Si pedimos dos grupos, obtendremos dos. Si pedimos cinco, obtendremos cinco. Eso no significa que la estructura real del problema tenga exactamente esa cantidad.

Pedir más grupos puede mostrar más detalle, mientras que pedir menos puede ofrecer un resumen mayor. Ninguna de las dos opciones es automáticamente correcta.

La elección del número de grupos debe evaluarse. Podemos usar criterios técnicos, visualizaciones y conocimiento del contexto. También debemos preguntarnos si los

grupos resultantes son interpretables y útiles. Un agrupamiento con muchos clusters puede ser difícil de comunicar. Uno con pocos puede ocultar diferencias importantes.

El agrupamiento también puede ser sensible a valores extremos. Un caso muy distinto al resto puede afectar el resultado o formar un grupo propio, según el algoritmo. Por eso conviene revisar outliers antes de interpretar clusters como segmentos significativos.

Un grupo pequeño o extraño puede representar un patrón real, un caso extremo, un error o una cantidad insuficiente de datos.

Esta es una de las razones por las que no debemos aceptar los clusters sin revisión. Agrupar no es lo mismo que descubrir categorías naturales definitivas. Es una herramienta para explorar una posible organización de los datos.

También debemos evitar sobreinterpretar. Si un algoritmo divide estudiantes, clientes o usuarios en grupos, esos grupos no deben convertirse automáticamente en etiquetas rígidas. Pueden ayudar a observar patrones, pero no deberían usarse sin cuidado para tomar decisiones sensibles.

Un cluster no define lo que una persona es; solo resume cómo aparece en ciertos datos.

Esto es especialmente importante cuando los agrupamientos pueden influir en oportunidades, accesos, recomendaciones o intervenciones. Las variables pueden estar incompletas, sesgadas o reflejar condiciones externas. Por eso, la interpretación debe ser responsable.

El agrupamiento puede ser muy útil como primer paso exploratorio. Puede revelar que los datos no son homogéneos, sugerir segmentos, ayudar a diseñar estrategias diferenciadas o mostrar casos que conviene estudiar por separado.

Por ejemplo, en una tienda, no todos los clientes se comportan igual. Algunos compran poco pero regularmente. Otros compran mucho en fechas específicas. Otros miran muchos productos pero compran poco. Agrupar puede ayudar a reconocer esas diferencias y pensar acciones distintas.

Agrupar ayuda a pasar de una masa de datos a un conjunto de perfiles exploratorios.

Pero, para que el resultado sea útil, necesitamos revisar los grupos con preguntas concretas:

Para interpretar los resultados conviene preguntarse: ¿Los grupos son interpretables? ¿Tienen tamaño razonable? ¿Son estables? ¿Ayudan a tomar decisiones? ¿Tienen sentido para el contexto?

Si la respuesta es no, quizá haya que cambiar variables, ajustar el preprocesamiento, probar otro número de grupos o incluso replantear la pregunta.

El agrupamiento busca formar grupos de casos parecidos sin usar etiquetas previas. Es una técnica no supervisada útil para explorar, segmentar y organizar datos. Pero los grupos encontrados dependen de las variables elegidas, de la escala, del algoritmo y de sus parámetros. Por eso, los clusters deben interpretarse con cuidado: son una propuesta de organización, no una verdad automática sobre los datos.

Reducción de dimensionalidad

Otra forma importante de aprendizaje no supervisado es la reducción de dimensionalidad. Su objetivo es representar datos con muchas variables usando una cantidad menor de dimensiones, intentando conservar la mayor parte posible de la información relevante.

La palabra “dimensionalidad” se refiere, en este contexto, a la cantidad de variables o columnas que usamos para describir cada caso. Si un dataset tiene dos variables, podemos imaginarlo como puntos en un plano. Si tiene tres, podemos imaginarlo como puntos en el espacio. Pero si tiene diez, cincuenta, cien o miles de variables, ya no podemos visualizarlo de manera directa.

El proceso parte de Muchas variables, continúa con datos difíciles de visualizar y conduce finalmente a representación más compacta.

La reducción de dimensionalidad busca construir una versión más simple del dataset. En lugar de trabajar con todas las variables originales, intentamos crear nuevas variables que resuman parte importante de la información.

Por ejemplo, imaginemos que tenemos datos de estudiantes con muchas variables relacionadas con actividad en una plataforma: cantidad de ingresos, tiempo conectado, actividades iniciadas, actividades completadas, cantidad de consultas, participación en foros, entregas realizadas y días desde el último acceso. Es posible que varias de esas variables estén relacionadas entre sí.

Muchas variables de actividad pueden expresar patrones parecidos.

Un método de reducción de dimensionalidad podría resumir parte de esa información en pocas dimensiones nuevas. Una de ellas podría capturar una idea general de actividad. Otra podría reflejar regularidad o constancia. Esas nuevas dimensiones no son necesariamente variables originales, sino combinaciones o transformaciones construidas a partir de ellas.

Esto puede servir para explorar mejor los datos. Si logramos representar muchos datos en dos dimensiones, podemos graficarlos y observar si aparecen grupos, tendencias o casos inusuales.

El proceso parte de Datos con muchas columnas, continúa con reducción a dos dimensiones y conduce finalmente a visualización en un plano.

La reducción de dimensionalidad no siempre se usa para visualizar. También puede servir para simplificar un dataset antes de aplicar otro modelo, reducir ruido, disminuir redundancia entre variables o hacer que ciertos análisis sean más manejables.

Por ejemplo, en un conjunto de datos con muchas variables muy correlacionadas, algunas columnas pueden estar contando una historia parecida. Si dos o más variables se mueven juntas, quizá podamos resumir parte de esa información en una dimensión común.

El proceso parte de Variables correlacionadas, continúa con información redundante y conduce finalmente a posible resumen en menos dimensiones.

Una técnica clásica de reducción de dimensionalidad es PCA, sigla de *Principal Component Analysis*, o análisis de componentes principales. PCA busca construir nuevas variables, llamadas componentes principales, que capturan la mayor cantidad posible de variación en los datos.

No vamos a estudiar PCA en detalle todavía, porque tendrá su propio capítulo. Por ahora, nos alcanza con entender su propósito general: transformar un conjunto de variables originales en un conjunto menor de componentes que resumen parte importante de la información.

El proceso parte de Variables originales, continúa con PCA y conduce finalmente a componentes principales.

Es importante notar que las nuevas dimensiones no siempre son fáciles de interpretar. Una componente principal puede combinar varias variables originales con distintos pesos. Por eso, aunque reduzca la complejidad del dataset, también puede volver menos directa la explicación de cada dimensión.

Por ejemplo, si una componente combina frecuencia de compra, gasto promedio y cantidad de productos distintos, quizá podamos interpretarla como una dimensión de “actividad comercial”. Pero esa interpretación no siempre será evidente ni completamente precisa.

Reducir dimensiones puede facilitar el análisis, pero también puede dificultar la interpretación directa.

Esta es una diferencia importante respecto de seleccionar columnas. Si simplemente elegimos dos variables originales, como `frecuencia_compra` y `gasto_promedio`, sabemos exactamente qué significa cada eje. En cambio, si usamos dos componentes principales, esos ejes son construcciones matemáticas derivadas de varias variables.

Concepto	Descripción
Seleccionar variables	conservamos columnas originales.
Reducir dimensionalidad	creamos nuevas dimensiones resumidas.

La reducción de dimensionalidad también depende mucho del escalado. Si una variable tiene valores muy grandes y otra valores pequeños, una técnica como PCA puede quedar dominada por la variable de mayor escala. Por eso, antes de aplicar ciertos métodos, suele ser necesario estandarizar las variables numéricas.

Las variables expresadas en escalas diferentes pueden afectar las dimensiones encontradas.

Esto conecta con una idea que ya vimos varias veces: el preprocesamiento no es un detalle menor. En técnicas basadas en distancias, geometría, varianza o relaciones entre variables, la escala puede cambiar de manera importante el resultado.

Un flujo típico podría incluir escalado y luego reducción de dimensionalidad:

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

pipeline = Pipeline(steps=[
    ("scaler", StandardScaler()),
    ("pca", PCA(n_components=2))
])
```

En este ejemplo, el pipeline primero estandariza las variables y luego reduce los datos a dos componentes principales. Si el objetivo es visualizar, dos dimensiones pueden ser convenientes porque permiten construir un gráfico en el plano.

El proceso parte de Escalado, continúa con reducción de dimensionalidad y conduce finalmente a visualización o análisis posterior.

Cuando usamos reducción de dimensionalidad dentro de un flujo de Machine Learning, también debemos evitar fuga de datos. Si el método aprende una transformación a partir de los datos, esa transformación debe ajustarse solo con el conjunto de entrenamiento. Luego se aplica al conjunto de prueba.

Esto es especialmente importante si la reducción se usa antes de un modelo supervisado. Aunque PCA sea una técnica no supervisada, aprende a partir de la estructura de X . Si ajustamos PCA usando todos los datos antes de separar, estaríamos usando información del test durante el entrenamiento del flujo.

Una transformación no supervisada también puede producir fuga de datos si se ajusta utilizando todo el dataset.

Por eso, si combinamos reducción de dimensionalidad con un clasificador o regresor, conviene incluir todo dentro de un pipeline.

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.linear_model import LogisticRegression

pipeline = Pipeline(steps=[
    ("scaler", StandardScaler()),
    ("pca", PCA(n_components=5)),
    ("modelo", LogisticRegression())
])
```

Este pipeline asegura que el escalado, la reducción de dimensionalidad y el modelo se ajusten siguiendo el flujo correcto durante la evaluación.

La reducción de dimensionalidad también puede ayudar cuando queremos observar datos complejos. Si tenemos muchas variables, quizá no sepamos si hay grupos, transiciones o casos extraños. Una representación en dos dimensiones puede mostrar patrones iniciales, aunque no debemos tomar esa visualización como prueba definitiva.

Una visualización reducida puede sugerir patrones, pero no demuestra por sí sola que esos patrones sean definitivos.

Esto es importante porque al reducir dimensiones siempre se pierde algo de información. Pasar de muchas variables a dos componentes implica resumir. Ese resumen puede ser útil, pero no contiene todos los detalles del dataset original.

Por eso, una gráfica en dos dimensiones debe interpretarse como una ayuda exploratoria. Si dos grupos parecen separados, puede ser una pista. Si se superponen, también puede ser informativo. Pero la interpretación debe complementarse con análisis adicional.

Reducir dimensiones es resumir, y todo resumen deja parte de la información afuera.

También debemos diferenciar reducción de dimensionalidad de eliminación de variables inútiles. A veces reducimos dimensiones para resumir variables relacionadas. Otras veces eliminamos columnas porque son irrelevantes, redundantes, tienen demasiados faltantes o no estarían disponibles en el momento de predicción. Son decisiones distintas.

Concepto	Descripción
Eliminar variables	quitar columnas.
Reducir dimensionalidad	transformar variables en una representación más compacta.

En aprendizaje no supervisado, la reducción de dimensionalidad puede usarse como herramienta de exploración. En aprendizaje supervisado, puede formar parte del preprocesamiento antes de entrenar un modelo. En ambos casos, necesitamos comprender qué transformación estamos aplicando y cómo afecta la interpretación.

Una ventaja es que puede hacer más manejables datasets complejos. Una limitación es que las nuevas dimensiones pueden ser menos intuitivas que las variables originales. Además, el resultado depende de la técnica usada, del escalado, de los datos y de la cantidad de dimensiones elegida.

La reducción de dimensionalidad ayuda a simplificar, pero no elimina la necesidad de interpretar los resultados con cuidado.

La reducción de dimensionalidad busca representar datos con muchas variables usando menos dimensiones. Puede ayudar a visualizar, resumir información, reducir redundancia o preparar datos para otros modelos. Técnicas como PCA construyen nuevas dimensiones que capturan parte importante de la variación de los datos. Sin embargo, reducir dimensiones implica perder información y, muchas veces, sacrificar interpretabilidad directa. Por eso, debe usarse como una herramienta de análisis y simplificación, no como una garantía automática de comprensión.

Detección de anomalía

Otra tarea frecuente dentro del aprendizaje no supervisado es la detección de anomalías. En este caso, el objetivo no es formar grupos ni reducir la cantidad de variables, sino identificar casos que se comportan de manera muy distinta al patrón general de los datos.

Una anomalía es una observación inusual. Puede ser un caso raro, un error de carga, una situación especial, una falla, un posible fraude o simplemente un ejemplo poco frecuente. El algoritmo no sabe cuál de esas explicaciones es correcta. Solo puede señalar que cierto caso se aparta de lo habitual según las variables disponibles.

El proceso parte de Datos habituales, continúa con patrón general, luego casos que se alejan mucho y conduce finalmente a posibles anomalías.

Esto es importante: una anomalía no es necesariamente algo malo. En algunos contextos puede indicar un problema, pero en otros puede ser un caso interesante, una oportunidad o una variación legítima.

Por ejemplo, en una tabla de ventas, una compra muy grande puede ser una anomalía. Pero esa compra podría ser un error, una operación fraudulenta, una venta corporativa legítima o una promoción especial. El modelo puede llamar la atención sobre el caso, pero no puede explicar por sí solo qué ocurrió.

Que un caso sea anómalo no significa automáticamente que sea incorrecto, fraudulento o peligroso.

La detección de anomalías es útil cuando no tenemos suficientes ejemplos etiquetados de eventos raros. En un problema supervisado de fraude, necesitaríamos transacciones marcadas como fraudulentas y no fraudulentas. Pero muchas veces esas etiquetas no existen, son incompletas o llegan tarde. En ese caso, podemos intentar detectar transacciones que se alejan del comportamiento normal.

El proceso parte de Sin etiquetas de fraude, continúa con buscar comportamiento habitual, luego detectar transacciones inusuales y conduce finalmente a revisión posterior.

Lo mismo puede ocurrir con sensores industriales. Una máquina puede producir miles de mediciones normales y muy pocas mediciones asociadas a fallas. Si no tenemos etiquetas confiables sobre fallas, podemos buscar registros que se apartan de la operación habitual.

También puede usarse en seguridad informática. Un acceso desde un lugar inesperado, en un horario poco común o con un patrón de actividad distinto al

habitual puede ser una anomalía. Eso no prueba que exista un ataque, pero puede justificar una revisión.

Una anomalía puede ser una señal de alerta, pero no una conclusión definitiva.

En educación, podríamos detectar patrones de uso muy distintos en una plataforma. Un estudiante que nunca ingresa, otro que entrega todo en pocos minutos o uno que accede desde muchos dispositivos pueden aparecer como casos inusuales. Pero cada uno requiere interpretación: puede haber problemas técnicos, hábitos personales, dificultades de acceso o situaciones externas.

Esto muestra que la detección de anomalías debe usarse con prudencia, especialmente cuando puede afectar a personas. Un resultado algorítmico no debería convertirse automáticamente en una acusación o en una decisión sensible.

Detectar un caso inusual no autoriza a sacar conclusiones sin una revisión humana.

La idea general de muchas técnicas de anomalías es definir qué se considera “normal” y luego medir qué tan lejos está cada caso de ese patrón. Algunas técnicas se apoyan en distancias. Otras en densidad, aislamiento, reconstrucción o modelos estadísticos. No necesitamos estudiar todas ahora. Lo importante es comprender el objetivo.

Concepto	Descripción
Primero	describir lo habitual.
Después	buscar casos que se alejan de ese comportamiento.

Un caso puede considerarse anómalo porque está lejos de la mayoría de los datos. También puede ser anómalo porque aparece en una zona con pocos vecinos. O porque su combinación de valores es rara aunque cada valor por separado no parezca extremo.

Por ejemplo, una compra de \$200.000 puede ser normal para ciertos clientes y anómala para otros. Una compra nocturna puede ser normal en algunos servicios y extraña en otros. Un gasto alto no siempre es raro; depende del contexto.

La rareza de un caso depende del conjunto de datos y del contexto del problema.

Esto es una diferencia importante respecto de buscar valores extremos columna por columna. Una anomalía no siempre es un valor máximo o mínimo en una variable. A veces surge de una combinación poco común.

Supongamos una tabla de viajes. Un viaje de 5 kilómetros no es raro. Un viaje de 10 minutos tampoco. Pero un viaje de 5 kilómetros con un costo extremadamente alto puede ser inusual. La anomalía aparece en la relación entre variables.

Dos valores que por separado son normales pueden formar una combinación inusual.

Por eso, la detección de anomalías puede ser más rica que una simple búsqueda de outliers en una columna. Sin embargo, también puede ser más difícil de interpretar.

Otro punto importante es que las anomalías suelen ser pocas. Si un algoritmo marca demasiados casos como anómalos, puede perder utilidad. Si marca muy pocos, puede pasar por alto situaciones relevantes. Algunos métodos permiten ajustar esa proporción esperada de anomalías mediante parámetros.

Concepto	Descripción
Demasiadas alertas	difíciles de revisar.
Muy pocas alertas	riesgo de no detectar casos importantes.

En la práctica, la detección de anomalías suele producir una lista de casos para revisar. No reemplaza el análisis posterior. Puede servir para priorizar atención, reducir el volumen de datos a inspeccionar o encontrar ejemplos que merecen una mirada más detallada.

Por ejemplo, en una mesa de soporte con miles de registros, un sistema puede marcar los casos más raros para que una persona los revise. En una base de transacciones, puede priorizar operaciones inusuales. En sensores, puede señalar intervalos de tiempo que conviene investigar.

El modelo puede ayudar a ordenar prioridades, pero la revisión sigue siendo necesaria.

La calidad de las variables es fundamental. Si las variables no describen bien el comportamiento normal, el modelo no podrá detectar anomalías útiles. Si usamos variables irrelevantes, mal escaladas o con errores, los casos marcados pueden no tener sentido.

También debemos prestar atención al preprocesamiento. En métodos basados en distancia, la escala importa mucho. Una variable con valores grandes puede dominar la definición de rareza. Además, los valores faltantes, duplicados o errores de carga pueden aparecer como anomalías, aunque no representen fenómenos reales.

Algunas anomalías detectadas pueden ser problemas de calidad de los datos y no eventos reales.

Esto no es necesariamente negativo. Detectar errores de carga también puede ser útil. Pero debemos distinguir entre una anomalía estadística y una conclusión sobre el mundo.

Una anomalía estadística significa que el caso es raro en los datos. La interpretación depende del contexto. Puede ser un fraude, una falla, un cliente especial, una oportunidad comercial, un error de registro o una situación legítima poco frecuente.

El algoritmo detecta rareza, pero la explicación de esa rareza requiere contexto.

En `scikit-learn` existen modelos para detección de anomalías, como `IsolationForest`, `LocalOutlierFactor` y `OneClassSVM`. No necesitamos profundizar en ellos ahora, pero es útil reconocer que la detección de anomalías puede abordarse con distintas técnicas.

```
from sklearn.ensemble import IsolationForest

modelo = IsolationForest(
    contamination=0.05,
    random_state=42
)
```

En este ejemplo, `contamination=0.05` indica que esperamos que aproximadamente un 5% de los casos sean anómalos. Este valor debe elegirse con cuidado. No siempre sabemos de antemano cuántos casos raros deberían existir.

El parámetro `contamination` no descubre mágicamente la proporción real de anomalías, sino que representa una decisión de modelado.

Cuando tenemos etiquetas reales de anomalías, aunque sean pocas, podemos evaluar mejor. Pero si no las tenemos, la evaluación es más exploratoria. Debemos revisar ejemplos, contrastar con conocimiento del dominio y analizar si los casos señalados son útiles para el objetivo.

También debemos considerar el costo de los errores. Un falso positivo ocurre cuando el sistema marca como anómalo un caso que en realidad no requiere atención. Un falso negativo ocurre cuando no marca un caso que sí era importante. Aunque no tengamos etiquetas completas, esos costos existen.

Concepto	Descripción
Marcar demasiado	genera trabajo innecesario.
Marcar poco	puede dejar pasar casos relevantes.

En problemas sensibles, como fraude, seguridad, salud o educación, estos costos deben analizarse con responsabilidad. La detección automática debe apoyar la revisión humana, no reemplazarla sin controles.

La detección de anomalías busca identificar casos que se apartan del comportamiento habitual de los datos. Es útil cuando no tenemos etiquetas confiables o cuando los eventos importantes son raros. Puede aplicarse a transacciones, sensores, accesos, ventas, plataformas educativas y muchos otros contextos. Sin embargo, una anomalía no es una explicación: es una señal para investigar. El significado de un caso inusual depende de las variables, el preprocesamiento, el algoritmo, los parámetros y el conocimiento del contexto.

Cuidado al interpretar resultados no supervisados

El aprendizaje no supervisado puede ser muy útil para explorar datos, encontrar grupos, reducir dimensiones o detectar anomalías. Sin embargo, sus resultados requieren una interpretación especialmente cuidadosa. Al no existir una variable objetivo conocida, no tenemos una respuesta correcta directa contra la cual comparar lo que el algoritmo produce.

En aprendizaje supervisado, si un modelo clasifica un correo como spam, podemos comparar esa predicción con una etiqueta real. Si predice un precio, podemos comparar con el precio observado. En aprendizaje no supervisado, en cambio, el modelo puede sugerir grupos, dimensiones o anomalías, pero no hay una etiqueta previa que indique si esas estructuras son “verdaderas”.

Concepto	Descripción
Aprendizaje supervisado	predicción vs. respuesta conocida.
Aprendizaje no supervisado	estructura sugerida sin respuesta correcta directa.

Esto no significa que los resultados no sirvan. Significa que deben entenderse como propuestas de organización, señales exploratorias o resúmenes posibles de los datos. El algoritmo puede ayudarnos a mirar mejor, pero no reemplaza el análisis del contexto.

Por ejemplo, si un algoritmo agrupa clientes en tres clusters, esos grupos no tienen significado automático. El cluster 0 no es necesariamente “mejor” que el cluster 1. Tampoco es una categoría natural que exista por sí misma. Es un conjunto de casos que el algoritmo consideró parecidos según las variables utilizadas.

Cluster 0 Cluster 1 Cluster 2

Son etiquetas técnicas, no explicaciones.

Para interpretar esos grupos, debemos revisar sus características. Podemos calcular promedios, medianas, distribuciones, tamaños, variables destacadas y ejemplos representativos. Recién después de ese análisis podemos proponer nombres o explicaciones.

Tal vez el cluster 0 reúna clientes frecuentes y de alto gasto. Tal vez el cluster 1 reúna clientes nuevos con poca actividad. Tal vez el cluster 2 reúna clientes antiguos que dejaron de comprar. Pero esas interpretaciones no vienen garantizadas por el algoritmo.

El algoritmo agrupa los datos, mientras que las personas interpretan qué significa ese agrupamiento.

Lo mismo ocurre con la reducción de dimensionalidad. Si usamos PCA y vemos una separación visual entre dos zonas del gráfico, esa separación puede ser una pista interesante. Pero no prueba automáticamente que existan dos grupos reales, ni que esa división tenga una explicación clara.

Una visualización reducida es un resumen. Puede ayudar a ver patrones, pero también puede ocultar información. Al pasar de muchas variables a dos dimensiones, siempre se pierde parte de la estructura original.

Una visualización en dos dimensiones puede ser útil, pero no contiene toda la información original.

Por eso, no conviene sacar conclusiones fuertes solo a partir de un gráfico reducido. Debe combinarse con análisis adicional: revisión de variables originales, comparación entre grupos, estabilidad de los resultados y conocimiento del dominio.

En detección de anomalías, el cuidado debe ser todavía mayor. Un caso marcado como anómalo no es necesariamente un fraude, una falla o un error. Solo significa que el caso se aparta del patrón habitual según el algoritmo y las variables utilizadas.

Anomalía: caso inusual en los datos.

No equivale automáticamente a problema o fraude.

Una transacción rara puede ser legítima. Un acceso inusual puede tener una explicación razonable. Un estudiante con actividad muy distinta puede atravesar una situación particular. Una medición extrema puede deberse a un sensor defectuoso, pero también a un evento real importante.

Por eso, los resultados no supervisados suelen ser puntos de partida para investigar, no conclusiones finales.

El proceso parte de Resultado no supervisado, continúa con preguntas nuevas, luego revisión y conduce finalmente a interpretación contextual.

Otro aspecto importante es que los resultados dependen de las variables elegidas. Si agrupamos clientes por gasto y frecuencia de compra, encontraremos una estructura. Si los agrupamos por edad y zona geográfica, encontraremos otra. Si agregamos o quitamos variables, los grupos pueden cambiar.

Esto no significa que el algoritmo esté fallando. Significa que no existe una única estructura inevitable en los datos. La estructura encontrada depende de la mirada que construimos mediante la selección de variables.

Si cambian las variables, también pueden cambiar los resultados.

También influye el preprocesamiento. Valores faltantes, escalado, codificación de categorías, tratamiento de outliers y selección de columnas pueden cambiar la estructura que el modelo encuentra. En métodos basados en distancia, el escalado puede ser decisivo.

Por ejemplo, si una variable está medida en pesos y otra en cantidad de compras, la variable en pesos puede tener valores mucho más grandes. Si no escalamos, el algoritmo puede agrupar principalmente por esa variable, aunque no sea la más importante para la pregunta que queremos explorar.

El preprocesamiento no solo prepara los datos, sino que también define qué patrones podrá encontrar el modelo.

Los parámetros del algoritmo también importan. En K-Means, por ejemplo, debemos elegir la cantidad de clusters. Si pedimos tres grupos, el algoritmo devolverá tres. Si pedimos cinco, devolverá cinco. El hecho de que un resultado exista no significa que esa sea la división más adecuada.

En detección de anomalías, si configuramos una proporción esperada de casos anómalos, el modelo tenderá a marcar una cantidad acorde. Esa elección puede cambiar mucho la lista de casos señalados.

Los parámetros también pueden cambiar la estructura encontrada.

Por eso, en aprendizaje no supervisado conviene probar, comparar y revisar. No se trata de ejecutar un algoritmo una vez y aceptar su salida como definitiva. Debemos

observar si el resultado es estable, si tiene sentido, si ayuda a comprender el problema y si es útil para tomar decisiones.

Un resultado puede ser técnicamente ordenado pero poco útil. Por ejemplo, un agrupamiento puede separar clientes por edad, pero quizá la pregunta del negocio era entender hábitos de compra. Una reducción de dimensionalidad puede producir un gráfico visualmente atractivo, pero no revelar patrones relevantes. Una detección de anomalías puede marcar muchos casos que nadie puede revisar.

Un resultado puede ser matemáticamente válido y, aun así, poco útil para el objetivo real.

La utilidad debe evaluarse en relación con la pregunta. Si la pregunta era explorar segmentos de comportamiento, los grupos deberían describir comportamientos. Si la pregunta era detectar casos para revisión, las anomalías deberían ser revisables y relevantes. Si la pregunta era visualizar datos complejos, la reducción debería ayudar a observar patrones que luego puedan analizarse.

También debemos tener cuidado con los sesgos. Los datos no supervisados no son neutrales. Pueden reflejar desigualdades, errores, ausencias o decisiones previas. Si usamos esos datos para agrupar personas, priorizar casos o detectar comportamientos inusuales, podemos reproducir o amplificar sesgos existentes.

No tener etiquetas no significa no tener sesgos.

Esto es especialmente importante cuando los resultados pueden afectar a personas. Un cluster no debe transformarse automáticamente en una etiqueta rígida. Una anomalía no debe convertirse automáticamente en sospecha. Una visualización no debe usarse para justificar decisiones sin análisis adicional.

El aprendizaje no supervisado puede orientar decisiones, pero debe hacerlo con supervisión humana, documentación y revisión crítica.

Los resultados no supervisados requieren contexto, criterio y responsabilidad.

Una buena práctica es documentar las decisiones tomadas: qué variables se usaron, cuáles se descartaron, cómo se trataron los valores faltantes, si se escaló, qué algoritmo se eligió, qué parámetros se probaron y cómo se interpretaron los resultados. Esa documentación permite revisar el análisis y evita presentar los resultados como si hubieran surgido de manera automática e indiscutible.

También conviene comunicar las limitaciones. En lugar de decir “descubrimos tres tipos reales de clientes”, sería más responsable decir “con estas variables y este

método, aparecen tres grupos de comportamiento que pueden analizarse como posibles segmentos”.

Es más apropiado comunicar que “el modelo sugiere grupos” que afirmar que “el modelo descubrió la verdad”.

Esta diferencia de lenguaje es importante. Ayuda a evitar la sobreinterpretación y deja claro que el resultado depende del método usado.

Los resultados no supervisados deben interpretarse con prudencia. Los clusters, las dimensiones reducidas y las anomalías son estructuras sugeridas por un algoritmo a partir de ciertos datos, variables, transformaciones y parámetros. Pueden ser muy útiles para explorar, resumir y orientar decisiones, pero no son conclusiones automáticas. Su significado debe construirse con análisis, contexto, revisión humana y comunicación responsable.

Aprender sin respuestas conocidas

En este capítulo abrimos la puerta al aprendizaje no supervisado. Hasta ahora habíamos trabajado principalmente con modelos que aprenden a partir de ejemplos con respuesta conocida. En clasificación, esa respuesta era una categoría. En regresión, un valor numérico. En ambos casos, el modelo tenía una variable objetivo y que guiaba el entrenamiento.

En aprendizaje no supervisado, esa variable objetivo no está disponible. Tenemos datos, pero no etiquetas. El modelo no aprende a predecir una respuesta conocida, sino que intenta encontrar estructura dentro de las variables disponibles.

Concepto	Descripción
Aprendizaje supervisado	$X + y$
Aprendizaje no supervisado	solo X

Este cambio modifica la naturaleza del problema. Ya no preguntamos “¿qué clase corresponde?” o “¿qué valor debería predecirse?”. Preguntamos si existen patrones, grupos, relaciones internas, dimensiones resumidas o casos que se apartan del comportamiento habitual.

Sin etiquetas, el objetivo no es predecir una respuesta, sino explorar la estructura de los datos.

Vimos que los datos no etiquetados son muy comunes. Muchas organizaciones tienen registros de clientes, transacciones, documentos, sensores, usuarios o actividades sin

que cada caso esté clasificado manualmente. Etiquetar datos puede ser costoso, lento o directamente imposible en ciertas situaciones. Por eso, los métodos no supervisados pueden ser una primera forma de explorar información disponible.

Una de las tareas más conocidas es el agrupamiento. En clustering, el algoritmo intenta formar grupos de casos parecidos entre sí. Esos grupos no existen previamente como etiquetas verdaderas. Son una propuesta de organización basada en las variables utilizadas.

El agrupamiento busca casos parecidos y forma grupos sugeridos.

También estudiamos la reducción de dimensionalidad. Esta tarea busca representar datos con muchas variables usando menos dimensiones. Puede servir para visualizar, resumir información, reducir redundancia o preparar datos para otros análisis. Técnicas como PCA permiten crear componentes que capturan parte importante de la variación del dataset.

La reducción de dimensionalidad resume muchas variables en una representación más compacta.

Otra tarea importante es la detección de anomalías. En lugar de formar grupos o resumir columnas, buscamos casos que se comportan de manera inusual respecto del patrón general. Una anomalía puede ser un error, una falla, un fraude, una oportunidad o simplemente un caso raro. El modelo señala rareza; la explicación requiere contexto.

La detección de anomalías busca casos que se alejan de lo habitual.

A lo largo del capítulo insistimos en una idea central: los resultados no supervisados deben interpretarse con cuidado. Un cluster no es una categoría natural garantizada. Una componente reducida no siempre tiene una interpretación directa. Una anomalía no equivale automáticamente a un problema. Son señales, resúmenes o propuestas de organización.

El algoritmo puede sugerir una estructura, pero su significado debe construirse mediante el análisis humano.

Esto hace que la evaluación sea más delicada que en aprendizaje supervisado. Cuando tenemos etiquetas, podemos comparar predicciones contra respuestas reales. Cuando no las tenemos, debemos usar otros criterios: coherencia, estabilidad, utilidad, visualizaciones, revisión de ejemplos, conocimiento del dominio y documentación de decisiones.

También vimos que las decisiones previas influyen mucho. Las variables elegidas, el tratamiento de valores faltantes, la codificación de categorías, el escalado, el algoritmo y sus parámetros pueden modificar la estructura encontrada. En aprendizaje no supervisado, el preprocesamiento no solo prepara los datos: también condiciona lo que el modelo podrá descubrir.

Si cambian las variables, el escalado o los parámetros, también pueden cambiar los resultados.

Por eso, no debemos presentar estos resultados como verdades automáticas. Es más responsable decir que, con ciertos datos, variables y métodos, aparecen determinados grupos, dimensiones o casos inusuales. Esa forma de comunicar deja claro que el resultado depende del análisis realizado.

Aprender sin etiquetas significa trabajar sin una respuesta conocida que guíe el entrenamiento. En lugar de predecir y , buscamos estructura dentro de X . El aprendizaje no supervisado puede ayudarnos a agrupar, resumir, visualizar y detectar casos inusuales, pero sus resultados requieren interpretación contextual y prudente. Es una herramienta poderosa para explorar datos, siempre que recordemos que encontrar una estructura no significa, por sí solo, haber encontrado una explicación definitiva.

Capítulo 26 · Clustering con K-Means

¿Qué significa agrupar datos?

Agrupar datos significa organizar casos parecidos entre sí en conjuntos o grupos. En Machine Learning, esta tarea se conoce como *clustering* y pertenece al aprendizaje no supervisado, porque no parte de etiquetas conocidas.

En clasificación, el modelo aprende a partir de ejemplos que ya tienen una clase asignada. Por ejemplo, correos etiquetados como spam o no spam, clientes etiquetados como abandona o no abandona, reseñas etiquetadas como positivas o negativas. En clustering, en cambio, esas etiquetas no existen.

Enfoque	Característica
Clasificación	Las clases ya están presentes en los datos.
Clustering	Los grupos se sugieren a partir de similitudes.

Esto cambia profundamente el objetivo. No le pedimos al modelo que aprenda a reconocer una clase previamente definida. Le pedimos que observe los datos y proponga una forma de organizarlos.

Supongamos que tenemos información sobre clientes de una tienda: cantidad de compras, gasto promedio, días desde la última compra y cantidad de categorías diferentes compradas. No sabemos qué tipo de cliente es cada uno. No tenemos una columna llamada `tipo_cliente`. Pero quizá existan patrones.

cliente	compras	gasto_promedio	dias_ultima_compra	categorias
---------	---------	----------------	--------------------	------------

Un algoritmo de clustering podría intentar formar grupos de clientes con comportamientos parecidos. Tal vez algunos compren seguido y gastan bastante. Otros compraron una sola vez. Otros compraban antes, pero hace mucho que no vuelven. El algoritmo no conoce esos nombres, pero puede detectar similitudes en los datos.

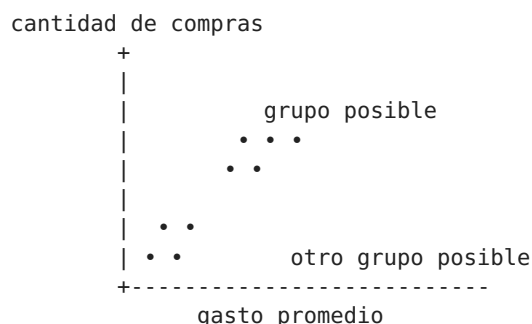
El proceso parte de los datos de clientes, busca similitudes entre los casos, propone grupos y requiere después una interpretación.

K-Means es uno de los algoritmos más conocidos para clustering. Su objetivo es dividir los datos en una cantidad determinada de grupos, llamados clusters. Cada cluster reúne puntos que, según las variables utilizadas, quedan relativamente cerca entre sí.

La idea central es sencilla: si representamos cada caso como un punto en un espacio de variables, agrupar significa juntar puntos cercanos.

Los puntos cercanos posiblemente pertenezcan al mismo grupo.

Por ejemplo, si usamos dos variables, podemos imaginar los datos en un plano. Cada cliente sería un punto. El eje horizontal podría representar la cantidad de compras y el eje vertical el gasto promedio. Si varios puntos están cerca, podrían formar un grupo.



En la práctica, los datasets suelen tener más de dos variables. Podemos agrupar usando tres, cinco, diez o más columnas. Aunque no podamos visualizar fácilmente todas esas dimensiones, la idea sigue siendo la misma: el algoritmo busca casos parecidos en el espacio definido por las variables.

Cada fila del dataset puede pensarse como un punto dentro de un espacio de variables.

Pero “parecido” no tiene un significado universal. Dos clientes pueden parecerse por su gasto, por su frecuencia de compra, por su antigüedad, por los productos que eligen o por su ubicación. El clustering depende de las variables que decidimos incluir.

Si usamos variables de comportamiento de compra, los grupos reflejarán comportamiento de compra. Si usamos edad y zona geográfica, los grupos reflejarán otra estructura. Si usamos variables irrelevantes, el algoritmo puede encontrar grupos poco útiles.

Agrupar exige decidir según qué variables consideraremos que dos casos son parecidos.

Esta pregunta es fundamental. K-Means no sabe cuál es el objetivo humano del análisis. Solo recibe datos numéricos y busca una organización interna. Por eso, antes de aplicar clustering, debemos pensar qué queremos explorar.

Por ejemplo, si queremos segmentar clientes para diseñar acciones comerciales, tendría sentido usar variables como frecuencia de compra, gasto promedio, recencia, variedad de productos o respuesta a promociones. Si queremos estudiar distribución geográfica, necesitaríamos otras variables. El algoritmo no corrige una mala selección de columnas.

El algoritmo agrupa según la información que recibe. Por eso, la elección de variables define qué estructura puede aparecer.

También debemos recordar que los grupos encontrados no vienen con significado propio. K-Means puede devolver etiquetas como 0, 1 y 2. Esas etiquetas solo identifican clusters. No significan “mejor”, “peor”, “alto”, “bajo” ni “importante”.

Los nombres `cluster 0`, `cluster 1` y `cluster 2` son identificadores técnicos, no interpretaciones.

Después de agrupar, debemos analizar cada cluster. Podemos mirar el promedio de cada variable, revisar casos representativos, observar tamaños de grupos y comparar perfiles. Recién entonces podemos proponer nombres interpretativos.

Por ejemplo:

Cluster	Interpretación posible
Cluster 0	Clientes frecuentes y de alto gasto.
Cluster 1	Clientes ocasionales.
Cluster 2	Clientes antiguos con baja actividad reciente.

Estos nombres no los descubre K-Means. Los construimos nosotros al analizar los resultados. Por eso, clustering no es solo ejecutar un algoritmo. Es un proceso de exploración, interpretación y validación contextual.

Otra cuestión importante es que K-Means necesita que indiquemos cuántos grupos queremos formar. Ese número suele representarse con la letra `k`. Si pedimos tres clusters, el algoritmo intentará formar tres. Si pedimos cinco, intentará formar cinco.

El valor de `k` indica la cantidad de clusters que le pedimos al algoritmo.

Esto no significa que los datos “tengan” exactamente esa cantidad de grupos. La elección de `k` es una decisión de modelado. En próximas secciones veremos cómo pensar esta decisión y qué ayuda puede ofrecer el método del codo.

Agrupar datos con K-Means, entonces, no es descubrir una verdad única escondida en el dataset. Es proponer una organización posible, basada en similitudes y en una cantidad de grupos elegida por quien analiza.

K-Means propone grupos, pero no garantiza que sean categorías naturales y definitivas.

También debemos cuidar el escalado. Como K-Means se apoya en distancias, las variables con valores numéricos más grandes pueden dominar la formación de clusters. Si una variable está medida en pesos y otra en cantidades pequeñas, la primera puede influir mucho más solo por su escala.

En K-Means, la escala de las variables puede cambiar los grupos encontrados.

Por eso, antes de aplicar K-Means, suele ser necesario preparar los datos: seleccionar variables relevantes, tratar faltantes, codificar categorías si corresponde y escalar variables numéricas. El algoritmo parece simple, pero sus resultados dependen mucho de esas decisiones previas.

Un flujo conceptual podría verse así:

El flujo parte de los datos originales, continúa con la selección de variables y el preprocesamiento, aplica K-Means, obtiene clusters y finaliza con su interpretación.

En `scikit-learn`, el modelo se encuentra como `KMeans`.

```
from sklearn.cluster import KMeans

modelo = KMeans(
    n_clusters=3,
    random_state=42
)
```

El parámetro `n_clusters` indica cuántos grupos queremos formar. En este ejemplo, estamos pidiendo tres clusters. El parámetro `random_state` ayuda a que el resultado sea reproducible, porque K-Means incluye una etapa inicial que puede depender de cierta aleatoriedad.

No es necesario entrar todavía en todos los detalles técnicos. Por ahora, lo importante es comprender qué significa agrupar: tomar datos sin etiquetas y buscar casos parecidos entre sí para formar grupos exploratorios.

Agrupar datos significa organizar casos similares cuando no tenemos etiquetas previas.

El clustering busca encontrar grupos dentro de datos no etiquetados. K-Means es un algoritmo clásico para esta tarea: divide los datos en k clusters según la cercanía entre puntos. Pero los grupos encontrados dependen de las variables elegidas, del escalado, del valor de k y del contexto del problema. Por eso, agrupar no es solo obtener etiquetas numéricas; es interpretar si esos grupos tienen sentido y si ayudan a comprender mejor los datos.

Centroides

Para entender K-Means, necesitamos incorporar una idea central: cada cluster se organiza alrededor de un punto llamado centroide. El centroide representa el centro del grupo según las variables utilizadas.

Si imaginamos los datos como puntos en un plano, cada cluster tendrá un punto central. Ese punto no necesariamente corresponde a un caso real del dataset. Puede ser un punto promedio, calculado a partir de las posiciones de los casos que pertenecen al grupo.

Cada cluster reúne un conjunto de puntos y posee un centroide, que representa el centro promedio del grupo.

Supongamos que agrupamos clientes usando dos variables: cantidad de compras y gasto promedio. Cada cliente puede representarse como un punto. Si un conjunto de clientes queda agrupado, el centroide estará ubicado en una posición promedio respecto de esos puntos.

• • • X • •

X = centroide.

El centroide resume la ubicación general del grupo. Si los clientes de un cluster tienen, en promedio, muchas compras y gasto alto, el centroide quedará en una zona asociada con ese comportamiento. Si otro grupo tiene pocas compras y bajo gasto, su centroide estará en otra zona.

K-Means usa los centroides para decidir a qué cluster pertenece cada punto. La regla básica es simple: cada caso se asigna al centroide más cercano.

Cada punto se asigna al centroide más cercano.

Por ejemplo, si tenemos tres centroides, cada cliente se compara con esos tres puntos centrales. El cliente queda asignado al cluster cuyo centroide esté más cerca de él.

De esta manera, los centroides funcionan como referencias que organizan el espacio de datos.

Para asignar un cliente nuevo o un punto del dataset, se calcula su distancia a cada centroide y se lo incorpora al cluster cuyo centroide esté más cerca.

El algoritmo K-Means no conoce de antemano dónde deberían estar esos centroides. Debe encontrarlos. Para hacerlo, sigue un proceso iterativo. Primero ubica centroides iniciales. Luego asigna cada punto al centroide más cercano. Después recalcula cada centroide usando los puntos asignados a ese cluster. Este proceso se repite varias veces hasta que los centroides se estabilizan o cambian muy poco.

1. Elegir centroides iniciales.
2. Asignar cada punto al centroide más cercano.
3. Recalcular los centroides.
4. Repetir el proceso hasta que se estabilice.

Esta idea iterativa es importante. K-Means no encuentra los clusters en un solo paso. Va ajustando los centroides a medida que reorganiza los puntos. En cada iteración, los grupos pueden cambiar, porque al moverse los centroides también cambia qué puntos quedan más cerca de cada uno.

Imaginemos una primera ubicación de centroides bastante imperfecta. Algunos puntos quedarán asignados a grupos poco adecuados. Pero luego, al recalcular los centroides, estos se desplazarán hacia el centro de los puntos que recibieron. En la siguiente iteración, algunos puntos pueden cambiar de grupo. El proceso continúa hasta alcanzar una organización relativamente estable.

Los centroides se desplazan hasta representar mejor a los puntos que tienen asignados.

Por eso, los centroides no son elegidos por una persona como etiquetas de grupo. Son calculados por el algoritmo. La persona que analiza decide cuántos clusters pedir, pero la posición final de los centroides surge del proceso de ajuste.

En `scikit-learn`, cuando entrenamos un modelo `KMeans`, podemos acceder a los centroides mediante el atributo `cluster_centers_`.

```
from sklearn.cluster import KMeans

modelo = KMeans(
    n_clusters=3,
    random_state=42
)

modelo.fit(X)
```

```
centroides = modelo.cluster_centers_
```

El resultado contiene las coordenadas de cada centroide en el espacio de variables. Si usamos dos variables, cada centroide tendrá dos valores. Si usamos cinco variables, cada centroide tendrá cinco valores. Cada valor indica la posición del centroide en una de las dimensiones utilizadas.

Cantidad de variables	Coordenadas de cada centroide
2	2
5	5

Esto nos ayuda a interpretar los clusters. Si las variables originales son comprensibles, podemos observar los centroides para entender el perfil promedio de cada grupo. Por ejemplo, un centroide puede indicar alta frecuencia de compra y alto gasto promedio. Otro puede indicar baja frecuencia y baja antigüedad.

Sin embargo, debemos tener cuidado. Si antes de aplicar K-Means escalamos las variables, los centroides estarán expresados en la escala transformada, no necesariamente en las unidades originales. Esto puede dificultar la interpretación directa.

Si entrenamos con datos escalados, los centroides quedan expresados en la escala transformada.

Por ejemplo, si usamos `StandardScaler`, las variables quedan expresadas en términos de desviaciones respecto de la media. Un centroide con valor positivo en una variable indica que ese cluster está por encima del promedio en esa dimensión. Un valor negativo indica que está por debajo. Pero ya no estamos viendo directamente pesos, cantidades o metros cuadrados.

Esto no es un problema, pero hay que tenerlo presente. Para comunicar resultados, muchas veces conviene analizar los clusters usando las variables originales. Podemos asignar cada fila a su cluster y luego calcular promedios reales por grupo en el dataset original.

Para interpretar los clusters, asignamos a cada fila su etiqueta de cluster y luego analizamos las variables originales de cada grupo.

De esta manera, podemos decir algo como: “el cluster 0 tiene un gasto promedio de \$18.500 y una frecuencia media de 6 compras mensuales”, en lugar de hablar solo de valores estandarizados.

Los centroides también ayudan a entender por qué K-Means tiende a formar grupos compactos y aproximadamente redondeados alrededor de centros. Como cada punto se asigna al centroide más cercano, el algoritmo funciona mejor cuando los clusters tienen una forma relativamente simple.

K-Means funciona mejor cuando los grupos pueden representarse alrededor de centros claros.

Si los datos tienen grupos alargados, curvos, con formas complejas o densidades muy distintas, K-Means puede tener dificultades. Esto lo veremos con más detalle al hablar de sus limitaciones.

Otra característica importante es que los centroides iniciales pueden influir en el resultado. Como el algoritmo empieza desde una posición inicial y luego ajusta, diferentes puntos de partida pueden llevar a soluciones distintas. Por eso, las implementaciones modernas suelen repetir el proceso varias veces con inicializaciones diferentes y quedarse con la mejor solución encontrada según su criterio interno.

El resultado de K-Means puede depender de la inicialización de los centroides.

En `scikit-learn`, el parámetro `n_init` controla cuántas veces se ejecuta el algoritmo con distintas inicializaciones. En muchas versiones actuales, su valor por defecto ya maneja esta cuestión de manera razonable, pero es útil saber que la inicialización existe y puede afectar el resultado.

```
modelo = KMeans(  
    n_clusters=3,  
    n_init="auto",  
    random_state=42  
)
```

El parámetro `random_state` ayuda a reproducir la misma inicialización cuando repetimos el experimento. Como en otros modelos, esto es útil para que los resultados puedan revisarse, compararse y documentarse.

Los centroides, entonces, son el corazón de K-Means. Cada cluster se define por un centro, y cada punto se asigna al centro más cercano. El algoritmo ajusta esos centros de manera iterativa hasta encontrar una organización estable.

K-Means combina centroides, asignación por cercanía y actualización iterativa.

Un centroide es el punto central de un cluster. K-Means empieza con centroides iniciales, asigna cada punto al centroide más cercano, recalcula los centroides a partir de los puntos asignados y repite el proceso hasta estabilizarse. Los centroides permiten resumir la posición de cada grupo, pero deben interpretarse con cuidado, especialmente si los datos fueron escalados. Comprenderlos es clave para entender cómo K-Means construye sus agrupamientos.

Distancia a los centros

K-Means agrupa los datos usando una idea muy concreta: cada punto se asigna al centroide más cercano. Por eso, la noción de distancia es fundamental. El algoritmo necesita medir qué tan lejos está cada caso de cada centroide para decidir a qué cluster pertenece.

Si trabajamos con dos variables, podemos imaginar los puntos en un plano. Cada punto representa una fila del dataset y cada centroide representa el centro de un cluster. La distancia entre un punto y un centroide indica qué tan parecido es ese caso al perfil central de ese grupo.

Para cada punto del dataset se calcula la distancia a todos los centroides y se lo asigna al más cercano.

Supongamos que agrupamos clientes usando dos variables: cantidad de compras y gasto promedio. Un cliente con muchas compras y gasto alto probablemente quede cerca del centroide de un grupo de clientes frecuentes. Un cliente con pocas compras y gasto bajo quedará más cerca de otro centroide.

La regla de K-Means es simple:

Cada punto pertenece al cluster cuyo centroide se encuentra más cerca.

Esta cercanía se calcula a partir de las variables utilizadas. Si usamos dos variables, la distancia se mide en ese plano. Si usamos cinco variables, la distancia se mide en un espacio de cinco dimensiones. Aunque no podamos visualizarlo fácilmente, la lógica es la misma.

Al incorporar más variables aumentan las dimensiones y las distancias se calculan dentro de ese espacio de variables.

En K-Means, la distancia más habitual es la distancia euclidiana. Es la idea de distancia “en línea recta” que usamos intuitivamente en geometría. En un plano, sería la distancia directa entre dos puntos.

No necesitamos memorizar la fórmula para comprender el modelo, pero sí entender su efecto: dos puntos son cercanos si sus valores son parecidos en las variables utilizadas. Si sus valores son muy diferentes, estarán más lejos.

Relación entre valores	Distancia esperada
Valores parecidos	Menor distancia
Valores diferentes	Mayor distancia

Por ejemplo, si dos clientes tienen una frecuencia de compra parecida y un gasto promedio parecido, quedarán cerca. Si uno compra muchas veces y otro casi nunca, o si uno gasta mucho más que el otro, la distancia aumentará.

Esta idea parece natural, pero tiene una consecuencia importante: la distancia depende totalmente de las variables que elegimos y de la escala en que están medidas.

Si usamos variables adecuadas, las distancias pueden reflejar parecidos útiles. Si usamos variables irrelevantes, las distancias pueden organizar los datos de una manera poco significativa.

La distancia no mide el parecido en términos generales, sino según las variables que decidimos incluir.

Esto es clave para interpretar K-Means. El algoritmo no sabe si dos clientes se parecen desde el punto de vista comercial, educativo, social o geográfico. Solo sabe calcular distancias usando las columnas que le dimos.

Si incluimos `frecuencia_compra`, `gasto_promedio` y `dias_ultima_compra`, el parecido estará relacionado con comportamiento de compra. Si incluimos `edad`, `zona` e `ingresos`, aparecerá otra noción de parecido. Si mezclamos variables sin criterio, el resultado puede ser difícil de interpretar.

Si cambian las variables, cambia la distancia y, como consecuencia, también pueden cambiar los clusters.

La escala de las variables también puede modificar completamente las distancias. Supongamos que usamos dos variables: ingresos mensuales en pesos y cantidad de compras. Los ingresos pueden tener valores como 250.000, 480.000 o 900.000. La cantidad de compras puede tener valores como 1, 5 o 12.

Si calculamos distancias sin escalar, la variable ingresos puede dominar el resultado simplemente porque sus números son mucho más grandes.

Variable	Valores de ejemplo
ingresos_mensuales	250000, 480000, 900000
cantidad_compras	1, 5, 12

Sin escalado, los ingresos pueden dominar el cálculo de la distancia.

Esto no significa que los ingresos sean necesariamente más importantes. Significa que, por su escala numérica, pesan mucho más en el cálculo. K-Means no distingue entre importancia conceptual y magnitud numérica. Para el algoritmo, una diferencia de 100.000 en una variable puede ser mucho más grande que una diferencia de 5 en otra, aunque desde el punto de vista del problema ambas puedan ser relevantes.

Por eso, en K-Means el escalado suele ser una etapa fundamental. Si queremos que las variables participen de manera más equilibrada, necesitamos llevarlas a escalas comparables.

K-Means utiliza distancias. Si las escalas son muy distintas, esas distancias pueden quedar distorsionadas.

Más adelante veremos el escalado como sección específica del capítulo, pero conviene anticipar la idea porque está directamente conectada con la distancia a los centros. No podemos entender bien K-Means sin entender que sus decisiones dependen de esas distancias.

En `scikit-learn`, cuando usamos `KMeans`, el modelo calcula internamente las distancias entre los puntos y los centroides durante el entrenamiento. Nosotros no tenemos que escribir ese cálculo a mano.

```
from sklearn.cluster import KMeans

modelo = KMeans(
    n_clusters=3,
    random_state=42
)

modelo.fit(X)
```

Después del entrenamiento, cada fila queda asignada a un cluster. Esas asignaciones se pueden consultar con `labels_`.

```
etiquetas_cluster = modelo.labels_
```

Cada valor en `etiquetas_cluster` indica cuál fue el centroide más cercano para esa fila después de que el algoritmo terminó de ajustar los clusters.

El atributo `labels_` contiene el cluster asignado a cada fila según el centroide más cercano.

También podemos usar `predict` para asignar nuevos puntos a los clusters ya aprendidos. En ese caso, el modelo no recalcula los centroides; simplemente toma los centroides existentes y asigna cada nuevo caso al más cercano.

```
nuevos_clusters = modelo.predict(nuevos_datos)
```

Esto puede ser útil si queremos clasificar nuevos clientes dentro de segmentos ya contruidos. Pero debemos recordar que esas asignaciones dependen de los centroides aprendidos y del mismo preprocesamiento usado durante el entrenamiento.

Un punto nuevo se compara con los centroides existentes y se asigna al cluster más cercano.

Si el modelo fue entrenado con datos escalados, los nuevos datos también deben pasar por el mismo escalado antes de usar `predict`. De lo contrario, las distancias no estarán en la misma escala y la asignación será incorrecta.

Por eso, nuevamente, conviene trabajar con pipelines cuando el flujo incluye transformaciones. El pipeline asegura que el mismo preprocesamiento se aplique durante el entrenamiento y durante la asignación de nuevos casos.

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans

pipeline = Pipeline(steps=[
    ("scaler", StandardScaler()),
    ("modelo", KMeans(n_clusters=3, random_state=42))
])
```

En este flujo, primero se escalan las variables y luego se aplica K-Means. Si después usamos `predict`, los nuevos datos pasarán por el mismo escalador antes de ser asignados al cluster más cercano.

La distancia a los centros también se relaciona con una medida interna del modelo llamada inercia. La inercia resume qué tan cerca están los puntos de sus centroides. Si los puntos quedan muy cerca de los centroides, la inercia será menor. Si quedan muy dispersos, será mayor.

Situación	Efecto sobre la inercia
Menor distancia a los centroides	Menor inercia

Situación	Efecto sobre la inercia
Mayor dispersión dentro de los clusters	Mayor inercia

La inercia será importante cuando estudiemos el método del codo, porque ayuda a comparar qué ocurre al cambiar la cantidad de clusters. Por ahora, nos alcanza con entender que K-Means intenta formar grupos compactos, donde los puntos estén lo más cerca posible de sus centros.

Sin embargo, compactar puntos alrededor de centros no siempre produce grupos con sentido. Un algoritmo puede reducir distancias internas y aun así formar clusters difíciles de interpretar. Por eso, la distancia es el criterio técnico del algoritmo, pero no debe ser el único criterio de análisis.

K-Means optimiza distancias, pero nosotros debemos evaluar si los clusters obtenidos tienen sentido.

K-Means asigna cada punto al centroide más cercano. Esa cercanía se mide mediante distancias calculadas con las variables disponibles. Por eso, las variables elegidas y sus escalas influyen directamente en los clusters obtenidos. La distancia a los centros permite que el algoritmo forme grupos compactos, pero no garantiza por sí sola que esos grupos sean útiles o interpretables. Como siempre en aprendizaje no supervisado, el resultado técnico debe revisarse con contexto y criterio.

Número de clusters

K-Means necesita que indiquemos de antemano cuántos clusters queremos formar. Ese número suele representarse con la letra **k**. Si elegimos **k = 3**, el algoritmo intentará organizar los datos en tres grupos. Si elegimos **k = 5**, intentará formar cinco.

El valor de **k** representa la cantidad de clusters que le pedimos a K-Means.

Esta decisión es una de las más importantes del algoritmo. K-Means no descubre por sí solo cuántos grupos “deberían” existir. Recibe un valor de **k** y ajusta los centroides para formar esa cantidad de clusters.

Por ejemplo, si agrupamos clientes y configuramos **k = 3**, el modelo devolverá tres grupos. Pero eso no significa que en la realidad existan exactamente tres tipos naturales de clientes. Significa que le pedimos al algoritmo que resuma los datos en tres grupos.

Si pedimos tres clusters, K-Means devolverá tres clusters. Esto no demuestra que los datos contengan exactamente tres grupos reales.

Esta diferencia es fundamental. En aprendizaje no supervisado, los resultados dependen de decisiones tomadas por quien analiza. Elegir el número de clusters es una de esas decisiones.

Si elegimos pocos clusters, el resultado será más general. Los grupos pueden ser grandes y fáciles de comunicar, pero quizá mezclen casos bastante diferentes entre sí. Si elegimos muchos clusters, el resultado puede capturar más detalle, pero también volverse difícil de interpretar.

Cantidad de clusters	Consecuencia
Pocos clusters	Más resumen y menos detalle
Muchos clusters	Más detalle y mayor complejidad

Supongamos que tenemos clientes de una tienda. Con $k = 2$, tal vez el algoritmo separe clientes de alta actividad y baja actividad. Esa división puede ser simple y útil, pero quizás demasiado amplia.

Con $k = 4$, podrían aparecer perfiles más específicos: clientes frecuentes de alto gasto, clientes frecuentes de bajo gasto, clientes ocasionales y clientes inactivos recientes. Esa segmentación puede ser más informativa, pero también exige más análisis.

Valor de k	Tipo de segmentación
2	Más simple
4	Más detallada

No existe una respuesta universal. El número adecuado de clusters depende de los datos, de las variables usadas y del objetivo del análisis.

Si el objetivo es comunicar una segmentación sencilla a un equipo de trabajo, quizá convenga usar pocos grupos. Si el objetivo es explorar diferencias finas entre comportamientos, tal vez tenga sentido probar más clusters. Si el objetivo es priorizar acciones, necesitamos una cantidad de grupos que sea útil y manejable.

Elegir k no es solamente una decisión técnica; también depende del uso que tendrán los clusters.

Esto no significa que podamos elegir cualquier valor sin criterio. Si elegimos un número demasiado bajo, los clusters pueden quedar muy mezclados. Si elegimos uno demasiado alto, podemos obtener grupos artificiales, muy pequeños o poco estables.

Por ejemplo, si tenemos 200 clientes y pedimos 50 clusters, probablemente aparezcan grupos con muy pocos casos. Tal vez eso no aporte una segmentación útil. En cambio, si pedimos solo 2 clusters para datos con perfiles muy diversos, podemos perder información importante.

Elección de k	Posible consecuencia
Demasiado bajo	Grupos demasiado generales
Demasiado alto	Grupos demasiado fragmentados

K-Means siempre intentará cumplir con el número de clusters solicitado. El algoritmo no se niega si el valor no tiene sentido. Por eso, la responsabilidad de elegir y justificar **k** queda en el análisis.

En `scikit-learn`, el número de clusters se indica con el parámetro `n_clusters`.

```
from sklearn.cluster import KMeans

modelo = KMeans(
    n_clusters=3,
    random_state=42
)
```

En este ejemplo, el modelo buscará tres clusters. Si cambiamos `n_clusters` a 4, buscará cuatro. Si lo cambiamos a 6, buscará seis.

```
modelo = KMeans(
    n_clusters=4,
    random_state=42
)
```

Cambiar `n_clusters` puede cambiar por completo la organización encontrada. No solo aparece un cluster nuevo; también pueden moverse los centroides y reasignarse muchos puntos.

Cambiar **k** puede modificar todos los clusters, no solamente agregar o quitar uno.

Por eso, cuando usamos K-Means, suele ser conveniente probar varios valores de **k** y comparar los resultados. No alcanza con ejecutar el algoritmo una sola vez y aceptar la primera salida.

Podemos observar cómo cambian los grupos, revisar sus tamaños, analizar sus perfiles y evaluar si la segmentación tiene sentido. También podemos usar algunas ayudas técnicas, como la inercia y el método del codo, que veremos en la próxima sección.

Para elegir k conviene revisar la compactación de los clusters, el tamaño de los grupos, su interpretabilidad y su utilidad para el problema.

La inercia mide, de manera simplificada, qué tan cerca quedan los puntos de sus centroides. Cuando aumentamos k , la inercia normalmente baja, porque con más centroides es más fácil que cada punto quede cerca de alguno.

Si usamos un cluster para todos los datos, muchos puntos quedarán lejos del único centroide. Si usamos muchos clusters, cada grupo será más pequeño y los puntos estarán más cerca de sus centros. Por eso, la inercia disminuye al aumentar k .

Al aumentar la cantidad de clusters, los puntos suelen quedar más cerca de sus centroides y la inercia disminuye.

Pero esto no significa que siempre convenga elegir más clusters. Si aumentamos mucho k , podemos reducir la inercia, pero a costa de crear grupos demasiado específicos o poco útiles. En el extremo, si cada punto fuera su propio cluster, la distancia interna sería mínima, pero el resultado no serviría como resumen.

Reducir la distancia interna no alcanza: los clusters también deben ser interpretables y útiles.

Esta es una idea importante. K-Means optimiza un criterio matemático, pero nuestro análisis necesita algo más que optimización. Necesita sentido.

Podemos imaginar la elección de k como un equilibrio entre simplicidad y detalle. Un buen número de clusters debería resumir los datos sin destruir sus diferencias importantes. Debería ser lo suficientemente pequeño para poder interpretarse y lo suficientemente grande para no mezclar patrones claramente distintos.

Un buen valor de k resume sin simplificar demasiado y diferencia sin fragmentar en exceso.

También debemos considerar el contexto. En una estrategia comercial, quizá cinco segmentos sean manejables, pero veinte no. En una exploración técnica, quizá probar diez clusters sea razonable. En una comunicación para un público amplio, tres o cuatro grupos pueden ser más claros.

El valor de k no debe decidirse solamente por una métrica. Las métricas ayudan, pero la interpretación es indispensable. Un resultado con buena compactación puede no tener sentido para el problema. Otro con clusters algo menos compactos puede ser más útil porque produce perfiles claros y accionables.

La elección de k combina criterio técnico, interpretación y el objetivo del análisis.

También conviene revisar el tamaño de los clusters. Si un valor de `k` produce un grupo enorme y varios grupos muy pequeños, debemos preguntarnos si eso refleja una estructura real o si el algoritmo está separando casos extremos. Los grupos pequeños pueden ser interesantes, pero no deben aceptarse sin análisis.

Los clusters muy pequeños pueden indicar perfiles poco frecuentes, outliers o una segmentación inestable.

Otra buena práctica es comparar resultados con distintos valores de `k`. Podemos entrenar modelos con `k = 2`, `k = 3`, `k = 4`, `k = 5` y observar qué cambia. A veces, la estructura general se mantiene y solo se subdivide un grupo. Otras veces, los resultados cambian mucho, lo que indica que la segmentación puede ser sensible.

```
for k in range(2, 7):
    modelo = KMeans(
        n_clusters=k,
        random_state=42
    )
    modelo.fit(X)
    print(k, modelo.inertia_)
```

Este código entrena varios modelos con distintos valores de `k` y muestra la inercia de cada uno. No decide automáticamente cuál es el mejor, pero nos da información para comparar.

Después de elegir un valor tentativo, conviene analizar los clusters en términos reales: qué características tienen, cuántos casos contienen, si son coherentes, si pueden describirse y si aportan algo al objetivo del trabajo.

La elección de `k` no termina al entrenar el modelo, sino cuando interpretamos si los grupos resultan útiles.

K-Means requiere elegir previamente el número de clusters. Ese valor, `k`, determina cuántos grupos intentará formar el algoritmo. Un `k` bajo produce una segmentación más simple; un `k` alto produce más detalle, pero también más complejidad y riesgo de fragmentación. La elección debe combinar criterios técnicos, como la inercia, con interpretación contextual, tamaño de los grupos y utilidad para el problema. K-Means siempre devolverá la cantidad de clusters solicitada; por eso, elegir `k` con cuidado es parte esencial del análisis.

Método del codo

Como K-Means necesita que indiquemos previamente el número de clusters, una pregunta aparece de inmediato: ¿cómo elegir un valor razonable para `k`?

No existe una respuesta única, pero hay herramientas que pueden ayudarnos. Una de las más conocidas es el método del codo. Este método no decide mágicamente cuál es el número correcto de grupos, pero permite observar cómo cambia la compactación de los clusters cuando probamos distintos valores de k .

La idea se apoya en la inercia. En K-Means, la inercia resume qué tan cerca quedan los puntos de sus centroides. Si los puntos están muy cerca de los centroides, la inercia es menor. Si están más dispersos, la inercia es mayor.

La inercia es la suma de las distancias internas entre los puntos y sus centroides.

Cuando aumentamos la cantidad de clusters, la inercia suele disminuir. Esto es esperable: si tenemos más centroides disponibles, cada punto puede quedar más cerca de alguno de ellos.

Por ejemplo, si usamos un solo cluster, todos los puntos deben organizarse alrededor de un único centroide. Si usamos dos, los datos pueden dividirse mejor. Si usamos tres, todavía más. A medida que agregamos clusters, la distancia interna tiende a bajar.

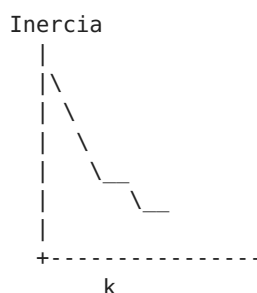
Más clusters implican más centroides; los puntos quedan más cerca de ellos y la inercia disminuye.

Pero esta mejora no continúa con la misma intensidad para siempre. Al principio, agregar clusters puede reducir mucho la inercia. Después de cierto punto, agregar más clusters puede producir mejoras cada vez más pequeñas.

El método del codo consiste en graficar la inercia para distintos valores de k y observar dónde la curva empieza a aplanarse.

El método consiste en probar varios valores de k , calcular la inercia, representar k frente a la inercia y buscar un cambio de pendiente.

Ese cambio de pendiente se conoce como “codo”, porque en el gráfico la curva puede parecerse a un brazo doblado. Antes del codo, aumentar k reduce bastante la inercia. Después del codo, agregar clusters mejora menos.



La interpretación es la siguiente: el código sugiere un punto de equilibrio entre simplicidad y compactación. Antes de ese punto, usar menos clusters puede ser demasiado pobre. Después de ese punto, usar más clusters puede agregar complejidad sin mejorar demasiado la estructura interna.

El código sugiere el punto a partir del cual agregar clusters aporta cada vez menos mejora.

Supongamos que probamos valores de `k` entre 1 y 8. La inercia baja mucho al pasar de 1 a 2 clusters, vuelve a bajar bastante al pasar de 2 a 3, pero después de 3 la mejora se vuelve más pequeña. En ese caso, `k = 3` podría ser un valor razonable para explorar.

No significa que 3 sea “la verdad” del dataset. Significa que, según la inercia, tres clusters ofrecen un equilibrio interesante entre resumir los datos y no fragmentarlos demasiado.

El método del código sugiere un valor razonable, pero no demuestra cuál es el número real de grupos.

En `scikit-learn`, podemos calcular la inercia de un modelo K-Means con el atributo `inertia_`. Para aplicar el método del código, entrenamos varios modelos con distintos valores de `k` y guardamos la inercia de cada uno.

```
from sklearn.cluster import KMeans

inercias = []

for k in range(1, 9):
    modelo = KMeans(
        n_clusters=k,
        random_state=42,
        n_init="auto"
    )
    modelo.fit(X)
    inercias.append(modelo.inertia_)
```

Luego podemos graficar esos valores.

```
import matplotlib.pyplot as plt

plt.plot(range(1, 9), inercias, marker="o")
plt.xlabel("Número de clusters (k)")
plt.ylabel("Inercia")
plt.title("Método del código")
plt.show()
```

El gráfico no nos entrega una respuesta automática. Debemos observar la forma de la curva. Si hay un cambio claro de pendiente, ese punto puede ser un buen candidato.

Si la curva baja de manera suave, sin un codo evidente, la decisión será más ambigua.

En algunos casos aparece un codo claro y en otros no.

Esta ambigüedad es normal. Los datos reales no siempre tienen grupos perfectamente separados. Puede haber transiciones graduales, solapamientos, ruido, outliers o estructuras que K-Means no representa bien.

Por eso, el método del codo debe usarse como una ayuda, no como una regla absoluta. Puede orientar la elección de k , pero no reemplaza la interpretación de los clusters.

El método del codo ayuda a elegir, pero no interpreta los grupos por nosotros.

Después de identificar un posible valor de k , conviene entrenar el modelo con ese número de clusters y analizar el resultado. Debemos revisar cuántos casos tiene cada cluster, qué valores promedio presentan las variables, si los grupos son coherentes y si tienen utilidad para el problema.

Por ejemplo, si el método del codo sugiere $k = 4$, no alcanza con aceptar cuatro clusters. Debemos preguntarnos qué representa cada grupo. Quizá los cuatro clusters sean interpretables y útiles. O quizá dos de ellos sean muy parecidos y no convenga separarlos. O quizá uno sea demasiado pequeño y esté capturando outliers.

Después de identificar el codo, todavía queda interpretar los clusters.

También puede ocurrir que el codo sugiera un valor técnicamente razonable, pero poco práctico. Supongamos que el gráfico sugiere siete clusters, pero el objetivo es comunicar una segmentación simple a un equipo comercial. Tal vez siete grupos sean demasiados para trabajar. En ese caso, podemos evaluar una solución con menos clusters, aunque la inercia sea un poco mayor.

La elección de k combina criterio técnico, claridad, utilidad y contexto.

El método del codo tampoco garantiza que K-Means sea el algoritmo adecuado. Si los datos tienen grupos con formas no redondeadas, tamaños muy distintos o densidades desiguales, la inercia puede no reflejar bien una estructura útil. En esos casos, aunque el gráfico muestre algún cambio de pendiente, los clusters pueden ser poco interpretables.

Además, la inercia siempre disminuye al aumentar k . Por eso, no debemos elegir el valor que tenga la menor inercia sin más. Si seguimos aumentando k , la inercia seguirá bajando, pero el resultado será cada vez más fragmentado.

La menor inercia no implica necesariamente el mejor agrupamiento.

En el extremo, si cada punto fuera su propio cluster, la distancia interna sería casi nula. Pero esa “solución” no nos serviría para resumir ni comprender los datos. El objetivo no es minimizar la inercia a cualquier costo, sino encontrar una segmentación útil.

También debemos recordar que el gráfico del codo depende del preprocesamiento. Si no escalamos variables que están en unidades muy distintas, la inercia estará dominada por algunas columnas. En ese caso, el codo puede reflejar más la escala de las variables que una estructura significativa.

Si las variables no están bien preparadas, el método del codo también puede orientar mal la decisión.

Por eso, antes de calcular el codo, conviene aplicar el mismo preprocesamiento que usaríamos para entrenar K-Means: selección de variables relevantes, tratamiento de faltantes y escalado cuando corresponda.

Un flujo razonable sería:

1. Elegir variables relevantes.
2. Preparar y escalar los datos.
3. Probar varios valores de k .
4. Graficar k frente a la inercia.
5. Elegir valores candidatos.
6. Interpretar los clusters resultantes.

El plural “valores candidatos” es importante. Muchas veces conviene comparar dos o tres opciones cercanas. Si el codo parece estar entre 3 y 4, podemos analizar ambos resultados. Quizá uno sea más claro o más útil que el otro.

El método del codo no siempre señala un único valor; también puede sugerir varias alternativas para comparar.

El método del codo ayuda a elegir un número razonable de clusters en K-Means. Consiste en entrenar modelos con distintos valores de k , calcular la inercia y graficar cómo disminuye al aumentar la cantidad de clusters. El “codo” de la curva sugiere un punto donde agregar más clusters empieza a aportar menos mejora. Sin embargo, no es una prueba definitiva. La elección final debe combinar la forma del gráfico con la interpretación de los clusters, el contexto del problema, la utilidad de la segmentación y la calidad del preprocesamiento.

Escalado de variables

K-Means depende de distancias. Esta idea apareció varias veces a lo largo del capítulo, pero merece una sección propia porque afecta directamente la calidad de los clusters encontrados. Si las variables están en escalas muy diferentes, el algoritmo puede agrupar los datos de una manera dominada por las columnas con valores numéricos más grandes.

Supongamos que queremos agrupar clientes usando dos variables:

Por ejemplo, podemos trabajar con las variables `ingresos_mensuales` y `cantidad_compras`.

La primera puede tener valores como 250.000, 500.000 o 900.000. La segunda puede tener valores como 1, 4, 8 o 15. Si calculamos distancias usando esas variables

Variable	Diferencia
900.000 - 500.000	= 400.000
Compras 15 - 8	= 7

Para K-Means, esos números forman parte del cálculo de distancia. El algoritmo no sabe que una variable está expresada en pesos y otra en cantidad de compras. Tampoco sabe cuál es más importante desde el punto de vista del análisis. Solo ve magnitudes numéricas.

Como consecuencia, los clusters pueden quedar organizados casi exclusivamente por ingresos, aunque nuestro objetivo fuera encontrar patrones de comportamiento de compra.

Sin escalado, una variable con valores grandes puede dominar la formación de los clusters.

Esto no significa que los ingresos sean irrelevantes. Significa que su escala puede darles un peso exagerado. Una variable no debería dominar el agrupamiento solo porque está medida en unidades más grandes.

El escalado busca evitar este problema. Consiste en transformar las variables numéricas para que estén en escalas comparables. De esa manera, la distancia refleja mejor las diferencias relativas entre los casos y no queda tan condicionada por las unidades originales.

Escalar variables significa llevar las columnas numéricas a una escala comparable antes de calcular distancias.

Una técnica habitual es la estandarización, que transforma cada variable para que tenga media cercana a 0 y desviación estándar cercana a 1. En `scikit-learn`, esto se hace con `StandardScaler`.

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
```

Cuando usamos K-Means, una forma recomendable de organizar el flujo es usar un pipeline que primero escale los datos y luego aplique el modelo.

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans

pipeline = Pipeline(steps=[
    ("scaler", StandardScaler()),
    ("modelo", KMeans(
        n_clusters=3,
        random_state=42,
        n_init="auto"
    ))
])
```

Este pipeline asegura que el escalado forme parte del proceso completo. Si después usamos el modelo para asignar nuevos casos a clusters, esos nuevos datos pasarán por el mismo escalador antes de calcular distancias.

Los datos nuevos deben recibir el mismo escalado aprendido; luego se calculan sus distancias a los centroides y se asigna el cluster correspondiente.

Esto es importante porque no alcanza con escalar una vez de manera informal. El mismo criterio de transformación debe aplicarse a todos los datos que se comparen con los centroides. Si entrenamos K-Means con datos escalados y luego intentamos predecir clusters usando datos sin escalar, las distancias estarán mal calculadas.

También debemos recordar que el escalador aprende información de los datos, como la media y la desviación estándar de cada variable. Si usamos K-Means solo como herramienta exploratoria sobre todo el dataset, esto no plantea el mismo problema que en una evaluación supervisada. Pero si K-Means forma parte de un flujo que luego se evalúa con datos separados, el escalado debe ajustarse solo con entrenamiento para evitar fuga de datos.

Si existe una separación entre entrenamiento y prueba, el escalado debe aprenderse únicamente con los datos de entrenamiento.

Esta idea ya apareció en capítulos anteriores. Aunque K-Means sea no supervisado, las transformaciones que aprenden estructura de los datos también deben manejarse con cuidado cuando forman parte de un proceso evaluado.

El escalado también afecta el método del codo. La inercia se calcula a partir de distancias entre puntos y centroides. Si las variables no están escaladas, la curva de inercia puede reflejar principalmente las columnas con valores grandes. En ese caso, el codo puede sugerir una estructura dominada por esas variables.

Sin un escalado adecuado, el método del codo puede orientar mal la elección de `k`.

Por eso, si vamos a usar el método del codo, conviene calcularlo con los datos ya preprocesados de la misma forma en que entrenaremos K-Means.

```
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans

X_escalado = StandardScaler().fit_transform(X)

inercias = []

for k in range(1, 9):
    modelo = KMeans(
        n_clusters=k,
        random_state=42,
        n_init="auto"
    )
    modelo.fit(X_escalado)
    inercias.append(modelo.inertia_)
```

El escalado no garantiza que los clusters sean buenos, pero evita una fuente importante de distorsión. Permite que las variables participen en el cálculo de distancias de forma más equilibrada.

El escalado no crea buenos clusters por sí solo, pero ayuda a evitar que las distancias queden dominadas por las unidades de medida.

También debemos tener cuidado con la interpretación. Si entrenamos K-Means sobre datos escalados, los centroides quedan expresados en la escala transformada. Esto puede ser útil para analizar si un cluster está por encima o por debajo del promedio en ciertas variables, pero no siempre es cómodo para comunicar.

Por ejemplo, un centroide puede tener valor 1.2 en `frecuencia_compra` y -0.8 en `dias_ultima_compra`. Eso indica que el grupo está por encima del promedio en frecuencia de compra y por debajo del promedio en días desde la última compra. Pero no nos dice directamente cuántas compras realiza ni cuántos días pasaron.

Los centroides escalados son útiles para comparar, aunque resultan menos directos para comunicar.

Para interpretar resultados, suele ser mejor asignar cada fila a un cluster y luego volver al dataset original para calcular estadísticas reales por grupo.

```
datos["cluster"] = pipeline.fit_predict(X)

resumen = datos.groupby("cluster")[columnas_originales].mean()
```

De esta forma, podemos describir cada cluster usando unidades comprensibles: pesos, cantidades, días, metros, edades o porcentajes. Esto facilita la comunicación.

Entrenamos con datos escalados, pero interpretamos los clusters utilizando las variables en sus unidades originales.

También hay que recordar que no todas las variables deben tratarse igual. K-Means trabaja naturalmente con variables numéricas. Si tenemos variables categóricas, no conviene incluirlas como texto. Deben transformarse antes, por ejemplo mediante One-Hot Encoding, aunque esto también puede afectar la noción de distancia.

Tipo de variable	Preparación
Numérica	Puede escalarse
Categórica	Debe codificarse antes de usarse en K-Means

Cuando mezclamos muchas variables categóricas codificadas con variables numéricas, debemos revisar si la distancia resultante sigue teniendo sentido. No todo dataset es adecuado para K-Means solo porque podamos convertirlo en números.

El escalado tampoco resuelve una mala selección de variables. Si incluimos columnas irrelevantes, redundantes o desconectadas del objetivo exploratorio, el agrupamiento puede seguir siendo poco útil. Escalar mejora la comparación numérica, pero no reemplaza el criterio de selección.

Escalar las variables no corrige una selección inadecuada de variables.

Por eso, antes de entrenar K-Means, conviene pensar el flujo completo:

1. Definir qué similitud queremos explorar.
2. Elegir variables relevantes.
3. Tratar los valores faltantes.
4. Codificar las categorías, si corresponde.
5. Escalar las variables numéricas.

6. Entrenar K-Means.

7. Interpretar los clusters en las unidades originales.

El escalado es una etapa fundamental al trabajar con K-Means porque el algoritmo usa distancias para asignar puntos a centroides. Si las variables están en escalas muy distintas, las columnas con valores grandes pueden dominar los clusters. Escalar ayuda a que las distancias reflejen mejor las diferencias relativas entre los casos. Sin embargo, no reemplaza la selección cuidadosa de variables ni la interpretación contextual. En K-Means, preparar bien las variables es tan importante como elegir el número de clusters.

Limitaciones de K-Mean*

K-Means es uno de los algoritmos de clustering más conocidos porque su intuición es clara y su implementación es relativamente simple. Sin embargo, esa simplicidad también trae limitaciones. Para usarlo correctamente, no alcanza con saber que agrupa puntos alrededor de centroides. También debemos entender cuándo esa forma de agrupar puede ser poco adecuada.

Una primera limitación es que K-Means necesita que indiquemos previamente el número de clusters. El algoritmo no decide por sí solo cuántos grupos existen en los datos. Si le pedimos tres clusters, devolverá tres. Si le pedimos cinco, devolverá cinco.

K-Means siempre intenta formar la cantidad de clusters que le pedimos.

Esto puede ser útil cuando tenemos una idea previa de la segmentación que queremos explorar, pero también puede ser problemático. Si elegimos un valor de `k` poco adecuado, el algoritmo igual construirá una solución. Esa solución puede parecer ordenada desde el punto de vista técnico, aunque no tenga demasiado sentido para el problema.

El método del codo puede ayudar, pero no resuelve completamente esta decisión. A veces el codo no es claro. Otras veces sugiere una cantidad de grupos que no resulta práctica o interpretable. Por eso, elegir `k` sigue siendo una decisión de análisis, no una respuesta automática.

El método del codo orienta la elección, pero no garantiza cuál es el número correcto de clusters.

Otra limitación importante es que K-Means funciona mejor cuando los grupos tienen formas compactas y relativamente redondeadas alrededor de un centro. Esto se debe a que el algoritmo organiza los datos según la distancia a centroides.

Si los clusters reales tienen formas alargadas, curvas, irregulares o entrelazadas, K-Means puede separarlos mal. El modelo seguirá buscando centros y asignando puntos por cercanía, aunque esa lógica no represente bien la estructura de los datos.

K-Means tiende a buscar grupos compactos alrededor de centroides.

Imaginemos datos distribuidos en forma de media luna o en anillos. Aunque visualmente podamos reconocer dos grupos, K-Means puede tener dificultades porque esos grupos no se organizan naturalmente alrededor de centros simples. El algoritmo puede cortar las formas de manera artificial para cumplir con la cantidad de clusters solicitada.

Si los grupos tienen formas complejas, K-Means puede imponer divisiones poco naturales.

Otra dificultad aparece cuando los grupos tienen tamaños muy diferentes. Si un cluster contiene muchos puntos y otro contiene pocos, K-Means puede no representar bien esa diferencia. Como intenta reducir distancias internas, puede dividir un grupo grande o absorber puntos de un grupo pequeño según la distribución de los datos.

También puede tener problemas cuando los clusters tienen densidades distintas. Un grupo puede estar muy concentrado y otro muy disperso. K-Means tiende a usar una lógica de cercanía a centros, pero no modela de forma explícita la densidad de los puntos. Esto puede llevar a asignaciones poco intuitivas.

Los clusters con tamaños o densidades muy diferentes pueden complicar el resultado.

La sensibilidad a valores extremos es otra limitación. Como los centroides se calculan a partir de promedios, los outliers pueden desplazar la posición de los centros. Un caso muy alejado puede influir en el centroide o incluso provocar la formación de un cluster poco representativo.

Por ejemplo, si agrupamos clientes y aparece una compra extremadamente grande por un error de carga o por una situación excepcional, ese valor puede afectar el resultado. El algoritmo no sabe si ese caso es un error, una oportunidad comercial o un registro legítimo poco frecuente.

Los outliers pueden desplazar los centroides y modificar los clusters.

Por eso, antes de aplicar K-Means conviene revisar valores extremos. No siempre hay que eliminarlos, pero sí debemos saber que existen y decidir cómo tratarlos. A veces conviene corregir errores de carga. Otras veces conviene mantener casos raros porque son parte importante del fenómeno. La decisión depende del contexto.

Otra limitación muy importante es la sensibilidad al escalado. Ya vimos que K-Means usa distancias. Si una variable tiene valores mucho más grandes que otra, puede dominar la formación de clusters. Esto puede hacer que el algoritmo agrupe principalmente según esa variable, aunque no sea la más importante desde el punto de vista del análisis.

Sin un escalado adecuado, K-Means puede agrupar según la escala numérica y no según la importancia conceptual.

El escalado ayuda, pero también introduce una decisión. Al escalar, modificamos la forma en que las variables participan en la distancia. Esto suele ser necesario, pero no significa que todas las variables deban importar exactamente lo mismo. En algunos problemas, quizá una variable debería tener más peso que otra. K-Means básico no resuelve esa cuestión automáticamente.

Otra limitación es que K-Means trabaja naturalmente con variables numéricas. Si tenemos variables categóricas, necesitamos transformarlas antes. Una codificación como One-Hot Encoding puede permitir incluir categorías, pero también cambia la noción de distancia y puede aumentar mucho la cantidad de columnas.

K-Means no trabaja directamente con categorías de texto. Primero debemos transformarlas, y esa transformación afecta el agrupamiento.

Esto requiere cuidado. Si una variable categórica tiene muchas categorías, la representación puede volverse grande y dispersa. Además, no siempre la distancia euclidiana sobre variables codificadas representa bien la similitud que nos interesa.

Por ejemplo, si codificamos barrios, profesiones o tipos de producto, el algoritmo puede tratar diferencias categóricas de una manera matemática que no necesariamente coincide con el significado real de esas categorías.

Convertir categorías en números no garantiza que la distancia resultante tenga sentido.

Otra limitación es que K-Means puede depender de la inicialización de los centroides. El algoritmo empieza con centros iniciales y luego los va ajustando. Diferentes puntos de partida pueden llevar a resultados distintos. Las implementaciones actuales reducen este problema repitiendo el proceso varias veces, pero no desaparece completamente como idea.

K-Means puede llegar a soluciones diferentes según los centroides iniciales.

El uso de `random_state` ayuda a reproducir resultados, y `n_init` permite ejecutar varias inicializaciones. Pero sigue siendo conveniente revisar si la solución es estable y si los clusters tienen sentido, en lugar de confiar ciegamente en una única ejecución.

También debemos recordar que K-Means siempre asigna todos los puntos a algún cluster. No deja casos sin clasificar. Esto puede ser una limitación cuando hay outliers o puntos que no pertenecen claramente a ningún grupo. El algoritmo igualmente los asignará al centroide más cercano.

K-Means obliga a que cada punto pertenezca a algún cluster.

En algunos problemas, esto puede ser poco realista. Tal vez ciertos casos deberían tratarse como anomalías o revisarse por separado, en lugar de incluirse dentro de un segmento. K-Means no distingue automáticamente entre un miembro típico de un cluster y un punto lejano asignado por obligación.

Otra limitación es que los clusters obtenidos no tienen significado automático. Aunque el algoritmo devuelva etiquetas de grupo, esas etiquetas no explican nada por sí mismas. Debemos analizar las variables originales, tamaños de grupo, perfiles y ejemplos para construir una interpretación.

Los nombres `Cluster 0`, `Cluster 1` o `Cluster 2` no adquieren significado hasta que analizamos sus características.

Esta limitación no es exclusiva de K-Means, sino del aprendizaje no supervisado en general. Pero en K-Means puede ser fácil caer en la tentación de aceptar los grupos como segmentos reales solo porque el algoritmo los produjo.

También puede ocurrir que los clusters sean técnicamente compactos pero poco útiles. El algoritmo optimiza distancias internas, no utilidad práctica. Puede encontrar grupos que reducen la inercia, pero que no ayudan a tomar decisiones, comunicar un análisis o comprender mejor el problema.

K-Means optimiza la compactación de los grupos, no su significado.

Por eso, después de entrenar el modelo, debemos interpretar los resultados en función del objetivo. Si queríamos segmentar clientes para diseñar acciones comerciales, los clusters deberían describir comportamientos útiles. Si queríamos explorar perfiles de uso, deberían ayudar a comprender diferencias relevantes. Si no aportan claridad, quizá haya que cambiar variables, preprocesamiento, valor de `k` o algoritmo.

K-Means es simple y poderoso como primera herramienta de clustering, pero tiene limitaciones claras. Requiere elegir previamente k , es sensible al escalado, puede verse afectado por outliers e inicialización, trabaja mejor con grupos compactos y aproximadamente redondeados, y no maneja naturalmente variables categóricas. Además, siempre asigna todos los puntos a algún cluster y no garantiza que los grupos encontrados tengan significado práctico. Por eso, sus resultados deben interpretarse como una propuesta de organización, no como una verdad automática sobre los datos.

Agrupar alrededor de centros

En este capítulo estudiamos K-Means, uno de los algoritmos más conocidos para realizar clustering. Su objetivo es agrupar datos no etiquetados formando clusters de casos parecidos entre sí. A diferencia de la clasificación, aquí no existen clases conocidas de antemano. El algoritmo propone una organización posible a partir de las variables disponibles.

La idea central de K-Means es representar cada caso como un punto en un espacio de variables y formar grupos alrededor de centros llamados centroides.

K-Means parte de datos sin etiquetas, representa los casos como puntos en un espacio de variables, calcula centroides y forma clusters.

Cada centroide representa el centro de un cluster. K-Means asigna cada punto al centroide más cercano, recalcula los centroides según los puntos asignados y repite el proceso hasta que la organización se estabiliza. Por eso decimos que es un algoritmo iterativo.

1. Elegir centroides iniciales.
2. Asignar cada punto al centroide más cercano.
3. Recalcular los centroides.
4. Repetir el proceso hasta que se estabilice.

La distancia a los centros es fundamental. K-Means agrupa según cercanía, por lo que la elección de variables y sus escalas influyen directamente en los clusters encontrados. El algoritmo no sabe qué significa “parecido” desde el punto de vista del problema. Solo mide distancias usando las columnas que recibe.

K-Means mide el parecido según las variables utilizadas.

Por eso, antes de aplicar el modelo debemos elegir variables relevantes. Si queremos segmentar clientes por comportamiento de compra, necesitaremos variables

relacionadas con compras. Si usamos variables geográficas, demográficas o administrativas, la estructura encontrada reflejará otro tipo de similitud.

También vimos que K-Means requiere indicar previamente el número de clusters, representado por `k`. Si pedimos tres grupos, el algoritmo devolverá tres. Si pedimos cinco, devolverá cinco. Esa cantidad no se descubre automáticamente: es una decisión de modelado.

El valor de `k` indica la cantidad de clusters que le pedimos al algoritmo.

El método del codo ayuda a explorar valores posibles de `k`. Para usarlo, entrenamos K-Means con distintas cantidades de clusters, calculamos la inercia y observamos cómo cambia. La inercia mide qué tan cerca quedan los puntos de sus centroides. Al aumentar `k`, la inercia suele bajar, pero después de cierto punto la mejora puede volverse menor.

El método del codo consiste en probar varios valores de `k`, graficar la inercia y buscar un cambio de pendiente.

El codo no ofrece una respuesta definitiva. Sugiere valores candidatos. La elección final debe combinar la forma del gráfico con la interpretación de los clusters, sus tamaños, su estabilidad y su utilidad para el objetivo del análisis.

Otro tema central fue el escalado. Como K-Means trabaja con distancias, una variable con valores grandes puede dominar el agrupamiento aunque no sea conceptualmente más importante. Por eso, muchas veces conviene usar `StandardScaler` u otra transformación adecuada antes de entrenar el modelo.

Sin escalado, la escala numérica puede confundirse con importancia para el algoritmo.

Un flujo típico puede combinar escalado y K-Means dentro de un pipeline:

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans

pipeline = Pipeline(steps=[
    ("scaler", StandardScaler()),
    ("modelo", KMeans(
        n_clusters=3,
        random_state=42,
        n_init="auto"
    ))
])
```

Este tipo de flujo ayuda a mantener ordenado el preprocesamiento. Si luego asignamos nuevos casos a clusters ya aprendidos, esos casos deben pasar por el mismo escalado antes de calcular distancias a los centroides.

También analizamos las limitaciones de K-Means. El algoritmo funciona mejor cuando los grupos son compactos, relativamente redondeados y pueden representarse bien alrededor de centros. Puede tener dificultades con grupos alargados, curvos, de tamaños muy distintos o con densidades diferentes.

K-Means busca grupos compactos alrededor de centroides y no representa correctamente todas las formas posibles de clusters.

Además, puede verse afectado por outliers, por la inicialización de los centroides y por la elección de k . También trabaja naturalmente con variables numéricas, por lo que las variables categóricas requieren transformaciones previas que deben usarse con cuidado.

Una idea importante del capítulo es que K-Means optimiza distancias, no significado. Puede producir clusters compactos desde el punto de vista matemático, pero eso no garantiza que sean útiles, interpretables o relevantes para el problema.

El algoritmo produce clusters, pero el análisis humano decide si esos clusters tienen sentido.

Por eso, después de entrenar K-Means, no alcanza con mirar las etiquetas 0, 1 o 2. Debemos analizar cada cluster con las variables originales, revisar promedios, distribuciones, tamaños y ejemplos representativos. Recién entonces podemos proponer interpretaciones como “clientes frecuentes”, “usuarios inactivos” o “perfiles de alto gasto”.

K-Means es una herramienta simple y poderosa para explorar datos no etiquetados mediante clustering. Agrupa puntos según su cercanía a centroides y requiere elegir previamente la cantidad de clusters. Su funcionamiento depende de las variables usadas, del escalado, del valor de k y de la estructura real de los datos. Por eso, sus resultados deben entenderse como una propuesta de organización que necesita interpretación, revisión y contexto.

Capítulo 27 · Reducción de dimensionalidad con PCA

¿Por qué reducir dimensiones?

En muchos problemas de Machine Learning trabajamos con datasets que tienen varias columnas. Cada columna describe una característica del caso observado: edad, ingreso, frecuencia de compra, cantidad de productos, tiempo de uso, temperatura, presión, superficie, antigüedad, cantidad de visitas o cualquier otra variable relevante.

Cuando el número de variables es pequeño, podemos razonar sobre ellas con relativa comodidad. Si tenemos dos variables, podemos imaginar los datos como puntos en un plano. Si tenemos tres, podemos imaginar un espacio tridimensional. Pero cuando el dataset tiene diez, veinte, cien o miles de variables, la situación se vuelve más difícil.

Concepto	Descripción
2 variables	podemos graficar en un plano.
3 variables	podemos imaginar un espacio.
Muchas variables	la visualización directa se vuelve difícil.

La cantidad de variables de un dataset se conoce como dimensionalidad. Si una tabla tiene cinco columnas predictoras, decimos que cada caso está representado en un espacio de cinco dimensiones. Si tiene cincuenta columnas predictoras, cada caso está representado en cincuenta dimensiones.

Aunque no podamos visualizar esos espacios, los modelos sí trabajan con ellos. Cada fila del dataset puede pensarse como un punto ubicado según los valores de sus variables.

Cada fila puede pensarse como un punto situado en un espacio definido por las variables.

Reducir dimensiones significa construir una representación más compacta de esos datos. En lugar de trabajar con todas las variables originales, intentamos expresar parte importante de la información usando menos dimensiones.

La pregunta central es:

¿Podemos resumir muchas variables sin perder demasiada información?

En algunos casos, la respuesta puede ser sí. Esto ocurre especialmente cuando varias variables están relacionadas entre sí o contienen información parcialmente redundante.

Por ejemplo, imaginemos un dataset sobre viviendas. Podemos tener variables como superficie cubierta, cantidad de ambientes, cantidad de dormitorios, cantidad de baños, superficie total y precio estimado. Algunas de estas variables pueden estar relacionadas. Una vivienda más grande suele tener más ambientes, más dormitorios o mayor superficie total.

Variables relacionadas: superficie, ambientes, dormitorios, baños.

Todas pueden expresar, en parte, una idea de tamaño.

Si varias columnas están contando aspectos parecidos de una misma característica general, tal vez podamos resumir parte de esa información en una dimensión más compacta. Esa dimensión no sería exactamente una de las columnas originales, sino una combinación construida a partir de ellas.

Algo similar puede ocurrir con datos de actividad en una plataforma educativa. Podemos tener cantidad de accesos, minutos conectados, actividades iniciadas, actividades completadas, entregas realizadas y consultas enviadas. Varias de esas columnas pueden estar relacionadas con una idea general de participación o actividad.

Muchas variables observadas pueden reflejar un patrón más general.

Reducir dimensiones puede ayudarnos a descubrir o representar esos patrones generales. En lugar de analizar muchas columnas por separado, podemos construir componentes que resumen direcciones importantes de variación en los datos.

Una de las razones más comunes para reducir dimensiones es la visualización. Si tenemos un dataset con muchas variables, no podemos graficarlo directamente en todas sus dimensiones. Pero si logramos resumirlo en dos componentes, podemos construir un gráfico en el plano y observar si aparecen grupos, tendencias o casos inusuales.

El proceso parte de Datos con muchas variables, continúa con reducción a 2 dimensiones y conduce finalmente a gráfico exploratorio.

Este tipo de visualización no reemplaza el análisis completo, pero puede ser muy útil para explorar. A veces permite ver si los datos parecen agruparse, si hay observaciones aisladas o si ciertas clases conocidas se distribuyen de manera diferenciada.

Por ejemplo, si tenemos datos de flores con varias mediciones, podríamos reducirlos a dos dimensiones para observar si las especies tienden a separarse visualmente. Si tenemos clientes, podríamos visualizar si aparecen perfiles distintos. Si tenemos documentos, podríamos ver si ciertos temas quedan cerca unos de otros.

Reducir dimensiones puede convertir datos complejos en una visualización inicial más manejable.

Otra razón para reducir dimensiones es simplificar el análisis. Cuando hay muchas variables, puede ser difícil entender cuáles son importantes, cuáles son redundantes y cuáles agregan ruido. Una representación más compacta puede facilitar una primera lectura del dataset.

También puede ayudar cuando algunas variables están muy correlacionadas. Si dos o más columnas se mueven juntas, quizá no aporten información completamente independiente. Reducir dimensiones puede condensar parte de esa información compartida.

El proceso parte de Variables muy relacionadas, continúa con información parcialmente repetida, luego posible resumen y conduce finalmente a en menos dimensiones.

Esto no significa que debamos reducir dimensiones siempre. A veces las variables originales son pocas, claras e interpretables. En esos casos, conservarlas puede ser mejor. Reducir dimensiones puede hacer que el análisis sea más compacto, pero también puede dificultar la interpretación.

Por ejemplo, una variable como `edad` es fácil de explicar. Una componente construida a partir de muchas variables puede ser más difícil de comunicar. Podemos saber que resume mucha variación, pero no siempre resulta sencillo decir qué significa exactamente.

Reducir dimensiones puede simplificar el dataset, pero también puede hacer menos directa la interpretación.

Por eso, la reducción de dimensionalidad debe entenderse como una herramienta, no como una obligación. La usamos cuando aporta algo: visualizar, resumir, explorar, reducir redundancia o preparar datos para otros métodos.

Una técnica clásica para hacer esto es PCA, sigla de *Principal Component Analysis*, o análisis de componentes principales. PCA busca construir nuevas variables, llamadas componentes principales, que capturan la mayor cantidad posible de variación en los datos.

PCA transforma las variables originales en componentes principales.

La idea de “variación” es importante. Si una dirección de los datos concentra mucha variabilidad, PCA intenta capturarla en una primera componente. Luego busca una segunda componente que capture otra parte de la variación, y así sucesivamente.

No necesitamos entrar todavía en todos los detalles matemáticos. Por ahora, lo importante es comprender que PCA no selecciona simplemente algunas columnas originales. Construye nuevas dimensiones a partir de combinaciones de las variables existentes.

PCA no selecciona columnas existentes, sino que construye nuevas dimensiones que resumen la variación de los datos.

Esta diferencia es fundamental. Si tenemos diez variables y usamos PCA para obtener dos componentes, esas dos componentes no son dos variables originales elegidas por el método. Son dos nuevas coordenadas que combinan información de las diez variables.

Eso puede ser muy útil, pero también exige cuidado. Como las componentes son combinaciones, su interpretación puede no ser inmediata. Una componente puede estar relacionada con tamaño, actividad, intensidad, consumo o comportamiento general, pero esa interpretación debe analizarse mirando cómo se construye y cómo se relaciona con las variables originales.

Las componentes pueden resumir patrones, pero su significado debe interpretarse con cuidado.

Reducir dimensiones también implica pérdida de información. Si pasamos de diez variables a dos componentes, estamos construyendo un resumen. Todo resumen deja algo afuera. La pregunta no es si se pierde información, sino cuánta se pierde y si el resumen sigue siendo útil para el objetivo.

Reducir dimensiones siempre implica resumir, y todo resumen deja parte de la información afuera.

Por eso, PCA permite analizar cuánta variación explica cada componente. Más adelante veremos la idea de varianza explicada, que ayuda a decidir cuántas componentes conservar y qué tan representativo es el resumen.

Otra razón para reducir dimensiones puede ser preparar datos para un modelo posterior. Si tenemos muchas variables correlacionadas, una representación compacta puede ayudar a ciertos modelos. Sin embargo, esto debe evaluarse. No

siempre reducir dimensiones mejora el desempeño predictivo. A veces ayuda, a veces no, y a veces perjudica.

Reducir dimensiones puede ayudar a algunos modelos, pero no garantiza mejores predicciones.

Si el objetivo es la predicción, la reducción de dimensionalidad debe incorporarse dentro de un flujo correctamente evaluado. PCA aprende una transformación a partir de X . Por lo tanto, si estamos trabajando con entrenamiento y prueba, debe ajustarse solo con los datos de entrenamiento para evitar fuga de datos.

PCA aprende una transformación a partir de los datos. Por eso, si existe una separación entre entrenamiento y prueba, debe ajustarse únicamente con el conjunto de entrenamiento.

Como ocurre con otras transformaciones, una forma segura de organizar el flujo es usar un pipeline.

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

pipeline = Pipeline(steps=[
    ("scaler", StandardScaler()),
    ("pca", PCA(n_components=2))
])
```

El escalado aparece porque PCA es sensible a las escalas de las variables. Si una columna tiene valores mucho más grandes que otra, puede dominar las componentes principales. Por eso, muchas veces se usa `StandardScaler` antes de PCA.

El proceso parte de Escalado, continúa con PCA y conduce finalmente a componentes principales.

Reducimos dimensiones para representar datos complejos de una manera más compacta. Esto puede servir para visualizar, explorar, resumir información, reducir redundancia o preparar datos para otros análisis. PCA es una técnica clásica que construye nuevas dimensiones llamadas componentes principales, intentando capturar la mayor variación posible. Pero reducir dimensiones no es gratis: implica pérdida de información y puede dificultar la interpretación. Por eso, debemos usarlo cuando aporta claridad o utilidad, no como una transformación automática.

Variables correlacionadas

Una de las situaciones donde PCA puede resultar útil es cuando tenemos variables correlacionadas. Dos variables están correlacionadas cuando tienden a moverse juntas de alguna manera. Si una aumenta y la otra también suele aumentar, hablamos de una relación positiva. Si una aumenta y la otra tiende a disminuir, hablamos de una relación negativa.

Por ejemplo, en un dataset de viviendas, es razonable esperar cierta relación entre superficie, cantidad de ambientes, cantidad de dormitorios y cantidad de baños. No son la misma variable, pero suelen estar conectadas. Las viviendas más grandes tienden a tener más ambientes; las que tienen más ambientes suelen tener más dormitorios; las que tienen más dormitorios pueden tener más baños.

superficie ambientes dormitorios baños

Variables distintas, pero relacionadas entre sí.

Cuando varias variables están correlacionadas, puede haber información repetida o parcialmente redundante. No significa que las columnas sean inútiles, ni que podamos borrar todas menos una sin pensar. Significa que algunas de ellas pueden estar describiendo aspectos relacionados de un mismo patrón general.

En el ejemplo de viviendas, muchas columnas pueden estar reflejando una dimensión amplia que podríamos llamar “tamaño” o “magnitud de la propiedad”. PCA puede aprovechar este tipo de relación para construir una componente que resuma buena parte de esa variación compartida.

El proceso parte de Varias variables relacionadas, continúa con patrón común, luego posible resumen y conduce finalmente a en una componente principal.

Algo similar puede ocurrir con datos de actividad en una plataforma. Variables como cantidad de accesos, minutos conectados, actividades completadas, entregas realizadas y consultas enviadas pueden estar relacionadas con una idea general de participación. No son idénticas, pero pueden moverse en conjunto.

Un estudiante muy activo quizá tenga muchos accesos, muchas actividades completadas y varias entregas. Un estudiante con baja actividad quizá tenga pocos accesos, poco tiempo conectado y pocas entregas. En ese caso, PCA podría encontrar una dirección principal que capture esa diferencia general entre mayor y menor actividad.

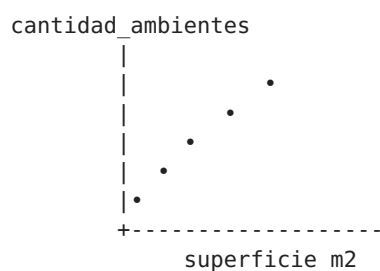
Muchas señales de actividad pueden resumirse parcialmente en una dimensión común.

Esta es una de las ideas más importantes de PCA: no busca simplemente eliminar columnas, sino construir nuevas dimensiones que capturen patrones de variación compartidos entre variables.

Si las variables no tienen relación entre sí, reducir dimensiones puede ser más difícil. Cada variable podría aportar información distinta y no habría tanto para resumir. En cambio, cuando hay correlación, suele existir cierta redundancia que PCA puede aprovechar.

El proceso parte de Más relación entre variables, continúa con más posibilidad de resumir información, luego Menos relación entre variables y conduce finalmente a menos redundancia para aprovechar.

Para entender esto, imaginemos una tabla con dos variables muy relacionadas: `superficie_m2` y `cantidad_ambientes`. Si graficamos los puntos, quizá formen una nube alargada. A medida que aumenta la superficie, también tiende a aumentar la cantidad de ambientes.



En ese caso, los datos tienen una dirección principal de variación. La nube no se extiende de cualquier manera, sino principalmente a lo largo de una diagonal. PCA intenta encontrar esa dirección donde los datos varían más.

La primera componente principal se orienta en la dirección que captura la mayor variación posible. En este ejemplo simple, podría representar una combinación de superficie y ambientes. Esa componente no sería exactamente superficie ni exactamente cantidad de ambientes, sino una nueva dimensión que resume la tendencia compartida.

Componente principal: nueva dimensión que captura una dirección importante de variación en los datos.

La segunda componente, si existe, capturaría una dirección distinta, perpendicular a la primera, donde queda otra parte de la variación. En un caso de dos variables muy correlacionadas, esa segunda dirección puede contener menos información, porque la mayor parte de la variación ya estaba alineada con la primera.

Esta intuición se extiende a más variables. Si tenemos diez columnas, PCA busca una primera dirección que explique la mayor variación posible, luego una segunda dirección que explique otra parte, y así sucesivamente. Cada componente es una combinación de las variables originales.

PCA busca direcciones: primera componente, segunda componente, tercera componente, ...

La correlación entre variables ayuda a que una o pocas componentes puedan resumir bastante información. Por eso, antes de aplicar PCA, puede ser útil revisar la matriz de correlación o explorar si algunas variables están fuertemente relacionadas.

```
correlaciones = datos[columnas_numericas].corr()
```

Esta matriz nos muestra cómo se relacionan las variables numéricas entre sí. Valores cercanos a 1 indican relación positiva fuerte. Valores cercanos a -1 indican relación negativa fuerte. Valores cercanos a 0 sugieren poca relación lineal.

Concepto	Descripción
Correlación cercana a 1	las variables tienden a aumentar juntas.
Correlación cercana a -1	una tiende a aumentar cuando la otra disminuye.
Correlación cercana a 0	no hay relación lineal clara.

PCA se apoya en relaciones lineales entre variables. Esto significa que captura mejor estructuras donde las variables se combinan de manera aproximadamente lineal. Si la relación entre variables es muy compleja o no lineal, PCA puede no resumirla bien.

Este punto es importante. PCA no descubre cualquier tipo de patrón. Es una técnica potente y clásica, pero tiene una forma específica de mirar los datos: busca direcciones lineales de máxima variación.

PCA resume especialmente bien las relaciones lineales entre variables.

También debemos tener cuidado con la escala. Una variable puede parecer muy importante para PCA simplemente porque tiene valores grandes o mucha varianza en sus unidades originales. Por ejemplo, una variable medida en pesos puede tener números mucho mayores que una variable medida en cantidad de compras. Si no escalamos, la primera puede dominar las componentes.

PCA es sensible a la escala de las variables, por lo que antes de aplicarlo suele ser necesario estandarizar los datos.

Por eso, muchas veces se usa `StandardScaler` antes de PCA. El escalado permite que las variables se comparen de manera más equilibrada, especialmente cuando están medidas en unidades distintas.

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

pipeline = Pipeline(steps=[
    ("scaler", StandardScaler()),
    ("pca", PCA(n_components=2))
])
```

Si todas las variables están en la misma unidad y la varianza tiene un significado directo que queremos conservar, tal vez no siempre sea necesario escalar. Pero en la mayoría de los datasets mixtos, donde combinamos edades, montos, cantidades, porcentajes y tiempos, escalar antes de PCA suele ser una decisión razonable.

Las variables correlacionadas también pueden generar dificultades en algunos modelos. En modelos lineales, por ejemplo, la presencia de columnas muy relacionadas puede complicar la interpretación de coeficientes. En esos casos, PCA puede transformar variables correlacionadas en componentes no correlacionadas entre sí. Esto puede ser útil en ciertos flujos, aunque también reduce la interpretación directa de las variables.

PCA puede transformar variables correlacionadas en componentes no correlacionadas.

Pero no debemos concluir que PCA siempre es la mejor solución frente a variables correlacionadas. A veces basta con elegir variables, eliminar redundancias claras o usar modelos capaces de manejar correlaciones. PCA es una herramienta posible, no una obligación.

También debemos recordar que una correlación alta no implica causalidad. Si superficie y precio están correlacionados, eso no significa que una única variable explique todo el fenómeno. Si actividad en plataforma y rendimiento académico se relacionan, no significa automáticamente que una cause la otra. PCA resume variación, no explica causas.

PCA encuentra patrones de variación, pero no demuestra relaciones causales.

Esta advertencia es importante para comunicar resultados. Una componente principal puede resumir un conjunto de variables correlacionadas, pero el significado de esa componente debe interpretarse con prudencia. Podemos decir que parece asociada a actividad, tamaño, intensidad o consumo, pero esa interpretación debe apoyarse en el análisis de las variables que más contribuyen a ella.

PCA es especialmente útil cuando varias variables están correlacionadas y contienen información parcialmente redundante. En lugar de analizar muchas columnas relacionadas por separado, PCA puede construir componentes principales que resumen direcciones importantes de variación. Estas componentes pueden ayudar a visualizar, simplificar o preparar datos para otros análisis. Sin embargo, PCA captura principalmente relaciones lineales, es sensible al escalado y no convierte correlaciones en explicaciones causales. Por eso, las variables correlacionadas son una oportunidad para reducir dimensiones, pero siempre requieren interpretación cuidadosa.

Componentes principales

PCA transforma las variables originales en nuevas variables llamadas componentes principales. Estas componentes son el corazón del método. No son columnas tomadas directamente del dataset, sino nuevas dimensiones construidas a partir de combinaciones de las variables originales.

Esta idea es fundamental. PCA no elige “las mejores columnas” del dataset. No selecciona `edad`, `ingresos` o `frecuencia_compra` y descarta otras. En cambio, crea nuevas dimensiones que resumen patrones de variación presentes en los datos.

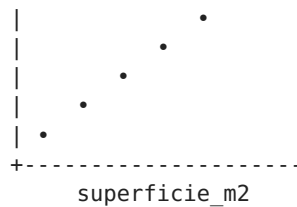
PCA transforma las variables originales en componentes principales.

Cada componente principal es una dirección en el espacio de los datos. Esa dirección indica una forma importante en que los datos varían. La primera componente principal es la dirección que captura la mayor variación posible. La segunda captura la mayor variación restante, con una condición: debe ser independiente de la primera en el sentido geométrico del método.

Concepto	Descripción
Primera componente	mayor variación posible.
Segunda componente	mayor variación restante.
Componentes siguientes	otras direcciones de variación.

Para entenderlo, imaginemos nuevamente un dataset con dos variables relacionadas: superficie de una vivienda y cantidad de ambientes. Si las viviendas más grandes tienden a tener más ambientes, los puntos del gráfico formarán una nube alargada. Esa nube tendrá una dirección principal.

cantidad_ambientes
|



La primera componente principal se orientaría aproximadamente siguiendo esa nube alargada. No sería exactamente el eje de superficie ni exactamente el eje de cantidad de ambientes. Sería una nueva dirección que combina ambas variables y resume la tendencia principal de los datos.

Componente principal: una nueva dirección que combina variables originales para resumir variación.

En este ejemplo, podríamos interpretar esa primera componente como una dimensión general de “tamaño de la vivienda”, porque aumenta cuando aumentan tanto la superficie como la cantidad de ambientes. Pero esa interpretación no viene escrita automáticamente. Debemos construirla observando qué variables participan más en la componente y en qué sentido.

La segunda componente, en este caso de dos variables, capturaría la variación que queda en una dirección perpendicular a la primera. Si la nube está muy alargada, esa segunda dirección puede contener poca información. Si los puntos están más dispersos en varias direcciones, la segunda componente puede ser más relevante.

Una componente resume la dirección principal.

Otra componente puede capturar variación que no quedó resumida por la primera.

Cuando tenemos más variables, ocurre algo similar, aunque sea más difícil de visualizar. Si el dataset tiene diez variables, PCA puede construir hasta diez componentes. La primera capturará la mayor variación posible, la segunda otra parte, la tercera otra, y así sucesivamente.

Pero normalmente no usamos todas las componentes. Si usáramos todas, no habríamos reducido la dimensionalidad. El objetivo suele ser conservar solo las primeras componentes, aquellas que explican una parte importante de la variación total.

El proceso parte de Muchas variables originales, continúa con muchas componentes posibles, luego elegimos algunas y conduce finalmente a representación reducida.

Por ejemplo, si tenemos diez variables originales, podríamos reducir el dataset a dos componentes para visualizarlo en un plano. O podríamos conservar cinco componentes si queremos mantener más información para un modelo posterior.

En `scikit-learn`, esto se indica con el parámetro `n_components`.

```
from sklearn.decomposition import PCA  
  
pca = PCA(n_components=2)
```

En este ejemplo, pedimos que PCA transforme los datos en dos componentes principales. Si el dataset original tenía diez variables, la salida tendrá dos columnas nuevas.

Concepto	Descripción
Entrada	10 variables originales.
Salida con PCA(n_components=2)	2 componentes principales.

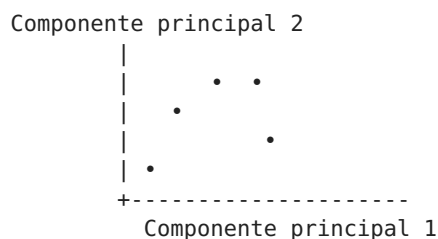
Una vez ajustado, PCA puede transformar los datos originales en esa nueva representación.

```
X_pca = pca.fit_transform(X)
```

El resultado `X_pca` contiene los datos expresados en términos de componentes principales. Cada fila sigue representando un caso, pero las columnas ya no son las variables originales. Son las componentes construidas por PCA.

Concepto	Descripción
Antes	fila descrita por variables originales.
Después	fila descrita por componentes principales.

Esto permite graficar datos de muchas variables en dos dimensiones, si elegimos dos componentes. Por ejemplo, la primera componente puede ir en el eje horizontal y la segunda en el eje vertical.



Este gráfico no muestra dos variables originales. Muestra dos dimensiones construidas por PCA. Por eso, su interpretación debe ser cuidadosa. Si vemos grupos o separaciones, son patrones en la representación reducida, no necesariamente divisiones completas y definitivas del dataset original.

Las componentes principales tienen una propiedad importante: están ordenadas. La primera componente explica más variación que la segunda. La segunda explica más que la tercera, y así sucesivamente. Este orden permite decidir cuántas componentes conservar.

Las componentes aparecen ordenadas desde la que más variación explica hasta la que menos variación explica.

En la próxima sección estudiaremos la varianza explicada, que nos ayuda a cuantificar cuánta información conserva cada componente y cuánta se pierde al reducir dimensiones.

También es importante entender que las componentes son combinaciones lineales de las variables originales. Esto significa que cada componente se construye combinando las columnas originales con distintos pesos. Algunas variables pueden aportar mucho a una componente, otras poco.

Cada componente combina las variables originales con distintos pesos.

En `scikit-learn`, esos pesos pueden consultarse mediante el atributo `components_`.

```
pca.components_
```

Cada fila de `components_` corresponde a una componente principal, y cada columna corresponde a una variable original. Estos valores ayudan a analizar qué variables influyen más en cada componente.

Sin embargo, no siempre es sencillo interpretar una componente. Si una componente combina muchas variables en distintos sentidos, asignarle un nombre puede ser difícil. A veces podemos decir que una componente parece asociada a tamaño, actividad, consumo o intensidad. Otras veces, la interpretación puede ser menos clara.

Las componentes pueden ser útiles aunque no siempre resulten fáciles de nombrar.

Esto marca una diferencia importante entre reducción de dimensionalidad e interpretación directa. PCA puede simplificar el dataset, pero esa simplificación puede alejarse de las variables originales. Ganamos una representación compacta, pero podemos perder claridad conceptual.

Por ejemplo, si usamos variables de actividad educativa y la primera componente aumenta con accesos, entregas y actividades completadas, podríamos interpretarla como “actividad general”. Pero si una componente mezcla accesos, tiempo conectado, consultas, retrasos y participación de formas distintas, su significado puede ser más difícil.

Una componente no debe nombrarse sin revisar cómo se relaciona con las variables originales.

También debemos recordar que PCA es sensible al escalado. Si las variables originales están en escalas muy diferentes, las componentes pueden quedar dominadas por las columnas con mayor varianza numérica. Por eso, muchas veces aplicamos `StandardScaler` antes de PCA.

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

pipeline = Pipeline(steps=[
    ("scaler", StandardScaler()),
    ("pca", PCA(n_components=2))
])
```

En este pipeline, primero se estandarizan las variables y luego se calculan las componentes principales. Esto ayuda a que las diferencias de escala no dominen la reducción.

Si PCA se usa dentro de un flujo predictivo, el pipeline también ayuda a evitar fuga de datos. El escalado y PCA deben ajustarse solo con los datos de entrenamiento durante la evaluación, porque ambos aprenden información a partir de `X`.

PCA aprende una transformación a partir de los datos.

Por eso debe integrarse correctamente al flujo de evaluación.

Otra cuestión importante es que las componentes principales no tienen unidades originales. Si las variables originales estaban en pesos, años, metros o cantidades, las componentes ya no se interpretan directamente en esas unidades. Son coordenadas en un nuevo sistema de referencia.

Concepto	Descripción
Variables originales	tienen unidades comprensibles.
Componentes principales	son nuevas coordenadas sin una unidad directa evidente.

Esto no impide usarlas, pero afecta la comunicación. Para un público técnico, un gráfico con componentes principales puede ser útil. Para un informe dirigido a personas no técnicas, tal vez necesitemos explicar con cuidado qué representan o complementar el análisis con variables originales.

Las componentes principales son nuevas dimensiones creadas por PCA a partir de combinaciones de las variables originales. La primera componente captura la mayor variación posible, la segunda captura otra parte de la variación restante, y así sucesivamente. Al conservar solo algunas componentes, podemos reducir la dimensionalidad del dataset y construir representaciones más compactas. Sin embargo, estas componentes no son variables originales, no tienen una interpretación automática y deben analizarse con cuidado, especialmente cuando queremos comunicar resultados.

Varianza explicada

Cuando usamos PCA para reducir dimensiones, necesitamos saber cuánta información conserva la nueva representación. Si pasamos de muchas variables originales a pocas componentes principales, inevitablemente estamos resumiendo. La pregunta es cuánto de la variación original queda representada en esas componentes.

Para eso usamos la idea de varianza explicada.

En PCA, la varianza representa cuánto cambian o se dispersan los datos en una determinada dirección. Una componente principal explica mucha varianza si, al proyectar los datos sobre esa componente, conserva una parte importante de la variación que existía en el dataset original.

El proceso parte de Más varianza explicada, continúa con la componente conserva, luego más información sobre la variación y conduce finalmente a de los datos..

La primera componente principal es la que explica la mayor cantidad posible de varianza. La segunda explica la mayor cantidad posible de la varianza restante. La tercera explica otra parte, y así sucesivamente.

Concepto	Descripción
Componente 1	explica la mayor parte posible.
Componente 2	explica otra parte.
Componente 3	explica otra parte adicional.

Esto permite ordenar las componentes según su importancia para resumir los datos. No todas las componentes aportan lo mismo. Las primeras suelen concentrar más información; las últimas pueden explicar variaciones más pequeñas.

Supongamos que tenemos un dataset con diez variables originales. PCA puede construir hasta diez componentes principales. Pero quizá las dos primeras expliquen una parte considerable de la variación total. En ese caso, podríamos usar esas dos componentes para visualizar los datos, sabiendo que estamos conservando una porción importante de su estructura.

El proceso parte de 10 variables originales, continúa con PCA, luego 10 componentes posibles, luego quizá las primeras 2 o 3 y conduce finalmente a resumen gran parte de la variación..

La varianza explicada suele expresarse como proporción o porcentaje. Por ejemplo, si la primera componente explica el 55% de la varianza y la segunda explica el 25%, juntas explican el 80% de la variación total.

Concepto	Descripción
Componente 1	55% Componente 2: 25%
Total con 2 componentes	80%

Esto significa que, al representar los datos con dos componentes, estamos conservando aproximadamente el 80% de la variación capturada por las variables originales. El 20% restante queda fuera de esa representación reducida.

Reducir dimensiones significa conservar una parte y dejar otra parte afuera.

En `scikit-learn`, después de ajustar PCA, podemos consultar la proporción de varianza explicada mediante `explained_variance_ratio_`.

```
from sklearn.decomposition import PCA

pca = PCA(n_components=2)
X_pca = pca.fit_transform(X)

pca.explained_variance_ratio_
```

Si el resultado fuera:

```
array([0.55, 0.25])
```

podríamos interpretarlo como que la primera componente explica el 55% de la varianza y la segunda el 25%.

Para conocer la varianza explicada acumulada, podemos sumar esos valores.

```
pca.explained_variance_ratio_.sum()
```

En este ejemplo, el resultado sería 0.80, es decir, 80%.

Cuando usamos más componentes, también podemos calcular la varianza acumulada componente por componente. Esto ayuda a decidir cuántas conservar.

```
import numpy as np

varianza_acumulada = np.cumsum(pca.explained_variance_ratio_)
```

La varianza acumulada permite responder preguntas como:

¿Cuántas componentes necesito para conservar el 80%, 90% o 95% de la variación?

Por ejemplo, si las primeras tres componentes explican el 72%, las primeras cinco explican el 88% y las primeras siete explican el 95%, podemos decidir cuántas usar según el objetivo.

No existe un porcentaje obligatorio. Para visualización, a veces usamos solo dos componentes aunque expliquen una proporción moderada, porque queremos construir un gráfico. Para preparar datos para otro modelo, quizá prefiramos conservar más componentes. Para exploración inicial, puede alcanzar con una representación más simple.

La cantidad de componentes depende del objetivo: visualizar, resumir, modelar o explorar.

También podemos pedirle a PCA que conserve automáticamente la cantidad de componentes necesaria para alcanzar cierta proporción de varianza explicada. En `scikit-learn`, `n_components` puede recibir un número decimal entre 0 y 1.

```
from sklearn.decomposition import PCA

pca = PCA(n_components=0.90)
```

En este caso, PCA conservará la cantidad de componentes necesarias para explicar aproximadamente el 90% de la varianza.

Esto puede ser útil cuando no sabemos cuántas componentes elegir, pero sí tenemos un criterio de conservación de información. Sin embargo, tampoco debe usarse de manera mecánica. Conservar el 90% de la varianza no garantiza que el resultado sea útil o interpretable para el problema.

Más varianza explicada no significa automáticamente mejor análisis.

La varianza explicada mide cuánta variación conserva la representación, pero no nos dice si esa variación es relevante para nuestro objetivo. Puede ocurrir que una componente explique mucha varianza, pero esté asociada a una diferencia poco importante para la pregunta que nos interesa. También puede ocurrir que una componente de menor varianza contenga información relevante para distinguir ciertos casos.

Por ejemplo, en un dataset de clientes, la mayor variación puede estar asociada al monto de compra. Pero si nuestro interés es detectar abandono, quizá otras variables menos dominantes sean importantes. PCA no sabe cuál es nuestro objetivo; solo resume variación en X .

PCA conserva variación, no necesariamente lo más importante para una decisión.

Esto es especialmente importante si usamos PCA antes de un modelo supervisado. PCA no usa la variable objetivo y . Por lo tanto, las componentes que más varianza explican no son necesariamente las que mejor predicen la clase o el valor objetivo.

PCA analiza solamente X ; no sabe qué variables son más útiles para predecir y .

Por eso, si incorporamos PCA a un flujo predictivo, debemos evaluar el pipeline completo con datos no vistos. No alcanza con conservar mucha varianza. Debemos comprobar si el modelo resultante predice mejor, igual o peor.

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.linear_model import LogisticRegression

pipeline = Pipeline(steps=[
    ("scaler", StandardScaler()),
    ("pca", PCA(n_components=0.90)),
    ("modelo", LogisticRegression())
])
```

Este pipeline primero escala, luego conserva suficientes componentes para explicar aproximadamente el 90% de la varianza y finalmente entrena un modelo de clasificación. La evaluación debe realizarse sobre el flujo completo.

La varianza explicada también ayuda a interpretar visualizaciones. Si graficamos las dos primeras componentes y juntas explican el 85% de la varianza, la visualización representa una parte importante de la estructura del dataset. Si juntas explican solo el 30%, el gráfico puede seguir siendo útil como exploración, pero debemos recordar que deja fuera mucha información.

Concepto	Descripción
Gráfico con 85% de varianza	representación bastante informativa.
Gráfico con 30% de varianza	representación más parcial.

Esto no significa que un gráfico con baja varianza explicada no sirva. Puede mostrar patrones interesantes. Pero debemos comunicarlo con prudencia. No deberíamos sacar conclusiones fuertes de una representación que conserva solo una parte pequeña de la variación total.

También podemos visualizar la varianza explicada mediante un gráfico. Una opción es graficar cuánta varianza explica cada componente. Otra es graficar la varianza acumulada.

```
import matplotlib.pyplot as plt
import numpy as np

pca = PCA()
X_pca = pca.fit_transform(X)

varianza_acumulada = np.cumsum(pca.explained_variance_ratio_)

plt.plot(
    range(1, len(varianza_acumulada) + 1),
    varianza_acumulada,
    marker="o"
)
plt.xlabel("Número de componentes")
plt.ylabel("Varianza explicada acumulada")
plt.title("Varianza explicada por PCA")
plt.show()
```

Este gráfico permite observar cuántas componentes necesitamos para alcanzar cierto nivel de información conservada. Si la curva sube rápido, pocas componentes resumen mucha variación. Si sube lentamente, necesitamos más componentes para conservar una proporción alta.

Concepto	Descripción
Curva que sube rápido	pocas componentes resumen bastante.
Curva que sube lentamente	la información está más distribuida.

La varianza explicada, entonces, es una herramienta para decidir. No reemplaza el criterio, pero aporta una medida concreta de cuánta información conserva cada componente.

La varianza explicada indica qué proporción de la variación original del dataset conserva cada componente principal. Las primeras componentes suelen explicar más varianza que las siguientes, y la varianza acumulada nos ayuda a decidir cuántas componentes conservar. Sin embargo, explicar mucha varianza no significa necesariamente explicar lo más importante para el objetivo del análisis. PCA resume estructura en X , no interpreta el problema por nosotros. Por eso, la varianza explicada debe usarse junto con visualizaciones, evaluación e interpretación contextual.

Usos frecuentes

PCA puede usarse con distintos propósitos dentro de un flujo de análisis de datos o Machine Learning. No siempre se aplica por la misma razón. A veces se usa para visualizar, otras para resumir información, otras para reducir redundancia y, en algunos casos, como parte del preprocesamiento antes de entrenar otro modelo.

Uno de sus usos más frecuentes es la exploración inicial de datos con muchas variables. Cuando un dataset tiene muchas columnas numéricas, puede ser difícil observar su estructura general. PCA permite construir una representación más compacta y analizar si aparecen patrones, grupos o casos inusuales.

El proceso parte de Dataset con muchas variables, continúa con PCA, luego representación más compacta y conduce finalmente a exploración inicial.

Por ejemplo, si tenemos datos de clientes con variables de frecuencia de compra, gasto promedio, antigüedad, cantidad de productos, uso de promociones y días desde la última compra, podemos usar PCA para observar si existen perfiles generales de comportamiento.

Quizá la primera componente esté asociada a una dimensión amplia de actividad comercial. Tal vez los clientes con valores altos en esa componente compren más seguido, gasten más y tengan mayor variedad de productos. La segunda componente podría reflejar otra diferencia, como clientes recientes frente a clientes antiguos, o compradores constantes frente a compradores estacionales.

PCA puede ayudar a descubrir direcciones generales de variación en los datos.

Estas interpretaciones no aparecen automáticamente. Debemos revisar las variables originales y cómo contribuyen a cada componente. Pero PCA puede ayudarnos a formular hipótesis que luego podemos investigar con más detalle.

Otro uso muy común es la visualización. Si reducimos el dataset a dos componentes principales, podemos graficar los casos en un plano. Si reducimos a tres, podemos construir una visualización tridimensional, aunque muchas veces dos dimensiones son más cómodas para comunicar.

El proceso parte de Muchas variables, continúa con dos componentes principales y conduce finalmente a gráfico en 2D.

Esto es especialmente útil cuando tenemos etiquetas conocidas y queremos observar si las clases se separan en la representación reducida. Por ejemplo, si tenemos flores clasificadas por especie, podemos usar PCA para proyectar las mediciones en dos dimensiones y colorear los puntos según la especie. Si las especies aparecen separadas, eso puede sugerir que las variables contienen información útil para distinguirlas.

PCA + colores por clase puede ayudar a observar si las clases se separan visualmente.

En ese caso, PCA no está clasificando. Solo reduce las variables para permitir una visualización. La separación visual puede ser una pista, pero no reemplaza la evaluación de un modelo supervisado.

También podemos usar PCA sin etiquetas, para observar si los datos forman grupos naturales o zonas de concentración. En ese caso, el gráfico puede sugerir si tendría sentido probar clustering o revisar posibles anomalías.

Una visualización con PCA ofrece una pista exploratoria, no una conclusión definitiva.

Otro uso frecuente es reducir redundancia entre variables correlacionadas. Cuando muchas columnas están relacionadas, PCA puede resumir parte de esa información en menos componentes. Esto puede hacer que el dataset sea más manejable.

Por ejemplo, en mediciones físicas o sensores, distintas variables pueden capturar aspectos relacionados de un mismo proceso. En datos económicos, algunas variables pueden moverse juntas. En datos educativos, distintas medidas de participación pueden reflejar una dimensión general de actividad.

El proceso parte de Variables correlacionadas, continúa con información parcialmente repetida, luego componentes que resumen y conduce finalmente a esa variación compartida.

PCA también puede usarse antes de ciertos modelos para reducir la cantidad de columnas. Si un dataset tiene muchas variables, entrenar algunos modelos puede

volverse más costoso o menos estable. Una representación con menos componentes puede simplificar el entrenamiento.

Sin embargo, esto debe evaluarse con cuidado. Reducir dimensiones no garantiza mejorar el desempeño predictivo. A veces mejora, a veces no cambia demasiado y a veces empeora porque se pierde información relevante para la variable objetivo.

PCA puede simplificar los datos antes de aplicar un modelo, pero no garantiza mejores predicciones.

Esto ocurre porque PCA no usa y . Busca conservar variación en X , pero no sabe cuáles direcciones son más importantes para predecir una clase o un valor. Una componente que explica mucha varianza puede no ser la más útil para la predicción. Una componente que explica poca varianza podría contener información importante para distinguir casos.

Por eso, si PCA se usa como parte de un modelo supervisado, debe evaluarse el pipeline completo.

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.linear_model import LogisticRegression

pipeline = Pipeline(steps=[
    ("scaler", StandardScaler()),
    ("pca", PCA(n_components=0.90)),
    ("modelo", LogisticRegression())
])
```

En este ejemplo, PCA conserva suficientes componentes para explicar aproximadamente el 90% de la varianza. Luego, esas componentes se usan como entrada para una regresión logística. La pregunta importante no es solo cuánta varianza se conserva, sino cómo rinde el modelo completo en datos no vistos.

Cuando PCA forma parte de un modelo supervisado, debe evaluarse junto con todo el pipeline.

Otro uso posible es reducir ruido. Si las primeras componentes concentran patrones principales y las últimas capturan variaciones pequeñas, ruido o detalles poco relevantes, conservar solo algunas componentes puede ayudar a obtener una representación más estable. Pero esta idea también debe aplicarse con prudencia. No toda variación pequeña es ruido, y no todo ruido queda necesariamente en las últimas componentes.

PCA puede ayudar a reducir ruido, pero no sabe distinguir por sí solo qué variación es importante.

PCA también puede servir para comprimir datos. Si podemos representar un conjunto de muchas variables con menos componentes conservando una proporción alta de varianza, estamos construyendo una versión más compacta. Esto puede ser útil para almacenamiento, transmisión, visualización o análisis posterior.

En imágenes, por ejemplo, cada píxel puede considerarse una variable. Una imagen pequeña en escala de grises puede tener cientos o miles de valores. PCA puede usarse para representar esas imágenes mediante menos componentes, conservando parte importante de la información visual. Este tipo de uso muestra que la reducción de dimensionalidad puede aplicarse más allá de tablas tradicionales.

El proceso parte de Muchas variables, continúa con representación comprimida, luego menos dimensiones, luego con parte importante y conduce finalmente a de la información.

También puede usarse para preparar datos antes de clustering. Si tenemos muchas variables correlacionadas, aplicar K-Means directamente puede ser difícil de interpretar o verse afectado por redundancia. Una alternativa es reducir dimensiones con PCA y luego aplicar clustering sobre las componentes. Esto puede facilitar ciertas exploraciones, aunque no siempre será la mejor opción.

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.cluster import KMeans

pipeline = Pipeline(steps=[
    ("scaler", StandardScaler()),
    ("pca", PCA(n_components=2)),
    ("cluster", KMeans(n_clusters=3, random_state=42, n_init="auto"))
])
```

Este flujo escala las variables, reduce a dos componentes y luego aplica K-Means. Puede servir para explorar grupos en una representación más simple. Sin embargo, debemos recordar que los clusters se formarán en el espacio reducido, no en el espacio completo original.

Clustering después de PCA: los grupos dependen de la representación reducida.

Por eso, si usamos PCA antes de clustering, debemos revisar cuánta varianza conservamos, si la representación tiene sentido y si los clusters resultantes son interpretables en términos de las variables originales.

PCA también puede ser útil para detectar casos inusuales. Una visualización en componentes principales puede mostrar puntos alejados del resto. Esos puntos pueden ser outliers, errores de carga, casos especiales o simplemente observaciones

poco frecuentes. Como siempre, la detección visual debe tomarse como una pista, no como una conclusión definitiva.

Puntos aislados en PCA pueden sugerir casos para revisar.

En todos estos usos, hay una condición común: PCA debe aplicarse con preprocesamiento adecuado. Como es sensible a la escala, muchas veces conviene estandarizar las variables numéricas antes de calcular componentes. Si mezclamos variables en pesos, edades, cantidades, porcentajes y tiempos sin escalar, las componentes pueden quedar dominadas por las columnas con mayor varianza numérica.

Antes de PCA, la escala de las variables debe revisarse.

También debemos recordar que PCA trabaja con variables numéricas. Si tenemos variables categóricas, necesitamos decidir cómo tratarlas. En muchos casos, PCA se aplica sobre columnas numéricas continuas. Si usamos variables categóricas codificadas, debemos tener cuidado con la interpretación de las componentes.

PCA se usa con frecuencia para visualizar datos de muchas variables, explorar estructuras, resumir información, reducir redundancia, comprimir representaciones, preparar datos para otros modelos o detectar casos inusuales. Su utilidad depende del objetivo. Puede aportar claridad cuando las variables contienen patrones compartidos, pero también puede dificultar la interpretación si se usa sin cuidado. Por eso, PCA debe aplicarse como una herramienta de análisis, evaluando siempre qué información conserva, qué información pierde y cómo se interpretan las componentes resultantes.

Visualización de datos

Uno de los usos más habituales de PCA es facilitar la visualización de datasets con muchas variables. Cuando tenemos más de tres variables, no podemos representar directamente todos los datos en un gráfico común. PCA permite construir una versión reducida, generalmente de dos componentes, que podemos graficar en un plano.

La idea es transformar muchas columnas originales en dos nuevas dimensiones:

El proceso parte de Variables originales, continúa con PCA, luego Componente principal 1, luego Componente principal 2 y conduce finalmente a gráfico en 2D.

Ese gráfico no muestra dos variables originales. No muestra, por ejemplo, **edad** en un eje y **ingresos** en otro. Muestra dos componentes principales: dos dimensiones construidas a partir de combinaciones de las variables originales.

Esto es muy importante para interpretar correctamente la visualización. Si el eje horizontal se llama “Componente principal 1”, no debemos leerlo como una variable concreta. Representa la dirección de mayor variación encontrada por PCA. El eje vertical, “Componente principal 2”, representa otra dirección de variación.

Concepto	Descripción
Eje horizontal	primera componente principal.
Eje vertical	segunda componente principal.

Al graficar los datos en ese nuevo plano, cada punto sigue representando una fila del dataset. Lo que cambia es el sistema de coordenadas. En lugar de ubicar cada punto según sus variables originales, lo ubicamos según sus valores en las componentes principales.

Por ejemplo, si tenemos un dataset de clientes con diez variables de comportamiento, PCA puede reducir esas diez variables a dos componentes. Luego podemos graficar cada cliente como un punto.

El proceso parte de Cliente, continúa con 10 variables originales, luego 2 componentes principales y conduce finalmente a punto en un gráfico.

Esta visualización puede ayudarnos a observar patrones generales. Tal vez veamos zonas con muchos puntos concentrados, puntos aislados, transiciones graduales o posibles grupos. Es una forma de mirar datos complejos de manera más accesible.

Gráfico PCA: permite observar estructura general, concentraciones, separaciones o casos aislados.

Si además tenemos etiquetas conocidas, podemos colorear los puntos según la clase. Por ejemplo, si trabajamos con flores y conocemos la especie de cada una, podemos reducir las mediciones con PCA y colorear los puntos según la especie. Si las especies aparecen separadas, eso sugiere que las variables originales contienen información útil para diferenciarlas.

PCA reduce dimensiones.

El color puede mostrar una etiqueta conocida, si existe.

Es importante notar que, en ese caso, PCA no usa la etiqueta para construir las componentes. Si aplicamos PCA sobre **X**, la técnica mira solo las variables

predictoras. Los colores se agregan después, para interpretar visualmente cómo se distribuyen las clases en el espacio reducido.

PCA trabaja con `X`. La clase `y` puede utilizarse después únicamente para colorear el gráfico.

Esto hace que PCA sea útil como herramienta exploratoria antes de entrenar un modelo supervisado. Si las clases aparecen bastante separadas en las primeras componentes, puede ser una señal de que hay patrones distinguibles. Si aparecen muy mezcladas, quizá el problema sea más difícil o quizá las dos primeras componentes no estén mostrando la información más relevante para separar las clases.

Pero debemos ser prudentes. Un gráfico en dos dimensiones no contiene toda la información del dataset. Si las dos primeras componentes explican una parte alta de la varianza, el gráfico será más representativo. Si explican poca varianza, la visualización será más parcial.

Antes de interpretar un gráfico PCA, conviene revisar cuánta varianza explican las componentes graficadas.

Por ejemplo, si las dos primeras componentes explican el 85% de la varianza, el gráfico conserva una parte importante de la estructura general. Si explican el 30%, el gráfico puede seguir siendo útil, pero deja afuera mucha información. En ese caso, una aparente mezcla o separación debe interpretarse con más cuidado.

En `scikit-learn`, podemos obtener una representación en dos componentes de esta manera:

```
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

X_escalado = StandardScaler().fit_transform(X)

pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_escalado)
```

Luego podemos construir un gráfico de dispersión usando esas dos componentes.

```
import matplotlib.pyplot as plt

plt.scatter(X_pca[:, 0], X_pca[:, 1])
plt.xlabel("Componente principal 1")
plt.ylabel("Componente principal 2")
plt.title("Visualización con PCA")
plt.show()
```

Si tenemos una variable de clase, podemos usarla para colorear los puntos. Esto no cambia PCA; solo modifica la forma de visualizar los resultados.

```
plt.scatter(X_pca[:, 0], X_pca[:, 1], c=y)
plt.xlabel("Componente principal 1")
plt.ylabel("Componente principal 2")
plt.title("PCA coloreado por clase")
plt.show()
```

En un cuaderno práctico, este tipo de gráfico puede ser muy útil para discutir si las clases parecen separarse o superponerse. Pero no debemos confundir una separación visual con una evaluación formal del modelo.

Concepto	Descripción
Separación visual	pista exploratoria.
Métricas con datos no vistos	evaluación del modelo.

También podemos usar PCA para visualizar resultados de clustering. Por ejemplo, podríamos aplicar K-Means sobre datos con muchas variables y luego graficar los puntos en dos componentes principales, coloreados según el cluster asignado.

El proceso parte de Datos con muchas variables, continúa con clustering, luego PCA para visualizar y conduce finalmente a puntos coloreados por cluster.

Esta estrategia puede ayudar a comunicar los grupos encontrados. Sin embargo, también requiere cuidado. Si el clustering se hizo en el espacio original y la visualización en dos componentes, puede ocurrir que algunos clusters parezcan superpuestos en el gráfico aunque estén separados en dimensiones que no se ven. También puede ocurrir lo contrario: una separación visual puede parecer más fuerte de lo que realmente es en el espacio completo.

Un gráfico PCA es una proyección y, como toda proyección, puede ocultar o simplificar parte de la estructura original.

Otra posibilidad es aplicar primero PCA y luego clustering sobre las componentes. En ese caso, los clusters sí se forman en el espacio reducido. Esta decisión puede ser útil, pero cambia el análisis: estamos agrupando sobre un resumen de los datos, no sobre todas las variables originales.

Por eso, al comunicar visualizaciones con PCA, conviene aclarar qué se hizo:

Para interpretar el resultado conviene preguntarse: ¿PCA se usó solo para graficar? ¿O PCA se usó antes de entrenar otro modelo? ¿Los clusters se calcularon en el espacio original o en el espacio reducido?

Estas diferencias son importantes para interpretar correctamente el resultado.

PCA también puede ayudar a detectar casos inusuales. En un gráfico de las dos primeras componentes, algunos puntos pueden aparecer muy alejados del resto. Esos casos pueden ser outliers, errores de carga, situaciones especiales o simplemente observaciones poco frecuentes.

Puntos alejados en el gráfico pueden ser casos que conviene revisar.

Pero, nuevamente, el gráfico solo muestra una parte de la información. Un punto que parece normal en las dos primeras componentes podría ser raro en otra dimensión. Y un punto que parece aislado en el gráfico podría no ser problemático al revisar las variables originales. La visualización ayuda a detectar preguntas, no a cerrar conclusiones.

Un gráfico PCA sirve para explorar, no para obtener conclusiones sin un análisis adicional.

La escala de las variables también influye en la visualización. Si aplicamos PCA sin escalar datos que están en unidades muy distintas, el gráfico puede quedar dominado por las variables con mayor varianza numérica. En ese caso, la visualización puede reflejar más las escalas originales que una estructura interesante.

Por eso, en muchos casos el flujo recomendable es:

El proceso parte de seleccionar variables numéricas, continúa con escalar, luego aplicar PCA, luego graficar componentes y conduce finalmente a interpretar con cuidado.

Si las variables tienen sentido en sus unidades originales y están en escalas comparables, puede haber casos donde el escalado no sea indispensable. Pero cuando mezclamos montos, edades, cantidades, porcentajes o tiempos, escalar suele ser una decisión prudente.

También debemos revisar la varianza explicada antes de mostrar o interpretar un gráfico. Esto puede incluir una frase simple en el análisis:

Las dos primeras componentes explican el 72% de la varianza total.

Esa información le da al lector una idea de cuán representativa es la visualización. No la vuelve perfecta, pero ayuda a ponerla en contexto.

PCA es muy útil para visualizar datos con muchas variables, reduciéndolos a dos componentes principales que pueden graficarse en un plano. Esta visualización permite explorar concentraciones, separaciones, grupos y casos inusuales. También puede combinarse con colores para mostrar clases conocidas o clusters asignados. Sin embargo, un gráfico PCA es una representación reducida: no muestra toda la información original y no debe interpretarse como prueba definitiva. Para usarlo correctamente, debemos revisar la varianza explicada, cuidar el escalado y complementar la visualización con análisis de las variables originales.

Limitaciones interpretativas

PCA puede ser una herramienta muy útil para resumir datos, visualizar estructuras y reducir dimensionalidad. Sin embargo, sus resultados no siempre son fáciles de interpretar. Esta es una de sus limitaciones más importantes: al transformar las variables originales en componentes principales, ganamos una representación más compacta, pero podemos perder claridad conceptual.

Las variables originales suelen tener nombres y unidades comprensibles. Podemos interpretar `edad`, `ingresos`, `superficie_m2`, `cantidad_compras` o `dias_ultima_compra`. En cambio, una componente principal no corresponde directamente a una variable del mundo real. Es una combinación de varias variables originales.

Concepto	Descripción
Variables originales	nombres claros, unidades conocidas, interpretación directa.
Componentes principales	combinaciones de variables, sin significado automático.

Esto significa que un eje llamado “Componente principal 1” no tiene por sí solo una explicación sencilla. No podemos decir automáticamente que representa edad, gasto, tamaño o actividad. Para interpretarlo, debemos revisar cómo se relaciona esa componente con las variables originales.

Por ejemplo, si aplicamos PCA a variables de viviendas y la primera componente aumenta cuando aumentan superficie, cantidad de ambientes, dormitorios y baños, podríamos interpretarla como una dimensión general de tamaño de la propiedad.

Concepto	Descripción
Componente 1 alta	más superficie, más ambientes, más dormitorios, más baños.
Interpretación posible	tamaño general de la vivienda.

Pero esa interpretación es una construcción nuestra. PCA no etiqueta la componente como “tamaño”. El algoritmo solo encuentra una dirección de variación. El nombre aparece después, cuando analizamos qué variables aportan a esa dirección.

En otros casos, la interpretación puede ser mucho menos clara. Una componente puede combinar variables de manera compleja: algunas con peso positivo, otras con peso negativo, otras con aportes pequeños. Puede mezclar actividad, antigüedad, gasto, frecuencia y recencia. Darle un nombre simple puede ser difícil o incluso engañoso.

No toda componente principal tiene una interpretación clara o fácil de comunicar.

Esta dificultad aumenta cuando el dataset tiene muchas variables. Si PCA se aplica sobre cincuenta columnas, cada componente puede ser una combinación de muchas de ellas. Aunque técnicamente sea útil, explicar qué significa cada componente puede ser complejo.

En `scikit-learn`, podemos revisar los pesos de cada variable original en las componentes mediante el atributo `components_`.

```
pca.components_
```

Cada fila corresponde a una componente y cada columna corresponde a una variable original. Estos valores ayudan a ver qué variables participan más en cada componente y en qué dirección.

Una forma habitual de analizarlos es construir una tabla que relacione variables originales con componentes.

```
import pandas as pd

pesos = pd.DataFrame(
    pca.components_,
    columns=columnas_originales,
    index=["PC1", "PC2"]
)

pesos
```

Esta tabla puede mostrar, por ejemplo, que `PC1` tiene pesos altos en variables de gasto y frecuencia, mientras que `PC2` tiene más relación con antigüedad o recencia. A partir de eso podemos proponer interpretaciones.

Los pesos de las componentes ayudan a interpretar, pero no siempre producen una explicación simple.

Otra limitación interpretativa aparece cuando usamos escalado. Muchas veces estandarizamos las variables antes de PCA, lo cual suele ser correcto porque PCA es sensible a la escala. Pero, una vez escalados los datos, las componentes ya no se expresan en las unidades originales.

Si antes teníamos pesos, años, metros o cantidades, después del escalado y PCA trabajamos con coordenadas transformadas. Esto mejora el cálculo, pero puede dificultar la comunicación.

PCA con datos escalados: mejor comparación entre variables, pero menor interpretación directa en unidades originales.

Por eso, cuando comunicamos resultados basados en PCA, conviene volver a las variables originales para explicar qué está ocurriendo. Si un grupo de puntos aparece separado en un gráfico PCA, debemos revisar qué características tienen esos casos en la tabla original.

No alcanza con decir que “están más a la derecha en la componente 1”. Debemos analizar qué significa estar más a la derecha en términos de variables comprensibles.

La visualización con PCA debe conectarse después con las variables originales.

También debemos tener cuidado con las conclusiones visuales. Un gráfico de dos componentes puede mostrar grupos, separaciones o casos aislados, pero ese gráfico es una proyección reducida. No contiene toda la información original.

Si las dos primeras componentes explican una proporción alta de varianza, la visualización puede ser bastante representativa. Si explican poca varianza, el gráfico deja afuera mucha información. En ese caso, una separación visual puede ser parcial, y una superposición visual puede no significar que los casos sean realmente indistinguibles en el espacio completo.

Un gráfico PCA muestra una versión resumida de los datos.

No muestra todo el dataset original.

Esto es especialmente importante cuando usamos PCA para apoyar una explicación. Un gráfico puede ser muy convincente visualmente, pero debemos evitar presentar una proyección como si fuera la estructura completa de los datos.

Por ejemplo, si dos clases aparecen separadas en un gráfico PCA, podemos decir que en esa representación reducida se observa cierta separación. No deberíamos decir, sin más, que las clases están perfectamente separadas en el problema original. Para eso necesitaríamos evaluación adicional.

Concepto	Descripción
Más responsable	"En las dos primeras componentes se observa una separación parcial."
Menos responsable	"Las clases están separadas."

Otra limitación es que PCA captura principalmente relaciones lineales. Si la estructura importante de los datos es no lineal, PCA puede no representarla bien. Puede reducir dimensiones, pero no necesariamente conservar la forma más relevante del problema.

PCA busca direcciones lineales de variación.

No está diseñado para capturar toda estructura no lineal posible.

Esto no significa que PCA sea malo. Significa que tiene una forma particular de resumir. Si el patrón de los datos se adapta a esa forma, puede ser muy útil. Si no, puede producir una representación limitada.

También debemos recordar que PCA conserva variación, no significado. Una componente puede explicar mucha varianza porque captura una diferencia grande entre casos, pero esa diferencia no siempre es la más relevante para nuestra pregunta.

Por ejemplo, en datos de clientes, la mayor variación puede estar relacionada con el monto de compra. Pero si nuestro interés es detectar abandono, quizá una variación menor relacionada con recencia o cambios de comportamiento sea más importante.

Mucha varianza explicada no significa necesariamente mayor relevancia para el objetivo.

Esto es clave cuando PCA se usa antes de un modelo supervisado. Como PCA no mira y , no sabe qué componentes son más predictivas. Solo sabe qué componentes conservan más variación en X .

Por eso, si usamos PCA dentro de un flujo predictivo, debemos evaluar el desempeño del pipeline completo con datos no vistos. No alcanza con elegir componentes porque explican mucha varianza.

PCA resume X .

La evaluación debe decir si ese resumen ayuda al objetivo predictivo.

Otra limitación interpretativa aparece cuando se intenta nombrar las componentes con demasiada confianza. A veces una componente parece asociada a una idea general, pero esa idea no debe exagerarse. Si una componente combina variables de

frecuencia, gasto y antigüedad, tal vez podamos decir que está relacionada con “actividad comercial”, pero debemos aclarar que es una interpretación aproximada.

Asignar un nombre a una componente es una interpretación humana, no un resultado automático del algoritmo.

También puede haber componentes que no convenga nombrar. Si su combinación de variables es difícil de explicar, quizá sea más honesto describir qué variables pesan más en ella en lugar de forzar una etiqueta conceptual.

En informes o comunicaciones, PCA debe presentarse con un lenguaje prudente. Conviene explicar que se trata de una técnica de reducción de dimensionalidad, que las componentes son combinaciones de variables, que la visualización conserva cierta proporción de varianza y que las interpretaciones se apoyan en el análisis posterior.

Comunicación responsable: componentes como resumen, no como variables naturales.

También debemos evitar que PCA oculte decisiones importantes. La selección de variables, el escalado, el tratamiento de faltantes y la cantidad de componentes elegida influyen en el resultado. Dos análisis PCA pueden producir representaciones distintas si se aplican sobre conjuntos de variables diferentes.

Cambian las variables, el escalado o la cantidad de componentes, y puede cambiar la interpretación.

Por eso, al usar PCA conviene documentar el flujo: qué variables se incluyeron, cómo se preprocesaron, cuántas componentes se conservaron y cuánta varianza explican. Esa documentación permite que el análisis sea revisable y no parezca una transformación misteriosa.

PCA reduce dimensiones construyendo componentes principales a partir de combinaciones de variables originales. Esto puede facilitar la visualización y el resumen de datos complejos, pero también dificulta la interpretación directa. Las componentes no tienen significado automático, no conservan unidades originales y pueden mezclar muchas variables. Además, una visualización reducida no muestra toda la información del dataset. Por eso, los resultados de PCA deben interpretarse con prudencia, conectarse con las variables originales y comunicarse como representaciones resumidas, no como explicaciones definitivas.

Reducir dimensiones para mirar mejor

En este capítulo estudiamos PCA, una técnica clásica de reducción de dimensionalidad. Su objetivo es construir una representación más compacta de datos que tienen muchas variables, intentando conservar una parte importante de la información original.

La idea de dimensionalidad se relaciona con la cantidad de variables que usamos para describir cada caso. Con dos variables podemos graficar puntos en un plano. Con tres, podemos imaginar un espacio. Pero con muchas variables, la visualización directa se vuelve difícil.

El proceso parte de Muchas variables, continúa con datos difíciles de visualizar, luego reducción de dimensionalidad y conduce finalmente a representación más compacta.

PCA no selecciona simplemente algunas columnas del dataset. En lugar de elegir variables originales, construye nuevas variables llamadas componentes principales. Estas componentes son combinaciones de las variables originales y están ordenadas según la cantidad de varianza que explican.

PCA transforma las variables originales en componentes principales.

La primera componente principal captura la mayor variación posible en los datos. La segunda captura otra parte de la variación restante. Las siguientes componentes continúan ese proceso. Al conservar solo algunas de ellas, podemos reducir la cantidad de dimensiones.

Vimos que PCA puede ser especialmente útil cuando hay variables correlacionadas. Si varias columnas se mueven juntas o representan aspectos relacionados de un mismo fenómeno, puede existir información parcialmente redundante. PCA puede resumir esa variación compartida en menos componentes.

El proceso parte de Variables correlacionadas, continúa con información parcialmente redundante, luego componentes que resumen y conduce finalmente a patrones de variación.

También analizamos la varianza explicada. Esta medida indica qué proporción de la variación original conserva cada componente. Si las dos primeras componentes explican, por ejemplo, el 80% de la varianza, significa que una representación en dos dimensiones conserva una parte importante de la estructura del dataset, aunque no toda.

Varianza explicada: cuánta variación conserva cada componente principal.

La varianza acumulada ayuda a decidir cuántas componentes conservar. Sin embargo, explicar mucha varianza no significa necesariamente conservar lo más importante para una decisión. PCA mira solo X : no sabe cuál es la variable objetivo ni qué información será más útil para predecir.

Uno de los usos más comunes de PCA es la visualización. Al reducir los datos a dos componentes, podemos construir un gráfico en el plano. Ese gráfico puede mostrar concentraciones, separaciones, posibles grupos o casos aislados.

El proceso parte de Datos con muchas variables, continúa con dos componentes principales y conduce finalmente a gráfico exploratorio.

Pero una visualización con PCA debe interpretarse con prudencia. Los ejes no son variables originales, sino componentes construidas. Además, el gráfico muestra una proyección reducida de los datos. Si las primeras componentes explican poca varianza, la visualización deja fuera mucha información.

También vimos que PCA puede usarse para resumir datos, reducir redundancia, preparar información para otros modelos, explorar posibles clusters o detectar casos inusuales. En todos los casos, su utilidad depende del objetivo del análisis y de una interpretación cuidadosa.

PCA puede ayudar a: visualizar, resumir, explorar, reducir redundancia.

Una condición importante es el escalado. PCA es sensible a la escala de las variables. Si una columna tiene valores mucho más grandes que otra, puede dominar las componentes principales. Por eso, en muchos datasets conviene aplicar `StandardScaler` antes de PCA.

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

pipeline = Pipeline(steps=[
    ("scaler", StandardScaler()),
    ("pca", PCA(n_components=2))
])
```

Cuando PCA forma parte de un flujo supervisado, también debe integrarse correctamente para evitar fuga de datos. El escalado y PCA deben ajustarse solo con los datos de entrenamiento dentro de cada evaluación. Por eso, el uso de pipelines vuelve a ser una buena práctica.

La principal limitación interpretativa de PCA es que las componentes no tienen significado automático. Una componente puede estar asociada a tamaño, actividad,

consumo o intensidad, pero esa interpretación debe construirse revisando cómo se relaciona con las variables originales.

Las componentes principales son resúmenes matemáticos, no variables naturales con un nombre propio.

Por eso, no conviene presentar un gráfico PCA o una componente principal como una explicación definitiva. PCA ayuda a mirar mejor datos complejos, pero no reemplaza el análisis contextual. Sus resultados dependen de las variables elegidas, del escalado, de la cantidad de componentes conservadas y de la estructura de los datos.

PCA permite reducir dimensiones construyendo componentes principales que capturan direcciones importantes de variación. Puede ser muy útil para visualizar y resumir datasets con muchas variables, especialmente cuando existen correlaciones. Sin embargo, reducir dimensiones implica perder información y puede dificultar la interpretación directa. PCA no descubre causas ni significados por sí solo: ofrece una representación compacta que debe analizarse, evaluarse y comunicarse con cuidado.

Capítulo 28 · Interpretar modelos

¿Qué significa interpretar un modelo?

Entrenar un modelo y obtener una métrica aceptable no siempre alcanza. En muchos problemas también necesitamos entender, al menos en parte, cómo llega el modelo a sus predicciones, qué variables parecen influir más, qué patrones está usando y qué límites tienen esas interpretaciones. A eso nos referimos cuando hablamos de interpretar modelos.

Interpretar un modelo significa intentar explicar su comportamiento. No necesariamente significa conocer cada detalle interno del algoritmo, ni convertirlo en una regla perfecta y sencilla. Significa construir una comprensión razonable de cómo el modelo relaciona las variables de entrada con las predicciones que produce.

Interpretar un modelo: entender qué señales usa, cómo influyen las variables y qué tan confiables son esas explicaciones.

Por ejemplo, si entrenamos un modelo para predecir abandono de clientes, no solo podría interesarnos saber si obtiene buen recall o buen F1-score. También podríamos querer saber qué factores están asociados con mayor riesgo de abandono: poca actividad reciente, baja frecuencia de compra, muchos reclamos, menor antigüedad o cambios en el uso del servicio.

Concepto	Descripción
Predicción	este cliente puede abandonar.
Interpretación	¿qué variables parecen influir en esa predicción?

Esta diferencia es importante. Una predicción responde qué espera el modelo para un caso. Una interpretación intenta explicar por qué el modelo produjo esa predicción o qué patrones generales aprendió.

En algunos contextos, la interpretación es necesaria para confiar en el resultado. Si un modelo recomienda revisar ciertas transacciones como posibles fraudes, una persona puede necesitar entender qué señales motivaron esa alerta. Si un modelo estima riesgo en un contexto educativo, médico, financiero o institucional, las explicaciones se vuelven todavía más importantes.

Cuanto más sensible es la decisión, más importante es entender cómo se usa el modelo.

Interpretar no significa justificar cualquier resultado. Un modelo puede producir una predicción y aun así estar usando señales incorrectas, sesgadas o filtradas. Por eso, la interpretación también sirve para auditar. Nos ayuda a detectar si el modelo está aprendiendo algo razonable o si está aprovechando atajos problemáticos.

Por ejemplo, si un modelo para predecir aprobación de un examen usa una variable que solo se conoce después del examen, tendríamos fuga de datos. La métrica podría parecer excelente, pero la interpretación de variables revelaría que el modelo está usando información que no debería estar disponible al momento de predecir.

Una buena métrica no garantiza un buen modelo.

Interpretar ayuda a revisar qué está aprendiendo.

También puede ocurrir que una variable aparezca como muy importante, pero en realidad esté funcionando como sustituto de otra información sensible o problemática. Por ejemplo, una variable geográfica podría estar asociada a diferencias socioeconómicas. Una variable de comportamiento digital podría reflejar acceso desigual a conectividad. Interpretar exige mirar más allá del algoritmo.

Esto conecta con una idea que veremos con más detalle en el capítulo siguiente: los modelos no aprenden en el vacío. Aprenden desde datos que fueron producidos en un contexto social, técnico e institucional. Por eso, sus patrones deben analizarse con responsabilidad.

Interpretar modelos no es solo mirar números.

También es preguntarse si esos números tienen sentido en el contexto.

Hay distintos niveles de interpretación. A veces queremos entender el comportamiento global del modelo: qué variables parecen ser más importantes en general, qué relaciones aprendió, qué patrones predominan. Otras veces queremos interpretar una predicción individual: por qué este caso particular recibió cierta clase o cierto valor.

Concepto	Descripción
Interpretación global	cómo se comporta el modelo en conjunto.
Interpretación local	por qué produjo una predicción particular.

Por ejemplo, en un modelo de precios de viviendas, una interpretación global podría decir que la superficie, la ubicación y la cantidad de ambientes son variables importantes. Una interpretación local podría explicar por qué una vivienda específica

recibió una predicción alta: tiene gran superficie, está en una zona de alto valor y posee varias habitaciones.

Ambos niveles son útiles, pero responden preguntas distintas. La interpretación global ayuda a comprender el modelo como sistema. La interpretación local ayuda a revisar casos concretos.

También debemos diferenciar interpretación de causalidad. Si un modelo aprende que cierta variable está asociada con una predicción, eso no significa automáticamente que esa variable cause el resultado. Un modelo predictivo puede usar correlaciones sin demostrar causas.

Concepto	Descripción
Interpretación de modelo	qué señales usa para predecir.
Causalidad	qué factores producen realmente un efecto.

Esta distinción es fundamental. Si un modelo encuentra que los clientes que llaman mucho al soporte abandonan más, eso no significa necesariamente que llamar al soporte cause abandono. Puede ser que los clientes llamen porque ya tienen problemas con el servicio, y esos problemas sean la verdadera causa del abandono.

El modelo puede usar la cantidad de reclamos como señal predictiva, pero la interpretación causal requiere otro tipo de análisis.

Una variable puede ser útil para predecir sin ser la causa del fenómeno.

Interpretar modelos también implica reconocer sus límites. Algunos modelos son más fáciles de interpretar que otros. Una regresión lineal o una regresión logística permiten observar coeficientes. Un árbol pequeño puede seguirse mediante reglas. En cambio, modelos como Random Forest, Gradient Boosting o SVM con kernels pueden ser más difíciles de explicar directamente.

Algunos modelos son más transparentes.

Otros son más potentes, pero menos fáciles de explicar.

A veces existe un equilibrio entre desempeño e interpretabilidad. Un modelo más complejo puede obtener mejores métricas, pero ser más difícil de comunicar. Un modelo más simple puede rendir un poco menos, pero ser más comprensible y suficiente para el objetivo.

No siempre necesitamos el modelo más interpretable. Tampoco siempre necesitamos el modelo más complejo. La decisión depende del problema, del riesgo de los errores, de las personas que usarán el resultado y de la necesidad de justificar decisiones.

Elegir un modelo también implica considerar qué tan explicable debe ser.

Por ejemplo, si usamos un modelo para organizar una lista interna de productos recomendados, quizá aceptemos mayor complejidad. Si lo usamos para tomar decisiones que afectan oportunidades de personas, la exigencia de interpretación y revisión debería ser mayor.

Interpretar también ayuda a comunicar resultados. Un informe de Machine Learning no debería limitarse a decir “el modelo obtuvo 0.87 de F1-score”. Conviene explicar qué variables parecen relevantes, qué errores se observaron, qué casos son difíciles y qué limitaciones tiene el análisis.

Comunicar un modelo: métricas, variables relevantes, errores, limitaciones, contexto.

Sin embargo, debemos evitar interpretar de más. No toda técnica de interpretación ofrece una verdad absoluta. La importancia de variables puede cambiar según el modelo, el dataset, el preprocesamiento y la métrica usada. Una explicación local puede ser aproximada. Un gráfico puede sugerir una relación, pero no demostrar una causa.

La interpretación debe ser útil, pero prudente.

Interpretar no es inventar certezas. Es construir explicaciones razonables con límites claros.

En este capítulo estudiaremos distintas formas de interpretar modelos. Veremos la diferencia entre modelos transparentes y modelos opacos, cómo leer coeficientes, cómo pensar la importancia de variables, qué aporta la permutation importance, y qué diferencia hay entre interpretaciones locales y globales. También dedicaremos atención a los riesgos de sobreinterpretar.

Interpretar un modelo significa analizar cómo se comporta, qué variables usa, qué patrones parece haber aprendido y qué límites tienen esas explicaciones. La interpretación ayuda a confiar, auditar, comunicar y mejorar modelos. Pero debe hacerse con cuidado: una explicación de modelo no es necesariamente una explicación causal, ni una garantía de justicia, ni una verdad definitiva sobre el fenómeno. Es una herramienta para entender mejor el modelo y usarlo de manera más responsable.

Modelos transparentes y modelos opacos

No todos los modelos son igual de fáciles de interpretar. Algunos permiten observar con bastante claridad cómo relacionan las variables de entrada con la predicción.

Otros pueden producir muy buenos resultados, pero su funcionamiento interno es más difícil de explicar. Por eso suele hablarse de modelos transparentes y modelos opacos.

Un modelo transparente es aquel cuya lógica puede entenderse de manera relativamente directa. Esto no significa que sea trivial, ni que no requiera conocimiento técnico, pero sí que sus decisiones pueden examinarse con cierta claridad.

Modelo transparente: su funcionamiento puede explicarse de forma relativamente directa.

La regresión lineal es un ejemplo clásico. En una regresión lineal, cada variable tiene un coeficiente. Ese coeficiente indica cómo cambia la predicción cuando aumenta esa variable, manteniendo las demás constantes. Si el modelo está bien planteado, podemos analizar esos coeficientes y construir una explicación razonable.

También ocurre algo parecido con la regresión logística. Aunque se use para clasificación, permite observar coeficientes asociados a las variables. Esos coeficientes ayudan a entender qué variables empujan la predicción hacia una clase u otra, siempre con cuidado en la interpretación.

Concepto	Descripción
Regresión lineal	coeficientes asociados a cambios en un valor numérico.
Regresión logística	coeficientes asociados a la probabilidad de una clase.

Un árbol de decisión pequeño también puede ser bastante transparente. Podemos seguir el camino desde la raíz hasta una hoja y observar qué preguntas hizo el modelo. Por ejemplo: si la antigüedad del cliente es menor que cierto valor, si tuvo reclamos recientes, si bajó su frecuencia de compra, y así sucesivamente.

Árbol pequeño: pregunta por una variable, divide los datos, vuelve a preguntar, llega a una decisión.

Esa estructura permite explicar una predicción como una secuencia de condiciones. Por eso, los árboles pequeños suelen ser útiles cuando necesitamos reglas comprensibles. Sin embargo, si el árbol crece demasiado, también puede volverse difícil de interpretar. La transparencia no depende solo del tipo de modelo, sino también de su tamaño y complejidad.

Un árbol pequeño puede ser claro.

Un árbol muy profundo puede ser difícil de seguir.

En el otro extremo aparecen modelos más opacos. Un modelo opaco, a veces llamado “caja negra”, es aquel cuyo comportamiento no puede explicarse fácilmente mirando directamente su estructura interna. Puede ser muy preciso, pero menos transparente.

Modelo opaco: puede predecir bien, pero su lógica interna no es fácil de explicar directamente.

Random Forest, por ejemplo, combina muchos árboles. Cada árbol puede tomar decisiones distintas y el resultado final surge de una votación o promedio. Aunque podamos analizar el modelo de distintas maneras, no es práctico explicar una predicción siguiendo manualmente cientos de árboles.

Gradient Boosting también puede ser difícil de interpretar directamente. Su predicción se construye como una secuencia de modelos que corrigen errores anteriores. El resultado puede ser muy potente, pero la explicación interna completa no suele ser sencilla.

Concepto	Descripción
Random Forest	muchos árboles votando o promediando.
Gradient Boosting	muchos modelos pequeños corrigiendo errores paso a paso.

SVM con kernels no lineales también puede resultar opaco. Aunque entendamos la idea del margen y los vectores de soporte, la frontera de decisión puede ser compleja. Explicar por qué un caso quedó de un lado u otro no siempre es directo.

Lo mismo ocurre con otros modelos más avanzados. A medida que aumenta la flexibilidad del modelo, muchas veces aumenta también la dificultad para explicar su comportamiento en términos simples.

Más flexibilidad puede traer mayor capacidad predictiva, pero menor interpretabilidad directa.

Esto no significa que los modelos opacos sean malos. Muchas veces son necesarios o convenientes. Si el objetivo principal es lograr el mejor desempeño posible en un problema complejo, un modelo opaco puede ser una buena elección. El problema aparece cuando usamos modelos difíciles de explicar en contextos donde la explicación es importante.

Por ejemplo, si un modelo se usa para priorizar campañas publicitarias, quizá la opacidad sea aceptable si el impacto de los errores es bajo y se monitorean los resultados. Pero si el modelo influye en decisiones educativas, financieras, médicas o legales, la necesidad de interpretación aumenta.

La exigencia de interpretación depende del riesgo y del uso del modelo.

También debemos evitar una idea demasiado simple: no siempre hay que elegir entre precisión e interpretabilidad como si fueran opuestos absolutos. A veces un modelo simple rinde casi tan bien como uno complejo. A veces un modelo complejo mejora mucho. A veces una pequeña pérdida de desempeño puede valer la pena si ganamos claridad, confianza y facilidad de comunicación.

La elección del modelo debe equilibrar: desempeño, interpretabilidad, costo, riesgo y contexto.

Supongamos que entrenamos dos modelos para predecir abandono de clientes. Una regresión logística obtiene un F1-score de 0.81. Un Gradient Boosting obtiene 0.83. La mejora existe, pero no es enorme. Si el equipo necesita entender y comunicar claramente los factores de abandono, quizá la regresión logística sea una opción suficiente y más conveniente.

En cambio, si el Gradient Boosting obtiene 0.90 y la mejora permite intervenir mucho mejor sobre clientes en riesgo, tal vez valga la pena aceptar mayor complejidad, siempre acompañada de técnicas de interpretación y monitoreo.

Un modelo más complejo debe justificar su complejidad con una mejora útil para el problema.

La transparencia también ayuda a detectar errores. Si un modelo interpretable muestra que una variable inesperada tiene mucho peso, podemos investigar. Tal vez haya fuga de datos. Tal vez la variable esté mal calculada. Tal vez represente un sesgo. En un modelo muy opaco, esos problemas pueden ser más difíciles de notar.

Pero tampoco debemos confiar ciegamente en los modelos transparentes. Que un modelo sea fácil de leer no significa que sea correcto, justo o causal. Una regresión logística puede usar variables problemáticas. Un árbol pequeño puede aprender una regla espuria. La interpretabilidad facilita la revisión, pero no reemplaza el criterio.

Transparente no significa automáticamente correcto o justo.

Los modelos opacos, por su parte, pueden analizarse mediante herramientas de interpretación. Podemos estudiar importancia de variables, permutation importance, gráficos parciales, explicaciones locales u otros métodos. Estas herramientas no vuelven al modelo completamente transparente, pero ayudan a comprender su comportamiento.

Un modelo opaco puede interpretarse parcialmente con herramientas adicionales.

Este punto es importante. La interpretabilidad no es una propiedad única y fija. Hay modelos naturalmente más transparentes, pero también existen técnicas para aproximarnos al comportamiento de modelos más complejos. Lo que cambia es el grado de esfuerzo, precisión y prudencia necesarios.

También debemos considerar a quién va dirigida la explicación. Una explicación técnica para un equipo de ciencia de datos puede incluir coeficientes, importancia de variables o curvas. Una explicación para una persona usuaria del sistema debe ser más clara, concreta y relacionada con el caso. Una explicación para una decisión institucional puede requerir documentación, validación y revisión ética.

No existe una única explicación.

La explicación adecuada depende de quién la necesita y para qué decisión.

Por ejemplo, decir “la variable X tiene alta importancia por permutation importance” puede ser útil para un equipo técnico. Pero para un informe general quizás convenga decir: “el modelo parece apoyarse principalmente en la actividad reciente, la antigüedad y la frecuencia de uso”. Para una persona afectada por una predicción, tal vez se necesite una explicación todavía más específica y cuidadosa.

La diferencia entre modelos transparentes y opacos, entonces, no debe verse como una clasificación rígida. Es más bien un continuo. Algunos modelos son muy fáciles de interpretar. Otros son parcialmente interpretables. Otros requieren herramientas externas. Otros pueden ser tan complejos que solo entendemos ciertos aspectos de su comportamiento.

Interpretabilidad: no es todo o nada.

Hay distintos grados de transparencia.

Los modelos transparentes permiten examinar su lógica de manera más directa, como ocurre con regresiones o árboles pequeños. Los modelos opacos pueden ser más difíciles de explicar, como Random Forest, Gradient Boosting o SVM con kernels, pero a veces ofrecen mejor desempeño. La elección entre modelos debe considerar no solo métricas, sino también el riesgo del problema, la necesidad de explicación, el público destinatario y la posibilidad de auditar el comportamiento del modelo. Interpretar no es un lujo: en muchos proyectos, es parte de usar Machine Learning de manera responsable.

Coeficientes

En algunos modelos, una de las formas más directas de interpretación aparece a través de los coeficientes. Los coeficientes son valores que indican cómo participa cada variable en la predicción del modelo. Son especialmente importantes en modelos lineales, como la regresión lineal y la regresión logística.

En una regresión lineal, el modelo construye una predicción sumando el aporte de distintas variables. Cada variable tiene asociado un coeficiente. Ese coeficiente indica cuánto cambia la predicción cuando esa variable aumenta una unidad, manteniendo las demás constantes.

El proceso parte de Variable, continúa con coeficiente y conduce finalmente a aporte a la predicción.

Por ejemplo, si entrenamos un modelo para predecir el precio de una vivienda, podríamos tener variables como superficie, cantidad de ambientes y antigüedad. Si el coeficiente de `superficie_m2` es positivo, el modelo está asociando una mayor superficie con un precio más alto.

Concepto	Descripción
Coeficiente positivo	cuando la variable aumenta, la predicción tiende a aumentar.
Coeficiente negativo	cuando la variable aumenta, la predicción tiende a disminuir.

Supongamos un ejemplo simplificado:

`precio_estimado = 50000 + 1200 * superficie_m2.`

En este caso, el coeficiente de `superficie_m2` es 1200. Eso significa que, según este modelo, por cada metro cuadrado adicional, el precio estimado aumenta en 1200, manteniendo todo lo demás constante.

`p><p>Coeficiente de superficie_m2 = 1200</p>`

`p><p>Interpretación: un metro cuadrado adicional se asocia con 1200 más en la predicción del modelo.`

La expresión “manteniendo todo lo demás constante” es importante. En un modelo con varias variables, el coeficiente intenta describir el efecto de una variable cuando las demás no cambian. En la práctica, eso puede ser una simplificación fuerte, porque las variables muchas veces están relacionadas entre sí.

Por ejemplo, la superficie y la cantidad de ambientes suelen estar correlacionadas. Las viviendas más grandes tienden a tener más ambientes. Entonces, interpretar el

coeficiente de una variable aislada puede ser más difícil cuando las variables están conectadas.

Coeficientes: útiles para interpretar, pero sensibles a la relación entre variables.

En regresión logística también existen coeficientes, pero su interpretación es menos directa que en regresión lineal. La regresión logística se usa para clasificación y estima probabilidades. Sus coeficientes indican cómo las variables empujan la predicción hacia una clase u otra, pero no representan un cambio directo en la probabilidad en unidades simples.

Aun así, el signo del coeficiente sigue siendo informativo. Un coeficiente positivo indica que, cuando esa variable aumenta, el modelo tiende a aumentar la evidencia a favor de la clase positiva. Un coeficiente negativo indica que la variable tiende a alejar la predicción de esa clase.

El proceso parte de Regresión logística: coeficiente positivo, continúa con más evidencia hacia la clase positiva, luego coeficiente negativo y conduce finalmente a menos evidencia hacia la clase positiva.

Por ejemplo, si el modelo predice abandono de clientes y `dias_desde_ultima_compra` tiene un coeficiente positivo, podría indicar que a mayor cantidad de días sin comprar, mayor tendencia del modelo a predecir abandono. Si `frecuencia_compra` tiene un coeficiente negativo, podría indicar que una mayor frecuencia de compra reduce la tendencia a predecir abandono.

El proceso parte de Más días sin comprar, continúa con mayor riesgo estimado de abandono, luego Mayor frecuencia de compra y conduce finalmente a menor riesgo estimado de abandono.

Esta interpretación puede ser muy útil para comunicar patrones generales. Pero debemos recordar que estamos interpretando el comportamiento del modelo, no una verdad causal. El modelo puede aprender una asociación entre días sin compra y abandono, pero eso no prueba por sí solo que los días sin compra causen el abandono.

Un coeficiente describe cómo usa una variable el modelo.

No demuestra por sí solo una causa.

En `scikit-learn`, los coeficientes de muchos modelos lineales se encuentran en el atributo `coef_`, después de entrenar el modelo.

```
modelo.coef_
```

En una regresión lineal con varias variables, `coef_` contiene un coeficiente por variable. Si tenemos los nombres de las columnas, podemos construir una tabla para interpretarlos mejor.

```
import pandas as pd

coeficientes = pd.DataFrame({
    "variable": columnas,
    "coeficiente": modelo.coef_
})

coeficientes
```

En una regresión logística binaria, la estructura es similar, aunque `coef_` suele aparecer como una matriz. Podemos convertirla en una tabla de manera parecida.

```
coeficientes = pd.DataFrame({
    "variable": columnas,
    "coeficiente": modelo.coef_[0]
})

coeficientes.sort_values("coeficiente")
```

Ordenar los coeficientes puede ayudarnos a ver qué variables empujan más hacia una clase y cuáles empujan en sentido contrario. Pero esta comparación debe hacerse con mucho cuidado, especialmente si las variables están en escalas distintas.

Si una variable está medida en pesos y otra en unidades pequeñas, sus coeficientes no son directamente comparables. Un coeficiente pequeño en una variable con valores enormes puede tener un efecto importante. Un coeficiente grande en una variable binaria puede representar otra cosa. Por eso, la escala afecta la interpretación.

Comparar coeficientes solo tiene sentido si entendemos la escala de las variables.

Cuando las variables numéricas se estandarizan, los coeficientes pueden compararse de manera más razonable porque las variables quedan en escalas similares. Pero incluso en ese caso, debemos interpretar con prudencia.

Con variables escaladas, los coeficientes son más comparables.

Pero siguen siendo coeficientes del modelo, no explicaciones causales.

El preprocesamiento también afecta los coeficientes. Si usamos One-Hot Encoding para variables categóricas, una categoría puede transformarse en una columna binaria. El coeficiente de esa columna indica cómo cambia la predicción al pertenecer a esa categoría, en comparación con una categoría de referencia o con la codificación utilizada.

Por ejemplo, si una variable `medio_pago` se transforma en columnas como `medio_pago_tarjeta`, `medio_pago_transferencia` y `medio_pago_efectivo`, cada coeficiente se refiere a una de esas columnas transformadas, no a la variable categórica original como un todo.

Después de One-Hot Encoding, los coeficientes corresponden a columnas transformadas.

Esto puede hacer que la interpretación sea más detallada, pero también más extensa. En datasets con muchas categorías, la cantidad de coeficientes puede crecer mucho. Por eso, al interpretar modelos con variables categóricas codificadas, conviene revisar cuidadosamente cómo se construyeron las columnas.

También debemos prestar atención a variables correlacionadas. Cuando dos variables están muy relacionadas, el modelo puede repartir el peso entre ellas de una manera difícil de interpretar. A veces una variable parece tener poco coeficiente no porque no sea relevante, sino porque su información está compartida con otra.

Variables correlacionadas pueden dificultar la interpretación individual de coeficientes.

Por ejemplo, si usamos superficie, cantidad de ambientes y cantidad de dormitorios en un modelo de precio de viviendas, esas variables pueden estar muy relacionadas. El coeficiente de cada una puede depender de la presencia de las otras. Si quitamos una variable, los coeficientes de las restantes podrían cambiar.

Esto muestra que los coeficientes no deben leerse como atributos absolutos de las variables. Son parte de un modelo específico, entrenado con un conjunto específico de datos y variables.

Un coeficiente depende de: los datos, las variables incluidas, el preprocesamiento, y el modelo elegido.

La regularización también puede modificar los coeficientes. Como vimos en capítulos anteriores, Ridge y Lasso penalizan la magnitud de los coeficientes. Lasso incluso puede llevar algunos coeficientes a cero. Esto puede ayudar a controlar la complejidad, pero también afecta la interpretación.

Regularización: controla coeficientes y puede cambiar su magnitud.

Por eso, si interpretamos coeficientes de un modelo regularizado, debemos recordar que esos valores no solo reflejan asociaciones con la variable objetivo. También reflejan la penalización aplicada por el modelo.

Aun con estas precauciones, los coeficientes son una herramienta interpretativa muy valiosa. Permiten observar la dirección de la relación aprendida por el modelo y, en algunos casos, la magnitud relativa del aporte de cada variable.

Coeficientes útiles para: ver dirección, comparar aportes, detectar señales inesperadas, comunicar patrones generales.

Si una variable tiene un coeficiente con signo contrario al esperado, eso puede ser una pista para revisar los datos. Quizá haya un problema de codificación, una fuga de datos, una relación más compleja o una interpretación equivocada. Los coeficientes pueden funcionar como una herramienta de diagnóstico.

Los coeficientes permiten interpretar modelos lineales observando cómo cada variable participa en la predicción. En regresión lineal, indican cambios en el valor predicho. En regresión logística, indican cómo las variables empujan la predicción hacia una clase u otra. Son útiles, pero deben leerse con cuidado: dependen de la escala, del preprocesamiento, de las variables incluidas, de la correlación entre ellas y de la regularización. Sobre todo, debemos recordar que un coeficiente describe el comportamiento del modelo, no una causa demostrada del fenómeno.

Importancia de variables

Además de los coeficientes, otra forma frecuente de interpretar modelos consiste en analizar la importancia de variables. La idea general es identificar qué variables parecen tener mayor peso en el comportamiento del modelo. Dicho de manera simple: queremos saber qué columnas influyen más en las predicciones.

Importancia de variables: intento de estimar qué variables pesan más en el modelo.

Esta pregunta aparece en muchos proyectos. Si un modelo predice abandono de clientes, queremos saber si usa principalmente la actividad reciente, la antigüedad, la cantidad de reclamos, la frecuencia de compra o el monto gastado. Si predice precios de viviendas, queremos saber si influyen más la superficie, la ubicación, la cantidad de ambientes o la antigüedad. Si clasifica correos, queremos conocer qué términos ayudan más a distinguir spam de no spam.

La importancia de variables puede ayudar a comprender el modelo, comunicar resultados y detectar problemas. Si una variable aparece como muy importante, podemos investigarla. Si una variable que esperábamos relevante no aparece, también conviene revisar. Si una variable sospechosa domina el modelo, quizá exista fuga de datos o una señal poco apropiada.

Variables muy importantes: pueden revelar señales útiles, pero también posibles problemas.

Sin embargo, debemos tener cuidado desde el comienzo. “Importancia” no significa lo mismo en todos los modelos. Cada técnica puede calcularla de una manera distinta. Por eso, la importancia de variables no debe interpretarse como una verdad universal sobre los datos, sino como una descripción del comportamiento de un modelo específico.

Una variable puede ser importante para un modelo y menos importante para otro.

En modelos lineales, muchas veces usamos coeficientes para interpretar el aporte de cada variable. En árboles de decisión y ensambles basados en árboles, como Random Forest o Gradient Boosting, suelen existir medidas internas de importancia de variables. Estas medidas intentan estimar cuánto contribuye cada variable a mejorar las divisiones del modelo.

En `scikit-learn`, muchos modelos basados en árboles tienen el atributo `feature_importances_`.

```
modelo.feature_importances_
```

Si tenemos los nombres de las columnas, podemos construir una tabla para revisar esos valores.

```
import pandas as pd

importancias = pd.DataFrame({
    "variable": columnas,
    "importancia": modelo.feature_importances_
})

importancias.sort_values("importancia", ascending=False)
```

El resultado puede mostrarnos, por ejemplo, que `dias_desde_ultima_compra`, `frecuencia_compra` y `cantidad_reclamos` son variables destacadas en un modelo de abandono. Eso nos da una pista sobre qué señales está usando el modelo.

Importancias altas: el modelo parece usar mucho esas variables para construir sus predicciones.

Pero estas importancias deben leerse con prudencia. En árboles, una variable puede aparecer como importante porque permite buenas divisiones en el entrenamiento. Eso no significa automáticamente que sea la causa del resultado. Tampoco significa que sea estable en otros datasets.

Por ejemplo, si una variable relacionada con reclamos aparece como importante para predecir abandono, podemos decir que el modelo la está usando como señal. No podemos afirmar solo con eso que los reclamos causan abandono. Puede ser que los reclamos sean consecuencia de problemas previos que también explican el abandono.

Importancia predictiva no equivale a importancia causal.

También puede haber variables correlacionadas. Si dos variables contienen información parecida, el modelo puede repartir la importancia entre ellas o preferir una por sobre la otra. En un Random Forest, por ejemplo, si `superficie_m2` y `cantidad_ambientes` están muy relacionadas, la importancia puede distribuirse de una manera que no representa claramente el valor conceptual de cada una.

Variables correlacionadas pueden complicar la interpretación de la importancia individual.

Si quitamos una variable, la otra podría ganar importancia. Si cambiamos el dataset o el modelo, la distribución puede cambiar. Por eso, no conviene leer la importancia de variables como un ranking definitivo e inmutable.

Otra precaución aparece con variables categóricas codificadas. Si usamos One-Hot Encoding, una variable categórica original puede convertirse en muchas columnas binarias. Por ejemplo, `barrio` puede transformarse en decenas de columnas, una por barrio. La importancia puede quedar repartida entre esas columnas.

Concepto	Descripción
Variable categórica original	barrio
Después de One-Hot Encoding	barrio_A barrio_B barrio_C ...

Si miramos cada columna por separado, quizá ninguna parezca demasiado importante. Pero, tomadas en conjunto, las columnas derivadas de `barrio` podrían tener mucho peso. Para interpretar correctamente, a veces necesitamos agrupar importancias por variable original.

También puede ocurrir lo contrario: una variable categórica con muchas categorías puede recibir mucha atención porque ofrece muchas oportunidades de división, aunque no necesariamente sea tan útil desde el punto de vista general. Algunas medidas internas de importancia en árboles pueden estar sesgadas hacia variables con más posibilidades de partición.

Las importancias internas pueden favorecer variables con muchas formas de dividir los datos.

Esto no significa que debamos ignorarlas, sino que debemos tomarlas como una primera aproximación. Son útiles para explorar, pero conviene complementarlas con otros análisis.

La importancia de variables también depende del preprocesamiento. Si escalamos, codificamos, imputamos o transformamos datos, el modelo ya no trabaja exactamente con las variables originales, sino con columnas transformadas. La interpretación debe tener en cuenta ese flujo.

El modelo interpreta las columnas que recibe, no necesariamente las variables originales tal como las pensamos.

Por ejemplo, si una variable numérica fue transformada, si se agregaron columnas indicadoras de valores faltantes o si una categoría fue expandida en muchas columnas, las importancias se refieren a esa representación final.

Por eso, cuando trabajamos con pipelines complejos, interpretar variables exige reconstruir qué columnas llegaron realmente al modelo. En proyectos simples esto puede ser directo. En proyectos con muchas transformaciones, puede requerir más cuidado.

Otra forma de usar la importancia de variables es como herramienta de revisión. Si una variable aparece con importancia muy alta y no debería estar disponible al momento de predicción, eso puede indicar fuga de datos.

El proceso parte de Variable muy importante, continúa con + no disponible al momento de predecir y conduce finalmente a posible fuga de datos..

Por ejemplo, en un modelo para predecir abandono, una variable como `fecha_baja` no debería usarse, porque solo se conoce después del abandono. Si aparece en el modelo, la evaluación estará contaminada. Una métrica alta en ese caso no sería confiable.

También puede ser sospechosa una variable que parezca una versión encubierta de la respuesta. Por ejemplo, `estado_final_cliente`, `motivo_cierre` o `resultado_revision` podrían contener información posterior al evento que queremos predecir.

La importancia de variables puede ayudarnos a detectar estos errores, pero no los detecta automáticamente. Necesitamos conocer el problema y revisar la disponibilidad temporal de cada variable.

Interpretar variables importantes también implica preguntar: ¿esta información existía en el momento de predicción?

Otro uso práctico es comunicar el modelo. En lugar de presentar solo métricas, podemos mostrar las variables que más influyen en las predicciones. Esto ayuda a que el análisis sea más comprensible para personas que no participaron del entrenamiento.

Pero la comunicación debe ser responsable. No conviene decir “estas son las causas del abandono” si lo que tenemos es una importancia predictiva. Es mejor decir “el modelo se apoya especialmente en estas variables” o “estas variables aparecen como señales relevantes para la predicción”.

Concepto	Descripción
Más responsable	"variables relevantes para el modelo".
Menos responsable	"causas del resultado".

También conviene acompañar la importancia con una explicación del contexto. Una variable puede ser importante porque contiene una señal legítima, porque está correlacionada con otra variable, porque refleja un sesgo o porque hay un problema de datos. La importancia por sí sola no distingue estas posibilidades.

La importancia de variables busca identificar qué columnas parecen influir más en el comportamiento de un modelo. Puede ayudarnos a interpretar, comunicar, revisar y auditar. Pero debe usarse con prudencia: depende del modelo, del preprocesamiento, de las variables incluidas, de las correlaciones y de la forma en que se calcula. Una variable importante para predecir no es necesariamente una causa, ni un resultado universal. Es una pista sobre cómo está funcionando un modelo específico y debe analizarse con contexto.

Permutation importance

La importancia de variables puede calcularse de distintas maneras. Algunas medidas vienen incorporadas en ciertos modelos, como ocurre con `feature_importances_` en árboles y ensambles. Pero también existe una técnica más general, que puede aplicarse a muchos tipos de modelos: la *permutation importance*, o importancia por permutación.

La idea es muy intuitiva. Si una variable es importante para el modelo, entonces desordenar sus valores debería empeorar el desempeño. Si una variable no aporta demasiado, desordenarla no debería cambiar mucho la métrica.

El proceso parte de Variable importante, continúa con si la desordenamos,, luego el modelo rinde peor., luego Variable poco importante, luego si la desordenamos, y conduce finalmente a el rendimiento cambia poco..

Imaginemos un modelo que predice abandono de clientes. Supongamos que una variable muy útil es `dias_desde_ultima_compra`. Si mezclamos al azar los valores de esa columna, cada cliente queda asociado con un valor que no le corresponde. El modelo pierde una señal importante y sus predicciones deberían empeorar.

En cambio, si desordenamos una variable poco relacionada con el abandono, el desempeño quizás cambie muy poco. Eso indicaría que el modelo no dependía demasiado de esa columna.

Permutation importance pregunta:

¿cuánto empeora el modelo cuando rompemos la información de una variable?

El procedimiento general es el siguiente. Primero evaluamos el modelo normalmente, usando una métrica elegida. Luego tomamos una variable, mezclamos sus valores entre las filas y volvemos a evaluar. La diferencia entre la métrica original y la métrica después de mezclar la variable indica su importancia.

1. Evaluar el modelo original.
2. Elegir una variable.
3. Mezclar sus valores.
4. Evaluar de nuevo.
5. Medir cuánto empeoró.

Si el desempeño cae mucho, la variable era importante para ese modelo en ese conjunto de datos. Si cae poco, la variable parecía tener poca influencia. Si no cambia, quizá el modelo no la estaba usando de manera relevante. Si incluso mejora, puede ser una señal de ruido, azar o inestabilidad del resultado.

Esta técnica tiene una ventaja importante: no depende de la estructura interna del modelo. Puede usarse con modelos lineales, Random Forest, Gradient Boosting, SVM, KNN u otros modelos, siempre que podamos medir su desempeño antes y después de permutar una columna.

Permutation importance observa el efecto sobre el rendimiento.

No necesita mirar cómo está construido internamente el modelo.

Por eso se considera una técnica relativamente general. Nos permite comparar la importancia de variables usando el mismo criterio: cuánto se deteriora la métrica cuando destruimos la información de una columna.

En `scikit-learn`, podemos usar `permutation_importance`.

```
from sklearn.inspection import permutation_importance

resultado = permutation_importance(
    modelo,
    X_test,
    y_test,
    scoring="f1",
    n_repeats=10,
    random_state=42
)
```

En este ejemplo, la importancia se calcula sobre `X_test` y `y_test`, usando F1-score como métrica. El parámetro `n_repeats` indica cuántas veces se repite la permutación para cada variable, porque mezclar al azar puede producir pequeñas variaciones.

`n_repeats`: repite el proceso varias veces para obtener una estimación más estable.

El resultado incluye, entre otros valores, `importances_mean`, que contiene la importancia promedio de cada variable, e `importances_std`, que indica la variabilidad de esa estimación.

Podemos organizar esos valores en una tabla.

```
import pandas as pd

importancias = pd.DataFrame({
    "variable": columnas,
    "importancia_media": resultado.importances_mean,
    "desviacion": resultado.importances_std
})

importancias.sort_values(
    "importancia_media",
    ascending=False
)
```

Una variable con alta importancia media es aquella cuya permutación redujo mucho la métrica. La desviación ayuda a ver si esa importancia fue estable entre repeticiones. Si la desviación es alta, la estimación puede ser más incierta.

Concepto	Descripción
Importancia media alta	al desordenar esa variable, el modelo pierde rendimiento.
Desviación alta	la estimación varía bastante entre repeticiones.

La elección de la métrica es fundamental. La permutation importance no mide una importancia abstracta. Mide cuánto cambia una métrica concreta. Si usamos accuracy, obtendremos una lectura. Si usamos recall, podemos obtener otra. Si usamos F1-score, otra.

Esto es especialmente importante en problemas desbalanceados. Una variable puede no afectar demasiado la accuracy, pero sí afectar mucho el recall de la clase minoritaria. Por eso, la métrica elegida debe estar alineada con el objetivo del problema.

La importancia depende de la métrica usada.

Cambiar la métrica puede cambiar el ranking de variables.

Por ejemplo, en detección de fraude, quizá nos interese especialmente el recall de la clase fraude. Si calculamos permutation importance usando accuracy, podríamos subestimar variables relevantes para detectar casos raros. En cambio, una métrica enfocada en la clase de interés puede mostrar mejor qué variables sostienen esa capacidad del modelo.

Otra decisión importante es sobre qué datos calculamos la importancia. Conviene hacerlo sobre datos no usados para entrenar, como un conjunto de prueba o validación. Si calculamos la importancia sobre entrenamiento, podríamos medir cómo el modelo usa variables para ajustarse a datos ya vistos, no necesariamente cómo generaliza.

Mejor práctica: calcular importancia sobre datos no vistos o de validación.

Esto conecta con una idea que apareció muchas veces: no basta con que algo funcione en entrenamiento. Queremos entender qué variables sostienen el desempeño en datos nuevos.

Permutation importance también tiene limitaciones. Una de las más importantes aparece cuando las variables están correlacionadas. Si dos variables contienen información parecida, desordenar una sola puede no empeorar demasiado el modelo, porque la otra todavía conserva parte de la señal.

Variables correlacionadas: si desordenamos una, otra puede seguir aportando información similar.

Por ejemplo, en un modelo de precios de viviendas, `superficie_m2` y `cantidad_ambientes` pueden estar relacionadas. Si permutamos `superficie_m2`, el modelo quizá todavía use `cantidad_ambientes` como señal de tamaño. Entonces la importancia de `superficie_m2` podría parecer menor de lo que sería si estuviera sola.

Esto no significa que permutation importance esté mal. Significa que mide el efecto de romper una variable manteniendo las demás disponibles. Si hay variables redundantes, el resultado debe interpretarse con cuidado.

Permutation importance responde: ¿qué pasa si rompo esta columna y dejo las demás intactas?

Otra limitación es que puede ser costosa en tiempo. Para cada variable, el modelo debe evaluarse varias veces. Si tenemos muchas columnas, muchos datos o un modelo lento, el cálculo puede tardar. El parámetro `n_repeats` mejora la estabilidad, pero también aumenta el costo.

Además, cuando trabajamos con pipelines y preprocesamiento, debemos tener cuidado con los nombres de variables. Si el modelo recibe columnas transformadas, como variables escaladas o columnas generadas por One-Hot Encoding, la interpretación puede referirse a esas columnas transformadas, no siempre a las variables originales.

Con preprocesamiento complejo, la variable interpretada puede no coincidir directamente con una columna original.

En algunos casos, conviene calcular la importancia sobre el pipeline completo usando los datos originales de entrada. De esa manera, la permutación afecta las columnas originales antes del preprocesamiento. En otros casos, puede interesarnos interpretar columnas ya transformadas. La decisión depende de la pregunta.

Por ejemplo, si queremos saber si `barrio` como variable original es importante, quizás necesitemos agrupar el efecto de todas las columnas generadas por One-Hot Encoding. Si miramos cada categoría por separado, podemos perder la visión de conjunto.

El proceso parte de Variable original categórica, continúa con muchas columnas codificadas, luego importancia repartida y conduce finalmente a entre varias columnas..

También debemos recordar que permutation importance sigue siendo una técnica de interpretación del modelo, no una prueba causal. Si una variable tiene alta importancia, significa que el modelo depende de ella para sostener su rendimiento. No significa necesariamente que esa variable cause el resultado.

Alta permutation importance: la variable sostiene el rendimiento del modelo.

No significa automáticamente causalidad.

Por ejemplo, si `cantidad_reclamos` es importante para predecir abandono, el modelo usa esa señal. Pero eso no demuestra que los reclamos causen abandono. Puede haber factores previos que generen tanto reclamos como abandono.

A pesar de estas limitaciones, permutation importance es una herramienta muy útil. Permite interpretar modelos de distintos tipos con una lógica común. También ayuda a detectar variables sospechosas. Si al permutar una variable el rendimiento cae de manera enorme y esa variable no debería estar disponible al momento de predecir, puede haber fuga de datos.

El proceso parte de Variable demasiado importante, continúa con + disponibilidad sospechosa y conduce finalmente a revisar posible fuga de datos..

También puede ayudar a simplificar modelos. Si varias variables tienen importancia muy baja de manera estable, podríamos considerar si realmente aportan algo. Pero no conviene eliminarlas automáticamente. Antes de quitar variables, debemos evaluar el modelo completo y revisar si la eliminación afecta el desempeño.

Baja importancia no siempre implica eliminar la variable.

Primero hay que evaluar el efecto de quitarla.

Permutation importance mide cuánto empeora el desempeño del modelo cuando desordenamos los valores de una variable. Si al romper una columna la métrica cae mucho, esa variable parece importante para el modelo. Es una técnica general, aplicable a muchos tipos de modelos, y especialmente útil para interpretar modelos más opacos. Sin embargo, depende de la métrica elegida, de los datos usados para evaluarla, de las correlaciones entre variables y del preprocesamiento. Como toda interpretación, debe leerse como una pista sobre el comportamiento del modelo, no como una explicación causal definitiva.

Interpretación local y global

Cuando interpretamos un modelo, podemos hacerlo desde dos niveles distintos: interpretación global e interpretación local. Ambos niveles son útiles, pero responden preguntas diferentes.

La interpretación global intenta explicar el comportamiento general del modelo. Busca responder qué variables parecen importantes en conjunto, qué patrones predominan y cómo se comporta el modelo a nivel general.

Interpretación global: ¿cómo se comporta el modelo en términos generales?

Por ejemplo, en un modelo de abandono de clientes, una interpretación global podría mostrar que las variables más relevantes son los días desde la última compra, la frecuencia de compra, la cantidad de reclamos y la antigüedad. Esa lectura no explica un caso específico, sino el comportamiento general del modelo sobre muchos casos.

El proceso parte de Modelo de abandono, continúa con variables globalmente importantes: actividad reciente,, luego frecuencia,, luego reclamos, y conduce finalmente a antigüedad..

La importancia de variables y la permutation importance son ejemplos de herramientas de interpretación global. Nos ayudan a entender qué variables sostienen el desempeño del modelo en conjunto. También podemos usar gráficos, análisis de errores por grupo o comparaciones de métricas para obtener una mirada global.

La interpretación local, en cambio, intenta explicar una predicción individual. No pregunta qué hace el modelo en promedio, sino por qué produjo una predicción específica para un caso concreto.

Interpretación local: ¿por qué el modelo predijo esto para este caso particular?

Por ejemplo, un modelo puede predecir que un cliente tiene alto riesgo de abandono. La interpretación local intenta explicar qué características de ese cliente contribuyeron a esa predicción: muchos días desde la última compra, baja frecuencia reciente, reclamos abiertos o disminución en el uso del servicio.

Concepto	Descripción
Predicción individual	cliente con alto riesgo.
Interpretación local	qué señales de ese cliente empujaron esa predicción.

Esta diferencia es importante porque una variable puede ser muy importante a nivel global y no ser decisiva en todos los casos individuales. También puede ocurrir que una variable con importancia global moderada sea clave para una predicción específica.

Por ejemplo, en general, la frecuencia de compra puede ser una de las variables más importantes del modelo. Pero para un cliente particular, la predicción puede estar más influida por un reclamo reciente o por un cambio brusco en su patrón de uso.

Lo importante en promedio no siempre es lo que explica cada caso individual.

La interpretación global ayuda a comprender el modelo como sistema. La interpretación local ayuda a revisar decisiones puntuales. En muchos proyectos necesitamos ambas.

Supongamos que presentamos un modelo a un equipo de trabajo. A nivel global, podríamos explicar que el modelo se apoya principalmente en actividad reciente, frecuencia y reclamos. Pero si una persona pregunta por qué cierto cliente fue marcado como de alto riesgo, necesitamos una explicación local de ese caso.

Concepto	Descripción
Global	qué patrones usa el modelo.
Local	qué ocurrió en una predicción concreta.

En modelos transparentes, algunas interpretaciones locales pueden obtenerse de manera directa. En un árbol de decisión pequeño, podemos seguir el camino que recorrió un caso desde la raíz hasta la hoja final. Esa secuencia de preguntas explica por qué llegó a cierta predicción.

El proceso parte de Árbol de decisión: caso individual, continúa con camino de decisiones y conduce finalmente a predicción final.

En una regresión lineal o logística, podemos analizar cómo las variables del caso contribuyen al resultado según sus coeficientes. Si conocemos los valores de las variables y los coeficientes, podemos observar qué aportes empujan la predicción hacia arriba, hacia abajo o hacia una clase determinada.

En modelos más opacos, la interpretación local suele requerir herramientas adicionales. Existen técnicas diseñadas para aproximar qué variables influyeron en una predicción individual. No siempre son exactas en sentido absoluto, pero pueden ofrecer explicaciones útiles si se usan con cuidado.

Modelos opacos: la interpretación local requiere herramientas auxiliares y lectura prudente.

Una interpretación local puede ser especialmente valiosa cuando una predicción afecta una decisión concreta. Si un sistema marca una transacción como sospechosa, conviene saber qué señales motivaron la alerta. Si un modelo sugiere priorizar un caso para revisión, necesitamos saber qué información llevó a esa sugerencia.

Pero también puede ser riesgosa si se presenta como una explicación perfecta. Muchas técnicas locales son aproximaciones. Pueden depender del método, de los datos cercanos al caso, de la representación usada y de la estabilidad del modelo.

Una explicación local puede ayudar, pero no debe confundirse con una verdad definitiva.

También debemos recordar que una explicación local describe el comportamiento del modelo, no necesariamente la causa real del fenómeno. Si el modelo predijo alto riesgo de abandono porque hubo muchos días sin compra, eso indica que esa variable influyó en la predicción. No demuestra, por sí solo, la causa del abandono.

Concepto	Descripción
Explicación local	por qué predijo el modelo.
No necesariamente	por qué ocurrió el fenómeno.

La interpretación global también tiene límites. Un ranking de variables importantes puede ocultar diferencias entre subgrupos. El modelo puede apoyarse en ciertas variables para la mayoría de los casos, pero usar otras señales en grupos específicos. Si miramos solo el promedio, podemos perder esas diferencias.

Por ejemplo, un modelo de riesgo puede comportarse distinto con clientes nuevos y clientes antiguos. Una variable puede ser muy relevante para un grupo y poco relevante para otro. La interpretación global ofrece una visión general, pero no siempre captura todos los matices.

La interpretación global resume el comportamiento, pero puede ocultar diferencias entre subgrupos.

Por eso, a veces conviene combinar ambos niveles. Podemos empezar con una visión global para entender el modelo en conjunto y luego revisar casos individuales representativos: aciertos, errores, falsos positivos, falsos negativos, casos cercanos al umbral o casos de alto impacto.

Interpretar bien puede incluir: mirada global, casos individuales, errores, subgrupos, y contexto.

Los errores son especialmente importantes. Una interpretación global puede decirnos qué variables usa el modelo en general, pero revisar errores individuales puede mostrar en qué situaciones falla. Tal vez el modelo se confunde con clientes nuevos, con casos atípicos o con registros incompletos. Esos hallazgos pueden ser más útiles que mirar solo importancias promedio.

Revisar errores individuales ayuda a entender los límites reales del modelo.

También es útil analizar casos cercanos al umbral de decisión. En clasificación binaria, algunos casos tienen probabilidades cercanas al umbral elegido. Esos casos

suelen ser más inciertos. Una explicación local puede ayudar a entender por qué quedaron de un lado u otro.

Casos cerca del umbral: decisiones más inciertas, más necesidad de revisión.

En cambio, casos muy alejados del umbral pueden ser más fáciles para el modelo. Pero incluso allí conviene revisar si la confianza aparente está justificada, especialmente si el modelo puede estar usando variables problemáticas.

A nivel práctico, la interpretación local y global deben estar alineadas con el uso del modelo. Si el modelo se usa para análisis general, quizá alcance con una interpretación global y ejemplos. Si se usa para decisiones individuales, la interpretación local se vuelve mucho más importante.

Concepto	Descripción
Uso analítico general	más peso de la interpretación global.
Decisiones caso por caso	más peso de la interpretación local.

También debemos adaptar la explicación al público. Un equipo técnico puede necesitar detalles sobre métricas, importancia de variables y métodos de interpretación. Una persona no técnica puede necesitar una explicación más concreta: qué factores principales influyeron y qué límites tiene la predicción.

La interpretación global busca entender cómo se comporta el modelo en conjunto, mientras que la interpretación local intenta explicar predicciones individuales. La primera ayuda a comprender patrones generales, variables relevantes y comportamiento promedio. La segunda ayuda a revisar casos concretos, decisiones puntuales y errores específicos. Ambas son complementarias, y ambas deben interpretarse con prudencia: explican el comportamiento del modelo, no necesariamente las causas reales del fenómeno.

Riesgos de interpretar de más

Interpretar modelos es necesario, pero también puede llevarnos a errores si convertimos las explicaciones en certezas demasiado fuertes. Un modelo puede ayudarnos a entender patrones, revisar variables importantes y comunicar resultados, pero sus interpretaciones tienen límites. Si olvidamos esos límites, corremos el riesgo de interpretar de más.

Interpretar de más significa sacar conclusiones que van más allá de lo que el modelo realmente permite afirmar.

Concepto	Descripción
Interpretar bien	entender el comportamiento del modelo con prudencia.
Interpretar de más	convertir señales del modelo en conclusiones exageradas.

Un primer riesgo es confundir importancia predictiva con causalidad. Si una variable aparece como importante, eso significa que el modelo la usa para predecir. No significa necesariamente que esa variable cause el resultado.

Por ejemplo, si un modelo de abandono de clientes muestra que la cantidad de reclamos es importante, podemos decir que el modelo usa esa variable como señal. Pero no podemos afirmar automáticamente que los reclamos causan el abandono. Puede ocurrir que los reclamos sean consecuencia de problemas previos en el servicio, y que esos problemas sean los que realmente explican la baja.

Variable importante para predecir no equivale a causa del fenómeno.

Esta diferencia es fundamental. Los modelos predictivos suelen aprender asociaciones. Algunas asociaciones son útiles para anticipar un resultado, pero no necesariamente explican por qué ocurre. Para hablar de causas necesitamos otros diseños de análisis, otros datos y otros criterios.

Un segundo riesgo es olvidar que la interpretación depende del modelo. Si entrenamos una regresión logística, un Random Forest y un Gradient Boosting sobre el mismo dataset, cada modelo puede apoyarse en las variables de manera diferente. Una variable puede ser muy importante en un modelo y menos importante en otro.

La interpretación pertenece a un modelo específico, entrenado con datos específicos.

Por eso, no conviene decir “esta variable es la más importante del problema” solo porque aparece primera en un ranking. Es más responsable decir “en este modelo, con estos datos y este preprocesamiento, esta variable aparece como muy relevante”.

La diferencia puede parecer pequeña, pero cambia mucho el sentido de la conclusión. La primera frase suena definitiva. La segunda deja claro que hablamos de un resultado situado.

Concepto	Descripción
Más responsable	"En este modelo, esta variable parece relevante."
Menos responsable	"Esta es la variable más importante del fenómeno."

Un tercer riesgo es no considerar el preprocesamiento. Los modelos no siempre trabajan con las variables originales tal como aparecen en la tabla. Pueden recibir columnas escaladas, imputadas, codificadas, combinadas o transformadas. Si

interpretamos sin mirar ese flujo, podemos atribuir significado a columnas que ya no coinciden directamente con las variables iniciales.

Esto ocurre mucho con One-Hot Encoding. Una variable categórica puede transformarse en muchas columnas binarias. Si interpretamos cada columna por separado, podemos perder la visión de la variable original. También puede ocurrir que una categoría parezca importante, pero solo en relación con la forma en que se codificaron los datos.

El modelo interpreta la representación final de los datos.

Nosotros debemos reconstruir qué significa esa representación.

Un cuarto riesgo es interpretar rankings como si fueran exactos. Las importancias de variables suelen ordenarse de mayor a menor, pero ese orden no siempre debe leerse con precisión absoluta. Dos variables con valores muy parecidos pueden intercambiar posiciones si cambia la muestra, el entrenamiento, la métrica o el modelo.

Un ranking de variables puede orientar, pero no siempre indica diferencias exactas y estables.

Si una variable tiene importancia 0.21 y otra 0.20, no deberíamos construir una conclusión fuerte diciendo que la primera es claramente más importante. Puede ser una diferencia pequeña dentro de la variabilidad del análisis.

Por eso, cuando sea posible, conviene revisar estabilidad: repetir validaciones, comparar modelos, observar desviaciones en permutation importance o analizar si el patrón se mantiene en distintos subconjuntos de datos.

Interpretaciones más confiables suelen ser aquellas que se mantienen ante cambios razonables del análisis.

Un quinto riesgo es ignorar variables correlacionadas. Cuando dos columnas contienen información parecida, la importancia puede repartirse, ocultarse o desplazarse. Una variable puede parecer poco importante porque otra variable relacionada ya aporta la misma señal. O una puede aparecer como dominante solo porque el modelo la eligió antes.

Por ejemplo, en viviendas, superficie, ambientes y dormitorios pueden estar relacionadas. Si superficie aparece como la variable más importante, no significa que las demás no importen. Puede significar que superficie resume parte de la información que también contienen las otras.

Variables correlacionadas pueden hacer que la importancia sea difícil de repartir e interpretar.

Esto también afecta a los coeficientes. En modelos lineales, la presencia de variables correlacionadas puede cambiar la magnitud y el signo de los coeficientes. Si quitamos o agregamos una variable, los coeficientes pueden modificarse. Por eso, leerlos como efectos aislados puede ser engañoso.

Un sexto riesgo es interpretar patrones aprendidos de datos sesgados como si fueran patrones justos o neutrales. Si los datos reflejan desigualdades, exclusiones, errores históricos o decisiones previas, el modelo puede aprender esas estructuras. Interpretar el modelo sin revisar el origen de los datos puede llevarnos a normalizar esos sesgos.

Un modelo puede aprender patrones presentes en los datos, aunque esos patrones sean injustos o problemáticos.

Por ejemplo, si un modelo se entrena con datos de oportunidades pasadas y esas oportunidades estuvieron distribuidas de manera desigual, el modelo puede reproducir esa desigualdad. Una variable puede aparecer como predictiva, pero su uso puede ser cuestionable desde el punto de vista ético o institucional.

Por eso, interpretar no es solo mirar qué variable pesa más. También es preguntar si esa variable debería usarse, qué representa realmente y qué consecuencias puede tener.

Concepto	Descripción
Pregunta técnica	¿la variable ayuda a predecir?
Pregunta responsable	¿es correcto usarla para esta decisión?

Un séptimo riesgo es confiar demasiado en explicaciones locales. Una explicación local puede ayudarnos a entender una predicción individual, pero muchas técnicas locales son aproximadas. Pueden cambiar si modificamos el método de explicación, el conjunto de referencia o algunos detalles del modelo.

Esto no significa que no sirvan. Significa que deben leerse como ayudas interpretativas, no como una reconstrucción perfecta del razonamiento del modelo.

Una explicación local puede orientar una revisión, pero no siempre agota todo lo que hizo el modelo.

También puede ocurrir que una explicación local sea convincente, pero incompleta. Tal vez muestre las variables que más influyeron en una predicción, pero no indique

que el modelo tiene bajo desempeño en cierto subgrupo. O puede explicar un caso particular sin revelar un problema global de sesgo o calibración.

Por eso, las explicaciones locales deben complementarse con análisis global, revisión de errores y evaluación por grupos cuando corresponda.

Buena interpretación: mirada global + casos individuales + errores + contexto.

Un octavo riesgo es interpretar probabilidades sin revisar calibración. Un modelo puede asignar una probabilidad alta a una clase, pero eso no significa necesariamente que esa probabilidad esté bien calibrada. Como vimos en otros capítulos, algunos modelos producen puntuaciones o probabilidades que requieren cuidado.

Si un modelo dice 0.90, no siempre significa que, de cada 100 casos similares, 90 pertenecerán realmente a esa clase. Puede ser una señal fuerte para el modelo, pero no una probabilidad confiable en sentido estricto.

Probabilidad alta no siempre significa certeza bien calibrada.

Esto es especialmente importante cuando las probabilidades se usan para tomar decisiones, fijar umbrales o comunicar riesgo. En esos casos, interpretar la salida del modelo requiere evaluar no solo la métrica de clasificación, sino también la calidad de las probabilidades.

Un noveno riesgo es olvidar que los modelos pueden cambiar con nuevos datos. Una interpretación válida hoy puede dejar de serlo si cambian los hábitos de los usuarios, las condiciones del mercado, los procesos de registro, las políticas de la organización o la forma en que se generan los datos.

La interpretación de un modelo puede envejecer si cambia el contexto que produce los datos.

Por eso, los modelos no deberían interpretarse una vez y considerarse cerrados para siempre. En proyectos reales, conviene monitorear desempeño, revisar errores y actualizar interpretaciones cuando el contexto cambia.

Un décimo riesgo es comunicar con un lenguaje demasiado fuerte. Palabras como “demuestra”, “prueba”, “causa” o “garantiza” pueden ser problemáticas si el análisis solo permite hablar de asociaciones o señales predictivas.

Es mejor usar expresiones más cuidadosas:

"El modelo sugiere..." "En estos datos se observa..." "Esta variable aparece asociada..." "El modelo se apoya en..." "Esta interpretación debe revisarse con..."

Ese tipo de lenguaje no debilita el análisis. Al contrario, lo vuelve más honesto y confiable. Un buen informe no exagera lo que el modelo sabe. Explica qué se encontró, cómo se obtuvo, qué límites tiene y qué revisiones serían necesarias.

Interpretar modelos es valioso, pero interpretar de más puede llevar a conclusiones equivocadas. Los coeficientes, las importancias de variables, las explicaciones locales y los gráficos ayudan a entender el comportamiento del modelo, pero no demuestran causalidad ni garantizan justicia, estabilidad o verdad definitiva. Toda interpretación depende de los datos, el preprocesamiento, el modelo, la métrica y el contexto. Por eso, interpretar responsablemente significa explicar con claridad, reconocer límites y evitar afirmaciones más fuertes que la evidencia disponible.

Interpretar para usar mejor

En este capítulo trabajamos una idea central: un modelo no debería evaluarse únicamente por sus métricas. Las métricas nos dicen cómo rinde bajo ciertas condiciones, pero no siempre nos dicen cómo está tomando sus decisiones, qué variables utiliza, qué patrones aprendió ni qué riesgos puede esconder. Para eso necesitamos interpretar.

Interpretar un modelo significa analizar su comportamiento. Queremos entender qué señales usa, qué variables parecen influir más, cómo se relacionan con las predicciones y qué límites tienen esas explicaciones.

Interpretar un modelo: entender cómo se comporta, qué señales usa y qué precauciones exige.

Vimos que no todos los modelos son igual de transparentes. Algunos, como regresiones lineales, regresiones logísticas o árboles pequeños, permiten una lectura relativamente directa. Otros, como Random Forest, Gradient Boosting o SVM con kernels, pueden ser más opacos y requieren herramientas adicionales para aproximarnos a su comportamiento.

Concepto	Descripción
Modelos transparentes	más fáciles de explicar directamente.
Modelos opacos	pueden requerir herramientas de interpretación adicionales.

También analizamos los coeficientes. En modelos lineales, los coeficientes permiten observar cómo participa cada variable en la predicción. En regresión lineal, pueden

interpretarse como cambios en el valor predicho. En regresión logística, indican cómo las variables empujan la predicción hacia una clase u otra.

Pero los coeficientes deben leerse con cuidado. Su interpretación depende de la escala de las variables, del preprocesamiento, de la presencia de variables correlacionadas, de la regularización y del conjunto de variables incluidas. Además, un coeficiente describe el comportamiento del modelo, no una causa demostrada del fenómeno.

Coeficiente: señal de cómo usa una variable el modelo.

No equivale automáticamente a una explicación causal.

Otra herramienta importante fue la importancia de variables. Esta idea busca identificar qué columnas parecen pesar más en el modelo. Puede ayudarnos a comunicar, revisar y auditar. Si una variable aparece como muy importante, puede ser una señal útil, pero también puede revelar un problema, como fuga de datos o una variable que no debería estar disponible al momento de predecir.

Variable importante: puede ser una señal útil, o una pista para revisar posibles problemas.

Estudiamos también la permutation importance. Esta técnica mide cuánto empeora el desempeño del modelo cuando desordenamos los valores de una variable. Si la métrica cae mucho, esa variable parece importante para sostener el rendimiento. Es una herramienta flexible porque puede aplicarse a distintos tipos de modelos.

Permutation importance: romper una variable y medir cuánto cae el desempeño del modelo.

Sin embargo, también tiene límites. Depende de la métrica elegida, de los datos usados para calcularla, de la presencia de variables correlacionadas y del preprocesamiento. Como toda técnica de interpretación, debe leerse como una pista sobre el comportamiento del modelo, no como una verdad definitiva.

Después diferenciamos interpretación global e interpretación local. La interpretación global intenta explicar el comportamiento general del modelo: qué variables son relevantes en conjunto, qué patrones predominan, cómo parece funcionar el modelo en promedio. La interpretación local intenta explicar una predicción individual.

Concepto	Descripción
Interpretación global	cómo se comporta el modelo en conjunto.
Interpretación local	por qué produjo una predicción para un caso concreto.

Ambas miradas son complementarias. Una importancia global puede ayudarnos a entender el modelo como sistema, pero no explica necesariamente cada caso particular. Una explicación local puede ayudarnos a revisar una predicción concreta, pero no describe por sí sola todo el comportamiento del modelo.

También dedicamos atención a los riesgos de interpretar de más. Este punto es fundamental. Una variable importante no es necesariamente una causa. Una explicación local no es necesariamente una reconstrucción perfecta del razonamiento del modelo. Un ranking de variables no es una verdad absoluta. Una probabilidad alta no siempre está bien calibrada. Un modelo entrenado con datos sesgados puede aprender patrones problemáticos.

Interpretar responsablemente significa explicar, pero también reconocer límites.

Por eso, el lenguaje importa. Es preferible decir “el modelo se apoya en estas variables” antes que “estas variables causan el resultado”. Es más prudente decir “en esta representación se observa una separación” antes que “los grupos existen de manera definitiva”. Una buena interpretación no exagera lo que el modelo puede afirmar.

Interpretar modelos también ayuda a mejorar el trabajo técnico. Puede revelar variables mal construidas, señales inesperadas, fuga de datos, errores de preprocesamiento, sesgos, inestabilidad o diferencias entre subgrupos. No es solo una etapa de comunicación; también es una herramienta de revisión.

Interpretar sirve para: comprender, comunicar, auditar, detectar problemas y mejorar modelos.

Interpretar un modelo es una parte esencial del trabajo con Machine Learning. Las métricas indican desempeño, pero la interpretación ayuda a entender qué está haciendo el modelo y si ese comportamiento tiene sentido. Coeficientes, importancia de variables, permutation importance e interpretaciones locales y globales son herramientas útiles, siempre que se usen con prudencia. Un modelo bien interpretado no es aquel al que le inventamos una explicación perfecta, sino aquel cuyo comportamiento podemos analizar, comunicar y revisar reconociendo sus límites.

Capítulo 29 · Sesgos, datos y decisiones

Sesgo en los datos

Un modelo de Machine Learning aprende a partir de datos. Esta idea apareció muchas veces a lo largo del libro, pero en este capítulo vamos a mirarla desde una perspectiva más crítica. Si los datos contienen errores, ausencias, desigualdades o decisiones previas problemáticas, el modelo puede aprender esos patrones y reproducirlos en sus predicciones.

Cuando hablamos de sesgo en los datos, nos referimos a una distorsión que hace que la información disponible no represente de manera adecuada el fenómeno que queremos estudiar o la población sobre la que queremos tomar decisiones.

Sesgo en los datos: la información disponible no representa bien la realidad que queremos modelar.

Esto puede ocurrir por muchas razones. A veces ciertos grupos están poco representados. A veces los datos fueron recolectados en un contexto específico y luego se quieren aplicar en otro. A veces faltan variables importantes. A veces las etiquetas reflejan decisiones humanas previas que ya tenían errores o desigualdades.

Por ejemplo, imaginemos un modelo que intenta predecir el rendimiento de estudiantes en una plataforma educativa usando datos de actividad: cantidad de accesos, tiempo conectado, actividades entregadas y participación en foros. Si algunos estudiantes tienen mala conectividad o comparten dispositivo, su actividad digital puede verse más baja, aunque eso no refleje necesariamente menor compromiso o menor capacidad.

Baja actividad registrada puede significar muchas cosas: poco interés, problemas de conexión, falta de tiempo, dispositivo compartido u otras condiciones externas.

Si el modelo aprende que baja actividad equivale directamente a bajo desempeño o bajo compromiso, puede reforzar una interpretación injusta. El problema no está solo en el algoritmo. Está en cómo los datos capturan parcialmente una realidad más compleja.

Algo similar puede ocurrir en datos comerciales. Una empresa puede tener más información sobre clientes frecuentes que sobre clientes ocasionales. Puede registrar mejor los comportamientos digitales que las compras presenciales. Puede contar con datos completos de ciertas zonas y datos incompletos de otras. Si entrenamos un

modelo con esa información, sus predicciones estarán condicionadas por esas diferencias.

Los datos no son neutrales: son el resultado de cómo se mide, qué se registra y qué queda fuera.

Una fuente común de sesgo es la subrepresentación. Si un grupo aparece muy poco en los datos de entrenamiento, el modelo tendrá menos ejemplos para aprender patrones asociados a ese grupo. Como consecuencia, puede rendir peor sobre esos casos.

Por ejemplo, si un modelo de clasificación se entrena principalmente con datos de usuarios adultos y luego se aplica a adolescentes, puede fallar más en ese grupo. Si un sistema de reconocimiento se entrenó con imágenes tomadas en ciertas condiciones de iluminación y luego se aplica en condiciones muy distintas, el rendimiento puede caer.

El proceso parte de poca representación, continúa con menos ejemplos para aprender, luego mayor riesgo de errores y conduce finalmente a en ese grupo..

Esto conecta con la idea de generalización. Un modelo generaliza bien cuando los datos de entrenamiento representan razonablemente los casos nuevos que verá después. Si entrenamiento y aplicación pertenecen a contextos distintos, el modelo puede parecer bueno en evaluación y fallar en la práctica.

Otra fuente de sesgo aparece en las etiquetas. En aprendizaje supervisado, la variable objetivo **y** funciona como respuesta conocida. Pero esa respuesta no siempre es una verdad perfecta. Puede ser una decisión humana, un registro administrativo, una evaluación previa o una clasificación hecha bajo ciertos criterios.

La etiqueta **y** no siempre representa una verdad objetiva. A veces surge de una decisión, un registro o una interpretación previa.

Por ejemplo, si entrenamos un modelo para predecir si una solicitud será aprobada usando decisiones históricas, el modelo aprenderá patrones presentes en esas decisiones. Si en el pasado hubo criterios inconsistentes, errores o desigualdades, esos patrones pueden quedar incorporados en el modelo.

Esto es especialmente delicado cuando el modelo se usa para apoyar decisiones futuras. Puede terminar reproduciendo el pasado en lugar de ayudar a tomar decisiones mejores.

El proceso parte de Datos históricos, continúa con modelo, luego decisiones futuras, luego Si el pasado tenía sesgos, y conduce finalmente a el modelo puede arrastrarlos..

También puede haber sesgo por variables faltantes. Si no registramos información importante, el modelo puede apoyarse en señales indirectas. A veces esas señales son útiles; otras veces pueden ser problemáticas.

Supongamos que queremos predecir abandono de clientes, pero no tenemos una variable que indique si tuvieron problemas con el servicio. Tal vez el modelo use reclamos, baja actividad o zona geográfica como señales indirectas. Algunas pueden tener sentido, pero otras pueden reflejar desigualdades o diferencias de acceso.

Cuando falta una variable importante, el modelo puede apoyarse en sustitutos imperfectos.

A esos sustitutos a veces se los llama proxies. Una variable proxy es una variable que no mide directamente algo, pero está relacionada con eso. El problema aparece cuando el proxy representa información sensible, desigualdades o condiciones externas que no deberían influir de esa manera.

Por ejemplo, la zona de residencia puede funcionar como proxy de nivel socioeconómico. El tipo de dispositivo puede funcionar como proxy de capacidad de acceso tecnológico. El horario de conexión puede estar relacionado con obligaciones laborales o familiares. Estas relaciones no siempre son evidentes, pero pueden afectar al modelo.

Una variable aparentemente neutral puede actuar como proxy de una condición sensible o desigual.

El sesgo en los datos también puede surgir por la forma de recolección. Si una encuesta solo se responde en línea, quedan fuera personas sin acceso o con baja conectividad. Si los datos provienen de una aplicación móvil, representan mejor a quienes usan esa aplicación. Si un registro depende de que una persona denuncie un problema, puede haber grupos que reporten menos aunque sufran el mismo problema.

Lo que no se registra también forma parte del problema de datos.

Este punto es importante porque muchas veces trabajamos con los datos disponibles como si fueran “la realidad”. Pero los datos son una representación de la realidad, no la realidad completa. Fueron producidos por sistemas, formularios, sensores, plataformas, instituciones o personas. Cada forma de registro tiene límites.

También puede haber sesgo temporal. Los datos de un período pueden no representar otro período. Un modelo entrenado con datos de ventas de un año normal puede fallar durante una crisis económica, una pandemia, una campaña excepcional o un cambio fuerte de precios. Un modelo entrenado con hábitos de uso antiguos puede no adaptarse a cambios en una plataforma.

Los datos tienen un contexto temporal: lo que fue válido antes puede dejar de serlo después.

Esto no significa que no podamos usar datos históricos. Significa que debemos preguntarnos si siguen siendo representativos del contexto actual. La calidad de un modelo depende también de la vigencia de los datos.

Otra fuente de sesgo es la medición desigual. Algunas variables pueden medirse con más precisión en ciertos grupos que en otros. Por ejemplo, una plataforma puede registrar con más detalle la actividad de usuarios que usan la versión web que la de quienes usan una versión móvil limitada. Un sistema puede registrar mejor transacciones digitales que operaciones en efectivo. Un sensor puede funcionar mejor en ciertas condiciones que en otras.

Mediciones desiguales pueden producir modelos desiguales.

Cuando los datos tienen estos problemas, el modelo puede aprender diferencias que en realidad vienen de la forma de medición, no del fenómeno que queremos analizar.

Por eso, revisar sesgos en los datos no es una etapa opcional. Forma parte de construir modelos responsables. Antes de entrenar, conviene preguntarnos:

Para revisar este aspecto conviene preguntarse: ¿Quiénes están representados? ¿Quiénes faltan? ¿Cómo se obtuvieron los datos? ¿Qué etiquetas se usaron? ¿Hay variables sensibles o proxies? ¿El contexto cambió?

Estas preguntas no siempre tienen respuestas simples. Pero hacerlas ayuda a evitar una confianza excesiva en el dataset.

También debemos recordar que los sesgos no se eliminan solo con técnicas automáticas. Limpiar datos, imputar faltantes o balancear clases puede ayudar en algunos aspectos, pero no resuelve por sí solo problemas de representación, desigualdad o mala definición del objetivo.

La limpieza técnica no siempre corrige un sesgo de origen.

Por ejemplo, si una etiqueta histórica refleja decisiones injustas, imputar valores faltantes no corrige ese problema. Si un grupo no está representado, escalar

variables no lo hace aparecer. Si una variable funciona como proxy de una condición sensible, el modelo puede seguir usando esa señal aunque el nombre de la columna parezca neutral.

El primer paso es reconocer que los datos tienen historia. Fueron generados por procesos concretos, con criterios concretos y límites concretos. El modelo aprende desde esa historia.

Un modelo no aprende desde una realidad pura, sino desde datos producidos por un contexto.

El sesgo en los datos aparece cuando la información disponible no representa adecuadamente el fenómeno o la población sobre la que queremos modelar. Puede surgir por subrepresentación, etiquetas históricas problemáticas, variables faltantes, proxies, mediciones desiguales, cambios temporales o formas limitadas de recolección. Como los modelos aprenden desde los datos, pueden reproducir esos sesgos si no los revisamos. Por eso, trabajar con Machine Learning de manera responsable exige mirar el dataset no solo como una tabla, sino como una representación parcial y contextual de la realidad.

Sesgo en el modelo

El sesgo no aparece solo en los datos. También puede aparecer en el modelo, en la forma en que aprende, en las variables que usa, en el objetivo que optimiza y en las decisiones técnicas que tomamos durante el proceso. Aunque el dataset fuera razonablemente bueno, un modelo puede producir resultados sesgados si el enfoque elegido no representa bien el problema o si optimiza una métrica que deja fuera aspectos importantes.

Cuando hablamos de sesgo en el modelo, podemos referirnos a distintas cosas. En un sentido técnico, ya vimos que un modelo demasiado simple puede tener alto sesgo porque no captura patrones relevantes. Pero en este capítulo estamos usando la palabra sesgo en un sentido más amplio: como una tendencia del sistema a producir resultados desiguales, parciales o problemáticos.

Sesgo en el modelo: tendencia del sistema a producir resultados distorsionados, desiguales o poco adecuados para el problema real.

Un modelo puede amplificar sesgos que ya estaban en los datos. Si ciertos grupos están poco representados, el modelo puede rendir peor en ellos. Si las etiquetas históricas reflejan decisiones injustas, el modelo puede aprenderlas y reproducirlas. Pero además, las propias decisiones de modelado pueden intensificar el problema.

Por ejemplo, supongamos que entrenamos un modelo de clasificación y evaluamos solo con accuracy. Si la clase mayoritaria domina el dataset, el modelo puede obtener una buena métrica acertando principalmente esa clase y fallando en la minoritaria. Desde el punto de vista global parece funcionar bien, pero para la clase menos representada puede ser poco útil.

Una buena accuracy general puede ocultar un mal desempeño en una clase minoritaria.

Esto ya lo vimos al estudiar métricas de clasificación. La diferencia es que ahora lo miramos desde el punto de vista de las decisiones. Si la clase minoritaria representa casos importantes, como fraude, abandono, riesgo, fallas o situaciones que requieren apoyo, el modelo puede dejar sin detectar justamente aquello que más nos importa.

En este caso, el sesgo no está solo en los datos. También está en la forma en que decidimos evaluar y seleccionar el modelo. Si elegimos una métrica inadecuada, favorecemos modelos que parecen buenos bajo esa métrica, aunque fallen en aspectos relevantes.

La métrica elegida orienta qué modelo consideramos mejor.

Otro ejemplo aparece con los umbrales de decisión. En clasificación binaria, muchos modelos producen probabilidades o puntuaciones. Luego usamos un umbral para convertir esa salida en una clase. Si usamos siempre 0.5 sin analizar el problema, podemos producir decisiones que no se ajustan al costo real de los errores.

Por ejemplo, en un sistema de detección de casos que requieren revisión, un umbral muy alto puede reducir falsos positivos, pero aumentar falsos negativos. Eso significa que muchos casos que necesitaban atención podrían no ser señalados. Un umbral más bajo puede detectar más casos, pero también generar más alertas innecesarias.

Cambiar el umbral modifica el tipo de errores que comete el modelo.

Si esos errores afectan de manera desigual a distintos grupos, el umbral puede contribuir al sesgo del sistema. Por eso, no alcanza con entrenar un modelo y aceptar su salida por defecto. Debemos analizar cómo se traducen las predicciones en decisiones.

El modelo también puede sesgarse por las variables que incluimos. Algunas variables pueden parecer útiles para predecir, pero representar condiciones externas, desigualdades o proxies de información sensible. Si el modelo se apoya mucho en ellas, puede producir decisiones problemáticas aunque la métrica sea alta.

Una variable predictiva no siempre es una variable apropiada para tomar decisiones.

Por ejemplo, la zona de residencia puede ayudar a predecir ciertos comportamientos comerciales, pero también puede estar relacionada con condiciones socioeconómicas. El tipo de dispositivo puede estar asociado al acceso tecnológico. La actividad en una plataforma puede depender no solo del interés, sino también de conectividad, tiempo disponible o recursos.

Si el modelo usa estas variables sin análisis, puede confundir patrones de acceso con características personales o de desempeño. Esto puede ser especialmente delicado en contextos educativos, laborales, financieros o institucionales.

Otra fuente de sesgo está en la elección del algoritmo. Un modelo muy simple puede no capturar patrones relevantes para ciertos grupos. Un modelo muy complejo puede ajustar ruido o patrones espurios. Un modelo entrenado sin revisar subgrupos puede rendir bien en promedio y mal en casos específicos.

El desempeño promedio puede ocultar diferencias entre subgrupos.

Por ejemplo, un modelo puede tener buen F1-score general, pero cometer muchos errores en personas nuevas en una plataforma, en clientes de cierta región, en productos poco frecuentes o en estudiantes con patrones de conexión distintos. Si solo miramos la métrica global, no veremos esas diferencias.

Por eso, cuando el contexto lo requiere, conviene evaluar el desempeño por subgrupos. No siempre tendremos o deberemos usar variables sensibles, pero sí debemos preguntarnos si el modelo funciona de manera razonablemente consistente en los conjuntos relevantes para el problema.

Evaluar por subgrupos puede revelar errores que la métrica global no muestra.

El sesgo del modelo también puede surgir por sobreajuste. Si un modelo aprende demasiado bien particularidades del conjunto de entrenamiento, puede fallar en casos nuevos. Y ese fallo no siempre se distribuye de manera uniforme. Puede afectar más a casos menos frecuentes, perfiles raros o grupos poco representados.

En este sentido, el sobreajuste no es solo un problema técnico de generalización. También puede convertirse en un problema de equidad y confiabilidad si los errores se concentran en ciertos tipos de casos.

Sobreajustar puede afectar especialmente a casos poco frecuentes o mal representados.

También puede haber sesgo por la forma en que definimos el objetivo. Un modelo aprende a optimizar lo que le pedimos, no necesariamente lo que nos importa en

sentido amplio. Si definimos mal la variable objetivo, el modelo puede resolver una tarea técnicamente correcta pero conceptualmente pobre.

Por ejemplo, si usamos “cantidad de accesos” como indicador de compromiso estudiantil, el modelo puede aprender a valorar actividad visible en la plataforma. Pero compromiso no siempre se mide bien con accesos. Puede haber estudiantes que estudian offline, comparten dispositivo o tienen conectividad limitada. El objetivo elegido reduce una realidad compleja a una señal medible.

Un objetivo mal definido puede producir un modelo técnicamente correcto, pero conceptualmente injusto.

Esta idea es central: el modelo no decide qué significa éxito, riesgo, mérito, interés, abandono o desempeño. Esa definición viene de quienes construyen el sistema. Si la definición es estrecha o problemática, el modelo heredaré ese problema.

También debemos pensar en el proceso de selección del modelo. Si probamos muchos modelos e hiperparámetros y elegimos el que obtiene la mejor métrica en validación, podemos terminar seleccionando un modelo que aprovecha particularidades del conjunto usado para evaluar. Por eso es importante separar adecuadamente entrenamiento, validación y prueba, y evitar usar el test como espacio de experimentación.

La selección del modelo también puede introducir sesgos si la evaluación no es cuidadosa.

En contextos sensibles, no alcanza con elegir el modelo de mayor métrica global. Puede ser necesario comparar modelos considerando estabilidad, desempeño por subgrupos, interpretabilidad, calibración de probabilidades y tipos de error.

Otro punto importante es que el modelo puede cambiar el comportamiento de las personas o sistemas sobre los que se aplica. Si un modelo prioriza ciertos casos para revisión, esos casos recibirán más atención y generarán más datos. Los casos no priorizados quizá queden menos observados. Con el tiempo, el sistema puede reforzar su propia mirada.

Las decisiones del modelo pueden modificar los datos futuros.

Esto se conoce a veces como retroalimentación o feedback loop. No necesitamos estudiarlo formalmente ahora, pero sí entender la intuición: cuando un modelo influye en decisiones, también puede influir en qué información se registra después. Si no se controla, puede reforzar patrones iniciales.

Por ejemplo, si un sistema marca siempre ciertos perfiles como de mayor riesgo, esos perfiles pueden ser revisados más, generando más registros de incidentes. Otros perfiles, menos revisados, pueden parecer más seguros simplemente porque se los observa menos. Así, el modelo puede contribuir a crear datos que luego confirman su propia decisión inicial.

Un modelo en uso no solo predice la realidad.

También puede influir en cómo esa realidad se registra.

Esto muestra que el sesgo del modelo no debe analizarse solo antes del despliegue. También debe monitorearse durante el uso. Las métricas pueden cambiar, los datos pueden cambiar, los comportamientos pueden cambiar y los errores pueden concentrarse de maneras nuevas.

El sesgo en el modelo también puede aparecer en la comunicación. Si presentamos un modelo como objetivo, neutral o infalible, damos una falsa sensación de certeza. Un modelo puede ser útil, pero sigue siendo una construcción humana basada en datos, decisiones técnicas y supuestos.

Un modelo no es neutral solo por ser matemático.

Por eso, comunicar sus límites forma parte de usarlo responsablemente. Debemos explicar qué datos usó, qué objetivo optimizó, qué métricas obtuvo, qué errores comete, qué variables influyen, dónde puede fallar y qué decisiones no deberían automatizarse sin revisión.

Una forma responsable de trabajar con sesgos en el modelo incluye varias preguntas:

Para revisar este aspecto conviene preguntarse: ¿Qué métrica estamos optimizando? ¿Qué errores son más costosos? ¿El desempeño es similar en subgrupos relevantes? ¿Las variables usadas son apropiadas? ¿El objetivo representa bien el problema? ¿El modelo será monitoreado después de usarse?

Estas preguntas no eliminan todos los sesgos, pero ayudan a hacerlos visibles. Y hacer visible un problema es el primer paso para poder gestionarlo.

El sesgo en el modelo puede surgir por la métrica elegida, el umbral de decisión, las variables incluidas, el algoritmo, el sobreajuste, la definición del objetivo, la selección del modelo y el uso posterior del sistema. Un modelo puede rendir bien en promedio y aun así fallar en grupos específicos o producir decisiones problemáticas. Por eso, construir modelos responsables exige mirar más allá de la métrica global: debemos analizar errores, subgrupos, interpretabilidad, variables, objetivos y consecuencias.

Datos faltantes y representación desigual

Los datos faltantes son un problema habitual en Machine Learning, pero no siempre deben interpretarse solo como una dificultad técnica. A veces un valor faltante es simplemente un error de carga o una omisión accidental. Otras veces, en cambio, la ausencia de un dato dice algo sobre la forma en que se recolectó la información, sobre quiénes quedaron menos representados o sobre qué situaciones no fueron registradas.

Dato faltante: puede ser un error técnico, pero también una señal de cómo se produjo el dataset.

En capítulos anteriores vimos que los valores faltantes pueden tratarse con estrategias como imputación, eliminación de filas o uso de modelos que los manejan de alguna manera. Desde una mirada técnica, el objetivo era evitar que el modelo recibiera datos incompletos o que fallara durante el entrenamiento.

Pero en este capítulo necesitamos mirar un paso más atrás. Antes de decidir cómo completar o eliminar valores faltantes, conviene preguntarse por qué faltan.

Concepto	Descripción
Pregunta técnica	¿cómo trato los faltantes?
Pregunta crítica	¿por qué faltan esos datos?

No todos los faltantes tienen el mismo significado. Si una persona no responde una encuesta sobre ingresos, ese dato faltante puede estar relacionado con privacidad, desconfianza, desconocimiento, incomodidad o falta de tiempo. Si una plataforma no registra actividad de ciertos usuarios, puede deberse a problemas de conectividad, uso compartido de dispositivos o interacción por canales no registrados. Si una institución tiene información incompleta de cierto grupo, quizá ese grupo fue menos observado o peor atendido por el sistema de registro.

En esos casos, tratar el faltante como un simple espacio vacío puede ocultar un problema más profundo.

La ausencia de datos también puede contener información sobre el proceso de recolección.

Imaginemos un modelo educativo que usa datos de una plataforma virtual. Para algunos estudiantes tenemos muchas señales: ingresos, actividades, entregas, participación en foros y tiempo de conexión. Para otros, hay registros incompletos. Podríamos pensar que esos estudiantes tienen “menor actividad”, pero quizás parte de la actividad ocurrió fuera de la plataforma, desde otro dispositivo o con dificultades de acceso.

Si el modelo interpreta la falta de registros como desinterés o bajo compromiso, puede producir conclusiones injustas.

Pocos registros digitales no siempre significan poca participación real.

Algo parecido puede ocurrir con datos de clientes. Una empresa puede registrar muy bien las compras realizadas en línea, pero no las compras presenciales. Si ciertos clientes usan más canales presenciales, parecerán menos activos en el dataset digital. El modelo podría interpretar esa menor actividad registrada como menor interés, cuando en realidad el problema está en la cobertura de los datos.

Menos datos registrados no siempre significa menos comportamiento real.

Este punto se relaciona con la representación desigual. Un dataset tiene representación desigual cuando algunos grupos, situaciones o comportamientos aparecen mejor registrados que otros. El modelo aprende más sobre lo que está bien representado y menos sobre lo que aparece poco, aparece incompleto o no aparece.

Representación desigual: algunos grupos o situaciones aparecen mejor registrados que otros.

La representación desigual puede producir modelos que funcionan bien en promedio, pero mal en ciertos grupos. Si la mayoría de los datos proviene de un tipo de usuario, zona, dispositivo, edad, canal o contexto, el modelo puede generalizar peor a casos distintos.

Por ejemplo, si un modelo comercial se entrena principalmente con datos de clientes urbanos y luego se aplica a clientes de zonas rurales, puede fallar más. Si un modelo se entrena con registros de usuarios que usan una app moderna y luego se aplica a usuarios que interactúan por canales más limitados, también puede haber problemas.

El proceso parte de Más representación, continúa con más ejemplos para aprender, luego Menos representación, luego más incertidumbre y conduce finalmente a y mayor riesgo de error..

La representación desigual no siempre es evidente al mirar el tamaño total del dataset. Podemos tener miles o millones de filas y aun así tener pocos casos de ciertos grupos. Un dataset grande no garantiza representación adecuada.

Disponer de muchos datos no significa necesariamente que sean representativos.

Por eso, al explorar un dataset, conviene revisar no solo cuántas filas hay en total, sino también cómo se distribuyen esas filas. ¿Hay grupos con pocos ejemplos? ¿Hay zonas con poca cobertura? ¿Hay categorías raras? ¿Hay períodos temporales con menos registros? ¿Hay variables faltantes concentradas en ciertos sectores?

Estas preguntas ayudan a detectar problemas que una métrica global podría ocultar.

Explorar representación: cuántos casos hay, dónde faltan datos, qué grupos aparecen poco, qué períodos están incompletos.

En clasificación, la representación desigual puede aparecer como desbalance de clases. Si una clase aparece mucho más que otra, el modelo puede aprender principalmente la clase mayoritaria. Pero la representación desigual también puede ocurrir dentro de cada clase.

Por ejemplo, en un modelo de detección de fraude, puede haber suficientes casos de fraude en general, pero muy pocos de ciertos tipos de fraude nuevos o menos frecuentes. En un modelo educativo, puede haber suficientes estudiantes aprobados y desaprobados, pero poca diversidad de contextos de acceso, escuelas, turnos o trayectorias.

El desbalance no solo puede estar entre clases.

También puede estar dentro de grupos, contextos o subpoblaciones.

Esto vuelve importante revisar el desempeño por segmentos relevantes cuando sea posible. Un modelo puede tener buen resultado general y fallar en los casos menos representados. Si esos casos son importantes para la decisión, no deberíamos ignorarlos.

También debemos tener cuidado al imputar datos faltantes. Imputar significa completar valores ausentes usando alguna estrategia, como la media, la mediana, la moda o un modelo. Esta técnica puede ser útil, pero si los faltantes no ocurren al azar, la imputación puede ocultar diferencias importantes.

Imputar puede resolver un problema técnico, pero no necesariamente un problema de representación.

Por ejemplo, si los ingresos faltan principalmente en un grupo específico y los completamos con la media general, podemos introducir valores poco realistas para ese grupo. Si los tiempos de conexión faltan por problemas de registro en ciertos dispositivos, imputarlos como si fueran valores normales puede ocultar una falla del sistema de medición.

Esto no significa que no debamos imputar. Significa que debemos hacerlo conscientemente, documentando la decisión y revisando si los faltantes tienen un patrón.

Antes de imputar, conviene revisar si los faltantes se concentran en ciertos grupos o situaciones.

Una práctica útil es crear indicadores de faltantes. Por ejemplo, además de imputar una variable, podemos agregar una columna que indique si ese valor estaba originalmente ausente. Esto permite que el modelo capture, si corresponde, que la ausencia del dato también puede tener significado.

```
datos["ingresos_faltante"] = datos["ingresos"].isna().astype(int)
```

Sin embargo, también esta decisión debe pensarse con cuidado. Si la ausencia de un dato está relacionada con condiciones sensibles o desigualdades de acceso, permitir que el modelo use ese indicador puede tener consecuencias. Puede mejorar la predicción, pero también puede reforzar diferencias que deberíamos analizar éticamente.

Un indicador de faltante puede aportar información, pero también puede capturar desigualdades del proceso de registro.

Otra estrategia frecuente es eliminar filas con datos faltantes. Esto puede parecer limpio, pero puede empeorar la representación desigual. Si eliminamos muchas filas de un grupo que ya estaba poco representado, el modelo aprenderá todavía menos sobre él.

Eliminar filas incompletas puede reducir la representación de ciertos grupos.

Por eso, eliminar datos no debería ser una decisión automática. Debemos revisar cuántas filas se pierden, qué características tienen y si la eliminación cambia la distribución del dataset.

También puede haber columnas completas con mucha ausencia. A veces conviene descartarlas porque contienen demasiados faltantes. Pero antes de hacerlo, conviene preguntarse si esa ausencia se debe a un problema de medición o a que la variable solo aplica a ciertos casos. En algunos contextos, que una variable no aplique también puede ser información.

Un dato faltante puede significar: no se midió, no respondió, no aplica, se perdió, o no fue registrado.

Distinguir estas situaciones es importante. No es lo mismo que falte la edad por error de carga que que falte una fecha porque el evento nunca ocurrió. No es lo mismo no registrar una compra que registrar que no hubo compra. En datos reales, estas diferencias pueden tener consecuencias importantes.

La representación desigual también puede aparecer en los datos de evaluación. Si el conjunto de prueba no representa adecuadamente los casos donde se usará el modelo, las métricas pueden ser optimistas. Podemos creer que el modelo generaliza bien porque el test se parece al entrenamiento, pero luego fallar en contextos menos representados.

Una evaluación confiable también necesita datos de prueba representativos.

Esto es especialmente importante cuando el modelo se aplicará en distintos lugares, períodos o grupos. Si entrenamos y evaluamos solo con datos de una región, no deberíamos asumir automáticamente que funcionará igual en otra. Si los datos provienen de un período estable, no deberíamos asumir que funcionarán igual durante una crisis o un cambio fuerte de comportamiento.

Los datos faltantes y la representación desigual son problemas que van más allá de la limpieza técnica. Los faltantes pueden revelar errores de registro, diferencias de acceso, respuestas incompletas o procesos de medición desiguales. La representación desigual puede hacer que un modelo funcione bien en promedio, pero mal en grupos poco registrados. Por eso, antes de imputar, eliminar o entrenar, debemos revisar quiénes están representados, quiénes faltan, dónde se concentran los valores ausentes y cómo esas decisiones pueden afectar las predicciones. En Machine Learning, lo que falta en los datos también puede ser parte de la historia que necesitamos entender.

Variables sensibles

Al construir modelos de Machine Learning, no todas las variables tienen el mismo nivel de riesgo. Algunas columnas pueden describir características especialmente

delicadas de las personas o de los grupos. A estas variables suele llamárselas variables sensibles.

Una variable sensible es aquella que, si se usa de manera incorrecta, puede contribuir a decisiones injustas, discriminatorias o difíciles de justificar. No se trata solo de si la variable ayuda a predecir. También importa si es apropiado usarla para el tipo de decisión que queremos tomar.

Variable sensible: información que puede afectar derechos, oportunidades o trato hacia personas o grupos.

Ejemplos frecuentes de variables sensibles son edad, género, nacionalidad, origen étnico, religión, discapacidad, salud, situación socioeconómica o lugar de residencia. El listado exacto depende del contexto, de la legislación aplicable y del tipo de decisión involucrada. Lo importante es entender la idea general: hay datos que requieren un cuidado especial.

En algunos proyectos, estas variables pueden estar explícitamente presentes. Por ejemplo, una columna puede llamarse `edad`, `genero`, `pais_origen` o `zona_residencia`. En otros casos, la información sensible puede aparecer de manera indirecta, a través de variables que funcionan como aproximaciones.

Concepto	Descripción
Variable sensible explícita	edad.
Posible variable proxy	tipo de dispositivo, zona, horario de conexión u otros datos relacionados.

Como vimos en secciones anteriores, una variable proxy es una columna que no mide directamente una característica sensible, pero puede estar asociada con ella. Esto vuelve más complejo el problema, porque eliminar la variable sensible explícita no siempre alcanza para evitar que el modelo use información relacionada.

Por ejemplo, supongamos que decidimos no usar nivel socioeconómico en un modelo. Aun así, el modelo podría apoyarse en zona de residencia, tipo de institución, tipo de dispositivo, canal de compra o forma de pago, si esas variables están relacionadas con condiciones socioeconómicas.

Eliminar una variable sensible no garantiza eliminar toda la información relacionada con ella.

Esto no significa que toda variable relacionada deba eliminarse automáticamente. Significa que debemos analizar qué representa, cómo se usará y qué consecuencias

puede tener. Una variable puede ser predictiva, pero no necesariamente adecuada para una decisión.

Por ejemplo, la zona geográfica puede ser relevante para planificar logística, estimar tiempos de entrega o analizar cobertura. Pero puede ser problemática si se usa para limitar oportunidades, negar acceso o clasificar personas de manera injusta. La misma variable puede tener riesgos distintos según el objetivo.

Una variable no es riesgosa solamente por su nombre; el riesgo depende también del uso que se le dará.

En Machine Learning, esta distinción es fundamental. El modelo no sabe si una variable es ética, justa o apropiada. Solo detecta patrones estadísticos. Si una variable mejora la métrica, el modelo puede usarla. La responsabilidad de decidir si corresponde usarla es humana.

Concepto	Descripción
El modelo pregunta	¿esta variable ayuda a predecir?
Nosotros debemos preguntar	¿es correcto usarla para decidir?

Imaginemos un sistema que prioriza estudiantes para recibir acompañamiento. Si usamos variables de actividad en una plataforma, debemos preguntarnos si esas variables reflejan realmente necesidades pedagógicas o si también capturan diferencias de conectividad, disponibilidad de dispositivos, horarios familiares o condiciones de vida. Si no hacemos esa revisión, podemos confundir desigualdad de acceso con falta de compromiso.

En un sistema comercial, un modelo puede detectar que ciertas zonas compren menos cierto producto. Esa información puede ser útil para entender mercados. Pero si se usa para excluir sistemáticamente a clientes de promociones, créditos, beneficios o atención, puede generar decisiones injustas o reforzar desigualdades.

Una señal predictiva puede ser útil para analizar, pero riesgosa para decidir.

También hay contextos donde una variable sensible puede usarse justamente para auditar. Por ejemplo, quizá no queramos que el modelo use cierta variable para predecir, pero sí necesitemos revisarla para evaluar si el desempeño es desigual entre grupos. En ese caso, la variable puede ser importante para el análisis de equidad, aunque no forme parte de las variables de entrada del modelo.

Una variable sensible puede excluirse del entrenamiento y, aun así, utilizarse para auditar los resultados.

Esta diferencia es importante. No registrar variables sensibles puede proteger privacidad en algunos casos, pero también puede dificultar la detección de desigualdades. Si no tenemos forma de comparar el desempeño entre grupos relevantes, puede ser difícil saber si el modelo falla más en algunos de ellos.

Por eso, el manejo de variables sensibles no tiene una receta única. Depende del problema, del marco legal, del riesgo, de la finalidad del modelo y de las políticas de la organización. Lo que sí debe evitarse es usarlas sin reflexión o ignorar que pueden existir proxies.

Variables sensibles: no deben usarse ni descartarse sin análisis del contexto.

Otra dificultad aparece cuando el objetivo del modelo está relacionado con decisiones históricas. Si usamos datos pasados para predecir qué personas recibirán cierta oportunidad, y esas oportunidades estuvieron distribuidas de manera desigual, el modelo puede aprender patrones injustos aunque no use variables sensibles explícitas.

Por ejemplo, si en el pasado ciertos grupos recibieron menos atención, menos créditos, menos recomendaciones o menos aprobaciones, un modelo entrenado sobre esas decisiones puede reproducir esa desigualdad. Eliminar una columna sensible no corrige necesariamente ese problema.

Si las etiquetas históricas ya contienen sesgos, el modelo puede aprenderlos aunque no use variables sensibles explícitas.

Esto muestra que la revisión de variables sensibles debe acompañarse con una revisión de etiquetas, métricas, errores y consecuencias. El problema no está solo en una columna aislada. Está en el flujo completo de datos y decisiones.

También conviene analizar los errores. Un modelo puede tener buen desempeño general, pero cometer más falsos positivos o falsos negativos en ciertos grupos. Si esos errores tienen consecuencias importantes, la diferencia debe tomarse en serio.

No alcanza con preguntar qué variables usa el modelo; también debemos analizar a quién perjudican sus errores.

Por ejemplo, en un modelo de detección de riesgo, un falso positivo puede hacer que una persona sea revisada innecesariamente. Un falso negativo puede hacer que alguien no reciba ayuda. El impacto de cada error depende del contexto, y puede afectar de manera desigual.

En problemas sensibles, una buena práctica es separar la utilidad predictiva de la aceptabilidad de uso. Primero podemos reconocer que una variable tiene señal. Luego debemos decidir si usar esa señal es compatible con el propósito del sistema.

Una variable puede ser: predictivamente útil y éticamente problemática.

También es importante documentar estas decisiones. Si se excluye una variable sensible, conviene explicar por qué. Si se conserva, también. Si se usa solo para auditoría, debe quedar claro. La documentación ayuda a que el modelo sea revisable y a que las decisiones no queden ocultas en el código.

Documentar: qué variables sensibles existen, cuáles se usaron, cuáles se excluyeron, y con qué justificación.

Además, las variables sensibles deben manejarse con cuidado en términos de privacidad. No siempre es necesario ni adecuado recolectar más información personal. La minimización de datos es una idea importante: usar solo la información necesaria para el objetivo legítimo del análisis. Cuantos más datos sensibles se recolectan, mayor responsabilidad existe sobre su protección y uso.

Más datos sensibles implican más responsabilidad sobre privacidad, seguridad y uso adecuado.

Este punto se conecta con una regla general: no todo lo que se puede medir debe usarse. Un dataset puede contener muchas columnas, pero construir un modelo responsable exige seleccionar con criterio. Algunas variables pueden descartarse no porque sean inútiles para predecir, sino porque no son apropiadas para la decisión.

Las variables sensibles requieren una revisión especial porque pueden afectar derechos, oportunidades o trato hacia personas y grupos. Pueden aparecer de forma explícita o indirecta mediante proxies. Eliminarlas no siempre elimina el sesgo, y usarlas sin cuidado puede reforzar desigualdades. En algunos casos pueden excluirse del entrenamiento, pero usarse para auditar el desempeño. La decisión depende del contexto, del riesgo, de la finalidad del modelo y de las consecuencias de sus errores. Trabajar responsablemente con Machine Learning implica no solo preguntar qué variables predicen mejor, sino también cuáles es correcto usar y bajo qué condiciones.

Correlación y causalidad

Una de las confusiones más frecuentes al interpretar modelos es confundir correlación con causalidad. Esta diferencia es fundamental para trabajar con Machine Learning de manera responsable.

Dos variables están correlacionadas cuando tienden a variar juntas. Si una aumenta y la otra también suele aumentar, hablamos de una correlación positiva. Si una aumenta y la otra tiende a disminuir, hablamos de una correlación negativa. Pero que dos variables estén relacionadas no significa necesariamente que una cause la otra.

Concepto	Descripción
Correlación	dos variables se mueven juntas.
Causalidad	una variable produce un efecto sobre otra.

Por ejemplo, en un dataset de viviendas, la superficie suele estar correlacionada con el precio. Las viviendas más grandes tienden a ser más caras. En este caso, esa relación tiene sentido: la superficie puede formar parte de los factores que influyen en el precio. Pero aun así, el precio no depende solo de la superficie. También intervienen ubicación, estado, antigüedad, demanda, servicios, contexto económico y muchas otras variables.

Superficie y precio pueden estar correlacionados.

Pero el precio no se explica solo por la superficie.

En otros casos, la relación puede ser más engañosa. Supongamos que un modelo encuentra que los estudiantes con más tiempo de conexión en una plataforma tienen mejores resultados. Podríamos sentir la tentación de concluir que conectarse más tiempo causa mejores resultados. Pero esa conclusión puede ser apresurada.

Quizá los estudiantes más organizados se conectan más y también estudian más fuera de la plataforma. Quizá quienes tienen mejor conectividad pueden usar más recursos digitales. Quizá algunos estudiantes se conectan poco porque descargan materiales y estudian sin permanecer en línea. La correlación existe en los datos, pero la causa real puede ser más compleja.

Más conexión puede estar asociada a mejor resultado.

Pero eso no prueba que la conexión sea la causa directa.

Machine Learning suele trabajar muy bien con correlaciones. De hecho, muchos modelos predictivos se basan en encontrar asociaciones útiles entre variables de

entrada y resultados. Si una variable ayuda a anticipar una clase o un valor, el modelo puede usarla aunque no sea una causa.

Esto no es necesariamente un problema para predecir. Un modelo puede ser útil aunque use señales correlacionadas. El problema aparece cuando interpretamos esas señales como explicaciones causales o cuando tomamos decisiones como si modificar esa variable fuera a producir automáticamente el resultado deseado.

Una variable puede ayudar a predecir sin ser la causa del fenómeno.

Por ejemplo, si un modelo de abandono de clientes detecta que la cantidad de reclamos está asociada con mayor riesgo de abandono, puede usar esa variable para anticipar casos. Pero no podemos concluir automáticamente que “los reclamos causan abandono”. Tal vez los reclamos son consecuencia de problemas previos con el servicio. Tal vez los clientes reclaman cuando ya están insatisfechos. Tal vez el reclamo es una señal visible de un malestar más profundo.

Reclamos pueden ser señal de abandono, pero no necesariamente la causa principal.

Esta distinción tiene consecuencias prácticas. Si interpretamos mal la relación, podríamos tomar malas decisiones. Si creemos que el problema son los reclamos en sí, podríamos intentar reducir la cantidad de reclamos dificultando los canales de atención. Eso bajaría el número registrado, pero no resolvería la insatisfacción. Incluso podría empeorarla.

En cambio, si entendemos los reclamos como una señal, podemos usarlos para investigar qué problemas del servicio están generando malestar.

Concepto	Descripción
Mala interpretación	reducir el registro de reclamos.
Mejor interpretación	usar los reclamos para detectar problemas reales.

Otro ejemplo aparece en detección de fraude. Una variable puede estar correlacionada con transacciones fraudulentas, como horario, ubicación o tipo de operación. El modelo puede usar esa información para priorizar revisiones. Pero no significa que el horario cause el fraude. Significa que, en los datos disponibles, ciertos patrones horarios aparecen asociados a mayor riesgo.

En este contexto, la correlación puede ser útil para decidir qué revisar, pero debe manejarse con cuidado para evitar conclusiones injustas o simplistas.

Concepto	Descripción
Correlación útil	puede orientar una alerta.

Concepto	Descripción
Causalidad no demostrada	no alcanza para explicar por qué ocurre el evento.

La diferencia entre correlación y causalidad también es importante cuando pensamos en intervenciones. Un modelo predictivo puede decirnos qué casos tienen mayor riesgo, pero no siempre nos dice qué acción reducirá ese riesgo.

Supongamos que un modelo educativo identifica estudiantes con baja participación en foros como grupo de mayor riesgo. Eso no significa necesariamente que obligar a participar en foros mejore el aprendizaje. Tal vez la baja participación sea una señal de otras dificultades: falta de tiempo, inseguridad, problemas de comprensión, falta de conectividad o poca familiaridad con la plataforma.

Concepto	Descripción
Modelo predictivo	quiénes parecen estar en riesgo.
Análisis causal	qué intervención podría modificar ese riesgo.

Para responder preguntas causales necesitamos otros enfoques. Puede ser necesario diseñar experimentos, comparar grupos de manera controlada, usar métodos estadísticos específicos, estudiar el contexto o combinar datos cuantitativos y cualitativos. Machine Learning predictivo, por sí solo, no alcanza para demostrar causalidad.

Esto no le quita valor al modelo. Simplemente ubica sus resultados en el lugar correcto. Un modelo puede ser muy útil para detectar patrones, anticipar casos, priorizar recursos o generar hipótesis. Pero debemos evitar convertir esas hipótesis en causas demostradas sin análisis adicional.

El Machine Learning predictivo puede sugerir hipótesis, pero no demuestra causas por sí solo.

También debemos tener cuidado con variables que parecen muy importantes. Una variable puede tener alta importancia en un modelo porque está fuertemente asociada con la variable objetivo. Pero esa importancia predictiva no nos dice si intervenir sobre ella cambiará el resultado.

Por ejemplo, si `dias_desde_ultima_compra` es importante para predecir abandono, eso no significa que simplemente lograr una compra aislada resolverá el problema. Puede ser que esos días sin compra reflejen pérdida de interés, problemas económicos, mala experiencia previa o cambio de necesidades.

Que una variable sea importante para el modelo no significa automáticamente que sea una palanca de intervención.

Esta diferencia es clave para tomar decisiones. Un modelo puede ayudar a identificar dónde mirar. Pero decidir qué hacer requiere entender el fenómeno.

También existen correlaciones espurias. Son relaciones que aparecen en los datos, pero que no tienen una conexión significativa con el fenómeno. Pueden surgir por azar, por un período particular, por errores de medición o por variables que coinciden temporalmente sin relación real.

Correlación espuria: relación aparente que no representa una conexión significativa.

Si un modelo encuentra una correlación espuria en los datos de entrenamiento, puede usarla para predecir aparentemente bien, pero fallar cuando cambie el contexto. Esto se relaciona con el sobreajuste y con la necesidad de evaluar en datos no vistos.

Por ejemplo, si durante una campaña especial cierto producto se vendió mucho en una región y luego usamos esos datos para predecir comportamiento general, el modelo puede aprender una relación temporal que no se mantiene después. La correlación existía en ese período, pero no era una regla estable.

Una correlación observada en el pasado puede dejar de ser útil si cambia el contexto.

Por eso, además de medir desempeño, conviene revisar si las relaciones aprendidas tienen sentido. La interpretación de modelos puede ayudar a detectar variables sospechosas o patrones difíciles de justificar. Si una variable inesperada aparece como muy importante, no deberíamos aceptarla sin pensar. Debemos investigarla.

El proceso parte de Variable inesperadamente importante, continúa con no aceptar automáticamente, luego revisar datos,, luego contexto y conduce finalmente a y posible fuga o sesgo..

La fuga de datos también puede producir relaciones engañosas. Si una variable contiene información posterior al evento que queremos predecir, el modelo puede encontrar una relación muy fuerte. Esa relación no es útil ni legítima para predecir en la práctica.

Por ejemplo, si intentamos predecir abandono y usamos una variable como `fecha_baja`, tendremos una correlación casi perfecta con la respuesta. Pero esa variable no estaría disponible antes de que ocurra el abandono. No es una causa ni una señal anticipada: es información posterior.

Una relación fuerte no siempre indica que la variable sea válida, porque también podría existir fuga de datos.

En problemas sensibles, confundir correlación con causalidad puede tener consecuencias graves. Si un modelo encuentra que cierta zona está asociada con mayor riesgo, no podemos concluir que vivir en esa zona cause el riesgo. Puede haber factores históricos, económicos, institucionales o de medición detrás de esa relación. Usarla sin cuidado puede reforzar desigualdades.

Una asociación en los datos puede reflejar condiciones sociales, históricas o de medición.

Por eso, al comunicar resultados, conviene usar un lenguaje prudente. En lugar de decir “esta variable provoca el resultado”, muchas veces es mejor decir “esta variable está asociada con la predicción”, “el modelo la usa como señal” o “aparece relacionada en estos datos”.

Concepto	Descripción
Lenguaje prudente	"se asocia con" "aparece como señal" "el modelo se apoya en"
Lenguaje riesgoso	"causa" "demuestra" "garantiza"

Este cuidado no debilita el análisis. Al contrario, lo vuelve más honesto. Reconocer que un modelo predictivo no demuestra causalidad evita malas decisiones y mejora la calidad de las conclusiones.

Correlación y causalidad no son lo mismo. Una variable puede estar asociada con una predicción y ser útil para un modelo sin ser la causa del fenómeno. Machine Learning detecta patrones y relaciones en los datos, pero no demuestra automáticamente qué produce qué. Para tomar decisiones responsables, debemos distinguir señales predictivas de explicaciones causales, revisar el contexto, evitar afirmaciones exageradas y recordar que un buen modelo para predecir no siempre indica cuál es la mejor intervención.

Impacto de las decisiones automatizadas

Cuando un modelo de Machine Learning se usa solo para explorar datos, su impacto puede ser limitado. Pero cuando sus predicciones se convierten en decisiones, recomendaciones, alertas, prioridades o acciones concretas, el efecto puede ser mucho mayor. En ese momento, el modelo deja de ser solo una herramienta técnica y pasa a formar parte de un sistema de decisión.

Una decisión automatizada ocurre cuando un sistema usa datos y reglas algorítmicas para decidir o influir en lo que sucede después. A veces la decisión es completamente

automática. Otras veces, el modelo no decide por sí solo, pero orienta fuertemente la decisión humana.

El proceso parte de Modelo, continúa con predicción o puntuación, luego decisión, alerta o recomendación y conduce finalmente a impacto sobre personas o procesos.

Por ejemplo, un modelo puede ordenar clientes según riesgo de abandono, priorizar transacciones para revisión, recomendar productos, detectar posibles fraudes, clasificar reclamos, señalar estudiantes que podrían necesitar apoyo o estimar la probabilidad de que una solicitud sea aprobada.

En todos esos casos, la predicción no queda aislada. Se usa para hacer algo.

Una predicción puede convertirse en: prioridad, alerta, recomendación, rechazo, aprobación, seguimiento o intervención.

Por eso, al evaluar un modelo, no alcanza con mirar si predice bien en términos estadísticos. También debemos preguntar qué consecuencias tiene su uso. Un falso positivo o un falso negativo no son solo celdas de una matriz de confusión. Pueden transformarse en acciones concretas.

Supongamos un modelo que detecta posibles fraudes. Un falso positivo ocurre cuando una operación legítima es marcada como sospechosa. Eso puede generar una revisión innecesaria, una demora, una incomodidad para el usuario o incluso el bloqueo temporal de una operación válida. Un falso negativo ocurre cuando una operación fraudulenta no es detectada. Eso puede generar pérdidas económicas o problemas de seguridad.

Concepto	Descripción
Falso positivo	señalar un caso que no correspondía.
Falso negativo	no señalar un caso que sí importaba.

La importancia de cada error depende del contexto. En algunos problemas, un falso positivo puede ser tolerable si solo genera una revisión rápida. En otros, puede ser muy grave si produce una sanción, una exclusión o una pérdida de oportunidad. Del mismo modo, un falso negativo puede ser menor en ciertos casos y crítico en otros.

Esto significa que las métricas deben interpretarse junto con el impacto real de las decisiones.

La métrica indica cuánto falla el modelo, mientras que el contexto permite entender qué significa ese error.

En problemas sensibles, las decisiones automatizadas pueden afectar oportunidades, acceso a recursos, trato institucional, seguimiento educativo, condiciones financieras o reputación de personas. Por eso, la exigencia de revisión debe ser mayor.

Un modelo usado para recomendar una canción no tiene el mismo nivel de riesgo que un modelo usado para priorizar becas, detectar abandono escolar, evaluar solicitudes o señalar conductas sospechosas. Aunque ambos sean modelos predictivos, sus consecuencias son muy distintas.

No todas las predicciones tienen el mismo impacto.

A mayor consecuencia, mayor responsabilidad.

También debemos considerar que una decisión automatizada puede parecer neutral porque se apoya en números. Sin embargo, esos números provienen de datos, variables, modelos, métricas, umbrales y decisiones humanas. El hecho de que un sistema sea automático no lo vuelve objetivo por sí mismo.

Que una decisión sea automática no significa que sea neutral, justa o infalible.

Un modelo puede reproducir sesgos de los datos. Puede apoyarse en variables inadecuadas. Puede fallar más en ciertos grupos. Puede estar desactualizado. Puede haber sido evaluado con una métrica que no refleja bien el problema. Si sus resultados se usan para decidir, esos errores pueden tener consecuencias reales.

Por eso, antes de automatizar una decisión, debemos preguntarnos si el modelo está listo para ese uso. Una cosa es explorar un patrón en un cuaderno. Otra muy distinta es incorporarlo a un proceso que afecta personas o recursos.

Explorar con un modelo no es lo mismo que delegarle una decisión.

Muchas veces, una opción más prudente es usar el modelo como apoyo a la decisión, no como reemplazo completo. El sistema puede priorizar casos para revisión, sugerir alertas o mostrar información relevante, pero una persona debe analizar el contexto antes de actuar.

Esto es especialmente importante cuando el modelo puede equivocarse de manera costosa o injusta. La revisión humana no debe ser decorativa. Debe tener capacidad real de cuestionar, corregir o rechazar la recomendación del sistema.

Concepto	Descripción
Modelo como apoyo	sugiere, ordena o alerta.
Revisión humana	interpreta, contextualiza y decide con responsabilidad.

Sin embargo, también debemos ser honestos: incluir una persona en el proceso no garantiza automáticamente una decisión justa. Si la persona acepta siempre la recomendación del modelo sin revisarla, la supervisión humana se vuelve superficial. Si no tiene información suficiente para cuestionar el resultado, tampoco puede cumplir bien su rol.

Por eso, un sistema responsable necesita explicaciones, documentación, criterios claros y mecanismos de revisión.

Supervisión humana real requiere: información, criterio, tiempo, autoridad y posibilidad de corregir.

Otro aspecto importante es el umbral de decisión. En muchos modelos de clasificación, la salida puede ser una probabilidad o puntuación. Para convertirla en una acción, necesitamos definir un umbral. Por ejemplo, revisar casos con probabilidad mayor a 0.70, enviar una alerta si supera 0.80 o priorizar los 100 casos con mayor riesgo.

Ese umbral no es solo técnico. Define cuántos casos serán intervenidos, cuántos quedarán afuera y qué tipo de errores aceptamos.

Concepto	Descripción
Umbral más bajo	más casos señalados, más falsos positivos posibles.
Umbral más alto	menos casos señalados, más falsos negativos posibles.

Elegir un umbral sin analizar las consecuencias puede producir decisiones inadecuadas. En un sistema de apoyo educativo, un umbral demasiado alto podría dejar fuera a estudiantes que necesitaban acompañamiento. Uno demasiado bajo podría saturar al equipo con demasiadas alertas. La decisión debe equilibrar recursos, riesgos y objetivos.

También puede haber impactos indirectos. Un modelo que prioriza ciertos casos puede hacer que esos casos reciban más atención, más registros y más seguimiento. Los casos no priorizados pueden quedar menos observados. Con el tiempo, esto puede cambiar los datos futuros y reforzar el comportamiento del sistema.

El modelo no solo predice.

Cuando se usa, puede modificar qué casos se observan y qué datos se registran después.

Este tipo de retroalimentación puede ser útil si se controla, pero también puede ser problemático. Si un sistema revisa siempre los mismos perfiles, encontrará más

información sobre ellos y menos sobre otros. Luego, el modelo futuro puede aprender desde datos generados por esa misma selección.

Por eso, los modelos en uso deben monitorearse. No alcanza con evaluarlos una vez antes de aplicarlos. Debemos revisar si el desempeño se mantiene, si los errores se concentran, si cambió la población, si el umbral sigue siendo adecuado y si las decisiones están produciendo efectos no deseados.

El proceso parte de Modelo en uso, continúa con monitoreo, luego revisión de errores y conduce finalmente a ajustes responsables.

También es importante comunicar el rol del modelo. Las personas que usan sus resultados deben saber qué significa la predicción, qué datos la alimentan, qué limitaciones tiene y qué no debe concluirse a partir de ella.

Por ejemplo, “alto riesgo” no significa certeza. “Anómalo” no significa culpable. “Baja probabilidad” no significa imposibilidad. “Recomendado” no significa obligatorio. Estas diferencias deben quedar claras para evitar usos excesivos del sistema.

Una puntuación del modelo no debe convertirse automáticamente en una etiqueta definitiva sobre una persona o una situación.

En contextos sensibles, también debe existir posibilidad de revisión o apelación. Si una decisión apoyada por un modelo afecta a alguien de manera importante, debería haber una forma de revisar el caso, corregir datos incorrectos o considerar información que el modelo no tuvo en cuenta.

Esto es especialmente relevante porque ningún dataset captura toda la realidad. Puede haber datos desactualizados, incompletos o mal registrados. Una persona puede tener una situación que el modelo no representa. Un proceso justo debe permitir revisar esas excepciones.

Los datos pueden estar incompletos.

Por eso, las decisiones importantes deben poder revisarse.

El impacto de las decisiones automatizadas también depende de la escala. Un error aislado puede ser corregible. Pero un modelo aplicado a miles o millones de casos puede multiplicar errores rápidamente. Una regla problemática puede afectar a muchas personas antes de que alguien la detecte.

La automatización puede multiplicar los aciertos, pero también los errores.

Esta es una de las razones por las que la responsabilidad aumenta cuando un modelo pasa de experimento a sistema real. En un cuaderno, un error puede ser parte del aprendizaje. En producción, puede afectar procesos, recursos y personas.

Las decisiones automatizadas transforman predicciones en acciones. Por eso, debemos analizar no solo la métrica del modelo, sino también las consecuencias de sus errores, el umbral usado, los grupos afectados, la posibilidad de revisión humana, los mecanismos de monitoreo y la forma de comunicación. Un modelo puede ser útil para apoyar decisiones, pero no debería presentarse como neutral, infalible o suficiente por sí mismo. Cuanto mayor sea el impacto de la decisión, mayor debe ser el cuidado en el diseño, la evaluación, la interpretación y el uso del sistema.

Responsabilidad humana

El uso de Machine Learning no elimina la responsabilidad humana. Aunque un modelo pueda procesar grandes cantidades de datos, detectar patrones y producir predicciones, las decisiones importantes no deberían delegarse sin criterio en un sistema automático. Detrás de cada modelo hay personas que eligieron los datos, definieron el problema, seleccionaron variables, entrenaron algoritmos, eligieron métricas, interpretaron resultados y decidieron cómo usar las predicciones.

Un modelo no aparece por sí solo: es el resultado de decisiones humanas sobre datos, objetivos, métricas, variables y usos.

Esta idea es central. A veces se habla de los modelos como si fueran entidades neutrales que simplemente “dicen lo que muestran los datos”. Pero los datos no hablan solos. Fueron recolectados en un contexto, con ciertos criterios, mediante determinados sistemas de registro. Además, el modelo aprende solo aquello que le permitimos ver.

Por eso, si un modelo produce una decisión injusta, no alcanza con decir “lo decidió el algoritmo”. El algoritmo forma parte de un sistema diseñado, configurado y usado por personas. La responsabilidad no desaparece porque haya código de por medio.

Decir “lo decidió el modelo” no reemplaza la responsabilidad de quienes diseñan y utilizan el sistema.

La responsabilidad humana comienza mucho antes del entrenamiento. Aparece al definir el problema. No es lo mismo predecir abandono de clientes, detectar estudiantes que podrían necesitar apoyo, estimar riesgo de fraude o priorizar solicitudes. Cada formulación implica una forma de mirar la realidad.

Si definimos mal el objetivo, el modelo puede ser técnicamente correcto y aun así producir resultados problemáticos. Por ejemplo, si confundimos baja actividad digital con bajo compromiso, el modelo aprenderá desde una definición limitada. Si usamos decisiones históricas como si fueran verdades objetivas, el modelo puede reproducir criterios pasados sin cuestionarlos.

Un problema mal formulado puede producir un modelo bien entrenado para una tarea equivocada.

También hay responsabilidad en la elección de variables. Algunas columnas pueden ser útiles para predecir, pero no apropiadas para decidir. Otras pueden funcionar como proxies de información sensible. Algunas pueden no estar disponibles en el momento real de predicción y generar fuga de datos. Revisar estas cuestiones no es una tarea que el modelo pueda resolver por sí solo.

El modelo puede detectar patrones, pero las personas deben decidir si esos patrones son válidos, apropiados y responsables.

La responsabilidad también aparece en la elección de métricas. Una métrica global puede ocultar errores importantes. Accuracy puede ser engañosa en clases desbalanceadas. Un buen promedio puede ocultar mal desempeño en subgrupos. Un modelo puede reducir falsos positivos a costa de aumentar falsos negativos, o al revés. Elegir qué error toleramos más no es solo una decisión matemática; depende del impacto del problema.

Elegir una métrica también es elegir qué tipo de error miramos con más atención.

En muchos casos, la decisión sobre el umbral de clasificación es igual de importante. Un modelo puede producir probabilidades o puntuaciones, pero alguien debe decidir a partir de qué valor se activa una alerta, se prioriza un caso o se recomienda una intervención.

Esa decisión afecta directamente a las personas o procesos involucrados. Si el umbral es demasiado bajo, habrá muchas alertas. Si es demasiado alto, algunos casos importantes quedarán fuera. No existe un umbral universalmente correcto. Debe elegirse considerando recursos, riesgos, costos de error y consecuencias.

El umbral traduce una puntuación del modelo en una decisión concreta.

La responsabilidad humana también implica revisar los resultados antes de usarlos. No alcanza con entrenar un modelo, obtener una métrica aceptable y aplicarlo automáticamente. Debemos revisar errores, falsos positivos, falsos negativos, casos límite, subgrupos, variables importantes y posibles sesgos.

Si el modelo falla sistemáticamente en cierto tipo de casos, eso debe investigarse. Si una variable inesperada domina las predicciones, también. Si el desempeño cae cuando cambia el período o el contexto, el modelo necesita revisión.

Evaluar un modelo responsablemente incluye mirar aciertos, errores, subgrupos, variables y contexto.

En sistemas sensibles, la revisión humana debe ser real. No basta con que una persona esté formalmente en el proceso si no tiene tiempo, información o autoridad para cuestionar la salida del modelo. Una supervisión humana débil puede terminar funcionando como una aprobación automática de lo que el sistema recomienda.

Para que la revisión sea significativa, las personas deben entender qué representa la predicción, qué datos se usaron, qué limitaciones tiene el modelo y qué alternativas existen.

Supervisión humana real: entender, cuestionar, corregir y decidir con criterio.

También es importante definir qué decisiones no deberían automatizarse por completo. No todos los problemas requieren el mismo nivel de intervención humana. Un modelo que recomienda productos puede aceptar más automatización. Un modelo que afecta trayectorias educativas, acceso a beneficios, condiciones financieras o revisiones institucionales necesita más cuidado.

A mayor impacto de la decisión, mayor necesidad de revisión, explicación y control humano.

La responsabilidad humana no termina cuando el modelo empieza a usarse. Los datos cambian, las personas cambian, los contextos cambian y los modelos pueden degradarse. Un sistema que funcionaba bien en un período puede fallar en otro. Una variable que era útil puede dejar de serlo. Un patrón aprendido puede volverse obsoleto.

Por eso, los modelos deben monitorearse. Debemos revisar si las métricas se mantienen, si aparecen nuevos tipos de error, si cambió la distribución de los datos y si las decisiones siguen siendo adecuadas.

El proceso parte de Modelo en uso, continúa con monitoreo, luego revisión, luego ajuste o retiro y conduce finalmente a si deja de ser confiable..

También debe existir documentación. Un modelo responsable debería tener registrada información básica: qué problema intenta resolver, con qué datos fue entrenado, qué variables usa, qué variables se excluyeron, qué métricas obtuvo, qué limitaciones se conocen y en qué contexto debería aplicarse.

La documentación no es un trámite burocrático. Permite revisar, explicar y mejorar. También evita que un modelo se use fuera del contexto para el que fue pensado.

Documentar un modelo ayuda a saber qué puede hacer, qué no puede hacer y bajo qué condiciones fue evaluado.

La responsabilidad humana también incluye comunicar con claridad. Si un modelo produce una puntuación de riesgo, quienes la usen deben entender que no es una certeza. Si detecta una anomalía, debe quedar claro que una anomalía no equivale automáticamente a fraude, error o culpa. Si sugiere un grupo o segmento, debe explicarse que es una clasificación construida a partir de ciertas variables.

Una predicción no debe comunicarse como una verdad absoluta.

El lenguaje que usamos puede cambiar la forma en que otras personas interpretan el resultado. Decir “el modelo detectó estudiantes con bajo compromiso” no es lo mismo que decir “el modelo identificó estudiantes con baja actividad registrada en la plataforma”. La primera frase interpreta de manera fuerte. La segunda describe con más precisión lo que los datos realmente muestran.

Concepto	Descripción
Más prudente	baja actividad registrada.
Más riesgoso	bajo compromiso.

Esta diferencia es importante porque los modelos suelen trabajar con indicadores parciales. Una variable puede medir algo observable, pero no capturar todo el fenómeno. Actividad registrada no es lo mismo que aprendizaje. Reclamos no son lo mismo que insatisfacción completa. Zona de residencia no es lo mismo que capacidad de pago. Cantidad de accesos no es lo mismo que esfuerzo.

La responsabilidad humana consiste, en parte, en no olvidar esas diferencias.

Los datos miden señales, no la totalidad de una persona o de un fenómeno.

También debemos pensar quién puede verse afectado por el modelo y quién participa en su diseño. En muchos proyectos, las personas que sufren las consecuencias de una decisión automatizada no son las mismas que construyen el sistema. Por eso, cuando el impacto es importante, conviene incorporar miradas del dominio, personas usuarias, equipos responsables y, cuando corresponda, marcos legales o éticos.

Un modelo no debería diseñarse solo desde la comodidad técnica. Debe pensarse dentro del proceso real donde será usado.

Un modelo responsable se diseña considerando el contexto donde actuará y las personas afectadas.

Por último, la responsabilidad humana implica reconocer cuándo no usar un modelo. A veces los datos son insuficientes, la etiqueta es demasiado problemática, el riesgo es alto, la explicación es débil o el impacto potencial no justifica la automatización. En esos casos, la decisión responsable puede ser no desplegar el modelo, limitar su uso o mantenerlo solo como herramienta exploratoria.

No todo problema debe convertirse en una decisión automatizada.

Esto no es una derrota técnica. Es una muestra de criterio. Machine Learning puede ser muy útil, pero no es siempre la respuesta adecuada.

La responsabilidad humana atraviesa todo el ciclo de Machine Learning: definir el problema, seleccionar datos, elegir variables, entrenar modelos, evaluar métricas, interpretar errores, decidir umbrales, comunicar resultados y monitorear el uso. Un modelo puede apoyar decisiones, pero no reemplaza el juicio, el contexto ni la responsabilidad de las personas. Usar Machine Learning de manera responsable implica entender que cada predicción forma parte de un sistema más amplio de datos, decisiones y consecuencias.

Decidir con datos y con criterio

En este capítulo analizamos una dimensión fundamental del Machine Learning: los modelos no existen aislados de los datos, las personas y las decisiones. Un modelo puede producir predicciones útiles, pero esas predicciones nacen de información parcial, de decisiones técnicas y de contextos concretos. Por eso, trabajar con modelos no es solo una tarea matemática o computacional. También exige responsabilidad.

Comenzamos con el sesgo en los datos. Vimos que un dataset no es una copia perfecta de la realidad, sino una representación construida. Los datos dependen de qué se midió, cómo se registró, quiénes aparecen, quiénes faltan y qué decisiones anteriores quedaron reflejadas.

El proceso parte de Datos, continúa con representación parcial y conduce finalmente a de una realidad más compleja.

Si ciertos grupos están poco representados, si algunas situaciones no se registran, si las etiquetas históricas reflejan decisiones problemáticas o si el contexto cambió, el

modelo puede aprender patrones distorsionados. En esos casos, una buena métrica no garantiza que el sistema sea justo, robusto o adecuado.

También estudiamos el sesgo en el modelo. Un modelo puede amplificar problemas presentes en los datos, pero también puede introducir sesgos por la métrica elegida, el umbral de decisión, las variables incluidas, el algoritmo, el sobreajuste o la forma en que se define el objetivo.

El modelo aprende según los datos disponibles, pero también según las decisiones de modelado.

Una métrica global puede ocultar errores importantes. Un modelo puede funcionar bien en promedio y fallar en subgrupos relevantes. Un umbral puede reducir un tipo de error y aumentar otro. Por eso, evaluar un modelo responsablemente requiere mirar más allá de un único número.

Luego analizamos los datos faltantes y la representación desigual. Los valores ausentes no siempre son un simple problema técnico. A veces revelan diferencias en el proceso de recolección, desigualdades de acceso, errores de medición o grupos menos observados.

Lo que falta en los datos también puede revelar cómo se produjo el dataset.

Imputar o eliminar datos puede ser necesario, pero esas decisiones deben revisarse. Si los faltantes se concentran en ciertos grupos, eliminarlos puede reducir todavía más su representación. Si se imputan sin analizar su origen, podemos ocultar un problema importante.

También trabajamos la idea de variables sensibles. Algunas columnas pueden afectar derechos, oportunidades o trato hacia personas y grupos. Pueden aparecer de manera explícita, como edad, género o lugar de residencia, o de manera indirecta, a través de variables proxy.

Una variable predictiva no siempre es una variable apropiada para decidir.

El modelo solo detecta patrones. No sabe si usar una variable es justo, legal o adecuado. Esa decisión corresponde a las personas responsables del sistema. En algunos casos, una variable sensible puede excluirse del entrenamiento, pero usarse para auditar si el modelo falla más en ciertos grupos. En otros casos, puede ser necesario revisar si variables aparentemente neutrales funcionan como proxies.

Una sección clave fue la diferencia entre correlación y causalidad. Machine Learning predictivo suele encontrar asociaciones útiles entre variables, pero una asociación no

demuestra una causa. Una variable puede ayudar a predecir sin ser la razón real por la que ocurre el fenómeno.

Concepto	Descripción
Correlación	una variable se asocia con otra.
Causalidad	una variable produce un efecto sobre otra.

Esta diferencia es importante para no tomar malas decisiones. Si una variable aparece como importante, podemos decir que el modelo la usa como señal. Pero no deberíamos afirmar automáticamente que modificar esa variable cambiará el resultado. Para responder preguntas causales hacen falta otros enfoques.

Después analizamos el impacto de las decisiones automatizadas. Cuando una predicción se transforma en una alerta, prioridad, recomendación, aprobación, rechazo o intervención, el modelo pasa a formar parte de un sistema de decisión. En ese punto, los errores dejan de ser solo métricas: pueden tener consecuencias reales.

El proceso parte de Predicción, continúa con decisión y conduce finalmente a consecuencia.

Por eso, debemos analizar falsos positivos, falsos negativos, umbrales, revisión humana, comunicación, monitoreo y posibilidad de corrección. A mayor impacto, mayor debe ser el cuidado.

Finalmente, trabajamos la responsabilidad humana. Un modelo no decide en el vacío. Personas definen el problema, eligen datos, seleccionan variables, entrenan modelos, eligen métricas, ajustan umbrales, comunican resultados y deciden cómo usar las predicciones.

El modelo puede apoyar decisiones, pero no reemplaza el juicio, el contexto ni la responsabilidad humana.

Decir “lo decidió el algoritmo” no alcanza. Si el sistema afecta personas, recursos u oportunidades, debe haber supervisión real, documentación, revisión, monitoreo y posibilidad de corregir errores. También debe existir la posibilidad de decidir no usar un modelo cuando los datos son insuficientes, el riesgo es alto o el problema está mal formulado.

Los modelos de Machine Learning aprenden desde datos producidos en contextos concretos y se usan dentro de sistemas de decisión. Por eso, sus resultados pueden reflejar sesgos, ausencias, desigualdades, errores de medición o decisiones históricas. Trabajar responsablemente implica revisar datos, variables, métricas, errores, subgrupos, correlaciones, umbrales, consecuencias y comunicación. Machine Learning puede ayudar a decidir mejor, pero solo si se usa con criterio, prudencia y responsabilidad humana.

Capítulo 30 · Errores comunes al construir modelos

Entrenar y evaluar con los mismos datos

Uno de los errores más comunes al construir modelos de Machine Learning es entrenar y evaluar con los mismos datos. A primera vista puede parecer una práctica razonable: tenemos un dataset, entrenamos un modelo con él y luego medimos qué tan bien predice esos mismos casos. El problema es que esa evaluación no mide lo que realmente necesitamos saber.

Cuando entrenamos un modelo, le mostramos ejemplos para que aprenda patrones. Si después lo evaluamos sobre esos mismos ejemplos, estamos revisando qué tan bien recuerda o ajusta los datos que ya vio. Pero el objetivo de Machine Learning no es funcionar bien solo sobre el pasado conocido, sino generalizar a casos nuevos.

Concepto	Descripción
Entrenar	el modelo aprende desde ciertos datos.
Evaluar	queremos saber si funciona con datos no vistos.

Si usamos los mismos datos para ambas cosas, la evaluación puede ser demasiado optimista. El modelo puede parecer muy bueno porque ya tuvo acceso a esos casos durante el entrenamiento. Esto es especialmente problemático en modelos flexibles, capaces de ajustarse mucho a los detalles del dataset.

Por ejemplo, supongamos que entrenamos un árbol de decisión muy profundo. Si le permitimos dividir los datos muchas veces, puede llegar a memorizar particularidades del conjunto de entrenamiento. Luego, si lo evaluamos sobre ese mismo conjunto, quizá obtenga una accuracy altísima. Pero eso no significa que vaya a funcionar igual con datos nuevos.

Un buen resultado en entrenamiento no garantiza un buen resultado en datos nuevos.

Este error se relaciona directamente con el sobreajuste. Un modelo sobreajustado aprende demasiado bien los detalles, ruidos o particularidades del conjunto de entrenamiento. Puede acertar mucho sobre esos datos, pero fallar cuando aparecen casos distintos.

La evaluación con los mismos datos no permite detectar bien ese problema. Al contrario, puede ocultarlo. Nos da una sensación de éxito que no se sostiene cuando el modelo se enfrenta a información nueva.

Evaluar sobre los mismos datos utilizados para entrenar puede ocultar el sobreajuste.

Por eso, una práctica básica es separar los datos en entrenamiento y prueba. El conjunto de entrenamiento se usa para ajustar el modelo. El conjunto de prueba se reserva para evaluar cómo se comporta con datos que no vio durante el entrenamiento.

El proceso parte de Datos disponibles, continúa con entrenamiento y conduce finalmente a prueba.

Este esquema no es un formalismo innecesario. Es una forma de simular, de manera controlada, lo que ocurrirá cuando el modelo reciba casos nuevos en el futuro. El conjunto de prueba actúa como una muestra de datos no vistos.

Concepto	Descripción
Train	datos para aprender.
Test	datos para evaluar generalización.

En `scikit-learn`, esta separación suele hacerse con `train_test_split`.

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.25,
    random_state=42
)
```

Luego entrenamos el modelo solo con los datos de entrenamiento.

```
modelo.fit(X_train, y_train)
```

Y evaluamos con los datos de prueba.

```
modelo.score(X_test, y_test)
```

Esta diferencia es esencial. El modelo puede aprender de `X_train` e `y_train`, pero no debería ver `X_test` ni `y_test` durante el entrenamiento. Si de alguna manera usamos información del test para entrenar, ajustar o decidir, contaminamos la evaluación.

El test debe representar datos no vistos.

Si lo usamos para aprender, deja de cumplir esa función.

Un error frecuente es mirar primero el resultado sobre entrenamiento y quedarse con esa métrica. Por ejemplo, alguien puede decir: “mi modelo tiene 98% de accuracy”, pero al revisar el código descubrimos que calculó esa accuracy sobre los mismos datos usados para entrenar. Esa métrica no es una estimación confiable de desempeño futuro.

Más útil es comparar desempeño en entrenamiento y en prueba. Si el resultado en entrenamiento es muy alto, pero en prueba baja mucho, puede haber sobreajuste. Si ambos son bajos, puede haber subajuste o falta de señal. Si ambos son razonables y similares, el modelo podría estar generalizando mejor.

El proceso parte de Entrenamiento muy alto, continúa con + prueba mucho más baja, luego posible sobreajuste., luego Entrenamiento bajo, luego + prueba baja y conduce finalmente a posible subajuste..

Esto no significa que las métricas de entrenamiento sean inútiles. Pueden servir para diagnosticar. Pero no deberían presentarse como evaluación final del modelo.

Otro error relacionado es usar todo el dataset para entrenar porque “no tenemos muchos datos”. Esta tentación es comprensible, especialmente con datasets pequeños. Pero si usamos todo para entrenar, nos quedamos sin una forma honesta de estimar generalización. En esos casos, puede ser mejor usar validación cruzada, que permite evaluar el modelo en varias particiones.

Cuando disponemos de pocos datos, la validación cruzada puede ser más informativa que una única separación.

La validación cruzada no elimina la necesidad de cuidado. Si la usamos para comparar modelos o ajustar hiperparámetros, debemos evitar que el conjunto de prueba final se convierta en parte del proceso de selección. Pero ayuda a obtener una estimación más estable cuando una única división podría ser demasiado dependiente del azar.

También debemos tener cuidado con datos temporales. Si el problema tiene una dimensión de tiempo, separar aleatoriamente puede no ser correcto. Por ejemplo, si queremos predecir ventas futuras, abandono futuro o demanda futura, no deberíamos entrenar con datos posteriores y evaluar con datos anteriores. La evaluación debe respetar la lógica temporal.

Si el problema depende del tiempo, la evaluación debe respetar el orden temporal.

En un dataset temporal, una separación razonable puede ser entrenar con datos antiguos y evaluar con datos más recientes. Esto se parece más al uso real del modelo: aprender del pasado para predecir el futuro.

El principio general es siempre el mismo: la evaluación debe parecerse al escenario real de aplicación. Si en la práctica el modelo verá clientes nuevos, documentos nuevos, transacciones nuevas o períodos futuros, la evaluación debe intentar reflejar esa situación.

Una buena evaluación simula de manera razonable el uso futuro del modelo.

Entrenar y evaluar con los mismos datos rompe ese principio. Nos dice cómo se comporta el modelo sobre ejemplos conocidos, no cómo responderá ante casos nuevos.

Este error también puede afectar la comunicación. Presentar métricas calculadas sobre entrenamiento puede generar confianza injustificada. Un modelo puede parecer listo para usarse cuando en realidad no fue evaluado correctamente. En contextos de bajo riesgo, esto puede llevar a malas decisiones. En contextos sensibles, puede ser mucho más grave.

Una métrica obtenida mediante un procedimiento inadecuado puede llevarnos a confiar en un modelo que no generaliza.

Por eso, al revisar cualquier modelo, una de las primeras preguntas debería ser:

Para revisar este punto conviene preguntarse: ¿Con qué datos fue entrenado? ¿Con qué datos fue evaluado? ¿El modelo ya había visto los casos usados para medirlo?

Si la respuesta es que entrenó y evaluó con los mismos datos, la métrica debe considerarse no confiable como estimación de desempeño futuro.

Entrenar y evaluar con los mismos datos es un error porque produce una evaluación demasiado optimista y no mide generalización. Un modelo debe aprender con datos de entrenamiento y evaluarse con datos no vistos. La separación train/test, la validación cruzada y las divisiones temporales cuando corresponda son herramientas para construir evaluaciones más honestas. El objetivo no es que el modelo recuerde bien los ejemplos que ya vio, sino que funcione de manera razonable cuando enfrente casos nuevos.

Fuga de datos

La fuga de datos, o *data leakage*, es uno de los errores más peligrosos en Machine Learning. Ocurre cuando el modelo recibe, directa o indirectamente, información que no debería estar disponible en el momento real de la predicción. Como consecuencia,

el rendimiento durante la evaluación puede parecer excelente, pero ese resultado no se sostendrá cuando el modelo se use con datos nuevos.

Existe fuga de datos cuando el modelo accede a información que no tendría disponible al momento de realizar la predicción.

La fuga de datos es peligrosa porque puede ser difícil de detectar. Muchas veces no genera errores en el código. El modelo entrena correctamente, las métricas se calculan sin problemas y los resultados parecen muy buenos. Justamente por eso puede pasar inadvertida.

El problema no es que el modelo “haga trampa” de manera intencional. El problema es que nosotros, al preparar los datos, podemos incluir información que solo se conocería después del evento que queremos predecir, o permitir que el conjunto de prueba influya en alguna etapa del entrenamiento.

El código puede ejecutarse correctamente y, aun así, la evaluación puede estar contaminada.

Un ejemplo simple aparece cuando queremos predecir si un cliente abandonará un servicio. Supongamos que en el dataset tenemos una columna llamada `fecha_baja`. Si usamos esa variable para entrenar el modelo, el resultado será artificialmente alto, porque la variable contiene información directa sobre lo que queremos predecir.

Concepto	Descripción
Queremos predecir	¿el cliente abandonará?
Variable problemática	<code>fecha_baja</code>

En la práctica, antes de que el cliente abandone, no conoceríamos la fecha de baja. Por lo tanto, esa variable no puede usarse para anticipar el abandono. Si la incluimos, el modelo no aprende señales previas; aprende información posterior.

Algo similar puede ocurrir con variables más sutiles. Una columna como `días_desde_baja`, `motivo_baja`, `recuperado_despues_de_baja` o `encuesta_post_baja` también podría filtrar información del futuro. Aunque no sean exactamente la variable objetivo, contienen pistas que solo existen después del evento.

Las variables posteriores al evento pueden filtrar información sobre la respuesta.

La pregunta clave es temporal: ¿esta variable estaría disponible en el momento en que el modelo debe hacer la predicción? Si la respuesta es no, entonces no debería usarse como variable de entrada.

La pregunta central es si esa variable existe antes del momento en que debe realizarse la predicción.

La fuga de datos también puede ocurrir durante el preprocesamiento. Este caso es muy frecuente y menos evidente. Si calculamos transformaciones usando todo el dataset antes de separar entrenamiento y prueba, el conjunto de prueba influye en el proceso de aprendizaje.

Por ejemplo, supongamos que escalamos una variable usando la media y la desviación estándar calculadas sobre todos los datos. Luego separamos en train y test. En ese caso, la media y la desviación usada para transformar el entrenamiento ya contienen información del test.

Concepto	Descripción
Incorrecto	escalar todo el dataset y después separar.
Correcto	separar primero, aprender la escala con train y aplicarla a test.

La diferencia puede parecer pequeña, pero conceptualmente es importante. El conjunto de prueba debe simular datos no vistos. Si usamos sus valores para calcular medias, desviaciones, imputaciones, codificaciones o selecciones de variables, deja de ser completamente independiente.

Por eso, las transformaciones que “aprenden” algo de los datos deben ajustarse solo con el conjunto de entrenamiento. Luego se aplican al conjunto de prueba usando lo aprendido en entrenamiento.

Concepto	Descripción
fit	aprender desde entrenamiento.
transform	aplicar lo aprendido a entrenamiento o prueba.

Este principio aparece en muchas transformaciones. La imputación aprende medias, medianas o modas. El escalado aprende medias, desviaciones, mínimos o máximos. One-Hot Encoding aprende categorías presentes. La selección de variables puede aprender qué columnas parecen más importantes. PCA aprende componentes. Todas estas etapas deben aprenderse sin mirar el test.

La imputación, el escalado, la codificación, la selección de variables o PCA pueden producir fuga de datos si se ajustan utilizando el conjunto de prueba.

Por eso usamos `Pipeline`. Un pipeline permite encadenar preprocesamiento y modelo de manera correcta. Cuando lo entrenamos con `fit`, cada transformación

aprende solo desde los datos de entrenamiento. Cuando evaluamos, aplica esas transformaciones al test sin volver a aprender desde él.

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

pipeline = Pipeline([
    ("scaler", StandardScaler()),
    ("modelo", LogisticRegression())
])

pipeline.fit(X_train, y_train)
pipeline.score(X_test, y_test)
```

En este ejemplo, `StandardScaler` calcula la media y la desviación estándar usando solo `X_train`. Luego transforma `X_test` con esos valores ya aprendidos. Así evitamos que el conjunto de prueba influya en el entrenamiento.

También puede haber fuga de datos al hacer búsqueda de hiperparámetros. Si usamos el conjunto de prueba para probar muchas combinaciones y elegir la mejor, ese test deja de ser una evaluación final independiente. En la práctica, lo estamos usando para tomar decisiones.

Si usamos el conjunto de prueba para elegir el modelo, deja de ser una evaluación final independiente.

La forma correcta es usar entrenamiento y validación, o validación cruzada, para ajustar hiperparámetros. El conjunto de prueba debe reservarse para el final, cuando ya elegimos el enfoque.

Concepto	Descripción
Train	aprender.
Validación	comparar y ajustar.
Test	evaluar al final.

La fuga también puede aparecer cuando hay registros relacionados entre sí. Por ejemplo, si tenemos varias filas del mismo cliente, estudiante, paciente, producto o dispositivo, y algunas quedan en entrenamiento y otras en prueba, el modelo puede aprender información específica de esa entidad. Luego parece generalizar, pero en realidad está reconociendo casos muy parecidos a los que ya vio.

La presencia de filas relacionadas puede hacer que el conjunto de prueba no sea verdaderamente independiente.

En esos casos, puede ser necesario separar por grupo. Por ejemplo, asegurarse de que todos los registros de un mismo cliente queden solo en entrenamiento o solo en prueba. Lo importante es que la evaluación represente el escenario real de uso.

También hay fuga cuando se agregan datos calculados usando información futura. Supongamos que queremos predecir si una persona comprará el próximo mes, pero usamos como variable el total de compras del año completo. Si ese total incluye meses posteriores al momento de predicción, estamos usando información del futuro.

Concepto	Descripción
Variable agregada	total_compras_anual
Problema	si incluye compras futuras, puede haber fuga de datos.

Este error es común en variables agregadas. Totales, promedios, rankings, tasas, recuentos o historiales deben calcularse respetando el momento de predicción. Solo debería usarse información disponible hasta ese momento.

Para predecir el futuro, las variables deben construirse únicamente con información disponible en el pasado.

La fuga de datos también puede estar escondida en variables generadas después de una decisión. Por ejemplo, si queremos predecir si una solicitud será aprobada, no podemos usar variables que se completan durante o después del proceso de aprobación. Si queremos predecir riesgo antes de una revisión, no podemos usar datos producidos por esa revisión.

Si una variable se genera después de la decisión, no puede utilizarse para predecir antes de que esa decisión ocurra.

Una señal de alerta es obtener resultados demasiado buenos para el problema. Si un modelo logra métricas casi perfectas en un problema realista y complejo, conviene sospechar. Puede ser que el problema sea realmente fácil, pero también puede haber fuga de datos, duplicados, etiquetas filtradas o una separación incorrecta.

Los resultados demasiado buenos para ser ciertos merecen una revisión cuidadosa.

No deberíamos celebrar automáticamente una métrica altísima. Debemos revisar qué variables se usaron, cómo se separaron los datos, cuándo se generó cada columna y si el test estuvo aislado durante todo el proceso.

Ante una métrica excelente: revisar variables, fechas, preprocesamiento, duplicados y separación train/test.

La fuga de datos puede afectar también la comunicación. Si presentamos un modelo con métricas infladas, podemos convencer a otras personas de que el sistema está listo cuando en realidad fallará al aplicarse. Esto es especialmente grave en contextos de decisión.

Por eso, evitar fuga de datos no es solo una buena práctica técnica. Es una condición básica para producir evaluaciones honestas.

Sin control de fuga, la métrica puede medir un escenario artificial, no el uso real del modelo.

Una buena forma de prevenir este error es pensar siempre desde el momento de predicción. Antes de usar una variable, conviene preguntarse:

Para revisar este punto conviene preguntarse: ¿La variable existe antes de predecir? ¿Se calcula solo con datos disponibles hasta ese momento? ¿El test influyó en alguna transformación? ¿Hay registros relacionados entre train y test? ¿Estoy usando el test para tomar decisiones?

Estas preguntas ayudan a detectar fugas antes de que contaminen el modelo.

La fuga de datos ocurre cuando el modelo accede a información que no tendría disponible en el momento real de la predicción, o cuando el conjunto de prueba influye en el entrenamiento, el preprocesamiento o la selección del modelo. Puede aparecer por variables posteriores al evento, transformaciones hechas antes de separar, uso indebido del test, registros relacionados, agregaciones con información futura o resultados sospechosamente altos. Prevenirlo exige pensar temporalmente, separar correctamente, usar pipelines y reservar el test para una evaluación final honesta.

Preprocesar antes de separar

Otro error frecuente al construir modelos es preprocesar todo el dataset antes de separar los datos en entrenamiento y prueba. A primera vista parece una decisión cómoda: limpiamos la tabla completa, imputamos valores faltantes, escalamos variables, codificamos categorías y recién después dividimos los datos. El problema es que algunas transformaciones aprenden información del conjunto completo. Si el conjunto completo incluye los datos de prueba, entonces el test empieza a influir en el entrenamiento.

Este error se relaciona directamente con la fuga de datos, pero merece una sección propia porque es muy común y muchas veces ocurre sin que nos demos cuenta.

Concepto	Descripción
Error común	preprocesar todo y separar después.
Práctica correcta	separar primero y aprender transformaciones solo con entrenamiento.

No todo preprocesamiento tiene el mismo riesgo. Algunas operaciones son simples correcciones que no aprenden información estadística del dataset. Por ejemplo, renombrar columnas, corregir tipos evidentes, eliminar espacios al comienzo o al final de textos, o reemplazar nombres mal escritos en una categoría conocida pueden formar parte de una limpieza inicial.

El problema aparece con las transformaciones que calculan algo a partir de los datos. Si una transformación aprende medias, desviaciones estándar, medianas, modas, categorías, componentes principales o variables seleccionadas usando todo el dataset, entonces también está usando información del conjunto de prueba.

Transformaciones que aprenden: imputación, escalado, codificación, selección de variables, PCA.

Deben aprender solo desde train.

Pensemos en la imputación. Si una variable numérica tiene valores faltantes, podemos completar esos valores con la media. Pero esa media debe calcularse usando solo el conjunto de entrenamiento. Si calculamos la media sobre todos los datos antes de separar, el conjunto de prueba participa en el valor usado para completar los faltantes del entrenamiento.

Concepto	Descripción
Incorrecto	media calculada con todo el dataset.
Correcto	media calculada con entrenamiento y aplicada luego a prueba.

En código, el error puede verse así:

```
from sklearn.impute import SimpleImputer
from sklearn.model_selection import train_test_split

imputer = SimpleImputer(strategy="mean")

X_imputado = imputer.fit_transform(X)

X_train, X_test, y_train, y_test = train_test_split(
    X_imputado,
    y,
    test_size=0.25,
    random_state=42
)
```

El problema está en esta línea:

```
X_imputado = imputer.fit_transform(X)
```

El `fit_transform` se aplicó sobre todo `X`. Eso significa que el imputador aprendió la media usando datos que luego formarán parte del conjunto de prueba. Aunque parezca un detalle menor, la evaluación ya no es completamente honesta.

La forma correcta es separar primero y ajustar el imputador solo con entrenamiento.

```
from sklearn.impute import SimpleImputer
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.25,
    random_state=42
)

imputer = SimpleImputer(strategy="mean")

X_train_imp = imputer.fit_transform(X_train)
X_test_imp = imputer.transform(X_test)
```

La diferencia entre `fit_transform` y `transform` es importante. Con `fit_transform`, el objeto aprende los valores necesarios y transforma los datos. Con `transform`, solo aplica lo que ya aprendió.

Concepto	Descripción
fit	aprender parámetros.
transform	aplicar parámetros ya aprendidos.
fit_transform	aprender y aplicar.

El conjunto de entrenamiento se usa con `fit_transform` porque allí sí queremos aprender. El conjunto de prueba se usa con `transform` porque queremos simular datos nuevos: se transforman con las reglas aprendidas en entrenamiento, no con reglas aprendidas desde el propio test.

Algo parecido ocurre con el escalado. Si usamos `StandardScaler`, el transformador calcula la media y la desviación estándar de cada variable. Esos valores deben aprenderse solo desde entrenamiento.

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
```

```
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

Si escalamos antes de separar, el conjunto de prueba participa en la media y la desviación estándar. El efecto puede ser pequeño en algunos datasets, pero el procedimiento deja de representar correctamente el uso real del modelo.

En uso real, los datos nuevos no participan en el cálculo de la escala.

Por eso, el test tampoco debería hacerlo.

También sucede con One-Hot Encoding. El codificador aprende qué categorías existen en los datos. Si lo ajustamos sobre todo el dataset, puede aprender categorías que solo estaban presentes en el test. En un escenario real, esas categorías todavía no habrían aparecido durante el entrenamiento.

One-Hot Encoding aprende las categorías observadas.

Debe aprenderlas desde train, no desde todo el dataset.

En este caso, además, conviene configurar el codificador para que pueda manejar categorías nuevas al momento de predecir.

```
from sklearn.preprocessing import OneHotEncoder

encoder = OneHotEncoder(handle_unknown="ignore")
```

Con `handle_unknown="ignore"`, si aparece una categoría no vista durante el entrenamiento, el modelo no falla por ese motivo. Simplemente la codifica sin activar ninguna de las columnas conocidas para esa variable.

Una forma más robusta de evitar este error es usar `Pipeline` y `ColumnTransformer`. Estas herramientas permiten definir el preprocesamiento y el modelo como una única secuencia. Así, cuando entrenamos el pipeline, todas las transformaciones se ajustan solo con el conjunto de entrenamiento.

```
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression

variables_numericas = ["edad", "ingresos"]
variables_categoricas = ["zona", "tipo_cliente"]

preprocesamiento = ColumnTransformer([
    ("num", Pipeline([
        ("imputer", SimpleImputer(strategy="median")),
        ("scaler", StandardScaler())
    ]), variables_numericas),
```



```

        ("cat", Pipeline([
            ("imputer", SimpleImputer(strategy="most_frequent")),
            ("encoder", OneHotEncoder(handle_unknown="ignore"))
        ]), variables_categoricas)
    ])

modelo = Pipeline([
    ("preprocesamiento", preprocesamiento),
    ("clasificador", LogisticRegression())
])

```

Luego separamos los datos y entrenamos el pipeline solo con entrenamiento.

```

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.25,
    random_state=42
)

modelo.fit(X_train, y_train)
modelo.score(X_test, y_test)

```

Esta estructura tiene una ventaja importante: reduce el riesgo de aplicar transformaciones en el orden incorrecto. El pipeline organiza el flujo completo y evita que tengamos que transformar manualmente entrenamiento y prueba en pasos separados.

Pipeline: aprende con train, transforma test sin volver a aprender desde test.

El mismo principio aplica a PCA. Como vimos en capítulos anteriores, PCA aprende componentes principales a partir de las variables. Si calculamos esos componentes usando todo el dataset, el conjunto de prueba participa en la construcción de la representación. La forma correcta es ajustar PCA solo con entrenamiento y aplicar luego esa transformación al test.

También ocurre con la selección de variables. Si elegimos las columnas “más importantes” usando todo el dataset antes de separar, estamos permitiendo que el test influya en la selección. Después, cuando evaluamos, el resultado puede ser optimista porque las variables fueron elegidas teniendo en cuenta información que debería haber permanecido oculta.

Seleccionar variables antes de separar también puede contaminar la evaluación.

Este punto es especialmente importante cuando usamos métodos automáticos de selección. Por ejemplo, elegir las mejores variables según su relación con y debe hacerse dentro del proceso de entrenamiento, no usando todo el conjunto antes de dividir.

La regla general es simple:

Toda transformación que aprenda algo de los datos debe ajustarse únicamente con el conjunto de entrenamiento.

El conjunto de prueba debe atravesar el mismo proceso, pero sin influir en él. Debe ser transformado como si fuera información nueva que llega después de haber construido el modelo.

Una señal de alerta es tener líneas de código que transforman `X` completo antes del `train_test_split`.

```
X_transformado = transformador.fit_transform(X)

X_train, X_test, y_train, y_test = train_test_split(
    X_transformado,
    y,
    test_size=0.25,
    random_state=42
)
```

No siempre estará mal transformar antes de separar, pero si el transformador usa `fit`, hay que revisar con cuidado. En la mayoría de los casos de Machine Learning supervisado, es más seguro separar primero y luego ajustar las transformaciones dentro de un pipeline.

Si aparece `fit_transform(X)` antes de `train_test_split`, conviene revisar si existe fuga de datos.

También debemos cuidar este problema durante la validación cruzada. Si hacemos preprocesamiento antes de llamar a `cross_val_score`, la validación cruzada también queda contaminada. El preprocesamiento debe estar dentro del pipeline para que, en cada fold, aprenda solo desde la parte de entrenamiento correspondiente.

```
from sklearn.model_selection import cross_val_score

scores = cross_val_score(
    modelo,
    X,
    y,
    cv=5
)
```

En este ejemplo, si `modelo` es un pipeline que contiene preprocesamiento y clasificador, cada partición de la validación cruzada se maneja correctamente. En cada fold, el preprocesamiento se ajusta solo con los datos de entrenamiento de ese fold.

En validación cruzada, el preprocesamiento también debe repetirse correctamente dentro de cada fold.

Preprocesar antes de separar puede contaminar la evaluación si las transformaciones aprenden información del conjunto completo. Imputar, escalar, codificar, aplicar PCA o seleccionar variables deben ajustarse solo con los datos de entrenamiento. El conjunto de prueba debe transformarse usando lo aprendido en entrenamiento, sin participar en el aprendizaje. Para evitar errores, conviene separar primero y usar Pipeline y ColumnTransformer, de modo que todo el flujo de preprocesamiento y modelado respete la separación entre aprender y evaluar.

Decidir con datos y con criterio

En este capítulo analizamos una dimensión fundamental del Machine Learning: los modelos no existen aislados de los datos, las personas y las decisiones. Un modelo puede producir predicciones útiles, pero esas predicciones nacen de información parcial, de decisiones técnicas y de contextos concretos. Por eso, trabajar con modelos no es solo una tarea matemática o computacional. También exige responsabilidad.

Comenzamos con el sesgo en los datos. Vimos que un dataset no es una copia perfecta de la realidad, sino una representación construida. Los datos dependen de qué se midió, cómo se registró, quiénes aparecen, quiénes faltan y qué decisiones anteriores quedaron reflejadas.

El proceso parte de Datos, continúa con representación parcial y conduce finalmente a de una realidad más compleja.

Si ciertos grupos están poco representados, si algunas situaciones no se registran, si las etiquetas históricas reflejan decisiones problemáticas o si el contexto cambió, el modelo puede aprender patrones distorsionados. En esos casos, una buena métrica no garantiza que el sistema sea justo, robusto o adecuado.

También estudiamos el sesgo en el modelo. Un modelo puede amplificar problemas presentes en los datos, pero también puede introducir sesgos por la métrica elegida, el umbral de decisión, las variables incluidas, el algoritmo, el sobreajuste o la forma en que se define el objetivo.

El modelo aprende según los datos disponibles, pero también según las decisiones de modelado.

Una métrica global puede ocultar errores importantes. Un modelo puede funcionar bien en promedio y fallar en subgrupos relevantes. Un umbral puede reducir un tipo de error y aumentar otro. Por eso, evaluar un modelo responsablemente requiere mirar más allá de un único número.

Luego analizamos los datos faltantes y la representación desigual. Los valores ausentes no siempre son un simple problema técnico. A veces revelan diferencias en el proceso de recolección, desigualdades de acceso, errores de medición o grupos menos observados.

Lo que falta en los datos también puede decir algo sobre cómo se produjo el dataset.

Imputar o eliminar datos puede ser necesario, pero esas decisiones deben revisarse. Si los faltantes se concentran en ciertos grupos, eliminarlos puede reducir todavía más su representación. Si se imputan sin analizar su origen, podemos ocultar un problema importante.

También trabajamos la idea de variables sensibles. Algunas columnas pueden afectar derechos, oportunidades o trato hacia personas y grupos. Pueden aparecer de manera explícita, como edad, género o lugar de residencia, o de manera indirecta, a través de variables proxy.

Una variable predictiva no siempre es una variable apropiada para decidir.

El modelo solo detecta patrones. No sabe si usar una variable es justo, legal o adecuado. Esa decisión corresponde a las personas responsables del sistema. En algunos casos, una variable sensible puede excluirse del entrenamiento, pero usarse para auditar si el modelo falla más en ciertos grupos. En otros casos, puede ser necesario revisar si variables aparentemente neutrales funcionan como proxies.

Una sección clave fue la diferencia entre correlación y causalidad. Machine Learning predictivo suele encontrar asociaciones útiles entre variables, pero una asociación no demuestra una causa. Una variable puede ayudar a predecir sin ser la razón real por la que ocurre el fenómeno.

Concepto	Descripción
Correlación	una variable se asocia con otra.
Causalidad	una variable produce un efecto sobre otra.

Esta diferencia es importante para no tomar malas decisiones. Si una variable aparece como importante, podemos decir que el modelo la usa como señal. Pero no deberíamos afirmar automáticamente que modificar esa variable cambiará el resultado. Para responder preguntas causales hacen falta otros enfoques.

Después analizamos el impacto de las decisiones automatizadas. Cuando una predicción se transforma en una alerta, prioridad, recomendación, aprobación, rechazo o intervención, el modelo pasa a formar parte de un sistema de decisión. En ese punto, los errores dejan de ser solo métricas: pueden tener consecuencias reales.

El proceso parte de Predicción, continúa con decisión y conduce finalmente a consecuencia.

Por eso, debemos analizar falsos positivos, falsos negativos, umbrales, revisión humana, comunicación, monitoreo y posibilidad de corrección. A mayor impacto, mayor debe ser el cuidado.

Finalmente, trabajamos la responsabilidad humana. Un modelo no decide en el vacío. Personas definen el problema, eligen datos, seleccionan variables, entrenan modelos, eligen métricas, ajustan umbrales, comunican resultados y deciden cómo usar las predicciones.

El modelo puede apoyar decisiones.

No reemplaza el juicio, el contexto ni la responsabilidad humana.

Decir “lo decidió el algoritmo” no alcanza. Si el sistema afecta personas, recursos u oportunidades, debe haber supervisión real, documentación, revisión, monitoreo y posibilidad de corregir errores. También debe existir la posibilidad de decidir no usar un modelo cuando los datos son insuficientes, el riesgo es alto o el problema está mal formulado.

Los modelos de Machine Learning aprenden desde datos producidos en contextos concretos y se usan dentro de sistemas de decisión. Por eso, sus resultados pueden reflejar sesgos, ausencias, desigualdades, errores de medición o decisiones históricas. Trabajar responsablemente implica revisar datos, variables, métricas, errores, subgrupos, correlaciones, umbrales, consecuencias y comunicación. Machine Learning puede ayudar a decidir mejor, pero solo si se usa con criterio, prudencia y responsabilidad humana.

Elegir métricas inadecuadas

Otro error muy común es elegir una métrica que no representa bien el problema. Una métrica es una forma de resumir el desempeño del modelo, pero no todas las métricas responden la misma pregunta. Si usamos una métrica inadecuada, podemos seleccionar un modelo que parece bueno en números, pero que no sirve para el objetivo real.

Una métrica no es solo un número: también expresa qué entendemos por buen desempeño.

Este problema aparece con mucha frecuencia en clasificación. La accuracy, por ejemplo, mide la proporción total de aciertos. Es una métrica simple y útil en algunos casos, pero puede ser engañosa cuando las clases están desbalanceadas o cuando distintos tipos de error tienen consecuencias muy diferentes.

Supongamos un dataset donde el 95% de los casos pertenece a la clase negativa y solo el 5% a la clase positiva. Un modelo que predice siempre “negativo” tendrá 95% de accuracy. A primera vista parece excelente. Pero en realidad no detecta ningún caso positivo.

Concepto	Descripción
Dataset	95% negativos 5% positivos
Modelo	predice siempre negativo
Accuracy	95%
Problema	no detecta la clase positiva.

En un problema así, la accuracy puede ocultar el fracaso del modelo. Si la clase positiva representa algo importante, como fraude, enfermedad, abandono, falla, riesgo o necesidad de intervención, no detectarla puede ser justamente el peor resultado posible.

Por eso, antes de elegir una métrica, debemos preguntarnos qué tipo de error importa más. En clasificación binaria, no todos los errores tienen el mismo significado. Un falso positivo y un falso negativo pueden tener costos muy distintos.

Concepto	Descripción
Falso positivo	el modelo señala algo que no correspondía.
Falso negativo	el modelo no señala algo que sí correspondía.

En detección de spam, un falso positivo puede enviar un correo legítimo a la carpeta de spam. Un falso negativo deja pasar un mensaje no deseado. En detección de fraude, un falso positivo puede bloquear una operación válida. Un falso negativo permite que pase una operación fraudulenta. En un sistema de apoyo educativo, un falso positivo puede señalar a un estudiante que no necesitaba intervención, mientras que un falso negativo puede dejar sin apoyo a alguien que sí lo necesitaba.

Una misma métrica puede resultar insuficiente si no distingue qué tipos de error son más importantes.

Cuando queremos saber qué proporción de los casos señalados como positivos realmente eran positivos, usamos precision. Esta métrica es útil cuando los falsos positivos son costosos o problemáticos.

Precision: de todos los casos que el modelo marcó como positivos, cuántos realmente lo eran.

Cuando queremos saber qué proporción de los positivos reales logró detectar el modelo, usamos recall o sensibilidad. Esta métrica es importante cuando los falsos negativos son costosos.

Recall: de todos los casos positivos reales, cuántos logró detectar el modelo.

El F1-score combina precision y recall en una sola medida. Puede ser útil cuando necesitamos equilibrar ambas, especialmente en problemas con clases desbalanceadas. Sin embargo, tampoco es una métrica mágica. Al resumir, también oculta detalles. Siempre conviene mirar la matriz de confusión y entender los errores concretos.

F1-score: resume precision y recall, pero no reemplaza la revisión de errores.

Elegir una métrica inadecuada puede llevarnos a comparar mal los modelos. Por ejemplo, un modelo puede tener mayor accuracy pero peor recall sobre la clase importante. Si elegimos solo por accuracy, seleccionaremos un modelo que acierta más en promedio, pero que falla donde más nos interesa.

Modelo A: accuracy alta, recall bajo.

Modelo B: accuracy menor, recall más alto.

El mejor depende del objetivo del problema.

No existe una única métrica correcta para todos los casos. La métrica debe elegirse según la pregunta, los datos, el desbalance de clases, el costo de los errores y el uso real del modelo.

También debemos prestar atención a qué promedio usamos en problemas multiclase. En clasificación con varias clases, métricas como precision, recall y F1 pueden calcularse por clase y luego promediarse de distintas maneras. El promedio macro trata a todas las clases por igual. El promedio weighted pondera según la cantidad de casos de cada clase.

Concepto	Descripción
Promedio macro	cada clase pesa igual.

Concepto	Descripción
Promedio weighted	las clases con más casos pesan más.

Si tenemos clases minoritarias importantes, el promedio weighted puede ocultar un mal desempeño en ellas, porque las clases grandes dominan el resultado. En ese caso, conviene mirar las métricas por clase y no solo el promedio general.

Una métrica promedio puede ocultar un mal desempeño en una clase específica.

En regresión ocurre algo parecido. No todas las métricas de error cuentan la misma historia. El MAE mide el error absoluto promedio en las unidades del problema. El RMSE también mide error en unidades reales, pero penaliza más los errores grandes. El R^2 expresa qué proporción de la variabilidad de la variable objetivo logra explicar el modelo, pero puede ser difícil de interpretar si se usa sin contexto.

Concepto	Descripción
MAE	error promedio en unidades reales.
RMSE	penaliza más los errores grandes.
R^2	resume ajuste relativo, pero necesita contexto.

Si para un problema los errores grandes son especialmente graves, RMSE puede ser más informativo que MAE. Si queremos explicar el error promedio de manera sencilla, MAE suele ser más claro. Si queremos comparar ajuste general, R^2 puede ayudar, pero no debería reemplazar la lectura de errores en unidades reales.

Por ejemplo, en un modelo que predice precios de viviendas, un MAE de 8000 significa que, en promedio, el modelo se equivoca por 8000 unidades monetarias. Esa interpretación es directa. En cambio, un R^2 de 0.82 puede indicar buen ajuste relativo, pero no nos dice por sí solo si el error es aceptable para la decisión que queremos tomar.

Una métrica útil debe poder relacionarse con el significado real del error.

También hay problemas donde la métrica debe considerar el impacto operativo. En un sistema que prioriza casos para revisión, tal vez no importa solo la clasificación correcta, sino cuántos casos puede revisar el equipo. En ese caso, puede ser útil analizar el desempeño sobre los primeros casos ordenados por riesgo, no solo una métrica global.

Si solo se revisarán los 100 casos de mayor riesgo, la métrica debe ayudar a evaluar esa forma de uso.

Esto muestra que la métrica debe estar conectada con el modo en que el modelo será utilizado. Un modelo no se evalúa en el vacío. Se evalúa para una tarea, un contexto y una decisión.

Otro error frecuente es elegir la métrica después de mirar los resultados. Si probamos varias métricas y luego destacamos solo la que mejor se ve, podemos comunicar una imagen sesgada del modelo. Lo ideal es definir de antemano qué métricas son relevantes y por qué.

Concepto	Descripción
Primero	definir qué importa medir.
Después	entrenar, evaluar y comparar modelos.

Esto no significa que no podamos explorar métricas adicionales. Podemos hacerlo, pero debemos distinguir entre la métrica principal y las métricas complementarias. La métrica principal orienta la selección del modelo. Las complementarias ayudan a entender mejor su comportamiento.

También es importante que la métrica coincida con el tipo de problema. No tiene sentido evaluar clasificación con métricas de regresión ni regresión con métricas de clasificación. Si transformamos un problema continuo en categorías, debemos tener claro qué se pierde en esa decisión. Si tratamos un problema de categorías ordenadas como si todas las diferencias fueran iguales, también podemos simplificar demasiado.

La métrica debe corresponder al tipo de predicción y al uso real del resultado.

En problemas con probabilidades, además, puede interesarnos evaluar si las probabilidades están bien calibradas. Un modelo puede ordenar bien los casos, pero producir probabilidades demasiado confiadas o demasiado conservadoras. Si vamos a usar esas probabilidades para tomar decisiones, no alcanza con mirar solo la clase final.

Predecir correctamente la clase no garantiza que la probabilidad estimada sea confiable.

Por ejemplo, si un modelo asigna 0.90 de probabilidad a muchos casos, esperamos que aproximadamente el 90% de esos casos sean realmente positivos. Si eso no ocurre, la probabilidad puede no estar bien calibrada. Esto importa cuando usamos probabilidades para definir umbrales, prioridades o niveles de riesgo.

Elegir métricas inadecuadas puede llevarnos a confiar en modelos que no resuelven bien el problema real. Accuracy puede engañar en clases desbalanceadas; los promedios pueden ocultar errores en clases minoritarias; MAE, RMSE y R^2 cuentan historias distintas en regresión; y las probabilidades requieren cuidado cuando se usan para decidir. Una buena evaluación comienza antes de entrenar: debemos definir qué errores importan, qué consecuencias tienen y qué métrica representa mejor el objetivo del modelo.

Comparar modelos de forma injusta

Comparar modelos es una parte normal del trabajo en Machine Learning. Podemos probar una regresión logística, un árbol de decisión, un Random Forest, un modelo de Gradient Boosting o cualquier otro algoritmo, y luego elegir el que mejor funcione para el problema. El error aparece cuando esa comparación no se hace en condiciones equivalentes.

Una comparación injusta ocurre cuando los modelos no fueron entrenados, ajustados o evaluados bajo las mismas reglas. En ese caso, la diferencia de resultados puede deberse al procedimiento y no a que un modelo sea realmente mejor que otro.

Comparación justa: mismos datos, misma separación, mismo preprocesamiento, métrica adecuada y evaluación equivalente.

Por ejemplo, supongamos que entrenamos un modelo con variables escaladas y otro con variables sin escalar. Si uno de los modelos es sensible a la escala, como KNN, SVM o regresión logística con regularización, puede quedar en desventaja. No estaríamos comparando solo algoritmos, sino también condiciones de preparación de datos.

Concepto	Descripción
Modelo A	datos correctamente escalados.
Modelo B	datos sin escalar.
Resultado	la comparación puede ser injusta.

Esto no significa que todos los modelos necesiten exactamente el mismo preprocesamiento. Algunos modelos requieren ciertas transformaciones y otros no. Los árboles de decisión y Random Forest, por ejemplo, no suelen necesitar escalado para funcionar correctamente. En cambio, modelos basados en distancias o márgenes sí pueden necesitarlo. Lo importante es que cada modelo reciba un tratamiento razonable y que la comparación refleje una oportunidad real para cada uno.

Comparar justamente no siempre significa aplicar idénticos pasos.

Significa dar a cada modelo un flujo adecuado y evaluarlo con el mismo criterio.

Una forma clara de lograrlo es usar pipelines. Cada modelo puede tener su propio pipeline de preprocesamiento y estimador, pero todos deben evaluarse con la misma separación de datos y la misma métrica principal.

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier

modelo_logistico = Pipeline([
    ("scaler", StandardScaler()),
    ("modelo", LogisticRegression())
])

modelo_rf = Pipeline([
    ("modelo", RandomForestClassifier(random_state=42))
])
```

En este ejemplo, la regresión logística incluye escalado porque puede necesitarlo. Random Forest no lo incluye porque no depende de distancias ni de la escala de las variables del mismo modo. Ambos pueden compararse, pero deben evaluarse sobre la misma división de datos.

```
modelo_logistico.fit(X_train, y_train)
modelo_rf.fit(X_train, y_train)

score_logistico = modelo_logistico.score(X_test, y_test)
score_rf = modelo_rf.score(X_test, y_test)
```

Otro error común es cambiar la separación train/test para cada modelo. Si un modelo se evalúa con una división y otro modelo con otra, las diferencias pueden deberse al azar de la separación. Tal vez un conjunto de prueba resultó más fácil que otro, o una clase quedó distribuida de manera distinta.

Concepto	Descripción
Modelo A	evaluado con un test.
Modelo B	evaluado con otro test.
Problema	los resultados no son directamente comparables.

Para comparar modelos, conviene usar la misma separación. Si usamos `train_test_split`, todos los modelos deberían entrenarse con `X_train` e `y_train`, y evaluarse con `X_test` e `y_test`. Si usamos validación cruzada, todos deberían evaluarse con la misma estrategia de partición.

Mismo train. Mismo test. Misma métrica.

Recién entonces la comparación tiene sentido.

La validación cruzada puede ayudar a comparar con más estabilidad, porque no dependemos de una única división. Pero también allí debemos cuidar la justicia del procedimiento. Todos los modelos deben evaluarse con el mismo número de folds y, si corresponde, con la misma estratificación.

```
from sklearn.model_selection import cross_val_score

scores_logistico = cross_val_score(
    modelo_logistico,
    X,
    y,
    cv=5,
    scoring="f1"
)

scores_rf = cross_val_score(
    modelo_rf,
    X,
    y,
    cv=5,
    scoring="f1"
)
```

Aquí ambos modelos se evalúan con 5 folds y con la misma métrica. Eso permite comparar no solo el promedio, sino también la variabilidad de los resultados.

No conviene mirar únicamente el promedio; también debemos observar si el desempeño es estable o muy variable.

Otro problema aparece cuando ajustamos mucho un modelo y casi nada otro. Por ejemplo, probamos decenas de combinaciones de hiperparámetros para Random Forest, pero usamos la regresión logística con valores por defecto. Si Random Forest gana, no sabemos si ganó por ser mejor para el problema o porque recibió más búsqueda y ajuste.

Concepto	Descripción
Modelo A	muchos hiperparámetros probados.
Modelo B	valores por defecto.
Resultado	comparación posiblemente injusta.

Esto no significa que debamos probar exactamente la misma cantidad de combinaciones para todos. Algunos modelos tienen más hiperparámetros relevantes que otros. Pero sí debemos reconocer cuándo una comparación enfrenta un modelo muy trabajado contra otro apenas probado.

Una práctica razonable es definir búsquedas proporcionales y transparentes. Por ejemplo, podemos comparar una versión base de cada modelo y luego, en una segunda etapa, ajustar los candidatos más prometedores. Lo importante es comunicar qué se hizo.

Comparación responsable: aclarar qué modelos se probaron, con qué hiperparámetros y bajo qué evaluación.

También puede ser injusto comparar modelos usando métricas distintas. Si evaluamos un modelo con accuracy y otro con F1-score, la comparación no tiene sentido directo. Cada métrica resume una dimensión diferente del desempeño.

Concepto	Descripción
Modelo A	accuracy.
Modelo B	F1-score.
Problema	no estamos midiendo lo mismo.

Podemos mirar varias métricas, por supuesto. Pero para decidir cuál modelo es mejor, debemos definir cuál es la métrica principal o cómo vamos a equilibrar las métricas relevantes. En un problema de clases desbalanceadas, por ejemplo, quizá la métrica principal sea recall, F1-score o precision, según el objetivo. Luego podemos reportar accuracy como información adicional, pero no usarla como criterio único si no representa el problema.

Otro error frecuente es comparar modelos después de aplicar distintas selecciones de variables sin justificarlo. Si un modelo recibe variables cuidadosamente seleccionadas y otro recibe variables sin revisar, el resultado puede depender de esa diferencia. Lo mismo ocurre si un modelo recibe variables creadas con conocimiento adicional y otro no.

Cuando cambian las variables, también cambia el problema que resuelve cada modelo.

A veces queremos comparar no solo modelos, sino pipelines completos. Eso es válido. Podemos comparar, por ejemplo, una estrategia con pocas variables interpretables contra otra con muchas variables y mayor complejidad. Pero en ese caso debemos decirlo con claridad: no estamos comparando solo algoritmos, sino enfoques completos.

Concepto	Descripción
Comparar algoritmos	misma representación de datos.
Comparar enfoques	pueden cambiar variables, preprocesamiento y modelo.

Esta distinción ayuda a comunicar mejor. Si decimos “Random Forest fue mejor que regresión logística”, deberíamos asegurarnos de que la comparación fue justa. Si en realidad Random Forest usó otro conjunto de variables, otro tratamiento de datos y otra búsqueda de hiperparámetros, tal vez la frase correcta sea: “el pipeline basado en Random Forest obtuvo mejor desempeño que el pipeline lineal probado”.

También hay que tener cuidado con el conjunto de prueba. Si probamos muchos modelos y elegimos el mejor según el test, estamos usando el test como herramienta de selección. Con cada prueba, el test deja de ser una evaluación final completamente independiente.

El conjunto de prueba no debería utilizarse para probar opciones una y otra vez.

La forma más ordenada es usar entrenamiento y validación, o validación cruzada, para comparar modelos y ajustar decisiones. El test se reserva para el final, cuando el proceso de selección ya terminó.

Concepto	Descripción
Entrenamiento	ajustar modelos.
Validación	comparar opciones.
Test	medir el resultado final.

En datasets pequeños, esto puede ser difícil porque separar demasiados conjuntos reduce los datos disponibles. En esos casos, la validación cruzada puede ser una alternativa. Pero el principio se mantiene: no conviene convertir el test en un espacio de experimentación.

Otro aspecto de justicia es el tiempo y la complejidad. Un modelo puede obtener una pequeña mejora de métrica a costa de mucha más complejidad, más tiempo de entrenamiento, menos interpretabilidad o mayor dificultad de mantenimiento. Decir que “es mejor” solo por una diferencia mínima puede ser apresurado.

El mejor modelo no siempre es el que obtiene la métrica más alta por una diferencia mínima.

Por ejemplo, si un modelo obtiene F1-score de 0.842 y otro 0.846, la diferencia puede no ser significativa en la práctica. Si el segundo es mucho más complejo y difícil de explicar, tal vez el primero sea una mejor elección para el contexto. La comparación debe considerar desempeño, estabilidad, interpretabilidad, costo y riesgo.

Comparar modelos implica mirar: métrica, estabilidad, simplicidad, interpretabilidad y aplicabilidad.

También conviene revisar los errores concretos. Dos modelos pueden tener métricas similares, pero equivocarse en casos distintos. Uno puede detectar mejor la clase minoritaria. Otro puede cometer menos falsos positivos. Otro puede funcionar mejor en cierto subgrupo. Mirar solo una métrica agregada puede ocultar diferencias importantes.

Dos modelos con métricas parecidas pueden cometer errores diferentes.

Por eso, una comparación responsable no se limita a una tabla con números. Incluye revisar matriz de confusión, métricas por clase, desempeño en validación, estabilidad, errores individuales y sentido práctico de las variables usadas.

Comparar modelos de forma injusta puede llevarnos a elegir un modelo por razones equivocadas. Para que la comparación sea válida, debemos usar la misma separación de datos, la misma estrategia de evaluación, métricas coherentes, pipelines adecuados y un tratamiento razonable para cada modelo. También debemos evitar usar el test como espacio de experimentación y considerar no solo la métrica, sino la estabilidad, la complejidad, la interpretabilidad y el contexto de uso. Comparar bien no significa elegir automáticamente el número más alto, sino entender en qué condiciones se obtuvo ese resultado y si realmente representa una mejora útil.

Confiar en un único resultado

Otro error frecuente al construir modelos es confiar demasiado en un único resultado. A veces entrenamos un modelo, obtenemos una métrica sobre una separación train/test y, a partir de ese número, sacamos conclusiones fuertes: “este modelo funciona”, “este algoritmo es mejor” o “el desempeño esperado es este”. El problema es que un único resultado puede depender mucho de la división de datos, de la semilla aleatoria, de la composición del conjunto de prueba o de pequeñas particularidades del dataset.

Un único resultado puede ser una señal útil, pero no siempre alcanza para concluir con seguridad.

Cuando usamos `train_test_split`, una parte de los datos queda para entrenamiento y otra para prueba. Si cambiamos la semilla aleatoria, puede cambiar qué filas caen en cada conjunto. En muchos problemas, especialmente con datasets pequeños o clases desbalanceadas, esa diferencia puede modificar bastante la métrica obtenida.

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
```

```
test_size=0.25,  
random_state=42  
)
```

El parámetro `random_state` nos ayuda a reproducir la misma separación. Eso es importante para que otra persona pueda obtener el mismo resultado. Pero reproducible no significa necesariamente representativo. Una división puede ser perfectamente reproducible y, aun así, ser una muestra particularmente fácil o difícil.

Que un resultado sea reproducible no significa automáticamente que sea suficiente.

Supongamos que entrenamos un clasificador y obtenemos un F1-score de 0.82. Ese valor puede ser una estimación razonable, pero no deberíamos interpretarlo como una verdad exacta. Con otra separación, quizá el resultado sea 0.78 o 0.85. Si el dataset es pequeño, esa variabilidad puede ser considerable.

Una métrica no es una propiedad fija grabada en el modelo, sino un resultado observado bajo una evaluación concreta.

Por eso conviene mirar la estabilidad. Un modelo no solo debería tener buen desempeño en una división, sino comportarse de manera razonablemente consistente en distintas evaluaciones. La validación cruzada ayuda a analizar esto.

En validación cruzada, el dataset se divide en varias partes. El modelo se entrena y evalúa varias veces, usando distintos bloques como prueba. Luego miramos el promedio de las métricas y también su dispersión.

Validación cruzada: varias evaluaciones sobre distintas particiones del dataset.

En `scikit-learn`, podemos usar `cross_val_score`.

```
from sklearn.model_selection import cross_val_score  
  
scores = cross_val_score(  
    modelo,  
    X,  
    y,  
    cv=5,  
    scoring="f1"  
)  
  
scores
```

El resultado no es un único número, sino una lista de puntajes, uno por cada fold.

Por ejemplo, una validación cruzada podría producir los puntajes `[0.81, 0.79, 0.84, 0.80, 0.83]`.

Luego podemos calcular el promedio y la desviación estándar.

```
scores.mean(), scores.std()
```

El promedio resume el desempeño general. La desviación estándar indica cuánto varían los resultados entre particiones.

Concepto	Descripción
Promedio alto	buen desempeño general.
Desviación baja	resultado más estable.
Desviación alta	resultado más variable.

Un modelo con promedio 0.82 y desviación 0.02 parece más estable que otro con promedio 0.83 y desviación 0.10. Aunque el segundo tenga un promedio apenas mayor, su desempeño depende más de la partición. En la práctica, esa inestabilidad puede ser importante.

Modelo A: promedio 0.82 desviación 0.02

Modelo B: promedio 0.83 desviación 0.10

El modelo B no necesariamente es mejor para el uso real.

Esto muestra que comparar modelos no debería limitarse al número más alto. También debemos mirar la variabilidad. Un modelo estable suele ser más confiable que uno que funciona muy bien en algunas particiones y mal en otras.

Sin embargo, la validación cruzada tampoco es perfecta. Sigue dependiendo del dataset disponible. Si el dataset no representa bien el contexto real, la validación cruzada repetirá ese problema. Si hay fuga de datos, registros relacionados o una mala definición temporal, la validación cruzada puede producir resultados engañosos.

La validación cruzada mejora la evaluación, pero no corrige un dataset mal representado ni una fuga de datos.

Por eso, confiar en un único resultado no se resuelve solo repitiendo cálculos. También debemos revisar si la evaluación refleja el uso futuro del modelo.

Otro aspecto importante es la variabilidad por clases o subgrupos. Una métrica global puede ser estable y, aun así, ocultar errores en ciertos casos. Por ejemplo, el promedio de F1 puede mantenerse parecido, pero el desempeño sobre una clase minoritaria puede variar mucho entre particiones.

La estabilidad global no garantiza estabilidad en cada clase o subgrupo.

Cuando el problema lo requiere, conviene mirar métricas por clase, matrices de confusión y desempeño en segmentos relevantes. Esto permite detectar si el modelo depende demasiado de cómo quedaron distribuidos ciertos casos.

También debemos tener cuidado con los resultados obtenidos después de muchas pruebas. Si entrenamos decenas de modelos, cambiamos variables, ajustamos hiperparámetros y repetimos evaluaciones, es posible que encontremos una combinación que funciona muy bien por casualidad en nuestro esquema de evaluación. Si luego comunicamos solo ese mejor resultado, podemos sobreestimar el desempeño real.

Probar muchas opciones aumenta la posibilidad de encontrar un resultado favorable por azar.

Esto no significa que esté mal experimentar. Experimentar es parte del trabajo. Pero debemos separar exploración, selección y evaluación final. Si usamos un conjunto de validación para probar opciones, el test final debería reservarse para una medición honesta al final del proceso.

Concepto	Descripción
Exploración	probar ideas.
Validación	comparar opciones.
Test final	estimar desempeño después de decidir.

Confiar en un único resultado también puede afectar la comunicación. Es mejor decir “en esta evaluación el modelo obtuvo...” que “el modelo tiene...”. La primera frase reconoce que el número depende de un procedimiento. La segunda puede sonar como si la métrica fuera una propiedad absoluta del modelo.

Concepto	Descripción
Más prudente	"en esta evaluación obtuvo 0.82".
Más riesgoso	"el modelo tiene 0.82".

Este cuidado es importante porque las métricas pueden cambiar al modificar los datos, el período, la población, las variables o la forma de evaluación. Un modelo que rinde bien en un dataset puede no rendir igual en otro.

También es útil revisar errores individuales. Un buen promedio no nos dice qué casos falló el modelo. Al mirar ejemplos concretos de errores, podemos descubrir patrones:

casos ambiguos, etiquetas dudosas, datos incompletos, subgrupos mal representados, variables insuficientes o problemas de definición del objetivo.

La métrica resume el desempeño, mientras que los errores concretos ayudan a entender qué está ocurriendo.

Por ejemplo, en un modelo de clasificación de reclamos, revisar errores puede mostrar que el modelo falla especialmente en reclamos breves, en textos con faltas de ortografía o en categorías muy parecidas entre sí. Esa información no aparece claramente en una métrica única.

En regresión, mirar solo MAE o RMSE puede ocultar que el modelo falla mucho en valores extremos. Tal vez el error promedio sea aceptable, pero las predicciones sean malas en viviendas muy caras, ventas excepcionales o demandas muy altas. Revisar gráficos de error y casos individuales ayuda a detectar estos problemas.

Un error promedio aceptable puede ocultar fallos importantes en casos extremos.

Confiar en un único resultado también puede hacernos ignorar la incertidumbre. En Machine Learning aplicado, rara vez deberíamos presentar una métrica como exacta y definitiva. Es más responsable comunicar rangos, variabilidad o al menos advertir que el resultado depende de los datos evaluados.

No siempre hace falta una explicación estadística compleja. A veces alcanza con decir que se usó validación cruzada y que los resultados fueron consistentes, o que hubo variabilidad entre folds y por eso la conclusión debe tomarse con cautela.

Comunicar la variabilidad ayuda a evitar conclusiones exageradas.

Por último, confiar en un único resultado puede llevarnos a elegir un modelo demasiado rápido. Antes de decidir, conviene revisar si el resultado se mantiene, si el modelo es estable, si los errores son aceptables, si la métrica representa el problema y si el procedimiento de evaluación fue correcto.

Antes de confiar: revisar estabilidad, errores, métrica, datos y procedimiento de evaluación.

Un único resultado puede ser insuficiente para evaluar un modelo. La métrica obtenida en una separación train/test depende de la partición, de la semilla, de los datos disponibles y del procedimiento usado. La validación cruzada, la comparación de promedios y desviaciones, el análisis por clases o subgrupos y la revisión de errores concretos ayudan a construir una evaluación más confiable. Un modelo no debería elegirse solo porque obtuvo un buen número una vez, sino porque su desempeño es razonable, estable y coherente con el problema real.

No revisar errores individuales

Una métrica resume el desempeño de un modelo, pero no muestra toda la historia. Podemos obtener una accuracy, un F1-score, un MAE o un RMSE y tener una idea general de cómo funcionó el modelo. Sin embargo, esos números no nos dicen cuáles fueron los casos en los que falló, qué tienen en común esos errores ni si hay algún patrón importante escondido detrás del promedio.

No revisar errores individuales es un error común porque nos deja con una visión demasiado superficial del modelo. Sabemos cuánto falló, pero no sabemos cómo falló.

Concepto	Descripción
La métrica responde	¿cuánto se equivocó?
La revisión de errores responde	¿en qué casos se equivocó y por qué podría haber ocurrido?

Por ejemplo, imaginemos un modelo de clasificación que predice si un cliente abandonará o no un servicio. El modelo obtiene un F1-score aceptable. A primera vista, podríamos pensar que funciona bien. Pero si revisamos los errores, quizá descubramos que falla especialmente en clientes nuevos, en clientes con pocos datos históricos o en clientes de una región determinada.

Ese hallazgo cambia la interpretación. El problema ya no es solo “el modelo tiene cierto F1-score”, sino “el modelo parece menos confiable en ciertos tipos de clientes”.

Concepto	Descripción
Métrica general	resultado promedio.
Errores individuales	pistas sobre casos difíciles, grupos mal representados o problemas de datos.

En clasificación, una buena forma de empezar es revisar falsos positivos y falsos negativos. Los falsos positivos son casos que el modelo marcó como positivos, pero

que en realidad eran negativos. Los falsos negativos son casos positivos reales que el modelo no detectó.

Concepto	Descripción
Falso positivo	el modelo señaló un caso que no correspondía.
Falso negativo	el modelo no señaló un caso que sí correspondía.

Cada tipo de error puede tener un significado distinto. En detección de fraude, un falso positivo puede bloquear o revisar una operación legítima. Un falso negativo puede dejar pasar una operación fraudulenta. En un sistema educativo, un falso positivo puede señalar a un estudiante que no necesitaba apoyo, mientras que un falso negativo puede dejar sin acompañamiento a alguien que sí lo necesitaba.

Por eso, revisar errores individuales ayuda a conectar el modelo con el contexto real del problema.

Los errores no son solo números: pueden convertirse en acciones, omisiones o decisiones concretas.

En `scikit-learn`, después de entrenar un modelo, podemos generar predicciones y comparar `y_test` con `y_pred`.

```
y_pred = modelo.predict(X_test)
```

Luego podemos construir una tabla con los casos reales, las predicciones y si hubo error.

```
import pandas as pd

revision = X_test.copy()
revision["real"] = y_test
revision["prediccion"] = y_pred
revision["error"] = revision["real"] != revision["prediccion"]

revision.head()
```

A partir de esa tabla, podemos filtrar los casos donde el modelo se equivocó.

```
errores = revision[revision["error"]]

errores.head()
```

Esto permite mirar ejemplos concretos. ¿Son casos con valores faltantes? ¿Tienen categorías poco frecuentes? ¿Corresponden a una clase minoritaria? ¿Aparecen en

cierto período? ¿Tienen valores extremos? ¿Son casos ambiguos incluso para una persona?

Para revisar este punto conviene preguntarse: Preguntas útiles: ¿los errores se parecen entre sí? ¿aparecen en ciertas clases? ¿hay valores extremos? ¿faltan datos? ¿son casos ambiguos?

En problemas de clasificación binaria, también puede ser útil separar falsos positivos y falsos negativos.

```
falsos_positivos = revision[
    (revision["real"] == 0) &
    (revision["prediccion"] == 1)
]

falsos_negativos = revision[
    (revision["real"] == 1) &
    (revision["prediccion"] == 0)
]
```

Esta separación ayuda a entender si el modelo se equivoca más de una forma que de otra. A veces la métrica general parece razonable, pero al mirar los errores encontramos que el modelo falla justamente en el tipo de error más costoso para el problema.

No todos los errores tienen el mismo impacto, por lo que conviene analizarlos por separado.

También conviene revisar errores junto con las probabilidades, cuando el modelo las produce. No es lo mismo equivocarse con baja confianza que equivocarse con mucha confianza. Un modelo que predice mal con probabilidad 0.51 está cerca del umbral. Un modelo que predice mal con probabilidad 0.98 está cometiendo un error más preocupante.

Concepto	Descripción
Error con baja confianza	caso cercano al límite.
Error con alta confianza	señal de revisión más importante.

Podemos obtener probabilidades con `predict_proba`, si el modelo lo permite.

```
probas = modelo.predict_proba(X_test)[: , 1]
revision["probabilidad_clase_positiva"] = probas
```

Luego podemos revisar los errores con mayor confianza.

```
errores_confiados = revision[revision["error"]].sort_values(
    "probabilidad_clase_positiva",
    ascending=False
)

errores_confiados.head()
```

Este tipo de revisión puede revelar problemas de calibración, variables engañosas, casos mal etiquetados o patrones que el modelo aprendió de manera excesiva.

En regresión, la revisión de errores individuales también es importante. Una métrica como MAE o RMSE resume el error promedio, pero no muestra en qué casos el modelo se equivoca mucho.

Concepto	Descripción
MAE o RMSE	resumen del error.
Errores individuales	dónde el modelo falla más.

Podemos calcular el error de cada caso restando el valor real y el predicho.

```
y_pred = modelo.predict(X_test)

revision = X_test.copy()
revision["real"] = y_test
revision["prediccion"] = y_pred
revision["error"] = revision["real"] - revision["prediccion"]
revision["error_abs"] = revision["error"].abs()
```

Luego podemos ordenar los casos por mayor error absoluto.

```
revision.sort_values("error_abs", ascending=False).head()
```

Esto permite detectar casos extremos. Por ejemplo, en un modelo de precios de viviendas, tal vez el error más alto ocurre en propiedades muy grandes, ubicaciones poco frecuentes o viviendas con características especiales que no están bien representadas en las variables.

Errores grandes pueden señalar: valores extremos, variables faltantes, casos raros, datos incorrectos o fenómenos no capturados.

También podemos analizar si el modelo tiende a sobreestimar o subestimar. Si el error real menos predicción es positivo, el modelo predijo por debajo del valor real. Si es negativo, predijo por encima. Detectar una tendencia sistemática puede ayudar a mejorar el modelo o a comunicar sus límites.

Concepto	Descripción
Subestimación	el modelo predice menos que el valor real.
Sobreestimación	el modelo predice más que el valor real.

Por ejemplo, un modelo puede predecir bastante bien valores medios, pero subestimar los valores altos. Esto es común cuando hay pocos ejemplos extremos o cuando el modelo tiende a suavizar las predicciones hacia el promedio.

Un buen error promedio puede ocultar un mal desempeño en los valores extremos.

Revisar errores individuales también puede mostrar problemas en las etiquetas. A veces el modelo “se equivoca” porque la etiqueta del dataset está mal, es ambigua o fue cargada con criterios inconsistentes. En esos casos, la revisión no solo evalúa el modelo, sino también la calidad de los datos.

Algunos errores del modelo pueden revelar errores presentes en los datos.

Por ejemplo, en clasificación de textos, tal vez un reclamo está etiquetado como “facturación”, pero su contenido habla principalmente de “soporte técnico”. Si el modelo lo clasifica como soporte técnico, la métrica lo cuenta como error, aunque la etiqueta original pueda ser discutible.

Esto muestra que la evaluación automática tiene límites. Las métricas dependen de que las etiquetas sean confiables. Si las etiquetas tienen ruido, los errores individuales pueden ayudarnos a detectarlo.

Revisar los errores también implica revisar la calidad de las etiquetas.

Otra ventaja de mirar errores individuales es que ayuda a mejorar el modelo con criterio. Si descubrimos que los errores se concentran en valores faltantes, podemos revisar la imputación. Si aparecen en categorías raras, podemos agrupar categorías o conseguir más datos. Si ocurren en textos muy cortos, quizá necesitamos mejores variables. Si afectan a un subgrupo, debemos revisar representación y sesgo.

Los errores pueden orientar: mejorar variables, revisar datos, ajustar umbrales, cambiar métricas o recolectar más información.

Sin esta revisión, las mejoras suelen ser más ciegas. Probamos modelos más complejos, cambiamos hiperparámetros o agregamos transformaciones sin entender cuál es el problema principal.

También ayuda a comunicar mejor. Un informe responsable no debería mostrar solo una métrica final. Puede incluir ejemplos de errores típicos, tipos de casos difíciles y

limitaciones observadas. Esto no debilita el trabajo. Al contrario, muestra comprensión del modelo.

Comunicar los errores no significa mostrar un fracaso, sino demostrar conocimiento del sistema y de sus límites.

Por supuesto, no siempre podemos revisar todos los errores si el dataset es grande. Pero podemos revisar una muestra. Podemos mirar los errores más confiados, los errores más grandes, errores por clase, errores por subgrupo o ejemplos cercanos al umbral. Lo importante es no quedarse solamente con el promedio.

No hace falta revisar todos los errores, pero sí una muestra útil y representativa.

No revisar errores individuales nos deja con una evaluación incompleta. Las métricas resumen el desempeño, pero los errores concretos muestran cómo falla el modelo, en qué casos se equivoca, si hay patrones problemáticos, si existen etiquetas dudosas o si el desempeño cambia según grupos, valores extremos o situaciones específicas. Un buen análisis de Machine Learning no termina en una métrica: continúa mirando los casos donde el modelo no funcionó como esperábamos.

Sobreinterpretar conclusiones

El último error que veremos en este capítulo es sobreinterpretar las conclusiones. Esto ocurre cuando extraemos afirmaciones más fuertes de lo que el modelo, los datos o la evaluación realmente permiten sostener. Es un error especialmente importante porque puede aparecer incluso cuando el código está bien escrito y las métricas son correctas.

Un modelo puede ayudarnos a detectar patrones, comparar alternativas, estimar valores, clasificar casos o generar alertas. Pero sus resultados siempre dependen de los datos usados, de las variables disponibles, de la forma de evaluación, del modelo elegido y del contexto donde se aplicará.

Un resultado de Machine Learning no habla por sí solo: debe interpretarse dentro de las condiciones en las que fue obtenido.

Por ejemplo, si un modelo obtiene buen desempeño en un dataset de prueba, podemos decir que funcionó bien en esa evaluación. Pero no deberíamos afirmar automáticamente que funcionará igual en cualquier contexto, en cualquier período o con cualquier población.

Concepto	Descripción
Más prudente	el modelo obtuvo buen desempeño en este conjunto de prueba.
Más riesgoso	el modelo funciona bien en general.

Esta diferencia puede parecer pequeña, pero es importante. La primera frase describe lo que efectivamente medimos. La segunda extiende la conclusión más allá de la evidencia disponible.

Sobreinterpretar también ocurre cuando confundimos predicción con explicación. Un modelo puede predecir bien sin explicar de manera completa el fenómeno. Puede encontrar asociaciones útiles entre variables, pero eso no significa que haya descubierto las causas reales.

Predecir bien no significa necesariamente explicar completamente el fenómeno.

Por ejemplo, un modelo puede predecir abandono de clientes usando frecuencia de uso, reclamos, antigüedad y cambios recientes en el consumo. Si funciona bien, esas variables son señales útiles para anticipar abandono. Pero no podemos concluir automáticamente que esas variables sean las causas profundas. Detrás puede haber problemas de precio, calidad del servicio, cambios personales, competencia, contexto económico u otros factores que no están en el dataset.

El modelo puede encontrar señales, pero las causas requieren un análisis más profundo.

También debemos tener cuidado con las variables importantes. Si una variable aparece como muy relevante para el modelo, eso no significa que sea la causa del resultado ni que modificarla vaya a cambiar la situación. Significa que, dentro de ese modelo y con esos datos, esa variable ayudó a predecir.

Concepto	Descripción
Variable importante	el modelo la usó como señal.
No necesariamente	la variable causa el resultado.

Esta confusión puede llevar a decisiones equivocadas. Si una plataforma educativa observa que la cantidad de accesos está asociada con mejores resultados, no debería concluir sin más que aumentar accesos producirá mejores aprendizajes. Tal vez los accesos sean una señal de organización, conectividad, disponibilidad de tiempo o familiaridad con la plataforma. Forzar más accesos no necesariamente resuelve el problema de fondo.

Concepto	Descripción
Asociación observada	más accesos, mejores resultados.
Conclusión riesgosa	más accesos causan mejor aprendizaje.

Sobreinterpretar también puede aparecer al comunicar clusters. En aprendizaje no supervisado, un algoritmo puede agrupar datos según similitudes en las variables disponibles. Pero esos grupos no son necesariamente categorías naturales, definitivas o universales. Son agrupamientos contruidos por un método, con ciertos datos y ciertos parámetros.

Un cluster no es una verdad natural.

Es un grupo sugerido por un algoritmo según ciertas variables.

Si usamos K-Means para segmentar clientes, por ejemplo, podemos obtener tres grupos. Pero no deberíamos presentar esos grupos como si fueran tipos reales e inmutables de personas. Son una forma útil de resumir patrones, siempre que se interpreten con cuidado.

Lo mismo ocurre con PCA. Dos componentes principales pueden ayudarnos a visualizar datos, pero no debemos olvidar que son combinaciones matemáticas de variables. Si vemos separación entre puntos en un gráfico, eso puede ser interesante, pero no necesariamente demuestra que existan grupos claramente definidos en la realidad.

Una visualización puede sugerir patrones interesantes.

No siempre demuestra conclusiones definitivas.

Otro caso de sobreinterpretación aparece cuando una métrica mejora levemente. Supongamos que un modelo obtiene F1-score de 0.842 y otro 0.846. Técnicamente, el segundo número es mayor. Pero no siempre esa diferencia justifica afirmar que el segundo modelo es claramente superior. Puede deberse a variabilidad de la partición, azar, detalles del dataset o diferencias poco relevantes en la práctica.

Una mejora pequeña en una métrica no siempre implica una mejora importante en la práctica.

Antes de concluir que un modelo es mejor, conviene revisar estabilidad, validación cruzada, errores concretos, complejidad, interpretabilidad y costo de implementación. A veces un modelo apenas menos preciso, pero más simple y explicable, puede ser más adecuado.

También debemos evitar conclusiones demasiado amplias a partir de datos limitados. Si entrenamos un modelo con datos de una ciudad, no podemos asumir automáticamente que funcionará igual en todo un país. Si usamos datos de un período particular, no podemos garantizar que se mantendrá en otro. Si el dataset proviene de una plataforma específica, no necesariamente representa a quienes no usan esa plataforma.

Los datos limitados producen conclusiones limitadas.

Esto no significa que el trabajo no sea útil. Significa que debemos comunicar su alcance. Un buen informe no solo dice qué se encontró, sino también bajo qué condiciones se encontró.

Una conclusión responsable incluye: resultado, contexto, límites y cautelas.

Sobreinterpretar también puede ocurrir cuando olvidamos la calidad de las etiquetas. En aprendizaje supervisado, tratamos `y` como la respuesta correcta. Pero en muchos problemas reales, esa respuesta puede ser una decisión humana, un registro imperfecto o una clasificación discutible. Si las etiquetas tienen errores o sesgos, el modelo puede aprenderlos.

Por eso, decir “el modelo predice correctamente” significa, en realidad, “el modelo coincide con las etiquetas disponibles” bajo cierta evaluación. Si las etiquetas son problemáticas, esa coincidencia debe interpretarse con cuidado.

Buen desempeño respecto de las etiquetas no garantiza verdad absoluta sobre el fenómeno.

También debemos evitar presentar probabilidades como certezas. Si un modelo dice que un caso tiene 0.80 de probabilidad de pertenecer a una clase, eso no significa que el resultado sea seguro. Además, esa probabilidad puede no estar perfectamente calibrada. Es una estimación del modelo, no una garantía.

Una probabilidad alta no significa certeza, y una probabilidad baja no significa imposibilidad.

En contextos de decisión, esta diferencia es fundamental. Una puntuación de riesgo no debería convertirse automáticamente en una etiqueta definitiva sobre una persona, un cliente, un estudiante o una situación. Debe usarse como una señal, junto con contexto y revisión cuando corresponda.

El lenguaje ayuda a evitar sobreinterpretaciones. Hay expresiones más prudentes y expresiones más riesgosas.

Concepto	Descripción
Más prudente	"el modelo sugiere" "en estos datos se observa" "la variable aparece asociada" "la evaluación indica"
Más riesgoso	"el modelo demuestra" "la variable causa" "esto siempre ocurre" "el sistema sabe"

Usar un lenguaje prudente no debilita el análisis. Al contrario, lo vuelve más riguroso. Reconoce la diferencia entre evidencia, interpretación e hipótesis.

También es importante no presentar el modelo como solución completa del problema. Machine Learning puede ser una herramienta dentro de un proceso más amplio, pero rara vez reemplaza por completo el conocimiento del dominio, la revisión de datos, el criterio profesional o la evaluación de consecuencias.

Un modelo puede apoyar una decisión, pero no debería reemplazar todo el análisis del problema.

Por ejemplo, un modelo puede ayudar a priorizar casos de posible abandono escolar. Pero decidir qué hacer con esa alerta requiere conocer la situación del estudiante, los recursos disponibles, la institución, la familia, la conectividad y otros factores que el modelo no necesariamente captura.

Una buena conclusión debería distinguir entre lo que el modelo muestra, lo que inferimos y lo que todavía no sabemos. Esta separación ayuda a evitar afirmaciones exageradas.

Concepto	Descripción
El modelo muestra	patrones en los datos.
Podemos inferir	posibles explicaciones o hipótesis.
Todavía no sabemos	causas definitivas ni comportamiento universal.

Sobreinterpretar conclusiones significa decir más de lo que los datos y el modelo permiten afirmar. Un buen resultado no garantiza validez universal. Una variable importante no prueba causalidad. Un cluster no define una categoría natural. Una probabilidad no es una certeza. Una mejora pequeña no siempre es relevante. Para comunicar Machine Learning con responsabilidad, debemos explicar resultados junto con sus límites, usar lenguaje prudente y distinguir entre predicción, interpretación, hipótesis y decisión.

Capítulo 31 · Comunicar resultados de Machine Learning

Explicar el problema

Comunicar resultados de Machine Learning no consiste solamente en mostrar métricas, gráficos o nombres de modelos. Antes de hablar del algoritmo, de las variables o del rendimiento, es necesario explicar con claridad qué problema se intentó resolver. Si el problema no está bien presentado, el resto del informe pierde contexto.

Un modelo siempre responde a una pregunta. Puede intentar predecir si un cliente abandonará un servicio, estimar el precio de una vivienda, detectar una transacción sospechosa, clasificar reclamos, agrupar usuarios o identificar casos que podrían necesitar revisión. Esa pregunta debe quedar clara desde el comienzo.

Primera pregunta de un informe: ¿qué problema intentamos resolver?

Explicar el problema significa describir la situación, el objetivo del análisis y el tipo de decisión o conocimiento que se espera obtener. No alcanza con decir “se entrenó un modelo de clasificación” o “se aplicó Random Forest”. Eso describe una herramienta, no el problema.

Concepto	Descripción
Herramienta	modelo de clasificación.
Problema	anticipar qué clientes podrían abandonar el servicio.

La diferencia es importante. Una persona que lee el informe necesita entender por qué se construyó el modelo y para qué podría servir. El algoritmo viene después. Primero debe aparecer la necesidad que motiva el trabajo.

Por ejemplo, en lugar de comenzar así:

Se entrenó un modelo KNN con una accuracy de 0.84.

Sería mejor comenzar así:

El objetivo del análisis fue construir un modelo capaz de anticipar qué clientes presentan mayor riesgo de abandonar el servicio.

La segunda versión permite entender el propósito. Luego sí tendrá sentido explicar qué datos se usaron, qué modelo se entrenó y qué rendimiento obtuvo.

También es importante indicar si el problema es de clasificación, regresión, agrupamiento u otro tipo. Pero esa clasificación técnica debería conectarse con el objetivo real.

Concepto	Descripción
Objetivo real	anticipar abandono.
Tipo de problema	clasificación binaria.

Si el modelo predice una categoría, estamos ante un problema de clasificación. Si predice un valor numérico, es un problema de regresión. Si busca grupos sin una etiqueta previa, es un problema de clustering. Esta aclaración ayuda a ubicar el análisis y a entender qué métricas serán relevantes más adelante.

Por ejemplo:

Como la variable objetivo indica si el cliente abandonó o no, el problema se formuló como una clasificación binaria.

O bien:

Como se busca estimar el precio de una vivienda, el problema se formuló como una regresión.

En aprendizaje no supervisado, la explicación debe ser todavía más cuidadosa, porque no hay una respuesta correcta conocida. Por ejemplo, si usamos K-Means para segmentar clientes, no deberíamos decir que el modelo “descubrió los tipos reales de clientes”. Es más prudente decir que el objetivo fue explorar posibles agrupamientos según las variables disponibles.

En clustering, el problema no es predecir una respuesta, sino explorar estructura en los datos.

Además del objetivo, conviene explicar el contexto de uso. ¿El modelo se usaría para apoyar una decisión? ¿Para ordenar casos por prioridad? ¿Para explorar patrones? ¿Para comparar alternativas? ¿Para generar hipótesis? Esta diferencia cambia la forma en que se interpretan los resultados.

No es lo mismo: explorar datos, apoyar una decisión, automatizar una acción o generar una hipótesis.

Por ejemplo, un modelo que predice riesgo de abandono puede usarse para priorizar acciones de retención. Un modelo que detecta posibles fraudes puede usarse para enviar casos a revisión. Un modelo que estima demanda puede ayudar a planificar stock. En cada caso, el resultado del modelo se conecta con una acción distinta.

Si el informe no explica esa conexión, la métrica queda flotando. Saber que un modelo tiene F1-score de 0.78 no significa mucho si no sabemos qué decisión intenta apoyar ni qué errores son más importantes.

Una métrica tiene sentido cuando sabemos qué problema evalúa y qué consecuencias tienen sus errores.

También conviene aclarar el momento de predicción. Esta idea ya apareció en capítulos anteriores, pero en la comunicación es fundamental. No basta con decir qué se quiere predecir; también hay que decir cuándo se quiere predecir y con qué información disponible.

Concepto	Descripción
Pregunta incompleta	¿qué clientes abandonarán?
Pregunta más clara	¿qué clientes tienen riesgo de abandonar durante el próximo mes, según la información disponible hasta hoy?

Esta precisión ayuda a evitar ambigüedades. También muestra si las variables usadas tienen sentido. Si el objetivo es anticipar un evento, el lector debe saber que el modelo no usó información posterior a ese evento.

En problemas educativos, por ejemplo, no es lo mismo predecir desempeño final al inicio del curso que después del primer examen. Las variables disponibles cambian. El propósito también puede cambiar.

Concepto	Descripción
Mismo fenómeno	desempeño final.
Distintos momentos	antes de comenzar, a mitad del curso, después de una evaluación.

Cada formulación produce un modelo distinto. Por eso, explicar el momento de predicción es parte de explicar el problema.

También es útil mencionar el alcance. ¿El modelo se entrenó para una institución, una plataforma, una región, un período o un tipo de caso específico? ¿Se espera que funcione solo dentro de ese contexto? ¿Se está haciendo una prueba exploratoria o una propuesta de uso más formal?

Alcance: dónde, cuándo y sobre qué casos tiene sentido interpretar el resultado.

Un informe responsable no debería sugerir que el modelo sirve para cualquier situación si solo fue evaluado en un contexto limitado. Por ejemplo, si los datos corresponden a ventas de una empresa durante un año, las conclusiones deben

ubicarse en ese marco. Si los datos provienen de una plataforma educativa específica, no necesariamente representan a todos los estudiantes.

Explicar el problema también implica decir qué queda fuera. Un modelo de predicción de abandono puede anticipar casos con mayor riesgo, pero no necesariamente explicar todas las causas del abandono. Un modelo de precios puede estimar valores según ciertas variables, pero no capturar factores que no están en los datos. Un agrupamiento puede sugerir segmentos, pero no definir categorías definitivas.

Explicar el problema también ayuda a explicar qué no puede responder el modelo.

Esta aclaración evita sobreinterpretaciones. Si el objetivo fue predecir, no deberíamos presentar el resultado como explicación causal. Si el objetivo fue explorar, no deberíamos presentarlo como una clasificación definitiva. Si el modelo es un prototipo, no deberíamos comunicarlo como un sistema listo para producción.

Una forma sencilla de redactar el inicio de un informe es responder cuatro preguntas:

Para analizar este punto conviene preguntarse: ¿Qué se quiere predecir o analizar? ¿Por qué es relevante? ¿Qué tipo de problema es? ¿En qué contexto se usaría el resultado?

Por ejemplo:

El objetivo del trabajo fue construir un modelo para anticipar qué clientes podrían abandonar el servicio durante el próximo mes.

El problema se formuló como una clasificación binaria, donde la clase positiva representa el abandono.

El resultado podría usarse como apoyo para priorizar acciones de retención, no como una decisión automática.

Este tipo de introducción es clara porque conecta el objetivo, el tipo de problema y el uso previsto. No entra todavía en detalles técnicos, pero prepara al lector para entenderlos.

En regresión, una redacción similar podría ser:

El objetivo del trabajo fue estimar el precio de viviendas a partir de sus características.

Como la variable objetivo es un valor numérico, el problema se formuló como una regresión.

El modelo busca apoyar una estimación inicial, pero no reemplaza una tasación profesional ni el análisis del contexto inmobiliario.

En clustering:

El objetivo del trabajo fue explorar posibles grupos de clientes según sus patrones de compra.

Como no se dispone de una etiqueta previa, el problema se abordó mediante aprendizaje no supervisado.

Los grupos obtenidos deben interpretarse como segmentos sugeridos por el modelo, no como categorías definitivas.

Estos ejemplos muestran que explicar el problema no requiere un lenguaje complicado. Requiere precisión.

Un error frecuente es empezar el informe desde el código: “se importaron librerías”, “se cargó el dataset”, “se entrenó un modelo”. Ese orden puede servir en un cuaderno de trabajo, pero en un informe final conviene comenzar desde la pregunta y el propósito.

Concepto	Descripción
Cuaderno de trabajo	puede seguir el orden del código.
Informe	debe seguir el orden de la explicación.

La comunicación no tiene que reproducir cada paso técnico en el mismo orden en que fue programado. Debe organizar la información para que otra persona entienda qué se hizo, por qué se hizo y qué se puede concluir.

También es importante adaptar la explicación al público. Si el informe está dirigido a personas técnicas, podemos incluir más detalles sobre algoritmos, validación y métricas. Si está dirigido a personas no técnicas, debemos enfatizar el problema, los datos, los resultados principales, los errores y las limitaciones.

Comunicar bien no significa decir todo.

Significa decir lo necesario para que el resultado se entienda correctamente.

En cualquier caso, el problema debe quedar claro. Incluso un público técnico necesita saber qué se intentó resolver. Incluso un público no técnico puede comprender si el objetivo está bien formulado.

Comunicar resultados de Machine Learning comienza por explicar el problema. Antes de hablar del modelo, debemos describir qué se quiere predecir o analizar, por qué importa, qué tipo de problema es, cuándo se haría la predicción, qué información estaría disponible y cómo podría usarse el resultado. Una buena explicación inicial da contexto a las métricas, evita interpretaciones exageradas y permite que el resto del informe tenga sentido.

Describir los datos

Después de explicar el problema, un informe de Machine Learning debe describir los datos utilizados. Esta descripción es fundamental porque el modelo aprende a partir de esos datos. Si no sabemos de dónde vienen, qué contienen, qué período cubren, qué variables incluyen o qué limitaciones tienen, no podemos interpretar correctamente los resultados.

Un modelo no se entiende solo mirando el algoritmo. También hay que entender el dataset.

El proceso parte de Problema, continúa con datos disponibles, luego modelo y conduce finalmente a resultados.

Describir los datos no significa listar todas las columnas sin contexto. Significa presentar la información necesaria para que otra persona comprenda con qué evidencia se trabajó y qué alcance tienen las conclusiones.

Por ejemplo, no alcanza con decir:

Se usó un dataset de clientes.

Es mucho más útil decir:

Se utilizó un dataset con registros de clientes activos durante el período enero-junio de 2025.

Cada fila representa un cliente y las columnas incluyen información de uso del servicio, antigüedad, reclamos, plan contratado y estado de abandono al mes siguiente.

Esta segunda versión permite entender qué representa cada fila, qué tipo de información contiene la tabla y cuál es la relación con el problema.

Uno de los primeros puntos que conviene aclarar es la unidad de análisis. Es decir, qué representa cada fila del dataset. Puede ser una persona, una transacción, una vivienda, una venta, un reclamo, un producto, una sesión, un estudiante o un período de tiempo.

Unidad de análisis: qué representa cada fila del dataset.

Este detalle parece simple, pero evita muchas confusiones. Si cada fila representa una compra, no debemos interpretarla como si representara un cliente. Si cada fila representa un registro mensual, no debemos interpretarla como si fuera una persona única. Si cada fila representa una vivienda, no debemos hablar de transacciones si el dataset no las contiene.

Cada fila puede representar: un cliente, una operación, un producto, una vivienda, un reclamo, un estudiante o un período.

También conviene indicar el tamaño del dataset. La cantidad de filas y columnas ayuda a dimensionar el análisis. No se interpreta igual un modelo entrenado con 300 registros que uno entrenado con 300.000. Tampoco es lo mismo trabajar con 8 variables que con 200.

Tamaño del dataset: cantidad de filas, cantidad de columnas y, si corresponde, cantidad de clases.

Por ejemplo:

El dataset contiene 12.450 registros y 18 variables.

Luego de la limpieza inicial, se conservaron 11.980 registros para el entrenamiento y la evaluación.

Si hubo eliminación de filas o columnas, conviene mencionarlo. No hace falta detallar cada operación menor, pero sí las decisiones que afectan el alcance de los datos.

Otro aspecto importante es la variable objetivo, cuando el problema es supervisado. Debemos explicar qué representa **y**, cómo fue definida y qué valores puede tomar.

Variable objetivo: lo que el modelo intenta predecir.

En clasificación, conviene indicar cuáles son las clases. En clasificación binaria, cuál es la clase positiva y cuál la negativa. Esto es especialmente importante porque

métricas como precision, recall y F1-score dependen de qué clase estamos analizando como positiva.

Clase positiva: el evento que queremos detectar o priorizar.

Por ejemplo:

La variable objetivo indica si el cliente abandonó el servicio durante el mes siguiente.

Se codificó como 1 para abandono y 0 para permanencia.

En este análisis, la clase positiva corresponde al abandono.

En regresión, debemos indicar la unidad del valor que se predice. Si el modelo estima precios, ventas, consumo, temperatura, tiempo o demanda, el informe debe dejar claro en qué unidades se mide.

En regresión, la unidad de la variable objetivo es clave para interpretar el error.

Por ejemplo:

La variable objetivo es el precio de venta de cada vivienda, expresado en pesos.

Por lo tanto, los errores del modelo también se interpretan en pesos.

También es importante describir las variables de entrada. No hace falta explicar una por una si hay muchas, pero sí agruparlas por tipo o por función. Podemos decir, por ejemplo, que se usaron variables demográficas, variables de comportamiento, variables de historial de compra, características del producto o información temporal.

Variables de entrada: qué información usa el modelo para producir la predicción.

Una descripción clara podría ser:

Las variables de entrada incluyen: antigüedad del cliente, cantidad de reclamos recientes, tipo de plan, frecuencia de uso, monto mensual promedio y canal principal de atención.

O, en un problema de viviendas:

Las variables predictoras incluyen superficie, cantidad de ambientes, antigüedad, ubicación, estado general y presencia de cochera.

Esta descripción permite que el lector entienda qué información tuvo disponible el modelo. También ayuda a detectar posibles limitaciones. Si un factor importante no está en los datos, conviene decirlo.

Describir variables también ayuda a mostrar qué información no está disponible.

Por ejemplo, en un modelo de precios de viviendas, tal vez no tengamos información sobre estado real de mantenimiento, cercanía a transporte o calidad del barrio. En un modelo educativo, quizá no tengamos información sobre conectividad, contexto familiar o estudio fuera de la plataforma. Esas ausencias limitan lo que el modelo puede aprender.

Otro punto clave es el período temporal. Si los datos corresponden a un período específico, debe indicarse. Esto ayuda a interpretar si el modelo podría seguir siendo válido más adelante.

Los datos tienen tiempo: período observado, fecha de corte y posible vigencia.

Por ejemplo:

Los datos corresponden a operaciones realizadas entre enero de 2024 y marzo de 2025.

El modelo fue evaluado con registros posteriores dentro de ese mismo período.

En problemas donde el tiempo importa, también conviene explicar cómo se hizo la separación entre entrenamiento y prueba. Si se usó una separación aleatoria, se puede mencionar. Si se respetó el orden temporal, también.

Cuando el problema es temporal, la forma de separar datos forma parte de la descripción.

Otro aspecto importante es la distribución de clases. En clasificación, especialmente cuando hay desbalance, conviene mostrar cuántos casos hay en cada clase. Esto ayuda a entender por qué se eligieron ciertas métricas.

Distribución de clases: cuántos casos hay de cada categoría.

Por ejemplo:

La clase de abandono representa el 18% de los casos, mientras que la clase de permanencia representa el 82%.

Por este desbalance, la evaluación no se basó únicamente en accuracy.

Esta explicación conecta directamente los datos con las métricas. No aparece como un detalle aislado, sino como una razón para tomar decisiones de evaluación.

También conviene mencionar valores faltantes, duplicados, categorías raras o problemas de calidad. No siempre hace falta entrar en todos los detalles, pero sí señalar los aspectos que pueden afectar el modelo.

Calidad de datos: faltantes, duplicados, valores extremos, categorías poco frecuentes y posibles inconsistencias.

Por ejemplo:

Se detectaron valores faltantes en las variables de ingresos y antigüedad.

Los faltantes se trataron mediante imputación dentro del pipeline de entrenamiento, para evitar fuga de datos.

Esta frase comunica dos cosas importantes: que había faltantes y que el tratamiento se hizo de manera correcta, respetando la separación entre entrenamiento y prueba.

También podemos mencionar si hubo variables excluidas. Esto es especialmente importante si se descartaron variables por fuga de datos, disponibilidad futura, sensibilidad o baja calidad.

Variables excluidas: también forman parte de una descripción responsable del dataset.

Por ejemplo:

Se excluyeron variables posteriores al abandono, como fecha de baja y motivo de baja, porque no estarían disponibles en el momento real de predicción.

Esta aclaración muestra criterio y evita dudas. También indica que se revisó el riesgo de fuga de datos.

En problemas sensibles, describir los datos también implica mencionar limitaciones de representación. Si algunos grupos aparecen poco, si ciertos canales están mejor registrados que otros o si la información proviene solo de una fuente, el informe debería advertirlo.

Un dataset puede ser grande y aun así no representar bien a todos los grupos o contextos.

Por ejemplo:

El dataset proviene de interacciones registradas en la plataforma digital.

Por lo tanto, puede representar mejor a usuarios con mayor conectividad y menor uso de canales presenciales.

Este tipo de aclaración es importante porque evita que el modelo se interprete como si estuviera basado en una visión completa de la realidad.

También es útil mostrar una vista resumida de los datos. En un informe, puede incluirse una tabla con cantidad de registros, período, variable objetivo, número de variables, porcentaje de faltantes y distribución de clases. En un cuaderno, además, podemos mostrar `df.head()`, `df.info()` o `df.describe()`, pero en un informe final conviene traducir esos resultados a lenguaje explicativo.

Concepto	Descripción
Cuaderno	muestra salidas técnicas.
Informe	explica qué significan esas salidas.

Por ejemplo, en vez de pegar una salida extensa de `df.info()`, podemos escribir:

El dataset combina variables numéricas y categóricas. Las variables numéricas principales son antigüedad, frecuencia de uso y monto promedio. Las variables categóricas incluyen plan, zona y canal de atención.

La descripción de los datos debe ser suficiente para que otra persona entienda qué evidencia sostiene el modelo. No necesita ser interminable, pero sí clara, honesta y conectada con el problema.

Una forma sencilla de organizar esta sección es responder estas preguntas:

Para analizar este punto conviene preguntarse: ¿Qué representa cada fila? ¿Cuántos registros y variables hay? ¿Cuál es la variable objetivo? ¿Qué variables de entrada se usaron? ¿Qué período cubren los datos? ¿Qué problemas de calidad se detectaron? ¿Qué limitaciones tiene el dataset?

Describir los datos es una parte esencial de comunicar resultados de Machine Learning. El lector debe saber qué representa el dataset, cuántos casos contiene, qué variable se intenta predecir, qué información usa el modelo, qué período cubren los registros, qué problemas de calidad existen y qué limitaciones deben tenerse en cuenta. Sin esta descripción, las métricas y conclusiones quedan incompletas. Un modelo solo puede interpretarse correctamente cuando entendemos los datos desde los que aprendió.

Justificar el enfoque

Después de explicar el problema y describir los datos, un informe de Machine Learning debe justificar el enfoque utilizado. Esto significa explicar por qué se eligió cierto tipo de modelo, cierto preprocesamiento, cierta forma de evaluación y ciertas métricas. No alcanza con decir qué se hizo; también conviene explicar por qué tenía sentido hacerlo.

Comunicar un enfoque: qué se hizo y por qué se hizo.

Justificar el enfoque ayuda a que el trabajo sea comprensible y evaluable. Si una persona lee que se usó regresión logística, Random Forest, K-Means o PCA, necesita saber por qué esa herramienta era adecuada para el problema. La elección no debería parecer arbitraria.

Por ejemplo, no es lo mismo escribir:

Se entrenó un modelo Random Forest.

Que escribir:

Se entrenó un modelo Random Forest porque permite capturar relaciones no lineales entre las variables y suele funcionar bien como modelo base en problemas tabulares.

La segunda versión no solo informa la herramienta usada, sino que conecta la elección con una razón.

La justificación debe partir del tipo de problema. Si la variable objetivo es una categoría, tiene sentido usar modelos de clasificación. Si es un valor numérico, corresponde usar modelos de regresión. Si no hay variable objetivo y se busca explorar estructura, puede tener sentido usar clustering o reducción de dimensionalidad.

El proceso parte de Tipo de problema y conduce a familia de modelos adecuada.

Por ejemplo:

Como la variable objetivo indica si el cliente abandonó o no, el problema se abordó como una clasificación binaria.

O bien:

Como se busca estimar un valor numérico continuo, el problema se abordó como una regresión.

Esta aclaración parece básica, pero es importante porque muestra coherencia entre el objetivo y la técnica.

También conviene justificar el preprocesamiento. Si el dataset contiene variables numéricas y categóricas, valores faltantes o variables con escalas muy distintas, el informe debería explicar cómo se trataron y por qué.

El proceso parte de Datos reales, continúa con limpieza y transformación y conduce finalmente a modelo.

Por ejemplo:

Las variables numéricas se imputaron con la mediana y luego se escalaron.

Las variables categóricas se imputaron con la categoría más frecuente y se codificaron mediante One-Hot Encoding.

Pero una justificación más completa agrega el motivo:

Se utilizó imputación para conservar registros con valores faltantes.

Las variables categóricas se codificaron con One-Hot Encoding porque los modelos de scikit-learn requieren variables numéricas.

El escalado se incluyó porque algunos modelos evaluados son sensibles a la escala de las variables.

Esta explicación muestra que el preprocesamiento no fue una serie de pasos automáticos, sino una respuesta a características concretas de los datos y de los modelos.

También es importante mencionar si el preprocesamiento se hizo dentro de un pipeline. Esto muestra que se cuidó la separación entre entrenamiento y prueba y que se evitó fuga de datos.

El preprocesamiento se integró en un Pipeline para que las transformaciones se ajustaran solo con los datos de entrenamiento.

Esta frase comunica una decisión técnica y, al mismo tiempo, una buena práctica metodológica.

La elección de modelos también debe justificarse según el objetivo del análisis. A veces queremos un modelo simple e interpretable. Otras veces buscamos mayor capacidad predictiva. En algunos casos comparamos un modelo base con uno más complejo. Esa intención debe quedar clara.

No siempre buscamos el modelo más complejo.

A veces buscamos: simplicidad, interpretabilidad, estabilidad o buen desempeño predictivo.

Por ejemplo:

Se entrenó una regresión logística como modelo inicial, porque ofrece una línea de base simple, rápida de entrenar y relativamente interpretable.

Luego podría agregarse:

También se evaluó Random Forest para comparar el desempeño con un modelo más flexible, capaz de capturar relaciones no lineales.

Esta estructura es clara porque diferencia el rol de cada modelo. La regresión logística funciona como línea de base; Random Forest como alternativa más flexible.

En regresión, una justificación similar podría ser:

Se utilizó regresión lineal como modelo inicial porque permite estimar una relación simple entre las variables y el valor objetivo.

Luego se comparó con modelos más flexibles para evaluar si la mejora de desempeño justificaba una mayor complejidad.

En aprendizaje no supervisado, la justificación debe explicar qué se esperaba obtener, porque no hay una métrica única equivalente a la accuracy o al MAE. Por ejemplo:

Se aplicó K-Means para explorar posibles grupos de clientes según patrones de compra.

El método se eligió como una primera aproximación porque permite obtener segmentos simples y fáciles de analizar a partir de centroides.

También podría decirse:

Se aplicó PCA para reducir la dimensionalidad y visualizar la estructura general de los datos en dos componentes principales.

En estos casos, la justificación debe evitar exageraciones. K-Means no “descubre” grupos definitivos; propone agrupamientos según las variables y el número de clusters elegido. PCA no explica todo el fenómeno; construye una representación reducida.

También hay que justificar la forma de evaluación. Si se usó train/test split, validación cruzada o una separación temporal, conviene explicar por qué.

Evaluar no es solo calcular una métrica.

También implica decidir cómo se separan los datos para estimar generalización.

Por ejemplo:

Los datos se separaron en entrenamiento y prueba para evaluar el desempeño del modelo sobre casos no vistos durante el entrenamiento.

Si se usó validación cruzada:

Se utilizó validación cruzada de 5 folds para obtener una estimación más estable del desempeño y reducir la dependencia de una única partición.

Si el problema tiene dimensión temporal:

Como el objetivo es predecir eventos futuros, la evaluación respetó el orden temporal: el modelo se entrenó con datos anteriores y se evaluó con datos posteriores.

Esta última justificación es especialmente importante porque muestra que la evaluación intenta parecerse al uso real del modelo.

También debemos justificar las métricas. No alcanza con reportar accuracy, F1-score, MAE o RMSE. Conviene explicar por qué esas métricas son adecuadas para el problema.

La métrica elegida debe corresponder al objetivo del modelo y al tipo de error relevante.

En clasificación con clases desbalanceadas, por ejemplo:

Debido al desbalance entre clases, la evaluación no se basó únicamente en accuracy.

Se reportaron precision, recall y F1-score para analizar mejor el desempeño sobre la clase de interés.

Si los falsos negativos son especialmente importantes:

Se prestó especial atención al recall, porque en este problema no detectar casos positivos tiene un costo mayor que generar algunas alertas adicionales.

Si los falsos positivos son más costosos:

Se priorizó precisión, porque una predicción positiva incorrecta puede generar una revisión innecesaria o una acción no deseada.

En regresión:

Se utilizó MAE porque expresa el error promedio en las mismas unidades que la variable objetivo, lo que facilita su interpretación.

O:

También se reportó RMSE porque penaliza con más fuerza los errores grandes, relevantes en este problema.

Justificar métricas ayuda a evitar una comunicación superficial. El lector entiende no solo qué número obtuvo el modelo, sino por qué ese número importa.

También es útil justificar decisiones relacionadas con el umbral de clasificación. Si se usa el umbral por defecto de 0.5, puede mencionarse. Si se ajusta, debe explicarse con claridad.

El umbral de decisión define cuándo una probabilidad se transforma en clase positiva.

Por ejemplo:

Se evaluó el modelo con el umbral estándar de 0.5 como punto de partida.

Luego se analizaron umbrales alternativos para observar el equilibrio entre falsos positivos y falsos negativos.

Si se eligió un umbral específico por razones operativas:

Se eligió un umbral más bajo para aumentar la detección de casos positivos, aceptando un mayor número de falsos positivos que deberán ser revisados manualmente.

Esta explicación conecta el modelo con la decisión real. No presenta el umbral como un detalle técnico menor.

También puede ser necesario justificar qué no se hizo. Por ejemplo, si no se usó un modelo muy complejo, si no se aplicó cierta técnica o si no se incluyó una variable, conviene explicar el motivo cuando sea relevante.

Una buena justificación también puede explicar por qué se descartaron ciertas alternativas.

Por ejemplo:

No se incluyeron variables posteriores al evento, como fecha de baja o motivo de baja, porque no estarían disponibles en el momento real de predicción.

O:

No se priorizó el modelo de mayor complejidad porque la mejora obtenida fue pequeña y se buscó conservar interpretabilidad.

Este tipo de aclaración muestra criterio. No todo lo que se puede probar debe incorporarse al modelo final.

En un informe, justificar el enfoque no significa escribir una explicación extensa de cada algoritmo. No hace falta desarrollar toda la teoría de Random Forest, SVM o Gradient Boosting si el objetivo es comunicar un resultado aplicado. Lo importante es explicar por qué esa elección fue razonable para ese problema y esos datos.

No se trata de enseñar todo el algoritmo.

Se trata de explicar por qué fue una elección adecuada en este análisis.

Una forma sencilla de organizar esta sección es responder:

Para analizar este punto conviene preguntarse: ¿Qué tipo de problema es? ¿Qué modelos se probaron y por qué? ¿Qué preprocesamiento fue necesario? ¿Cómo se evitó fuga de datos? ¿Cómo se evaluó el modelo? ¿Por qué se eligieron esas métricas?

Con esas respuestas, el lector puede seguir el razonamiento del trabajo. Aunque no conozca todos los detalles técnicos, entiende que las decisiones no fueron arbitrarias.

Justificar el enfoque significa explicar las decisiones metodológicas del proyecto: por qué el problema se formuló como clasificación, regresión o clustering; por qué se eligieron ciertos modelos; qué preprocesamiento fue necesario; cómo se cuidó la evaluación; qué métricas se usaron y por qué. Una buena justificación no recarga el informe con teoría innecesaria, pero sí muestra que cada decisión técnica está conectada con el problema, los datos y el uso previsto del modelo.

Presentar métricas relevantes

Una vez explicado el problema, descriptos los datos y justificado el enfoque, llega el momento de presentar los resultados. En Machine Learning, esos resultados suelen expresarse mediante métricas. Pero comunicar métricas no significa simplemente

escribir una lista de números. Hay que elegir cuáles mostrar, explicar qué significan y relacionarlas con el problema que se está intentando resolver.

Una métrica relevante es aquella que ayuda a entender si el modelo cumple el objetivo para el cual fue construido.

Métrica relevante: un número que ayuda a evaluar si el modelo sirve para el problema planteado.

No todas las métricas tienen la misma importancia en todos los problemas. La accuracy puede ser útil en algunas clasificaciones, pero insuficiente en clases desbalanceadas. El recall puede ser central si nos importa detectar la mayor cantidad posible de casos positivos. La precision puede ser más importante si queremos evitar falsas alarmas. En regresión, MAE, RMSE y R^2 responden preguntas distintas.

Por eso, antes de mostrar métricas, debemos recordar qué problema estamos evaluando.

El proceso parte de Problema, continúa con tipo de errores importantes y conduce finalmente a métricas adecuadas.

En un informe, conviene evitar presentar métricas como si fueran números aislados. Por ejemplo, una frase como esta resulta incompleta:

El modelo obtuvo un F1-score de 0.81.

Es mejor acompañarla con una interpretación:

El modelo obtuvo un F1-score de 0.81 sobre el conjunto de prueba.

Esta métrica resume el equilibrio entre precision y recall para la clase de abandono.

La segunda versión no solo informa el resultado, sino que ayuda a entender qué mide.

También es importante aclarar sobre qué conjunto se calculó la métrica. No es lo mismo una métrica de entrenamiento, de validación cruzada o de prueba. Si el resultado corresponde al conjunto de prueba, debe decirse. Si es un promedio de validación cruzada, también.

Una métrica debe indicar: qué mide, sobre qué datos se calculó y cómo debe interpretarse.

Por ejemplo:

En el conjunto de prueba, el modelo obtuvo una accuracy de 0.86 y un F1-score de 0.74 para la clase positiva.

O bien:

Mediante validación cruzada de 5 folds, el modelo obtuvo un F1-score promedio de 0.78, con una desviación estándar de 0.04.

En el segundo caso, se comunica no solo el promedio, sino también la variabilidad. Eso ayuda a interpretar si el desempeño fue estable o dependió mucho de las particiones.

Concepto	Descripción
Promedio	resultado general.
Desviación	variabilidad entre evaluaciones.

En clasificación, una forma útil de presentar métricas es comenzar por la matriz de confusión. La matriz muestra cuántos casos fueron clasificados correctamente y qué tipos de errores cometió el modelo. Ayuda a que precision, recall y F1-score no aparezcan como fórmulas desconectadas.

Matriz de confusión: muestra aciertos, falsos positivos y falsos negativos.

Por ejemplo, en un problema de abandono de clientes, podemos explicar:

La matriz de confusión permite observar cuántos clientes que abandonaron fueron detectados correctamente y cuántos quedaron sin detectar.

Luego se pueden presentar métricas más resumidas. Si la clase positiva es abandono, recall indica qué proporción de clientes que realmente abandonaron fue detectada. Precision indica qué proporción de los clientes señalados como riesgo efectivamente abandonó.

Concepto	Descripción
Recall alto	se detectan más casos positivos.
Precision alta	hay menos falsas alarmas entre los casos señalados.

La elección de cuál destacar depende del objetivo. Si el modelo se usará para generar alertas que luego serán revisadas manualmente, quizá se acepte menor precision a cambio de mayor recall. Si cada alerta tiene un costo alto, puede priorizarse precision.

La métrica principal debe conectarse con el uso previsto del modelo.

Por ejemplo:

En este problema se priorizó recall, porque el objetivo principal es no dejar sin detectar casos de abandono potencial.

Esto implica aceptar un mayor número de falsos positivos, que deberán revisarse antes de actuar.

Esta explicación muestra que la métrica no fue elegida al azar. También reconoce el costo de esa elección.

En cambio, si el problema exige evitar falsas alarmas:

En este caso se priorizó precisión, porque cada predicción positiva activa una revisión manual costosa.

Por eso, se buscó que los casos señalados tuvieran una alta probabilidad de corresponder realmente a la clase positiva.

En problemas multiclase, conviene presentar métricas por clase. Un promedio global puede ocultar que el modelo funciona bien en algunas clases y mal en otras.

En clasificación multiclase, mirar solo el promedio puede ocultar errores en clases específicas.

Por ejemplo:

Aunque el F1-score promedio fue 0.82, la clase "reclamos técnicos" obtuvo un F1-score de 0.61.

Esto indica que el modelo tuvo más dificultades para identificar correctamente esa categoría.

Este tipo de lectura es mucho más informativa que presentar solo el promedio.

En clasificación desbalanceada, también conviene aclarar la proporción de clases. Si una clase representa solo el 5% de los casos, una accuracy alta puede ser poco significativa. La métrica debe leerse junto con esa distribución.

Una métrica de clasificación se interpreta mejor cuando conocemos la distribución de clases.

Por ejemplo:

La accuracy fue de 0.93, pero la clase positiva representa solo el 7% de los casos.

Por ese motivo, se analizaron también recall y F1-score para evaluar la detección de la clase minoritaria.

En regresión, presentar métricas relevantes implica traducir el error a unidades comprensibles. Si usamos MAE, conviene explicar qué significa en el contexto del problema.

En regresión, un error se entiende mejor cuando se expresa en unidades reales.

Por ejemplo:

El modelo obtuvo un MAE de 8.500 pesos.

Esto significa que, en promedio, sus predicciones se alejaron 8.500 pesos del valor real.

Si usamos RMSE, conviene aclarar que penaliza más los errores grandes.

El RMSE fue de 13.200 pesos.

Como esta métrica penaliza más los errores grandes, sugiere que existen algunos casos con diferencias importantes entre valor real y predicho.

El R^2 también puede ser útil, pero debe explicarse con cuidado. No siempre es la métrica más intuitiva para públicos no técnicos. Puede acompañar al MAE o RMSE, pero no reemplazar la interpretación del error en unidades reales.

R^2 ayuda a resumir ajuste, pero MAE o RMSE ayudan a interpretar la magnitud del error.

Por ejemplo:

El modelo alcanzó un R^2 de 0.79, lo que indica que capturó una parte importante de la variabilidad del precio.

De todos modos, el MAE permite interpretar mejor el tamaño promedio del error en unidades monetarias.

También es útil comparar las métricas con una línea de base. Un modelo puede tener una métrica que parece razonable, pero necesitamos saber si mejora realmente una estrategia simple.

Una línea de base permite preguntar: ¿el modelo mejora una solución simple?

En clasificación, una línea de base puede ser predecir siempre la clase mayoritaria. En regresión, puede ser predecir siempre la media o la mediana de la variable

objetivo. Si nuestro modelo apenas mejora esa referencia, tal vez no aporte demasiado.

Modelo útil: debería superar una estrategia simple de referencia.

Por ejemplo:

La línea de base, que predice siempre la clase mayoritaria, obtiene una accuracy de 0.82.

El modelo propuesto obtiene una accuracy de 0.86, pero además mejora el recall de la clase positiva.

Esta comparación permite entender si el modelo agrega valor.

También conviene presentar las métricas de manera ordenada. Si se comparan varios modelos, una tabla puede ser útil. Pero la tabla debe ir acompañada de una interpretación. No basta con dejar los números para que el lector saque conclusiones.

Una tabla de métricas debe acompañarse con una lectura clara.

Por ejemplo:

El modelo Random Forest obtuvo el mejor F1-score, pero la regresión logística mostró un desempeño cercano con mayor simplicidad e interpretabilidad.

Por ese motivo, la elección final dependerá de si se prioriza desempeño o facilidad de explicación.

Esta lectura evita caer en la idea de que el número más alto siempre gana. El informe debe explicar qué significan las diferencias.

También debemos evitar sobrecargar con demasiadas métricas. Mostrar muchas columnas puede dar una apariencia técnica, pero confundir al lector. Es mejor seleccionar las métricas principales y agregar otras como apoyo cuando sean necesarias.

Más métricas no siempre significan mejor comunicación.

Una buena práctica es distinguir entre métrica principal y métricas complementarias.

Concepto	Descripción
Métrica principal	orienta la evaluación del modelo.
Métricas complementarias	ayudan a entender mejor sus errores y comportamiento.

Por ejemplo:

La métrica principal fue F1-score, por el desbalance entre clases.

También se reportaron precision y recall para analizar el tipo de errores cometidos por el modelo.

En problemas donde se usan probabilidades, puede ser relevante presentar curvas ROC, AUC o curvas Precision-Recall. Pero estas métricas deben aparecer solo si ayudan a responder la pregunta del problema. No deberían incluirse como adornos técnicos.

Una métrica avanzada solo es útil si aporta información para interpretar o decidir.

Si el informe menciona AUC, conviene explicar que mide la capacidad del modelo para ordenar casos positivos por encima de negativos en distintos umbrales. Si la clase positiva es muy minoritaria, puede ser útil complementar con una curva Precision-Recall.

Pero, nuevamente, lo importante es conectar la métrica con el problema, no acumular nombres.

El objetivo no es mostrar todas las métricas posibles.

El objetivo es mostrar las métricas que ayudan a tomar una conclusión responsable.

Finalmente, al presentar métricas, debemos usar un lenguaje prudente. Una métrica no garantiza desempeño futuro en cualquier contexto. Representa el comportamiento observado bajo una evaluación concreta.

Concepto	Descripción
Más prudente	en esta evaluación, el modelo obtuvo...
Más riesgoso	el modelo garantiza...

Por ejemplo:

En el conjunto de prueba, el modelo obtuvo un F1-score de 0.81.

Este resultado sugiere un desempeño razonable para detectar la clase positiva en datos similares a los evaluados.

Esta frase comunica el resultado y su alcance. No exagera.

Presentar métricas relevantes significa elegir números que tengan sentido para el problema, explicar qué miden, indicar sobre qué datos se calcularon y relacionarlos con los errores que importan. En clasificación, puede ser necesario mirar matriz de confusión, precision, recall, F1-score y métricas por clase. En regresión, conviene traducir MAE, RMSE o R^2 a una interpretación comprensible. Una buena comunicación no muestra métricas por acumulación, sino por relevancia: cada número debe ayudar a entender si el modelo es útil, confiable y adecuado para el objetivo planteado.

Mostrar errores y limitaciones

Un informe de Machine Learning no debería mostrar únicamente los buenos resultados. También debe mostrar los errores y las limitaciones del modelo. Esto puede parecer una debilidad, pero en realidad fortalece el análisis. Un informe que reconoce dónde falla el modelo es más confiable que uno que solo presenta métricas favorables.

Todo modelo se equivoca. La pregunta no es si hay errores, sino qué tipo de errores comete, en qué casos aparecen y qué consecuencias podrían tener.

Comunicar errores: no es admitir fracaso.

Es mostrar cómo se comporta realmente el modelo.

En clasificación, una forma clara de mostrar errores es usar la matriz de confusión. La matriz permite observar aciertos, falsos positivos y falsos negativos. A diferencia de una métrica única, muestra la estructura de los errores.

Matriz de confusión: aciertos, errores, falsos positivos y falsos negativos.

Por ejemplo, en un modelo de abandono de clientes, la matriz de confusión permite responder preguntas importantes: ¿cuántos clientes que abandonaron fueron detectados? ¿Cuántos quedaron sin detectar? ¿Cuántos clientes fueron señalados como riesgo aunque finalmente no abandonaron?

No basta con saber cuánto acertó el modelo.

También importa saber qué tipo de errores cometió.

Un falso negativo puede representar un cliente que efectivamente abandonó, pero que el modelo no detectó como riesgo. Un falso positivo puede representar un cliente señalado como riesgo, aunque finalmente permaneció. Según el uso del modelo, cada error tendrá un costo distinto.

Concepto	Descripción
Falso negativo	caso importante no detectado.
Falso positivo	caso señalado innecesariamente.

Por eso, al comunicar errores, conviene conectarlos con el problema real. No alcanza con decir que hubo cierta cantidad de falsos positivos y falsos negativos. Debemos explicar qué significan en el contexto del análisis.

El proceso parte de Error técnico y conduce a consecuencia práctica.

Por ejemplo:

Los falsos negativos representan clientes que abandonaron el servicio sin haber sido identificados previamente como casos de riesgo.

Este tipo de error es relevante porque limita la posibilidad de realizar acciones de retención.

O bien:

Los falsos positivos representan clientes señalados como riesgo que finalmente no abandonaron.

Estos casos pueden generar acciones innecesarias, por lo que deben considerarse al definir el umbral de decisión.

Este tipo de explicación ayuda a que el lector comprenda el impacto de los errores.

También es importante revisar si los errores se concentran en ciertas clases, grupos o situaciones. Un modelo puede tener buen desempeño global y, aun así, fallar en una clase minoritaria, en ciertos valores extremos, en una región, en un período o en casos con pocos datos.

Una métrica global puede ocultar errores concentrados en ciertos casos.

Por ejemplo:

El modelo mostró buen desempeño general, pero tuvo más dificultades para detectar casos de abandono en clientes con poca antigüedad.

Esta frase comunica una limitación concreta. No invalida todo el modelo, pero advierte dónde debe usarse con más cuidado.

En clasificación multiclase, mostrar errores por clase es especialmente importante. Si un modelo clasifica reclamos en varias categorías, puede funcionar bien en

categorías frecuentes y mal en categorías raras. En ese caso, el promedio general puede ser engañoso.

En clasificación multiclase, conviene revisar qué clases se confunden entre sí.

Por ejemplo:

El modelo confundió con frecuencia reclamos de facturación con reclamos administrativos.

Esto sugiere que ambas categorías comparten vocabulario o que las etiquetas podrían necesitar una definición más precisa.

Este tipo de análisis no solo evalúa el modelo. También puede revelar problemas en los datos o en la definición de las clases.

En regresión, mostrar errores implica analizar cuánto se alejan las predicciones de los valores reales. Las métricas como MAE o RMSE resumen esos errores, pero también conviene revisar casos con errores grandes.

En regresión: no mirar solo el error promedio.

También revisar los casos donde el error fue especialmente alto.

Por ejemplo:

Aunque el MAE fue razonable, los errores más altos aparecieron en viviendas de gran superficie y ubicaciones poco frecuentes.

Esto indica que el modelo puede ser menos confiable en propiedades alejadas del perfil típico del dataset.

Esta explicación es mucho más útil que presentar solo el MAE. Muestra dónde el modelo funciona peor y ayuda a definir límites de uso.

También puede ser útil analizar si el modelo tiende a sobreestimar o subestimar. En un modelo de precios, por ejemplo, quizá las viviendas más caras se predicen por debajo de su valor real. En un modelo de demanda, quizá los picos de consumo se subestiman. Esa información es relevante para la toma de decisiones.

Concepto	Descripción
Error promedio	cuánto se equivoca.
Tendencia del error	si suele predecir por encima o por debajo.

Por ejemplo:

El modelo tiende a subestimar los valores más altos de demanda.

Por lo tanto, puede ser menos adecuado para planificar situaciones de pico sin una revisión adicional.

Además de los errores, un informe debe mostrar limitaciones. Una limitación no es necesariamente un error del modelo. Puede ser una condición que restringe el alcance de los resultados.

Limitación: condición que reduce el alcance o la confianza de una conclusión.

Las limitaciones pueden venir de los datos. Por ejemplo, el dataset puede cubrir solo un período corto, una región, una plataforma, ciertos tipos de usuarios o una cantidad limitada de casos. También puede tener valores faltantes, clases desbalanceadas, etiquetas dudosas o variables importantes ausentes.

Limitaciones de datos: período limitado, poca representación, faltantes, etiquetas imperfectas, variables ausentes.

Por ejemplo:

Los datos corresponden únicamente a interacciones registradas en la plataforma digital.

Por eso, el modelo puede representar mejor a usuarios con actividad online que a quienes usan canales presenciales.

Esta aclaración evita que el lector generalice el resultado más allá de lo que los datos permiten.

También hay limitaciones del modelo. Un modelo simple puede no capturar relaciones complejas. Un modelo complejo puede ser menos interpretable. Un modelo entrenado con datos históricos puede no adaptarse bien si cambia el contexto. Un modelo con buen desempeño promedio puede fallar en casos raros.

Limitaciones del modelo: simplicidad, complejidad, interpretabilidad, estabilidad, sensibilidad a cambios.

Por ejemplo:

Random Forest obtuvo el mejor desempeño, pero ofrece menor interpretabilidad que una regresión logística.

Por ese motivo, sus predicciones deberían acompañarse con análisis de importancia de variables y revisión de errores.

O bien:

El modelo fue entrenado con datos históricos.

Si cambian fuertemente los hábitos de los usuarios o las condiciones del servicio, su desempeño podría degradarse.

Otra limitación puede estar en la evaluación. Si se usó una única separación train/test, el resultado puede depender de esa partición. Si el dataset es pequeño, la métrica puede tener mayor variabilidad. Si no hubo prueba en un período futuro, no deberíamos afirmar que el modelo funcionará igual más adelante.

Limitaciones de evaluación: una sola partición, pocos datos, sin prueba temporal, muestra poco representativa.

Por ejemplo:

La evaluación se realizó con una única separación train/test.

Sería conveniente complementarla con validación cruzada para estimar mejor la estabilidad del desempeño.

O:

Aunque el modelo fue evaluado en un conjunto de prueba separado, no se realizó una validación sobre datos de un período posterior.

Por eso, la capacidad de generalizar a escenarios futuros debe interpretarse con cautela.

Estas frases no desvalorizan el trabajo. Lo vuelven más honesto.

También es importante mostrar limitaciones relacionadas con el uso. Un modelo puede estar pensado para apoyar decisiones, no para automatizarlas. Puede servir para priorizar casos, pero no para etiquetar personas de manera definitiva. Puede sugerir riesgo, pero no demostrar una causa.

El uso previsto también define las limitaciones del modelo.

Por ejemplo:

El modelo debe interpretarse como una herramienta de apoyo para priorizar revisiones, no como un sistema de decisión automática.

O:

Las predicciones indican mayor o menor probabilidad estimada, pero no deben interpretarse como certezas sobre cada caso individual.

Este tipo de aclaración es especialmente importante en contextos sensibles, educativos, financieros, laborales o institucionales.

Mostrar errores y limitaciones también ayuda a proponer mejoras. Si sabemos dónde falla el modelo, podemos sugerir pasos concretos: recolectar más datos, mejorar etiquetas, agregar variables, revisar el umbral, probar modelos alternativos, evaluar por subgrupos o monitorear desempeño en el tiempo.

El proceso parte de Error detectado y conduce a posible mejora.

Por ejemplo:

Dado que el modelo presenta más errores en clientes nuevos, una mejora posible sería incorporar variables específicas de los primeros días de uso o recolectar más ejemplos de ese segmento.

O:

Como las clases minoritarias tienen menor desempeño, sería conveniente revisar su representación, analizar ejemplos mal clasificados y considerar estrategias de balanceo.

Una buena sección de errores y limitaciones no debe convertirse en una lista interminable de advertencias. Debe concentrarse en los puntos que realmente afectan la interpretación del resultado.

No se trata de mencionar todas las dudas posibles.

Se trata de señalar las limitaciones relevantes para interpretar bien el modelo.

También es importante el tono. Mostrar limitaciones no significa disculparse por el trabajo. Significa explicar con precisión qué puede afirmarse y qué no.

Buen tono: claro, prudente, concreto y orientado a mejorar.

Comunicar errores y limitaciones es una parte esencial de un informe de Machine Learning. Las métricas muestran el desempeño general, pero los errores revelan cómo falla el modelo y en qué casos debe usarse con cuidado. Las limitaciones explican el alcance de los datos, del modelo, de la evaluación y del uso previsto. Un informe responsable no oculta estos aspectos: los presenta con claridad para que las conclusiones sean útiles, honestas y proporcionadas a la evidencia disponible.

Comparar modelos sin exagerar

Comparar modelos es una parte importante de comunicar resultados de Machine Learning. Muchas veces probamos más de un algoritmo, más de una estrategia de preprocesamiento o más de un conjunto de hiperparámetros. El informe debe explicar esas comparaciones con claridad, pero también con prudencia. No se trata solo de decir cuál obtuvo el número más alto, sino de interpretar si esa diferencia realmente importa.

Comparar modelos: mirar desempeño, estabilidad, errores, complejidad e interpretación.

Un error frecuente es presentar una tabla de métricas y declarar automáticamente ganador al modelo con el mejor valor. Por ejemplo, si un modelo obtiene F1-score de 0.812 y otro 0.816, técnicamente el segundo resultado es mayor. Pero esa diferencia puede ser demasiado pequeña para sostener una conclusión fuerte.

Modelo A: F1-score = 0.812

Modelo B: F1-score = 0.816

¿B es claramente mejor? No necesariamente.

Antes de afirmar que un modelo es superior, conviene revisar si la diferencia es consistente, si se mantiene en validación cruzada, si aparece también en otras métricas relevantes y si tiene impacto práctico. Una mejora mínima puede no justificar mayor complejidad, menor interpretabilidad o más costo de implementación.

Una mejora numérica pequeña no siempre equivale a una mejora útil.

Por eso, el lenguaje del informe debe ser cuidadoso. En lugar de decir “el modelo B es mejor”, puede ser más preciso decir “el modelo B obtuvo el mayor F1-score en esta evaluación, aunque la diferencia con el modelo A fue pequeña”.

Concepto	Descripción
Más prudente	obtuvo el mejor resultado en esta evaluación.
Más riesgoso	es el mejor modelo en general.

Esta forma de comunicar evita exagerar. El resultado queda asociado al procedimiento de evaluación usado, no a una afirmación universal.

También debemos considerar la estabilidad. Un modelo puede obtener un resultado alto en una partición train/test, pero variar mucho en validación cruzada. Otro puede tener un promedio apenas menor, pero resultados más consistentes. En muchos contextos, la estabilidad es tan importante como el desempeño promedio.

Concepto	Descripción
Modelo estable	resultados similares en distintas particiones.
Modelo inestable	resultados muy variables según cómo se dividan los datos.

Por ejemplo:

El modelo de Gradient Boosting obtuvo el F1-score promedio más alto, pero también presentó mayor variabilidad entre folds.

La regresión logística tuvo un desempeño menor, aunque más estable.

Esta comparación es más rica que decir simplemente cuál ganó. Permite que el lector entienda el intercambio entre desempeño y estabilidad.

También hay que comparar el tipo de errores. Dos modelos pueden tener métricas globales parecidas, pero equivocarse de maneras distintas. Uno puede tener mayor recall y detectar más casos positivos, pero generar más falsos positivos. Otro puede tener mayor precision, pero dejar más casos positivos sin detectar.

Métricas similares pueden esconder errores diferentes.

Por ejemplo:

Random Forest obtuvo mayor recall, por lo que detectó más casos de abandono.

Sin embargo, también produjo más falsos positivos, lo que implicaría revisar más clientes que finalmente no abandonarían.

O bien:

La regresión logística obtuvo menor recall, pero mayor precision.

Esto significa que señaló menos casos, aunque con menor proporción de falsas alarmas.

Este tipo de comparación conecta los modelos con las consecuencias reales de sus errores. No reduce la decisión a un ranking.

También debemos considerar la interpretabilidad. Un modelo más complejo puede tener mejor métrica, pero ser más difícil de explicar. En algunos contextos, eso no es un problema grave. En otros, puede ser una limitación importante.

Mayor desempeño puede venir acompañado de: mayor complejidad, menor interpretabilidad o mayor costo de mantenimiento.

Por ejemplo:

Random Forest obtuvo mejor desempeño que la regresión logística.

Sin embargo, la regresión logística permite una interpretación más directa de la influencia de las variables, lo que puede ser valioso si el objetivo incluye explicar el modelo.

Esta comparación no niega el buen desempeño de Random Forest. Simplemente muestra que la elección depende del contexto.

También puede ocurrir que un modelo simple sea suficiente. Si un modelo lineal obtiene un resultado cercano al de un modelo mucho más complejo, quizá convenga elegir el más simple, especialmente si es más fácil de explicar, mantener o implementar.

Si dos modelos rinden parecido, la simplicidad puede ser un criterio importante.

Por ejemplo:

Aunque Gradient Boosting obtuvo el mejor desempeño, la mejora frente a la regresión logística fue reducida.

Dado que la regresión logística es más simple e interpretable, podría ser una opción razonable para una primera implementación.

Este tipo de conclusión es responsable porque no se deja llevar únicamente por el número más alto.

También conviene comparar los modelos con una línea de base. Una línea de base es una estrategia simple que sirve como referencia. En clasificación, puede ser predecir

siempre la clase mayoritaria. En regresión, puede ser predecir siempre la media o la mediana.

Línea de base: referencia simple para saber si el modelo realmente aporta valor.

Por ejemplo:

La línea de base obtuvo una accuracy de 0.82 al predecir siempre la clase mayoritaria.

Los modelos entrenados superaron esa referencia, especialmente en recall de la clase positiva.

Esta comparación ayuda a mostrar si el modelo agrega algo más que una regla trivial. Si un modelo complejo apenas mejora la línea de base, su utilidad debe ponerse en duda.

También es importante explicar si los modelos fueron comparados en condiciones justas. Deben evaluarse con la misma partición de datos, la misma métrica principal y un preprocesamiento adecuado para cada uno. Si un modelo recibió mucho ajuste de hiperparámetros y otro se probó con valores por defecto, conviene aclararlo.

Una comparación justa requiere: misma evaluación, misma métrica principal y condiciones razonables para cada modelo.

Por ejemplo:

Todos los modelos fueron evaluados con la misma separación train/test y la misma métrica principal.

Las transformaciones de preprocesamiento se aplicaron mediante pipelines para evitar fuga de datos.

O:

Random Forest fue ajustado mediante búsqueda de hiperparámetros, mientras que la regresión logística se evaluó con su configuración base.

Por lo tanto, la comparación debe interpretarse como una comparación entre estos pipelines probados, no como una conclusión definitiva sobre ambos algoritmos.

Esta aclaración evita conclusiones exageradas. Comunicar bien también implica decir qué tan justa o limitada fue la comparación.

En algunos informes, puede ser útil separar la comparación en dos etapas. Primero se comparan modelos base. Luego se ajustan uno o dos candidatos. Esto permite ordenar el proceso y evitar que parezca una búsqueda descontrolada.

Concepto	Descripción
Primera etapa	comparar modelos base.
Segunda etapa	ajustar los candidatos más prometedores.

Por ejemplo:

En una primera etapa, se compararon modelos base para identificar alternativas prometedoras.

Luego se ajustaron hiperparámetros en los dos modelos con mejor desempeño inicial.

También debe evitarse decir que un modelo “demostró ser mejor” si la evaluación fue limitada. Tal vez fue mejor en una partición, con un dataset específico y una métrica determinada. Eso no equivale a una superioridad general.

Un modelo puede ser mejor en esta evaluación sin ser mejor en todos los contextos.

Una redacción prudente podría ser:

En esta evaluación, Random Forest obtuvo el mejor equilibrio entre precision y recall.

Sin embargo, la diferencia con la regresión logística fue moderada, por lo que ambas alternativas podrían considerarse según el nivel de interpretabilidad requerido.

Esta conclusión deja espacio para una decisión contextual.

Además del desempeño, puede compararse la facilidad de implementación. Un modelo que requiere mucho ajuste, alto costo computacional o transformaciones difíciles de mantener puede no ser la mejor opción en un entorno real, aunque tenga una métrica levemente superior.

Modelo aplicable: no solo predice bien, también puede mantenerse, explicarse y usarse en el contexto real.

Por ejemplo:

Aunque el modelo más complejo obtuvo una mejora leve en la métrica, su costo de entrenamiento y menor interpretabilidad podrían dificultar su uso operativo.

Esto muestra que comunicar Machine Learning no es solo reportar resultados, sino ayudar a tomar decisiones informadas.

También es útil mostrar modelos descartados. No hace falta dedicar mucho espacio a cada uno, pero puede ser importante explicar por qué no se eligieron. Tal vez tuvieron bajo desempeño, alta variabilidad, demasiados falsos positivos o complejidad innecesaria.

Explicar modelos descartados ayuda a mostrar que la elección final no fue arbitraria.

Por ejemplo:

KNN fue descartado porque mostró menor desempeño y mayor sensibilidad al escalado en comparación con los otros modelos evaluados.

O:

SVM obtuvo resultados competitivos, pero el tiempo de entrenamiento fue considerablemente mayor y la mejora no justificó su complejidad para este análisis.

Estas frases son útiles porque dan razones concretas.

Finalmente, al comparar modelos, debemos recordar que la elección final no siempre es puramente técnica. Puede depender de los objetivos del proyecto, del público que necesita entender el resultado, del riesgo de la decisión, de los recursos disponibles y del nivel de mantenimiento esperado.

Elegir modelo: desempeño, errores, estabilidad, interpretabilidad, costo y contexto.

Comparar modelos sin exagerar significa presentar resultados con claridad, pero también con cautela. El mejor número no siempre implica el mejor modelo. Debemos considerar diferencias reales de desempeño, estabilidad, tipos de error, interpretabilidad, complejidad, línea de base, condiciones de evaluación y uso previsto. Una comparación responsable no busca declarar un ganador absoluto, sino explicar qué alternativa parece más adecuada para el problema, bajo qué condiciones y con qué límites.

Escribir conclusiones responsables

La conclusión de un informe de Machine Learning no debería limitarse a repetir la métrica principal ni a declarar que un modelo “funciona”. Una buena conclusión debe integrar el problema, los datos, el enfoque, los resultados, los errores y las limitaciones. Su objetivo es responder, de manera clara y prudente, qué se puede afirmar a partir del trabajo realizado.

Una conclusión responsable: resume el resultado, explica su alcance y reconoce sus límites.

Un error frecuente es cerrar el informe con una frase demasiado fuerte, por ejemplo:

El modelo predice correctamente el abandono de clientes.

Esta frase es problemática porque suena absoluta. No dice bajo qué condiciones se evaluó el modelo, qué desempeño obtuvo, qué errores comete ni qué límites tienen los datos. Una redacción más responsable sería:

En el conjunto de prueba, el modelo mostró un desempeño razonable para anticipar casos de abandono, especialmente al identificar una parte importante de los clientes que efectivamente abandonaron.

La segunda frase no exagera. Ubica el resultado en la evaluación realizada y evita convertir la predicción en una certeza.

Una conclusión responsable debe empezar por volver al objetivo. Si el informe comenzó diciendo que se quería anticipar abandono, estimar precios, clasificar reclamos o explorar segmentos, la conclusión debe responder esa pregunta inicial.

Concepto	Descripción
Pregunta inicial	¿qué queríamos resolver?
Conclusión	¿qué pudimos responder con los datos y modelos usados?

Por ejemplo:

El objetivo del trabajo fue anticipar qué clientes presentan mayor riesgo de abandonar el servicio.

Los resultados sugieren que el modelo puede servir como una herramienta de apoyo para priorizar casos de revisión, aunque no debería utilizarse como decisión automática.

Esta conclusión conecta el objetivo con el uso posible. También marca un límite importante: apoyo, no automatización completa.

En un problema de regresión, una conclusión responsable podría decir:

El modelo permitió estimar precios de viviendas con un error promedio interpretable en unidades monetarias.

Sin embargo, los errores fueron mayores en propiedades de características poco frecuentes, por lo que las predicciones deberían usarse como referencia inicial y no como tasación definitiva.

La frase no solo comunica que el modelo estima precios. También explica dónde falla más y cómo debería usarse.

En clustering, una conclusión prudente podría ser:

El análisis permitió identificar tres grupos de clientes con patrones de compra diferenciados.

Estos grupos deben interpretarse como segmentos sugeridos por el algoritmo a partir de las variables disponibles, no como categorías definitivas ni universales.

Aquí se evita sobreinterpretar. El modelo sugiere grupos, pero no descubre verdades absolutas.

Una conclusión responsable también debe mencionar la métrica principal, pero no como un número aislado. Debe explicar qué significa en relación con el problema.

Concepto	Descripción
No alcanza con	F1-score = 0.81.
Hay que explicar	qué indica ese valor para el objetivo planteado.

Por ejemplo:

El F1-score de 0.81 indica un equilibrio razonable entre detectar casos positivos y evitar falsas alarmas en el conjunto de prueba.

Si una métrica tiene una interpretación práctica, conviene incluirla. En regresión, por ejemplo:

El MAE de 8.500 pesos indica que, en promedio, las predicciones se alejaron 8.500 pesos del valor real.

Esa frase ayuda al lector a entender el resultado en unidades concretas.

También es importante no ocultar errores. Si el modelo tiene una limitación relevante, debe aparecer en la conclusión. No hace falta repetir toda la sección de errores, pero sí mencionar los aspectos principales que afectan el uso del modelo.

Una conclusión completa incluye tanto resultados como advertencias relevantes.

Por ejemplo:

El modelo tuvo buen desempeño general, pero mostró menor recall en la clase minoritaria.

Por eso, si el objetivo es detectar la mayor cantidad posible de casos positivos, sería necesario ajustar el umbral o revisar estrategias adicionales.

Esta conclusión es útil porque no solo informa el problema, sino que sugiere un camino de mejora.

También conviene mencionar las limitaciones de los datos. Si el dataset cubre un período específico, una región, una plataforma o un conjunto parcial de casos, la conclusión debe reflejarlo.

Los resultados valen en relación con los datos sobre los que fueron evaluados.

Por ejemplo:

Como los datos provienen únicamente de interacciones digitales, el modelo puede representar mejor a usuarios activos en la plataforma que a quienes utilizan otros canales.

Esta aclaración evita que el lector generalice demasiado.

Una conclusión responsable también debe distinguir predicción de causalidad. Si el modelo detecta variables asociadas al resultado, no debemos afirmar automáticamente que esas variables causan el fenómeno.

Concepto	Descripción
Más prudente	la variable aparece asociada con la predicción.
Más riesgoso	la variable causa el resultado.

Por ejemplo:

Las variables de frecuencia de uso y reclamos recientes aparecieron como señales relevantes para anticipar abandono.

Esto no implica que sean las causas únicas del abandono, sino que aportaron información predictiva en los datos analizados.

Esta distinción es especialmente importante cuando el informe será leído por personas que podrían tomar decisiones a partir del modelo.

También debe quedar claro si el modelo está listo para usarse o si todavía es exploratorio. No todos los trabajos de Machine Learning terminan en un sistema

aplicable. Algunos sirven para probar una idea, entender datos, comparar enfoques o generar hipótesis.

Conclusión responsable: indica si el modelo es exploratorio, experimental, de apoyo o candidato a implementación.

Por ejemplo:

Los resultados son prometedores como análisis exploratorio, pero antes de una implementación sería necesario validar el modelo con datos más recientes y revisar su desempeño por subgrupos.

Esta frase evita presentar un prototipo como si fuera una solución final.

En contextos de mayor impacto, la conclusión debe mencionar la necesidad de revisión humana o monitoreo. Un modelo puede apoyar decisiones, pero no debería reemplazar completamente el análisis del contexto cuando hay consecuencias importantes.

A mayor impacto, más importante es aclarar: revisión, monitoreo y límites de uso.

Por ejemplo:

El modelo podría utilizarse para ordenar casos según prioridad, pero las decisiones finales deberían mantenerse bajo revisión humana, especialmente en situaciones ambiguas o de alto impacto.

También puede ser necesario indicar próximos pasos. Una conclusión no tiene que cerrar todo definitivamente. Puede señalar qué debería hacerse para mejorar o validar el trabajo.

Próximos pasos: validar, monitorear, recolectar datos, revisar errores, ajustar umbrales o comparar alternativas.

Por ejemplo:

Como próximo paso, se recomienda evaluar el modelo con datos de un período posterior para verificar si el desempeño se mantiene en el tiempo.

O:

También sería conveniente revisar los falsos negativos para entender qué casos positivos no fueron detectados y si comparten características comunes.

Estas recomendaciones muestran que el análisis no se limita a una métrica final, sino que abre un camino de mejora.

El tono de la conclusión es muy importante. Debe ser claro, directo y prudente. No conviene usar expresiones como “demuestra”, “garantiza”, “siempre”, “nunca” o “predice perfectamente”, salvo que estemos en un contexto muy controlado y aun así con cuidado.

Lenguaje prudente: sugiere, indica, en esta evaluación, en estos datos, podría utilizarse, requiere revisión.

Un buen cierre podría tener esta estructura:

1. Recordar el objetivo.
2. Resumir el resultado principal.
3. Interpretar la métrica relevante.
4. Mencionar errores o limitaciones.
5. Indicar uso recomendado o próximos pasos.

Por ejemplo:

El objetivo del trabajo fue anticipar clientes con riesgo de abandono.

En el conjunto de prueba, el modelo Random Forest obtuvo el mejor equilibrio entre precisión y recall, con un F1-score de 0.81.

El resultado sugiere que el modelo podría servir como apoyo para priorizar acciones de retención.

Sin embargo, se observaron falsos negativos en clientes con poca antigüedad, por lo que sería conveniente revisar ese segmento antes de una implementación.

El modelo debería utilizarse como herramienta de apoyo y no como decisión automática.

Esta conclusión es breve, pero contiene los elementos esenciales: objetivo, resultado, interpretación, límite y uso recomendado.

Escribir conclusiones responsables significa no exagerar lo que el modelo puede afirmar. Una buena conclusión responde la pregunta inicial, interpreta las métricas en contexto, reconoce errores y limitaciones, distingue predicción de causalidad y aclara el uso adecuado del modelo. El objetivo no es vender el resultado como perfecto, sino comunicar de manera honesta qué aporta el modelo, qué dudas quedan y qué cuidados se necesitan antes de usarlo para decidir.

¿Qué debería incluir un informe básico?

Después de recorrer las distintas partes de la comunicación, podemos reunir las en una estructura sencilla. Un informe básico de Machine Learning no necesita ser excesivamente largo ni complejo, pero sí debe permitir que otra persona entienda qué problema se abordó, con qué datos, qué se hizo, qué resultados se obtuvieron y qué límites tienen esas conclusiones.

La idea central es que el informe no sea solo una colección de salidas de código. Debe ser una explicación ordenada del trabajo realizado.

Un informe básico debe responder: qué problema, qué datos, qué enfoque, qué resultados, qué errores y qué conclusiones.

Una posible estructura comienza con una introducción del problema. Allí se explica qué se quiere predecir o analizar, por qué es relevante y cómo se formula la tarea. Esta sección debe ubicar al lector antes de entrar en detalles técnicos.

Para analizar este punto conviene preguntarse: 1. Problema ¿Qué se quiere resolver? ¿Por qué importa? ¿Qué tipo de problema es?

Por ejemplo:

El objetivo del trabajo fue anticipar qué clientes podrían abandonar el servicio durante el mes siguiente.

Como la variable objetivo indica abandono o permanencia, el problema se formuló como una clasificación binaria.

La segunda parte debería describir los datos. Conviene explicar qué representa cada fila, cuántos registros hay, cuál es la variable objetivo, qué variables de entrada se usaron, qué período cubren los datos y qué limitaciones o problemas de calidad se detectaron.

Para analizar este punto conviene preguntarse: 2. Datos ¿Qué representa cada fila? ¿Cuántos registros hay? ¿Cuál es la variable objetivo? ¿Qué variables se usaron? ¿Qué limitaciones tiene el dataset?

Una redacción posible sería:

Cada fila representa un cliente.

El dataset contiene información de uso del servicio, antigüedad, plan contratado, reclamos recientes y estado de abandono.

La clase positiva corresponde a clientes que abandonaron el servicio durante el período analizado.

Si hay desbalance de clases, valores faltantes, variables excluidas o posibles problemas de representación, también deberían mencionarse.

Describir datos incluye mostrar lo que los datos permiten ver y lo que podrían dejar fuera.

La tercera parte corresponde al enfoque metodológico. Aquí se explica qué modelos se probaron, qué preprocesamiento se aplicó, cómo se separaron los datos y qué métricas se eligieron. No hace falta desarrollar toda la teoría de cada algoritmo, pero sí justificar las decisiones principales.

Para analizar este punto conviene preguntarse: 3. Enfoque ¿Qué modelos se probaron? ¿Qué preprocesamiento se aplicó? ¿Cómo se evaluó? ¿Qué métricas se eligieron?

Por ejemplo:

Se comparó una regresión logística con un modelo Random Forest.

Las variables numéricas se imputaron y escalaron, mientras que las variables categóricas se codificaron con One-Hot Encoding.

Todo el preprocesamiento se integró en pipelines para evitar fuga de datos.

También puede mencionarse la forma de evaluación:

Los datos se separaron en entrenamiento y prueba para evaluar el desempeño del modelo sobre casos no vistos.

Debido al desbalance entre clases, se reportaron precision, recall y F1-score, además de accuracy.

La cuarta parte presenta los resultados. Esta sección debe mostrar las métricas relevantes y explicar qué significan. Si se comparan modelos, puede incluirse una tabla. Si se trata de clasificación, es recomendable incluir o describir la matriz de confusión. Si se trata de regresión, conviene interpretar los errores en unidades reales.

Para analizar este punto conviene preguntarse: 4. Resultados ¿Qué métricas se obtuvieron? ¿Qué modelo funcionó mejor? ¿Qué significan esos números?

Un ejemplo de redacción para clasificación:

En el conjunto de prueba, Random Forest obtuvo el mejor F1-score para la clase positiva.

Esto indica un mejor equilibrio entre detectar clientes que abandonan y evitar alertas innecesarias.

Un ejemplo para regresión:

El modelo obtuvo un MAE de 8.500 pesos.

Esto significa que, en promedio, las predicciones se alejaron 8.500 pesos del valor real.

La quinta parte debe mostrar errores y limitaciones. Esta sección es clave para evitar conclusiones exageradas. Debe explicar qué tipos de errores cometió el modelo, dónde parece fallar más, qué limitaciones tienen los datos o la evaluación y qué cuidados deberían tenerse al usar el resultado.

Para analizar este punto conviene preguntarse: 5. Errores y limitaciones ¿Qué errores comete el modelo? ¿Dónde falla más? ¿Qué no cubren los datos? ¿Qué cuidados requiere el resultado?

Por ejemplo:

El modelo presentó falsos negativos en clientes con poca antigüedad, lo que indica que podría no detectar algunos abandonos tempranos.

Además, los datos provienen únicamente del canal digital, por lo que el desempeño podría diferir en clientes que usan canales presenciales.

Estas aclaraciones ayudan a interpretar el resultado de manera responsable.

La sexta parte corresponde a la conclusión. Allí se retoma el objetivo inicial, se resume el resultado principal, se mencionan límites importantes y se indica un uso recomendado o próximos pasos.

Para analizar este punto conviene preguntarse: 6. Conclusión ¿Qué se pudo responder? ¿Qué modelo parece más adecuado? ¿Qué límites hay? ¿Qué se recomienda hacer después?

Una conclusión breve podría ser:

Los resultados sugieren que el modelo puede servir como herramienta de apoyo para priorizar acciones de retención.

Sin embargo, no debería utilizarse como decisión automática, ya que aún requiere revisión de errores, validación en datos recientes y monitoreo de desempeño.

Esta estructura básica puede adaptarse según el tipo de trabajo. Un informe exploratorio quizá dedique más espacio a los datos y a las visualizaciones. Un informe comparativo puede centrarse en modelos y métricas. Un informe orientado a implementación debe agregar detalles sobre uso, monitoreo, actualización y riesgos.

La estructura del informe depende del objetivo, pero siempre debe mantener claridad, contexto y límites.

También es importante diferenciar entre un cuaderno y un informe. Un cuaderno de trabajo puede incluir código, pruebas intermedias, salidas parciales y exploraciones. Un informe final debe seleccionar y ordenar la información más relevante. No necesita mostrar todo lo que se hizo, sino lo necesario para comprender y evaluar el resultado.

Concepto	Descripción
Cuaderno	registro de trabajo.
Informe	comunicación ordenada de resultados y conclusiones.

Esto no significa ocultar decisiones importantes. Significa evitar que el lector tenga que reconstruir el razonamiento a partir de muchas celdas de código. El informe debe guiar la lectura.

Una estructura sencilla de informe podría ser:

Título

1. Objetivo del análisis
2. Descripción de los datos
3. Preparación y enfoque
4. Resultados principales
5. Errores y limitaciones
6. Conclusiones y próximos pasos.

Si el informe es más breve, estas secciones pueden combinarse. Por ejemplo, “Datos y metodología” puede reunir la descripción del dataset y el enfoque. “Resultados y limitaciones” puede presentar métricas y errores. Lo importante es que ninguna pieza esencial desaparezca.

Un informe puede ser breve, pero no debería omitir contexto, datos, evaluación y límites.

En algunos casos, también puede incluirse un resumen ejecutivo al comienzo. Este resumen es útil cuando el lector necesita una primera visión rápida del resultado. Debe ser corto y claro: qué se hizo, qué se encontró y qué se recomienda.

Resumen ejecutivo: problema, resultado principal, limitación clave y recomendación.

Por ejemplo:

Se construyó un modelo para anticipar abandono de clientes.

El mejor resultado lo obtuvo Random Forest, con buen equilibrio entre precisión y recall.

El modelo podría usarse para priorizar revisiones, pero requiere validación adicional antes de una implementación operativa.

También puede ser útil incluir anexos técnicos. Allí pueden quedar detalles de hiperparámetros, tablas completas de métricas, gráficos adicionales o fragmentos de código. De esta manera, el cuerpo principal del informe se mantiene legible y los detalles quedan disponibles para quien los necesite.

Concepto	Descripción
Cuerpo principal	explicación y resultados clave.
Anexo	detalles técnicos adicionales.

El nivel de detalle debe ajustarse al público. Si el informe es para un equipo técnico, puede incluir más información sobre validación cruzada, pipelines, hiperparámetros y curvas de evaluación. Si es para un público general, debe priorizar lenguaje claro, interpretación de métricas, errores relevantes y decisiones recomendadas.

El informe debe adaptarse a quienes lo van a leer y a las decisiones que necesitan tomar.

Una buena práctica es revisar si el informe podría responder estas preguntas sin necesidad de mirar el código:

Para analizar este punto conviene preguntarse: ¿Entiendo qué problema se resolvió? ¿Sé qué datos se usaron? ¿Comprendo cómo se evaluó? ¿Sé qué tan bien funcionó? ¿Conozco sus errores principales? ¿Entiendo sus límites? ¿Queda claro cómo debería usarse?

Si alguna de estas preguntas no tiene respuesta, el informe probablemente necesita mejorar.

También conviene evitar algunas frases excesivas. Por ejemplo, no decir “el modelo demuestra”, “el algoritmo garantiza”, “la predicción es segura” o “los grupos descubiertos son definitivos”. En su lugar, es preferible usar expresiones como “los resultados sugieren”, “en esta evaluación”, “el modelo mostró”, “podría utilizarse como apoyo” o “requiere validación adicional”.

Buen informe: claro, prudente, concreto y conectado con el problema.

Un informe básico de Machine Learning debería incluir el objetivo del análisis, la descripción de los datos, el enfoque metodológico, las métricas relevantes, los errores observados, las limitaciones y las conclusiones responsables. No necesita ser extenso, pero sí completo en lo esencial. Su propósito no es impresionar con términos técnicos, sino ayudar a comprender qué aporta el modelo, qué tan confiable es, dónde puede fallar y cómo debería interpretarse su resultado.

Comunicar para que el modelo pueda entenderse

En este capítulo trabajamos una idea central: un buen proyecto de Machine Learning no termina cuando el modelo produce una métrica. También es necesario comunicar qué se hizo, por qué se hizo, qué resultados se obtuvieron, qué errores aparecieron y qué límites deben tenerse en cuenta.

Un modelo puede estar correctamente entrenado y evaluado, pero si se comunica mal, puede ser interpretado de manera equivocada. Una métrica aislada, una conclusión exagerada o una explicación incompleta pueden llevar a decisiones poco responsables.

Construir un modelo es una parte del trabajo.

Comunicarlo bien también forma parte del trabajo.

El primer paso para comunicar resultados es explicar el problema. Antes de hablar del algoritmo, debemos aclarar qué se quería predecir o analizar, por qué era importante, qué tipo de problema era y en qué contexto podría usarse el resultado. Esta explicación da sentido a todo lo demás.

Concepto	Descripción
Primero	el problema.
Después	los datos, el modelo, las métricas y las conclusiones.

Luego vimos la importancia de describir los datos. El lector necesita saber qué representa cada fila, qué variables se usaron, cuál fue la variable objetivo, qué período cubren los registros, qué problemas de calidad aparecieron y qué limitaciones tiene el dataset. Sin esta información, las métricas quedan incompletas.

También analizamos la necesidad de justificar el enfoque. No alcanza con decir que se usó un modelo determinado. Conviene explicar por qué ese enfoque era razonable para el problema, qué preprocesamiento fue necesario, cómo se evitó fuga de datos, cómo se evaluó el modelo y por qué se eligieron ciertas métricas.

Comunicar el enfoque: qué se hizo, cómo se hizo y por qué tenía sentido.

Después trabajamos la presentación de métricas relevantes. Una buena comunicación no consiste en mostrar todos los números posibles, sino en seleccionar aquellos que ayudan a entender el desempeño del modelo. En clasificación, pueden ser importantes la matriz de confusión, precision, recall, F1-score o métricas por clase. En regresión, pueden ser más útiles MAE, RMSE o R^2 , siempre interpretados en el contexto del problema.

Una métrica útil debe responder una pregunta relevante sobre el modelo.

También dedicamos una sección a mostrar errores y limitaciones. Esto es esencial. Un informe responsable no oculta dónde falla el modelo. Al contrario, muestra qué tipos de errores comete, en qué casos parece menos confiable, qué limitaciones tienen los datos y qué cuidados requiere su uso.

Mostrar limitaciones no debilita el informe.

Lo vuelve más honesto y más útil.

Luego vimos cómo comparar modelos sin exagerar. El modelo con el número más alto no siempre es automáticamente el mejor. También importan la estabilidad, los errores, la complejidad, la interpretabilidad, el costo de implementación y el contexto

de uso. Una comparación responsable explica bajo qué condiciones se obtuvo cada resultado.

Comparar modelos no es solo ordenar métricas.

Es interpretar diferencias y decidir si importan.

Finalmente, trabajamos cómo escribir conclusiones responsables. Una conclusión debe responder la pregunta inicial, interpretar los resultados en contexto, reconocer límites, distinguir predicción de causalidad y aclarar el uso adecuado del modelo. No debe prometer más de lo que los datos y la evaluación permiten sostener.

Conclusión responsable: resultado, alcance, limitaciones y próximos pasos.

Un informe básico de Machine Learning debería incluir, al menos, el objetivo del análisis, la descripción de los datos, el enfoque utilizado, las métricas relevantes, los errores observados, las limitaciones y las conclusiones. Esta estructura puede adaptarse, pero esos elementos ayudan a que el trabajo sea comprensible y revisable.

Informe básico: problema, datos, enfoque, resultados, errores, limitaciones y conclusiones.

La comunicación también debe cuidar el lenguaje. Es preferible decir “el modelo sugiere”, “en esta evaluación se observó” o “los resultados indican” antes que afirmar que el modelo demuestra, garantiza o explica completamente un fenómeno. Machine Learning trabaja con datos, patrones y estimaciones, no con certezas absolutas.

Lenguaje prudente: sugiere, indica, en estos datos, en esta evaluación, podría usarse como apoyo.

Comunicar bien no significa simplificar en exceso ni esconder la técnica. Significa organizar la información para que otra persona pueda entender qué se hizo y qué puede concluirse. Un público técnico necesitará más detalles metodológicos. Un público general necesitará más contexto e interpretación. En ambos casos, el objetivo es el mismo: que el resultado se entienda correctamente.

Comunicar resultados de Machine Learning es parte esencial del proceso. Un modelo no se evalúa solo por sus métricas, sino también por la claridad con que podemos explicar su propósito, sus datos, su desempeño, sus errores, sus límites y su uso recomendado. Un buen informe no busca impresionar con nombres de algoritmos, sino ayudar a tomar decisiones mejor informadas. Comunicar bien es convertir un trabajo técnico en conocimiento útil, prudente y revisable.

Capítulo 32 · Cómo seguir aprendiendo Machine Learning

¿Qué temas quedaron fuera del libro?

Llegar al final de un libro de Machine Learning no significa haber agotado el tema. En realidad, significa algo mucho más interesante: ahora tenemos una base desde la cual seguir aprendiendo.

A lo largo de este recorrido trabajamos con ideas fundamentales: datos, variables, entrenamiento, prueba, modelos de clasificación, modelos de regresión, métricas, validación, sobreajuste, modelos más potentes, aprendizaje no supervisado, interpretación, sesgos y comunicación de resultados. Estos conceptos forman una base sólida porque aparecen una y otra vez en distintos problemas reales.

Pero Machine Learning es un campo amplio. Mucho más amplio que cualquier libro introductorio. Siempre quedan temas afuera, no porque sean poco importantes, sino porque requieren tiempo, práctica y conocimientos previos para ser comprendidos con profundidad.

Terminar este libro no significa cerrar el tema.

Significa tener una base para seguir avanzando.

En este libro nos concentramos principalmente en modelos clásicos de Machine Learning, especialmente aquellos que pueden trabajarse con datos tabulares y con herramientas como `pandas`, `scikit-learn` y los recursos habituales de análisis de datos. Este enfoque fue intencional. Antes de avanzar hacia técnicas más complejas, conviene comprender bien cómo se formula un problema, cómo se preparan los datos, cómo se evalúa un modelo y cómo se interpretan sus resultados.

Los modelos más avanzados no eliminan estas preguntas. Al contrario, muchas veces las vuelven más importantes.

Modelos más complejos no eliminan las preguntas básicas:

qué datos usamos, qué queremos predecir, cómo evaluamos y qué límites tiene el resultado.

Quedaron fuera, por ejemplo, muchos temas vinculados con Deep Learning. Las redes neuronales permiten abordar problemas muy potentes, como reconocimiento de imágenes, procesamiento de lenguaje, generación de texto, audio, visión por

computadora y muchas otras aplicaciones. Pero para estudiarlas con seriedad hace falta introducir nuevas ideas: capas, neuronas, funciones de activación, descenso por gradiente, entrenamiento con grandes volúmenes de datos, arquitecturas profundas y uso de hardware especializado.

Deep Learning: una familia de modelos basados en redes neuronales capaces de aprender representaciones muy complejas.

También quedó fuera un tratamiento profundo del procesamiento de lenguaje natural. Analizar texto tiene desafíos propios. Las palabras deben transformarse en números, el orden puede importar, el contexto cambia el significado y la misma idea puede expresarse de muchas maneras. En este libro apenas mencionamos algunos ejemplos, pero el tema merece un recorrido completo.

Texto: no es solo una columna más.

Tiene contexto, orden, ambigüedad y significado.

Otro tema importante que no desarrollamos en profundidad son las series temporales. Predecir valores a lo largo del tiempo, como ventas, demanda, temperatura, precios, consumo o tráfico, requiere cuidados especiales. No alcanza con separar datos al azar. Hay que respetar el orden temporal, analizar tendencias, estacionalidades, cambios de régimen y formas de validación adecuadas para el tiempo.

Series temporales: aprender del pasado para estimar el futuro, respetando el orden del tiempo.

También quedaron fuera los sistemas de recomendación. Estos sistemas aparecen en plataformas de música, video, comercio electrónico, noticias, cursos y redes sociales. Su objetivo no es solo predecir una clase o un valor, sino sugerir elementos relevantes para una persona o contexto. Para eso se combinan datos de usuarios, productos, interacciones, similitudes y comportamiento histórico.

Sistemas de recomendación: ayudar a elegir entre muchas opciones posibles.

Otro campo que apenas rozamos es MLOps. Entrenar un modelo en un cuaderno es solo una parte del trabajo. Usar un modelo en un sistema real implica guardar versiones, desplegar, monitorear, actualizar, controlar cambios en los datos, medir degradación, automatizar procesos y mantener trazabilidad. Es el puente entre el experimento y el uso sostenido.

MLOps: llevar modelos desde el experimento hacia sistemas mantenibles, monitoreados y actualizables.

También queda mucho por estudiar sobre ética, regulación y responsabilidad. En este libro dedicamos capítulos a sesgos, interpretación y comunicación, pero el tema es mucho más amplio. Los modelos pueden afectar decisiones educativas, laborales, financieras, comerciales, médicas, legales o institucionales. Por eso, a medida que las herramientas se vuelven más potentes, también crece la necesidad de pensar sus consecuencias.

Más capacidad técnica también exige más responsabilidad.

Que estos temas hayan quedado fuera no es una falta del libro. Es parte de una decisión de recorrido. Primero era necesario construir una base clara. Después vendrán los temas que requieren esa base para poder entenderse mejor.

En ese sentido, este capítulo no busca cerrar el camino, sino abrirlo. Si llegaste hasta acá, ya tenés herramientas para leer otros materiales, probar modelos, interpretar métricas, detectar errores comunes y hacer mejores preguntas. Eso es más importante que memorizar una lista de algoritmos.

Lo más valioso no es conocer todos los modelos.

Es aprender a pensar como alguien que trabaja con datos y predicciones.

Machine Learning cambia rápido. Aparecen nuevas librerías, nuevas arquitecturas, nuevos modelos y nuevas aplicaciones. Pero las preguntas fundamentales siguen siendo necesarias: qué datos tengo, qué representan, qué quiero predecir, cómo evalúo, dónde puede fallar, qué sesgos puede arrastrar y cómo comunico los resultados.

Este libro termina en ese punto: no con la idea de que el lector ya lo sabe todo, sino con la idea de que ahora puede seguir aprendiendo mejor.

En el futuro, muchos de los temas que quedaron fuera podrán formar parte de nuevos recorridos. Deep Learning, procesamiento de lenguaje natural, visión por computadora, modelos generativos, series temporales, sistemas de recomendación y MLOps merecen espacios propios. Este libro puede funcionar como la base sobre la que esos caminos se apoyen.

Este libro fue una base.

Lo que sigue es aprender a construir sobre esa base.

Quedaron fuera muchos temas importantes, pero no quedaron fuera del camino. Quedaron como próximos pasos. Machine Learning es demasiado amplio para agotarlo en un solo volumen, y justamente por eso vale la pena seguir explorándolo. Lo importante es avanzar sin perder de vista los fundamentos: datos, evaluación, interpretación, límites y responsabilidad.

Deep Learning

Uno de los caminos más importantes para seguir aprendiendo Machine Learning es Deep Learning. El nombre suele aparecer asociado a inteligencia artificial moderna, reconocimiento de imágenes, modelos de lenguaje, generación de texto, traducción automática, audio, video y muchas otras aplicaciones que en los últimos años se volvieron muy visibles.

Deep Learning puede entenderse como una familia de técnicas basadas en redes neuronales con muchas capas. Estas redes aprenden representaciones cada vez más elaboradas a partir de los datos. En lugar de depender solamente de variables construidas manualmente, pueden aprender patrones internos que resultan útiles para resolver una tarea.

En modelos clásicos de Machine Learning, muchas veces trabajamos con datos tabulares: columnas, filas, variables numéricas, variables categóricas y una variable objetivo. Ese enfoque sigue siendo muy importante. De hecho, en muchísimos problemas reales, los modelos clásicos continúan siendo una opción excelente.

Pero hay tipos de datos que requieren otro tratamiento. Una imagen no se parece a una tabla simple. Un texto no es solo una lista de columnas. Una señal de audio tiene forma temporal. Un video combina imágenes, movimiento y sonido. En esos casos, Deep Learning ofrece herramientas especialmente potentes.

Por ejemplo, en visión por computadora, una red puede aprender a detectar bordes, formas, texturas, partes de objetos y objetos completos. En procesamiento de lenguaje natural, puede aprender relaciones entre palabras, frases, temas, contexto y significado. En audio, puede detectar patrones sonoros. En modelos generativos, puede aprender a producir texto, imágenes, código, música u otros contenidos.

La diferencia no está solo en usar un algoritmo más complejo. También cambia la forma de pensar los datos. En Deep Learning, las representaciones internas del modelo tienen un papel central. Una red neuronal profunda puede transformar los datos originales en varias capas de representación antes de llegar a la predicción final.

Esto permite resultados muy potentes, pero también trae nuevos desafíos. Los modelos de Deep Learning suelen necesitar más datos, más tiempo de entrenamiento y más capacidad de cómputo. Además, pueden ser más difíciles de interpretar. Cuando una red profunda produce una predicción, no siempre es sencillo explicar de manera directa qué parte de la entrada fue decisiva o cómo combinó la información.

Por eso, Deep Learning no debe entenderse como un reemplazo automático de todo lo anterior. No es “mejor” en todos los problemas. Para muchos datasets tabulares pequeños o medianos, un modelo clásico bien preparado puede funcionar igual o mejor, con menos complejidad y más facilidad de interpretación.

Una buena pregunta no es “¿puedo usar Deep Learning?”, sino “¿este problema necesita Deep Learning?”. Si los datos son tabulares, el dataset es pequeño, las variables están bien definidas y se necesita interpretabilidad, quizás convenga comenzar con modelos clásicos. Si trabajamos con imágenes, texto, audio, grandes volúmenes de datos o patrones difíciles de diseñar manualmente, Deep Learning puede ser un camino natural.

También es importante entender que Deep Learning no elimina los fundamentos. Antes de entrenar una red neuronal, seguimos necesitando definir el problema, separar datos, evitar fuga de datos, elegir métricas adecuadas, revisar errores, cuidar sesgos y comunicar resultados con prudencia. Los modelos cambian, pero muchas preguntas siguen siendo las mismas.

En algunos casos, incluso se combinan enfoques. Podemos usar un modelo profundo para extraer representaciones de imágenes o textos y luego utilizar esas representaciones en otro modelo. También podemos usar modelos preentrenados, es decir, modelos que ya fueron entrenados con grandes volúmenes de datos y que luego se adaptan a una tarea más específica.

Esta idea de modelos preentrenados es clave en la inteligencia artificial moderna. En lugar de entrenar desde cero cada vez, muchas aplicaciones parten de modelos grandes que ya aprendieron patrones generales y luego los ajustan, los consultan o los integran en sistemas concretos. Esto cambió mucho la forma de trabajar con texto, imágenes y otros tipos de datos.

Pero cuanto más potentes son las herramientas, más importante se vuelve el criterio. Un modelo profundo puede producir resultados impresionantes y, aun así, equivocarse. Puede generar respuestas convincentes pero incorrectas. Puede aprender sesgos presentes en los datos. Puede fallar en casos que no se parecen a los de entrenamiento. Puede ser difícil de auditar. Por eso, aprender Deep Learning no significa abandonar la mirada crítica; significa aplicarla en un terreno más complejo.

Para seguir este camino, conviene avanzar de manera gradual. Primero, entender qué es una red neuronal simple. Luego, estudiar cómo aprende mediante entrenamiento y ajuste de pesos. Después, incorporar ideas como funciones de activación, pérdida, optimización, descenso por gradiente, épocas, batches y validación. Más adelante, se pueden estudiar arquitecturas específicas: redes convolucionales para imágenes, redes recurrentes y transformers para secuencias y lenguaje, autoencoders, modelos generativos y otras variantes.

El aprendizaje no debería comenzar por los modelos más grandes ni por las herramientas más llamativas. Es preferible construir una base clara. Una red neuronal pequeña, entrenada sobre un problema simple, puede enseñar más que intentar usar una arquitectura enorme sin entender qué ocurre.

Deep Learning será, probablemente, uno de los próximos grandes pasos naturales después de este libro. Muchos de los temas que quedaron fuera aquí merecen un recorrido propio, con ejemplos, código, visualizaciones y explicaciones cuidadosas. Especialmente porque no se trata solo de usar modelos poderosos, sino de entender cuándo tienen sentido, cómo se entrenan, cómo se evalúan y qué riesgos traen.

Deep Learning amplía enormemente el campo de Machine Learning. Permite trabajar con imágenes, texto, audio, video y modelos generativos de una manera que los enfoques clásicos no siempre pueden abordar. Pero también exige más datos, más recursos, más cuidado y más criterio. La base construida en este libro no queda atrás: se vuelve necesaria para avanzar con mayor seguridad hacia modelos más complejos.

Procesamiento de lenguaje natural

Otro camino importante para seguir aprendiendo es el procesamiento de lenguaje natural, conocido habitualmente como NLP por sus siglas en inglés. Este campo se ocupa de trabajar con texto y lenguaje humano mediante técnicas computacionales.

El lenguaje aparece en muchos problemas reales: comentarios de usuarios, reseñas de productos, reclamos, correos electrónicos, noticias, publicaciones en redes sociales, respuestas abiertas en encuestas, documentos, consultas de clientes, transcripciones de audio y conversaciones. En todos esos casos, la información no está organizada inicialmente como una tabla numérica. Está escrita en palabras.

Trabajar con texto plantea desafíos propios. Una palabra puede tener distintos significados según el contexto. Dos frases distintas pueden expresar la misma idea. Una oración puede incluir ironía, negación, ambigüedad o referencias implícitas.

Además, los textos pueden tener errores de escritura, abreviaturas, emojis, lenguaje informal o mezcla de idiomas.

Por eso, el texto no puede tratarse simplemente como una variable categórica más. Para que un modelo pueda usarlo, primero hay que transformarlo en una representación numérica. Esa transformación es una parte central del procesamiento de lenguaje natural.

En enfoques clásicos, una técnica común consiste en representar los textos según las palabras que contienen. Por ejemplo, podemos contar cuántas veces aparece cada término o indicar si una palabra está presente o no. Herramientas como `CountVectorizer` o `TfidfVectorizer` permiten convertir una colección de textos en una matriz que luego puede usarse con modelos de Machine Learning.

Este enfoque puede funcionar sorprendentemente bien en tareas simples. Por ejemplo, clasificar mensajes como spam o no spam, agrupar reseñas por temas, detectar sentimiento positivo o negativo, clasificar reclamos según categoría o identificar palabras frecuentes en un conjunto de documentos.

Pero también tiene límites. Contar palabras no siempre alcanza para entender significado. El orden puede importar. La negación puede cambiar completamente el sentido. No es lo mismo “bueno” que “no fue bueno”. Tampoco es lo mismo una palabra usada de manera literal que una palabra usada con ironía. El contexto modifica el significado.

Con el tiempo, el procesamiento de lenguaje natural avanzó hacia representaciones más ricas. En lugar de mirar solo palabras aisladas, los modelos modernos intentan representar relaciones entre palabras, frases y contextos. En ese camino aparecen conceptos como embeddings, modelos de lenguaje y transformers.

Un embedding es una representación numérica de palabras, frases o textos completos en un espacio donde ciertos significados o usos pueden quedar cercanos entre sí. Esta idea permite que el modelo no dependa solamente de coincidencias exactas de palabras, sino que pueda capturar cierta cercanía semántica.

Los modelos de lenguaje modernos llevan esta idea mucho más lejos. Son capaces de trabajar con grandes cantidades de texto, aprender patrones complejos del lenguaje y generar respuestas coherentes. Muchos de los sistemas actuales de inteligencia artificial generativa se apoyan en este tipo de modelos.

Aun así, los fundamentos siguen siendo importantes. Un modelo de lenguaje puede ser muy potente, pero no elimina la necesidad de definir bien el problema, revisar los datos, evaluar resultados y reconocer limitaciones. En tareas con texto, esto puede

ser incluso más delicado, porque el lenguaje humano es ambiguo y está lleno de matices.

Por ejemplo, si entrenamos un modelo para clasificar reclamos, debemos revisar si las categorías están bien definidas. Si dos categorías se superponen, el modelo puede confundirse. Si las etiquetas fueron asignadas con criterios distintos por diferentes personas, la evaluación puede ser inconsistente. Si algunos tipos de reclamos aparecen poco, el modelo puede rendir peor en ellos.

También debemos considerar sesgos. Los textos reflejan formas de hablar, contextos culturales, opiniones, prejuicios, diferencias regionales y desigualdades. Un modelo entrenado con textos puede aprender esas marcas. Por eso, usar NLP de manera responsable requiere revisar no solo la métrica, sino también qué tipos de errores comete y a quiénes pueden afectar.

El procesamiento de lenguaje natural tiene aplicaciones muy amplias. Puede usarse para resumir documentos, clasificar mensajes, detectar temas, buscar información, traducir, responder preguntas, generar borradores, extraer entidades, analizar opiniones o asistir en tareas de lectura y escritura. Pero cada aplicación necesita una evaluación distinta.

No es lo mismo clasificar textos cortos que resumir documentos largos. No es lo mismo buscar palabras clave que responder preguntas. No es lo mismo detectar sentimiento en reseñas de productos que analizar textos educativos o institucionales. El tipo de tarea define los datos, las métricas, los errores importantes y los límites de uso.

Además, en NLP muchas veces el resultado parece más humano que en otros tipos de modelos. Una respuesta redactada con fluidez puede sonar convincente, incluso cuando contiene errores. Esto exige una atención especial. La forma del texto no garantiza su verdad. Un sistema puede escribir bien y equivocarse.

Por eso, al seguir aprendiendo NLP, conviene mantener una mirada crítica. No alcanza con probar una herramienta que genera texto. Hay que preguntarse qué datos la sostienen, qué tarea queremos resolver, cómo vamos a evaluar la calidad, qué errores pueden aparecer, qué información no debería inventar y cuándo se necesita revisión humana.

Para comenzar, puede ser útil trabajar primero con problemas acotados: clasificar reseñas, detectar spam, agrupar noticias, analizar respuestas abiertas o comparar distintas formas de representar texto. Luego se puede avanzar hacia embeddings, modelos preentrenados, transformers y modelos generativos.

El procesamiento de lenguaje natural es uno de los campos más activos y visibles del Machine Learning actual. Seguramente tendrá un lugar importante en recorridos futuros, especialmente porque conecta directamente con herramientas que muchas personas ya usan para leer, escribir, resumir, buscar y conversar.

NLP permite aplicar Machine Learning al lenguaje humano. Abre la puerta a trabajar con textos, documentos, conversaciones y modelos generativos. Pero también trae desafíos especiales: ambigüedad, contexto, sesgos, evaluación difícil y riesgo de respuestas convincentes pero incorrectas. La base construida en este libro sigue siendo necesaria: definir bien el problema, preparar los datos, evaluar con criterio, revisar errores y comunicar límites.

Series temporales

Otro tema importante para seguir aprendiendo es el trabajo con series temporales. Una serie temporal es una secuencia de observaciones ordenadas en el tiempo. Puede representar ventas diarias, consumo eléctrico por hora, temperatura mensual, visitas a un sitio web, precios, demanda de productos, cantidad de reclamos, asistencia a clases o cualquier otra variable que se registra a lo largo de distintos momentos.

A diferencia de otros problemas, en una serie temporal el orden importa. No podemos mezclar los datos como si todas las filas fueran independientes. Una observación de marzo viene después de febrero y antes de abril. Ese orden contiene información.

En muchos problemas de Machine Learning trabajamos con una separación aleatoria entre entrenamiento y prueba. En series temporales, esa estrategia puede ser incorrecta. Si queremos predecir el futuro, no deberíamos entrenar con datos posteriores y evaluar con datos anteriores. Sería como permitir que el modelo conozca parte del futuro antes de hacer la predicción.

Por eso, en series temporales suele respetarse el orden cronológico. Se entrena con datos más antiguos y se evalúa con datos más recientes. De esta manera, la evaluación se parece más al uso real del modelo: aprender del pasado para estimar lo que vendrá después.

Por ejemplo, si tenemos ventas mensuales desde enero de 2021 hasta diciembre de 2024, podríamos entrenar con los datos de 2021 a 2023 y evaluar con los datos de 2024. Esa separación permite observar si el modelo logra anticipar un período que no usó para aprender.

Las series temporales también introducen ideas propias. Una de ellas es la tendencia. La tendencia describe un movimiento general de crecimiento, disminución o estabilidad a lo largo del tiempo. Por ejemplo, una empresa puede mostrar un aumento sostenido de ventas durante varios años, más allá de las variaciones de cada mes.

Otra idea importante es la estacionalidad. La estacionalidad aparece cuando ciertos patrones se repiten en intervalos regulares. Las ventas pueden subir todos los diciembres, el consumo eléctrico puede aumentar en verano, las visitas a una plataforma educativa pueden bajar durante vacaciones o los reclamos pueden concentrarse en ciertos días de la semana.

También puede haber ciclos, cambios bruscos, eventos especiales y comportamientos atípicos. Una promoción, una crisis económica, una modificación de precios, un cambio de plataforma o un evento externo pueden alterar temporalmente el patrón de la serie. Estos elementos hacen que el análisis temporal sea más delicado.

En series temporales, las variables de entrada pueden construirse a partir del pasado. Por ejemplo, para predecir las ventas de la próxima semana, podemos usar las ventas de semanas anteriores, promedios móviles, diferencias respecto del período anterior, indicadores de mes, día de la semana, feriados o campañas. Estas variables deben construirse con cuidado para evitar información futura.

Este punto es fundamental. Si calculamos un promedio usando valores posteriores al momento que queremos predecir, estamos introduciendo fuga de datos. En series temporales, la fuga puede aparecer fácilmente cuando se calculan agregados sin respetar la fecha de corte.

Por ejemplo, si queremos predecir demanda de abril, podemos usar información de enero, febrero y marzo. Pero no deberíamos usar datos de mayo o junio. Parece obvio al decirlo, pero en una tabla histórica completa puede ser fácil cometer ese error al crear variables agregadas.

También cambia la forma de validar modelos. Además de una separación simple entre pasado y futuro, pueden usarse esquemas de validación temporal, donde el modelo se entrena y evalúa en varias ventanas. Por ejemplo, se entrena con los primeros meses y se evalúa el mes siguiente; luego se amplía la ventana de entrenamiento y se evalúa otro período posterior. Esto permite ver si el desempeño se mantiene en distintos momentos.

Las métricas también deben interpretarse en contexto. En regresión temporal, podemos usar MAE, RMSE u otras métricas de error, pero conviene mirar los resultados junto con gráficos de la serie. Un error promedio puede ocultar que el

modelo falla especialmente en picos de demanda, cambios repentinos o períodos estacionales.

Por ejemplo, un modelo puede predecir bien meses normales, pero fallar durante diciembre. Si diciembre es un mes comercialmente importante, ese error no debería pasar desapercibido. El análisis temporal exige mirar no solo el promedio, sino también cuándo ocurren los errores.

Las series temporales también presentan un desafío adicional: el futuro puede cambiar. Un modelo entrenado con datos históricos supone, en alguna medida, que ciertos patrones seguirán siendo útiles. Pero si cambia el contexto, esa suposición puede fallar. Nuevos hábitos de consumo, cambios de precios, modificaciones en una plataforma, eventos económicos o decisiones institucionales pueden alterar el comportamiento de la serie.

Por eso, los modelos temporales suelen requerir monitoreo y actualización. No basta con entrenarlos una vez y asumir que seguirán funcionando igual. A medida que llegan nuevos datos, conviene revisar si el error aumenta, si aparecen nuevos patrones o si la serie cambió de comportamiento.

Existen métodos específicos para series temporales, como modelos estadísticos clásicos, suavizado exponencial, ARIMA y sus variantes. También pueden usarse modelos de Machine Learning que construyen variables a partir del pasado, e incluso enfoques de Deep Learning para secuencias más complejas. Cada familia de métodos tiene ventajas y limitaciones.

Lo importante, al comenzar, no es memorizar nombres de modelos, sino comprender qué vuelve especial a una serie temporal: el orden, la dependencia entre observaciones, la posibilidad de tendencia y estacionalidad, la validación hacia adelante y el riesgo de usar información futura sin querer.

También es importante recordar que no todo dataset con una columna de fecha es necesariamente un problema de serie temporal. A veces la fecha es solo una variable más. Otras veces, el objetivo realmente depende del orden temporal. La diferencia está en la pregunta que queremos responder.

Si queremos estimar el precio de una vivienda usando su fecha de publicación como una variable adicional, quizás no estemos haciendo un análisis de serie temporal completo. Pero si queremos predecir la evolución mensual del precio promedio, entonces el tiempo pasa a ser el eje central del problema.

Las series temporales permiten trabajar con datos que evolucionan en el tiempo. Son fundamentales para problemas de demanda, ventas, consumo, precios, clima, tráfico, actividad digital y muchos otros fenómenos. Pero requieren cuidados especiales: respetar el orden temporal, evitar información futura, construir variables desde el pasado, validar con períodos posteriores y monitorear cambios. Aprender series temporales es un paso natural para quienes quieren avanzar desde modelos generales hacia problemas donde el tiempo tiene un papel central.

Sistemas de recomendación

Otro tema que queda abierto para seguir aprendiendo es el de los sistemas de recomendación. Estos sistemas aparecen en muchas plataformas cotidianas: servicios de música y video, tiendas en línea, bibliotecas digitales, redes sociales, sitios de noticias, cursos en línea y aplicaciones que sugieren contenidos, productos o personas.

A diferencia de otros problemas de Machine Learning, en un sistema de recomendación la pregunta no siempre es “¿a qué clase pertenece este caso?” o “¿qué valor numérico tendrá esta variable?”. Muchas veces la pregunta es: entre muchas opciones posibles, ¿cuáles conviene mostrarle a esta persona, en este momento o bajo este contexto?

Un sistema de recomendación intenta ordenar alternativas. Puede sugerir películas, canciones, libros, cursos, productos, publicaciones, noticias o contactos. Para hacerlo, usa información sobre usuarios, elementos disponibles e interacciones previas: qué se vio, qué se compró, qué se calificó, qué se guardó, qué se escuchó, qué se abandonó o qué se ignoró.

Una idea básica es que las recomendaciones pueden apoyarse en similitudes. Si dos usuarios tuvieron comportamientos parecidos en el pasado, quizá a uno le interesen elementos que el otro ya valoró. Si dos productos son parecidos por sus características o por el tipo de usuarios que los consumen, pueden recomendarse en contextos similares.

Existen distintos enfoques. Algunos sistemas se basan en el contenido de los elementos. Por ejemplo, si una persona leyó libros de ciencia ficción, el sistema puede recomendar otros libros con género, autores, temas o descripciones parecidas. Este enfoque usa características de los elementos.

Otros sistemas se basan en colaboración entre usuarios. En ese caso, la recomendación surge de patrones de interacción: personas que eligieron elementos

similares podrían recibir sugerencias parecidas. Este enfoque se conoce como filtrado colaborativo.

También existen sistemas híbridos, que combinan información de contenido, comportamiento de usuarios, popularidad, contexto y reglas de negocio. En aplicaciones reales, muchas recomendaciones no dependen de una única técnica, sino de una combinación de señales.

Los sistemas de recomendación tienen desafíos propios. Uno de ellos es el problema del arranque en frío. Si una persona es nueva, el sistema sabe poco sobre sus preferencias. Si un producto es nuevo, todavía no hay muchas interacciones que permitan recomendarlo. En esos casos, puede ser necesario usar información inicial, popularidad general, características del contenido o preguntas explícitas al usuario.

Otro desafío es evitar recomendaciones demasiado repetitivas. Si el sistema solo muestra elementos muy parecidos a lo que la persona ya consumió, puede encerrar al usuario en una burbuja estrecha. A veces conviene equilibrar relevancia con diversidad, novedad y exploración.

También aparece el problema de la popularidad. Los elementos muy populares tienden a recibir más interacciones y, por lo tanto, más recomendaciones. Esto puede hacer que los elementos nuevos o menos visibles tengan pocas oportunidades de aparecer, aunque puedan ser relevantes para ciertos usuarios.

Evaluar recomendaciones tampoco es simple. Podemos medir si una persona hizo clic, compró, vio o calificó positivamente un elemento recomendado. Pero esas acciones no siempre representan satisfacción real. Un clic puede ser curioso pero poco útil. Una compra puede depender de precio, necesidad o promoción. Una reproducción puede no significar que el contenido haya gustado.

Además, las recomendaciones influyen en los datos futuros. Si el sistema muestra ciertos elementos y oculta otros, los usuarios solo pueden interactuar con lo que ven. Luego, el sistema aprende de esas interacciones, pero esas interacciones ya fueron condicionadas por recomendaciones anteriores. Esto puede generar ciclos de retroalimentación.

Por eso, los sistemas de recomendación son técnicamente interesantes y también delicados desde el punto de vista social. Pueden ayudar a descubrir contenidos valiosos, pero también pueden limitar la exposición a perspectivas distintas, reforzar preferencias previas o favorecer ciertos elementos por encima de otros.

En educación, por ejemplo, un sistema podría recomendar actividades, cursos o materiales según el avance de cada estudiante. Esto puede ser muy útil si se usa con

cuidado, pero también requiere atención: no debería reducir las oportunidades de aprendizaje ni encasillar tempranamente a una persona según patrones incompletos.

En comercio, una recomendación puede mejorar la experiencia del cliente y aumentar ventas. Pero también puede orientar consumos, priorizar productos específicos o influir en decisiones de compra. En plataformas de contenido, puede afectar qué noticias, videos o publicaciones reciben visibilidad.

Para estudiar sistemas de recomendación, conviene comenzar con ejemplos simples. Una primera práctica puede consistir en recomendar elementos populares como línea de base. Luego se puede avanzar hacia similitud entre productos, similitud entre usuarios, matrices de interacción, filtrado colaborativo y modelos más avanzados.

También es importante mantener la mirada que construimos durante este libro. Un sistema de recomendación no debería evaluarse solo por una métrica aislada. Hay que mirar qué recomienda, a quién, con qué diversidad, qué elementos quedan invisibles, qué sesgos puede reforzar y qué objetivo real está optimizando.

La pregunta “¿qué recomienda mejor?” no siempre tiene una respuesta única. Puede significar mayor probabilidad de clic, mayor satisfacción, mayor aprendizaje, mayor diversidad, mayor descubrimiento, mayor venta o mayor permanencia en la plataforma. Cada objetivo lleva a decisiones distintas.

Los sistemas de recomendación son una extensión natural del Machine Learning hacia problemas donde hay muchas opciones posibles y se busca ordenar o sugerir alternativas relevantes. Combinan datos de usuarios, elementos e interacciones, y pueden apoyarse en contenido, colaboración o enfoques híbridos. Son potentes, pero también requieren cuidado: sus recomendaciones pueden influir en lo que las personas ven, consumen, aprenden o descubren. Por eso, aprender recomendación implica estudiar técnicas, pero también criterios de evaluación, diversidad, sesgos y consecuencias.

MLOps

Otro tema importante para seguir aprendiendo es MLOps. La palabra surge de combinar Machine Learning con Operations, y se refiere al conjunto de prácticas necesarias para llevar modelos desde un experimento hasta un uso real, mantenible y controlado.

En este libro trabajamos principalmente desde la perspectiva del aprendizaje y la evaluación: preparar datos, entrenar modelos, medir resultados, revisar errores e

interpretar conclusiones. Ese recorrido es fundamental. Pero en un entorno real aparece una pregunta adicional: ¿qué ocurre después de entrenar el modelo?

Un modelo puede funcionar bien en un cuaderno, pero eso no significa que esté listo para integrarse en una aplicación, en un sistema de reportes, en una plataforma o en un proceso de decisión. Para usarlo de manera sostenida, hay que pensar cómo se guarda, cómo se actualiza, cómo recibe nuevos datos, cómo se monitorea y cómo se detecta si deja de funcionar correctamente.

MLOps se ocupa de ese puente entre el modelo como experimento y el modelo como parte de un sistema.

En una etapa inicial, podemos entrenar un modelo en un notebook y obtener una métrica. Pero si queremos que ese modelo se use más adelante, necesitamos guardar no solo el modelo final, sino también el preprocesamiento. Si el modelo fue entrenado con imputación, escalado, codificación de variables categóricas o selección de columnas, todo ese flujo debe conservarse de manera coherente.

Por eso, herramientas como `Pipeline` no solo ayudan a evitar fuga de datos durante la evaluación. También ayudan a empaquetar el flujo completo de transformación y predicción. En producción, no queremos aplicar manualmente pasos sueltos y correr el riesgo de transformar los datos de una manera distinta a la usada durante el entrenamiento.

Un modelo usado en un sistema real necesita recibir datos nuevos con la misma estructura esperada. Si durante el entrenamiento usó ciertas columnas, tipos de datos y categorías, el sistema debe controlar que los nuevos datos sean compatibles. Si aparece una columna faltante, un tipo inesperado o una categoría nueva, el sistema debe manejarlo de manera segura.

Aquí aparece una idea clave: el modelo no está solo. Depende de un flujo de datos. Si ese flujo cambia, el modelo puede fallar aunque el archivo del modelo no haya cambiado.

Por ejemplo, si una variable que antes se cargaba en pesos empieza a cargarse en otra unidad, las predicciones pueden volverse incorrectas. Si una categoría cambia de nombre, el codificador puede no reconocerla. Si una columna deja de actualizarse, el modelo puede recibir información incompleta. Estos problemas no siempre aparecen como errores evidentes; a veces producen predicciones silenciosamente malas.

MLOps también incluye versionado. En un proyecto serio, conviene saber qué versión de datos se usó, qué código generó el modelo, qué hiperparámetros se probaron, qué métricas se obtuvieron y qué archivo final fue desplegado. Sin esa trazabilidad, puede

ser difícil reproducir un resultado o entender por qué un modelo cambió su comportamiento.

Versionar modelos no significa solo guardar muchos archivos. Significa poder responder preguntas como: ¿con qué datos se entrenó este modelo?, ¿qué variables usaba?, ¿qué métrica obtuvo?, ¿cuándo se actualizó?, ¿qué cambió respecto de la versión anterior?

Otro aspecto importante es el despliegue. Desplegar un modelo significa ponerlo en condiciones de ser usado por otros sistemas o personas. Puede ser una API que recibe datos y devuelve una predicción, un proceso automático que genera un archivo con puntuaciones, una integración dentro de una aplicación o una actualización periódica de un tablero.

Cada forma de despliegue tiene sus cuidados. Si el modelo se ejecuta en tiempo real, debe responder rápido y manejar errores. Si se ejecuta por lotes, debe procesar correctamente muchos registros. Si se usa en un tablero, debe quedar claro cuándo fueron generadas las predicciones y con qué datos.

Pero el trabajo no termina al desplegar. Un modelo puede degradarse. Esto ocurre cuando los datos nuevos empiezan a diferir de los datos con los que fue entrenado, o cuando cambia la relación entre las variables y el resultado. A veces se habla de cambio en la distribución de datos o de drift.

Por ejemplo, un modelo de demanda entrenado antes de un cambio económico fuerte puede dejar de predecir bien. Un modelo de abandono de clientes puede degradarse si cambia la política comercial de la empresa. Un modelo de clasificación de reclamos puede fallar si aparecen nuevos tipos de reclamos que antes no existían. Un modelo educativo puede cambiar su comportamiento si se modifica la plataforma, la modalidad de cursada o la forma de registrar actividad.

Por eso, los modelos en uso deben monitorearse. No alcanza con decir que una vez obtuvieron buena métrica. Hay que revisar si siguen funcionando. Esto puede incluir monitorear la distribución de las variables de entrada, la cantidad de datos faltantes, la frecuencia de categorías, el número de predicciones positivas, los errores cuando se conocen las respuestas reales y la evolución de las métricas en el tiempo.

En algunos problemas, la respuesta real se conoce rápidamente. Por ejemplo, si predecimos si una transacción fue fraudulenta, tal vez podamos confirmarlo después de una investigación. En otros casos, la respuesta llega mucho más tarde. Si predecimos abandono, desempeño o demanda futura, puede pasar tiempo hasta conocer el resultado. Esto afecta cómo y cuándo podemos evaluar el modelo en producción.

MLOps también se relaciona con la actualización. Si el modelo deja de funcionar bien, puede ser necesario reentrenarlo con datos más recientes. Pero reentrenar no debería ser una acción improvisada. Debe haber criterios: cuándo actualizar, con qué datos, cómo validar la nueva versión y cómo compararla con la anterior.

Un modelo nuevo no debería reemplazar automáticamente al anterior solo porque fue entrenado con datos más recientes. Debe evaluarse. Puede mejorar en algunos aspectos y empeorar en otros. También puede introducir nuevos errores. Por eso, el cambio de versión debe ser controlado.

Otra parte importante es la documentación. Un modelo que será usado por otras personas necesita información clara: qué problema resuelve, qué datos espera, qué variables usa, qué salida produce, qué significan sus predicciones, qué límites tiene, cuándo fue entrenado, cómo se evaluó y en qué situaciones no debería usarse.

La documentación ayuda a evitar usos incorrectos. Un modelo entrenado para cierto contexto no debería aplicarse sin revisión en otro muy distinto. Un modelo diseñado para apoyar decisiones no debería convertirse en una decisión automática sin evaluar consecuencias. Un modelo que produce probabilidades no debería comunicarse como si produjera certezas.

MLOps también obliga a pensar en responsabilidades. ¿Quién mantiene el modelo? ¿Quién revisa si falla? ¿Quién decide cuándo actualizarlo? ¿Quién responde si una predicción genera una acción incorrecta? Estas preguntas no pertenecen solo al código, pero forman parte del uso real de Machine Learning.

Para alguien que está empezando, MLOps puede parecer un tema lejano. Sin embargo, conviene conocerlo desde temprano porque ayuda a entender que entrenar un modelo es solo una etapa. Incluso en proyectos pequeños, es útil guardar el pipeline, documentar los datos, registrar métricas y evitar depender de pasos manuales difíciles de reproducir.

Un primer acercamiento práctico puede consistir en organizar un proyecto de forma clara: separar notebooks exploratorios de scripts reutilizables, guardar modelos entrenados, conservar configuraciones, registrar versiones de datasets, documentar decisiones y escribir instrucciones para reproducir el entrenamiento. No hace falta comenzar con una infraestructura compleja para adoptar buenos hábitos.

Más adelante, se pueden estudiar herramientas específicas para seguimiento de experimentos, automatización de entrenamientos, despliegue de modelos, monitoreo, pruebas de datos y control de versiones. Pero la idea de fondo es simple: si un modelo se va a usar, debe poder mantenerse.

MLOps es el conjunto de prácticas que permite llevar modelos de Machine Learning más allá del cuaderno de experimentación. Incluye versionado, despliegue, monitoreo, documentación, actualización y control del flujo de datos. Su objetivo es que los modelos no solo funcionen una vez, sino que puedan usarse de manera confiable, revisable y sostenible. Aprender MLOps es entender que un modelo útil no termina en una métrica: necesita vivir dentro de un sistema que lo cuide, lo observe y lo actualice cuando sea necesario.

Ética y regulación

A medida que Machine Learning se vuelve más potente y más presente en distintos ámbitos, también crece la necesidad de pensar sus consecuencias. No alcanza con preguntarnos si un modelo funciona. También tenemos que preguntarnos si es apropiado usarlo, qué riesgos genera, a quién puede afectar y bajo qué condiciones debería aplicarse.

La ética en Machine Learning no es un agregado decorativo al final del proyecto. Forma parte del proceso completo. Aparece al elegir el problema, al recolectar datos, al definir la variable objetivo, al seleccionar variables de entrada, al entrenar el modelo, al evaluar errores, al comunicar resultados y al decidir cómo se usará el sistema.

Un modelo puede tener buena métrica y aun así ser problemático. Puede funcionar bien en promedio, pero fallar en ciertos grupos. Puede apoyarse en variables que no deberían usarse para tomar decisiones. Puede reproducir desigualdades presentes en datos históricos. Puede generar una sensación de objetividad que no corresponde. Puede automatizar decisiones que deberían mantenerse bajo revisión humana.

Por eso, la pregunta ética no es solamente “¿el modelo predice bien?”. También debemos preguntar: “¿qué consecuencias tiene equivocarse?”, “¿quién se beneficia?”, “¿quién puede verse perjudicado?”, “¿hay posibilidad de revisión?”, “¿los datos fueron obtenidos de manera adecuada?”, “¿se respeta la privacidad?”, “¿el uso del modelo es proporcional al problema?”.

En este libro ya trabajamos varias ideas relacionadas con estas preguntas: sesgo en los datos, variables sensibles, proxies, correlación y causalidad, decisiones automatizadas, interpretación, errores y limitaciones. Esas ideas son una base para seguir pensando la dimensión ética del Machine Learning.

Uno de los puntos centrales es la privacidad. Los modelos suelen entrenarse con datos que describen personas, comportamientos, consumos, trayectorias, interacciones o preferencias. Aunque esos datos parezcan técnicos, pueden revelar

información sensible. Por eso, no todo dato disponible debería usarse. Recolectar, almacenar y procesar información implica responsabilidad.

También importa la finalidad. Un mismo dato puede ser razonable para una tarea y problemático para otra. La ubicación aproximada de una persona puede ser necesaria para estimar tiempos de entrega, pero puede ser delicada si se usa para decidir acceso a oportunidades. Una variable de actividad puede ayudar a mejorar una plataforma educativa, pero puede ser injusta si se interpreta automáticamente como compromiso o capacidad.

Otro punto importante es la transparencia. Las personas afectadas por un sistema deberían poder entender, al menos en términos generales, que se está usando un modelo, qué tipo de información se considera y qué consecuencias puede tener. Esto no significa que todo modelo deba explicarse línea por línea, pero sí que su uso no debería quedar oculto cuando influye en decisiones relevantes.

La explicabilidad también es parte de este debate. En algunos contextos, no alcanza con que un modelo obtenga buena métrica. Puede ser necesario justificar por qué se tomó una decisión, qué variables influyeron y cómo se puede revisar el caso. Esto es especialmente importante cuando el modelo afecta acceso, oportunidades, seguimiento, beneficios, revisiones o sanciones.

Sin embargo, la explicación no debe convertirse en una apariencia de control. Una explicación simplificada puede ser útil, pero también puede ser engañosa si presenta como claro algo que en realidad es incierto. Por eso, interpretar modelos exige prudencia: explicar lo que se puede explicar, reconocer lo que no se sabe y evitar afirmaciones causales sin evidencia.

La regulación aparece porque muchas de estas cuestiones no pueden depender solo de la buena voluntad de quienes desarrollan modelos. Cuando los sistemas automatizados afectan derechos, oportunidades o decisiones importantes, las sociedades necesitan reglas. Esas reglas pueden referirse a privacidad, protección de datos, no discriminación, transparencia, seguridad, auditoría, responsabilidad y posibilidad de revisión.

Las regulaciones cambian según el país, el sector y el tipo de aplicación. No es lo mismo un modelo usado para recomendar música que uno usado en salud, educación, finanzas, empleo o justicia. A mayor impacto potencial, mayor debería ser el nivel de cuidado, documentación y control.

Para alguien que está aprendiendo Machine Learning, no es necesario convertirse de inmediato en especialista legal. Pero sí es importante adquirir una actitud responsable. Antes de usar datos personales o sensibles, hay que preguntarse si

corresponde usarlos, si se cuenta con autorización, si son necesarios para el objetivo y si pueden protegerse adecuadamente. Antes de automatizar una decisión, hay que preguntarse si debería automatizarse. Antes de comunicar un resultado, hay que preguntarse si puede ser malinterpretado.

También es importante entender que ética y regulación no son obstáculos externos al trabajo técnico. Son parte de construir sistemas confiables. Un modelo que no puede explicarse mínimamente, que no se monitorea, que no permite revisión o que produce errores injustos puede terminar siendo inútil o dañino, aunque tenga buen desempeño en un conjunto de prueba.

En proyectos reales, una práctica recomendable es documentar decisiones. Qué datos se usaron, qué variables se excluyeron, qué riesgos se identificaron, qué métricas se eligieron, qué errores se observaron, qué limitaciones se conocen, qué revisión humana se requiere y qué usos no están recomendados. Esa documentación ayuda a que el modelo pueda ser revisado y discutido.

También conviene incorporar distintas miradas. Un modelo que afecta un proceso educativo, por ejemplo, no debería pensarse solo desde el código. Hace falta entender el contexto pedagógico, las condiciones de acceso, las consecuencias de una alerta, la forma en que se usará la información y los límites de los datos disponibles. Lo mismo ocurre en otros ámbitos: el conocimiento del dominio es indispensable.

La ética en Machine Learning no ofrece recetas simples para todos los casos. Muchas veces hay tensiones reales: precisión frente a interpretabilidad, personalización frente a privacidad, automatización frente a revisión humana, eficiencia frente a equidad, detección temprana frente a riesgo de etiquetar injustamente. Pensar éticamente no significa encontrar respuestas perfectas, sino reconocer esas tensiones y tomar decisiones justificadas.

En el futuro, estos temas serán cada vez más importantes. A medida que los modelos se integran en herramientas cotidianas y sistemas institucionales, será necesario discutir no solo cómo funcionan, sino también cómo deberían usarse. La formación técnica deberá ir acompañada de formación crítica.

Ética y regulación forman parte del aprendizaje de Machine Learning. Los modelos pueden ayudar a resolver problemas, pero también pueden reproducir sesgos, afectar oportunidades, usar datos sensibles o generar decisiones difíciles de revisar. Aprender a construir modelos implica también aprender a preguntar por sus consecuencias. Un buen uso de Machine Learning no se define solo por la métrica, sino por la responsabilidad con que se diseña, se evalúa, se comunica y se aplica.

¿Cómo practicar con datasets reales?

Después de aprender los conceptos principales, la mejor forma de seguir avanzando es practicar con datasets reales. Leer teoría es necesario, pero Machine Learning se entiende de verdad cuando aparecen datos incompletos, variables confusas, clases desbalanceadas, errores inesperados y decisiones que no tienen una única respuesta correcta.

Los datasets reales rara vez llegan perfectamente preparados. Pueden tener valores faltantes, columnas mal documentadas, categorías escritas de distintas maneras, variables que no sirven, duplicados, valores extremos o información difícil de interpretar. Justamente por eso son valiosos para aprender. Nos obligan a pensar, no solo a ejecutar código.

Practicar con datos reales ayuda a desarrollar criterio. Al principio, puede ser tentador buscar rápidamente el modelo con mejor métrica. Pero un buen ejercicio consiste en recorrer todo el proceso: entender el problema, explorar los datos, definir la variable objetivo, revisar las variables disponibles, separar entrenamiento y prueba, construir un pipeline, entrenar modelos, evaluar, revisar errores y escribir conclusiones.

No hace falta empezar con datasets enormes. De hecho, para aprender, muchas veces conviene trabajar con datasets pequeños o medianos, que permitan inspeccionar filas, revisar columnas y entender qué está ocurriendo. Un dataset demasiado grande puede impresionar, pero no necesariamente enseña más si no podemos seguir el razonamiento.

Una buena práctica es elegir problemas variados. Por ejemplo, un dataset de viviendas para regresión, uno de abandono de clientes para clasificación, uno de reclamos para clasificación de texto, uno de ventas para series temporales o uno de clientes para clustering. Cada tipo de problema obliga a pensar distinto.

También conviene repetir ejercicios con distintos niveles de profundidad. La primera vez podemos buscar una solución simple. La segunda, mejorar el preprocesamiento. La tercera, comparar modelos. La cuarta, revisar errores. La quinta, escribir un informe. Volver sobre el mismo dataset desde nuevas preguntas puede ser más útil que saltar rápidamente a otro sin haber comprendido el anterior.

Una forma ordenada de practicar es comenzar siempre con una línea de base. En clasificación, puede ser un modelo simple o incluso una regla trivial, como predecir la clase mayoritaria. En regresión, puede ser predecir la media o la mediana. La línea de base nos ayuda a saber si el modelo realmente aporta algo.

Luego se puede avanzar hacia modelos más elaborados. Pero cada mejora debería tener una razón. No se trata de probar algoritmos al azar hasta que algún número suba. Se trata de entender qué problema tiene el modelo actual y qué cambio podría ayudar.

Si el modelo falla por valores faltantes, habrá que revisar la imputación. Si falla por clases desbalanceadas, habrá que mirar métricas adecuadas y quizá ajustar umbrales. Si falla en variables categóricas, habrá que revisar la codificación. Si se sobreajusta, habrá que controlar complejidad o validar mejor. Si los errores se concentran en ciertos casos, habrá que analizarlos.

Practicar también implica aprender a formular preguntas. Algunas preguntas útiles son: ¿qué representa cada fila?, ¿cuál es la variable objetivo?, ¿qué información estaría disponible al momento de predecir?, ¿hay fuga de datos?, ¿qué métrica representa mejor el problema?, ¿qué errores son más importantes?, ¿el modelo mejora una línea de base?, ¿los resultados son estables?, ¿qué limitaciones tienen los datos?

Estas preguntas pueden repetirse en casi cualquier proyecto. Con el tiempo, se vuelven parte de una forma de pensar.

También es importante practicar la lectura de documentación del dataset. Muchos datasets vienen acompañados por una descripción, un diccionario de variables o notas sobre cómo fueron recolectados. Esa información es tan importante como la tabla. Sin ella, podemos interpretar mal columnas, usar variables incorrectas o sacar conclusiones exageradas.

A veces, el dataset no tiene suficiente documentación. Eso también es parte del aprendizaje. En esos casos, conviene ser prudente. Podemos explorar, pero debemos reconocer que algunas interpretaciones son tentativas. No deberíamos afirmar con seguridad algo que los datos no permiten sostener.

Otro buen ejercicio es escribir pequeños informes, aunque nadie los pida. Después de entrenar un modelo, dedicar unos párrafos a explicar el problema, los datos, las métricas, los errores y las limitaciones ayuda muchísimo. Obliga a ordenar las ideas y a detectar huecos en el análisis.

Si no podemos explicar por qué elegimos una métrica, quizá todavía no entendimos bien el problema. Si no podemos explicar qué significan los errores, quizá solo miramos números. Si no podemos decir qué datos usó el modelo, quizá el pipeline necesita revisión.

También es útil guardar versiones del trabajo. Un proyecto de aprendizaje puede comenzar desordenado, pero conviene adquirir hábitos: mantener nombres claros,

separar exploración de resultados finales, conservar el pipeline, registrar métricas y anotar decisiones. Estos hábitos preparan el camino para proyectos más serios.

Al practicar con datasets reales, es normal equivocarse. De hecho, parte del aprendizaje consiste en detectar errores propios: preprocesar antes de separar, usar una métrica inadecuada, olvidarse del desbalance, interpretar como causal una relación que solo era predictiva, elegir un modelo demasiado complejo o confiar en un único resultado.

Cada error corregido mejora el criterio. En Machine Learning, aprender no significa evitar todos los errores desde el comienzo, sino desarrollar la capacidad de encontrarlos, entenderlos y corregirlos.

También conviene comparar el propio trabajo con soluciones de otras personas. En plataformas abiertas suele haber notebooks, discusiones y enfoques alternativos. Leerlos puede ser muy útil, pero no para copiar sin entender. Lo valioso es observar cómo otras personas exploran, qué variables crean, qué métricas eligen, cómo validan y cómo comunican resultados.

Una buena actitud es preguntarse: ¿qué hicieron distinto?, ¿por qué?, ¿qué puedo aprender?, ¿hay algo que no comparto?, ¿su evaluación es correcta?, ¿sus conclusiones están justificadas? Leer soluciones con mirada crítica es una forma poderosa de aprender.

También puede ser útil reconstruir proyectos desde cero. Elegir un dataset, borrar el notebook anterior y volver a empezar. Esto ayuda a comprobar si realmente entendimos el flujo o si solo seguimos pasos mecánicos. La repetición consciente construye autonomía.

Con el tiempo, el objetivo debería ser pasar de “qué código tengo que escribir” a “qué decisión debo tomar ahora”. El código es una herramienta. La habilidad más importante es decidir qué hacer con los datos, cómo evaluar y cómo interpretar.

Practicar con datasets reales es el paso necesario para transformar conceptos en criterio. Conviene empezar con problemas manejables, recorrer el flujo completo, usar líneas de base, construir pipelines, elegir métricas con sentido, revisar errores y escribir conclusiones. Cada dataset real trae dificultades propias, y justamente ahí está su valor. Machine Learning se aprende practicando, pero practicando con preguntas, cuidado y responsabilidad.

¿Cómo leer documentación?

Aprender Machine Learning no consiste solo en leer libros, seguir tutoriales o copiar ejemplos. También implica aprender a leer documentación. Esta habilidad es menos vistosa que entrenar un modelo, pero es una de las más importantes para ganar autonomía.

Las librerías cambian, agregan funciones, modifican parámetros, deprecian opciones y mejoran herramientas. Un ejemplo que funciona hoy puede necesitar ajustes en el futuro. Por eso, quien aprende a leer documentación no depende únicamente de recetas: puede entender qué hace una herramienta, cómo se usa y qué alternativas ofrece.

La documentación cumple varias funciones. Explica qué hace una clase, una función o un módulo. Muestra qué parámetros recibe. Indica qué valores devuelve. Advierte sobre detalles importantes. Ofrece ejemplos mínimos. A veces también enlaza a guías más completas o a conceptos relacionados.

En `scikit-learn`, por ejemplo, la documentación de un modelo suele indicar para qué sirve, qué parámetros se pueden configurar, qué atributos quedan disponibles después de entrenar y qué métodos podemos usar. Si leemos la documentación de `LogisticRegression`, `RandomForestClassifier`, `KMeans`, `PCA` o `Pipeline`, encontraremos una estructura similar.

Al comienzo, esa documentación puede parecer densa. Hay muchos nombres, parámetros y detalles. No hace falta entender todo de una vez. Una buena estrategia es leer con una pregunta concreta. Por ejemplo: “¿qué significa este parámetro?”, “¿qué método uso para predecir?”, “¿cómo manejo categorías desconocidas?”, “¿qué atributo me permite ver los componentes principales?”, “¿qué métrica espera esta función?”.

Leer documentación con una pregunta concreta evita perderse. En lugar de intentar memorizar toda una página, buscamos la parte que responde a la necesidad actual. Con el tiempo, uno empieza a reconocer patrones.

Una sección importante suele ser la descripción general. Allí se explica qué hace la herramienta y en qué tipo de problemas se usa. Esta parte ayuda a ubicar la función dentro del mapa general. Antes de mirar parámetros, conviene entender para qué sirve.

Luego aparecen los parámetros. Un parámetro es una opción que podemos configurar antes de entrenar o ejecutar una función. En un modelo, puede controlar

la complejidad, el criterio de división, la regularización, la cantidad de vecinos, el número de clusters o la forma de calcular una transformación.

No todos los parámetros son igual de importantes. Muchas veces podemos empezar con los valores por defecto y concentrarnos solo en unos pocos. Por ejemplo, en KNN nos interesa especialmente `n_neighbors`. En K-Means, `n_clusters`. En árboles, parámetros como `max_depth` o `min_samples_leaf`. En regresión logística, `C`, `penalty` o `max_iter` pueden ser relevantes según el caso.

La documentación también indica valores por defecto. Esto es muy útil porque nos permite saber qué ocurre cuando no configuramos nada. A veces creemos que un modelo está usando cierta opción, pero el valor por defecto es otro. Leer la documentación evita suposiciones incorrectas.

Otra parte importante son los atributos. A diferencia de los parámetros, los atributos suelen aparecer después de entrenar el modelo. Por ejemplo, algunos modelos tienen `coef_`, `intercept_`, `feature_importances_`, `classes_` o `components_`. Estos atributos nos permiten inspeccionar lo que el modelo aprendió.

Esta diferencia entre parámetros y atributos es importante. Los parámetros se configuran antes. Los atributos se consultan después del entrenamiento. Reconocer esta estructura facilita mucho la lectura de documentación en `scikit-learn`.

También aparecen los métodos. Los más frecuentes son `fit`, `transform`, `fit_transform`, `predict`, `predict_proba` y `score`. En este libro usamos varios de ellos. La documentación permite confirmar qué espera cada método como entrada y qué devuelve como salida.

Por ejemplo, `fit` entrena o ajusta un objeto. `transform` aplica una transformación ya aprendida. `predict` devuelve clases o valores predichos. `predict_proba` devuelve probabilidades cuando el modelo lo permite. Entender estos métodos ayuda a evitar errores de uso.

Los ejemplos de la documentación suelen ser breves. No siempre se parecen exactamente a nuestro problema, pero sirven para ver la forma básica de uso. Una buena práctica es copiarlos en un cuaderno aparte, ejecutarlos y modificarlos de a poco. Cambiar un parámetro, revisar la salida, probar otro dataset pequeño. Así la documentación deja de ser un texto abstracto y se convierte en experimentación.

También conviene leer las advertencias. A veces la documentación indica que una función espera datos escalados, que cierto parámetro será removido en una versión futura, que un método no acepta valores faltantes o que una métrica se interpreta de cierta manera. Estas notas pueden evitar errores difíciles de detectar.

Cuando una página de documentación resulte demasiado técnica, puede ser útil leerla en capas. Primero, entender para qué sirve la herramienta. Después, identificar el ejemplo básico. Luego, revisar los parámetros principales. Más adelante, volver para leer detalles avanzados. No es necesario absorber todo en una sola lectura.

Leer documentación también implica aprender a buscar. Muchas veces no sabemos exactamente qué página necesitamos. En esos casos, conviene buscar por el nombre de la herramienta y la librería. Por ejemplo, “scikit-learn ColumnTransformer”, “pandas groupby documentation” o “matplotlib scatter”. Si se trata de un error, copiar una parte del mensaje puede ayudar a encontrar la sección relevante.

Pero hay que tener cuidado con soluciones tomadas de foros o respuestas sueltas. Pueden estar desactualizadas, resolver otro problema o funcionar por casualidad. La documentación oficial suele ser el punto de referencia más confiable. Los ejemplos de la comunidad pueden ser útiles, pero conviene contrastarlos con la documentación.

También es importante mirar la versión. Una función puede comportarse distinto según la versión de la librería. Si un parámetro aparece en un ejemplo de internet pero no funciona en nuestro entorno, tal vez pertenece a otra versión. La documentación oficial suele indicar cambios, opciones deprecadas y novedades.

En Machine Learning, leer documentación no significa abandonar la comprensión conceptual. Al contrario, la documentación se entiende mejor cuando tenemos conceptos previos. Si sabemos qué es regularización, será más fácil entender el parámetro `C`. Si sabemos qué es One-Hot Encoding, será más fácil entender `handle_unknown`. Si sabemos qué es validación cruzada, será más fácil usar `cross_val_score` o `GridSearchCV`.

Por eso, teoría y documentación se complementan. La teoría nos ayuda a entender qué problema estamos resolviendo. La documentación nos muestra cómo usar una herramienta concreta para resolverlo.

Una buena costumbre es no quedarse solo con el primer ejemplo que funciona. Después de lograr que el código corra, conviene volver a la documentación y revisar qué opciones relevantes podríamos configurar. No para cambiar todo, sino para entender mejor qué estamos usando.

También conviene leer documentación cuando algo parece demasiado fácil. Por ejemplo, si usamos `score`, debemos saber qué métrica calcula por defecto. En clasificadores suele ser accuracy; en regresores suele ser R^2 . Si no lo sabemos, podemos interpretar mal el resultado. La documentación aclara estas decisiones.

Lo mismo ocurre con modelos que tienen muchas opciones. Un Random Forest puede parecer simple de usar porque se entrena con pocas líneas, pero su comportamiento depende de varios parámetros. No necesitamos ajustar todos desde el comienzo, pero sí entender cuáles controlan la cantidad de árboles, la profundidad, la aleatoriedad y el criterio de división.

Aprender a leer documentación también ayuda a escribir mejor código. Permite usar nombres correctos, evitar parámetros innecesarios, comprender errores y construir pipelines más claros. Además, mejora la capacidad de adaptar ejemplos a problemas propios.

Al principio, puede ser lento. Es normal. Leer documentación es una habilidad que se entrena. Con el tiempo, uno aprende dónde mirar: descripción, parámetros, atributos, métodos, ejemplos y advertencias. Esa estructura se vuelve familiar.

Leer documentación es una parte fundamental de seguir aprendiendo Machine Learning. Las herramientas cambian y ningún libro puede cubrir todas las opciones de cada librería. La documentación permite entender funciones, modelos, parámetros, métodos y ejemplos directamente desde la fuente. Aprender a usarla con preguntas concretas, revisando versiones y conectando cada opción con los conceptos del modelo, ayuda a construir autonomía. Quien aprende a leer documentación deja de depender de recetas y empieza a moverse con más seguridad en problemas nuevos.

¿Cómo construir criterio propio?

El último paso de este recorrido no consiste en aprender un algoritmo más, sino en construir criterio propio. Esa es, probablemente, una de las habilidades más importantes para seguir avanzando en Machine Learning.

Al principio, es natural buscar recetas. Qué modelo usar, qué métrica elegir, qué porcentaje reservar para test, qué parámetros configurar, qué función llamar. Las recetas ayudan a empezar. Permiten dar los primeros pasos sin perderse en demasiadas decisiones. Pero, con el tiempo, hace falta ir más allá.

Construir criterio significa poder mirar un problema y tomar decisiones razonadas. No porque una guía lo diga, no porque un tutorial lo hizo así, sino porque entendemos qué estamos intentando resolver, qué datos tenemos, qué riesgos aparecen y qué alternativas tienen sentido.

Una persona con criterio no necesariamente conoce todos los modelos. Nadie conoce todo. Pero sabe hacerse buenas preguntas. Antes de entrenar, pregunta qué

representa cada fila, cuál es la variable objetivo, cuándo se hará la predicción, qué datos estarían disponibles en ese momento y qué errores serían más importantes. Antes de elegir una métrica, piensa qué significa equivocarse. Antes de confiar en un resultado, revisa cómo fue evaluado. Antes de comunicar, piensa qué puede afirmarse y qué no.

Ese criterio se construye con práctica, lectura, errores y revisión. No aparece de golpe. Se desarrolla al trabajar con datasets reales, comparar enfoques, equivocarse, corregir, leer documentación, analizar errores individuales y volver sobre decisiones anteriores.

Una parte importante del criterio es saber empezar simple. Ante un problema nuevo, no siempre conviene usar el modelo más complejo. Muchas veces es mejor construir una línea de base, entender los datos, entrenar un modelo sencillo y ver qué ocurre. Si una solución simple funciona razonablemente bien, ya tenemos un punto de comparación. Si falla, podemos investigar por qué.

Empezar simple no significa quedarse siempre en lo básico. Significa avanzar con control. Primero entender, después complejizar. Un modelo avanzado puede ser útil, pero solo si sabemos qué problema intenta resolver y qué mejora aporta.

También forma parte del criterio desconfiar de resultados demasiado buenos. Una métrica altísima puede ser real, pero también puede indicar fuga de datos, una separación incorrecta, duplicados, una variable posterior al evento o una evaluación poco representativa. Un buen resultado no debería suspender la revisión; debería motivarla.

El criterio también aparece al elegir métricas. No hay una métrica universal. Accuracy, precision, recall, F1-score, MAE, RMSE, R^2 , AUC y otras medidas tienen sentido en contextos distintos. Elegir bien implica entender el problema, los datos y las consecuencias de los errores.

En clasificación, no es lo mismo dejar pasar un fraude que revisar una operación legítima. No es lo mismo no detectar a un estudiante que necesita apoyo que señalar a uno que no lo necesitaba. No es lo mismo clasificar mal un correo que bloquear una acción importante. Cada problema tiene costos distintos, y las métricas deben leerse a la luz de esos costos.

En regresión, tampoco alcanza con mirar un error promedio. Hay que preguntarse si ese error es aceptable en unidades reales, si los errores grandes importan más, si el modelo falla en valores extremos o si sus predicciones son útiles para la decisión que se quiere tomar.

Otra parte del criterio consiste en reconocer límites. Un modelo puede ser útil sin ser perfecto. Puede aportar información sin resolver todo el problema. Puede servir como apoyo, pero no como decisión automática. Puede detectar asociaciones, pero no demostrar causas. Puede funcionar en un contexto, pero no necesariamente en otro.

Esta forma de pensar evita dos extremos. Por un lado, evita rechazar Machine Learning porque “los modelos se equivocan”. Todo modelo se equivoca. La pregunta es si sus errores son conocidos, aceptables y gestionables. Por otro lado, evita confiar ciegamente en un modelo porque “lo dicen los datos”. Los datos también tienen historia, sesgos y ausencias.

Construir criterio propio también implica aprender a comparar. No solo comparar modelos por métrica, sino comparar soluciones posibles. A veces el mejor enfoque no es el que obtiene el número más alto, sino el que combina buen desempeño, simplicidad, interpretabilidad, estabilidad y facilidad de uso. A veces una regla simple bien justificada puede ser preferible a un modelo complejo difícil de explicar.

También hay que aprender a mirar los errores. Las métricas resumen, pero los errores enseñan. Un conjunto de falsos positivos puede mostrar que el umbral no es adecuado. Un grupo de falsos negativos puede revelar casos poco representados. Errores grandes en regresión pueden señalar valores extremos o variables faltantes. Casos mal etiquetados pueden mostrar problemas en el dataset.

Revisar errores transforma la evaluación en aprendizaje. El modelo deja de ser una caja que produce un número y se convierte en una herramienta para entender mejor los datos y el problema.

El criterio también se construye leyendo trabajos de otras personas con mirada activa. No alcanza con copiar notebooks. Conviene preguntarse por qué alguien eligió cierto modelo, si separó bien los datos, si evitó fuga, si la métrica es adecuada, si las conclusiones son prudentes y si el enfoque podría mejorarse.

Leer así ayuda a aprender mucho más que copiar código. Cada solución ajena se vuelve una oportunidad para analizar decisiones.

También es importante desarrollar paciencia. Machine Learning puede parecer, desde afuera, una sucesión rápida de modelos y métricas. Pero los buenos proyectos requieren tiempo para entender el problema, limpiar datos, revisar supuestos, corregir errores y explicar resultados. Muchas veces el trabajo más importante no está en la línea donde se entrena el modelo, sino en todo lo que ocurre antes y después.

Construir criterio propio significa aceptar esa complejidad. No buscar una respuesta automática para todo. No creer que existe siempre un algoritmo ganador. No pensar

que una métrica cierra la discusión. En cambio, aprender a formular mejores preguntas.

Algunas preguntas acompañan casi cualquier proyecto: ¿qué quiero predecir?, ¿por qué importa?, ¿qué datos tengo?, ¿qué datos faltan?, ¿qué información estaría disponible en producción?, ¿cómo separé entrenamiento y prueba?, ¿hay fuga de datos?, ¿qué métrica representa el objetivo?, ¿qué errores comete el modelo?, ¿a quién afectan esos errores?, ¿puedo explicar el resultado?, ¿qué límites debo comunicar?

Volver a estas preguntas no es repetir lo mismo. Es una forma de cuidar la calidad del trabajo.

El criterio también se relaciona con la responsabilidad. A medida que los modelos se vuelven más potentes, es más fácil producir resultados llamativos. Pero lo importante no es solo que un modelo funcione, sino que se use de manera adecuada. Un buen trabajo técnico debe estar acompañado por una buena comprensión del contexto y de las consecuencias.

Por eso, seguir aprendiendo Machine Learning no debería reducirse a acumular algoritmos. Los algoritmos importan, por supuesto. Pero importan dentro de una forma más amplia de pensar: datos, problema, evaluación, interpretación, comunicación y responsabilidad.

A partir de aquí, el camino queda abierto. Se puede avanzar hacia Deep Learning, procesamiento de lenguaje natural, series temporales, sistemas de recomendación, MLOps, modelos generativos y muchos otros temas. Cada uno traerá herramientas nuevas. Pero la base seguirá siendo necesaria.

Cuanto más compleja sea la herramienta, más importante será el criterio para usarla bien.

Construir criterio propio es aprender a tomar decisiones razonadas en proyectos de Machine Learning. Implica empezar simple, revisar datos, elegir métricas con sentido, desconfiar de resultados demasiado buenos, analizar errores, reconocer límites, comparar modelos con prudencia y comunicar conclusiones responsables. Ese criterio no se adquiere de una vez: se construye practicando, leyendo, equivocándose y volviendo a pensar. Es lo que permite pasar de ejecutar modelos a comprender realmente lo que hacemos con ellos.

Un camino que sigue

Llegar al final de este libro no significa llegar al final de Machine Learning. Significa haber recorrido una primera etapa importante. Una etapa en la que las palabras modelo, entrenamiento, prueba, métrica, validación, sesgo, interpretación y predicción dejaron de ser ideas sueltas para empezar a formar parte de una manera de pensar.

Ese es el objetivo principal de un libro introductorio: no cerrar el tema, sino abrir una puerta con suficiente claridad para que el lector pueda seguir caminando.

Machine Learning es un campo enorme. Cambia rápido, incorpora herramientas nuevas y se cruza cada vez más con otras áreas: educación, salud, economía, comunicación, industria, ciencia, arte, programación, gestión pública y muchas más. Pero, aunque cambien las herramientas, los fundamentos siguen siendo necesarios.

Antes de usar un modelo avanzado, seguimos necesitando entender qué datos tenemos. Antes de confiar en una predicción, seguimos necesitando saber cómo fue evaluada. Antes de comunicar un resultado, seguimos necesitando reconocer sus límites. Antes de automatizar una decisión, seguimos necesitando pensar sus consecuencias.

Por eso, lo aprendido hasta aquí no es un punto final. Es una base.

A partir de esta base se abren muchos caminos. Algunos llevan hacia Deep Learning y redes neuronales. Otros hacia procesamiento de lenguaje natural, modelos generativos, visión por computadora, series temporales, sistemas de recomendación o MLOps. Cada camino tiene sus propias herramientas, sus propios problemas y su propio vocabulario. Pero todos se apoyan, de una u otra manera, en preguntas que ya aparecieron en este recorrido.

- ¿Qué queremos resolver?
- ¿Qué datos tenemos?
- ¿Qué representan esos datos?
- ¿Qué modelo tiene sentido probar?
- ¿Cómo evaluamos sin engañarnos?
- ¿Qué errores aparecen?
- ¿Qué podemos concluir?
- ¿Qué deberíamos comunicar con cuidado?

Seguir aprendiendo Machine Learning no consiste solamente en acumular técnicas. También consiste en aprender a mirar mejor. Mirar los datos con atención. Mirar las métricas con prudencia. Mirar los errores como oportunidades de comprensión.

Mirar los modelos como herramientas útiles, pero no infalibles. Mirar las decisiones con responsabilidad.

Ese aprendizaje requiere práctica. Requiere volver a probar, equivocarse, revisar, comparar, leer documentación, estudiar ejemplos ajenos y construir proyectos propios. También requiere paciencia. No todo se entiende en la primera lectura. No todo modelo funciona en el primer intento. No toda métrica mejora por usar una técnica más compleja. Parte del oficio está en aprender a detenerse, revisar y preguntar mejor.

Si este libro cumplió su propósito, el lector no termina con una lista cerrada de respuestas, sino con mejores preguntas. Y en Machine Learning, eso es fundamental. Las buenas preguntas suelen ser más importantes que el impulso de entrenar rápidamente un modelo.

En los próximos pasos, habrá herramientas más potentes y problemas más complejos. Deep Learning permitirá trabajar con imágenes, textos, audio y modelos generativos. El procesamiento de lenguaje natural abrirá la puerta a documentos, conversaciones y sistemas capaces de producir o analizar texto. Las series temporales permitirán mirar fenómenos que evolucionan. Los sistemas de recomendación mostrarán cómo ordenar opciones para distintas personas. MLOps permitirá pensar modelos que no solo se entrenan, sino que se mantienen en uso.

Todo eso queda por delante.

Pero avanzar hacia esos temas será más valioso si no se pierde lo esencial: Machine Learning no es magia. Es trabajo con datos, modelos, evaluación, interpretación y decisiones. Sus resultados pueden ser muy útiles, pero siempre deben ser leídos dentro de un contexto. Un modelo aprende de datos disponibles, no de una realidad completa. Predice patrones, no verdades absolutas. Sugiere, estima, clasifica o agrupa, pero necesita criterio humano para ser usado correctamente.

Por eso, el cierre de este libro es también una invitación. A practicar con datos reales. A leer documentación. A probar modelos simples antes de correr hacia los más complejos. A desconfiar de resultados demasiado perfectos. A revisar errores. A escribir conclusiones prudentes. A seguir aprendiendo sin perder responsabilidad.

Machine Learning tiene mucho para ofrecer. Puede ayudar a entender fenómenos, anticipar comportamientos, organizar información, detectar patrones y apoyar decisiones. Pero su valor depende de cómo se lo use. La técnica importa, pero el criterio importa tanto como la técnica.

Este libro termina aquí, pero el recorrido continúa. Lo que sigue no es abandonar lo aprendido, sino construir sobre ello. Nuevos modelos, nuevas herramientas y nuevos

problemas aparecerán. La base queda: pensar con datos, evaluar con cuidado, interpretar con prudencia y comunicar con responsabilidad.

MATERIAL GRATUITO

Última versión y apoyo al proyecto

Este documento forma parte de VintaBytes y se comparte de manera gratuita.

Escaneá el código QR para acceder a la última versión disponible, encontrar otros recursos del proyecto o colaborar de forma voluntaria.

Si este material te resultó útil, cualquier aporte ayuda a seguir preparando libros, cuadernos y ejemplos en español. También ayuda compartirlo, recomendarlo o avisarme si encontrás algo para mejorar.

Y si querés colaborar con el café que acompaña muchas de estas páginas, prometo usarlo con responsabilidad pedagógica.

Muchas gracias por formar parte de VintaBytes.



Escaneá para ir a la página oficial

También podés entrar desde:

<https://vintabytes.github.io/>